

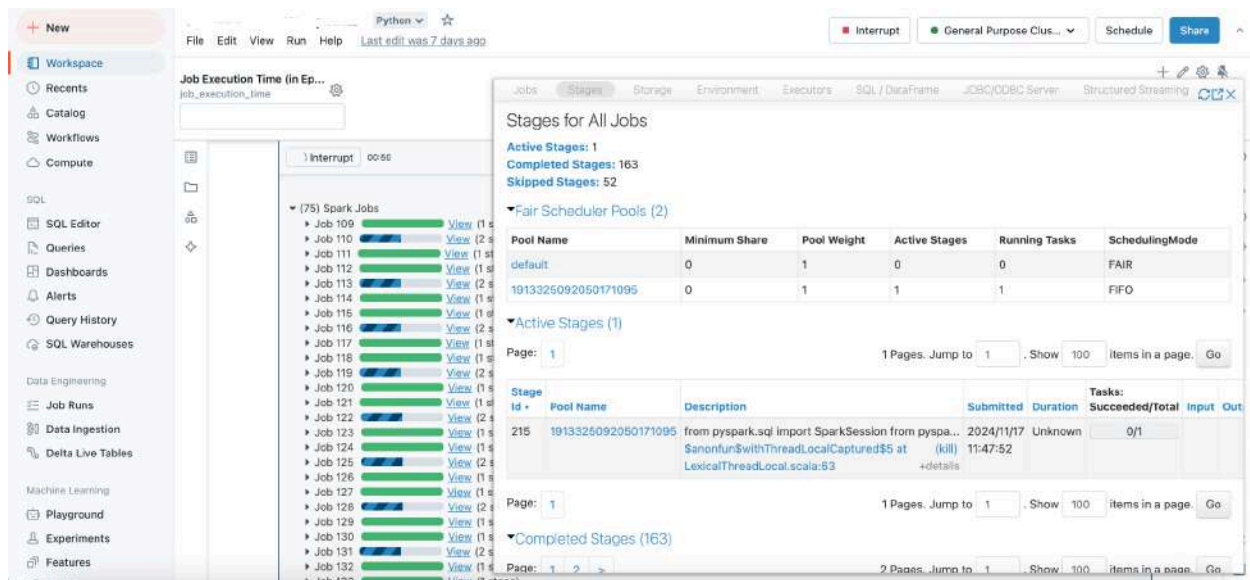
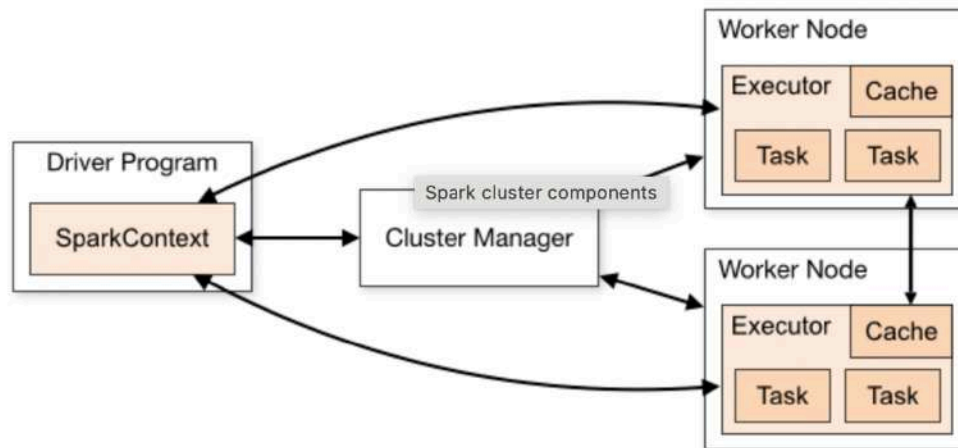
TechStackQs

Many similar tech stack questions can be checked on GPT or [opensource-github-repos](#).

Spark

# PySpark

## Batch Pipelines



- Spark turns the user's data processing commands into a Directed Acyclic Graph, or DAG, which determines the execution plan, i.e what tasks are executed on what nodes and in what sequence.
- Spark runs in a distributed fashion by combining a Driver core process that splits a Spark application into tasks & distributes them among many Executor processes that do the work. Driver sends tasks to executors via Cluster Manager.
- OpenSource vs Databricks managed Spark:



- **Native Spark** (open-source):
  - If running mode: `--deploy-mode cluster`:
    - spark-submit job → Resource Manager
      - submit-job always talks to RM first.
      - You must configure beforehand: driver-memory, driver-cores, num-executors, executor-memory, executor-cores, dynamic allocation, cluster manager settings, etc., when submitting a job.
      - Local machine can disconnect ("fire and forget") - Prod use.
    - Resource Manager launches Driver (random node inside the cluster)
    - Driver starts running your Spark app
    - Driver requests executors from Resource Manager
    - Resource Manager allocates executors on worker nodes
    - Executors register back with Driver
    - **Note:** If you request more Spark executor cores or instances than your infrastructure actually has, the resource manager would still allocate what is physically available rather than crashing the system.
  - If running mode: `--deploy-mode client`:
    - Driver runs on your machine
      - submit-job starts driver locally, then driver calls RM (next point).
      - In client mode, you "skip" the step of asking the Resource Manager to find a home for the Driver because it's already living on your machine.
      - Local machine must stay online for the job to finish - Local use.
    - Driver talks to Resource Manager
    - Resource Manager allocates executors

- **Databricks (managed Spark):** User submits job → Databricks handles almost EVERYTHING. You only configure high-level things.
- **SparkContext vs SparkSession:**
  - **SparkContext:** Only ONE per JVM; Core connection to cluster; Low level.
  - **SparkSession:** Multiple possible; Wrapper around SparkContext; Abstracted level.
    - `getOrCreate()` → returns existing session if present (same object)
    - `newSession()` → creates a new SparkSession (different object, same SparkContext)
    - `spark.stop()` → required to fully reset everything
- **Data Processing:**
  - **Resilient Distributed Dataset (RDD):** Low-level abstraction.
    - Compile-time checking before the job runs.

```
For ex., type-safety checked:
val df = Seq((1, "Alice")).toDF("id", "name")
df.select("nam").show()
// Typo in column name: "nam" instead of "name".
// Compilation error thrown in start, before job runs
```

- NOT Catalyst/Tungsten engine optimised hence comparatively slow.
  - Use - Low level data transformations, custom partitioning data etc.
- **DataFrame:** Medium-level abstraction.
  - NO compile-time checking.
  - Catalyst/Tungsten optimised.
  - Use - Analytical queries, SQL-like aggregations, joins, filters.
- **Dataset:** High-level abstraction.
  - Compile-time checking.
  - Catalyst/Tungsten optimised.
    - Hence best of dataframe & RDD.
  - Use - Type safety + functional + SQL benefits (Scala/Java only).

- Components: Driver node creates jobs/stages/tasks, and workers execute or process data partitions.
  - **Application** - A user program built on Spark using its APIs. It consists of a driver program and executors/workers on the cluster.
  - **Job** - Refers to a sequence of transformations on data in response to a Spark Action. Each job is converted into a DAG with execution plan/stages.
    - Execution: **Sequential**, unless you explicitly trigger multiple actions asynchronously (e.g., via threads, async APIs, or multiple Spark contexts).
    - Actions: **collect(), count(), take(n)/head(n)/tail(), first(), foreach(func), foreachPartition(func), saveAsTextFile(), saveAsParquet(), saveAsTable(), reduce(func), fold(), aggregate(), countByKey(), collectAsMap(), toPandas(), write.format("").save(), show(), rdd.takeSample(), rdd.saveAs...(), spark.sql("SELECT ...").show(), writeStream.start()**.
    - We see from earlier that **writes trigger a job**. But in case of: **spark.sql("SELECT ...") or spark.read.parquet("path") will only build a logical plan, no action**. If you couple it with **.show()**, then action is triggered.
      - **Exception**: Generally, when reading data, spark would know the schema like if reading parquet data, then from metadata of file, schema is known. But assume, reading CSV (comma-separated-values) then spark may inferSchema. **If it does then spark.read.csv("file.csv", inferSchema=True) could trigger 1 job** unlike our previously discussed point.
    - Note:
      - **count()** is an action that triggers a job. Computation happens on executors, and only a final aggregated count is returned to the driver → memory safe but requires full data scan.
      - **collect()** brings all data to the driver → can cause OOM, should be avoided for large datasets.

- `head(n)` is an action that retrieves first `n` rows and may stop early after scanning enough partitions → more **efficient** than full scan.
  - `tail(n)` is **inefficient** in distributed systems since Spark may need to scan the entire dataset → generally avoided.
- **Stage** - Each job gets divided into smaller sets of tasks called stages, in which a sequence of transformations can be executed in a single pass, i.e., without any shuffling of data. **The boundary between two stages is due to data shuffling.** Transformations never “bring data” immediately (they’re lazy - will discuss). But they define how tasks and stages will run once an action triggers them.
    - Execution: **Sequential**
    - Transformations:
      - **Wide (creates a new stage):** `reduceByKey()`, `groupByKey()`, `groupByKey()`, `join(df, on="id", how="left")`, `distinct()`, `repartition()`, `sortBy()`, `orderBy()`, `sortWithinPartitions()`, `cogroup()`, `cartesian()`, `aggregateByKey()`.
      - **Narrow:** `map()`, `filter()`, `union()`, `flatMap()`, `withColumn()`, `drop() / alias()`, `coalesce()`, `mapPartitions()`, `sample()`, `mapValues()`, `keyBy()`, `select() / where() / limit()`, `cache() / persist() / checkpoint()`, `createOrReplaceTempView()`.
      - Note:
        - If you do: `coalesce(numPartitions, shuffle=True)` - **then it becomes a wide transformation.** Acts like `repartition` only.
        - `limit(n)` is tricky: narrow if no global order, wide if combined with `orderBy`.

- **Task** - A task is the smallest unit of execution in Spark, executed on a single core of an executor. Each stage is divided into multiple tasks, where each task processes one partition of the data. A task runs all transformations in that stage on its assigned partition. The number of tasks equals the number of partitions, and the number of tasks that can run in parallel depends on the total available cores in the cluster.
  - Execution: Parallel
- Example:

```
spark = SparkSession.builder \
    .appName("MultipleJobsStagesTasksExample") \
    .master("local[4]") \
    # local[4] → uses 4 cores → at most 4 tasks can run in
parallel
    # local[*] → uses all available cores on the machine
    # NOTE:
    # - This controls PARALLELISM (concurrent task execution), NOT
number of partitions directly
    # - spark.default.parallelism may be derived from cores if not
explicitly set
    .config("spark.executor.memory", "2g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.sql.shuffle.partitions", "16") \
    # Default for spark.sql.shuffle.partitions = 200 → overridden
to 16 (affects shuffle stages like groupBy)
    .getOrCreate()

# DATA SOURCE
df = spark.createDataFrame(data, columns)

# NOTE:
# - createDataFrame from in-memory data → usually creates 1
partition
# - "size / 128MB" rule DOES NOT apply here
# - That rule applies when reading from files (parquet/orc/etc.):
#     partitions ≈ total_data_size /
```

```
spark.sql.files.maxPartitionBytes (default 128MB)

# WIDE TRANSFORMATION → causes shuffle
df = df.repartition(8, "group_id")
# After this → df has 8 partitions

# TRANSFORMATIONS (lazy)
df_filtered = df.filter(col("amount") > 20)
# narrow transformation → no shuffle → pipelined
df_mapped = df_filtered.withColumn("amount_plus_5", col("amount")
+ 5)
# still narrow → stays in same stage

# WIDE TRANSFORMATION → causes shuffle
df_grouped = df_mapped.groupBy("group_id") \
.agg(avg("amount_plus_5").alias("avg_amount"))

# ACTION 1 → triggers Job 1
df_grouped.show()

# ACTION 2 → triggers Job 2 (recomputes lineage since no cache)
df_mapped.count()

# -----
# Job 1 → df_grouped.show()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions of df (likely 1, since
createDataFrame)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → narrow transformations (pipelined)
# tasks = 8 (new partitions after repartition)
```

```

# Stage 3:
# groupBy → shuffle (map side)
# tasks = 8 (input partitions)

# Stage 4:
# groupBy → shuffle (reduce side / aggregation)
# tasks = 16 (controlled by spark.sql.shuffle.partitions)

# Driver collects limited rows (default 20) for show()

# -----
# Job 2 → df_mapped.count()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions (likely 1)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → pipelined
# tasks = 8

# Final:
# Each partition computes partial count
# → tree aggregation → final result returned to driver

# -----
# NOTES:
# 1. Partition vs Parallelism:
#   - Partitions → define number of tasks
#   - Cores (local[4]) → define how many tasks run in parallel
# 2. File-based reads:
#   - partitions ≈ data_size / 128MB (default)
#   - controlled by: spark.sql.files.maxPartitionBytes
# 3. In-memory (createDataFrame):
#   - usually 1 partition unless explicitly repartitioned

```

```
# 4. repartition(n):
#   - full shuffle
#   - sets number of partitions to n
# 5. spark.sql.shuffle.partitions:
#   - controls number of partitions AFTER shuffle (e.g., groupBy)
# -----
```

- Spark does **lazy evaluation**. Transformations (filter, map, groupBy) do NOT execute immediately. They build a logical plan (DAG). Only when you call an action like show(), count(), write(); Spark triggers execution.
  - Plans are of 2 types: Logical + Physical.
  - Evaluation Engines:
    - **Catalyst** (Spark's **query optimization** engine that converts a logical plan into an optimized physical execution plan using rule-based and cost-based optimizations)
    - **Tungsten** (Spark's **physical execution** engine that focuses on CPU and memory efficiency using code generation, off-heap memory, and vectorized processing)
  - Plan lifecycle:

```
Unresolved Logical Plan
  ↓ (Catalyst: Analysis)
Analyzed Logical Plan
  ↓ (Catalyst: Logical Optimization)
Optimized Logical Plan
  ↓ (Catalyst: Physical Planning + CBO)
Physical Plan
  ↓ (Tungsten: Whole-Stage Codegen)
Optimized Physical Plan (with fused operators)
  ↓ (Tungsten: Execution Engine)
Execution:
- Off-heap memory management
- UnsafeRow (binary format)
- Cache-aware execution
- Vectorized processing (batch execution)
  ↓
Tasks run on Executors (1 task per partition)
```



↓  
Result returned to Driver

- **Unresolved Logical Plan:** Just the code what you wrote.
- **Analyzed Logical Plan:** Resolves: Column names, Data types, Table references
- **Optimized Logical Plan:** Rule-based and cost-based optimizations applied by Catalyst like:
  - **Predicate Pushdown:** filter early → less data read
  - **Projection Pushdown (Column pruning):** read only required columns
  - **Constant Folding:** simplify expressions at compile-time rather than execution-time. Eg: `SELECT * FROM users WHERE age > (10 + 5)` --> `SELECT * FROM users WHERE age > (15)`; Other ex: `SELECT name FROM users WHERE age > 30 AND 1 = 1` --> Removing the `1=1`.
  - **Null Propagation:** If it detects that a transformation will inevitably result in null because one of the constants is null, it can skip the calculation entirely.
  - **Join Optimization:** Choosing joins (broadcast, shuffle, etc. - will discuss); Avoiding unnecessary shuffles.
  - **Operation Reordering:** Eg: `map → join → filter` to `filter → map → join` to reduce dataset size
  - **Pipeline Fusion:** Here Spark fuses multiple consecutive operations (like filter, map, and project) into a single, highly optimized function. So instead of moving data from one function to the next, data stays in CPU registers as long as possible.
- **Physical Plan:** Spark chooses how to execute based on cost (**Cost Based Optimizer**). Cost includes: Data size (bytes), Row count, Shuffle cost (network I/O), CPU cost, Memory usage. Config to enable it: `spark.conf.set("spark.sql.cbo.enabled", True)`.
- **Optimized Physical Plan:**
  - **Whole-Stage Code Generation:** Instead of processing data row-by-row with multiple function calls, Spark: generates

one optimized Java program for the entire stage - via Janino (JIT compiler) and runs directly on JVM. So: No virtual function calls, No object creation per row, Tight CPU loop → better performance.

Eg:

> Old (Iterator Model -- slow)

Each operator runs separately

Too many function calls

Row → Filter → Row → Map → Row → Output

```
while (iterator.hasNext()) {  
    Row row = iterator.next();  
    if (row.value > 10) {  
        output.append(row.value * 2);  
    }  
}
```

> New (Whole-stage codegen -- fast)

Spark merges all operations into one loop

```
for (int i = 0; i < numRows; i++) {  
    if (column[i] > 10) {  
        output[i] = column[i] * 2;  
    }  
}
```

- **Off-Heap Memory Management**: Instead of storing data as Java objects (which cause GC overhead): Spark stores data in raw binary format outside JVM heap. Uses low-level APIs like `sun.misc.Unsafe`.
- **Cache-Aware Execution (UnsafeRow)**: Data is stored in contiguous memory blocks. Instead of: Row → object → pointer → value; Spark uses: Flat memory → direct offset access. So no pointer chasing and faster data access.
  - **UnsafeRow is the internal binary format** Spark uses to represent a row of data in memory.
- **Vectorized Processing**: Instead of processing 1 row at a time: Spark processes batches of rows (column-wise). Files like parquet/ORC are perfect for vectorized processing due to columnar format.

- **Execution Runtime:**

- **Adaptive Query Execution (AQE)** happens at runtime, after the physical plan is created. Enable AQE via: `spark.conf.set("spark.sql.adaptive.enabled", "true")`. It does:

- **Dynamic Join Strategy Change:** Say planned was Shuffle Join but may switch to Broadcast Join if data is small.
- **Coalesce Shuffle Partitions:** Reduce 200 → 20 partitions to avoid too many small tasks.
- **Skew Join Optimization:** Detects skewed partitions (very large ones); Splits them → avoids slow tasks
- **Better Aggregation Decisions.**

- Partition is a logical chunk of data that Spark processes in a task on a core. Ideal number: 2-3 partitions on a core. Default partitioning: automatically partitions data based on the underlying file system block size — typically 128 MB or 256 MB per block in HDFS/S3. Eg: Reading 3 files of 1 GB →  $\approx 8 \times 3 = 24$  partitions (1 GB / 128 MB  $\approx 8$ ).

- Types of Partitioning:

- **Hash Partitioning:** Takes your key, applies a hash function, and assigns it to a partition by: `partition = hash(key) % numPartitions`. Deterministic distribution but if one key repeats a lot, that partition maybe skewed. Eg: `join()`, `groupByKey()`, `reduceByKey()`, `distinct()`. Code ex:

```
rdd = sc.parallelize([("A", 1), ("B", 2), ("A", 3), ("C", 4)], numSlices=3)
```

```
grouped = rdd.groupByKey() # triggers hash partitioning
```

Other ex:

Say data: 1,2,3,4,5,6... If hash partition, i.e `hash(id)%4` (Assume 4 constant used) then data would go accordingly to:

```
1 in partition p1
2 in partition p2
3 in partition p3
4 in partition p0
5 in partition p1, etc...
```

- **Range Partitioning:** Sorts keys and divides them into contiguous ranges, so each partition gets a range of keys, say:

partition 0: keys < 100, partition 1: 100–199, etc. Eg:  
sortByKey(), Range-based joins, etc. Code ex:

```
data = [(5, "A"), (1, "B"), (10, "C"), (20, "D"), (100, "E")]
rdd = sc.parallelize(data, 2)
sorted_rdd = rdd.sortByKey(numPartitions=3) # internally uses
RangePartitioner
```

- **Custom Partitioning**: By extending the `Partitioner` class (RDD-level only, not DataFrame).
- Adjust partitions using:
  - **repartition(n)** – increase/decrease partitions with full shuffle (expensive but creates balanced, equal-sized partitions).
  - **coalesce(n)** – reduce partitions without full shuffle (cheap but may create uneven partitions; good for writing smaller outputs). It has a `shuffle=True/False` flag.
  - Sidenote: You may see a similar partition function called: **partitionBy()**. It is NOT the same as `repartition`. **partitionBy() is used only during writes**. Physical data layout on disk is affected (organizing files), not execution-time partitioning. Useful for query pruning. It triggers a shuffle. Eg:

```
df.write.partitionBy("country").parquet("/path").
This creates directory structure like:
/path/
country=US/
country=IN/
country=UK/
```

- Partition strategy:
  - **Count**: If high -> excessive overhead in managing many small tasks, If low -> underutilised cores in cluster.
  - **Size**: If large -> long computation time, slow write times, executor/worker OOM, If small -> slow read time downstream, large task creation overhead, driver OOM
  - Avoid partition on high-cardinality keys (e.g., `user_id`) → causes small files problem, skew, catalogue explosion, and slow query planning.

- **Caching:** Cache intermediate DataFrames to avoid recomputation across stages. Although the number of stages/tasks created would be in expected lines and not reduced, but skipped. Cache vs Persist vs Checkpoint:
  - **Cache:** Stores the DataFrame in memory (RAM) by default. It's equivalent to: `df.persist(StorageLevel.MEMORY_ONLY)`. To remove cache ~ `df.unpersist()`
  - **Persist:** More flexible version of `cache()` — you can choose where and how to store data: Memory only, Memory + Disk, Disk only, Serialized, etc.
  - **Checkpoint:** It's like a savepoint. Breaks the lineage of a DataFrame (cuts off its dependency graph); Writes data to disk (HDFS/S3) as a new materialized copy; Used for fault tolerance and state management (streaming or long transformations). Forces a job execution. **Use a checkpoint if lineage is too long.** Eg:
 

```
# Set the checkpoint dir
spark.sparkContext.setCheckpointDir("/mnt/checkpoints")
# Doing the checkpoint
df.checkpoint()
```

Eg:

```
from pyspark.sql.functions import col
df = spark.read.parquet("big_data.parquet")
df1 = df.filter(col("amount") > 100)           # narrow
df2 = df1.withColumn("x", col("amount") * 2)   # narrow
# df2.cache() # caching if required
# Reused multiple times
df3 = df2.groupBy("category").count()          # shuffle
df4 = df2.groupBy("region").avg("x")           # shuffle
df3.show()   # Action 1 → Job 1
df4.show()   # Action 2 → Job 2
## Without cache df2 is recomputed twice for df3/df4
## With cache, work is reduced inside stages
```

- Joins:
  - **Broadcast Hash Join:** Send the small table to every machine, and join it with chunks of the large table locally. No need to move the large table around. Related config:  
spark.sql.autoBroadcastJoinThreshold

How it works (example)

Data:

Small table (users)

| id | name |
|----|------|
| 1  | A    |
| 2  | B    |

Large table (transactions)

| id | amount |
|----|--------|
| 1  | 100    |
| 2  | 200    |
| 1  | 300    |

Execution:

Small table **is** copied to all executors:

Executor 1 → gets full small table

Executor 2 → gets full small table

Each executor processes its chunk of large table:

Executor 1:

(1,100), (2,200)

Executor 2:

(1,300)

For each row **in** large table:

Look up matching id **in** small table

Join immediately

Result:

(1,A,100)

(2,B,200)

(1,A,300)

- **Shuffle Hash Join:** Move both tables so that same keys come together, then: Build a quick lookup (hash map) from smaller data; Then for each row in larger data → find matches. Less used these days.

How it works (example)

Data:

df1 (small)

df2 (large)

|          |          |
|----------|----------|
| id   val | id   val |
| 1   A    | 1   X    |
| 2   B    | 2   Y    |
| 1   C    | 3   Z    |

Step 1: Shuffle both tables by id

Partition 1 (id=1)      Partition 2 (id=2,3)

df1: (1,A), (1,C)      df1: (2,B)

df2: (1,X)      df2: (2,Y), (3,Z)

Now matching keys are together

Step 2: Inside each partition build lookup from smaller table;

HashMap:

1 → [(A), (C)]

Now process larger table rows:

Row: (1,X)

→ find id=1 in hashmap

→ matches (A), (C)

Result:

(1,A,X)

(1,C,X)

...

- **Sort Merge Join:** Move both tables so the same keys come together, then sort both sides; then walk through them together and match rows. Related config: `spark.sql.join.preferSortMergeJoin`

How it works (example)

Data:

|          |          |
|----------|----------|
| df1      | df2      |
| id   val | id   val |
| 1   A    | 1   X    |
| 2   B    | 2   Y    |
| 1   C    | 3   Z    |

Step 1: Shuffle both tables by id; Now partitions contain same keys

Step 2: Sort inside each partitions

Partition A:

df1: (1,A), (1,C)

```

df2: (1,X)
Partition B:
df1: (2,B)
df2: (2,Y), (3,Z)
# Already sorted but Spark ensures sorting
Step 3: Walk through both (like merging sorted arrays)
Compare:
1 vs 1 → Say in partition A -> 1A-1X, 1C-1X -> match → output.
Other id's no match.
2 vs 2 → Say in partition B -> 2B-2Y -> match → output. Other
id's no match.
Result: (1,A,X), (1,C,X), (2,B,Y)

```

**\*\*Note\*\*:**

- A Spark partition **is** NOT just raw bytes like a file block. A logical chunk of data processed by one task, exposed **as** an iterator of rows.
- Spark sees something like: Partition A: df1\_iter = Iterator[(1,A), (1,C)] **and** df2\_iter = Iterator[(1,X)
- It **is not** merged into one single contiguous array. It's **two independent streams of data in partition**. Contiguous bytes idea may come from thinking in terms of HDFS/storage, but during execution, Spark converts that into row iterators / UnsafeRows.

- **Bucketing** = pre-shuffling data by a key and storing it that way on disk. Data is divided into a fixed number of buckets using:  
`bucket = hash(join_key) % num_buckets.`  
Rows with the same key → always go to the same bucket.
  - **It happens only during write**, i.e saved as tables. Syntax: `.bucketBy(...).saveAsTable(...)`. It is not applied on normal dataframe in memory generally.
  - Bucketing is a storage-level optimization, not runtime transformation. Eg:

```

df1.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df1")

```



```
df2.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df2")
Now: Both tables have 4 buckets; Same hashing logic → id = 1 goes
to the same bucket number in both tables.
```

- If both tables are bucketed on the same key, and we have the same number of buckets, then Spark can do bucket-to-bucket join.
- Bucketing may not always avoid shuffling.
- Skewness in data: When some keys/partitions have much more data than others, causing: Few tasks run very slow (stragglers) + Other tasks finish quickly → resources idle. To solve, different ways like Broadcast join, Increase partitions, Adaptive Skew Join, Preaggregation (apply filters early), etc. We'll discuss more on Salting.
  - **Salting:** Break one heavy key into multiple smaller keys. So the heavy key split into multiple partitions. Load gets distributed.

Eg:

```
# Create a "Salted Key". Instead of '1', we get '1_0', '1_4',
'1_9', etc.
```

```
df_large_salted = df_large.withColumn(
    "salted_dept_id",
    concat(col("dept_id"), lit("_"), floor(rand() * 10))
)
```

```
salt_array = array([lit(i) for i in range(10)])
```

```
# Explode the small table so each dept_id exists 10 times
```

```
df_small_exploded = df_small.withColumn("salt",
explode(salt_array)).withColumn("salted_dept_id",
concat(col("dept_id"), lit("_"), col("salt")))
```

-----

Sidenote on: What explode does:

It converts one row with an array/map into multiple rows.

Input:

| id | items     |
|----|-----------|
| 1  | [A, B, C] |
| 2  | [D, E]    |

```
from pyspark.sql.functions import explode
df.select("id", explode("items").alias("item"))
```

Output:

| id | item |
|----|------|
| 1  | A    |
| 1  | B    |
| 1  | C    |
| 2  | D    |
| 2  | E    |

-----

Coming back, to salting: Assume data:

Large table (orders)

| dept_id | order            |
|---------|------------------|
| 1       | o1               |
| 1       | o2               |
| 1       | o3               |
| 1       | o4 ← 🔥 heavy key |
| 2       | o5               |

Small table (dept info)

| dept_id | name  |
|---------|-------|
| 1       | Sales |
| 2       | HR    |

What goes wrong without salting: During join: All dept\_id = 1 → go to SAME partition.

One task gets: (1,o1), (1,o2), (1,o3), (1,o4)

Other tasks are almost empty

One slow task → overall job slow (data skew)

Hence we add salting.

Step 1: Add salt to LARGE table: salt ∈ {0,1}.

| dept_id | order | salt |
|---------|-------|------|
| 1       | o1    | 0    |
| 1       | o2    | 1    |
| 1       | o3    | 0    |
| 1       | o4    | 1    |

```
2      | o5      | 0
```

Heavy key (1) is now split: (1,0) and (1,1). Load distributed.

Problem now: Small table is still:

```
1 | Sales
```

```
2 | HR
```

It does NOT have salt. So join like: (1,0) ≠ (1) ❌, (1,1) ≠ (1)

❌

Join will FAIL / miss data.

Step 2: Expand SMALL table: Duplicate small table for all salts;

It is intentional.

```
dept_id | name | salt
```

```
1      | Sales | 0
```

```
1      | Sales | 1
```

```
2      | HR    | 0
```

```
2      | HR    | 1
```

Step 3: Join on (dept\_id, salt). Now matching works:

```
(1,o1,0) → matches (1,Sales,0)
```

```
(1,o2,1) → matches (1,Sales,1)
```

```
(1,o3,0) → matches (1,Sales,0)
```

```
(1,o4,1) → matches (1,Sales,1)
```

Result:

```
dept_id | order | name
```

```
1      | o1     | Sales
```

```
1      | o2     | Sales
```

```
1      | o3     | Sales
```

```
1      | o4     | Sales
```

```
2      | o5     | HR
```

After join: Remove the salt: drop("salt")

- Executors (Fat/Tiny):
  - **Fat Executors:** Few executors with more cores & memory per executor

- **Tiny Executors:** Many executors with fewer cores & memory each
- **Spark UDFs:** UDF (User-Defined Function) is a custom function written by the user to extend Spark's capabilities beyond the built-in functions provided by Spark. Say you have to cater some complex business logic or nested JSON parsing, etc. It runs row by row on executors hence slow.
  - **Native Spark (No UDF):**
    - Execution: Logical plan → optimized by Catalyst
    - Converted to **compiled JVM code (WholeStageCodegen)**.  
Runs fully in JVM
    - Runs as a tight **loop inside the JVM**. Vectorized (batch processing). CPU-cache friendly. Predicate pushdown possible.
    - **Eg:** `df.filter(df.age > 30)`
  - **Spark (With UDF):**
    - Execution: JVM → serialize → Python → execute → serialize → JVM
    - Problems:
      - **Serialization Overhead** as data moves from JVM → Python → JVM. Uses Pickle. Happens repeatedly per batch
      - **Cross language:** JVM and Python = separate processes. Communication via sockets/pipes
      - **No Catalyst Optimization.** Spark cannot see inside UDF. So cannot apply push filters down, reorder operations, etc. Also since it executes on worker nodes, logs are not visible in driver node, but only worker nodes.
      - **No Code Generation:** Native → compiled Java. UDF → interpreted Python
      - **Breaks Vectorization:** Native → batch processing. UDF → often row-wise or less efficient batching
  - Pandas UDF could be a better alternative. As it uses **Apache Arrow**. Columnar data transfer. Vectorized execution. Faster than Python UDF, although slower than native spark.

- Spark always deals with serialization at some level, but DataFrames/Datasets minimize and optimize it, while RDDs rely on generic serialization. RDDs store data as JVM/Python objects requiring heavy serialization, while DataFrames use Tungsten's binary format, reducing serialization overhead and enabling optimized execution.
- Data Warehouse vs Lake vs Lakehouse:
  - **Warehouse:** Structured, curated data for analytics & BI.
  - **Lake:** Store everything (raw data) cheaply.
  - **Lakehouse** (Modern approach): Combine Data Lake + Data Warehouse.
  - Others:
    - **Schema-on-Read vs Schema-on-Write**
      - Schema-on-write → validate before storing (warehouse)
      - Schema-on-read → apply when reading (lake)
    - **ETL vs ELT**
      - ETL → Transform before loading (warehouse)
      - ELT → Load first, transform later (lake/lakehouse)
    - Medallion Architecture (Lakehouse): Bronze → raw data, Silver → cleaned, Gold → aggregated
- Copy-on-Write (COW) and Merge-on-Read (MOR) are two strategies for handling row-level updates (like UPDATE or DELETE) in modern data lake formats like Apache Iceberg, Hudi, and Delta Lake
  - **COW:**
    - Mechanism: Total Replacement. When a row is modified, the system locates the file containing that row and rewrites a brand-new version of the entire file with the changes applied.
    - Write Speed: Slow and expensive. Even a tiny change forces a rewrite of a large file (High Write Amplification).
    - Read Speed: Fast and efficient. The query engine reads clean, finished files with no extra "math" required.
    - **Best For: Read-heavy workloads** where data doesn't change often (e.g., historical archives).
  - **MOR:**

- Mechanism: Log-based Changes. Instead of rewriting the big data file, the system keeps the original file as-is and writes a small "sidecar" file (a log of deletes or updates).
- Write Speed: Very fast. It only records the delta (the change), not the whole file (Low Write Amplification).
- Read Speed: Slower. The query engine must read the base data PLUS apply the log of changes at runtime to show the final result.
- Best For: Write-heavy or streaming workloads (e.g., CDC pipelines where data flows in constantly).
- External vs Delta tables:
  - External:
    - Data stored outside Spark metastore, only metadata is stored.
    - Points to data in: S3 / HDFS / ADLS. Dropping table → data NOT deleted.
    - No ACID guarantees. Works with formats like Parquet, CSV, JSON.
  - Delta:
    - It is a storage format (table format), not a table type, i.e: Parquet files + \_delta\_log (metadata layer). It does NOT just “virtually apply changes”. Delta is not a row-level mutation system rather a file-level versioning system.
    - spark.read.format("delta").load(...) reads \_delta\_log and figures out valid files for the current version and reads only them.
    - Stores data as parquet files only internally. \_delta\_log stores metadata like: File additions/removals, schema, transactions, etc. You can perform DELETE/UPDATE row level operations (plain parquet external tables don't allow that). Eg:

```

-> deltaTable.delete("date < '2024-01-01'")
-> deltaTable.update(
  condition="status = 'pending'",
  set={"status": "'processed'"}
)

```

- Other properties of delta tables:

- **ACID Transactions**: Safe concurrent reads/writes
- **Schema Enforcement & Evolution**: Prevents bad writes, Supports schema changes
- **Time Travel**: Ability to query a table exactly as it existed at a specific point in history. Eg:  
`spark.read.format("delta").option("versionAsOf", 5).load(path)`
- **Change Data Feed (CDF)**: Read only changed data. Eg:  
`spark.read.format("delta").option("readChangeFeed", true).option("startingVersion", 2).option("endingVersion", 3)`
- **Optimistic Concurrency Control**: Multiple writers allowed, Conflict detection via versions. “Last writer wins” only if there is no conflict.
- **MERGE operation**: Upserts data. Eg: `MERGE INTO target USING source ON target.id = source.id WHEN MATCHED THEN UPDATE WHEN NOT MATCHED THEN INSERT`
- **INSERT OVERWRITE operation**: Replaces entire table. Eg: `INSERT OVERWRITE TABLE user_data SELECT * FROM new_snapshot;`
- **Problems: Small Files Problem** is a very common issue in Delta Lake. When you are doing: micro-batch/stream writes, frequent merge/DML operations, over-partitioning, etc. This causes: Metadata bloat (Delta tracks every file in its transaction log, which becomes huge overtime), Increased IO overhead, Higher costs.
- **Solution**:
  - **OPTIMIZE table**: Does compaction (bin-packing small files → large files)
  - **Z-ORDER**: It is a multi-dimensional clustering technique for data files - for delta lake, apache iceberg, apache hudi. It co-locates related column values together in the same set of files to make filtering and range scans faster. Normal external tables lack a metadata layer (like Delta's Transaction Log or

Iceberg's Manifests) to store the min/max statistics created by the Z-Order process. Without these statistics, the query engine cannot perform "data skipping," making the physical Z-Order arrangement of rows useless. Eg: OPTIMIZE events ZORDER BY (country, date)

Eg:

```
SELECT * FROM transactions WHERE country = 'IN' AND event_time BETWEEN '2025-10-01' AND '2025-10-10';
```

Delta physically rearranges data on disk so that rows with similar country and event\_time values end up close together (in the same files).

So when you filter by these columns, Delta needs to read fewer files – because the relevant data is tightly clustered instead of being scattered across many files.

Query: OPTIMIZE transactions ZORDER BY (country, event\_time);

Internally, it interleaves the bits and stores, like:

country bits: 0 1 1 0 0 1

event\_time bits: 1 1 1 0 1 0 0 1 0 1 0 1

Z-order bits: 0 1 1 1 0 0 1 1 0 0 1 0 1 0 1...

Rows are sorted by this interleaved single composite Z-order bit key.

Z-Ordering > Sorting by One Column

- **VACUUM:** Deletes old files (physically). Affects time travel though. Eg: VACUUM table RETAIN 168 HOURS
- Solved inefficiencies:
  - **Deletion Vectors (DVs):** Traditionally, Delta used **Copy-on-Write**—if you deleted one row, the entire Parquet file containing it had to be rewritten. **Deletion Vectors shift this to a Merge-on-Read paradigm.** Eg: delta.enableDeletionVectors



- **Liquid Clustering** is the successor to Partitioning and Z-Order. It is a flexible, "self-tuning" layout that automatically groups related data. How it works: It uses a tree-based algorithm (often Hilbert Curves) to organize data incrementally. Unlike Z-Order, it only clusters the newly arrived data rather than rewriting the entire table every time.
- For columns with high cardinality (like email, user\_id, or ip\_address) that don't cluster well, Delta uses Bloom Filter Indexes.
  - **Bloom filter** is a space-efficient, probabilistic data structure used to determine if an element is a member of a set. It is exceptionally fast and memory-efficient but comes with a unique trade-off: it can tell you if an item is "definitely not" in a set or "possibly" in a set.
 

**How it works:** "Instead of storing the data itself, we store its footprint in a bit array of size.

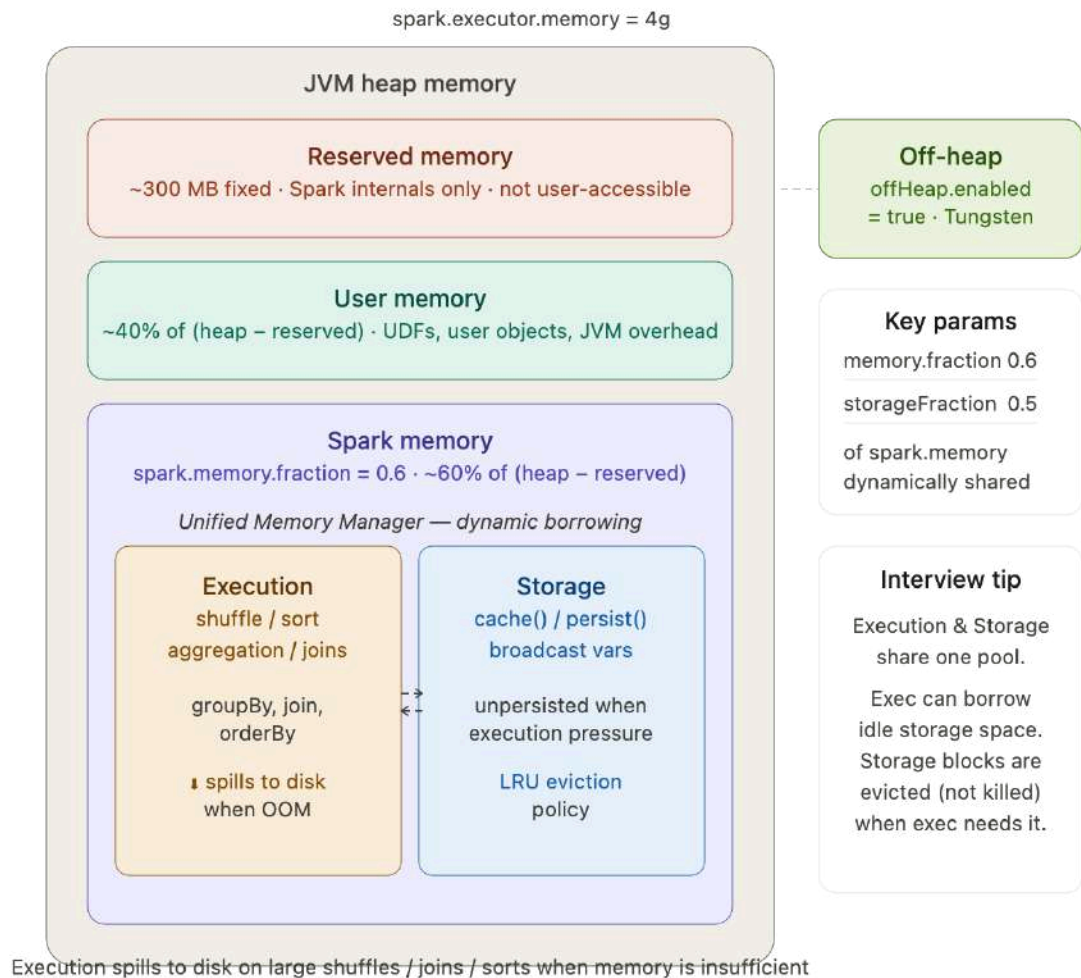
    - Insertion: We run an input through independent hash functions. Each hash outputs an index, and we set those specific bits to 1.
    - Lookup: We hash the query the same way. If we find even a single 0 at any resulting index, we know the item was never added. If all are 1, it's a 'maybe' due to potential bit-overlap from other items (as other items could have similar bits turned on) —which we call a false positive." Say inserting apples, searching for oranges, etc.
  - It performs well at scale:
    - Space: You can track millions of items in just a few megabytes because you aren't storing strings or objects, just bits.

- Speed: Complexity is  $O(k)$ . Whether you have a thousand items or a billion, the lookup time is the same because it's just 'k' simple math operations.

- **Spark Memory split** in workers: Assume a single 4GB worker:

Executor Heap Memory

- └ Reserved Memory (~300MB)
- └ User Memory
- └ Spark Memory
  - └ Execution Memory
  - └ Storage Memory



- Memory hierarchy — heap splits into 3 parts: Reserved (~300 MB hard floor), User (~40% of remaining), and Spark Memory (~60% of remaining via `spark.memory.fraction`).
- Unified Memory Manager: Execution and Storage are not fixed splits — they share a pool. Execution can borrow idle storage space. When execution pressure is high, cached blocks get LRU-evicted (not killed), freeing space. Storage, however, cannot forcibly reclaim memory held by active execution tasks.
- Spill to disk — happens automatically when execution memory is exhausted during shuffle/sort/join. Performance drops but no OOM.
- Off-heap — optional, outside JVM GC, enabled via `spark.memory.offHeap.enabled=true`. Used by Tungsten's binary processing to reduce GC pauses.
- Other tips:
  - `UNION` removes duplicate rows, `UNION ALL` does not. There is a performance hit when using `UNION` instead of `UNION ALL`, choose based on use case.
  - `foreach` runs function per row (use for debugging, not for I/O as could lead to millions of calls) and `foreachPartition` runs function per partition on executor (ideal for external system writes).
  - `EXISTS/NOT EXISTS` (Stops evaluating as soon as a match is found, faster) > `IN/NOT IN` (Subquery may be fully evaluated, slower on large datasets).
  - UDF and `foreachPartition` process data row by row, but they operate at completely different layers of Spark's architecture — and have very different purposes. UDF transform data inside the Spark SQL/DataFrame engine. Operates on data values (columns, rows) as part of a transformation (select, withColumn, filter, etc.) on executors. The output of a UDF is a new column value, part of the logical plan. Whereas `foreach()` perform actions with side effects, often for external I/O. Also runs on executors, but outside Spark's SQL planner. Operates on raw data (RDD/Row objects) per partition. Used after all transformations are done.

- Spark is faster than Hadoop/MapReduce due to in-memory data processing (in Hadoop every stage writes to disk) so disk I/O reduced, can spill memory to disk if required to handle large datasets, uses Catalyst optimizer, etc.
  - Eg:
    - Hadoop: step → disk → step → disk
    - Spark: step → memory → step → memory
  - Spark + HDFS >> MapReduce + HDFS.
  - Hadoop is better if very big datasets bigger than memory need to be processed.
- **Photon** is Databricks' next-generation vectorized execution engine, written in C++, designed to replace many of Spark's traditional JVM operators. Photon uses: SIMD vectorization, tighter memory layout, CPU instruction-level optimizations (AVX-512, etc.). Photon accelerates: File Scans (Parquet, Delta), Projection, Filter, Aggregations, Joins (hash, sort-merge). Where Photon does NOT work: Python / Scala UDFs, Complex nested types in some cases, MLlib operators, Graph workloads.
- **Use KryoSerializer instead of Java serializer** as when Spark runs distributed jobs: Data is sent between executors (e.g., for shuffles, caching, broadcasts). Each time, Spark must serialize (convert objects into byte form) to transfer them across the network or store them in memory/disk. Hence performance matters. Eg:
 

```
.config("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
```
- **Parquet vs Avro vs ORC:**

| Feature  | Parquet<br>(Columnar)                | Avro (Row-based)            | ORC (Columnar)                 |
|----------|--------------------------------------|-----------------------------|--------------------------------|
| Best For | Spark, general analytics, AWS Athena | Kafka, high-speed ingestion | Apache Hive, ACID transactions |

|                         |                                             |                                              |                                             |
|-------------------------|---------------------------------------------|----------------------------------------------|---------------------------------------------|
| <b>Query Speed</b>      | <b>Fastest</b> for reading specific columns | Slow for analytics (must read whole row)     | Fast, especially for filtering with indexes |
| <b>Compression</b>      | High (column-specific)                      | Moderate                                     | <b>Highest</b> (stripe-level optimization)  |
| <b>Schema Evolution</b> | Basic (mostly adding columns)               | <b>Superior</b> (renames, deletes, defaults) | Moderate                                    |

- Note:

--Note--

Parquet format supports Bloom filters, but they are **not** enabled by default. Standard Parquet metadata stores the minimum **and** maximum values **for** each row group. If you query WHERE id = 'abc', **and** the row group's range is aaa to zzz, the engine must scan that group because abc falls inside the range.

To use Bloom filters, you must enable them specifically during the write process so Spark can build the index and store it in the Parquet file's metadata.

```
Eg: df.write.mode("overwrite") \
    .option("parquet.bloom.filter.enabled", "true") \
    .option("parquet.bloom.filter.columns", "user_id, txn_id") \
    .option("parquet.bloom.filter.expected.ndv#user_id", "100000") \
    .parquet("path/to/output")
```

- 

---

## Structured Streaming [High-level basics]

- **Core mental model:** Spark treats streaming as incremental batch processing on an unbounded DataFrame. The engine reads new data on each trigger, builds an incremental logical plan, executes a Spark DAG, optionally updates state, writes to a sink, then commits offsets to

checkpoint. The abstraction you work with is a DataFrame that logically grows forever.

- **Pipeline:** Source → Trigger → Transformations → Output Mode → Sink, with Checkpoint running alongside.
- **Driver role:** coordinates triggers, tracks offsets, plans the query, manages checkpoint commits.
- **Executor role:** runs micro-batch tasks, maintains state store partitions, executes transformations.
- **Micro-batch vs Continuous Processing**

| Mode        | How it works                   | Latency       | Production status               |
|-------------|--------------------------------|---------------|---------------------------------|
| Micro-batch | Batches executed as Spark jobs | 100ms-seconds | Standard, full guarantees       |
| Continuous  | Long-running tasks, row-by-row | ~1ms          | Experimental, limited operators |

- Even Structured Streaming is fundamentally micro-batch in all real production systems. Continuous mode is rarely used.
- **What happens in each micro-batch trigger:**
  - Read new offsets from source
  - Build incremental logical plan
  - Execute Spark DAG on executors
  - Update state store (if stateful)
  - Write to sink
  - Commit offsets + checkpoint
- Triggers:
  - **Trigger.ProcessingTime("10 seconds")** – fires on fixed interval, default mode.

- **Trigger.AvailableNow** — (Spark 3.3+) processes all backlog across multiple safe micro-batches then stops; recommended for incremental ETL.
- **Trigger.Once** — single batch then stops; can OOM on large backlog, now superseded. Continuous — experimental, record-by-record, sub-millisecond latency, avoid in prod.

|               | <b>Trigger.Once</b> | <b>Trigger.AvailableNow</b> |
|---------------|---------------------|-----------------------------|
| Batch count   | Single huge batch   | Multiple micro-batches      |
| Large backlog | Can OOM             | Safe                        |
| Status        | Older               | Recommended                 |

- Time semantics:
  - **Processing time** — when Spark processed the record.
  - **Event time** — when the event actually occurred at the source. These can differ significantly.
  - Example: user clicked at 10:00, Kafka delayed delivery until 10:15 — event time is still 10:00. Watermarks operate on event time, not processing time. This distinction is critical for correctness of windowed aggregations.
- **Watermarking**
  - withWatermark("event\_time", "10 minutes"): Spark tracks the maximum event time seen so far and allows data to arrive up to 10 minutes late relative to that event time. Data older than the watermark is considered too late and may be discarded.
  - Why watermarks matter:
    - Correctness for Append mode on aggregations: Without a watermark, Spark cannot know whether a future late event

might still update an old aggregation/window result.

Therefore the result is never considered final, making Append mode invalid for aggregations whose results may still change.

With a watermark, Spark knows when a window is sufficiently old and can safely emit the final result in Append mode.

(Stateless append queries do not require watermarks because rows are emitted once and never updated.)

- **Bounding state size:** Stateful operations maintain intermediate state in the state store. Without watermarks, old aggregation/join state may accumulate forever, eventually causing excessive memory usage, GC pressure, spills, or OOM errors.
  - **State cleanup / eviction:** As the watermark advances, Spark evicts expired window state and old stream-stream join state from the state store.
- **Window types;**
    - **Tumbling window** — fixed size, non-overlapping: Eg: `window(col("ts"), "10 minutes")`
    - **Sliding window** — overlapping, a record can belong to multiple windows: Eg: `window(col("ts"), "10 minutes", "5 minutes")`
    - **Session window** — dynamic, closes after an inactivity gap: Eg: `session_window(col("ts"), "30 minutes")`
  - **Operations**
    - **Stateless** — filter, map, select. No state store touched. Rows are emitted once and never revised. Effectively Append behavior regardless of mode declared; Update mode is technically allowed but behaves identically.
    - **Stateful** — aggregations, windowed ops, stream-stream joins. Uses the StateStore API internally: versioned, checkpointed, one state partition per shuffle partition. Backed by HDFS/S3 (default) or RocksDB (Spark 3.2+, `spark.sql.streaming.stateStore.providerClass`). RocksDB is strongly preferred for large or long-lived state — avoids JVM GC pressure.
    - **State store internals:** large cardinality aggregation keys → massive state growth → GC pressure → OOM. Optimization levers:



watermarks, careful key design, RocksDB backend, proper partitioning.

- Stream joins
  - **Stream-Static:** join a stream against a static DataFrame. No watermark needed. The static side behaves like a normal batch DataFrame and is joined against each incoming micro-batch.
  - **Stream-Stream:** both sides unbounded. Watermark is mandatory on both sides to bound the join buffer. Without it, Spark must buffer all rows from both sides indefinitely.
- **Message delivery guarantees:** (how a system handles data as it moves from a source to a destination)
  - **At-Most-Once:** Data may be lost but is never duplicated (fastest, lowest reliability).
  - **At-Least-Once:** Data is never lost but may be duplicated (standard for most pipelines).
  - **Exactly-Once:** Data is neither lost nor duplicated (highest accuracy, requires checkpoints/idempotency).
- Output modes
  - **Append** — new rows only. Valid for: stateless ops, and stateful ops with watermark (once window is closed). Illegal on non-watermarked aggregations — Spark throws at runtime.
  - **Update** — only rows changed this batch. Most memory-efficient for aggregations. Not supported by file sinks.
  - **Complete** — entire result table every batch. Requires an aggregation. Expensive for large result sets.
- Source-side rate limiting
  - The old `spark.streaming.backpressure.enabled=true` is a DStreams config — it does not apply to Structured Streaming. For Structured Streaming, control ingestion rate through source-specific options:  
Eg: `.option("maxOffsetsPerTrigger", 100000)` # Kafka;  
`.option("maxFilesPerTrigger", 10)` # file source
- Sinks
  - **foreach** — row-level, runs on executors. Not idempotent by default. If a batch fails midway and retries, your writer runs again on already-processed rows. Spark's checkpoint only tracks source

offsets, not what your external writer did. Avoid for production DB writes.

- **foreachBatch** — batch-level callback. Callback executes on the driver; batch\_df operations inside run distributed on executors. batchId is provided — use it to make writes idempotent (e.g. deduplicate on batchId before insert). Exactly-once semantics are achievable if and only if your sink write is implemented idempotently using batchId or transactional logic. Spark cannot magically guarantee external DB semantics.
- **Built-in sinks:** Kafka (exactly-once natively), Parquet/JSON (micro-batch file writes), memory (testing only).
- **Delta Lake sink** — ACID transactional writes, exactly-once behavior, supports streaming reads and writes, CDC patterns, and the standard Bronze/Silver/Gold lakehouse architecture. Increasingly the default production sink. Eg:  
`df.writeStream.format("delta").option("checkpointLocation", "...").start("path/to/table")`
- **Fault tolerance and exactly-once:** Spark guarantees: exactly-once processing of input offsets (via checkpointed offsets + replayability). End-to-end exactly-once depends entirely on the sink:

| Sink                 | Guarantee                               |
|----------------------|-----------------------------------------|
| Kafka                | Exactly-once (native)                   |
| Delta Lake           | Exactly-once (transactional)            |
| foreachBatch + dedup | Exactly-once (if implemented correctly) |

|                |                                    |
|----------------|------------------------------------|
| JDBC / foreach | At-least-once unless you add dedup |
|----------------|------------------------------------|

- **Checkpointing:** Stores three things: source offsets (e.g. Kafka partition offsets), state store snapshots (for stateful ops), job metadata and progress. Stored on HDFS or S3. Without it, Spark cannot resume from failure.
- **Unsupported / restricted operations:** Global sort on an unbounded stream, certain non-time-bounded stream-stream joins, collect() and similar actions, non-deterministic operations. Root cause: streaming datasets are unbounded so these operations have no valid completion point.
- **Common production problems:** Small file problem from frequent micro-batches writing to object storage. State explosion from high-cardinality keys without watermarks. Checkpoint corruption (requires manual recovery or offset reset). Data skew in stateful aggregations. Kafka partition imbalance causing uneven executor load. Late data silently dropped when it exceeds the watermark – no error, just gone. Long GC pauses with the default JVM state store on large state (fix: RocksDB).
- **Kafka & Spark streaming:** Kafka partitions map to Spark tasks. Parallelism heavily depends on Kafka partition count. Spark stores consumed offsets in checkpoint, not Kafka consumer groups (usually). maxOffsetsPerTrigger controls ingestion rate. Too few Kafka partitions limits streaming parallelism.
- 

---

## Hadoop-MapReduce

- Hadoop = HDFS (storage) + MapReduce (compute)
- HDFS (Hadoop Distributed File System – Distributed Storage Layer)
  - Purpose: Reliable, fault-tolerant storage for very large files across a distributed cluster.

- Architecture:
  - **NameNode (Master): Stores metadata** – file names, block locations, permissions. Keeps a transaction log (EditLog) and a checkpoint (FsImage). Doesn't store actual data, only where blocks live. Single point of failure in early Hadoop versions (fixed later via Secondary NameNode / Standby NameNode / High Availability mode).
  - **DataNodes (Slaves): Store actual file blocks (default size: 128 MB or 256 MB)**. Send periodic heartbeats and block reports to the NameNode. Perform replication, deletion, and block creation as instructed by the NameNode.
- Key Concepts:
  - **Replication factor**: Default = 3 → improves fault tolerance and availability.
  - **Rack Awareness**: **NameNode knows rack topology** → places replicas on different racks to balance network efficiency and fault resilience.
  - **Read/Write Path**: Client → NameNode (for metadata) → Read from nearest Datanodes or write blocks of data.
- Benefits: High throughput, scalable storage, fault-tolerant, optimized for large sequential reads/writes.
- MapReduce (Distributed Processing Framework)
  - Purpose: Parallel computation model that processes massive data using Map() and Reduce() functions.
  - Flow:
    - Job Submission: Client submits job to JobTracker (Hadoop v1) or YARN ResourceManager (v2).
    - Split Phase: Input is divided into chunks (InputSplits), typically aligned with HDFS blocks.
    - Map Phase: Mapper processes each split → outputs intermediate key-value pairs.
    - Shuffle & Sort: Framework groups all values by key and transfers them to the appropriate reducer.
    - Reduce Phase: Reducer aggregates or combines values per key → produces final output.

- Key Points:
  - Idempotent: Multiple runs on same data yield same output.
  - Fault Tolerance: If a node fails, tasks are re-run on another node using data replicas.
  - Data Locality Optimization: Tries to schedule Map tasks on nodes where data already resides.
  - Output written back to HDFS.
- Limitation: High latency for iterative or real-time processing — hence, frameworks like Spark were built to overcome this.

#### Sample Problem:

Count frequency of each word in a large dataset. Step-by-step flow in Hadoop

##### 1. Input Splitting

File stored in HDFS

Split into blocks (e.g., 128 MB each)

Each block → processed by a Mapper

##### 2. Map Phase

Each mapper processes its chunk:

Input: "hello world hello"

Output (key-value pairs):

(hello, 1)

(world, 1)

(hello, 1)

##### 3. Shuffle & Sort (MOST IMPORTANT)

Hadoop groups same keys together across all mappers

(hello, [1,1])

(world, [1])

Happens across network (expensive step)

##### 4. Reduce Phase

Reducer aggregates:

(hello, 2)

(world, 1)

Final Output: Stored back in HDFS

- YARN (Yet Another Resource Negotiator — Cluster Resource Management Layer)

- Purpose: Decouples resource management from job scheduling, making Hadoop cluster multi-tenant and more efficient.

---

### Miscellaneous(s)

**Question1:** Assume daily data volume to be 1TB/day from Kafka. Discuss about infra requirements if you were to ingest/parse data and store in S3 - in daily basis (batch) or near real-time basis (streaming).

**Answer:**

Common Assumptions

| Parameter               | Value                                                                       | Notes                           |
|-------------------------|-----------------------------------------------------------------------------|---------------------------------|
| Daily data volume       | 1 TB/day ( $\approx 1000$ GB)                                               | Uncompressed JSON input         |
| Average record size     | 1 KB                                                                        | $\approx 1$ billion records/day |
| Storage layer           | HDFS / S3 / Delta Lake                                                      |                                 |
| Compression after parse | Parquet ( $\approx 5\times$ smaller)                                        |                                 |
| Cluster type            | Spark on YARN or Kubernetes                                                 |                                 |
| Task                    | JSON parsing $\rightarrow$ schema mapping $\rightarrow$ write Parquet/Delta |                                 |

Batch Job (Run Once per Day):

- Workload Pattern
  - One Spark job runs once a day.
  - Reads 1 TB JSON → parses → writes to Parquet.
  - Target: finish in ~1 hour.
- JSON Parsing Throughput
  - Average: 30 – 50 MB/sec per vCPU.
  - Assume 40 MB/sec/vCPU for estimation.
  - Formula: Total time (sec) =  $(1000 \text{ GB} \times 1024 \text{ MB/GB}) / (\text{Throughput} \times \text{vCPUs})$
  - Target time = 3600 sec →  $\text{vCPUs} \approx (1000 \times 1024) / (40 \times 3600) \approx 7$  cores (ideal)
  - Add ~5× overhead for Spark shuffles, GC, I/O, etc. →  $\approx 40\text{--}50$  vCPUs needed realistically.
- Suggested Infra (Batch): Note that if you're fine with 2–3 hr runtime, halve the cores (~20–25 vCPUs).

| Resource          | Estimate                |
|-------------------|-------------------------|
| Executors         | 10 – 12                 |
| Cores / executor  | 4 – 5                   |
| Total vCPUs       | 40 – 50                 |
| Memory / executor | 16 – 24 GB              |
| Total memory      | 200 – 250 GB            |
| Cluster nodes     | ~5–6 × m5.4xlarge (AWS) |

|      |                             |
|------|-----------------------------|
| Cost | ~ \$3-4 per hour of runtime |
|------|-----------------------------|

#### Near Real-time Streaming Job:

- Workload Pattern
  - Continuous Kafka ingestion rate:  $1 \text{ TB} / 24 \text{ h} = 1000 \text{ GB} / 86400 \text{ s} \approx 11.6 \text{ MB/sec}$
  - Goal: process each micro-batch within seconds or a minute.
- Efficiency
  - Structured Streaming is  $\sim 2-3\times$  less efficient than batch due to micro-batching and checkpointing.
  - Rule of thumb:  $\sim 2$  vCPUs per MB/sec input rate
  - At 11.6 MB/sec input:  $11.6 \times 2 \text{ vCPUs} \approx 23 \text{ vCPUs}$
  - Add 50 % headroom  $\rightarrow \approx 35-40 \text{ vCPUs total (continuous)}$ .
- Suggested Infra (Streaming): Note that streaming runs  $24\times 7$ , unlike batch.

| Resource          | Estimate                                  |
|-------------------|-------------------------------------------|
| Executors         | 8 – 10                                    |
| Cores / executor  | 4                                         |
| Total vCPUs       | 32 – 40                                   |
| Memory / executor | 16 – 24 GB                                |
| Total memory      | 160 – 200 GB                              |
| Cluster nodes     | $\sim 4-5 \times \text{m5.4xlarge (AWS)}$ |



|      |                                                                             |
|------|-----------------------------------------------------------------------------|
| Cost | $\$3\text{--}4/\text{hr} \times 24 \text{ h} = \$75\text{--}100/\text{day}$ |
|------|-----------------------------------------------------------------------------|

### Comparison Summary

| Aspect         | Batch (Daily)             | Streaming (Near Real-time)           |
|----------------|---------------------------|--------------------------------------|
| Data processed | 1 TB once/day             | Continuous 1 TB/day                  |
| Compute needed | ~40–50 vCPUs (1 hr run)   | ~35–40 vCPUs continuous              |
| Memory         | ~200–250 GB               | ~160–200 GB                          |
| Cost           | ~\$3–4/hr (only 1 hr/day) | ~\$75–100/day                        |
| Latency        | Up to 1 day               | Seconds – minutes                    |
| Complexity     | Easier                    | Higher (checkpointing, watermarking) |

### Key Takeaways

- Batch = cheaper, simpler if 1-day delay acceptable. Streaming = real-time insights but higher cost (~20–30× more).
- JSON parsing is CPU-intensive — prefer Avro or Parquet ingestion.
- Autoscaling or serverless Spark (Databricks, EMR on EKS) reduces idle cost.
- Can I use less infra for streaming to save cost?
  - Yes — if your streaming job does very little per-record work, one or two vCPUs may keep up with a modest Kafka throughput. But if your streaming job performs CPU-heavy JSON parsing, joins,

windowing, stateful operations, or frequent checkpoints, you need many more cores to avoid lag and operational issues.

- Trade-offs when you reduce infra
  - Increased processing latency — microbatches take longer to finish → higher end-to-end latency (seconds → minutes → hours).
  - Backpressure and Kafka lag — consumer lag will grow if processing rate < input rate; lag must be stored in Kafka (requires sufficient retention).
  - Larger state / checkpoint growth — slower processing increases state growth and checkpoint sizes, slowing recoveries.
  - Spill to disk / GC pressure — insufficient memory leads to disk spill and longer task times; long GC pauses may cause executor failures.
  - Lower fault tolerance and recovery speed — on failure, recovery needs more time if checkpoints are large and cluster is small.
  - Higher operational complexity — must monitor lag, tune partitions, tune microbatch durations, autoscale carefully.
  - Potential for data loss if retention insufficient — if Kafka retention expires before backlog is processed.
- Example scenarios (for 1 TB/day → 11.6 MB/s)
  - Scenario A — Light work (minimal parsing):
    - $\text{per\_core\_MBps} = 12 \text{ MB/s}$
    - $\text{required\_cores} = 11.6 / 12 \approx 1.0$
    - headroom 2× →  $\text{allocated\_cores} \approx 2$  vCPUs. When appropriate: messages are already compact (AVRO/Protobuf), transforms are trivial, no stateful operations.
  - Scenario B — Medium work (JSON parsing + enrichments)
    - $\text{per\_core\_MBps} = 4 \text{ MB/s}$
    - $\text{required\_cores} = 11.6 / 4 \approx 2.9$

- headroom  $2\times \rightarrow \text{allocated\_cores} \approx 6$  vCPUs  
When appropriate: parsing JSON, some UDFs, light joins, small state windows.
- Scenario C — Heavy work (stateful joins, large windows, heavy UDFs)
  - $\text{per\_core\_MBps} = 1$  MB/s
  - $\text{required\_cores} = 11.6 / 1 \approx 11.6$
  - headroom  $2\times \rightarrow \text{allocated\_cores} \approx 24$  vCPUs  
When appropriate: large keyed state, big aggregations, frequent checkpoints, complex UDFs.
- Note: memory per executor should be aligned to avoid spills (e.g., 16–32 GB per executor depending on JVM overhead and state size). Also distribute workload across many Kafka partitions to parallelize.

---

Question2: Why RAM faster than Disk (HDD/SSD)?

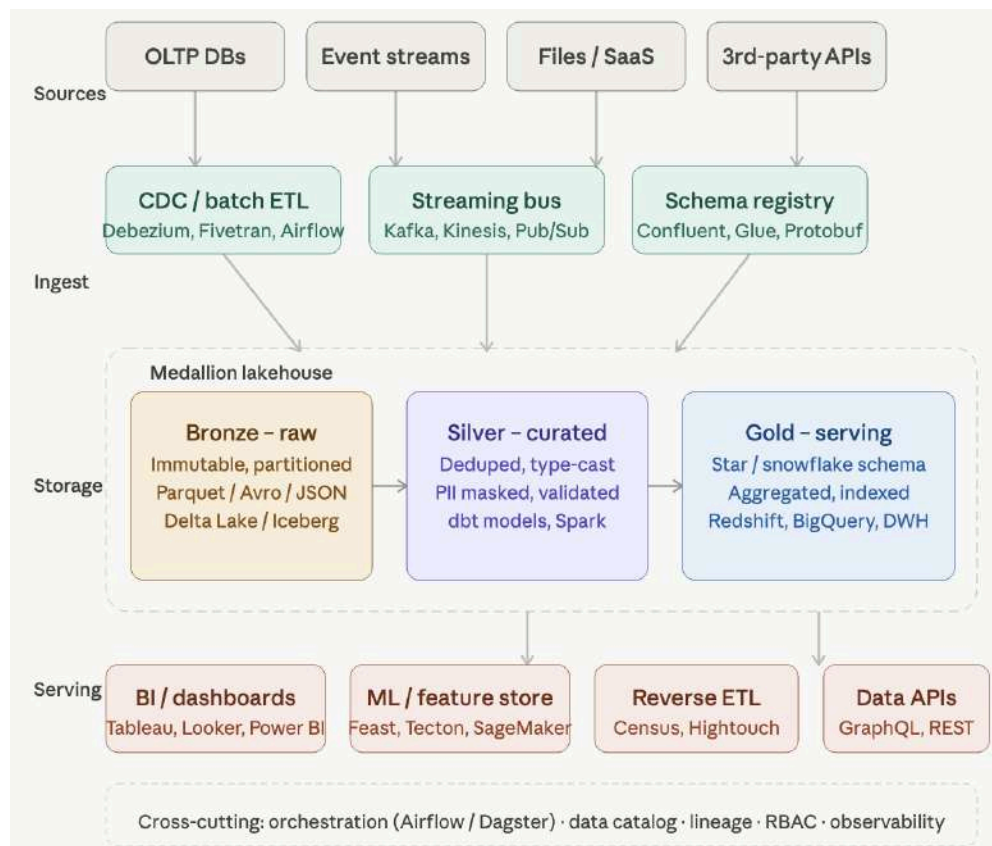
**Answer:**

- **RAM (Memory):** Fully electronic (no moving parts). Data stored using electrical charge in memory cells (transistors + capacitors)
  - Each bit: Charged  $\rightarrow 1$ , Not charged  $\rightarrow 0$
  - Access: Directly connected to CPU via memory bus. CPU can read/write almost instantly
  - Result: Access time: nanoseconds ( $10^{-9}$  sec), Very high bandwidth (GB/s)
  - Nature: Volatile  $\rightarrow$  data lost when power off, Built for speed
- **HDD (Hard Disk Drive - Disk):** Structure: Spinning platters (glass/aluminum base), Coated with magnetic material, Read/write head moves over surface. Data stored: Surface divided into tiny magnetic regions (bits).
  - Each region: Magnetized one way  $\rightarrow 1$ , Opposite direction  $\rightarrow 0$ .
  - Access: To read data: Disk must spin, Head must move (seek) to correct track, Wait for correct sector to rotate under head

- Result: This causes: Seek time + rotational latency, Time in milliseconds ( $10^{-3}$  sec)
  - Nature: Persistent (non-volatile), Optimized for storage capacity & cost, not speed
  - **SSD (Solid State Drive - Disk)**
    - No spinning, no mechanical parts
    - Data stored as electrical charge in flash memory cells
    - Faster than HDD, but still slower than RAM
- 

Question3: Design data warehouse system.

**Answer:**





---

**Question: What improvements have been made in the Spark 4.x version?**

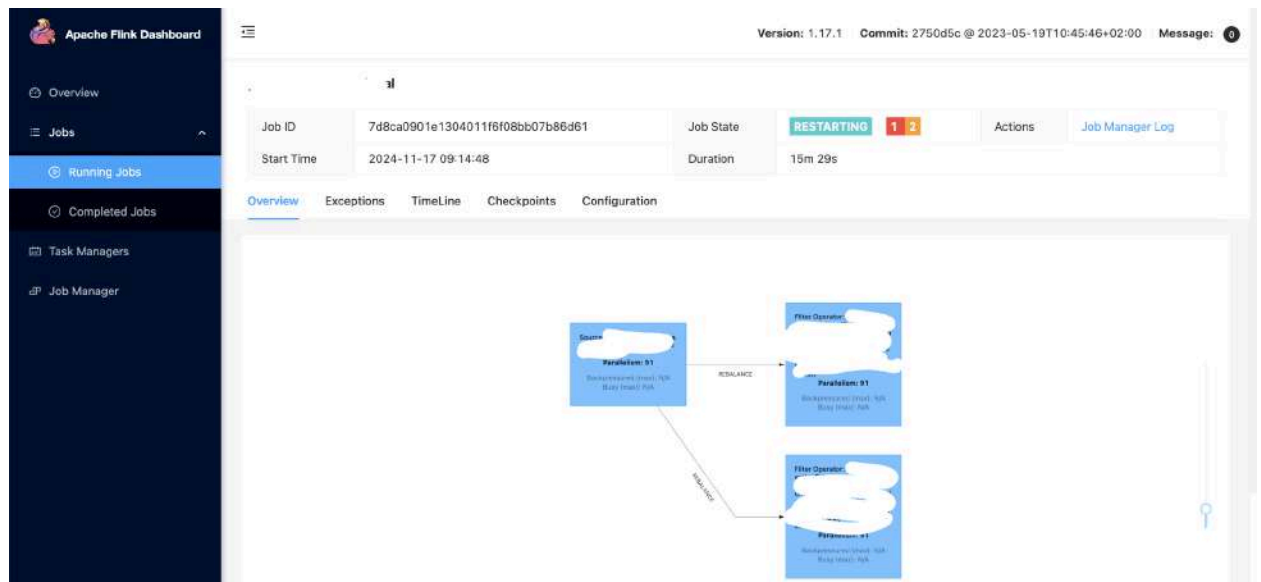
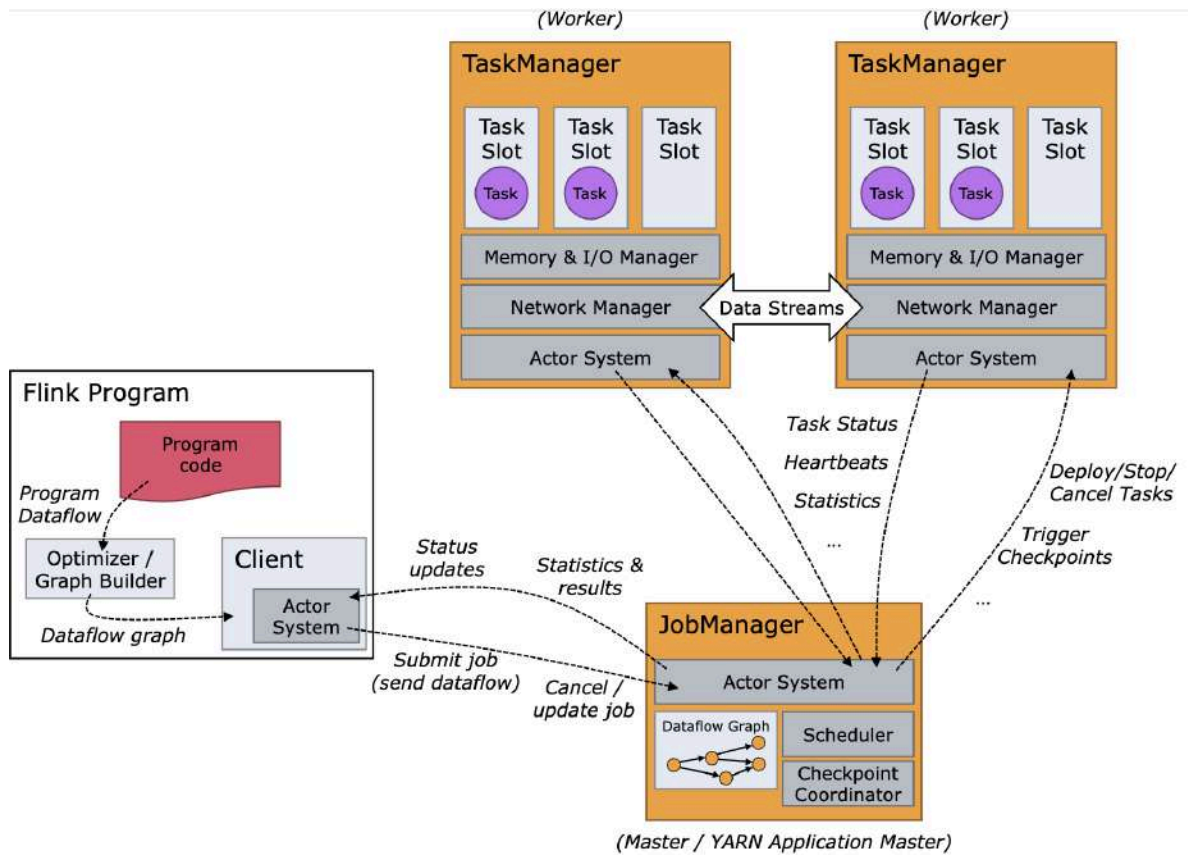
**Answer:**

Spark 4.x mainly improved: Structured Streaming state management, low-latency real-time streaming, Spark Connect, SQL capabilities, PySpark performance, and observability/debugging. Major highlights include: transformWithState API, AQE for streaming, Real-Time Mode, VARIANT type support, and production-ready Spark Connect.

---

Flink

# Flink



- True stream processing engine with event-by-event processing, unlike Spark's micro-batch.
- Components:

- Flink uses an internal RPC framework and Netty-based network stack for communication between JobManagers and TaskManagers. Older Flink versions relied heavily on Akka.
- JobManager: Coordinates distributed execution, Schedules tasks, Manages checkpoints, Tracks job status, Handles failures and recovery.
  - JobManager high availability (HA) works by having multiple JobManager instances, with one acting as the leader and others as standbys, to eliminate the single point of failure.
- TaskManager: Worker nodes that execute tasks, Manage task slots, Buffer and exchange data streams, Maintain state backends
  - Task Slots: Logical fraction of resources in TaskManager or a Container for multiple tasks. Memory isolated, CPU shared.
    - A slot is a resource container that can host multiple subtasks through slot sharing.
    - Total available parallelism = number of TaskManagers × slots per TaskManager
  - Task: One or more chained operators running in one thread. All tasks run simultaneously.
    - All tasks run simultaneously (pipelined execution model)
    - Tasks are separated by network shuffles (e.g., keyBy(), rebalance(), broadcast())
    - Compatible operators are chained into a single task to avoid serialization overhead
  - Threads: Typically, each subtask chain runs in one main task thread.
    - A slot may host multiple subtask threads via slot sharing. Chained operators run in the same thread.
    - Total threads =  $\Sigma(\text{Parallelism of each Task})$ .  
Approximate task execution threads  $\approx$  total parallel subtasks across chained tasks. Actual JVM thread count is higher due to GC, network, RocksDB, async I/O, etc.
    - Parallelism: Number of parallel instances of an operator/task. It can be set per operator using



.setParallelism(N) or globally. Match parallelism to bottleneck and throughput requirements of each stage.

- Client: Submits jobs to JobManager, Can be CLI, web UI, or programmatic
- ResourceManager: Manages task slots, Provisions containers (YARN, Kubernetes, Mesos).

Eg:

Infra:

Assume 2 TaskManagers, 1 JobManager

Machine: 64 cores, 256GB RAM per TaskManager

taskmanager.numberOfTaskSlots::4

Parallelism.default::8 -> Global parallelism, i.e for all operators

Sample Flink Job:

```
DataStream<SensorEvent> stream = env
    .addSource(new FlinkKafkaConsumer<>(...)) // Read from Kafka
    .name("Kafka Source")
    .filter(event-> event.temperature > 0) // Filter invalid data
    .name("Filter Invalid")
    .map(event-> enrichEvent(event)) // Transform/enrich
    .name("Enrich Data")
    .keyBy(event-> event.sensorId) // Partition by sensor- SHUFFLE
    .process(new ApiCallFunction()) // API call per sensor
    .name("API Call")
    .addSink(new JdbcSink<>(...)) // Write to database
    .name("Database Sink");
...
```

LOGICAL RESOURCE ALLOCATION

Step 1: Job Submission

1. Client submits job to JobManager
2. JobManager receives job and creates ExecutionGraph (logical plan)
3. JobManager decides how to chain operators together
4. JobManager assigns parallelism to each operator
5. ResourceManager allocates task slots from TaskManagers

## Step 2: Operator Chaining

Flink chains compatible operators into Tasks to avoid serialization overhead:

Resulting Tasks:

- Task 1: `[Kafka Source → Filter → Map]` (chained together)
- Task 2: `[API Call]` (separated by keyBy shuffle)
- Task 3: `[Database Sink]`

Rules for Chaining: Same parallelism across operators. No shuffle between them (no `keyBy`, `rebalance`, `broadcast`) as shuffle creates task boundary. Connected by forward partitioning (one-to-one). Chaining is enabled (default).

## Step 3: Task Distribution

With parallelism = 8:

```

```
Task 1: [Source → Filter → Map]    Parallelism: 8 → 8 threads
      ↓ (network shuffle via keyBy)
Task 2: [API Call]                  Parallelism: 8 → 8 threads
      ↓
Task 3: [Database Sink]             Parallelism: 8 → 8 threads
> Total threads = 3 tasks × 8 parallelism = 24 threads
```

## Step 4: Slot Allocation

With `taskmanager.numberOfTaskSlots: 4` and 2 TaskManagers:

Total slots available: 2 TaskManagers × 4 slots = 8 slots

This means 3 threads per slot, 24 threads total across 8 slots

## PHYSICAL RESOURCE ALLOCATION

(image pasted after this content \*\*1)

Each slot gets ~16 cores worth of CPU time

More Slots Than Cores (Oversubscription): Good for: I/O-bound workloads (waiting on network/disk/API calls), Bad for: CPU-bound workloads (heavy computation)

Opposite case with undersubscription.

One slot per core is recommended. Although it depends on the

running task if it's CPU or IO intensive.

Other notes:

Pipelining in 3 threads (Multiple events processing simultaneously at different stages)

(image pasted after this content \*\*2)

Backpressure is created if downstream threads queue is getting filled up.

-----

Other ex:

Job:

```
env.setParallelism(8); // Global default
DataStream<Event> stream = env
    .addSource(kafkaSource)
    .setParallelism(10)           // ← Override: 10 Kafka consumers
    .name("Source")
    .filter(e -> e.valid)         // Inherits 10
    .map(e -> enrich(e))          // Inherits 10
    .name("Transform")
    .keyBy(e -> e.key)            // ← SHUFFLE: Task boundary
    .window(...)
    .aggregate(...)
    .setParallelism(20)          // ← Override: 20 window operators
    .name("Windowed Agg")
    .keyBy(e -> e.userId)         // ← SHUFFLE: Task boundary
    .process(new ApiCall())
    .setParallelism(4)           // ← Override: Only 4 API callers
    .name("API")
    .map(e -> format(e))          // Inherits 4
    .keyBy(e -> e.region)         // ← SHUFFLE: Task boundary
    .addSink(jdbcSink)
    .setParallelism(16);         // ← Override: 16 DB writers
```

Logical Tasks Created

Task 1: [Source → Filter → Map] Parallelism: 10 → 10 threads  
↓ (keyBy shuffle)

Task 2: [Window → Aggregate]      Parallelism: 20 → 20 threads  
      ↓ (keyBy shuffle)  
Task 3: [Process (API) → Map]      Parallelism: 4 → 4 threads  
      ↓ (keyBy shuffle)  
Task 4: [Sink]                      Parallelism: 16 → 16 threads

Total threads = 10 + 20 + 4 + 16 = 50 threads

Slots needed = max(10, 20, 4, 16) = 20 slots (with slot sharing)

### Physical Slot Distribution

With 20 slots available (5 TaskManagers × 4 slots):

SLOT DISTRIBUTION WITH SLOT SHARING:

|          |          |             |            |             |             |
|----------|----------|-------------|------------|-------------|-------------|
| Slot 1:  | T1-inst0 | + T2-inst0  | + T3-inst0 | + T4-inst0  | = 4 threads |
| Slot 2:  | T1-inst1 | + T2-inst1  | + T3-inst1 | + T4-inst1  | = 4 threads |
| Slot 3:  | T1-inst2 | + T2-inst2  | + T3-inst2 | + T4-inst2  | = 4 threads |
| Slot 4:  | T1-inst3 | + T2-inst3  | + T3-inst3 | + T4-inst3  | = 4 threads |
| Slot 5:  | T1-inst4 | + T2-inst4  | + T3-empty | + T4-inst4  | = 3 threads |
| Slot 6:  | T1-inst5 | + T2-inst5  | + T3-empty | + T4-inst5  | = 3 threads |
| Slot 7:  | T1-inst6 | + T2-inst6  | + T3-empty | + T4-inst6  | = 3 threads |
| Slot 8:  | T1-inst7 | + T2-inst7  | + T3-empty | + T4-inst7  | = 3 threads |
| Slot 9:  | T1-inst8 | + T2-inst8  | + T3-empty | + T4-inst8  | = 3 threads |
| Slot 10: | T1-inst9 | + T2-inst9  | + T3-empty | + T4-inst9  | = 3 threads |
| Slot 11: | T1-empty | + T2-inst10 | + T3-empty | + T4-inst10 | = 2 threads |
| Slot 12: | T1-empty | + T2-inst11 | + T3-empty | + T4-inst11 | = 2 threads |
| Slot 13: | T1-empty | + T2-inst12 | + T3-empty | + T4-inst12 | = 2 threads |
| Slot 14: | T1-empty | + T2-inst13 | + T3-empty | + T4-inst13 | = 2 threads |
| Slot 15: | T1-empty | + T2-inst14 | + T3-empty | + T4-inst14 | = 2 threads |
| Slot 16: | T1-empty | + T2-inst15 | + T3-empty | + T4-inst15 | = 2 threads |
| Slot 17: | T1-empty | + T2-inst16 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 18: | T1-empty | + T2-inst17 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 19: | T1-empty | + T2-inst18 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 20: | T1-empty | + T2-inst19 | + T3-empty | + T4-empty  | = 1 thread  |

Total: 50 threads across 20 slots

Average: 2.5 threads per slot

## Per-TaskManager View

TaskManager 1 (32 cores, 4 slots):

Slot 1: 4 threads }  
Slot 2: 4 threads } 16 threads total  
Slot 3: 4 threads } competing for 32 cores  
Slot 4: 4 threads }

TaskManager 2 (32 cores, 4 slots):

Slot 5: 3 threads }  
Slot 6: 3 threads } 12 threads total  
Slot 7: 3 threads } competing for 32 cores  
Slot 8: 3 threads }

TaskManager 3 (32 cores, 4 slots):

Slot 9: 3 threads }  
Slot 10: 3 threads } 12 threads total  
Slot 11: 2 threads } competing for 32 cores  
Slot 12: 2 threads }

TaskManager 4 (32 cores, 4 slots):

Slot 13: 2 threads }  
Slot 14: 2 threads } 8 threads total  
Slot 15: 2 threads } competing for 32 cores  
Slot 16: 2 threads }

TaskManager 5 (32 cores, 4 slots):

Slot 17: 1 thread }  
Slot 18: 1 thread } 4 threads total  
Slot 19: 1 thread } competing for 32 cores (underutilized!)  
Slot 20: 1 thread }

How Data Flows with Different Parallelism

Network Shuffle Behavior

10 Source instances reading from Kafka



[Rebalance/Hash Partition by key]  
↓  
20 Window instances (each receives from ~0.5 sources on average)  
↓  
[Hash Partition by userId]  
↓  
4 API instances (each receives from ~5 window instances)  
↓  
[Hash Partition by region]  
↓  
16 Sink instances (each receives from ~0.25 API instances)

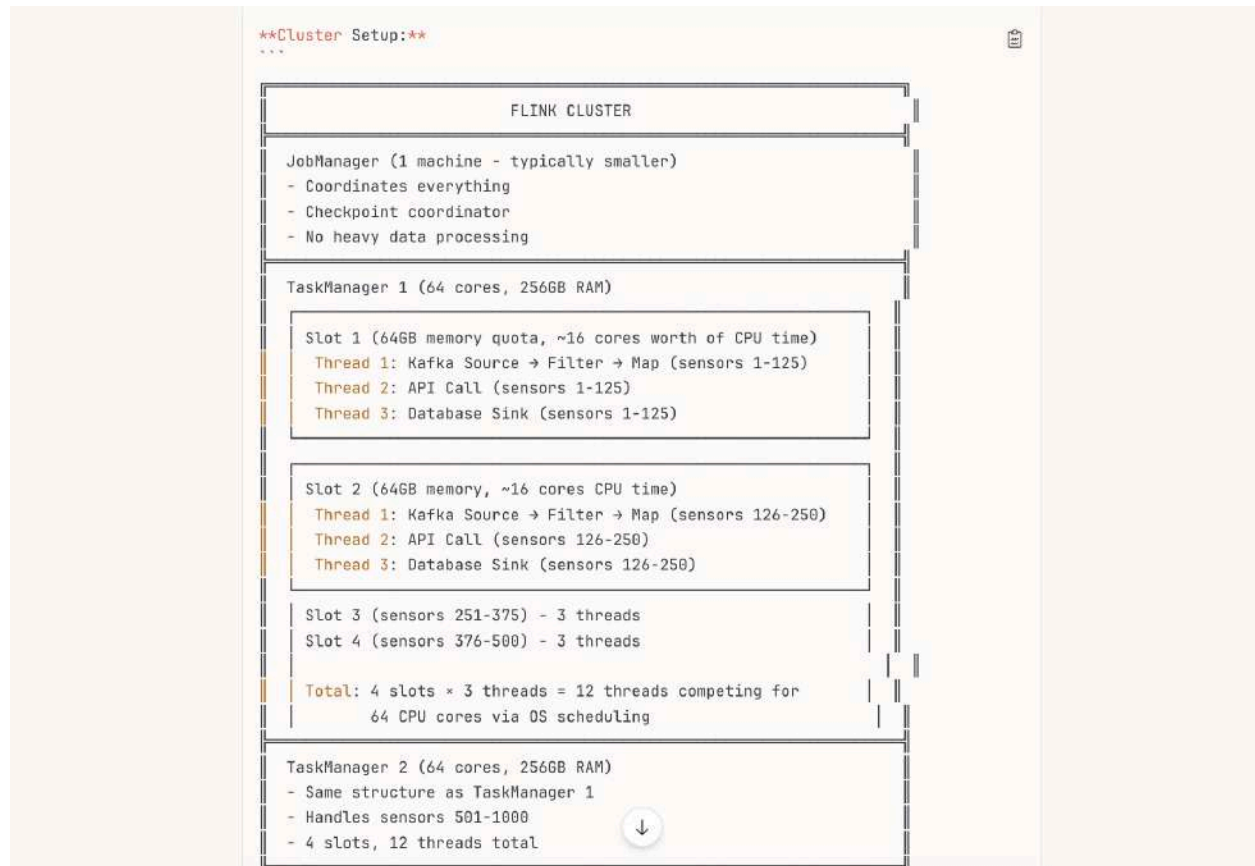
Example event flow:

Event with key="sensor123", userId="user456", region="us-east"

1. Read by Source-instance-7 (one of 10)
2. Filter + Map in Source-instance-7
3. Network shuffle:  $\text{hash}(\text{"sensor123"}) \% 20 = 13$   
→ Sent to Window-instance-13
4. Windowing in Window-instance-13
5. Network shuffle:  $\text{hash}(\text{"user456"}) \% 4 = 2$   
→ Sent to API-instance-2
6. API call in API-instance-2
7. Map in API-instance-2
8. Network shuffle:  $\text{hash}(\text{"us-east"}) \% 16 = 8$   
→ Sent to Sink-instance-8
9. DB write in Sink-instance-8

All threads use OS scheduling to compete for the 32 cores

(Physical resource allocation \*\*1)



(Pipelining in thread \*\*2)

### How Events Flow (Pipelined)

**\*\*3 threads are active simultaneously, processing different events:\*\***

| Time  | Thread 1<br>(Source) | Thread 2<br>(API) | Thread 3<br>(Sink) |
|-------|----------------------|-------------------|--------------------|
| 0ms   | Read e1              |                   |                    |
| 1ms   | Filter e1            |                   |                    |
| 3ms   | Read e2              | Start API(e1)     |                    |
| 4ms   | Filter e2            | API(e1) running   |                    |
| 6ms   | Read e3              | API(e1) running   |                    |
| 103ms | Read e35             | API(e1) done      | Start Write(e1)    |
| 104ms | Filter e35           | Start API(e2)     | Write(e1)          |
| 113ms | Read e36             | API(e2) running   | Write(e1) done     |
| 114ms | Filter e36           | API(e2) running   | Start Write(e2)    |

**\*\*Key Insights:\*\***

- Thread 1 doesn't wait for Thread 2 or 3
- Events are **\*\*buffered** in queues between operators
- Multiple events are "in flight" at different stages simultaneously
- Maximum throughput achieved through pipelining

- Watermarks:
  - Special markers in stream indicating event time has progressed to certain point. Tell Flink when to trigger window computations. Events older than the current watermark may be considered late depending on window boundaries and allowed lateness.
  - Operators forward minimum watermark from all input streams (input unions, joins, etc.). Ensures correctness across parallel operations.
  - Watermarking if: aggressive - data loss, conservative - data lag.
  - Types:
    - Periodic Watermarks:
      - Generated at fixed intervals (default: 200ms)
      - Based on maximum observed timestamp minus allowed lateness

```
Eg: DataStream<Event> stream = env
    .addSource(new FlinkKafkaConsumer<>("events", new
    EventDeserializationSchema(), props))
```



```

        .assignTimestampsAndWatermarks(
            WatermarkStrategy

.<Event>forBoundedOutOfOrderness(Duration.ofSeconds(5))
            .withTimestampAssigner((event, ts) ->
event.getEventTime())
            .withIdleness(Duration.ofMinutes(1)) // handle idle
Kafka partitions
        );
- Emits watermark every 200ms by default.
- forBoundedOutOfOrderness: Allows events that are up to 5s
  late (within watermark delay).
- withIdleness(): prevents watermark stalls if one Kafka
  partition stops producing.
- withTimestampAssigner(...): To extract event time from
  payload. Pattern: ISO-8601 string → epoch milliseconds, as
  Flink uses epoch millis internally for event time

```

#### ■ Punctuated Watermarks:

- Generated based on special markers in stream.
- Event-driven watermark generation

Eg:

```

DataStream<Event> stream = env
    .addSource(new CustomSourceFunction())
    .assignTimestampsAndWatermarks(
        new WatermarkStrategy<Event>() {
            @Override
            public WatermarkGenerator<Event>
createWatermarkGenerator(WatermarkGeneratorSupplier.Context
context) {
                return new WatermarkGenerator<Event>() {
                    private long currentMaxTimestamp =
Long.MIN_VALUE;

                    @Override
                    public void onEvent(Event event, long
eventTimestamp, WatermarkOutput output) {

```

```

        currentMaxTimestamp =
Math.max(currentMaxTimestamp, eventTimestamp);
        // Emit punctuated watermark when special flag is seen
        if (event.isEndOfBatch()) {
            output.emitWatermark(new
Watermark(currentMaxTimestamp - 1));
        }
    }
    @Override
    public void onPeriodicEmit(WatermarkOutput
output) {
        // No-op for punctuated watermark
    }
};
}
@Override
public TimestampAssigner<Event>
createTimestampAssigner(TimestampAssignerSupplier.Context ctx) {
    return (event, recordTimestamp) ->
event.getTime(); });

```

- To handle late arriving data:
  - Use Allowed Lateness: Eg:
 

```
stream.keyBy(...).window(TumblingEventTimeWindows.of(Time
e.minutes(1))).allowedLateness(Time.minutes(5)).sideOutputLa
teData(lateTag.aggregate(...))
```
  - Side Outputs: Redirect late data to separate stream for special
 handling, can reprocess, log, or send to dead letter queue.

```

final OutputTag<Event> lateEventsTag = new
OutputTag<Event>("late-events"){
};
SingleOutputStreamOperator<AggregatedEvent> result = stream
    .keyBy(Event::getKey)
    .window(TumblingEventTimeWindows.of(Time.minutes(1)))
    .allowedLateness(Time.minutes(5))
    .sideOutputLateData(lateEventsTag)
    .aggregate(new MyAggregateFunction());
// Retrieve late data for special handling

```

```
DataStream<Event> lateStream =
result.getSideOutput(lateEventsTag);
lateStream
    .addSink(new DeadLetterQueueSink<>("late-events-dlq"));
```

- Watermark Configuration: Increase allowed out-of-orderness; Tradeoff: Higher latency before window triggers

```
env.getConfig().setAutoWatermarkInterval(1000L); // emit every 1
second instead of 200ms
```

- Window Reassignment: Custom logic to reassign late events to appropriate windows
- Windows: A window defines how data in a stream is grouped for processing over time or count. Since data in streams is unbounded, windows help you create bounded slices of it for aggregation or analysis. Flink decides window boundaries based on
  - > Event time (Timestamp when event actually occurred. Embedded in the event record),
  - > Processing time (Timestamp when event processed by Flink operator,
  - > Non-deterministic (Depends on system speed)), or
  - > Ingestion time (Timestamp when event enters Flink source)
- Types:
  - Tumbling Windows: Fixed-size, non-overlapping. Each event belongs to exactly one window. Eg:
 

```
TumblingEventTimeWindows.of(Time.minutes(5)).
```

    - Use case: 5-minute aggregations, Periodic reports (e.g., sales per 5 minutes).
 

```
|----5m----|----5m----|----5m----|
```
  - Sliding Windows: Fixed size with configurable slide interval. Windows can overlap. Eg:
 

```
SlidingEventTimeWindows.of(Time.minutes(10),
Time.minutes(5))...
```

    - Use case: Moving averages, rolling metrics
 

```
|-----10m-----|
|-----10m-----|
```

|-----10m-----|

- Session Windows: Dynamic size based on inactivity gap. No fixed duration. The windows automatically merge when events arrive within the inactivity gap. Eg:  
`EventTimeSessionWindows.withGap(Time.minutes(10))`
  - Use case: User session analysis, burst detection  
User clicks → gap < 10m → same window  
User idle → gap > 10m → new session starts
- Global Windows: Single window containing all events, Requires custom trigger.
  - Use case: Custom windowing logic
- Count Windows: Based on number of elements. Eg:  
`countWindow(100) // tumbling. countWindow(100, 10) // sliding`
  - Use case: Batch-style aggregation
- Triggers: Control when window is evaluated
  - `EventTimeTrigger`: Fire when watermark passes window end
  - `ProcessingTimeTrigger`: Fire based on processing time
  - `CountTrigger`: Fire after N elements
  - `ContinuousEventTimeTrigger`: Fire periodically within window
  - Custom Triggers: Implement `TriggerResult` (CONTINUE, FIRE, PURGE, FIRE\_AND\_PURGE)
    - FIRE emits results but keeps data for future updates; FIRE\_AND\_PURGE emits and clears state to free memory.
- Evictors: Remove elements from window before/after computation
  - `TimeEvictor`: Keep only elements within time range
  - `CountEvictor`: Keep only last N elements
  - `DeltaEvictor`: Keep elements within delta threshold
  - Custom Evictors: Implement `Evictor` interface

Eg:  
stream

```

.keyBy(...)
.window(GlobalWindows.create())
.trigger(CountTrigger.of(1000))
.evictor(TimeEvictor.of(Time.hours(1)))
.reduce(...)

```

Visual:

|               |                                     |    |    |    |    |
|---------------|-------------------------------------|----|----|----|----|
| Event Stream: | E1                                  | E2 | E3 | E4 | E5 |
| Timestamps:   | 1s                                  | 2s | 4s | 7s | 9s |
| Window:       | [0 ----- 10)                        |    |    |    |    |
| Watermark:    | ↑ (fires trigger at 10s)            |    |    |    |    |
| Trigger:      | CONT → CONT → CONT → CONT → FIRE    |    |    |    |    |
| Evictor:      | (removes old events before compute) |    |    |    |    |

- State Management: “state” means remembering information about past events to influence future processing. Flink stores and manages this state fault-tolerantly across operators and keys.
  - Types:
    - Keyed State: Scoped per key (after a `keyBy()` operation). Only accessible in keyed streams. Each key has its own isolated state partition. Eg: `ValueState<T>` (Single value per key), `ListState<T>` (List of values), `MapState<K,V>`, `ReducingState<T>`, etc.
    - Operator State: Scoped per operator instance, not per key. Used when partitioning isn’t applied (e.g., non-keyed source/sink). Eg: `ListState`, `UnionListState`, `BroadcastState<K,V>`
  - The State Backend determines how and where state is stored:
    - `HashMapStateBackend`: Stores state in-memory (JVM heap), fast, snapshots to external storage, suitable for smaller state workloads but not ideal for very large production state.
    - `EmbeddedRocksDBStateBackend`: RocksDB (disk), slower but scalable, best for prod.

```

StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

```

```
env.setStateBackend(new EmbeddedRocksDBStateBackend());
env.getCheckpointConfig().setCheckpointStorage("s3://flink-checkpoints");
```

- State TTL (Time-To-Live): TTL prevents unbounded state growth. Automatic expiration of state after configured time. Can be incremental, snapshot based, compaction.

```
StateTtlConfig ttlConfig = StateTtlConfig
    .newBuilder(Time.days(7))
    .setUpdateType(StateTtlConfig.UpdateType.OnCreateAndWrite)
    .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)
    .cleanupInRocksdbCompactFilter()
    .build();
```

Types:

OnCreateAndWrite: Reset TTL on creation and writes

OnReadAndWrite: Reset TTL on reads and writes

Disabled: TTL never updated

- State Rescaling: When you change job parallelism, Flink redistributes state. Works accordingly for Keyed and Operator states.
- Checkpoints & Recovery: State is snapshotted periodically into durable storage (e.g., S3, HDFS) On recovery, Flink restores state exactly as before the failure. Steps:
  - Aligned Checkpoints:
    - Triggering:
      - The JobManager initiates a checkpoint periodically (as configured).
      - Checkpoint barriers are injected into the source tasks (Kafka, S3, etc.).
    - Barrier Propagation:
      - These barriers flow through the data stream, tagging all records as belonging to a specific checkpoint (say, #42).
      - Records before the barrier → belong to checkpoint 41.

- Records after the barrier are processed beyond checkpoint #42.
- State Snapshotting:
  - When an operator has received a barrier for checkpoint #42 from all its inputs, it:
    - Operators buffer channels that already received checkpoint barriers until all input barriers arrive.
    - Snapshots its current operator state + keyed state asynchronously.
    - Persists this state to a checkpoint storage backend (e.g., S3, HDFS).
- Barrier Acknowledgement:
  - Once all operators complete their snapshots, ack messages go to the JobManager.
  - When all are acked → checkpoint considered complete.
  - No global stop — processing continues concurrently (async snapshotting), just locally the operator may pause & faster upstream inputs would be buffered, downstream operators would still be running; Once checkpoint complete, then the local operator resumes.
- What causes checkpoint alignment delays?
  - Skewed input rates — one Kafka partition slower than others.
  - Large operator state — snapshot takes long.
  - High backpressure — data flow uneven, barriers delayed.
  - Network congestion — delays barrier propagation.
  - Asynchronous snapshot thread saturation in RocksDB.
- Unaligned checkpoints: Operators don't wait for barrier alignment, In-flight records are included in checkpoint snapshots hence big checkpoint size, Checkpoints complete even under heavy backpressure.
- Others:
  - Checkpoint Components: Operator state, Keyed state, In-flight records (for exactly-once).

- Flink guarantees exactly-once consistency for internal state. External exactly-once depends on sink support for transactional or idempotent writes.
- Checkpoint: Automatic, periodic, lightweight, may be deleted  
AND Savepoint: Manual, for versioning, upgrades, always retained

```
env.enableCheckpointing(60000); // Every 60 seconds
env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
env.getCheckpointConfig().setMinPauseBetweenCheckpoints(30000);
env.getCheckpointConfig().setCheckpointTimeout(600000);
env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
env.getCheckpointConfig().setTolerableCheckpointFailureNumber(3);
```

- Exactly-once in Flink:
  - Exactly-Once Processing: Internal state updated exactly once per event, Achieved through checkpointing, Flink guarantees this for state updates
  - Exactly-Once Delivery: Each event written to sink exactly once, Requires transactional or idempotent sinks, Harder to achieve, depends on external system. Delivery Guarantees:
    - At-Most-Once: Fire and forget, No checkpointing needed, Data loss possible, Lowest overhead
    - At-Least-Once: Checkpoint and replay on failure, Duplicates possible, Most sinks support this
    - Exactly-Once: Transactional sinks or idempotent writes, No duplicates or loss, Highest overhead.
  - Steps:
    - Source:
      - Must be replayable (Kafka offsets, file positions)
      - Checkpoint source positions
      - Reset to checkpoint on recovery
    - Internal Processing:
      - Barrier snapshotting ensures consistent state
      - All operators see same snapshot of inputs



- Sink:
  - Transactional Sinks: Two-phase commit (2PC)
  - Idempotent Sinks: Deterministic IDs, overwrite on replay.  
Eg: `env.enableCheckpointing(60000);`  
`sink.setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)`
- Partitioning Strategy:
  - It defines how records are distributed across parallel subtasks in Flink's dataflow graph. This directly impacts state locality, load balancing, and network shuffle costs.
  - Strategies:
    - KeyBy: `stream.keyBy(event -> event.getUserId())`
      - Hash partition by key
      - keyBy hashes records into key-groups which are assigned to subtasks. The same key always maps to the same subtask.
      - Enables keyed state
      - Can cause data skew if keys not uniformly distributed
    - Rebalance: `stream.rebalance()`
      - Round-robin distribution
      - Even load distribution
      - Causes network shuffle
      - Use when dealing with skewed data
    - Rescale: `stream.rescale()`
      - Local round-robin to subset of downstream tasks
      - Less network overhead than rebalance
      - Use when upstream and downstream parallelism are multiples
    - Forward:
      - One-to-one connection (no shuffle)
      - Preserves partitioning
      - Lowest overhead
    - Broadcast: `stream.broadcast()`
      - Send all elements to all parallel instances

- Expensive, use sparingly
  - Useful for small reference data
- Handle/detect data skew:
  - Flink UI to detect skewness.
  - Key Salting. Eg: `.keyBy(event -> event.getUserId() + "_" + (event.getUserId().hashCode() % 4))`
  - Rebalance Before Stateful Operator: Helps redistribute load before `keyBy` when skewed distribution is known. Eg: `rebalance().keyBy(...)`
  - Dynamic Load Balancing: Implement custom partitioner. Eg: `stream.partitionCustom(new CustomPartitioner(), event -> event.getKey());`
  - Scale Up Parallelism: Could operator parallelism. Eg: `env.setMaxParallelism(128);`
    - `maxParallelism` is different from runtime parallelism. It defines the number of key-groups used for keyed-state partitioning and future rescaling flexibility.
- Process Functions: Process Functions are low-level API for stream processing. They give fine-grained control over: Event-time / processing-time timers, Custom logic beyond windowing / simple transformations, State access (keyed or operator), Side outputs (for late data or special cases). Types:
  - `ProcessFunction<IN, OUT>`: Operates on non-keyed streams. Gives access to processing time, event time, and side outputs. Cannot use keyed state (only operator state).

```
stream.process(new ProcessFunction<Event, Result>() {
    @Override
    public void processElement(Event event, Context ctx,
        Collector<Result> out) {
        out.collect(new Result("Received: " + event));
    }
});
```

- `KeyedProcessFunction<K, IN, OUT>`: Operates on keyed streams (after `.keyBy()`). Can use keyed state (`ValueState`, `ListState`, etc.). Can

register event-time or processing-time timers per key.

- Non-keyed stream: A normal `DataStream` where all events are processed together by each parallel subtask, without partitioning by key. Every operator instance gets a subset of the stream — distributed in parallel — but no key-based grouping is done.
- Keyed Stream: Created by applying `.keyBy(...)` on a stream. Partitions the stream by key value, so all records with the same key go to the same parallel subtask. Usecase: If you need to track information per user, session, merchant, device, etc., like: Counting clicks per user:

Example for Keyed stream:

```
stream
    .keyBy(Event::getUserId)
    .process(new KeyedProcessFunction<String, Event,
UserCount>() {
        private ValueState<Integer> count;
        public void open(Configuration parameters) {
            count = getRuntimeContext().getState(new
ValueStateDescriptor<>("count", Integer.class));
        }
        public void processElement(Event event, Context ctx,
Collector<UserCount> out) throws Exception {
            int newCount = (count.value() == null ? 0 :
count.value()) + 1;
            count.update(newCount);
            out.collect(new UserCount(event.getUserId(),
newCount));
        }
    });
```

Example for Keyed Process function:

```
sstream.keyBy(Event::getUserId)
    .process(new KeyedProcessFunction<String, Event, Result>() {
        ...
    })
```

- `ProcessWindowFunction<IN, OUT, KEY, W extends Window>`: Used inside window operations. Gives access to window metadata (start/end timestamp). Provides all elements of the window for full aggregation.

```
stream.keyBy(Event::getUserId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .process(new ProcessWindowFunction<Event, Result, String,
TimeWindow>() {
        @Override
        public void process(String key, Context ctx,
Iterable<Event> events, Collector<Result> out) {
            long count =
StreamSupport.stream(events.spliterator(), false).count();
            out.collect(new Result(key, count,
ctx.window().getEnd()));
        }
    });
```

- `ProcessAllWindowFunction<IN, OUT, W extends Window>`: Same as above, but for non-keyed windows (global).

```
stream.windowAll(TumblingProcessingTimeWindows.of(Time.seconds(10)
))
    .process(new ProcessAllWindowFunction<Event, Result,
TimeWindow>() {
        @Override
        public void process(Context ctx, Iterable<Event> events,
Collector<Result> out) {
            out.collect(new Result("Total count = " +
StreamSupport.stream(events.spliterator(), false).count()));
        }
    });
```

- `CoProcessFunction<IN1, IN2, OUT>`: Processes two connected streams together. Can access state shared between both inputs.

```

DataStream<EventA> streamA = ...
DataStream<EventB> streamB = ...
streamA.connect(streamB)
    .process(new CoProcessFunction<EventA, EventB, Result>() {
        private ValueState<EventA> pendingA;
        ...
    })

```

- Async I/O in Flink: Avoid blocking operators waiting for external service calls. Increase throughput with concurrent requests. Reduce latency. Use in API calls, DB reqs and lookups, etc.
  - `orderedWait()` preserves output ordering but may introduce head-of-line blocking if slower requests delay later completed results.
  - `unorderedWait()`: Allow reordering (higher throughput)

Eg:

```

class AsyncDatabaseRequest extends RichAsyncFunction<String,
Tuple2<String, String>> {
    private transient DatabaseClient client;

    @Override
    public void open(Configuration parameters) {
        client = new DatabaseClient();
    }

    @Override
    public void asyncInvoke(String key,
ResultFuture<Tuple2<String, String>> resultFuture) {
        CompletableFuture<String> future = client.asyncQuery(key);

        future.whenComplete((result, error) -> {
            if (error != null) {
                resultFuture.completeExceptionally(error);
            } else {
                resultFuture.complete(Collections.singleton(new
Tuple2<>(key, result)));
            }
        });
    }
}

```

```

        }
    });
}

@Override
public void timeout(String input, ResultFuture

```

- Broadcast State pattern: Broadcasting rules/config to all parallel instances for enrichment. Each task (operator instance) maintains its own replica of the broadcast state (i.e its a type of operator state replicated across all parallel subtasks), ensuring every event is processed with the latest rules – without network lookups. Broadcast state is fully replicated across all parallel subtasks → memory pressure grows linearly with number of subtasks × state size. To Manage Broadcast State Growth: Evict stale entries, State TTL, Compaction, Shard/split broadcast stream (multiple streams), Use rocksdb backend.

```

// Stream 1: Rules/config (low volume)
BroadcastStream<Rule> broadcastStream = rulesStream
    .broadcast(rulesStateDescriptor);

// Stream 2: Events (high volume)
DataStream<Event> eventStream = ...;

```

```
// Connect and process
eventStream
    .connect(broadcastStream)
    .process(new BroadcastProcessFunction<Event, Rule, Result>() {
        @Override
        public void processElement(Event event, ReadOnlyContext
ctx, Collector<Result> out) {
            // Read broadcast state
            for (Map.Entry<String, Rule> entry :

ctx.getBroadcastState(rulesStateDescriptor).immutableEntries()) {
                Rule rule = entry.getValue();
                if (rule.matches(event)) {
                    out.collect(new Result(event, rule));
                }
            }
        }

        @Override
        public void processBroadcastElement(Rule rule, Context
ctx, Collector<Result> out) {
            // Update broadcast state

ctx.getBroadcastState(rulesStateDescriptor).put(rule.getId(),
rule);
        }
    });
```

- Side Output: Additional output streams from operators for specific use cases. Used for late data handling, error streams, etc.

```
final OutputTag<Event> lateDataTag = new
OutputTag<Event>("late-data"){ };
final OutputTag<Event> errorTag = new
OutputTag<Event>("errors"){ };
```

```

SingleOutputStreamOperator<Result> mainStream = stream
    .process(new ProcessFunction<Event, Result>() {
        @Override
        public void processElement(Event event, Context ctx,
Collector<Result> out) {
            try {
                if (event.getTimestamp() <
ctx.timerService().currentWatermark()) {
                    // Late data
                    ctx.output(lateDataTag, event);
                } else if (!event.isValid()) {
                    // Invalid data
                    ctx.output(errorTag, event);
                } else {
                    // Normal processing
                    out.collect(process(event));
                }
            } catch (Exception e) {
                ctx.output(errorTag, event);
            }
        }
    });

// Get side outputs
DataStream<Event> lateData =
mainStream.getSideOutput(lateDataTag);
DataStream<Event> errors = mainStream.getSideOutput(errorTag);

```

- A `SingleOutputStreamOperator<T>` represents a transformation in your Flink job that produces one output stream of type `T`. This operator produces exactly one output stream. For transformations that produce multiple output types (like side outputs), Flink uses: `ProcessFunction + OutputTag` to produce multiple outputs (main + side).
  - `DataStream` → the base abstraction.
  - `SingleOutputStreamOperator` → a `DataStream` that comes from an operation producing one output.
  - `KeyedStream`, `WindowedStream` → specializations for



keyed/windowed data.

- Async Functions:
  - AsyncFunction<IN, OUT>: Lightweight - just asyncInvoke()
  - RichAsyncFunction<IN, OUT>: + lifecycle hooks (open, close, getRuntimeContext). Any "Rich" prefix = Lifecycle + RuntimeContext access Examples: RichMapFunction, RichFlatMapFunction, RichAsyncFunction RuntimeContext: Provides metadata & utilities about running task
- Joins:
  - For event-time Join between two streams, can use intervalJoin.

```
SingleOutputStreamOperator<JoinedEvent> joinedStream =
    leftStream
        .keyBy(LeftEvent::getJoinKey)
        .intervalJoin(rightStream.keyBy(RightEvent::getJoinKey))
        .between(Time.seconds(-60), Time.seconds(60))
        .process(new ProcessJoinFunction<LeftEvent, RightEvent,
JoinedEvent>() {
            @Override
            public void processElement(
                LeftEvent left,
                RightEvent right,
                Context ctx,
                Collector<JoinedEvent> out
            ) throws Exception {
                JoinedEvent joined = new JoinedEvent();
                joined.setId(UUID.randomUUID().toString());
                joined.setTimestamp(right.getEventTimestamp());
                // Combine/enrich from both sides
                out.collect(joined);
            }
        });
```

How .between() Works

For each LEFT event at time T:

Match RIGHT events where:

$\text{rightTime} \in [\text{leftTime} - 60\text{s}, \text{leftTime} + 60\text{s}]$

Example:

Left event at 12:00:30

Matches right events from 11:59:30 to 12:01:30

Key Points

Requirement: Both streams must be keyed on join key

Time-based: Uses event time (not processing time)

Symmetric window: -60s to +60s means 2-minute total window

ProcessJoinFunction: Defines join logic & output type

- Sinks (S3/Kafka, etc):
  - FileSink API to write in S3. Kafka methods to publish in Kafka.

```
// FileSink API:
OutputFileConfig fileConfig = OutputFileConfig.builder()
    .withPartPrefix("user-data-")
    .withPartSuffix(".json")
    .build();
FileSink<T> sink = FileSink
    .forRowFormat(new Path(s3BasePath), new Encoder<T>() {
        @Override
        public void encode(T element, OutputStream stream) throws
IOException {
            String json =
objectMapper.writeValueAsString(element);
            stream.write(json.getBytes(StandardCharsets.UTF_8));
            stream.write('\n'); // Newline-delimited JSON
        }
    })
    .withBucketAssigner(new
DateTimeBucketAssigner<>("yyyy-MM-dd"))
    .withRollingPolicy(
        DefaultRollingPolicy.builder()
            .withRolloverInterval(5000) // Roll every 5s
            .withInactivityInterval(5000) // Roll after 5s idle
            .withMaxPartSize(1024) // Roll after 1KB
            .build()
```

```
)  
.withOutputFileConfig(fileConfig)  
.build();
```

Info:

BucketAssigner: Organizes files by date/time

RollingPolicy: When to close & start new files. Like: Rollover

interval: Time-based, Inactivity: Idle period, Max size: File size threshold.

Encoder: Custom serialization logic

-----

// Kafka Sink:

```
KafkaRecordSerializationSchema<Event> schema =  
    KafkaRecordSerializationSchema.<Event>builder()  
        .setTopic("output-topic")  
        .setValueSerializationSchema(new EventSchema())  
        .build();
```

```
KafkaSink<Event> kafkaSink = KafkaSink.<Event>builder()  
    .setBootstrapServers("broker1:9092,broker2:9092")  
    .setRecordSerializer(schema)  
    .build();
```

// Usage

```
stream.sinkTo(kafkaSink);
```

Info:

KafkaRecordSerializationSchema: How to serialize + which topic

ValueSerializationSchema: Custom serializer for your type

setRecordSerializer: Wires serialization logic to sink

- Flink's Table API and SQL API are high-level, declarative APIs built on top of the DataStream API. You can register DataStreams as Tables, perform Table or SQL queries, and then convert the results back to streams.

```
StreamTableEnvironment tableEnv =  
StreamTableEnvironment.create(env);
```

```
// Register a stream as a table with schema and watermark
```

```

tableEnv.createTemporaryView(
    "Events",
    eventStream,
    Schema.newBuilder()
        .column("userId", DataTypes.STRING())
        .column("amount", DataTypes.DOUBLE())
        .column("eventTime", DataTypes.TIMESTAMP(3))
        .watermark("eventTime", "eventTime - INTERVAL '5' SECOND")
        .build()
);

// Table API query
Table result = tableEnv.from("Events")
    .groupBy($"userId")
    .select($"userId", $"amount").sum().as("totalAmount"));

// Convert back to DataStream
DataStream<Row> resultStream = tableEnv.toDataStream(result);

// Equivalent SQL ex:
Table result = tableEnv.sqlQuery(
    "SELECT userId, SUM(amount) AS totalAmount " +
    "FROM Events GROUP BY userId"
);

```

- CEP (Complex Event Processing): Detect patterns in event streams. Used for fraud detection.

```

Pattern<Event, ?> pattern = Pattern.<Event>begin("start")
    .where(new SimpleCondition<Event>() {
        @Override
        public boolean filter(Event event) {
            return event.getType().equals("login");
        }
    })
    .next("middle")
    .where(event -> event.getType().equals("payment"))

```

```
.followedBy("end")  
.where(event -> event.getType().equals("logout"))  
.within(Time.minutes(10));
```

- Other notes:
  - Operations in Flink:

- Transformations:
  - map: One-to-one transformation that applies a function to each element. Eg: `dataStream.map(event -> event.toUpperCase());`
  - flatMap: One-to-many transformation that can produce zero, one, or multiple elements for each input. Eg: `dataStream.flatMap((event, out) -> {for (String word : event.split(" ")) { out.collect(word); }});`
  - filter: Keeps elements that satisfy a condition. Eg: `dataStream.filter(event -> event.contains("error"));`
- Keying & Partitioning:
  - keyBy: Groups the stream by a key (creates a `KeyedStream`). Eg: `dataStream.keyBy(event -> event.getUserId());`
  - shuffle: Redistributes elements randomly. Eg: `dataStream.shuffle();`
  - rebalance: Evenly distributes elements across tasks. Eg: `dataStream.rebalance();`
  - rescale: Redistributes elements in a round-robin fashion among downstream tasks. Eg: `dataStream.rescale();`
  - broadcast: Replicates each element to all parallel tasks. Eg: `dataStream.broadcast();`
- Aggregations & Windows:
  - reduce: Combines elements of a `KeyedStream` using a reduce function. Eg: `keyedStream.reduce((a, b) -> new Sum(a.value + b.value));`
  - aggregate: Applies an aggregation function (like sum, min, max). Eg: `keyedStream.sum("amount");`  
`keyedStream.min("latency");` `keyedStream.max("value");`
  - window: Groups elements of a stream into finite sets based on time or count. Eg:

```

keyedStream.window(TumblingEventTimeWindows.of(Time.seconds(5)));
keyedStream.window(SlidingProcessingTimeWindows.of(Time.seconds(10), Time.seconds(5)));
keyedStream.countWindow(100, 10); // sliding count window
- windowAll: Creates a window on a non-keyed stream. Eg:
  dataStream.windowAll(TumblingProcessingTimeWindows.of(Time.minutes(1)));
- Joining & Combining
  - join: Joins two data streams based on a key. Eg:
    stream1.join(stream2).where(e1 ->
      e1.getKey().equalTo(e2 ->
        e2.getKey()).window(TumblingEventTimeWindows.of(Time.seconds(5))).apply((e1, e2) -> new Tuple2<>(e1, e2)));
  - coGroup: Groups and joins two streams in a more flexible way than join. Eg: stream1.coGroup(stream2).where(e1 ->
    e1.getKey().equalTo(e2 ->
      e2.getKey()).window(TumblingEventTimeWindows.of(Time.seconds(5))).apply(new CoGroupFunction<...>() {...}));
  - union: Combines multiple streams of the same type. Eg:
    stream1.union(stream2, stream3);
  - connect: Combines two streams of potentially different types. Eg: stream1.connect(stream2).map(new
    CoMapFunction<Type1, Type2, ResultType>() {...});

```

- Flink Performance Optimization: Parallelism Tuning (setParallelism), State Optimization (ValueState or MapState, Implement state TTL, Incremental Checkpointing, RocksDB Backend state), Checkpoint Optimization, Resource Configs, Network Optimize (Chain operators, reduce shuffle, local aggregations before shuffle), Operator Optimization (Use AggregateFunction instead of ProcessWindowFunction, Batch operations if possible).
- Backpressure reasons: Slow operators, External system latency, Insufficient resources, Data skew (hot keys), GC pressure, Checkpoint alignment delays.

- In eks yaml deployment config file you may use: upgradeMode key. The upgradeMode setting determines how Flink jobs behave during upgrades or restarts:
    - last-state mode: When a job is upgraded or restarted, it will attempt to restore from the most recent successful checkpoint. This preserves the processing state and allows the job to continue from where it left off.
    - stateless mode: The job will start from scratch after an upgrade, without attempting to restore any previous state. This is like a completely fresh deployment.
    - Note that if you have checkpointing related configs in yaml; The checkpointing configuration remains active in both modes - it's just that the stateless mode ignores existing checkpoints when pipeline starting up.
  - Deployment modes in Flink:
    - Session Mode: Long-running cluster, Multiple jobs share same cluster, Resources shared across jobs, Use case: Development, multiple small jobs
    - Application Mode: One cluster per application, Main method runs on cluster, Better resource isolation, Use case: Production applications
    - Per-Job Mode (Deprecated): One cluster per job, Cluster terminated after job finishes, Use case: Batch jobs, isolation
-





Trino

(Note: Used default font, to preserve text style | Refactored with GPT)

## Table of Contents

---

1. What is Trino & Why
  2. Architecture — How Trino Works
  3. Deployment — GKE (Production)
  4. Deployment — Dataproc (Managed)
  5. Configuration Files
  6. Connectors — Catalog Configuration
  7. Trino SQL — Full Reference
  8. Table Formats — Iceberg, Delta, Hive
  9. Query Performance & Tuning
  10. Memory Management
  11. Security — Authentication & Authorization
  12. Hive Metastore & Schema Management
  13. Trino CLI & JDBC
  14. Monitoring — Metrics, JMX, Dashboards
  15. Fault Tolerance & Retry
  16. Cost-Based Optimizer (CBO)
  17. Operations — Common Tasks
  18. Troubleshooting Guide
- 

## 1. What is Trino & Why

---

Trino (formerly PrestoSQL) is an open-source, distributed SQL query engine. It was originally created at Facebook to query petabytes of data in Hive. The Facebook team that built it left and created the Trino project (maintaining the PrestoSQL fork as Trino).

### What Trino Is

- A **query engine**, not a database. It does not store data.
- It reads data from external storage (GCS, HDFS, S3) and databases (PostgreSQL, BigQuery, Bigtable, Kafka).
- It federates queries across multiple sources — you can JOIN a GCS Parquet file with a Cloud SQL table in a single SQL query.
- It is optimised for analytical (OLAP) workloads — large scans, aggregations, joins over millions to trillions of rows.

### What Trino is NOT

- Not a replacement for transactional databases (OLTP) — use Cloud SQL for that.

- Not a streaming engine — it processes data in batch. (Use Dataflow or Flink for streaming.)
- Not a storage layer — data lives in GCS, not Trino.
- Not a scheduler — orchestrate Trino queries from Airflow.

### Why Trino for the Data Platform

| Need                                       | How Trino addresses it                               |
|--------------------------------------------|------------------------------------------------------|
| Query raw files in GCS (Parquet, ORC, CSV) | Hive/Iceberg connector reads GCS directly            |
| Federation — JOIN across data sources      | Single SQL query spanning BigQuery, PostgreSQL, GCS  |
| Ad-hoc analytics without data movement     | Query in-place; no ETL needed                        |
| Standard SQL interface for all data        | One SQL dialect, multiple backends                   |
| Separation of storage and compute          | Scale workers independently of data volume           |
| Open ecosystem                             | Works with Airflow, dbt, Superset, Metabase, Jupyter |

### Trino vs Alternatives

| Feature       | Trino                     | BigQuery           | Spark SQL | Presto (Meta)      |
|---------------|---------------------------|--------------------|-----------|--------------------|
| Query model   | Interactive SQL           | Serverless SQL     | Batch SQL | Interactive SQL    |
| Data location | External (GCS, PG, BQ)    | BigQuery native    | External  | External           |
| Federation    | Yes — multiple connectors | Limited            | Limited   | Yes                |
| Latency       | Seconds to minutes        | Seconds to minutes | Minutes   | Seconds to minutes |

|                |                            |                       |                             |                    |
|----------------|----------------------------|-----------------------|-----------------------------|--------------------|
| Manag<br>ement | Self-managed<br>cluster    | Fully<br>serverless   | Self-managed or<br>Dataproc | Self-manag<br>ed   |
| Cost<br>model  | Pay for cluster<br>uptime  | Pay per TB<br>scanned | Pay for cluster             | Pay for<br>cluster |
| Best for       | Multi-source<br>federation | Pure BQ<br>analytics  | Large batch<br>transforms   | Meta<br>ecosystem  |

## 2. Architecture — How Trino Works



### Coordinator

The coordinator is a single node responsible for:

1. **Parsing** the SQL query
2. **Planning** — building a logical and physical query plan
3. **Optimising** — applying the Cost-Based Optimizer (CBO) rules
4. **Splitting** — dividing data into splits (chunks) that workers process in parallel
5. **Scheduling** — assigning splits to workers
6. **Merging** — collecting results from workers and returning to the client

The coordinator never reads data itself — it only orchestrates.

## Workers

Workers do the actual data processing:

- Read splits from data sources (GCS files, DB rows)
- Apply filters, projections, aggregations
- Exchange intermediate data between workers for joins and aggregations
- Return results to the coordinator

Workers communicate directly with each other for data exchange (shuffle) — not through the coordinator.

## Splits

A **split** is the smallest unit of parallelism — one worker processes one split at a time. For GCS Parquet files, a split is typically one file (or one row group within a file). For databases, splits are key ranges. More splits = more parallelism = faster queries (up to the number of available worker threads).

## Discovery Service

Workers register themselves with the coordinator via HTTP. The coordinator maintains a list of healthy workers. On GKE, the coordinator's service name is the discovery URI workers use.

## Query Lifecycle

Client sends SQL

→ Coordinator parses and validates SQL

→ Coordinator builds logical plan

→ Optimizer rewrites plan (predicate pushdown, join reordering, etc.)

→ Coordinator creates physical plan with stages

→ Coordinator splits data source into splits

→ Coordinator schedules splits to workers

→ Workers read data, process, exchange intermediate results

→ Coordinator collects final results

→ Results returned to client

---

### 3. Deployment — GKE (Production)

The production-recommended deployment is Trino on GKE using the official Helm chart.

#### Helm Chart Setup

```
helm repo add trino https://trinodb.github.io/charts
helm repo update
helm show values trino/trino > trino-default-values.yaml # review all options
```

#### values.yaml — Production Configuration

##### Sizing logic — how these numbers are chosen:

```
Node type for Trino workers: n2-highmem-16 (16 vCPU, 128 GB RAM)
  Usable RAM after OS + K8s overhead: ~120 GB
  One worker Pod: requests 30 GB RAM, 4 vCPU
  → 1 node fits ~3 worker Pods (3 × 30 GB = 90 GB, leaving headroom)
```

```
JVM heap (maxHeapSize: 28G) is set ~2 GB below the Pod memory request (30 GB)
  so the JVM has room for off-heap memory (NIO buffers, compiled code cache)
  without triggering the container's OOM kill limit.
```

```
Node type for coordinator: n2-standard-8 (8 vCPU, 32 GB RAM)
  Coordinator doesn't process data – it plans queries and orchestrates workers.
  10 GB RAM is plenty; more vCPU helps with query planning for complex queries.
```

```
query.max-memory = 50 GB (cluster-wide limit per query)
  = 5 workers × ~10 GB each for a single query
  Prevents one runaway query from consuming all workers' memory.
# values.yaml
```

```
image:
  repository: trinodb/trino
  tag: "440" # pin to a specific version
  pullPolicy: IfNotPresent
```

```
server:  
  workers: 5
```

```
coordinator:  
  jvm:  
    maxHeapSize: "8G"  
  config:  
    query.max-memory: "50GB"  
    query.max-memory-per-node: "4GB"  
    query.max-total-memory-per-node: "6GB"  
  nodeSelector:  
    workload: trino-coordinator  
  resources:  
    requests:  
      memory: "10Gi"  
      cpu: "2"  
    limits:  
      memory: "12Gi"  
      cpu: "4"  
  tolerations:  
    - key: dedicated  
      value: trino  
      operator: Equal  
      effect: NoSchedule
```

```
worker:  
  jvm:  
    maxHeapSize: "28G" # ~2 GB below memory request –  
leaves room for JVM off-heap  
  config:  
    query.max-memory-per-node: "20GB"  
    query.max-total-memory-per-node: "26GB"  
  nodeSelector:  
    workload: trino-worker  
  resources:  
    requests:  
      memory: "30Gi" # sized for n2-highmem-16 (128 GB  
/ ~3 workers per node)
```

```
    cpu: "4"
  limits:
    memory: "32Gi"
    cpu: "8"
  tolerations:
    - key: dedicated
      value: trino
      operator: Equal
      effect: NoSchedule
```

```
# Catalogs (data sources)
additionalCatalogs:
  gcs_hive: |
    connector.name=hive
    hive.metastore.uri=thrift://companyABC-metastore:9083
    hive.gcs.use-access-token=false
    hive.gcs.json-key-file-path=/etc/gcp/service-account.json
    hive.storage-format=PARQUET
    hive.compression-codec=SNAPPY
    hive.parquet.use-column-names=true
```

```
  iceberg_gcs: |
    connector.name=iceberg
    iceberg.catalog.type=hive_metastore
    hive.metastore.uri=thrift://companyABC-metastore:9083
    iceberg.file-format=PARQUET
    iceberg.compression-codec=SNAPPY
```

```
  bigquery: |
    connector.name=bigquery
    bigquery.project-id=companyABC-platform-prod
    bigquery.views-enabled=true
    bigquery.arrow-serialization.enabled=true
```

```
  postgres_prod: |
    connector.name=postgresql
    connection-url=jdbc:postgresql://10.10.0.5:5432/mydb
    connection-user=${ENV:POSTGRES_USER}
```



```
connection-password=${ENV:POSTGRES_PASSWORD}
```

```
# Service account for Workload Identity
```

```
serviceAccount:
```

```
  create: true
```

```
  name: trino-sa
```

```
  annotations:
```

```
    iam.gke.io/gcp-service-account:
```

```
trino-gcp-sa@my-project.iam.gserviceaccount.com
```

```
# Expose coordinator UI
```

```
service:
```

```
  type: ClusterIP
```

```
ingress:
```

```
  enabled: true
```

```
  annotations:
```

```
    kubernetes.io/ingress.class: gce
```

```
    networking.gke.io/managed-certificates: trino-cert
```

```
  hosts:
```

```
    - host: trino.internal.companyABC.com
```

```
    paths:
```

```
      - path: /
```

```
      pathType: Prefix
```

## Install

```
helm upgrade --install trino trino/trino \
```

```
  --namespace trino \
```

```
  --create-namespace \
```

```
  --values values.yaml \
```

```
  --wait
```

## Verify

```
kubectl get pods -n trino
```

```
kubectl get svc -n trino
```

```
# Access Trino UI via port-forward
```

```
kubectl port-forward svc/trino 8080:8080 -n trino
```

```
# Open http://localhost:8080
```

---

## 4. Deployment — Dataproc (Managed)

Dataproc runs Trino as an optional component on a managed cluster. Good for getting started quickly or running ephemeral query clusters.

```
gcloud dataproc clusters create companyABC-trino-cluster \
  --region=asia-south1 \
  --master-machine-type=n2-standard-8 \
  --master-boot-disk-size=500GB \
  --worker-machine-type=n2-standard-8 \
  --worker-boot-disk-size=500GB \
  --num-workers=4 \
  --optional-components=TRINO \
  --enable-component-gateway \           # exposes Trino UI via an
IAP-secured proxy

--dataproc-metastore=projects/my-project/locations/asia-south1/servi
ces/companyABC-metastore \
  --properties="trino:query.max-memory=50GB" \
  --scopes=cloud-platform \
  --service-account=trino-gcp-sa@my-project.iam.gserviceaccount.com
```

Access Trino UI via the Component Gateway URL in the GCP Console → Dataproc → Cluster → Web Interfaces.

**Ephemeral cluster pattern** — create for a workload, delete when done:

# Create

```
gcloud dataproc clusters create companyABC-trino-ephemeral
--region=asia-south1 --optional-components=TRINO ...
```

# Run query

```
gcloud dataproc jobs submit trino --region=asia-south1
--cluster=companyABC-trino-ephemeral \
  --execute="SELECT count(*) FROM hive.default.events WHERE dt =
'2026-06-07'"
```

# Delete when done

```
gcloud dataproc clusters delete companyABC-trino-ephemeral
--region=asia-south1
```

---

## 5. Configuration Files

---

Trino is configured via files in the `etc/` directory. On GKE via Helm, these are set in `values.yaml` and rendered as ConfigMaps.

### **config.properties (Coordinator)**

```
coordinator=true
node-scheduler.include-coordinator=false # don't schedule work on
coordinator
http-server.http.port=8080
discovery.uri=http://trino-coordinator:8080
```

```
query.max-memory=100GB
query.max-memory-per-node=20GB
query.max-total-memory-per-node=25GB
query.max-run-time=2h
query.max-execution-time=1h
```

```
# Spilling (use disk when memory is exceeded)
spill-enabled=true
spiller-spill-path=/tmp/trino-spill
spill-compression-enabled=true
spill-encryption-enabled=false
```

```
# Fault tolerance
retry-policy=TASK # retry failed tasks, not
entire queries
exchange.deduplication-buffer-size=32MB
fault-tolerant-execution-task-memory=5GB
```

```
# Performance
optimizer.join-reordering-strategy=AUTOMATIC
optimizer.default-filter-factor-when-exact-stats-unavailable=0.9
```

### **config.properties (Worker)**

```
coordinator=false
http-server.http.port=8080
discovery.uri=http://trino-coordinator:8080
```

```
query.max-memory-per-node=20GB
query.max-total-memory-per-node=25GB
```

```

spill-enabled=true
spiller-spill-path=/tmp/trino-spill
jvm.config
-server
-Xmx28G
-XX:+UseG1GC
-XX:G1HeapRegionSize=32M
-XX:+ExplicitGCInvokesConcurrent
-XX:+HeapDumpOnOutOfMemoryError
-XX:+ExitOnOutOfMemoryError
-XX:ReservedCodeCacheSize=512M
-XX:PerMethodRecompilationCutoff=10000
-XX:PerBytecodeRecompilationCutoff=10000
-Djdk.attach.allowAttachSelf=true
-Djdk.nio.maxCachedBufferSize=2000000
-XX:+UnlockDiagnosticVMOptions
-XX:+UseAioCbForEpollIo
node.properties
node.environment=production
node.id=<unique-uuid-per-node> # auto-generated by Helm
chart
node.data-dir=/data/trino
log.properties
io.trino=INFO
io.trino.plugin.hive=WARN
io.trino.plugin.iceberg=WARN

```

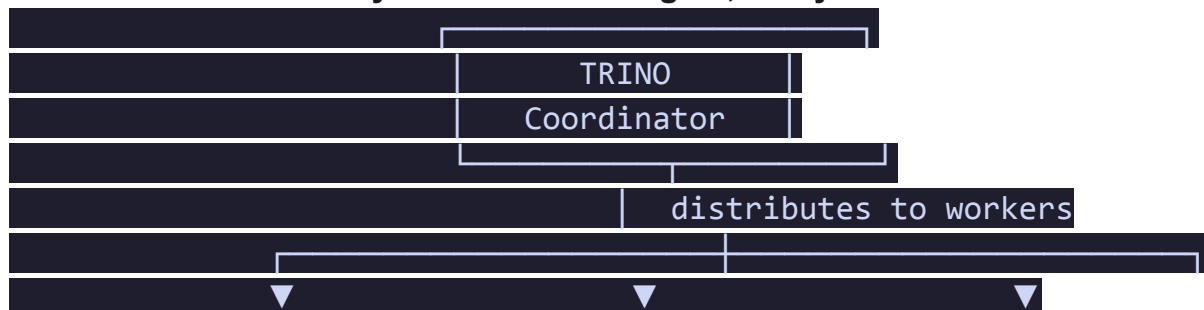
---

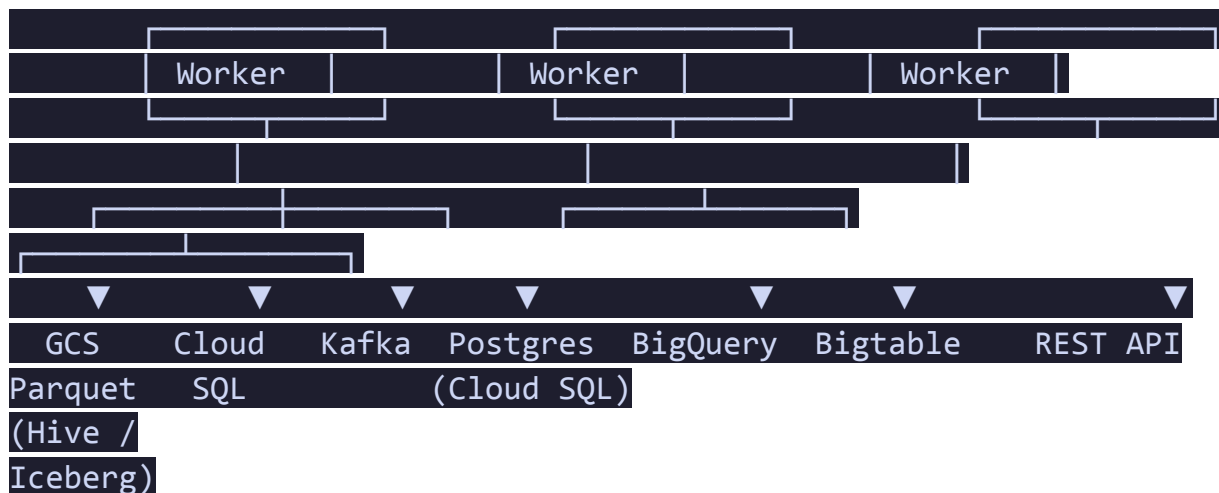
## 6. Connectors — Catalog Configuration

---

Each catalog = one data source. Place catalog config in `etc/catalog/<name>.properties`.

**Trino as a federation layer — one SQL engine, many sources:**





```
Catalog: hive postgres bigquery kafka http
         iceberg
         delta
```

Single SQL query can span ANY combination of these sources:

```
SELECT g.payment_id, p.payment_name, b.order_count
FROM   hive.payments.events g           -- GCS Parquet
JOIN   postgres.public.payments p       -- Cloud SQL
JOIN   bigquery.analytics.orders b      -- BigQuery
ON     g.payment_id = p.payment_id AND p.payment_id = b.payment_id
WHERE  g.dt = CURRENT_DATE
```

### Hive Connector (GCS — Parquet/ORC/CSV)

The most common connector for reading files in GCS via Hive Metastore.

```
connector.name=hive
```

```
hive.metastore.uri=thrift://companyABC-metastore:9083
```

```
# GCS Authentication (use Workload Identity on GKE – no key file
needed)
```

```
hive.gcs.use-access-token=true
```

```
# OR with a service account key file:
```

```
# hive.gcs.json-key-file-path=/etc/gcp/service-account.json
```

```
# Defaults
```

```
hive.storage-format=PARQUET
```

```
hive.compression-codec=SNAPPY
```

```
hive.parquet.use-column-names=true
hive.parquet.ignore-statistics=false
hive.orc.bloom-filters.enabled=true
```

#### # Performance

```
hive.max-split-size=128MB
hive.max-initial-splits-per-node=8
hive.max-splits-per-second=100
hive.recursive-directories=true          # read subdirectories
hive.file-status-cache-tables=*          # cache file listings
```

#### # GCS-specific

```
hive.gcs.max-batch-size=100
fs.cache.enabled=true
fs.cache.max-size=1GB
```

#### Iceberg Connector (GCS)

For Apache Iceberg format tables — supports ACID transactions, schema evolution, time travel.

```
connector.name=iceberg
iceberg.catalog.type=hive_metastore
hive.metastore.uri=thrift://companyABC-metastore:9083
```

```
iceberg.file-format=PARQUET
iceberg.compression-codec=SNAPPY
iceberg.target-max-file-size=512MB
```

#### # Time travel and metadata

```
iceberg.max-partitions-per-writer=100
iceberg.unique-table-location=true
iceberg.register-table-procedure.enabled=true  # allow registering
existing tables
```

#### # GCS auth (same as Hive)

```
hive.gcs.use-access-token=true
```

#### Delta Lake Connector (GCS)

For Delta Lake format tables.

```
connector.name=delta_lake
hive.metastore.uri=thrift://companyABC-metastore:9083
```

```
delta.hide-non-delta-lake-tables=false
```

```
hive.gcs.use-access-token=true
```

```
delta.target-max-file-size=512MB
```

```
delta.vacuum.min-retention=7d
```

### BigQuery Connector

```
connector.name=bigquery
```

```
bigquery.project-id=companyABC-platform-prod
```

```
bigquery.views-enabled=true # allow querying BQ views
```

```
bigquery.arrow-serialization.enabled=true # faster serialisation  
via Arrow
```

```
bigquery.read-timeout=PT10M
```

```
bigquery.parallelism=10 # parallel read streams
```

Usage:

```
-- Query a BigQuery table from Trino
```

```
SELECT * FROM bigquery.my_dataset.app_events
```

```
WHERE DATE(event_time) = CURRENT_DATE
```

```
LIMIT 100;
```

```
-- Join GCS data with BigQuery
```

```
SELECT g.payment_id, g.item_id, b.order_count
```

```
FROM hive.payments.events g
```

```
JOIN bigquery.analytics.daily_orders b ON g.payment_id =  
b.payment_id
```

```
WHERE g.dt = CURRENT_DATE;
```

### PostgreSQL Connector (Cloud SQL)

```
connector.name=postgresql
```

```
connection-url=jdbc:postgresql://10.10.0.5:5432/companyABC_db
```

```
connection-user=${ENV:PG_USER}
```

```
connection-password=${ENV:PG_PASSWORD}
```

```
# Performance
```

```
postgresql.array-mapping=AS_ARRAY
```

```
decimal-mapping=ALLOW_OVERFLOW
```

```
decimal-default-scale=12
```

Usage:

```
SELECT * FROM postgres.public.payments WHERE is_active = true;
```

```
-- Cross-source join
SELECT p.payment_name, COUNT(h.item_id) AS payX_picks
FROM postgres.public.payments p
JOIN hive.payments.events h ON p.payment_id = h.payment_id
WHERE h.dt = CURRENT_DATE
GROUP BY p.payment_name;
```

### MySQL Connector

```
connector.name=mysql
connection-url=jdbc:mysql://10.10.0.6:3306/mydb
connection-user=${ENV:MYSQL_USER}
connection-password=${ENV:MYSQL_PASSWORD}
```

### Kafka Connector

Query Kafka topics as tables.

```
connector.name=kafka
kafka.nodes=kafka-broker-1:9092,kafka-broker-2:9092
kafka.table-description-dir=/etc/trino/kafka/
kafka.hide-internal-columns=false
kafka.messages-per-split=50000
```

Table description file (`app-events.json`):

```
{
  "tableName": "app_events",
  "schemaName": "default",
  "topicName": "app-events",
  "key": {
    "dataFormat": "raw",
    "fields": [{ "name": "kafka_key", "dataFormat": "VARCHAR",
"type": "VARCHAR" }]
  },
  "message": {
    "dataFormat": "json",
    "fields": [
      { "name": "payment_id", "mapping": "payment_id", "type":
"VARCHAR" },
      { "name": "event_type", "mapping": "event_type", "type":
"VARCHAR" },
      { "name": "item_id", "mapping": "item_id", "type": "VARCHAR"
}
    ],
  },
}
```



```
{ "name": "event_time", "mapping": "event_time", "type":
"TIMESTAMP" }
]
}
```

## HTTP Connector

Query REST APIs as tables.

```
connector.name=http
```

```
http-client.base-url=https://api.companyABC.internal
```

## JMX Connector (Internal Diagnostics)

```
connector.name=jmx
```

Usage:

```
-- Check Trino's own JMX metrics
```

```
SELECT * FROM jmx.current."io.trino.execution:name=QueryManager"
WHERE attribute LIKE '%RunningQueries%';
```

---

## 7. Trino SQL

Trino supports ANSI SQL with extensions. DDL/DML/Select/Explain/Array functions/Different datatypes of columns, etc.

---

## 8. Table Formats — Iceberg, Delta, Hive

### Choosing a table format:

New table for the data platform?

```

|
| — Need ACID / row-level updates / time travel?
|   | — YES → Apache Iceberg ✓ (recommended default)
|
| — Already have Delta Lake tables from Databricks / Spark?
|   | — YES → Delta Lake (Trino reads well; writes less
mature)
|
| — Append-only event log, no updates, compatibility with
legacy Hive?
|   | — YES → Hive format (Parquet / ORC)
```

Feature comparison:

|  | Hive | Delta Lake | Iceberg |
|--|------|------------|---------|
|--|------|------------|---------|

|                         |         |         |          |                            |
|-------------------------|---------|---------|----------|----------------------------|
| ACID transactions       | X       | ✓       | ✓        |                            |
| Row-level UPDATE        | X       | ✓       | ✓        |                            |
| Schema evolution        | Limited | ✓       | ✓        | ←                          |
| add/rename/drop columns |         |         |          |                            |
| Partition evolution     | X       | X       | ✓        | ← no data rewrite needed   |
| Time travel             | X       | ✓       | ✓        |                            |
| Hidden partitioning     | X       | X       | ✓        | ← no dt= column in queries |
| Small file compaction   | Manual  | VACUUM  | OPTIMIZE |                            |
| Trino write support     | ✓       | Limited | ✓        | ← full read + write        |

### Apache Iceberg (Recommended for new tables)

Iceberg is the modern open table format. It provides:

- **ACID transactions** — concurrent reads and writes without corruption
- **Schema evolution** — add/rename/drop columns without rewriting data
- **Partition evolution** — change partitioning without rewriting data
- **Time travel** — query past snapshots of a table
- **Row-level deletes and updates** — without full partition rewrites

When NOT to Use Iceberg

- Strictly Low-Latency OLTP Systems: Iceberg is not a replacement for MySQL, PostgreSQL, or MongoDB for sub-second, transactional web applications.
- Small, Static Datasets: If your data is only a few gigabytes and rarely updates, simple Parquet files or traditional databases are easier to manage.

### How Iceberg stores data and enables time travel (GCS layout):

`gs://companyABC-data-lake/payments/events/`

||

```
├── metadata/
│   ├── v1.metadata.json ← snapshot 1 (after first INSERT)
│   ├── v2.metadata.json ← snapshot 2 (after UPDATE)
│   ├── v3.metadata.json ← snapshot 3 (after DELETE) ← current
│   └── snap-1.avro snap-2.avro snap-3.avro (manifest lists)
```

||

```
├── data/
│   ├── payment_id=payment-001/day=2026-06-07/
│   │   ├── file-a.parquet ← written by INSERT (snapshot 1)
│   │   └── file-b.parquet ← written by UPDATE (new rows, snapshot 2)
```

```
└─ payment_id=GOB002/day=2026-06-07/
  └─ file-c.parquet
```

Time travel:

```
SELECT * FROM events FOR TIMESTAMP AS OF '2026-06-06'
→ reads v1.metadata.json → finds snap-1.avro → reads only
file-a.parquet
```

```
SELECT * FROM events (current)
→ reads v3.metadata.json → finds snap-3.avro → reads
file-b.parquet
(file-a.parquet rows deleted are tracked via delete files,
not rewritten)
```

expire\_snapshots() removes old metadata + data files no longer referenced.

-- Create Iceberg table

```
CREATE TABLE iceberg.payments.events (
  event_id VARCHAR,
  payment_id VARCHAR,
  event_time TIMESTAMP(6) WITH TIME ZONE
)
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['day(event_time)', 'payment_id'],
  sorted_by = ARRAY['event_time']
);
```

-- Insert

```
INSERT INTO iceberg.payments.events VALUES ('e1', 'payment-001',
NOW());
```

-- Update and Delete

```
UPDATE iceberg.payments.events SET payment_id = 'GOB002' WHERE
event_id = 'e1';
DELETE FROM iceberg.payments.events WHERE event_id = 'e1';
```

-- Time travel

```
SELECT * FROM iceberg.payments.events FOR TIMESTAMP AS OF TIMESTAMP  
'2026-06-01 00:00:00';  
SELECT * FROM iceberg.payments.events FOR VERSION AS OF 12345; --  
snapshot ID
```

```
-- Inspect metadata
```

```
SELECT * FROM iceberg.payments."events$snapshots";  
SELECT * FROM iceberg.payments."events$history";  
SELECT * FROM iceberg.payments."events$files";  
SELECT * FROM iceberg.payments."events$partitions";  
SELECT * FROM iceberg.payments."events$manifests";
```

```
-- Schema evolution
```

```
ALTER TABLE iceberg.payments.events ADD COLUMN operator_id VARCHAR;  
ALTER TABLE iceberg.payments.events DROP COLUMN operator_id;  
ALTER TABLE iceberg.payments.events RENAME COLUMN event_time TO  
occurred_at;
```

```
-- Partition evolution (no data rewrite!)
```

```
ALTER TABLE iceberg.payments.events  
SET PROPERTIES partitioning = ARRAY['month(event_time)',  
'payment_id'];
```

```
-- Compaction – merge small files into larger ones
```

```
ALTER TABLE iceberg.payments.events EXECUTE OPTIMIZE;
```

```
-- Expire old snapshots (reduce GCS storage)
```

```
ALTER TABLE iceberg.payments.events  
EXECUTE expire_snapshots(retention_threshold => '7d');
```

```
-- Remove orphan files
```

```
ALTER TABLE iceberg.payments.events EXECUTE remove_orphan_files;
```

## Delta Lake

Delta Lake is another ACID table format (from Databricks). Trino reads Delta tables (writes are supported but less mature than Iceberg).

```
SELECT * FROM delta.payments.events;
```

```
-- Time travel
```

```
SELECT * FROM delta.payments.events FOR VERSION AS OF 5;  
SELECT * FROM delta.payments.events FOR TIMESTAMP AS OF TIMESTAMP  
'2026-06-01';
```

```
-- Vacuum (remove old versions)  
CALL delta.system.vacuum('payments', 'events', 7);
```

### Hive (Legacy — Parquet/ORC files)

Traditional Hive-format tables — no ACID, no row-level updates. Fine for append-only data like event logs.

```
-- Partitioned Hive table  
CREATE TABLE hive.payments.events (  
  event_id VARCHAR,  
  payment_id VARCHAR,  
  item_id VARCHAR,  
  event_time TIMESTAMP  
)  
WITH (  
  format = 'PARQUET',  
  partitioned_by = ARRAY['dt'],  
  external_location = 'gs://companyABC-data-lake/payments/events/'  
);  
  
-- Load a specific partition  
CALL hive.system.sync_partition_metadata('payments', 'events',  
'FULL');
```

---

## 9. Query Performance & Tuning

### Predicate Pushdown

Trino pushes predicates (WHERE clause filters) down to the data source to minimise data read. This is the single biggest performance lever.

**What happens without vs with predicate pushdown:**

```
WITHOUT pushdown (bad – function on partition column):  
WHERE YEAR(dt) = 2026
```

```
GCS bucket (all 365 partitions)  
├── dt=2026-01-01/ ← scanned  
├── dt=2026-01-02/ ← scanned  
└── ... ← scanned (ALL of them)
```

```
└── dt=2026-06-07/ ← scanned
```

```
└── Trino workers read ALL data, then apply filter ←  
slow + expensive
```

```
WITH pushdown (good – exact partition column):  
WHERE dt = DATE '2026-06-07'
```

```
GCS bucket
```

```
└── dt=2026-01-01/ ← SKIPPED (partition pruning)  
└── ... ← SKIPPED  
└── dt=2026-06-07/ ← ONLY this partition is opened and read
```

```
└── Trino workers read 1/365th of the data ← fast +  
cheap
```

- **Always filter on partition columns** — Trino skips entire partitions.
- **Filter early in CTEs** — push filters into subqueries so fewer rows flow through the plan.
- **For BigQuery connector:** filters on date and string columns are pushed to BigQuery.
- **For PostgreSQL connector:** most WHERE conditions are pushed to Postgres.

```
-- Good: partition filter – only reads the 2026-06-07 partition  
SELECT * FROM hive.payments.events WHERE dt = DATE '2026-06-07';
```

```
-- Bad: function on partition column – cannot push; reads all  
partitions  
SELECT * FROM hive.payments.events WHERE YEAR(dt) = 2026;
```

```
-- Good: filter before join  
WITH recent AS (  
  SELECT * FROM hive.payments.events  
  WHERE dt = CURRENT_DATE -- filter early  
  AND event_type = 'order_placed'  
)  
SELECT r.item_id, s.payment_name  
FROM recent r  
JOIN postgres.public.payments s ON r.payment_id = s.payment_id;
```

## Join Distribution Strategy

Trino has two join strategies:

| Strategy                  | When used                           | How                                  |
|---------------------------|-------------------------------------|--------------------------------------|
| <b>Broadcast</b>          | Small table joined with large table | Small table copied to all workers    |
| <b>Partitioned (Hash)</b> | Both tables are large               | Both sides redistributed by join key |

Trino's CBO chooses automatically with table statistics. Force a strategy:

```
SELECT /*+ BROADCAST(s) */ r.item_id, s.payment_name
FROM hive.payments.events r
JOIN postgres.public.payments s ON r.payment_id = s.payment_id;
```

## Column Pruning

Trino reads only the columns referenced in the query from Parquet/ORC files (columnar format). Always select only the columns you need — avoid **SELECT \*** on large tables.

## Parallelism

More splits = more parallelism. To increase the number of splits for a large GCS read:

```
SET SESSION hive.max_split_size = '64MB'; -- smaller splits = more workers engaged
```

For very wide Parquet files with many row groups, splits are per row group. Tune **hive.max-initial-splits-per-node**.

## Join Reordering

With CBO and table statistics, Trino reorders joins to start with the most selective table. Run **ANALYZE** to collect statistics:

```
ANALYZE hive.payments.events;
ANALYZE hive.payments.events WITH (partitions =
ARRAY[ARRAY['2026-06-07']]);
```

## Spilling

When a query uses more memory than the per-node limit, Trino can spill intermediate data to disk. Enable spilling to handle memory-intensive joins and aggregations:

```
spill-enabled=true
spiller-spill-path=/tmp/trino-spill
spill-compression-enabled=true
```

## File Size Optimisation

Small files are the biggest performance killer — each file requires a metadata round-trip to GCS plus separate processing. Target file sizes of 128–512 MB.

- For Iceberg: `ALTER TABLE ... EXECUTE OPTIMIZE` to compact small files.
- For Hive: rewrite partitions via `INSERT OVERWRITE`.

### Statistics Collection (CBO)

```
-- Collect stats for a table
ANALYZE hive.payments.events;
```

```
-- Show collected stats
SHOW STATS FOR hive.payments.events;
```

```
-- For Iceberg – stats are maintained automatically from manifest
files
```

---

## 10. Memory Management

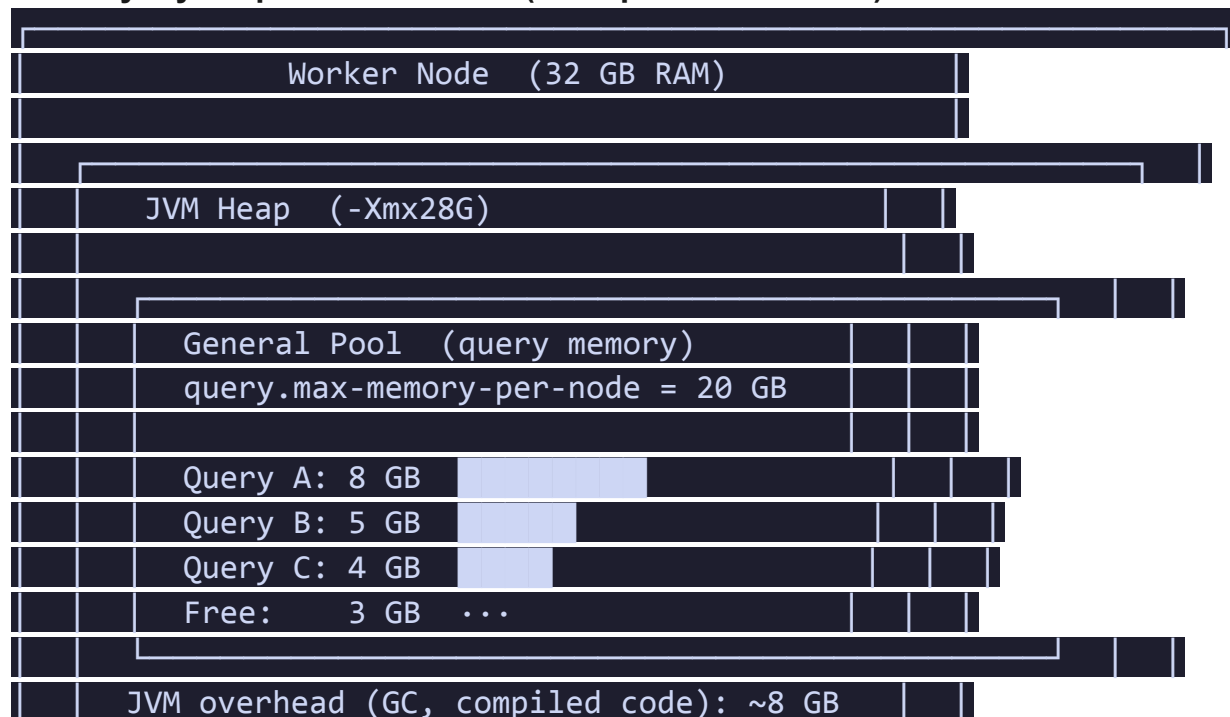
Understanding Trino's memory model is essential for production stability.

### Memory Pools

Each Trino worker has:

- **General pool**: for query execution memory (joins, aggregations)
- **Reserved pool** (deprecated in recent versions): for spilling

### Memory layout per worker node (example: 32 GB node):





```
OS + other processes: ~4 GB
```

If a query exceeds max-memory-per-node (20 GB):

spill-enabled=true → overflow written to disk (/tmp/trino-spill or GCS)

spill-enabled=false → query FAILS with "Query exceeded per-node memory limit"

Cluster-wide limit (all workers combined):

query.max-memory = 100 GB (e.g. 5 workers × 20 GB)

A single query cannot use more than this across the whole cluster.

Key config parameters:

# Per query, per node

query.max-memory-per-node=20GB # max general memory for one query on one worker

query.max-total-memory-per-node=25GB # includes revocable memory (used for spilling)

# Per query, cluster-wide (sum of all workers)

query.max-memory=100GB

# JVM heap

-Xmx28G # leave ~20-25% headroom above query memory for JVM overhead

## Memory Troubleshooting

-- Check current memory usage per query

```
SELECT query_id, state, user, memory_pool,
       total_memory_reservation / 1e9 AS memory_gb
FROM system.runtime.queries
WHERE state = 'RUNNING'
ORDER BY total_memory_reservation DESC;
```

-- Check node memory

```
SELECT node_id, free_memory / 1e9 AS free_gb, total_memory / 1e9 AS
total_gb
FROM system.runtime.nodes;
```

## Common memory errors and fixes:

| Error                                        | Cause                                    | Fix                                                                                                     |
|----------------------------------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Query exceeded per-node memory limit         | Single query using too much memory       | Enable spilling; increase <code>query.max-memory-per-node</code> ; optimise query to reduce data volume |
| Query exceeded distributed user memory limit | Cluster-wide query memory limit exceeded | Increase <code>query.max-memory</code> ; optimise join strategy; add worker nodes                       |
| Out of memory (JVM OOM)                      | JVM heap exceeded                        | Reduce <code>query.max-memory-per-node</code> ; increase worker node RAM; upgrade node type             |

## 11. Security — Authentication & Authorization

### Authentication

#### Password file (development/simple setups):

```
# config.properties
http-server.authentication.type=PASSWORD
```

```
# password-authenticator.properties
password-authenticator.name=file
file.password-file=/etc/trino/password.db
# Create password file (htpasswd format)
htpasswd -B -C 10 /etc/trino/password.db admin
```

#### LDAP/Active Directory:

```
http-server.authentication.type=PASSWORD
```

```
# password-authenticator.properties
password-authenticator.name=ldap
ldap.url=ldaps://ldap.companyABC.com:636
ldap.bind-dn=CN=trino-svc,OU=ServiceAccounts,DC=companyABC,DC=com
ldap.bind-password=${ENV:LDAP_BIND_PASSWORD}
ldap.user-bind-pattern=CN=${USER},OU=Engineering,DC=companyABC,DC=com
```

m

**JWT (recommended for service-to-service, Airflow integration):**

```
http-server.authentication.type=JWT
```

```
http-server.authentication.jwt.key-file=/etc/trino/public-key.pem
```

**OAuth 2.0 / OIDC (for browser UI + SSO):**

```
http-server.authentication.type=OAUTH2
```

```
http-server.authentication.oauth2.issuer=https://accounts.google.com
```

```
http-server.authentication.oauth2.client-id=<client-id>
```

```
http-server.authentication.oauth2.client-secret=${ENV:OAUTH_CLIENT_SECRET}
```

```
http-server.authentication.oauth2.scopes=openid,email,profile
```

**TLS (always enable in production):**

```
http-server.https.enabled=true
```

```
http-server.https.port=8443
```

```
http-server.https.keystore.path=/etc/trino/keystore.jks
```

```
http-server.https.keystore.key=${ENV:KEYSTORE_PASSWORD}
```

```
internal-communication.https.required=true
```

```
internal-communication.shared-secret=${ENV:INTERNAL_SHARED_SECRET}
```

**Authorization — Access Control**

**File-based rules** (`etc/rules.json`):

```
{
  "catalogs": [
    {
      "user": "admin",
      "catalog": ".*",
      "allow": "all"
    },
    {
      "group": "data-team",
      "catalog": "(hive|iceberg|bigquery)",
      "allow": "read-only"
    },
    {
      "user": "pipeline-svc",
      "catalog": "hive",
      "allow": "all"
    }
  ],
  "schemas": [
```

```

    {
      "user": "pipeline-svc",
      "catalog": "hive",
      "schema": "payments",
      "owner": true
    }
  ],
  "tables": [
    {
      "group": "data-team",
      "catalog": "hive",
      "schema": "payments",
      "table": "events",
      "privileges": ["SELECT"]
    }
  ],
  "columns": [
    {
      "catalog": "hive",
      "schema": "payments",
      "table": "user_sessions",
      "columns": [
        { "name": "email", "allow": false }      -- deny access to
PII column
      ]
    }
  ]
}

```

Enable in config.properties:

```
access-control.name=file
```

```
security.config-file=/etc/trino/rules.json
```

**OPA (Open Policy Agent)** — for more advanced, dynamic authorization:

```
access-control.name=opa
```

```
opa.policy.uri=http://opa-service:8181/v1/data/trino/allow
```

---

## 12. Hive Metastore & Schema Management

The Hive Metastore (HMS) stores table definitions, schemas, and partition metadata. On GCP, use **Dataproc Metastore**.

## Connecting Trino to Dataproc Metastore

```
# Create Dataproc Metastore
gcloud metastore services create companyABC-metastore \
  --location=asia-south1 \
  --hive-metastore-version=3.1.2 \
  --network=companyABC-vpc \
  --tier=DEVELOPER # or ENTERPRISE for production
HA
```

```
# Get the Thrift endpoint
gcloud metastore services describe companyABC-metastore \
  --location=asia-south1 \
```

```
--format="value(hiveMetastoreConfig.configOverrides.'hive.metastore.
uris')"
```

Configure in Trino catalog:

```
hive.metastore.uri=thrift://10.10.0.100:9083
```

### Common Metastore Operations

```
-- Register an existing GCS path as a table (useful for external
data)
```

```
CALL hive.system.register_partition(
  schema_name => 'payments',
  table_name => 'events',
  partition_columns => ARRAY['dt'],
  partition_values => ARRAY['2026-06-07'],
  location =>
'gs://companyABC-data-lake/payments/events/dt=2026-06-07/'
);
```

```
-- Sync all partitions from GCS (for tables with many partitions
added externally)
```

```
CALL hive.system.sync_partition_metadata(
  schema_name => 'payments',
  table_name => 'events',
  mode => 'FULL' -- ADD, DROP, or FULL
);
```

```
-- Show statistics
```

```
SHOW STATS FOR hive.payments.events;  
ANALYZE hive.payments.events;
```

---

### 13. Trino CLI & JDBC

---

#### Trino CLI

# Download

```
wget
```

```
https://repo1.maven.org/maven2/io/trino/trino-cli/440/trino-cli-440-  
executable.jar -O trino  
chmod +x trino
```

# Connect

```
./trino --server http://trino-coordinator:8080 \  
--user userABC \  
--catalog hive \  
--schema payments
```

# Execute a query and exit

```
./trino --server http://trino-coordinator:8080 \  
--execute "SELECT COUNT(*) FROM hive.payments.events WHERE  
dt = CURRENT_DATE"
```

# Output formats

```
./trino ... --output-format TSV # tab-separated  
./trino ... --output-format CSV  
./trino ... --output-format JSON
```

# Run a SQL file

```
./trino ... --file query.sql
```

# Ignore errors (useful in scripts)

```
./trino ... --ignore-errors
```

CLI keyboard shortcuts:

- **Ctrl+D** — exit
- **Up arrow** — previous query
- **Alt+Enter** — newline within query
- **;** — execute query

#### JDBC

Add the Trino JDBC driver to your project:

```
<!-- Maven -->
<dependency>
  <groupId>io.trino</groupId>
  <artifactId>trino-jdbc</artifactId>
  <version>440</version>
</dependency>
# Python via SQLAlchemy + trino-python-client
pip install trino sqlalchemy sqlalchemy-trino

from trino.dbapi import connect

conn = connect(
    host="trino-coordinator.trino.svc.cluster.local",
    port=8080,
    user="pipeline-svc",
    catalog="hive",
    schema="payments",
    http_scheme="https",
    auth=trino.auth.BasicAuthentication("pipeline-svc", "password"),
    verify=True,
    request_timeout=3600,
)

cursor = conn.cursor()
cursor.execute("SELECT payment_id, COUNT(*) FROM events WHERE dt =
CURRENT_DATE GROUP BY payment_id")
rows = cursor.fetchall()
# With SQLAlchemy (for pandas, dbt, etc.)
from sqlalchemy import create_engine
import pandas as pd

engine = create_engine(
    "trino://pipeline-svc:password@trino-coordinator:8080/hive/payments"
    ,
    connect_args={"http_scheme": "https", "verify": True}
)
```

```
df = pd.read_sql("SELECT * FROM events WHERE dt = CURRENT_DATE LIMIT 1000", engine)
```

### dbt with Trino

```
# profiles.yml
companyABC_data_platform:
  target: prod
  outputs:
    prod:
      type: trino
      method: none
      host: trino-coordinator.trino.svc.cluster.local
      port: 8080
      database: hive
      schema: payments
      user: dbt-svc
      threads: 8
-- models/payX_daily.sql
{{ config(
  materialized='incremental',
  incremental_strategy='insert_overwrite',
  partition_by=['dt'],
  file_format='parquet'
) }}
```

```
SELECT
  payment_id,
  DATE(event_time) AS dt,
  COUNT(*) AS event_count,
  COUNT(DISTINCT item_id) AS unique_payX
FROM {{ source('payments', 'events') }}
{% if is_incremental() %}
WHERE dt >= DATE_ADD('day', -3, CURRENT_DATE)
{% endif %}
GROUP BY 1, 2
```

---

## 14. Monitoring — Metrics, JMX, Dashboards

---

### JMX Metrics



Trino exposes metrics via JMX. Query them in Trino itself:

```
-- Active queries
SELECT * FROM jmx.current."io.trino.execution:name=QueryManager"
WHERE attribute IN ('RunningQueries', 'QueuedQueries',
'CompletedQueries', 'FailedQueries');
```

```
-- Memory pool
SELECT * FROM
jmx.current."io.trino.memory:type=ClusterMemoryPool,name=general";
```

```
-- Node stats
SELECT * FROM jmx.current."io.trino.server:name=NodeResource";
```

### **system.runtime Tables**

```
-- Running queries
SELECT query_id, state, user, elapsed_time,
total_memory_reservation,
        LEFT(query, 100) AS query_preview
FROM system.runtime.queries
WHERE state = 'RUNNING'
ORDER BY total_memory_reservation DESC;
```

```
-- All recent queries (last ~1000)
SELECT query_id, state, error_code, error_type, user,
        created, execution_start_time, end_time
FROM system.runtime.queries
WHERE state = 'FAILED'
ORDER BY end_time DESC LIMIT 20;
```

```
-- Tasks
SELECT task_id, node_id, state, splits, completed_splits
FROM system.runtime.tasks
WHERE query_id = 'my-query-id-here';
```

```
-- Nodes
SELECT node_id, http_uri, node_version, state,
memory_pool_total_bytes
FROM system.runtime.nodes;
```

### **Prometheus Metrics**

Trino exposes Prometheus metrics natively at <http://<coordinator>:8080/metrics>. Scrape with Google Managed Prometheus:

```
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: trino-coordinator-monitoring
  namespace: trino
spec:
  selector:
    matchLabels:
      component: coordinator
  endpoints:
    - port: 8080
      path: /metrics
      interval: 30s
```

**Key metrics to alert on:**

| Metric                                                                     | Alert threshold     | Meaning                     |
|----------------------------------------------------------------------------|---------------------|-----------------------------|
| <code>trino_execution_query_manager_running_queries</code>                 | > 50                | Too many concurrent queries |
| <code>trino_execution_query_manager_queued_queries</code>                  | > 10 for<br>> 5 min | Queries backing up          |
| <code>trino_execution_query_manager_failed_queries_one_minute_count</code> | > 5/min             | High failure rate           |
| <code>trino_memory_cluster_memory_pool_free_bytes{pool="general"}</code>   | < 10%<br>of total   | Memory pressure             |
| <code>trino_execution_executor_running_splits_count</code>                 | > 80%<br>of max     | Worker saturation           |
| <code>jvm_memory_used_bytes{area="heap"}</code>                            | > 85%<br>of max     | JVM heap pressure           |

## Recommended Grafana Dashboard

Create panels for:

1. **Query throughput** — completed/failed queries per minute (rate)
2. **Active queries** — running and queued count
3. **Query latency** — p50/p95/p99 from `system.runtime.queries` via Prometheus or BigQuery export
4. **Memory** — cluster total, per-node free memory
5. **CPU** — worker node CPU utilisation from GKE node metrics
6. **Splits** — active splits per worker (indicates parallelism)
7. **Coordinator JVM heap** — heap used vs max

---

## 15. Fault Tolerance & Retry

### Task-Level Retry (Trino 395+)

When enabled, Trino retries individual failed tasks rather than the entire query. This makes queries resilient to transient worker failures (Spot VM preemptions, network blips).

```
# config.properties
retry-policy=TASK
exchange.deduplication-buffer-size=32MB
fault-tolerant-execution-task-memory=5GB
fault-tolerant-execution-max-task-retry-attempts=4
```

```
# Use Exchange Manager for spooling intermediate results
exchange.base-directories=/tmp/trino-exchange
# On GCS (recommended for GKE):
exchange-manager.name=filesystem
exchange.base-directories=gs://companyABC-trino-exchange/
```

With GCS-based exchange, even if a worker Pod is killed and restarted on a different node, the intermediate data (shuffled partitions) is safe in GCS and the new worker picks up where the old one left off.

### Graceful Pod Shutdown on GKE

Configure Trino to drain gracefully when a Pod receives SIGTERM (node drain, Spot preemption):

```
spec:
  terminationGracePeriodSeconds: 300    # 5 min for in-flight tasks
  to complete
  containers:
    - lifecycle:
```

```
preStop:
exec:
command: ["/bin/sh", "-c", "sleep 10"] # wait for LB
to de-register
```

---

## 16. Cost-Based Optimizer (CBO)

---

Trino's CBO uses table statistics to choose the best query plan — join order, join strategy, aggregation method.

### Enabling CBO

CBO is enabled by default. It is only effective when statistics are available.

### Collecting Statistics

```
-- Hive tables – must be run manually
```

```
ANALYZE hive.payments.events;
```

```
-- With partition filter (faster for large partitioned tables)
```

```
ANALYZE hive.payments.events
```

```
WITH (partitions = ARRAY[ARRAY['2026-06-07'], ARRAY['2026-06-06']]);
```

```
-- Iceberg – statistics are maintained automatically from manifest
files
```

```
-- BigQuery – statistics come from BQ table metadata automatically
```

```
-- PostgreSQL – statistics come from pg_statistic automatically
```

### What CBO Does

With statistics, Trino can:

- **Reorder joins** — start with the most selective table to minimise intermediate data
- **Choose broadcast vs hash joins** — broadcast small tables, hash-join large tables
- **Estimate selectivity** — predict how many rows a filter will eliminate
- **Choose aggregation strategy** — single-phase vs two-phase aggregation

```
-- Check if CBO is using stats
```

```
EXPLAIN (TYPE DISTRIBUTED)
```

```
SELECT r.payment_id, COUNT(*)
```

```
FROM hive.payments.events r
```

```
JOIN postgres.public.payments s ON r.payment_id = s.payment_id
```

```
WHERE r.dt = CURRENT_DATE
```

```
GROUP BY r.payment_id;
```

```
-- Look for "Estimates" in the plan output
```

---

## 17. Operations — Common Tasks

---

### Scale Workers on GKE

# Manual scale

```
kubectl scale deployment trino-worker --replicas=10 -n trino
```

# Via Helm

```
helm upgrade trino trino/trino -n trino --set server.workers=10  
--reuse-values
```

### Upgrade Trino Version

# 1. Update the image tag in values.yaml

# 2. Helm upgrade

```
helm upgrade trino trino/trino -n trino --values values.yaml
```

# Rolling update — coordinator first, then workers

```
kubectl rollout status deployment/trino-coordinator -n trino
```

```
kubectl rollout status deployment/trino-worker -n trino
```

### Kill a Specific Query

-- From Trino CLI or UI

```
SELECT query_id FROM system.runtime.queries WHERE state = 'RUNNING'  
AND user = 'airflow';
```

# Via REST API

```
curl -X DELETE http://trino-coordinator:8080/v1/query/<query-id> \  
-u admin:password
```

Or click **Kill** in the Trino Web UI.

### Add a New Catalog

1. Add the catalog properties file (via Helm `additionalCatalogs`)
2. Helm upgrade — no cluster restart needed for most connectors
3. Verify: `SHOW CATALOGS;`

### Reload Catalog Without Restart

# Trino supports dynamic catalog reload via REST (Trino 370+)

```
curl -X POST http://trino-coordinator:8080/v1/catalog \  
-H "Content-Type: application/json" \  
-d '{"catalogName": "my_new_catalog", "connectorName":  
"postgresql", "properties": {...}}'
```

### Partition Management

-- Add a partition manually

```
CALL hive.system.register_partition(  

```

```
    schema_name => 'payments',
    table_name => 'events',
    partition_columns => ARRAY['dt'],
    partition_values => ARRAY['2026-06-07'],
    location =>
'gs://companyABC-data-lake/payments/events/dt=2026-06-07/'
);
```

```
-- Drop a partition
CALL hive.system.unregister_partition(
    schema_name => 'payments',
    table_name => 'events',
    partition_columns => ARRAY['dt'],
    partition_values => ARRAY['2026-06-01']
);
```

```
-- Sync all GCS partitions to metastore
CALL hive.system.sync_partition_metadata('payments', 'events',
'FULL');
```

```
-- Show partitions
SELECT * FROM hive.payments."events$partitions";
```

### **Iceberg Table Maintenance**

```
-- Compact small files
ALTER TABLE iceberg.payments.events EXECUTE OPTIMIZE
WHERE dt >= DATE_ADD('day', -7, CURRENT_DATE); -- only recent
partitions
```

```
-- Expire old snapshots (keeps last 7 days of history)
ALTER TABLE iceberg.payments.events
EXECUTE expire_snapshots(retention_threshold => '7d');
```

```
-- Remove orphan files (GCS objects not referenced by any snapshot)
ALTER TABLE iceberg.payments.events
EXECUTE remove_orphan_files(retention_threshold => '7d');
```

Schedule these via Airflow or a CronJob — weekly or daily depending on write frequency.

### **Backup Metastore**

Dataproc Metastore supports exports:

```
gcloud metastore services export gcs companyABC-metastore \
  --location=asia-south1 \
  --destination-gcs-folder=gs://companyABC-backups/metastore/
```

---

## 18. Troubleshooting Guide

---

### Query Slow

-- Step 1: run EXPLAIN ANALYZE to see where time is spent

```
EXPLAIN ANALYZE
```

```
SELECT payment_id, COUNT(*) FROM hive.payments.events
```

```
WHERE dt = CURRENT_DATE GROUP BY payment_id;
```

-- Step 2: check if partition pruning is happening

-- Look for "Partition filters applied" in EXPLAIN output

-- If not, ensure you're filtering on the exact partition column

-- Step 3: check data volume

```
SELECT COUNT(*) FROM hive.payments.events WHERE dt = CURRENT_DATE;
```

-- Step 4: check for small files (large split count with small data)

```
SELECT "$file_size", "$file_modified_time", "$path"
```

```
FROM hive.payments.events WHERE dt = CURRENT_DATE LIMIT 10;
```

-- If many small files: run OPTIMIZE on the table

### Query Failing with OOM

Query exceeded per-node memory limit of 20GB

Steps:

1. Enable spilling: `SET SESSION spill_enabled = true;`
2. Reduce data: add more filters, sample data, split into smaller chunks
3. Optimise join: ensure smaller table is broadcast, not hash-joined
4. Increase worker memory (bigger node type)

-- Check if join is using broadcast (small table should be on probe side)

```
EXPLAIN SELECT ... FROM large_table JOIN small_table ON ...;
```

-- If small\_table is being hash-joined: SET SESSION

```
join_distribution_type = 'BROADCAST';
```

### Metastore Connection Errors

Failed connecting to Hive Metastore:

```
thrift://companyABC-metastore:9083
```

```
# Check from inside a Trino pod
kubectl exec -it <trino-coordinator-pod> -n trino -- /bin/bash
curl telnet://companyABC-metastore:9083 # or nc -zv
companyABC-metastore 9083
```

```
# Check Dataproc Metastore status
gcloud metastore services describe companyABC-metastore
--location=asia-south1
```

Causes: VPC firewall blocking port 9083, metastore service down, wrong URI in catalog config.

### **Workers Not Registering**

```
Coordinator reports 0 workers
# Check worker pods
kubectl get pods -n trino -l component=worker
```

```
# Check worker logs
kubectl logs -l component=worker -n trino --tail=50
```

```
# Common cause: wrong discovery.uri in worker config
# Must match the coordinator's service DNS name exactly
kubectl describe configmap trino -n trino
```

### **GCS Access Denied**

```
AccessDeniedException: 403 trino-gcp-sa does not have
storage.objects.get access
# Check Workload Identity is set up correctly
kubectl describe sa trino-sa -n trino
# Look for: iam.gke.io/gcp-service-account annotation
```

```
# Check GCP SA has the right permission
gcloud projects get-iam-policy my-project \
  --flatten="bindings[].members" \
  --filter="bindings.members:trino-gcp-sa@..."
```

```
# Verify Workload Identity binding
gcloud iam service-accounts get-iam-policy trino-gcp-sa@... \
  --format=json | grep workloadIdentityUser
```

### **Queries Queued (Not Running)**

```
Query queued – too many concurrent queries
```



```
-- Check concurrency limit  
SHOW SESSION LIKE '%max_concurrent%';
```

```
-- Check running queries  
SELECT COUNT(*) FROM system.runtime.queries WHERE state = 'RUNNING';
```

```
-- Check if workers are overloaded  
SELECT node_id, total_memory_reservation, max_total_memory  
FROM system.runtime.nodes;
```

Fixes: add more workers, increase `query.max-concurrent-queries`, check if a runaway query is consuming all resources.

---

Airflow

(Note: Used default font, to preserve text style | Refactored with GPT)

## Table of Contents

---

1. Airflow Overview & Philosophy
  2. Architecture — Components
  3. Core Concepts — DAG, Task, Operator, Run
  4. Defining DAGs
  5. TaskFlow API (Airflow 2.x)
  6. Operators
  7. Sensors
  8. Task Dependencies
  9. Task Groups
  10. Dynamic DAGs
  11. XCom — Cross-Task Communication
  12. Variables & Connections
  13. Hooks
  14. Scheduling & Timetables
  15. Executors
  16. Airflow on Kubernetes
  17. Pools, Queues & Priorities
  18. DAG Versioning & Best Practices
  19. Monitoring, Alerting & SLAs
  20. Airflow vs Alternatives
- 

## 1. Airflow Overview & Philosophy

---

Apache Airflow is an **open-source workflow orchestration platform** for authoring, scheduling, and monitoring data pipelines.

**Three core ideas:**

- **Workflows as code** — DAGs are Python programs. Version-controlled, testable, reviewable like any other code.
- **Declarative dependency graph** — You define what depends on what; Airflow figures out the execution order.
- **Pluggable** — Custom operators, hooks, executors, and plugins can extend any part of Airflow.

**What Airflow is NOT:**

- Not a streaming system — it schedules and triggers batch/micro-batch workloads. For real-time, use Flink or Kafka Streams.





## Webserver

- Serves the Airflow UI (Flask-based).
- Exposes the REST API.
- Reads DAG state from the metadata DB.
- Does **NOT** schedule or execute tasks.

## Scheduler

- Parses DAG files continuously (every `min_file_process_interval` seconds).
- Checks which DAG runs should be triggered based on schedule.
- Creates `TaskInstance` records in the DB.
- Sends tasks to the executor queue.
- Multiple schedulers can run in HA mode (Airflow 2.0+) — they use row-level DB locking.

## Triggerer (Airflow 2.2+)

- Manages **deferrable tasks** — tasks that wait for an external condition without occupying a worker slot.
- Uses async I/O (`asyncio`); one Triggerer can manage thousands of waiting tasks.
- When a deferred task's condition is met, it's re-queued for a worker to resume.

## Metadata Database

- The central state store — PostgreSQL (production) or MySQL.
- Stores: DAG runs, task instances, connections, variables, XComs, logs (optionally), pools, SLA misses.
- **SQLite is NOT for production** — no concurrent writes.

## Executor

- Determines how tasks are run — locally, on Celery workers, in Kubernetes pods, etc.
- See Section 15 for details.

## Workers

- Processes that actually execute the task Python code.
- In CeleryExecutor: standalone processes listening to a message queue (Redis/RabbitMQ).
- In KubernetesExecutor: each task spins up its own pod.

## DAG Folder

- Python files parsed by the scheduler.
- Mounted as a shared filesystem or synced via Git (git-sync sidecar on Kubernetes).

---

### 3. Core Concepts — DAG, Task, Operator, Run

---

#### DAG (Directed Acyclic Graph)

A DAG is a collection of tasks with defined dependencies. "Directed" = dependencies have direction. "Acyclic" = no circular dependencies.

- Defined in a Python file placed in the `dags/` folder.
- The DAG object sets the schedule, start date, default args, and metadata.
- A DAG is **NOT** the same as a DAG Run — a DAG is the definition; a DAG Run is one execution instance of that definition.

#### Task

An instance of an Operator assigned a specific position in the DAG. A task has:

- A unique `task_id` within the DAG.
- Dependencies (upstream/downstream tasks).
- A state in each DAG Run: `queued` → `running` → `success` / `failed` / `skipped` / `upstream_failed`.

#### Operator

A template that defines a unit of work. Operators are instantiated as Tasks.

- `PythonOperator` — runs a Python function.
- `BashOperator` — runs a shell command.
- `SparkSubmitOperator` — submits a Spark job.
- `KubernetesPodOperator` — runs a task in a Kubernetes pod.
- Hundreds of community operators via `apache-airflow-providers-*` packages.

#### DAG Run

One execution of a DAG for a specific **logical date** (previously called `execution_date`). Each run has:

- A `run_id` (unique identifier).
- A `logical_date` (the data interval start — does NOT mean when the task ran).
- A `data_interval_start` and `data_interval_end`.
- A `run_type`: `scheduled`, `manual`, `dataset_triggered`, `backfill`.

#### Task Instance

One execution of a Task within a specific DAG Run. The combination of (DAG ID + task ID + logical date) uniquely identifies a task instance.

#### Logical Date vs Execution Date

Schedule: @daily (runs at midnight every day)

Logical date (data\_interval\_start): 2026-06-15 00:00:00

Actual trigger time: 2026-06-16 00:00:00

→ The DAG processes data FOR 2026-06-15 but RUNS on 2026-06-16.

This "one interval behind" behavior is important for partition-based data loading.

---

## 4. Defining DAGs

---

### Classic Style (context manager)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
```

```
default_args = {
    "owner": "data-platform",
    "depends_on_past": False,
    "email": ["alerts@acme.com"],
    "email_on_failure": True,
    "email_on_retry": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
}
```

```
with DAG(
    dag_id="iot_telemetry_pipeline",
    description="Daily ETL for robot telemetry data",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule="0 2 * * *",          # 02:00 UTC daily (cron
    expression)
    catchup=False,                 # Don't backfill past runs on
    first_deploy
    max_active_runs=1,             # Only 1 DAG run at a time
```

```

    max_active_tasks=5,                # Max concurrent tasks in this
DAG
    tags=["telemetry", "etl", "prod"],
    doc_md="""
    ## Robot Telemetry Pipeline
    Reads raw telemetry from GCS, transforms, writes to BigQuery.
    """,
) as dag:

    def extract():
        # read from GCS
        pass

    def transform():
        # clean + enrich
        pass

    t1 = PythonOperator(task_id="extract", python_callable=extract)
    t2 = PythonOperator(task_id="transform",
python_callable=transform)
    t3 = BashOperator(task_id="load", bash_command="bq load
--project=...")

    t1 >> t2 >> t3

```

### Important DAG Parameters

| Parameter               | Description                                                                                                    | Recommended                                     |
|-------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>schedule</code>   | Cron, <code>@daily</code> , <code>timedelta</code> , <code>Dataset</code> , or <code>None</code> (manual only) | Use cron strings                                |
| <code>start_date</code> | Logical start; first run covers <code>[start_date, start_date + interval)</code>                               | Fixed datetime, not <code>datetime.now()</code> |
| <code>catchup</code>    | If True, backfills all missed runs since <code>start_date</code>                                               | <code>False</code> for most production DAGs     |



|                               |                                                       |                                           |
|-------------------------------|-------------------------------------------------------|-------------------------------------------|
| <code>max_active_runs</code>  | Concurrent DAG runs allowed                           | 1 for sequential ETL pipelines            |
| <code>max_active_tasks</code> | Max concurrent task instances for this DAG            | Based on resource constraints             |
| <code>depends_on_past</code>  | Task only runs if same task in previous run succeeded | <code>False</code> unless strictly needed |
| <code>dagrun_timeout</code>   | Max time before a DAG run is marked failed            | Set to avoid hung runs                    |
| <code>tags</code>             | Organizational labels visible in the UI               | Always set                                |

## 5. TaskFlow API (Airflow 2.x)

The TaskFlow API uses decorators to define DAGs and tasks with less boilerplate. XCom passing happens automatically between decorated functions.

```
from airflow.decorators import dag, task
from datetime import datetime
```

```
@dag(
    dag_id="etl_pipeline_taskflow",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["telemetry"],
)
def robot_pipeline():

    @task()
    def extract() -> dict:
        # Airflow automatically pushes the return value to XCom
        return {"rows": 1000, "source": "gcs://bucket/raw/" }

    @task()
    def transform(raw_data: dict) -> dict:
        # raw_data is automatically pulled from XCom
        return {"rows": raw_data["rows"], "status": "cleaned" }
```

```
@task()
def load(clean_data: dict) -> None:
    print(f>Loading {clean_data['rows']} rows to BigQuery")
```

```
# Implicit dependency via function call chaining
raw = extract()
cleaned = transform(raw)
load(cleaned)
```

```
# Instantiate the DAG
robot_pipeline()
```

### TaskFlow vs Classic Operators

|                         | Classic Operators                | TaskFlow API                  |
|-------------------------|----------------------------------|-------------------------------|
| <b>Code style</b>       | Class-based                      | Decorator-based               |
| <b>XCom</b>             | Manual push/pull                 | Automatic via return values   |
| <b>Dependencies</b>     | >> / << operators                | Function call chaining        |
| <b>Readability</b>      | More verbose                     | Cleaner for Python-heavy DAGs |
| <b>Custom operators</b> | Still needed for non-Python work | Still use classic operators   |

### Mixing TaskFlow and Classic Operators

```
from airflow.decorators import dag, task
from airflow.providers.google.cloud.operators.bigquery import
BigQueryInsertJobOperator
```

```
@dag(schedule="@daily", start_date=datetime(2026, 1, 1),
catchup=False)
def mixed_dag():
```

```
@task()
def prepare_query() -> str:
    return "SELECT * FROM `project.dataset.table` WHERE date =
'2026-06-15'"
```

```

    # Classic operator used directly
    run_bq_job = BigQueryInsertJobOperator(
        task_id="run_bq_job",
        configuration={
            "query": {
                "query": "{{
task_instance.xcom_pull(task_ids='prepare_query') }}"},
            "useLegacySql": False,
        },
        project_id="my-project",
    )

    query = prepare_query()
    query >> run_bq_job

mixed_dag()

```

---

## 6. Operators

---

### PythonOperator

```

from airflow.operators.python import PythonOperator

def my_function(param1, param2, **context):
    # context contains: dag, task, run_id, logical_date, ds (date
    string), etc.
    logical_date = context["ds"] # "2026-06-15"
    print(f"Processing date: {logical_date}, param1={param1}")
    return "result_value" # pushed to XCom automatically

task = PythonOperator(
    task_id="process_data",
    python_callable=my_function,
    op_kwargs={"param1": "value1", "param2": "value2"},
)

```

### BashOperator

```

from airflow.operators.bash import BashOperator

```

```
task = BashOperator(
    task_id="run_script",
    bash_command="python /opt/scripts/process.py --date {{ ds }}",
    env={"MY_ENV_VAR": "value"},
)
```

### SparkSubmitOperator

```
from airflow.providers.apache.spark.operators.spark_submit import
SparkSubmitOperator
```

```
spark_task = SparkSubmitOperator(
    task_id="run_spark_job",
    application="/opt/spark/jobs/telemetry_etl.py",
    conn_id="spark_default",
    conf={
        "spark.executor.memory": "4g",
        "spark.executor.cores": "2",
        "spark.sql.shuffle.partitions": "100",
    },
    application_args=["--date", "{{ ds }}"],
    name="telemetry_etl_{{ ds }}",
    verbose=False,
)
```

### BigQuery Operators

```
from airflow.providers.google.cloud.operators.bigquery import (
    BigQueryInsertJobOperator,
    BigQueryCheckOperator,
    BigQueryCreateEmptyTableOperator,
)
```

# Run a SQL query job

```
bq_query = BigQueryInsertJobOperator(
    task_id="run_sql",
    configuration={
        "query": {
            "query": """
                INSERT INTO `project.dataset.output_table`
                SELECT * FROM `project.dataset.source_table`
                WHERE DATE(event_time) = '{{ ds }}'
            """
        }
    }
```

```

        "",
        "useLegacySql": False,
        "writeDisposition": "WRITE_APPEND",
    }
},
    project_id="my-gcp-project",
    location="US",
)

```

```

# Data quality check – fails task if query returns 0 rows
bq_check = BigQueryCheckOperator(
    task_id="check_row_count",
    sql="SELECT COUNT(*) FROM `project.dataset.output_table` WHERE
date = '{{ ds }}'",
    use_legacy_sql=False,
)

```

### EmptyOperator / DummyOperator

```

from airflow.operators.empty import EmptyOperator

```

```

start = EmptyOperator(task_id="start")
end = EmptyOperator(task_id="end")

```

```

start >> [task_a, task_b, task_c] >> end

```

### BranchPythonOperator

```

from airflow.operators.python import BranchPythonOperator

```

```

def choose_branch(**context):
    day = context["logical_date"].weekday()
    if day < 5:
        return "weekday_task"
    else:
        return "weekend_task"    # returns task_id of branch to
follow

```

```

branch = BranchPythonOperator(
    task_id="branch",
    python_callable=choose_branch,
)

```

```
weekday_task = PythonOperator(task_id="weekday_task",
python_callable=lambda: print("weekday"))
weekend_task = PythonOperator(task_id="weekend_task",
python_callable=lambda: print("weekend"))
```

```
branch >> [weekday_task, weekend_task]
```

### ShortCircuitOperator

```
from airflow.operators.python import ShortCircuitOperator
```

```
def should_run(**context):
    # Return True to continue, False to skip all downstream tasks
    return context["ds"] != "2026-01-01" # skip on New Year's Day
```

```
check = ShortCircuitOperator(
    task_id="check_should_run",
    python_callable=should_run,
    ignore_downstream_trigger_rules=True, # skips ALL downstream,
not just direct
)
```

### TriggerDagRunOperator

```
from airflow.operators.trigger_dagrun import TriggerDagRunOperator
```

```
trigger = TriggerDagRunOperator(
    task_id="trigger_downstream_dag",
    trigger_dag_id="downstream_pipeline",
    conf={"source_date": "{{ ds }}"},
    wait_for_completion=True, # block until triggered DAG
finishes
    poke_interval=30,
)
```

---

## 7. Sensors

Sensors are a special type of Operator that **wait for a condition** to be met before proceeding. They use either **poke** mode (occupies a worker slot while polling) or **reschedule** mode (releases the worker slot between polls).

### Poke vs Reschedule Mode

| Mode                           | Behavior                            | Use when                                |
|--------------------------------|-------------------------------------|-----------------------------------------|
| <code>poke</code><br>(default) | Worker slot held during entire wait | Short waits (< 5 min)                   |
| <code>reschedule</code>        | Releases slot between polls         | Long waits (hours); saves resources     |
| <code>deferrable</code>        | Async via Triggerer; no slot held   | Production best practice (Airflow 2.2+) |

```
from airflow.sensors.filesystem import FileSensor
from airflow.sensors.python import PythonSensor
from airflow.providers.google.cloud.sensors.gcs import
GCSObjectExistenceSensor
```

```
# Wait for a file to appear
file_sensor = FileSensor(
    task_id="wait_for_file",
    filepath="/data/incoming/{{ ds }}/manifest.json",
    mode="reschedule",
    poke_interval=60,      # check every 60 seconds
    timeout=3600,         # fail after 1 hour
)
```

```
# Wait for a GCS object
gcs_sensor = GCSObjectExistenceSensor(
    task_id="wait_for_gcs_file",
    bucket="my-data-bucket",
    object="raw/{{ ds }}/data.parquet",
    google_cloud_conn_id="google_cloud_default",
    mode="reschedule",
    poke_interval=300,
    timeout=7200,
)
```

```
# Custom condition sensor
def check_api(**context):
    import requests
```

```
resp = requests.get("https://api.example.com/status")
return resp.json()["status"] == "ready"
```

```
api_sensor = PythonSensor(
    task_id="wait_for_api",
    python_callable=check_api,
    mode="reschedule",
    poke_interval=120,
    timeout=3600,
)
```

### ExternalTaskSensor — Wait for Another DAG's Task

```
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import timedelta
```

```
wait_for_upstream = ExternalTaskSensor(
    task_id="wait_for_upstream_dag",
    external_dag_id="upstream_pipeline",
    external_task_id="load_complete", # None = wait for entire DAG
    to complete
    mode="reschedule",
    poke_interval=60,
    timeout=7200,
    # Use if schedules differ – offset to find the right run
    execution_delta=timedelta(hours=1),
)
```

### Deferrable Sensors (Airflow 2.2+)

Deferrable sensors use the Triggerer component — they do not occupy a worker slot while waiting.

```
from airflow.providers.google.cloud.sensors.gcs import
GCSObjectExistenceSensor
```

```
# Same operator but with deferrable=True – requires Triggerer
running
gcs_sensor = GCSObjectExistenceSensor(
    task_id="wait_for_gcs_file",
    bucket="my-bucket",
    object="data/{{ ds }}/file.parquet",
    google_cloud_conn_id="google_cloud_default",
```



```

    deferrable=True,      # Use Triggerer instead of polling in
worker
    poke_interval=60,
)

```

## 8. Task Dependencies

### >> and << Operators

```
# Sequential
```

```
t1 >> t2 >> t3
```

```
# Fan out – t1 runs, then t2 and t3 run in parallel
```

```
t1 >> [t2, t3]
```

```
# Fan in – t2 and t3 must complete before t4
```

```
[t2, t3] >> t4
```

```
# Full example
```

```
start >> extract >> [transform_a, transform_b] >> merge >> load >>
end
```

### set\_upstream / set\_downstream

```
t2.set_upstream(t1)      # equivalent to t1 >> t2
```

```
t2.set_downstream(t3)    # equivalent to t2 >> t3
```

### Trigger Rules

By default, a task runs only when **all upstream tasks succeed** (`all_success`).

Trigger rules change this behavior.

| Rule                     | Description                        | Use case                   |
|--------------------------|------------------------------------|----------------------------|
| <code>all_success</code> | All upstreams succeeded (default)  | Standard pipelines         |
| <code>all_failed</code>  | All upstreams failed               | Failure notification tasks |
| <code>all_done</code>    | All upstreams finished (any state) | Cleanup tasks              |
| <code>one_success</code> | At least one upstream succeeded    | OR logic                   |

|                                          |                                   |                            |
|------------------------------------------|-----------------------------------|----------------------------|
| <code>one_failed</code>                  | At least one upstream failed      | Alert on any failure       |
| <code>none_failed</code>                 | No upstreams failed (skipped OK)  | After branching            |
| <code>none_failed_min_one_success</code> | No failures, at least one success | After branching with skips |
| <code>always</code>                      | Run regardless of upstream state  | Teardown tasks             |

```
from airflow.utils.trigger_rule import TriggerRule
```

```
# Run cleanup even if upstream tasks fail
cleanup = PythonOperator(
    task_id="cleanup",
    python_callable=do_cleanup,
    trigger_rule=TriggerRule.ALL_DONE,
)
```

```
# Notification task – run only if any upstream failed
notify_failure = PythonOperator(
    task_id="notify_failure",
    python_callable=send_alert,
    trigger_rule=TriggerRule.ONE_FAILED,
)
```

---

## 9. Task Groups

---

Task Groups visually group related tasks in the Airflow UI without affecting execution.

```
from airflow.utils.task_group import TaskGroup
from airflow.operators.python import PythonOperator
```

```
with DAG("grouped_pipeline", schedule="@daily",
start_date=datetime(2026,1,1), catchup=False) as dag:
```

```
    with TaskGroup("extract", tooltip="All extraction tasks") as
extract_group:
```

```

        extract_gcs = PythonOperator(task_id="from_gcs",
python_callable=extract_from_gcs)
        extract_pg = PythonOperator(task_id="from_postgres",
python_callable=extract_from_pg)
        extract_api = PythonOperator(task_id="from_api",
python_callable=extract_from_api)

```

```

    with TaskGroup("transform") as transform_group:
        clean = PythonOperator(task_id="clean",
python_callable=clean_data)
        enrich = PythonOperator(task_id="enrich",
python_callable=enrich_data)
        clean >> enrich

```

```

    with TaskGroup("load") as load_group:
        load_bq = PythonOperator(task_id="to_bigquery",
python_callable=load_bigquery)
        load_gcs = PythonOperator(task_id="to_gcs_archive",
python_callable=archive_gcs)

```

```

extract_group >> transform_group >> load_group

```

In the UI, tasks appear as `extract.from_gcs`, `transform.clean`, etc. Task groups can be collapsed.

---

## 10. Dynamic DAGs

### Pattern 1 — Loop-based (single DAG, multiple tasks)

```

from airflow.decorators import dag, task
from datetime import datetime
# Fintech example: processing payments by region
REGIONS = ["EMEA", "APAC", "US", "LATAM"]
@dag(schedule="@daily", start_date=datetime(2026, 1, 1),
catchup=False)
def payment_batch_pipeline():
    for region in REGIONS:
        @task(task_id=f"process_{region}_payments")
        def process_region(region_code: str) -> dict:
            print(f"Processing payment batch for {region_code}")
            return {"region": region_code, "status": "settled"}

```

```
    process_region(region)
payment_batch_pipeline()
```

## Pattern 2 — Dynamic Task Mapping (Airflow 2.3+)

```
# Fintech example: reconciling across multiple payment gateways
```

```
from airflow.decorators import dag, task
```

```
from datetime import datetime
```

```
@dag(schedule="@daily", start_date=datetime(2026, 1, 1),
catchup=False)
```

```
def gateway_reconciliation_pipeline():
```

```
    @task()
```

```
    def get_active_gateways() -> list[str]:
```

```
        # Could query a DB to get active providers
```

```
        return ["STRIPE", "PAYPAL", "ADYEN"]
```

```
    @task()
```

```
    def reconcile_gateway(gateway_id: str) -> dict:
```

```
        print(f"Reconciling transactions for {gateway_id}")
```

```
        return {"gateway": gateway_id, "processed_count": 5000}
```

```
    @task()
```

```
    def generate_report(results: list[dict]) -> None:
```

```
        print(f"Report generated for {len(results)} gateways")
```

```
    gateways = get_active_gateways()
```

```
    # expand() creates one task instance per payment gateway
```

```
    results = reconcile_gateway.expand(gateway_id=gateways)
```

```
    generate_report(results)
```

```
gateway_reconciliation_pipeline()
```

## Pattern 3 — Generate Multiple DAGs from Config

```
# Fintech example: per-gateway pipeline configurations
```

```
from airflow.decorators import dag, task
```

```
from datetime import datetime
```

```
GATEWAY_CONFIG = [
```

```
    {"gateway_id": "stripe", "region": "us", "priority": "high"},
```

```
    {"gateway_id": "paypal", "region": "eu", "priority": "normal"},
```

```
    {"gateway_id": "adyen", "region": "ap", "priority": "low"},
```

```
]
```

```
def create_gateway_dag(gateway_id: str, region: str, priority: str):
```

```
    @dag(
```

```
        dag_id=f"gateway_pipeline_{gateway_id}",
```

```
        schedule="@hourly",
```

```

        start_date=datetime(2026, 1, 1),
        catchup=False,
        tags=["finance", "payment", "priority"],
    )

    def gateway_dag():
        @task()
        def process():
            print(f"Processing {gateway_id} batch for {region}")
            process()
        return gateway_dag()
# Create one DAG per gateway
for config in GATEWAY_CONFIG:
    globals()[f"gateway_pipeline_{config['gateway_id']}"] =
    create_gateway_dag(**config)

```

**Important:** The Airflow scheduler parses DAG files repeatedly. Keep DAG file-level code fast — no DB queries, no heavy I/O. Load config from Variables or a lightweight file.

---

## 11. XCom — Cross-Task Communication

XCom (cross-communication) allows tasks to share small amounts of data via the metadata database.

- **Push:** A task pushes a value with a key.
- **Pull:** A downstream task pulls the value by key and source task ID.
- **TaskFlow API:** Return values are automatically pushed/pulled.
- **Size limit:** XCom values are stored in the DB — keep them **small** (< 1MB). For large data, write to GCS/S3 and pass the path via XCom.

```

# Classic push/pull
def push_data(**context):
    context["task_instance"].xcom_push(key="row_count", value=5000)

def pull_data(**context):
    count = context["task_instance"].xcom_pull(task_ids="push_task",
    key="row_count")
    print(f"Row count from previous task: {count}")

push_task = PythonOperator(task_id="push_task",
python_callable=push_data)

```

```
pull_task = PythonOperator(task_id="pull_task",
python_callable=pull_data)
# TaskFlow API – automatic XCom
@task()
def extract() -> str:
    path = "gs://my-bucket/data/2026-06-16/data.parquet"
    return path # automatically pushed to XCom
```

```
@task()
def transform(file_path: str) -> dict:
    print(f"Processing {file_path}") # automatically pulled
    return {"rows": 1000}
```

### XCom in Jinja Templates

```
BashOperator(
    task_id="use_xcom_in_bash",
    bash_command="echo {{
task_instance.xcom_pull(task_ids='extract', key='return_value') }}",
)
```

### Custom XCom Backend

For large data, configure a custom XCom backend that stores to GCS instead of the DB:

```
# airflow.cfg or environment variable
AIRFLOW__CORE__XCOM_BACKEND =
"airflow.providers.google.cloud.utils.airflow_to_gcs_xcom.GCSXComBackend"
```

---

## 12. Variables & Connections

### Variables

Key-value pairs stored in the metadata DB. Used for configuration that changes across environments (dev/staging/prod).

```
from airflow.models import Variable
```

```
# Get a variable (with optional default)
project_id = Variable.get("gcp_project_id",
default_var="my-default-project")
```

```
# Get JSON-serialized variable
config = Variable.get("pipeline_config", deserialize_json=True)
```

```
# Returns a dict: {"batch_size": 1000, "timeout": 300}
```

```
# In Jinja templates
```

```
BashOperator(  
    task_id="use_variable",  
    bash_command="echo {{ var.value.gcp_project_id }}",  
    # or for JSON: {{ var.json.pipeline_config.batch_size }}  
)
```

```
# CLI management
```

```
airflow variables set gcp_project_id "my-project"
```

```
airflow variables get gcp_project_id
```

```
airflow variables import variables.json # bulk import
```

```
airflow variables export variables.json # backup
```

**Best practice:** Don't store secrets in Variables. Use Airflow Connections or a secrets backend (GCP Secret Manager, HashiCorp Vault).

## Connections

Connections store credentials and endpoints for external systems (databases, cloud providers, APIs).

```
from airflow.hooks.base import BaseHook
```

```
# Get connection details
```

```
conn = BaseHook.get_connection("my_postgres_conn")
```

```
print(conn.host, conn.port, conn.login, conn.password)
```

```
# CLI management
```

```
airflow connections add 'my_postgres_conn' \
```

```
--conn-type postgres \
```

```
--conn-host localhost \
```

```
--conn-port 5432 \
```

```
--conn-login myuser \
```

```
--conn-password mypassword \
```

```
--conn-schema mydb
```

```
airflow connections list
```

```
airflow connections delete my_postgres_conn
```

## Secrets Backend — GCP Secret Manager

```
# airflow.cfg
```

```
[secrets]
```

```

backend =
airflow.providers.google.cloud.secrets.secret_manager.CloudSecretMan
agerBackend
backend_kwargs = {"project_id": "my-project", "connections_prefix":
"airflow-connections", "variables_prefix": "airflow-variables"}

# Airflow will look up:
# Connection "postgres_default" → GCP secret
"airflow-connections-postgres-default"
# Variable "my_var" → GCP secret
"airflow-variables-my-var"

```

---

### 13. Hooks

Hooks are interfaces to external systems. They handle connection management and authentication. Operators typically use Hooks internally.

```

from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.google.cloud.hooks.gcs import GCSHook
from airflow.providers.google.cloud.hooks.bigquery import
BigQueryHook

```

```

# PostgreSQL Hook
def query_postgres(**context):
    hook = PostgresHook(postgres_conn_id="my_postgres_conn")
    records = hook.get_records("SELECT * FROM events WHERE date =
%s", parameters=[context["ds"]])
    return len(records)

```

```

# GCS Hook
def upload_to_gcs(**context):
    hook = GCSHook(gcp_conn_id="google_cloud_default")
    hook.upload(
        bucket_name="my-bucket",
        object_name=f"output/{context['ds']}/results.csv",
        filename="/tmp/results.csv",
    )

```

```

# BigQuery Hook – run a query and get results
def query_bq(**context):

```



```
hook = BigQueryHook(gcp_conn_id="google_cloud_default",
use_legacy_sql=False)
client = hook.get_client()
results = client.query("SELECT COUNT(*) FROM
`project.dataset.table`").result()
return [row[0] for row in results][0]
```

### Custom Hook

```
from airflow.hooks.base import BaseHook
import requests
```

```
class MyAPIHook(BaseHook):
    def __init__(self, conn_id: str = "my_api_default"):
        super().__init__()
        self.conn_id = conn_id

    def get_conn(self):
        conn = self.get_connection(self.conn_id)
        self.base_url = f"https://{conn.host}"
        self.token = conn.password
        return requests.Session()

    def get_data(self, endpoint: str) -> dict:
        session = self.get_conn()
        response = session.get(
            f"{self.base_url}/{endpoint}",
            headers={"Authorization": f"Bearer {self.token}"})
        response.raise_for_status()
        return response.json()
```

---

## 14. Scheduling & Timetables

### Cron Expressions

```
schedule="0 2 * * *"      # 02:00 UTC daily
schedule="0 */6 * * *"    # Every 6 hours
schedule="0 9 * * 1"      # Every Monday at 09:00 UTC
schedule="0 0 1 * * *"    # First day of every month
schedule="@daily"          # Midnight daily (same as "0 0 * * *")
schedule="@hourly"
```

```

schedule="@weekly"
schedule="@monthly"
schedule="@once"          # Run once and never again
schedule=None             # Manual trigger only

```

### Timedelta Scheduling

```

from datetime import timedelta
schedule=timedelta(hours=6)  # Every 6 hours from start_date

```

### Dataset-Based Scheduling (Airflow 2.4+)

Trigger a DAG when an upstream DAG produces a dataset (logical data dependency, not time-based).

```

from airflow import Dataset
from airflow.decorators import dag, task

```

```

# Define a dataset
robot_telemetry_dataset = Dataset("gs://my-bucket/telemetry/{{ ds
}}/")

```

```

# Producer DAG – marks dataset as updated when task completes
@dag(schedule="@daily", start_date=datetime(2026, 1, 1),
catchup=False)
def producer_dag():
    @task(outlets=[robot_telemetry_dataset]) # marks the dataset as
updated
    def produce():
        # write data to GCS
        pass
    produce()

```

```

# Consumer DAG – triggered when the dataset is updated
@dag(schedule=[robot_telemetry_dataset], catchup=False) # no
time-based schedule
def consumer_dag():
    @task()
    def consume():
        print("Dataset is ready, processing...")
    consume()

```

### Custom Timetable (Airflow 2.2+)

```

from airflow.timetables.base import DagRunInfo, DataInterval,
TimeRestriction, Timetable
from pendulum import DateTime, instance

class BusinessHoursTimetable(Timetable):
    """Run at 9am, 12pm, and 5pm on weekdays only."""
    ...

```

---

## 15. Executors

The executor determines **where and how tasks run**. Configured via `AIRFLOW__CORE__EXECUTOR`.

### SequentialExecutor

- Tasks run one at a time in the same process as the scheduler.
- Uses SQLite.
- **Development only** — not for production.

### LocalExecutor

- Tasks run in separate processes on the same machine as the scheduler.
- Supports parallelism via `parallelism` config.
- No external message broker required.
- **Good for small-scale production** or single-node deployments.

```

[core]
executor = LocalExecutor
parallelism = 32 # max concurrent tasks

```

### CeleryExecutor

- Tasks are sent to a **message broker** (Redis or RabbitMQ) and picked up by Celery workers.
- Workers can be on separate machines — horizontally scalable.
- Requires: Redis/RabbitMQ (broker), result backend (Redis or DB).

```

[core]
executor = CeleryExecutor

[celery]
broker_url = redis://redis:6379/0
result_backend = db+postgresql://airflow:airflow@postgres/airflow
worker_concurrency = 16 # tasks per worker process
# Start a Celery worker
airflow celery worker --queues default,high_priority

```

### KubernetesExecutor

- Each task runs in its **own Kubernetes Pod**.
- No permanent workers — pods are created on demand and deleted after task completes.
- Complete resource isolation per task.
- Slower startup (pod scheduling overhead ~5–30s).
- Best for tasks with varying resource requirements.

```
[core]
```

```
executor = KubernetesExecutor
```

```
[kubernetes]
```

```
namespace = airflow
```

```
worker_container_repository = my-registry/airflow
```

```
worker_container_tag = 2.9.0
```

```
delete_worker_pods = True
```

### CeleryKubernetesExecutor

- Hybrid: Routes tasks to CeleryExecutor (fast, low overhead) or KubernetesExecutor based on queue.
- Light tasks → Celery workers (fast start).
- Heavy/isolated tasks → Kubernetes pods.

```
# Route to Kubernetes executor by queue name
```

```
heavy_task = SparkSubmitOperator(
```

```
    task_id="heavy_spark_job",
```

```
    queue="kubernetes", # sends to KubernetesExecutor
```

```
    ...
```

```
)
```

### Executor Comparison

|                           | Sequential  | Local          | Celery          | Kubernetes |
|---------------------------|-------------|----------------|-----------------|------------|
| <b>Scalability</b>        | Single task | Single machine | Multi-machine   | Per-pod    |
| <b>Setup complexity</b>   | None        | Low            | Medium (broker) | High (K8s) |
| <b>Resource isolation</b> | None        | Process        | Process         | Pod        |

|                         |      |            |               |              |
|-------------------------|------|------------|---------------|--------------|
| <b>Startup overhead</b> | None | ~100ms     | ~1s           | 5–30s        |
| <b>Use case</b>         | Dev  | Small prod | Scalable prod | Cloud-native |

---

## 16. Airflow on Kubernetes

---

### Helm Deployment

```
helm repo add apache-airflow https://airflow.apache.org
helm repo update
```

```
helm upgrade --install airflow apache-airflow/airflow \
  --namespace airflow \
  --create-namespace \
  --values airflow-values.yaml \
  --version 1.14.0
```

### airflow-values.yaml — Key Production Settings

```
# Executor
executor: KubernetesExecutor # or CeleryKubernetesExecutor
```

```
# Image
images:
  airflow:
    repository: my-registry/airflow
    tag: 2.9.0-python3.11
    pullPolicy: IfNotPresent
```

```
# PostgreSQL metadata DB
postgresql:
  enabled: false # use external managed DB (Cloud SQL on GCP)
data:
  metadataConnection:
    user: airflow
    pass: airflow
    host: 10.0.0.5 # Cloud SQL private IP
    port: 5432
    db: airflow
```

```
# DAG source – git-sync sidecar
dags:
  gitSync:
    enabled: true
    repo: https://github.com/acme/data-platform
    branch: main
    subPath: dags/
    period: 60s           # sync every 60 seconds
    credentialsSecret: git-credentials
```

```
# Scheduler HA
scheduler:
  replicas: 2             # HA – 2 schedulers, one is active
  resources:
    requests:
      cpu: "500m"
      memory: "1Gi"
    limits:
      cpu: "2"
      memory: "2Gi"
```

```
# Webserver
webserver:
  replicas: 2
  service:
    type: ClusterIP
  resources:
    requests:
      cpu: "500m"
      memory: "1Gi"
```

```
# Triggerer (for deferrable tasks)
triggerer:
  enabled: true
  replicas: 1
  resources:
    requests:
```

```
    cpu: "200m"
    memory: "512Mi"
```

```
# Worker (for CeleryExecutor)
```

```
workers:
```

```
  replicas: 3
  resources:
    requests:
      cpu: "1"
      memory: "2Gi"
  limits:
    cpu: "4"
    memory: "8Gi"
```

```
# Redis (for CeleryExecutor)
```

```
redis:
  enabled: true
```

```
# Logs – store in GCS
```

```
logs:
  persistence:
    enabled: true
  gcs:
    enabled: true
    bucket: my-airflow-logs-bucket
```

```
# Extra environment variables
```

```
env:
  - name: AIRFLOW__CORE__FERNET_KEY
    valueFrom:
      secretKeyRef:
        name: airflow-secrets
        key: fernet-key
  - name: AIRFLOW__SECRETS__BACKEND
    value:
```

```
airflow.providers.google.cloud.secrets.secret_manager.CloudSecretManagerBackend
```

```
KubernetesPodOperator
```

Runs a task in an arbitrary Docker container inside Kubernetes. The most flexible operator for cloud-native workloads.

```
from airflow.providers.cncf.kubernetes.operators.pod import
KubernetesPodOperator
from kubernetes.client import models as k8s

spark_job = KubernetesPodOperator(
    task_id="spark_etl_job",
    name="spark-etl-{{ ds_nodash }}",          # pod name
    namespace="airflow-jobs",
    image="my-registry/spark-jobs:latest",
    image_pull_policy="Always",
    cmds=["python"],
    arguments=["/app/etl.py", "--date", "{{ ds }}"],
    env_vars={
        "GCP_PROJECT": "my-project",
        "LOGICAL_DATE": "{{ ds }}",
    },
    resources=k8s.V1ResourceRequirements(
        requests={"cpu": "2", "memory": "4Gi"},
        limits={"cpu": "4", "memory": "8Gi"},
    ),
    service_account_name="spark-sa",
    is_delete_operator_pod=True,                # delete pod on
success
    get_logs=True,                             # stream pod logs to
Airflow logs
    in_cluster=True,                           # Airflow itself is in
the cluster
    startup_timeout_seconds=300,
    on_finish_action="delete_pod",
)
```

### git-sync Sidecar Pattern

Instead of baking DAGs into the image, use a git-sync sidecar to keep DAGs up to date:

```
Airflow Scheduler Pod
├── scheduler container
└── git-sync container ← pulls from GitHub every 60s
```



└─ Shared volume (/opt/airflow/dags)

This means deploying new DAGs = merging to the `main` branch. No image rebuild needed.

---

## 17. Pools, Queues & Priorities

---

### Pools

Pools limit the number of concurrently running tasks of a certain type. Use them to protect downstream systems (databases, APIs) from being overwhelmed.

# Define pool in UI or CLI

```
# airflow pools set spark_pool 5 "Max 5 concurrent Spark jobs"
```

```
spark_job = SparkSubmitOperator(
    task_id="run_spark",
    pool="spark_pool",          # limited to 5 concurrent across all
                                DAGs
    pool_slots=1,              # this task uses 1 slot (default)
    ...
)
```

# A heavy job can use more slots

```
heavy_job = SparkSubmitOperator(
    task_id="heavy_spark_job",
    pool="spark_pool",
    pool_slots=3,              # uses 3 out of 5 pool slots
    ...
)
```

```
airflow pools list
```

```
airflow pools set spark_pool 5 "Spark job concurrency limit"
```

```
airflow pools delete spark_pool
```

### Queues (CeleryExecutor)

Route tasks to specific Celery workers. Workers listen to named queues.

# Task goes to high-priority queue

```
urgent_task = PythonOperator(
    task_id="urgent",
    python_callable=do_work,
    queue="high_priority",
)
```

# Start workers for specific queues

```
airflow celery worker --queues default,high_priority
airflow celery worker --queues gpu_jobs           # GPU-specific
workers
```

## Priority Weights

Higher priority tasks are scheduled before lower priority tasks when resources are limited.

```
high_priority_task = PythonOperator(
    task_id="important_job",
    python_callable=do_important_work,
    priority_weight=10,           # default is 1
    weight_rule="absolute",      # or "downstream" (sum of
    downstream_weights)
)
```

---

## 18. DAG Versioning & Best Practices

---

### DAG File Best Practices

```
# GOOD: Fixed start_date
start_date=datetime(2026, 1, 1)
```

```
# BAD: Dynamic start_date – changes every day, causes scheduler
confusion
start_date=datetime.now() - timedelta(days=7)
```

```
# GOOD: catchup=False for most pipelines
catchup=False
```

```
# BAD: catchup=True on a newly deployed DAG with old start_date
# → Airflow tries to backfill hundreds of runs
```

```
# GOOD: Idempotent tasks – running the same task twice produces the
same result
```

```
# BAD: Tasks that append data without checking for existing records
```

### Idempotency

Tasks should be safe to re-run. Design tasks to be idempotent:

```
# BAD – appends every time the task runs
def load_to_bq():
    df.to_gbq("project.dataset.table", if_exists="append")
```

```
# GOOD – overwrites the partition, safe to re-run
def load_to_bq(**context):
    partition = context["ds_nodash"] # "20260615"
    df.to_gbq(
        f"project.dataset.table${partition}",
        if_exists="replace"
    )
```

### Avoid Heavy Logic at Import Time

```
# BAD – DB query runs every time the scheduler parses the file
(every 30s)
SITES = query_database_for_sites() # at module level
```

```
# GOOD – move to a task function or use Airflow Variables
@task()
def get_sites():
    return query_database_for_sites()
```

### Template Variables

Airflow provides Jinja template variables available in most operator parameters:

| Variable                               | Value                                                                    |
|----------------------------------------|--------------------------------------------------------------------------|
| <code>{{ ds }}</code>                  | Logical date as <code>YYYY-MM-DD</code> (e.g., <code>2026-06-15</code> ) |
| <code>{{ ds_nodash }}</code>           | Logical date as <code>YYYYMMDD</code> (e.g., <code>20260615</code> )     |
| <code>{{ ts }}</code>                  | Logical date as ISO 8601 timestamp                                       |
| <code>{{ logical_date }}</code>        | Pendulum datetime object                                                 |
| <code>{{ data_interval_start }}</code> | Start of data interval                                                   |
| <code>{{ data_interval_end }}</code>   | End of data interval                                                     |
| <code>{{ dag.dag_id }}</code>          | DAG ID                                                                   |
| <code>{{ run_id }}</code>              | DAG run ID                                                               |
| <code>{{ task.task_id }}</code>        | Current task ID                                                          |
| <code>{{ var.value.my_var }}</code>    | Airflow Variable                                                         |

|                                         |                      |
|-----------------------------------------|----------------------|
| <code>{{ conn.my_conn.host }}</code>    | Connection attribute |
| <code>{{ macros.ds_add(ds, 7) }}</code> | Date arithmetic      |

## Testing DAGs

```
# Parse DAG (check for import errors)
```

```
python dags/my_dag.py
```

```
# Test a specific task without affecting the DB
```

```
airflow tasks test my_dag_id my_task_id 2026-06-15
```

```
# Backfill (creates real DB records)
```

```
airflow dags backfill my_dag_id --start-date 2026-06-01 --end-date 2026-06-15
```

```
# Trigger manual run
```

```
airflow dags trigger my_dag_id --conf '{"key": "value"}'
```

## Unit Testing DAG Structure

```
# tests/test_my_dag.py
```

```
import pytest
```

```
from airflow.models import DagBag
```

```
def test_dag_loads_without_errors():
```

```
    dagbag = DagBag(dag_folder="dags/", include_examples=False)
```

```
    assert "iot_telemetry_pipeline" in dagbag.dags
```

```
    assert len(dagbag.import_errors) == 0
```

```
def test_dag_structure():
```

```
    dagbag = DagBag(dag_folder="dags/", include_examples=False)
```

```
    dag = dagbag.get_dag("iot_telemetry_pipeline")
```

```
    assert dag is not None
```

```
    assert dag.schedule_interval == "0 2 * * *"
```

```
    task_ids = [t.task_id for t in dag.tasks]
```

```
    assert "extract" in task_ids
```

```
    assert "transform" in task_ids
```

```
    assert "load" in task_ids
```

```
def test_task_dependencies():
```

```

dagbag = DagBag(dag_folder="dags/")
dag = dagbag.get_dag("iot_telemetry_pipeline")
extract = dag.get_task("extract")
transform = dag.get_task("transform")
assert transform.task_id in [t.task_id for t in
extract.downstream_list]

```

---

## 19. Monitoring, Alerting & SLAs

---

### Email Alerts

```

default_args = {
    "email": ["data-alerts@acme.com"],
    "email_on_failure": True,
    "email_on_retry": False,
    "email_on_success": False,
}

```

### Callbacks

```

from airflow.models import TaskInstance
from airflow.utils.state import State

def on_failure_callback(context):
    dag_id = context["dag"].dag_id
    task_id = context["task"].task_id
    logical_date = context["ds"]
    exception = context.get("exception", "Unknown")

    # Send Slack alert
    import requests
    requests.post("https://hooks.slack.com/services/...", json={
        "text": f":red_circle: *Airflow Failure*\n"
                f"DAG: `{dag_id}`\nTask: `{task_id}`\n"
                f>Date: `{logical_date}`\nError: `{exception}`"
    })

def on_success_callback(context):
    print(f"Task succeeded: {context['task'].task_id}")

with DAG(
    "monitored_pipeline",

```

```

        on_failure_callback=on_failure_callback, # DAG-level: fires on
DAG run failure
        ...
    ) as dag:
        task = PythonOperator(
            task_id="my_task",
            python_callable=do_work,
            on_failure_callback=on_failure_callback, # task-level
callback
            on_success_callback=on_success_callback,
            on_retry_callback=on_failure_callback,
        )

```

### SLAs (Service Level Agreements)

SLAs define a time by which a task must complete relative to the logical date. If a task misses its SLA, Airflow logs the miss and can trigger a callback.

```

from datetime import timedelta

```

```

with DAG(
    "sla_pipeline",
    sla_miss_callback=on_sla_miss,
    ...
) as dag:

```

```

    critical_task = PythonOperator(
        task_id="must_finish_within_2h",
        python_callable=do_work,
        sla=timedelta(hours=2), # task must complete within 2h of
logical_date
    )

```

```

def on_sla_miss(dag, task_list, blocking_task_list, slas,
blocking_tis):
    print(f"SLA missed for tasks: {task_list}")
    # send alert

```

### Metrics — StatsD / OpenMetrics

Airflow emits metrics to StatsD or OpenMetrics (Prometheus format). Key metrics to monitor:

| Metric                                              | Description                     | Alert threshold |
|-----------------------------------------------------|---------------------------------|-----------------|
| <code>scheduler.heartbeat</code>                    | Scheduler is alive              | Missing > 60s   |
| <code>dagrun.duration.success.&lt;dag_id&gt;</code> | DAG run duration on success     | > p95 baseline  |
| <code>dagrun.duration.failed.&lt;dag_id&gt;</code>  | DAG run failure                 | Any             |
| <code>task_instance.duration</code>                 | Task execution time             | > SLA           |
| <code>scheduler.tasks.starving</code>               | Tasks waiting for executor slot | > 0 for > 5 min |
| <code>pool.starving_tasks.&lt;pool&gt;</code>       | Pool-starved tasks              | > 0             |
| <code>dag_processing.total_parse_time</code>        | DAG file parse time             | > 30s           |
| <code>scheduler.critical_section_duration</code>    | Scheduler loop duration         | > 10s           |

```
# airflow.cfg
[metrics]
statsd_on = True
statsd_host = statsd-exporter
statsd_port = 8125
statsd_prefix = airflow
```

## 20. Airflow vs Alternatives

| Feature               | Airflow             | Prefect             | Dagster         | Luigi          |
|-----------------------|---------------------|---------------------|-----------------|----------------|
| <b>DAG definition</b> | Python + decorators | Python + decorators | Python + assets | Python classes |

|                              |                                               |                                    |                                       |                   |
|------------------------------|-----------------------------------------------|------------------------------------|---------------------------------------|-------------------|
| <b>UI</b>                    | Rich (built-in)                               | Rich (cloud/self-hosted)           | Rich (built-in)                       | Basic             |
| <b>Scheduling</b>            | Cron, datasets, timedelta                     | Cron, event-based                  | Cron, asset-based                     | Manual /cron      |
| <b>Dynamic tasks</b>         | Yes (mapping, Airflow 2.3+)                   | Yes (native)                       | Yes (native)                          | Limited           |
| <b>Data-aware scheduling</b> | Datasets (Airflow 2.4+)                       | Yes                                | Yes (core concept)                    | No                |
| <b>Asset-based</b>           | Partial (Datasets)                            | Partial                            | First-class                           | No                |
| <b>Kubernetes native</b>     | Yes (KubernetesExecutor)                      | Yes                                | Yes                                   | Limited           |
| <b>Observability</b>         | Moderate                                      | Good                               | Excellent                             | Minimal           |
| <b>Backfills</b>             | Built-in                                      | Manual                             | Built-in                              | Manual            |
| <b>Community/ Ecosystem</b>  | Largest                                       | Growing                            | Growing                               | Mature but small  |
| <b>Managed offering</b>      | Cloud Composer (GCP), MWAA (AWS), Astronomer  | Prefect Cloud                      | Dagster Cloud                         | None              |
| <b>Best for</b>              | Complex enterprise pipelines, large ecosystem | Python-first teams, fast iteration | Data assets & lineage, platform teams | Simple batch jobs |





# Linux

## Linux

- **sudo:** Sudo is a command in Linux allows users to run commands with privileges that only root user have.
- **grep -i "UNix" geekfile.txt:** Case insensitive search
- **grep -c "unix" geekfile.txt:** Displaying the count of number of matches
- **grep -l "unix" \* (or) grep -l "unix" f1.txt f2.txt f3.txt f4.txt:** Display the file names that matches the pattern
- **grep -w "unix" geekfile.txt:** Checking for the whole words in a file
- **head <file>** Displays the first parts of files, with the syntax, head [options] [file\_name].
- **tail <file>:** Displays the last N number of lines of the given input.
  - **tail -f /var/log/syslog:** It only shows the last part of the logs, where problems usually lie. tail will continue watching the log file and print out the next line written to the file.
  - If you do: **pip install -r /tmp/requirements.txt --ignore-installed | tee /tmp/pip\_installation.log** - It will install pip requirements packages as well as output the process in pip\_installation.log file. You can watch this log file using tail command: **tail -f pip\_installation.log**
- **cat <file\_name>:** View file content
  - Eg: **cat <file> | head -88 | tail -1:** To display the 88th line of a file
- **fdisk -l:** The fdisk command is used for disk partitioning and disk management on Unix-like systems. The -l option lists information about all available disks and their partitions. This command is typically used to view the current disk layout and partitioning scheme of the system.
- **ps -ef:** The ps command is used to display information about running processes on Unix-like systems. The -ef options stand for "everyone" (-e) and "full-format" (-f), which together instruct ps to display information about all processes in a detailed format. This includes information such as the process ID (PID), the terminal associated with the process, the user running the process, CPU and memory usage, and the command being executed.
  - **ps -ef | grep pip:** Lists down all running processes by the name pip.
- **ls -l <dir path else it lists current path files>:** Lists files and directories in the specified directory (or the current directory if none is specified) with detailed information, including permissions, ownership, size, and modification date.

- `ls -lrt`: Lists files and directories in the current directory in long format (-l) sorted by modification time in reverse order (-r for reverse, -t for time).
- `pwd`: Prints the current working directory.
- `ls -la`: Lists all files (including hidden files) and directories in the current directory in long format, showing detailed information, including hidden files (-a for all).
- `su`: Allows switching to another user account. If no username is specified, it defaults to the superuser (root) account.
- `su root`: Switches to the root user account.
- `whoami`: Displays the username of the current user.
- `who`: Lists about users who are currently logged in to the system.
- `last`: This command in Linux is used to display the history of last logged in users.
- `which python`: Displays path where executable file of python is located.
- `top`: Displays real-time information about system processes, including CPU and memory usage.
  - In your keyboard if you press button 'c' after running top command - it will display the stats in detailed manner. There are other keyboard buttons as well to press which would display other functionalities with top command.
- `set | grep AWS_KEY`: Displays environment variables (set) and filters the output to show lines containing AWS\_KEY using grep.
- `screen -S vikas`: Starts a new screen session with the name vikas.
  - `screen -ls`: Lists currently running screen sessions. screen command in Linux provides the ability to launch and use multiple shell sessions from a single ssh session.
- `find ~ -name aws_credentials`: Searches for files named aws\_credentials in the user's home directory (~).
  - `find / -name "requirements.txt" 2>/dev/null`: Searches for file from root directory. The /dev/null command suppresses error messages about directories you don't have permission to read.
  - `find . -name "requirements.txt"`: Command will search for the file starting from the current directory (.) and look through all subdirectories.
  - `find / -type d -name "efs"`: Command will search for a directory with name "efs".
- `grep -iR AWS_KEY *`: Searches recursively (-R) for occurrences of AWS\_KEY in files in the current directory and its subdirectories (\*).

- `exit`: Exits the current shell or session.
- `grep -iR access <path>`: Searches recursively for occurrences of `access` in files within the specified path.
- `echo $ec2`: Prints the value of the environment variable `ec2`.
- `pip install -U s3fs=0.4.0`: Installs or upgrades the `s3fs` Python package to version 0.4.0 using `pip`.
- `cd ./venvs`: Changes the current directory to `venvs`, which is located in the current directory (`.`).
- `df -h`: The `df` command is used to display disk space usage on Unix-like systems. The `-h` option stands for "human-readable" and formats the output in a more easily understandable way, showing sizes in kilobytes, megabytes, gigabytes, etc., rather than in raw bytes.
- `du psycopg2-2.9.3.tar.gz`: Displays the disk usage of the `psycopg2-2.9.3.tar.gz` file.
- `du -sh psycopg2-2.9.3.tar.gz`: Displays the disk usage of the `psycopg2-2.9.3.tar.gz` file in a human-readable format (`-h`) and summarizes the total (`-s`).
- `du -sh *`: Displays the disk usage of all files and directories in the current directory in a human-readable format and summarizes the total.
  - `sudo du -sh /*`: Displays disk usage for all files/directories with root user privilege.
- `du -sh * | sort -h`: Displays the disk usage of all files and directories in the current directory in a human-readable format, summarizes the total, and sorts the output numerically and in a human-readable way (`-h`).
- `conda list`: Lists installed packages and their versions using Conda, a package manager for Python.
- `echo $PATH`: It prints out the value of the `PATH` environment variable.
- `sudo -su user`: It is short for `sudo -s -u user`. The `-s` option means to run the shell specified in the environment variable `SHELL` if this has been set, or else the user's login shell. The `-u user` option means to run the command as the specified user rather than root.
- `sudo su user`: It will use `sudo` to run the command `su user` as the root user. The `su` command will then invoke the login shell of the specified username. The `su user` command could be run without the use of `sudo`, but by running it as root it will not require the password of the target user.
- `rm /path/to/directory/*`: To remove all non-hidden files\* in a directory
- `rm -r /path/to/directory/*`: To remove all non-hidden files and sub-directories (along with all of their contents) in a directory

- `rm -rf "/path/to the/directory/"*`: Force removal of files
- `history`: Provides a chronological list of previously executed commands
- `bash`: `bash` is a command interpreter, a shell, a program with an interface that interprets the commands that you put into it.
- `pstree`: To know how many sub-shells deep you are.
- `netstat`: The network statistics command is a networking tool used for troubleshooting and configuration, that can also serve as a monitoring tool for connections over the network.
- `ps aux`: Similar to `ps -ef`. Displays information about running processes.
- `ssh -i`: To `ssh` into a machine. `-i` is `identity_file` which selects a file from which the identity (private key) for public key authentication is read.
- `ping -c 1 $(hostname)`: To get IP address of the machine
- `watch`: Used to execute a program periodically, showing output in fullscreen.
  - `watch -n 1 "ps -ef | grep ssh"`: Executes `ps -ef | grep ssh` command every second and displays output.
- `kill -9 <pid>`: To kill a process/pid.
  - In Linux, an executable stored on disk is called a program, and a program loaded into memory and running is called a process. When a process is started, it is given a unique number called process ID (PID) that identifies that process to the system. If you ever need to kill a process, for example, you can refer to it by its PID.
  - In addition to a unique process ID, each process is assigned a parent process ID (PPID) that tells which process started it. The PPID is the PID of the process's parent.
  - Eg case:

```
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17600
root      17600    17599    0 Jul04 pts/2      00:00:00 ssh
ubuntu@10.80.10.5 pip install -r /tmp/requirements.txt
--ignore-installed
ubuntu    47424    42931    0 09:41 pts/3      00:00:00 grep
--color=auto 17600
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17599
root      17599    17598    0 Jul04 pts/2      00:00:06
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
root      17600    17599    0 Jul04 pts/2      00:00:00 ssh
```

```

ubuntu@10.80.10.5 pip install -r /tmp/requirements.txt
--ignore-installed
ubuntu      47422   42931   0 09:41 pts/3      00:00:00 grep
--color=auto 17599
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17598
root        17598   17597   0 Jul04 pts/2      00:00:00 /bin/sh -c
echo BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ;
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
root        17599   17598   0 Jul04 pts/2      00:00:06
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
ubuntu      47429   42931   0 09:41 pts/3      00:00:00 grep
--color=auto 17598
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17597
root        17597   17596   0 Jul04 pts/2      00:00:00 sudo -H -S -n
-u root /bin/sh -c echo
BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ; /usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
root        17598   17597   0 Jul04 pts/2      00:00:00 /bin/sh -c
echo BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ;
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
ubuntu      47467   42931   0 09:43 pts/3      00:00:00 grep
--color=auto 17597

```

- `cd /proc/<your pid>/fd:` [To get process details from its PID, we use it.](#)  
 Post `cd` into the directory we can run `ls` or `cat` command and get infos on processes. Eg: `sudo ls -l /proc/330046/fd/1`
  - Cases:

```

/proc/cpuinfo: Information about the processor, such as its type,
make, model, and performance.

```

/proc/[pid]/cmdline: This holds the complete command line for the process, unless the whole process has been swapped out, or unless the process is a zombie. In either of these later cases, there is nothing in this file: i.e. a read on this file will return 0 characters. The command line arguments appear in this file as a set of null-separated strings, with a further null byte after the last string.

/proc/[pid]/cwd: This is a link to the current working directory of the process.

/proc/[pid]/environ: This file contains the environment for the process. The entries are separated by null characters, and there may be a null character at the end.

/proc/[pid]/exe: The exe file is a symbolic link containing the actual path name of the executed command. The exe symbolic link can be dereferenced normally - attempting to open exe will open the executable. You can even type /proc/[pid]/exe to run another copy of the same process as [pid].

/proc/[pid]/fd: This is a subdirectory containing one entry for each file which the process has opened, named by its file descriptor, and which is a symbolic link to the actual file (as the exe entry does). Thus, 0 is standard input, 1 standard output, 2 standard error, etc.

/proc/[pid]/maps: A file containing the currently mapped memory regions and their access permissions.

/proc/[pid]/mem: The mem file provides a means to access the process memory pages, using open, fseek and read commands.

/proc/[pid]/root: This is a link to the root directory which is seen by the process. This root directory is usually "/", but it can be changed by the chroot command.



/proc/[pid]/stat: This file provides status information about the process. This **is** used by the Process Show utility. It **is** defined **in** fs/proc/array.c source file **and** may differ **from** one distribution to another.

/proc/devices: List of device drivers configured into the currently running kernel.

/proc/dma: Shows which DMA channels are being used at the moment. etc etc...

- `cmp <source path file> <destination path file>`: It is used to compare the two files byte by byte and helps you to find out whether the two files are identical or not.
- `cp <source path file> <destination path file>`: Command creates a copy of the source\_file at the specified destination.
- `sh test_file.sh` : To run a shell script. sh is the bourne shell. There are several shells, of which bourne is the old standard, installed on all unix systems, and generally the one you can guarantee will exist. The shell is the command interpreter that takes your input, provides output back to the screen, to the correct files, etc, and provides all the basic built-in commands you need to manage jobs, kill, test expressions, etc. Your command above is saying to run that shell-script using the bourne shell. Different shells use different syntax, so using the correct shell is a requirement. The first line of the shell should also define which to use: `#!/bin/sh` says use `/bin/sh`.
  - The `-c` option in the sh command is used to execute a command or a series of commands provided as an argument to the `-c` option. Eg: `sh -c "command1; command2; command3"`
- `chmod a+x test_file.sh`: The `chmod a+x` command in Linux adds the execute permission to a file for all users (owner, group, and others). All [chmod commands](#) [al](#).
  - [File permissions control](#) [al](#) which actions can be performed by which users. Read, Write, and Execute are the three actions possible for every file. Three important commands you'll use when managing file permissions: `chmod` (Change mode), `chown` (Change ownership), `chgrp` (Change group).
  - Users are classified under three broad categories: Normal users, Groups, and Others. Linux allows users to set permissions at a very

granular level. You can secure your file or directory in every possible location of a file system.

- `chmod` is a command that lets you change the permissions of a file or directory to all types of users.
  - user level permissions: `u` – Grant permission to a user; `g` – Grant permission to a group (A Group of users); `o` – Grant permission to others
  - file level permissions: `r` – Grants read permission; `w` – Grant write permission; `x` – Grant execute permission
- We can execute `.sh` files by just running them like this, eg: `./install.sh`. But it is possible file may not execute, for such case, we need to change the relevant permissions, eg: `chmod +x install.sh` and then run. To remove permissions, one could run `chmod -x install.sh`. Similarly, say to remove read permissions: `chmod -r install.sh`.
- `sudo dmesg`: All the messages received from the kernel ring buffer is displayed when we execute the command “`dmesg`”, here only the latest messages will be shown.
  - The kernel ring buffer can be thought of as a log file, but for the kernel itself. However, unlike other log files, it's stored in memory rather than in a disk file.
- `sudo apt-get install iftop`: `apt-get` (Advanced Packaging Tool) is a command-line tool that helps in handling packages in Linux.
  - Similarly to `uninstall`:
    - `apt-get remove iftop`: will remove the binaries, but not the configuration or data files of the package `packagename`. It will also leave dependencies installed with it on installation time untouched.
    - `apt-get purge iftop`: will remove about everything regarding the package `packagename`, but not the dependencies installed with it on installation. Both commands are equivalent. Particularly useful when you want to 'start all over' with an application because you messed up the configuration. However, it does not remove configuration or data files residing in users home directories, usually in hidden folders there. There is no easy way to get those removed as well.
    - `apt-get autoremove`: removes orphaned packages, i.e. installed packages that used to be installed as an dependency,

but aren't any longer. Use this after removing a package which had installed dependencies you're no longer interested in.

- `aptitude remove iftop`: will also attempt to remove other packages which were required by packagename on but are not required by any remaining packages. Note that aptitude only remembers dependency information for packages that it has installed.
- [etc. etc... al](#)

- `clear`: Clear the terminal screen in Linux
- `date`: Displays date in UTC in linux
- `scp <file/host> <file/host>`: scp (secure copy) command in Linux system is used to copy file(s) between servers in a secure way. The SCP command or secure copy allows the secure transferring of files between the local host and the remote host or between two remote hosts. It uses the same authentication and security as it is used in the Secure Shell (SSH) protocol. SCP is known for its simplicity, security, and pre-installed availability.
  - Eg: `sp -r circleci@13.120.15.80:/home/repo/file/values.yml migrated_values.vml`
  - `scp user@remotehost:/home/user/file_name`: Securely copy a file from remote machine to our local machine
  - `scp [file_name] remoteuser@remotehost:/remote/directory`: Securely copy a file from a local machine to a remote machine
- `cat /etc/passwd`: To list all users in a Linux system you can read the file (as seen). Each line in this file represents a user account, and the first field in each line is the username.
- `cat /etc/group`: The `/etc/group` file is a configuration file on Unix-like systems that stores the group information for the system. It contains a list of groups, each with a unique group name and a list of users who are members of that group.
- `cat /etc/sudoers`: The `/etc/sudoers` file is a configuration file on Unix-like systems that specifies which users or groups are allowed to run commands with superuser (root) privileges using the `sudo` command.
- `tcpdump`: It is used to capture, filter, and analyze network traffic such as TCP/IP packets going through your system.
- In terminal, if you type a long command, and want to move the cursor to first alphabet/position, press: `Ctrl + a` in keyboard.

- In terminal, if you want to retrace and use a command previously/historically used, click Ctrl + r in keyboard and search the command in terminal, for further historical instance, click Ctrl + r again and again.
- `cat date_2024_08_04_data.csv | wc -l`: 'wc' stands for word count. In given command we count the no. of lines in the file.
- Run `gunzip` zip/unzip command(s) and tar file commands:
  - `gunzip originalGeoIP2-City.tar.gz` (unzip file) -> `tar -xvf originalGeoIP2-City.tar` (decompress tar file) -> `tar -cvf gunzipTarGeoIP2-City.tar GeoIP2-City_20230512/` (compress folder to tar file) -> `gzip gunzipTarGeoIP2-City.tar` (zip back)
- `md5sum file.yml`: The MD5 sum of a file is a 32-character hexadecimal number that verifies a file's integrity. It's commonly used to check if a file has changed due to a disk error, faulty transfer, or other reasons.
- `aws cloudwatch describe-alarms --query "MetricAlarms[][AlarmName, AlarmActions]" | jq -r '.[1] | select(.[1] != null) | "\(.[])"'`: The command you provided is a combination of AWS CLI and the jq tool, which is used for processing JSON data. The entire command retrieves a list of CloudWatch alarms, specifically their names and actions. It then filters this list to include only those alarms that have defined actions (i.e., their action field is not null) and outputs each alarm name and its associated actions as plain text strings. The pipe symbol takes the output from the previous command and feeds it into jq, just note.

---

#### Vim commands:

- Exiting Vim
  - `:q!`: Exit Vim terminal without saving
  - `:wq!`: Save and exit Vim terminal
- Editing Files
  - `:%d`: Clear all contents of a file
  - `:set nu`: Add line numbers to the file
- Navigating Files
  - `gg`: Move to the first line of the file
  - `G`: Move to the last line of the file
  - `gg=G`: Reindent the whole file
  - `gv`: Reselect the last visual selection
  - `^`: Move to the first non-blank character of the line
  - `g_`: Move to the last non-blank character of the line
  - `gf`: Jump to the file name under the cursor

- Manipulating Text
  - xp: Swap characters forward
  - Xp: Swap characters backward
  - yyp: Duplicate the current line
  - yapP: Duplicate the current paragraph
  - dat: Delete around an HTML tag, including the tag
  - dit: Delete inside an HTML tag, excluding the tag
  - w: Move one word to the right
  - b: Move one word to the left
  - dd: Delete the current line
  - <<: Outdent the current line
  - >>: Indent the current line
  - ~: Toggle the case of the current character
  - gUw: Uppercase until the end of the word
  - gUw: Uppercase the entire word
  - gUU: Uppercase the entire line
  - gu\$: Lowercase until the end of the line
  - da": Delete the next double-quoted string
  - +: Move to the first non-whitespace character of the next line
  - S: Delete the current line and go into insert mode
  - I: Insert at the beginning of the line
  - ci": Change what's inside the next double-quoted string
  - ca{: Change inside the curly braces (try [, (, etc.)
  - vaw: Visually select a word
  - dap: Delete the whole paragraph
  - r: Replace a character
- Navigating Changes
  - [: Jump to the beginning of the last yanked text
  - ]: Jump to the end of the last yanked text
  - g;: Jump to the last change you made
  - g,: Jump back forward through the change list
- Repeating Commands
  - &: Repeat the last substitution on the current line
  - g&: Repeat the last substitution on all lines
- Additional Commands
  - :%s/old/new/g: Replace all occurrences of "old" with "new" in the file
  - u: Undo the last action

- Ctrl+r: Redo the last undone action
  - %: Move to the matching parenthesis, bracket, or brace
  - \*: Search forward for the word under the cursor
  - #: Search backward for the word under the cursor
  - f{char}: Move forward to the next occurrence of {char}
  - F{char}: Move backward to the previous occurrence of {char}
  - t{char}: Move forward until before the next occurrence of {char}
  - T{char}: Move backward until before the previous occurrence of {char}
  - /{pattern}: Search forward for the {pattern}
  - ?{pattern}: Search backward for the {pattern}
  - n: Repeat the last search in the same direction
  - N: Repeat the last search in the opposite direction
  - Shift + v: Visual editor mode
- 

#### Misc Notes:

- /dev/null is a like a virtual device, which has a special property: Any data written to /dev/null vanishes or disappears. Because of this characteristic, it is also called bitbucket or blackhole.
  - Eg: echo "helloworld" > /dev/null; Then if you do: cat /dev/null; You won't get any output.
  - It is mainly used to discard standard output and standard error from an output.
- [Linux Directory Structure](#)
  - / – The root directory: Everything, all the files and directories, in Linux are located under 'root' represented by '/'.
    - /dev/null: can be sent to destroy any file or string
    - /dev/zero: contains an infinite sequence of 0
    - /dev/random: contains an infinite sequence of random values
  - /bin – Binaries: The '/bin' directly contains the executable files of many basic shell commands like ls, cp, cd etc. Mostly the programs are in binary format here and accessible by all the users in the Linux system.
  - /dev – Device files: This directory only contains special files, including those relating to the devices. These are virtual files, not physically on the disk. Some interesting examples of these files are:
    - /dev/null: can be sent to destroy any file or string
    - /dev/zero: contains an infinite sequence of 0
    - /dev/random: contains an infinite sequence of random values
  - /etc – Configuration files: The /etc directory contains the core configuration files of the system, use primarily by the administrator and services, such as the password file and networking files. If you need to

make changes in system configuration (for example, changing the hostname), the `etc` folder is where you'll find the respective files.

- `/usr` – User binaries and program data: In `'usr'` go all the executable files, libraries, source of most of the system programs. For this reason, most of the files contained therein is readonly (for the normal user)
- `/home` – User personal data: Home directory contains personal directories for the users. The home directory contains the user data and user-specific configuration files. As a user, you'll put your personal files, notes, programs etc in your home directory. When you create a user on your Linux system, it's a general practice to create a home directory for the user. Suppose your Linux system has two users, Alice and Bob. They'll have a home directory of their own at locations `/home/alice` and `/home/bob`. Do note that Bob won't have access to `/home/alice` and vice versa.
- `/lib` – Shared libraries: Libraries are basically codes that can be used by the executable binaries. The `/lib` directory holds the libraries needed by the binaries in `/bin` and `/sbin` directories. Libraries needed by the binaries in the `/usr/bin` and `/usr/sbin` are located in the directory `/usr/lib`.
- `/sbin` – System binaries: This is similar to the `/bin` directory. The only difference is that it contains the binaries that can only be run by root or a sudo user. You can think of the 's' in 'sbin' as super or sudo.
- `/tmp` – Temporary files: As the name suggests, this directory holds temporary files. Many applications use this directory to store temporary files. Even you can use directory to store temporary files. But do note that the contents of the `/tmp` directories are deleted when your system restarts. Some Linux system also delete files old files automatically so don't store anything important here.
- `/var` – Variable data files: Var, short for variable, is where programs store runtime information like system logging, user tracking, caches, and other files that system programs create and manage. The files stored here are NOT cleaned automatically and hence it provides a good place for system administrators to look for information about their system behavior. For example, if you want to check the login history in your Linux system, just check the content of the file in `/var/log/wtmp`.
- `/boot` – Boot files: The `'boot'` directory contains the files of the kernel and boot image, in addition to LILO and Grub. It is often advisable that the directory resides in a partition at the beginning of the disc.
- `/proc` – Process and kernel files: The `'proc'` directory contains the information about currently running processes and kernel parameters.



The content of the `proc` directory is used by a number of tools to get runtime system information. For example, if you want to check processor information in Linux, you can simply refer to the file `/proc/cpuinfo`. You want to check memory usage of your Linux system, just look at the content of `/proc/meminfo` file.

- `/opt` – Optional software: Traditionally, the `/opt` directory is used for installing/storing the files of third-party applications that are not available from the distribution's repository. The normal practice is to keep the software code in `opt` and then link the binary file in the `/bin` directory so that all the users can run it.
- `/root` – The home directory of the root: There is `/root` directory as well and it works as the home directory of the root user. So instead of `/home/root`, the home of root is located at `/root`. Do not confuse it with the root directory (`/`).
- `/media` – Mount point for removable media: When you connect a removable media such as USB disk, SD card or DVD, a directory is automatically created under the `/media` directory for them. You can access the content of the removable media from this directory.
- `/mnt` – Mount directory: This is similar to the `/media` directory but instead of automatically mounting the removable media, `mnt` is used by system administrators to manually mount a filesystem.
- `/srv` – Service data: The `/srv` directory contains data for services provided by the system. For example, if you run a HTTP server, it's a good practice to store the website data in the `/srv` directory.

---

Other ref: <https://github.com/chassing/linux-sysadmin-interview-questions>



# FastAPI-SpringBoot

# FastAPI & Spring Boot Notes

## FastAPI Notes

### Introduction

FastAPI is a modern high-performance Python framework used for building APIs. It is built on top of Starlette for the web layer and Pydantic for validation and serialization. FastAPI is designed for high concurrency, async support, automatic validation, and automatic API documentation generation.

Common use cases include microservices, AI/ML APIs, realtime systems, and backend APIs.

---

### Architecture

Typical request flow:

Client



Load Balancer



Uvicorn / Gunicorn Workers



ASGI Server



Middleware



Routing



Dependency Injection



Validation



Business Logic



Database / External Services



Serialization

↓

JSON Response

---

## ASGI

ASGI (Asynchronous Server Gateway Interface) is the asynchronous successor to WSGI. Unlike WSGI, ASGI supports non-blocking request handling, WebSockets, and streaming.

WSGI frameworks such as Flask follow a synchronous thread/process-based model, whereas ASGI frameworks like FastAPI use an event-loop-based concurrency model that scales efficiently for I/O-heavy workloads.

---

## Async Programming

FastAPI heavily relies on Python's `async` and `await` syntax for asynchronous programming.

```
@app.get("/")
async def get_users():
    users = await db.fetch()
    return users
```

When a coroutine waits for I/O, control is returned to the event loop, which schedules another coroutine. This allows a single thread to efficiently handle many concurrent requests.

Async programming is highly beneficial for:

- Database calls
- External API calls
- Network I/O

It is not useful for CPU-intensive tasks such as image processing or heavy computations because of Python's GIL.

---

## Event Loop

The event loop is responsible for scheduling and managing coroutines. When a coroutine pauses during I/O, the event loop switches execution to another task instead of blocking the thread.

Execution flow:

Request arrives

↓

Coroutine starts

↓

Coroutine waits on I/O

↓

Event loop schedules another coroutine

↓

Original coroutine resumes after I/O completion

This architecture significantly improves concurrency while reducing thread overhead.

---

## Dependency Injection

FastAPI provides lightweight dependency injection using `Depends()`.

```
from fastapi import Depends

def get_db():
    return Session()

@app.get("/")
def users(db=Depends(get_db)):
    pass
```

Dependency injection improves modularity, loose coupling, and testability. Common use cases include database sessions, authentication, shared services, Redis clients, and configuration management.

---

## Pydantic

Pydantic is used for request validation, parsing, serialization, and schema generation.

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
```

Pydantic automatically validates incoming payloads based on Python type hints. Invalid payloads return HTTP 422 responses automatically. Pydantic v2 internally

uses a Rust-based validation engine, making serialization and validation extremely fast.

---

## Middleware

Middleware executes before and after request processing. It is commonly used for authentication, logging, metrics collection, tracing, CORS handling, and rate limiting.

```
@app.middleware("http")
async def middleware(request, call_next):
    response = await call_next(request)
    return response
```

---

## Database Integration

FastAPI commonly uses SQLAlchemy with PostgreSQL or MySQL. Async database access is typically implemented using SQLAlchemy async engines. Connection pooling is important because creating database connections is expensive. Proper session handling prevents connection leaks and pool exhaustion.

---

## Scaling and Performance

FastAPI scales well because of its async non-blocking architecture. Common production scaling strategies include:

- Multiple worker processes
- Horizontal scaling
- Stateless services
- Redis caching
- Load balancing

A common mistake is performing blocking synchronous I/O inside async endpoints, which blocks the event loop and reduces concurrency benefits.

---

## Spring Boot Notes

### Introduction

Spring is a modular Java framework providing dependency injection, IoC container management, transaction management, AOP, and web framework

support. Spring Boot simplifies Spring development through auto-configuration, embedded servers, starter dependencies, and production-ready tooling.

---

## **Architecture**

Typical request flow:

Client



Tomcat



DispatcherServlet



Controller



Service



Repository



Hibernate/JPA



Database



Response

---

## **IoC and Dependency Injection**

Inversion of Control (IoC) means the Spring container manages object creation and lifecycle instead of the application manually creating dependencies.

Dependency Injection (DI) is the mechanism used to implement IoC.

Dependencies are injected into classes rather than instantiated directly.

Constructor injection is the preferred approach because it improves immutability and testability.

---

## **Bean Lifecycle**

A bean is an object managed by the Spring container.

Lifecycle flow:

Bean Instantiation



Dependency Injection



@PostConstruct



Bean Ready



Bean Usage



@PreDestroy



Bean Destroyed

Singleton is the default bean scope. However, singleton beans are not automatically thread-safe because multiple threads can access the same instance concurrently.

Best practice is to keep singleton services stateless.

---

## **Spring Boot Startup Flow**

Application startup flow:

main()



SpringApplication.run()



Create ApplicationContext



Component Scan



Auto Configuration



Bean Creation



Embedded Server Startup



Application Ready

Spring Boot auto-configuration works using `@Conditional` annotations and classpath detection.

Priority order:

Explicit user bean > Auto-configured bean

---

## **DispatcherServlet**

DispatcherServlet is the front controller of Spring MVC. It receives incoming requests, determines the correct controller, invokes business logic, and constructs the response.

Flow:

Request

↓

DispatcherServlet

↓

Handler Mapping

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Response

---

## **AOP and Proxies**

Spring uses Aspect-Oriented Programming (AOP) for transactions, logging, security, metrics, and other cross-cutting concerns.

Internally, Spring commonly uses:

- JDK Dynamic Proxies
- CGLIB Proxies

The proxy intercepts method calls and applies additional behavior such as transaction handling.

---



## Transaction Management

`@Transactional` enables declarative transaction management.

Execution flow:

Method Call

↓

Spring Proxy Intercepts

↓

Transaction Starts

↓

Business Logic Executes

↓

Commit / Rollback

An important interview topic is the self-invocation problem. Internal method calls bypass the Spring proxy, causing transactional annotations to not work in certain scenarios.

---

## Connection Pooling

Spring Boot uses HikariCP as the default connection pool.

Connection pooling:

- Reuses database connections
- Reduces latency
- Improves scalability
- Prevents expensive connection creation overhead

Improper pool sizing can cause throughput bottlenecks under high traffic.

---

## Spring MVC vs WebFlux

Spring MVC follows a blocking thread-per-request model. Each request occupies a worker thread until completion.

WebFlux is Spring's reactive non-blocking framework built on Reactor using `Mono` and `Flux`. It uses an event-loop-based concurrency model similar to FastAPI's async architecture.

WebFlux is useful for highly concurrent I/O-heavy workloads.

---

## Hibernate and JPA

JPA is a specification for ORM, while Hibernate is the most common JPA implementation.

Hibernate responsibilities include:

- SQL generation
- Entity mapping
- Dirty checking
- Caching

A major interview topic is the N+1 query problem, where one parent query triggers many additional child queries. Solutions include:

- JOIN FETCH
- EntityGraph
- Batch fetching

LAZY fetching is generally preferred over EAGER fetching to avoid unnecessary database queries.

---

## Caching

Spring supports caching using annotations such as:

- `@Cacheable`
- `@CacheEvict`
- `@CachePut`

Common providers include Redis, Caffeine, and EhCache.

Caching reduces database load and improves latency significantly.

---

## Messaging and Async Processing

Spring integrates well with Kafka and RabbitMQ for event-driven architectures.

Common annotations:

- `@KafkaListener`
- `@RabbitListener`
- `@Async`

Async processing is commonly used for background tasks, notifications, and decoupled communication between services.

---

## Production Concerns

Important production topics: Structured logging, Metrics collection, Distributed tracing, Correlation IDs, Monitoring, Horizontal scaling, Circuit breakers, Retry mechanisms.

Common tools: Micrometer, Prometheus, Grafana, OpenTelemetry, Resilience4j.

---

# InfluxDB

(Note: Used default font, to preserve text style | Refactored with GPT)

---

## Table of Contents

1. What Is InfluxDB & How It Works
2. InfluxDB vs Other Time Series Databases
3. Versions & Product Lineup (1.x → 2.x → 3.x)
4. Data Model
5. Query Languages: InfluxQL, Flux, SQL
6. Writes (Line Protocol & Write APIs)
7. Reads (Query APIs)
8. Tasks & the Processing Engine
9. Telegraf
10. InfluxDB on Kubernetes
11. High Availability & Clustering
12. Security & Authentication
13. Monitoring InfluxDB Itself
14. Backup & Restore

---

## 1. What Is InfluxDB & How It Works

InfluxDB is a purpose-built **time series database (TSDB)** created by InfluxData. A time series database is optimized for data points indexed by time — metrics, sensor/IoT readings, application telemetry, financial ticks, logs — where the dominant access patterns are:

- **High-volume, append-mostly writes** (rarely updates/deletes)
- **Time-bounded range queries** ("last 5 minutes", "this month")
- **Downsampling / aggregation over time windows** (avg, sum, rate, percentile)
- **Automatic expiration of old data** (retention policies)

### 1.1 Two generations, two storage engines

InfluxDB has shipped with **two fundamentally different storage engines**, which is the single most important thing to understand before designing anything on it.

#### Generation 1 — TSM (InfluxDB 1.x and 2.x)

- Engine: **TSM (Time-Structured Merge Tree)** — a custom LSM-tree-like engine written in Go.
- Writes land in a Write-Ahead Log (WAL) + in-memory cache, then get compacted into immutable, sorted, compressed TSM files on disk (similar lineage to LevelDB/Cassandra's SSTables).

- Data is organized into **shards**, grouped into **shard groups** by time range, each governed by a **retention policy (1.x)** or **bucket (2.x)**.
- An in-memory **series index (TSI)** maps each unique time series (measurement + tag set) to its data. This index is the engine's Achilles' heel: every unique tag combination is a new "series," and high **series cardinality** (millions of unique tag combinations) degrades performance and can exhaust memory.
- Single-node only in open source; clustering/HA required the commercial **InfluxDB Enterprise** product (1.x) — see §11.

## Generation 2 — InfluxDB 3.x ("IOx" engine)

- Rewritten from scratch in **Rust**, built on the open-source **Apache Arrow** ecosystem: Arrow (in-memory columnar format), **DataFusion** (query engine/optimizer), **Parquet** (columnar on-disk file format), and **Arrow Flight** (high-performance gRPC data transport, including Flight SQL).
- **Columnar storage** instead of the older log-structured engine — much better for analytical scans and compression.
- **Diskless / object-storage-backed architecture**: durable data lives as Parquet files in object storage (S3, GCS, Azure Blob, or local disk), decoupling compute from storage. This is what allows independent, horizontal scaling of ingest, query, and compaction.
- **Effectively unlimited series cardinality** — the cardinality bottleneck that plagued the TSM engine is designed away by indexing differently (no global in-memory series index that grows unboundedly with cardinality).
- Native **SQL** and **InfluxQL** support via DataFusion; a built-in **Processing Engine** (embedded Python) replaces/extends what Flux Tasks did in 2.x.

### 1.2 Conceptual write/read path (3.x)

Client (Telegraf / app / curl)

| line protocol over HTTP



Ingest node (WAL + in-memory buffer)

| periodic snapshot



Parquet files in Object Storage (S3 / GCS / Azure / local)



Compactor (merges small Parquet files, builds catalog/index)



Query node (DataFusion executes SQL / InfluxQL over Parquet + Arrow)



Client (Grafana, SQL client, Flight SQL driver, HTTP API)

Because storage is the object store itself, any number of stateless query or ingest nodes can attach to the same data — this is the basis of InfluxDB 3

Enterprise/Clustered HA (§11).

### 1.3 Why "time series" needs a specialized engine

A generic RDBMS *can* store time-stamped rows, but a TSDB earns its keep through:

- **Compression:** Time and numeric deltas compress extremely well (Gorilla-style/columnar compression) — 10–100x smaller than naive row storage.
- **Time-bucketed retention/downsampling:** Cheaply drop or roll up old data without expensive `DELETE` scans.
- **Cardinality-aware indexing:** Indexing strategies tuned for "tag = value" lookups across time, not arbitrary joins.
- **Purpose-built query semantics:** Built-in functions like `derivative()`, `rate()`, `moving_average()`, time-windowed `GROUP BY time()`.

---

## 2. InfluxDB vs Other Time Series Databases

| Database                      | Storage model                       | Query language                   | Cardinality handling                              | Clustering / HA                                    | Best fit                                          |
|-------------------------------|-------------------------------------|----------------------------------|---------------------------------------------------|----------------------------------------------------|---------------------------------------------------|
| <b>InfluxDB 1.x/2.x (TSM)</b> | LSM-style, row-oriented per series  | InfluxQL, Flux                   | Degrades at high cardinality (millions of series) | OSS = single node; HA via paid Enterprise (legacy) | General metrics/IoT, existing Flux/InfluxQL shops |
| <b>InfluxDB 3.x (IOx)</b>     | Columnar, Parquet on object storage | SQL, InfluxQL (Flux unsupported) | Designed for unlimited                            | Native multi-node (Enterprise/Clustered/Cloud)     | New deployments, high-cardinality                 |

|                        |                                                                 |                                                       |                                                                   |                                                                        |                                                              |
|------------------------|-----------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------|
|                        |                                                                 |                                                       | cardinality                                                       |                                                                        | workloads, analytics-heavy use cases                         |
| <b>Prometheus</b>      | Custom TSDB, local disk, pull-based scraping                    | PromQL                                                | Cardinality-sensitive (similar issue to TSM)                      | No built-in clustering; Thanos/Cortex/Mimir add HA & long-term storage | Kubernetes/infra monitoring, alerting (via Alertmanager)     |
| <b>TimescaleDB</b>     | PostgreSQL extension; hypertables (chunked, partitioned tables) | Full SQL (PostgreSQL)                                 | Handles high cardinality well via normal indexes/joins            | Native PostgreSQL replication; TimescaleDB Cloud for multi-node        | Teams wanting relational joins + SQL + time series in one DB |
| <b>VictoriaMetrics</b> | Custom columnar/LSM, Prometheus-compatible                      | PromQL / MetricsQL                                    | Very good — built specifically to fix Prometheus cardinality cost | Native clustering (vmstorage/vminsert/vmselect)                        | Drop-in Prometheus remote-storage replacement at scale       |
| <b>QuestDB</b>         | Columnar, append-only, custom engine (Java/C++)                 | SQL (with time-series extensions like <b>SAMPLE</b> ) | Good                                                              | Limited native clustering (enterprise feature)                         | Very high ingest rate analytics, financial tick data         |



|                                 |                                   |                                 |                                            |                                                         |                                                               |
|---------------------------------|-----------------------------------|---------------------------------|--------------------------------------------|---------------------------------------------------------|---------------------------------------------------------------|
|                                 |                                   | BY,<br>LATEST<br>ON)            |                                            |                                                         |                                                               |
| <b>OpenTSDB</b>                 | Built on HBase/Big table          | Custom JSON/HTTP query API      | Good (inherits HBase's distributed model)  | Inherits HBase clustering (heavy operational footprint) | Large Hadoop/HBase shops already running that stack           |
| <b>Graphite (whisper/ceres)</b> | Fixed-size round-robin files      | Graphite functions / render API | Poor at high cardinality (file-per-series) | Carbon-relay sharding, manual                           | Legacy ops monitoring, simple dashboards                      |
| <b>ClickHouse</b>               | Columnar OLAP (not TSDB-specific) | SQL                             | Excellent                                  | Native distributed/sharded clustering                   | Teams that want a general columnar warehouse doubling as TSDB |

### Key decision points

- **Choose InfluxDB 3.x over Prometheus** when you need long-term retention, high-cardinality tags (e.g., per-request trace IDs), SQL access, or you're not purely doing pull-based infra metrics.
- **Choose Prometheus** when you're deep in the Kubernetes ecosystem and want pull-based scraping, native **ServiceMonitor** CRDs, and tight Alertmanager integration — pair with VictoriaMetrics/Mimir/Thanos for retention & HA.
- **Choose TimescaleDB** when your team already lives in PostgreSQL and wants relational joins alongside time series (e.g., joining sensor readings to a **devices** table).

- **Choose InfluxDB 1.x/2.x** mainly if you're already invested in it, need Flux/Kapacitor-based tooling, or have moderate cardinality needs — but for new projects, InfluxData itself now steers new workloads to InfluxDB 3.x.
- **Choose VictoriaMetrics** for a cost-efficient, drop-in replacement for Prometheus long-term storage with native clustering and lower resource use.

### 3. Versions & Product Lineup

InfluxDB's versioning is genuinely confusing because "1, 2, 3" refers to **major engine generations**, not simple sequential releases, and each generation spawned several product editions (OSS, Enterprise, Cloud variants). As of mid-2026:

#### 3.1 Generation summary

| Generation          | Engine                              | Status (2026)                                     | Query languages                                       |
|---------------------|-------------------------------------|---------------------------------------------------|-------------------------------------------------------|
| <b>InfluxDB 1.x</b> | TSM                                 | Maintenance mode                                  | InfluxQL                                              |
| <b>InfluxDB 2.x</b> | TSM                                 | Maintenance mode                                  | Flux (primary), InfluxQL (limited)                    |
| <b>InfluxDB 3.x</b> | IOx (Rust/Arrow/Parquet/DataFusion) | Active development, recommended for new workloads | SQL (primary), InfluxQL; <b>Flux is not supported</b> |

**Flux's status:** InfluxData has placed Flux into maintenance mode. It is not officially supported on any InfluxDB 3 product (it may technically still run against InfluxDB Cloud Serverless but isn't optimized or guaranteed). If you're building anything new, plan to use SQL or InfluxQL instead of Flux.

#### 3.2 InfluxDB 3.x product editions

| Edition                | Deployment                              | Multi-node / HA  | Notes                                                                                                                  |
|------------------------|-----------------------------------------|------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>InfluxDB 3 Core</b> | Self-hosted, single binary, single node | No (single node) | Free, open source (MIT/Apache-2.0). Great for edge devices (runs on a Raspberry Pi), dev/test, non-critical workloads. |

|                                  |                                                                                         |                                                                      |                                                                                                                                                                                                |
|----------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>InfluxDB 3 Enterprise</b>     | Self-hosted (bare metal, VM, Kubernetes), single binary                                 | <b>Yes</b> — multi-node clusters, read replicas, dedicated compactor | Superset of Core. Adds HA, long-range historical query performance, advanced security, license required (free "Home" license available, paid commercial license for production).               |
| <b>InfluxDB Clustered</b>        | Kubernetes-native , InfluxData-managed Helm/operator pattern, self-hosted on your infra | <b>Yes</b>                                                           | The HA InfluxDB 3 cluster you run yourself on Kubernetes; decoupled, independently scalable ingest/query/compactor tiers.                                                                      |
| <b>InfluxDB Cloud Dedicated</b>  | Fully managed by InfluxData                                                             | <b>Yes</b>                                                           | Single-tenant managed cluster in InfluxData's cloud.                                                                                                                                           |
| <b>InfluxDB Cloud Serverless</b> | Fully managed, multi-tenant                                                             | Yes (managed)                                                        | Pay-as-you-go; v3 storage engine but a different/limited API surface (no native v3 write API — uses v1/v2-compatible write endpoints). Replaces legacy "InfluxDB Cloud (TSM)" for new signups. |

### 3.3 Legacy / maintenance products

| Product                     | What it was                                                                                                                                            |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>InfluxDB Cloud (TSM)</b> | Legacy managed v2.x cloud service. No longer accepting new signups; existing customers steered to migrate to InfluxDB 3.                               |
| <b>InfluxDB Cloud 1</b>     | Legacy managed v1.x (InfluxDB Enterprise-based) DBaaS on AWS with full TICK stack (Telegraf, InfluxDB, Chronograf, Kapacitor) support. No new signups. |

|                                  |                                                                                                                                                                           |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>InfluxDB Enterprise (1.x)</b> | Commercial clustered version of 1.x with separate <b>meta nodes</b> (Raft consensus) and <b>data nodes</b> . Now effectively legacy, superseded by InfluxDB 3 Enterprise. |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3.4 Which one should I pick today?

- **New project, single node / edge / dev: InfluxDB 3 Core**
- **New project, production, need HA: InfluxDB 3 Enterprise** (self-managed) or **InfluxDB Cloud Dedicated/Serverless** (managed)
- **Kubernetes-native self-hosted HA: InfluxDB Clustered**
- **Already on 1.x/2.x and stable, low urgency:** stay, but plan a migration path to v3 since 1.x/2.x get no new features (only maintenance fixes), and Flux has no future investment
- **Heavy existing Flux/Kapacitor tooling you can't easily port:** stay on 2.x longer, but budget time to migrate queries to SQL/InfluxQL eventually

### 3.5 Migration note

All InfluxDB 3 products expose **v1 (/write, /query)** and **v2 (/api/v2/write)** **compatibility endpoints**, plus authentication compatible with tokens or username/password — so existing Telegraf configs and client libraries usually keep working with minimal changes during a phased migration. v1 databases/retention policies map to v3 "databases"; v2 buckets map similarly.

---

## 4. Data Model

### 4.1 InfluxDB 1.x / 2.x (TSM) terminology

| Concept                                                 | Meaning                                                                                              |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Organization (org)</b>                               | (2.x only) A workspace/tenant boundary containing buckets, tasks, dashboards, tokens.                |
| <b>Bucket (2.x) / Database + Retention Policy (1.x)</b> | Named container for data with a defined retention period.                                            |
| <b>Measurement</b>                                      | Roughly analogous to a SQL "table name" — a logical grouping, e.g. <b>cpu</b> , <b>temperature</b> . |

|                            |                                                                                                                                                                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tag set</b>             | Indexed, string-only key/value metadata, e.g. <code>host=server01,region=us-west</code> . Tags are indexed — good for filtering/grouping, bad for high-cardinality unbounded values (e.g., never tag by raw UUID-per-event unless you accept the cardinality cost). |
| <b>Field set</b>           | The actual measured values (int, float, string, boolean) — <i>not</i> indexed by default. This is your data, e.g. <code>usage_idle=98.2,usage_user=1.1</code> .                                                                                                     |
| <b>Timestamp</b>           | Nanosecond-precision time associated with the point.                                                                                                                                                                                                                |
| <b>Point</b>               | One measurement + tag set + field set + timestamp = one row written via line protocol.                                                                                                                                                                              |
| <b>Series</b>              | The unique combination of measurement + tag set (excluding fields/time). Cardinality = number of unique series.                                                                                                                                                     |
| <b>Shard / Shard group</b> | Internal physical storage unit; shard groups bucket data by time range per retention policy.                                                                                                                                                                        |

#### Example line protocol point:

weather,location=us-midwest,sensor=A temperature=82.0,humidity=54

1717689600000000000

└─measurement─┐ └─tag set─┐ └─field set─┐  
└─timestamp (ns)─┐

## 4.2 InfluxDB 3.x terminology

| Concept         | Meaning                                                                                                                                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Database</b> | Top-level namespace (replaces bucket/db+RP). Equivalent to a v2 bucket or v1 db/retention-policy pair.                                                                                                                         |
| <b>Table</b>    | Equivalent to a v1/v2 "measurement". Contains columns.                                                                                                                                                                         |
| <b>Columns</b>  | Each table has typed columns: one <b>time</b> column, one or more <b>tag columns</b> (string, used for the primary key / sort order), and <b>field columns</b> (the measured values: float64, int64, uint64, string, boolean). |

|                        |                                                                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Primary key</b>     | The ordered set of tag columns + time that uniquely identifies a row and determines Parquet file sort order. Tag columns are immutable once set for a table.            |
| <b>Schema-on-write</b> | Still schema-on-write: the first write to a table establishes/extends its schema. Unlike TSM, growing cardinality on tag values does not blow up a global series index. |

### 4.3 Schema design best practices (applies broadly)

1. **Use tags for things you filter/group by** (host, region, device\_id) — keep tag *value* cardinality reasonable in TSM-era (1.x/2.x); v3 relaxes this constraint significantly.
2. **Use fields for the actual measurements** (temperature, cpu\_usage, bytes\_sent).
3. **Don't put high-churn, ever-increasing values in tags on TSM engines** (e.g., a raw event UUID) — this is the classic "cardinality explosion" mistake on 1.x/2.x.
4. **One measurement/table per logical entity type**, not one giant catch-all table — keeps schemas clean and query planning efficient.
5. **Pick retention windows deliberately**: keep raw high-resolution data for a short window, downsample (via Tasks / Processing Engine) into a longer-retention rollup table for historical queries.

---

## 5. Query Languages: InfluxQL, Flux, SQL

InfluxDB has had **three** query languages across its history. Knowing which applies to which version matters a lot.

| Language        | Introduced                            | Status (2026)                                                   | Style                                                              |
|-----------------|---------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|
| <b>InfluxQL</b> | InfluxDB 1.x                          | Still supported across 1.x, 2.x (limited), and all 3.x products | SQL-like, time-series-aware<br>( <code>GROUP BY time(...)</code> ) |
| <b>Flux</b>     | InfluxDB 2.x (also backported to 1.8) | Maintenance mode; not supported on InfluxDB 3 products          | Functional, pipe-forward<br>(`                                     |

|            |              |                                             |                                                                 |
|------------|--------------|---------------------------------------------|-----------------------------------------------------------------|
| <b>SQL</b> | InfluxDB 3.x | <b>Primary language for all v3 products</b> | Standard SQL via Apache DataFusion, plus time-series extensions |
|------------|--------------|---------------------------------------------|-----------------------------------------------------------------|

## 5.1 InfluxQL

Familiar to anyone who knows SQL, with time-bucketing built in.

-- Average temperature per 5-minute window, last 1 hour, grouped by location

```
SELECT MEAN("temperature") AS avg_temp
```

```
FROM "weather"
```

```
WHERE time > now() - 1h
```

```
GROUP BY time(5m), "location"
```

```
FILL(none)
```

-- Show measurements / tag keys / tag values (schema exploration)

```
SHOW MEASUREMENTS
```

```
SHOW TAG KEYS FROM "weather"
```

```
SHOW TAG VALUES FROM "weather" WITH KEY = "location"
```

Common InfluxQL functions: **MEAN**, **SUM**, **COUNT**, **MIN**, **MAX**, **MEDIAN**, **STDDEV**, **PERCENTILE**, **DERIVATIVE**, **DIFFERENCE**, **MOVING\_AVERAGE**, **NON\_NEGATIVE\_DERIVATIVE**, **TOP/BOTTOM**.

## 5.2 Flux (2.x / legacy)

Flux pipes data through a chain of transformation functions, somewhat like a Unix pipeline. Even though it's in maintenance mode, you'll likely encounter it in existing 1.8/2.x deployments.

```
from(bucket: "telegraf")
```

```
|> range(start: -1h)
```

```
|> filter(fn: (r) => r._measurement == "cpu" and r._field == "usage_idle")
```

```
|> filter(fn: (r) => r.host == "server01")
```

```
|> aggregateWindow(every: 5m, fn: mean, createEmpty: false)
```

```
|> yield(name: "mean_cpu_idle")
```

// Join two streams (e.g., compute a derived metric)

```
cpu = from(bucket: "telegraf")
```

```
|> range(start: -1h)
```

```
|> filter(fn: (r) => r._measurement == "cpu" and r._field == "usage_user")
```

```
mem = from(bucket: "telegraf")
```

```
|> range(start: -1h)
```

```
|> filter(fn: (r) => r._measurement == "mem" and r._field == "used_percent")
```

```
join(tables: {cpu: cpu, mem: mem}, on: ["_time", "host"])
```

```
// Downsample task body example
```

```
data = from(bucket: "raw_metrics")
```

```
|> range(start: -task.every)
```

```
|> filter(fn: (r) => r._measurement == "weather")
```

```
|> aggregateWindow(every: 10m, fn: mean)
```

```
data |> to(bucket: "downsampled_metrics", org: "myorg")
```

Key Flux concepts: **pipe-forward operator** `|>`, **tables** (Flux's core data structure — a stream of annotated CSV-like tables with `_time`, `_value`, `_field`, `_measurement` columns), `range()`, `filter()`, `group()`, `pivot()`, `window()/aggregateWindow()`, `join()`, `to()` (write back to a bucket).

If you're starting fresh today, **do not invest new effort in learning Flux** unless you're locked into 2.x — write SQL instead and target v3.

### 5.3 SQL (InfluxDB 3.x — primary language)

Standard SQL via DataFusion, with time-series helper functions (`date_bin`, `DATE_BIN`-based windowing, `now()`, interval arithmetic).

```
-- Basic range + filter
```

```
SELECT time, host, usage_idle
```

```
FROM cpu
```

```
WHERE time >= now() - INTERVAL '1 hour'
```

```
AND host = 'server01'
```

```
ORDER BY time DESC
```

```
LIMIT 100;
```

```
-- Time-bucketed aggregation (the SQL equivalent of GROUP BY time())
```

```
SELECT
```

```
date_bin(INTERVAL '5 minutes', time) AS bucket,
```

```
host,
```

```
AVG(usage_idle) AS avg_idle
```

```
FROM cpu
```

```
WHERE time >= now() - INTERVAL '6 hours'
```

```
GROUP BY bucket, host
```

```
ORDER BY bucket;
```



```
-- Schema exploration
SHOW TABLES;
SHOW COLUMNS FROM cpu;
SELECT DISTINCT host FROM cpu;

-- Joins across tables (something Flux made awkward — SQL makes natural)
SELECT c.time, c.host, c.usage_idle, m.used_percent
FROM cpu c
JOIN mem m ON c.time = m.time AND c.host = m.host
WHERE c.time >= now() - INTERVAL '1 hour';
```

InfluxDB 3 also exposes data via **Arrow Flight SQL**, enabling fast, typed, columnar transport for BI tools and drivers (Python, JDBC/ODBC bridges, etc.) — far more efficient than parsing JSON over HTTP for large result sets.

#### 5.4 Choosing a language today

- **New code on InfluxDB 3.x → SQL.** It's the most actively developed, most portable (every BI/SQL tool understands it), and has Flight SQL for performance.
- **InfluxQL** still works everywhere (1.x, 2.x, 3.x) and is a fine lightweight option for simple time-bucketed queries without learning full SQL semantics for windows.
- **Flux** only if you're maintaining existing 1.8/2.x dashboards, tasks, or Kapacitor-adjacent tooling.

---

## 6. Writes (Line Protocol & Write APIs)

### 6.1 Line Protocol — the universal write format

All generations accept the same simple text-based **line protocol**:

```
<measurement>[,<tag_key>=<tag_value>...]
<field_key>=<field_value>[,<field_key2>=<field_value2>...] [<timestamp>]
```

Rules:

- Measurement and tags come first (whitespace-free, comma-separated).
- A single space separates the tag set from the field set.
- Fields are comma-separated **key=value** pairs; string values are quoted, integers suffixed with **i**, unsigned with **u**, booleans **t/f/true/false**.
- Timestamp is optional (server assigns "now" if omitted) and is in nanoseconds by default, though you can specify write precision (**ns**, **us**, **ms**, **s**).

- No spaces allowed in measurement/tag/field names unless escaped with \.

```
cpu,host=server01,region=us-west usage_idle=98.2,usage_user=1.1
1717689600000000000
temperature,sensor_id=TLM0100 value=71.2 1717689600000000000
events,service=checkout level="error",message="payment gateway timeout"
1717689600000000000
```

## 6.2 Write APIs by version

| Version                   | Endpoint                                                  | Auth                                        |
|---------------------------|-----------------------------------------------------------|---------------------------------------------|
| 1.x                       | POST /write?db=mydb&rp=myrp                               | Basic auth / query param token (if enabled) |
| 2.x                       | POST /api/v2/write?org=myorg&bucket=mybucket&precision=ns | Authorization: Token <API_TOKEN>            |
| 3.x (native)              | POST /api/v3/write_lp?db=mydb&precision=ns                | Authorization: Bearer <token> (or Token)    |
| 3.x (v1/v2 compatibility) | /write, /api/v2/write also work                           | Same compatibility token/username schemes   |

### curl example (v3 native):

```
curl -X POST "http://localhost:8181/api/v3/write_lp?db=mydb&precision=ns" \
  --header "Authorization: Bearer ${INFLUXDB3_AUTH_TOKEN}" \
  --data-raw 'cpu,host=server01 usage_idle=98.2,usage_user=1.1'
```

### curl example (v2):

```
curl -X POST
"http://localhost:8086/api/v2/write?org=myorg&bucket=mybucket&precision=ns" \
  --header "Authorization: Token ${INFLUX_TOKEN}" \
  --data-raw 'cpu,host=server01 usage_idle=98.2,usage_user=1.1'
```

## 6.3 Batching and write performance

- **Always batch writes.** Sending one line-protocol line per HTTP request is extremely inefficient. Batch 5,000–10,000 points (or a few hundred KB to a few MB) per request as a starting point, then tune.
- Use **gzip compression** on the request body for large batches.
- Client libraries (Python, Go, Java, JS, C#) include built-in batching write APIs — prefer those over hand-rolled HTTP calls.
- For v3 Enterprise, dedicate **ingest-mode nodes** behind a load balancer for high write throughput; ingest nodes use IO threads for line-protocol parsing, so more cores + IO thread allocation = higher ingest throughput.
- Avoid writing **out-of-order / very old timestamps** in large volumes if avoidable — it produces extra compaction work (less of a hard limit on v3 than the old TSM engine, but still a performance consideration).

#### 6.4 Client libraries

Official client libraries exist for **Python, Go, Java, JavaScript/Node.js, C#, PHP, Ruby**, etc. — covering write batching, retries, and (on 1.x/2.x) Flux query execution; on 3.x, libraries increasingly favor SQL + Flight SQL.

---

#### 7. Reads (Query APIs)

| Version | Query endpoint                                                                    | Language(s)                                                              | Typical client                                                                     |
|---------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1.x     | GET/POST<br><code>/query?db=mydb&amp;q=...</code>                                 | InfluxQL                                                                 | curl, client libs, Grafana InfluxDB datasource (InfluxQL mode)                     |
| 2.x     | POST <code>/api/v2/query</code><br>(body = Flux script)                           | Flux (primary), limited InfluxQL via <code>/query</code> compat endpoint | influx CLI, Grafana (Flux mode), client libs                                       |
| 3.x     | POST <code>/api/v3/query_sql</code> ,<br>POST <code>/api/v3/query_influxql</code> | SQL, InfluxQL                                                            | influxdb3 CLI, Grafana (SQL/Flight SQL datasource), any SQL/BI tool via Flight SQL |

**curl example — InfluxQL (1.x/3.x compatible):**

```
curl -G "http://localhost:8086/query" \
```

```
--header "Authorization: Token ${TOKEN}" \  
--data-urlencode "db=mydb" \  
--data-urlencode "q=SELECT mean(usage_idle) FROM cpu WHERE time > now() -  
1h GROUP BY time(5m)"
```

### curl example — Flux (2.x):

```
curl -X POST "http://localhost:8086/api/v2/query?org=myorg" \  
--header "Authorization: Token ${TOKEN}" \  
--header "Content-Type: application/vnd.flux" \  
--data 'from(bucket:"telegraf") |> range(start:-1h) |>  
filter(fn:(r)=>r._measurement=="cpu")'
```

### curl example — SQL (3.x):

```
curl -X POST "http://localhost:8181/api/v3/query_sql" \  
--header "Authorization: Bearer ${INFLUXDB3_AUTH_TOKEN}" \  
--header "Content-Type: application/json" \  
--data '{  
  "db": "mydb",  
  "q": "SELECT * FROM cpu WHERE time > now() - INTERVAL \"1 hour\" LIMIT  
10"  
}'
```

## 7.1 Flight SQL (v3)

For analytical / BI workloads, v3 exposes **Arrow Flight SQL** on a gRPC port — significantly faster than HTTP/JSON for large result sets, with native drivers for Python ([pyarrow.flight](#) / ADBC), JDBC/ODBC bridges, and direct integration with tools that speak Arrow natively (e.g., pandas via Arrow).

## 7.2 Grafana integration

- **InfluxQL datasource**: works across all versions.
- **Flux datasource**: 1.8+/2.x only.
- **SQL / Flight SQL datasource**: for InfluxDB 3.x — recommended for new dashboards.

---

## 8. Tasks & the Processing Engine

### 8.1 Flux Tasks (1.8 / 2.x)

A **Task** is a saved Flux script that runs on a schedule (cron or "every" interval) inside InfluxDB itself — used for:

- **Downsampling**: aggregate raw high-resolution data into a lower-resolution, longer-retention bucket.

- **Alerting / monitoring "checks"**: evaluate thresholds and write notifications (in the 2.x Notifications/Checks system, often paired with Slack/PagerDuty endpoints).
- **Data transformation/ETL**: reshape, filter, or copy data between buckets, or even between InfluxDB instances.

option task = {name: "downsample-cpu", every: 1h}

```
from(bucket: "raw")
  |> range(start: -task.every)
  |> filter(fn: (r) => r._measurement == "cpu")
  |> aggregateWindow(every: 10m, fn: mean, createEmpty: false)
  |> to(bucket: "cpu_downsampled", org: "myorg")
```

Tasks can be managed via the UI, the **influx** CLI (**influx task create -f task.flux**), or the **/api/v2/tasks** REST API.

## 8.2 InfluxDB 3.x Processing Engine

v3 replaces Flux Tasks with an **embedded Python Processing Engine** running inside the database process — Python "plugins" can react to:

- **On-write triggers**: run logic immediately as data is written (real-time enrichment, validation, alerting).
- **Scheduled triggers**: cron-like periodic jobs (downsampling, rollups, housekeeping).
- **HTTP request triggers**: expose custom HTTP endpoints backed by Python logic.

InfluxData publishes a **plugin library** with ready-made plugins for common needs: system metrics collection, threshold monitoring/alerting (Slack, Discord, SMS, WhatsApp, webhook notification channels), anomaly detection, and downsampling.

# Example processing engine plugin skeleon (scheduled trigger)

```
def process_scheduled_call(influxdb3_local, call_time, args=None):
    query = "SELECT mean(usage_idle) AS avg_idle FROM cpu WHERE time > now() - INTERVAL '10 minutes'"
    result = influxdb3_local.query(query)
    for row in result:
        if row["avg_idle"] < 10:
            influxdb3_local.warn(f"Low idle CPU detected: {row['avg_idle']}")
            # send alert via configured notification plugin / webhook
```

This effectively merges what used to require **Kapacitor** (1.x's separate stream/batch processing engine) directly into the database.

---

## 9. Telegraf

**Telegraf** is InfluxData's open-source, plugin-driven metrics/agent collector — the "T" in the original **TICK stack** (Telegraf, InfluxDB, Chronograf, Kapacitor). It is written in Go, ships as a single static binary, and supports **300+ input and output plugins**.

### 9.1 Architecture

[Input Plugins] → [Processor Plugins] → [Aggregator Plugins] → [Output Plugins]

- **Input plugins:** pull or receive metrics — e.g. `cpu`, `mem`, `disk`, `net`, `docker`, `kubernetes`, `mysql`, `postgresql`, `redis`, `kafka_consumer`, `statsd`, `prometheus` (scrape), `snmp`, `mqtt_consumer`, `http`, `exec`.
- **Processor plugins:** transform metrics in-flight — e.g. `rename`, `regex`, `enum`, `converter`, `geoip`.
- **Aggregator plugins:** compute aggregates over a time window before output — e.g. `minmax`, `histogram`, `basicstats`.
- **Output plugins:** send metrics onward — e.g. `influxdb_v2` (or `influxdb` for v1/v3-compatible), `prometheus_client`, `kafka`, `elasticsearch`, `file`, `http`.

### 9.2 Example telegraf.conf

```
# Global agent settings
```

```
[agent]
```

```
interval = "10s"
```

```
round_interval = true
```

```
metric_batch_size = 1000
```

```
metric_buffer_limit = 10000
```

```
flush_interval = "10s"
```

```
flush_jitter = "2s"
```

```
# ---- INPUTS ----
```

```
[[inputs.cpu]]
```

```
percpu = true
```

```
totalcpu = true
```

```
[[inputs.mem]]
```

```
[[inputs.disk]]
```

```
ignore_fs = ["tmpfs", "devtmpfs", "overlay"]
```

```
[[inputs.net]]
```

```
[[inputs.docker]]
```

```
    endpoint = "unix:///var/run/docker.sock"
```

```
# ---- PROCESSORS ----
```

```
[[processors.rename]]
```

```
    [[processors.rename.replace]]
```

```
        measurement = "cpu"
```

```
        dest = "host_cpu"
```

```
# ---- OUTPUTS ----
```

```
# Output to InfluxDB 3 (Core/Enterprise) using v2-compatible API
```

```
[[outputs.influxdb_v2]]
```

```
    urls = ["http://influxdb:8181"]
```

```
    token = "$INFLUXDB3_AUTH_TOKEN"
```

```
    organization = ""          # not used by v3, leave blank or required by compat layer
```

```
    bucket = "mydb"
```

```
# Or output to InfluxDB 2.x natively
```

```
# [[outputs.influxdb_v2]]
```

```
#   urls = ["http://influxdb:8086"]
```

```
#   token = "$INFLUX_TOKEN"
```

```
#   organization = "myorg"
```

```
#   bucket = "telegraf"
```

Run with: `telegraf --config telegraf.conf --test` (dry run) or

`telegraf --config telegraf.conf` (run).

### 9.3 Deployment patterns

- **Host agent:** one Telegraf per VM/bare-metal host (systemd service / package install) collecting system + local app metrics.
- **Kubernetes DaemonSet:** one Telegraf pod per node for node-level metrics (cpu/mem/disk/net), often paired with the `kubernetes` or `kube_inventory` input plugins for cluster-level object metrics.
- **Sidecar container:** Telegraf running alongside an app container in the same pod, scraping app-specific metrics (e.g., `statsd`, `prometheus` input pointed at `localhost`).

- **Centralized/aggregator Telegraf:** a small fleet of Telegraf instances acting as Kafka/MQTT/StatsD consumers, fanning many upstream sources into InfluxDB.

## 9.4 Telegraf Enterprise / Controller (new, beta as of 2026)

InfluxData is rolling out a **Telegraf Controller** (beta) — a centralized control plane (UI + API) for managing Telegraf configuration and agent health across large fleets, plus a commercial **Telegraf Enterprise Support** offering. This targets organizations running Telegraf at scale who currently hand-manage hundreds/thousands of `telegraf.conf` files.

---

## 10. InfluxDB on Kubernetes

### 10.1 InfluxDB 2.x via Helm (single-node, StatefulSet pattern)

helm repo add influxdata <https://helm.influxdata.com/>

helm repo update

`influxdb-values.yaml`:

image:

repository: influxdb

tag: "2.7"

persistence:

enabled: true

size: 20Gi

storageClassName: standard

resources:

requests:

cpu: "500m"

memory: "1Gi"

limits:

cpu: "2"

memory: "4Gi"

service:

type: ClusterIP

port: 8086

adminUser:

organization: myorg



```
bucket: primary
user: admin
retention_policy: "30d"
existingSecret: influxdb-admin-secret
```

```
kubectl create namespace influxdb
kubectl create secret generic influxdb-admin-secret \
  --from-literal=admin-password='ChangeMe123!' \
  --from-literal=admin-token='replace-with-strong-token' \
  -n influxdb
helm install influxdb influxdata/influxdb2 -f influxdb-values.yaml -n influxdb
```

Because InfluxDB 2.x OSS is single-node, this Helm chart deploys a **single StatefulSet replica** with a **PersistentVolumeClaim** — Kubernetes here provides self-healing/restart and storage management, *not* true multi-node HA. For real HA on Kubernetes with 2.x, you'd need the (legacy, 1.x-based) Enterprise clustering approach or migrate to v3.

## 10.2 InfluxDB 3.x on Kubernetes

**Option A — InfluxDB 3 Core/Enterprise as a StatefulSet** (self-managed manifests), pointing each node at shared object storage:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: influxdb3-enterprise
  namespace: influxdb
spec:
  serviceName: influxdb3
  replicas: 2
  selector:
    matchLabels:
      app: influxdb3
  template:
    metadata:
      labels:
        app: influxdb3
    spec:
      containers:
        - name: influxdb3
          image: influxdb:3-enterprise
```

```

args:
  - serve
  - --node-id=$(POD_NAME)
  - --cluster-id=prod-cluster
  - --mode=ingest,query
  - --object-store=s3
  - --bucket=my-influxdb-data
  - --http-bind=0.0.0.0:8181
env:
  - name: POD_NAME
    valueFrom:
      fieldRef:
        fieldPath: metadata.name
  - name: AWS_ACCESS_KEY_ID
    valueFrom:
      secretKeyRef: { name: influxdb3-s3-creds, key: access-key-id }
  - name: AWS_SECRET_ACCESS_KEY
    valueFrom:
      secretKeyRef: { name: influxdb3-s3-creds, key: secret-access-key }
  - name: INFLUXDB3_ENTERPRISE_LICENSE_EMAIL
    valueFrom:
      secretKeyRef: { name: influxdb3-license, key: email }
ports:
  - containerPort: 8181
volumeMounts:
  - name: local-cache
    mountPath: /var/lib/influxdb3
volumeClaimTemplates:
  - metadata:
      name: local-cache
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 50Gi
---
apiVersion: v1
kind: Service
metadata:

```

```
name: influxdb3
namespace: influxdb
spec:
  selector:
    app: influxdb3
  ports:
    - port: 8181
      targetPort: 8181
```

Note: every node connects to the **same object store bucket** (S3/GCS/Azure Blob) — that shared object storage *is* the durable, replicated layer. The local `volumeClaimTemplate` above is just for local WAL/cache, not the source of truth.

**Option B — InfluxDB Clustered:** InfluxData's own Kubernetes-native, HA-by-design deployment pattern for InfluxDB 3, intended to be installed and managed via Kubernetes manifests/operator tooling provided by InfluxData (proof-of-concept and production install guides are published in InfluxData's docs). This is the recommended path if you want a fully HA, self-hosted InfluxDB 3 cluster on your own Kubernetes infrastructure without hand-rolling the StatefulSet approach above.

### 10.3 General Kubernetes considerations

- **Storage class:** use SSD-backed storage classes for local WAL/cache volumes; object storage (S3/GCS/Azure) handles durable Parquet data on v3.
- **Resource requests/limits:** size ingest nodes for CPU (line-protocol parsing is CPU-bound) and query nodes for memory (DataFusion query execution).
- **Service mesh / network policies:** restrict which namespaces/pods can reach the InfluxDB write/query ports.
- **Secrets:** store tokens, license keys, and object-store credentials in Kubernetes Secrets (or an external secrets manager via CSI driver), never in ConfigMaps or plain manifests.
- **Probes:** configure `livenessProbe/readinessProbe` against the `/health` or `/ping` endpoint.
- **Telegraf as DaemonSet:** pair with a Telegraf DaemonSet + the `kubernetes/kube_inventory` input plugins to monitor the cluster itself.

---

## 11. High Availability & Clustering

This is one of the most version-dependent topics in InfluxDB — read carefully for the version you're actually running.

### 11.1 InfluxDB 1.x OSS

- **No built-in clustering.** Single node only.
- HA required the commercial **InfluxDB Enterprise (1.x)** product: a cluster of **meta nodes** (odd number, minimum 3, using Raft consensus over ports 8089/8091) and **data nodes** (minimum 2, replicating shards, communicating over port 8088). You had to put your own load balancer in front of the data nodes' HTTP API (port 8086) — InfluxDB Enterprise did not include a built-in LB.
- This architecture is now legacy; not where InfluxData is investing.

### 11.2 InfluxDB 2.x OSS

- **Still single-node only** in open source. There was never a multi-node OSS 2.x clustering story — InfluxData's stated direction was instead "skip straight to rebuilding the engine" (which became v3) rather than retrofitting clustering onto TSM 2.x.
- HA for 2.x in practice meant: external redundancy (e.g., dual-write from Telegraf to two independent instances), or moving to InfluxDB Cloud (TSM) for a managed multi-tenant HA service.

### 11.3 InfluxDB 3 Core

- **Single node only** (by design — Core is the free/edge/dev tier). No clustering.

### 11.4 InfluxDB 3 Enterprise — native multi-node clustering

This is the modern, recommended way to get HA InfluxDB. Key architectural facts:

- **Diskless / shared-object-store architecture:** all nodes in a cluster point at the *same* object store (S3, S3-compatible, Azure Blob, or GCS). Because the durable data already lives in shared, replicated cloud object storage, "clustering" mostly means coordinating *compute* nodes around that shared data — there's no separate data-replication protocol to operate.
- **Node modes** (`--mode` flag): **ingest**, **query**, **compact**, **process**, or **all** (all-in-one, single-node only). A given node can run one or more modes.
  - Exactly **one node per cluster should run compact** — compacted Parquet data is meant for a single writer, many readers.
  - Multiple **ingest** nodes can write concurrently (each writes to its own area in object storage); downstream readers/compactors pick up data across all of them.
  - **Unlimited read replicas** are possible (**query** mode nodes) — capped only by your object storage's request-rate capacity, not by a fixed replica count.

- **Recommended cluster topologies:**

| Topology | Nodes | Use case |
|----------|-------|----------|
|----------|-------|----------|

|                                     |                                                                                                             |                                                                                      |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>2-node basic HA</b>              | 2 nodes, both read+write, one also runs compactor                                                           | Cost-efficient HA for smaller workloads; fault tolerant if one node dies             |
| <b>3-node (recommended default)</b> | 2 read/write nodes + 1 dedicated compactor                                                                  | Avoids compaction competing with read/write resources                                |
| <b>5+ node scaled cluster</b>       | Dedicated ingest nodes (multiple) + dedicated query nodes (multiple) + 1 compactor + optional process nodes | High-throughput production workloads needing independent scaling of ingest vs. query |

- **Failure handling:** if an ingest/query node fails, other nodes continue serving reads/writes from the shared object store and from already-compacted Parquet files — there's no single point of failure on the data path as long as the object store itself is durable/multi-AZ.
- **Licensing:** Enterprise requires license activation (free "Home" license for personal use, 30-day trial, or a paid commercial license for production — contact sales).

### 11.5 InfluxDB Clustered & Cloud Dedicated/Serverless

- **InfluxDB Clustered:** a packaged, Kubernetes-native HA deployment of the same v3 multi-node architecture above, intended for self-hosting "the managed-cloud experience" on your own infrastructure.
- **InfluxDB Cloud Dedicated / Serverless:** InfluxData operates the HA cluster for you; you just get an endpoint + token.

### 11.6 Open-source community alternative (legacy 1.x)

Worth knowing about if you're stuck on 1.x without an Enterprise license: community projects like **influxdb-cluster** (a fork that reimplements 1.x clustering as a free alternative to the discontinued OSS-Enterprise split) exist, but are third-party/community-maintained, not InfluxData products — evaluate operational risk carefully before relying on them in production.

### 11.7 Practical HA checklist (v3 Enterprise / Clustered)

1. Put a load balancer (cloud LB, HAProxy, or Kubernetes **Service**) in front of your ingest and query node pools.
2. Use durable, multi-AZ/multi-region object storage as the backing store.
3. Run an odd dedicated compactor strategy — exactly one compactor node; monitor its queue depth/lag.

4. Set up automated failover health checks (liveness/readiness probes) so the LB stops routing to a dead node quickly.
5. Test node-loss scenarios (kill an ingest node mid-write) before going to production.
6. Keep client write retries idempotent-friendly (line protocol writes are generally safe to retry).

---

## 12. Security & Authentication

### 12.1 InfluxDB 1.x

- Auth is **off by default** — must explicitly enable (`auth-enabled = true` in config).
- Username/password-based; HTTP Basic Auth or query-param token.
- Coarse-grained: admin vs. non-admin, per-database read/write grants. No fine-grained RBAC.

### 12.2 InfluxDB 2.x

- **Token-based authentication** is the default and primary mechanism — every request needs `Authorization: Token <token>`.
- **Organizations** scope resources (buckets, tasks, dashboards); tokens can be scoped to:
  - **All-access** (admin)
  - **Operator** (cross-org admin)
  - **Read/write scoped to specific buckets**
- Tokens are managed via UI, CLI (`influx auth create`), or API.
- **Stronger token security (OSS 2.9.0+)**: tokens are now **hashed on disk by default**. Existing plaintext tokens get hashed on first startup after upgrade and the plaintext can't be recovered afterward — **capture any plaintext tokens you still need before upgrading**. Hashed tokens authenticate identically from the client's perspective; nothing changes for integrations.
- **TLS**: configure `tls-cert/tls-key` to terminate HTTPS at InfluxDB, or terminate TLS at a reverse proxy/load balancer in front of it (common pattern).

### 12.3 InfluxDB 3.x (Core / Enterprise)

- **Token-based auth**, `Authorization: Bearer <token>` (or `Token` for compatibility).
- v3 products also expose **v1-compatible auth** (username/password) and **v2-compatible auth** (token + org/bucket) for migration ease.
- **InfluxDB 3 Enterprise** adds **fine-grained access controls**: resource-scoped tokens (specific databases/tables), **expiring tokens** (time-limited), and broader RBAC than Core.

- **Admin token:** created at first startup; treat like a root credential — store in a secrets manager, rotate periodically.
- **TLS:** terminate at the database (cert/key flags) or, more commonly in Kubernetes, at an Ingress/Service mesh sidecar in front of it.

---

### 13. Monitoring InfluxDB Itself

You should monitor InfluxDB the same way you'd monitor any critical data infrastructure — ironically, often *with* InfluxDB/Telegraf itself.

#### 13.1 InfluxDB 1.x/2.x internal monitoring

- 1.x exposes internal stats via the `_internal` database (when `[monitor]` is enabled in config) — measurements like `database`, `httpd`, `write`, `queryExecutor`, `runtime`, `tsm1_engine`, etc.
- 2.x ships a built-in **\_monitoring system bucket** that captures internal operational metrics (and, separately, the **Checks & Notifications** feature lets you build threshold-based alerting directly off any bucket's data, sending to Slack/PagerDuty/HTTP/etc.).
- A `/metrics` endpoint exposes **Prometheus-format** metrics directly from InfluxDB (2.x), so you can scrape InfluxDB with Prometheus if that's your existing stack.

#### 13.2 InfluxDB 3.x monitoring

- v3 exposes operational data via `system.*` **SQL tables** you can query directly, e.g.:

-- Check for high failed-query rate (parsing/auth/etc. issues)

```
SELECT
  count(*) AS total_queries,
  sum(CASE WHEN success = true THEN 1 ELSE 0 END) AS successful_queries,
  sum(CASE WHEN success = false THEN 1 ELSE 0 END) AS failed_queries
FROM system.queries
WHERE issue_time > now() - INTERVAL '5 minutes';
```

- Other useful `system` tables typically include things like query history, compaction status/summary, and node/catalog metadata — explore with `SHOW TABLES` against the `system` schema on your running instance, since exact table names can change between releases.
- The **Processing Engine plugin library** (see §8.2) includes ready-made **system metrics collection** and **threshold monitoring/alerting plugins** (CPU/mem/disk/network stats, configurable thresholds, multi-channel notifications via Slack/Discord/SMS/WhatsApp/webhook).

- For cluster-wide visibility, scrape each node's `/metrics` (Prometheus-format, where exposed) into your existing Prometheus/Grafana stack alongside the SQL-based `system` queries.

### 13.3 What to actually alert on

| Signal                                                            | Why it matters                                                                        |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Write success/error rate</b>                                   | Catches client misconfiguration, auth failures, malformed line protocol, or overload. |
| <b>Query latency (p50/p95/p99) &amp; failure rate</b>             | Detects degraded query nodes, bad query patterns, or cardinality issues.              |
| <b>WAL / buffer size and snapshot lag</b>                         | Growing WAL backlog = ingest outpacing snapshot/compaction.                           |
| <b>Compactor queue depth / lag (v3)</b>                           | A stalled compactor degrades long-range query performance over time.                  |
| <b>Object storage error rate / latency (v3)</b>                   | Since durability lives in the object store, its health directly impacts the database. |
| <b>Disk usage (1.x/2.x TSM, or local WAL/cache volumes in v3)</b> | Prevent disk-full outages.                                                            |
| <b>Series cardinality growth (1.x/2.x especially)</b>             | Early warning before performance cliff.                                               |
| <b>Node up/down + replica health (multi-node v3)</b>              | Confirms HA is actually intact, not silently degraded to one node.                    |

### 13.4 Dashboards

Grafana is the de facto standard dashboarding layer in front of InfluxDB (any version), using the InfluxQL, Flux, or SQL/Flight SQL datasource as appropriate. InfluxDB's own UI also ships built-in dashboards (1.x/2.x: Chronograf or the bundled 2.x UI; 3.x: the standalone **InfluxDB 3 Explorer** web app for querying/visualizing/administering Core and Enterprise).

---

## 14. Backup & Restore

### 14.1 InfluxDB 1.x

# Backup (full, to a local directory)



```
influxd backup -portable /path/to/backup
```

# Backup a specific database

```
influxd backup -portable -database mydb /path/to/backup
```

# Restore

```
influxd restore -portable /path/to/backup
```

- **-portable** format is the modern, recommended backup format (works across patch versions); legacy non-portable format exists for older installs.
- Can do incremental backups by re-running against the same backup directory.

## 14.2 InfluxDB 2.x

# Backup (entire instance, or scope to org/bucket)

```
influx backup /path/to/backup \  
--org myorg \  
--token $INFLUX_TOKEN
```

# Backup a single bucket

```
influx backup /path/to/backup \  
--bucket mybucket \  
--token $INFLUX_TOKEN
```

# Restore

```
influx restore /path/to/backup \  
--token $INFLUX_TOKEN
```

- The **influx** CLI backup/restore commands snapshot both the **bucket data** and the **metadata** (users, orgs, dashboards, tasks) needed for a full instance restore.
- For periodic automated backups, wrap this in a cron job / Kubernetes CronJob and ship the backup directory to durable storage (S3, GCS, etc.).

## 14.3 InfluxDB 3.x

Because v3's durable data already lives as Parquet files in object storage, backup strategy shifts:

- **Object storage-level backup:** rely on your cloud provider's native bucket versioning/replication/snapshotting (e.g., S3 Cross-Region Replication, versioning + lifecycle policies) as the primary durability mechanism — the Parquet files *are* the backup-able artifact.

- **influxdb3 export** (Enterprise): export compacted data as Parquet files for external tools — list databases/tables/compacted time windows, then export selected data; useful for archival, migration, or feeding into a separate analytics pipeline (Spark, DuckDB, etc.).
- **Multi-node cluster backup/restore**: for InfluxDB 3 Enterprise clusters, the documented approach is to **copy the object storage files in a specific recommended order per node type** (catalog/metadata first, then data) rather than a single CLI command — consult the current Enterprise docs for the exact sequence before performing a cluster-wide restore, since this is one of the areas most likely to evolve between releases.
- **Catalog/metadata**: since schema/catalog info also lives in object storage alongside the data, a full object-store backup captures both data and catalog consistently if done correctly (e.g., via storage-level snapshot rather than file-by-file copy mid-write).

#### 14.4 General backup/restore best practices

1. **Test restores regularly** — an untested backup is not a real backup.
  2. **Automate** backups via cron/CronJob, not manual runs.
  3. **Encrypt backups at rest** (most object storage does this by default; verify).
  4. **Retain multiple generations** (daily/weekly/monthly) with a clear retention policy for the backups themselves.
  5. **Separate backup storage from production storage** — different bucket/account/region where feasible, to survive a regional or account-level incident.
  6. **Document and rehearse the restore runbook** including expected RTO/RPO for your environment.
-

AWS

## 1. AWS Fundamentals

| Question                          | Answer                                                                                      |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| What is AWS?                      | Cloud platform offering on-demand compute, storage, and services via pay-as-you-go pricing. |
| Benefits of AWS?                  | Scalability, flexibility, cost efficiency, global reach, and managed services.              |
| Shared Responsibility Model?      | AWS secures the cloud infrastructure; you secure data, applications, and access.            |
| IaaS vs PaaS vs SaaS?             | IaaS: EC2, PaaS: Elastic Beanstalk, SaaS: WorkMail.                                         |
| Region vs AZ vs Edge Location?    | Region = geographic area; AZ = isolated data centers; Edge = CDN caching points.            |
| Global Accelerator vs CloudFront? | Accelerator = TCP/UDP optimization; CloudFront = HTTP caching.                              |
| AWS Local Zones?                  | Extension of regions for ultra-low latency applications near users.                         |

---

## 2. EC2 (Elastic Compute Cloud)

- An Amazon Machine Image (AMI) serves as a template for virtual servers, known as EC2 instances, enabling rapid and consistent provisioning.

Features:

- Root Volume: The AMI captures a state, including the operating system and pre-installed applications, and stores it as a template. This template is employed to establish the root volume of the EC2 instance.
- Customizations and Snapshots: Beyond the root volume, the AMI can include additional storage volumes. These might have specific data sets or applications. When the AMI is utilized to launch an EC2 instance, it ensures that these configured volumes are also established and linked to the new EC2 instance.

- In AWS, every EC2 instance automatically gets a Public IP and can optionally be assigned an Elastic IP for more flexibility.

| Question                                        | Answer                                                                                                               |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| What is EC2?                                    | Virtual servers in AWS cloud.                                                                                        |
| On-Demand vs Reserved vs Spot vs Savings Plans? | On-Demand = flexible; Reserved = 1-3yr commit; Spot = cheapest (interruptible); Savings Plans = flexible commitment. |
| What is an AMI?                                 | Pre-configured OS template for launching instances.                                                                  |
| Security Group vs NACL?                         | SG = stateful (instance-level); NACL = stateless (subnet-level).                                                     |
| Stop vs Terminate vs Hibernate?                 | Stop = pause + retain EBS; Terminate = delete; Hibernate = save RAM state.                                           |
| Instance Store vs EBS?                          | Instance Store = ephemeral (lost on stop); EBS = persistent block storage.                                           |
| EC2 Placement Groups?                           | Cluster = low latency; Spread = HA across hardware; Partition = distributed apps.                                    |
| Enhanced Networking?                            | SR-IOV for higher PPS, lower latency (ENA, Intel VF).                                                                |
| Nitro System?                                   | Hypervisor replacement providing better performance and security.                                                    |
| How to troubleshoot failed instance launch?     | Check instance limits, subnet capacity, AMI availability, EBS snapshot status.                                       |

---

### 3. S3 (Simple Storage Service)

| Question | Answer |
|----------|--------|
|----------|--------|

|                                                                  |                                                                                     |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| What is S3?                                                      | Object storage for unstructured data (unlimited scalability).                       |
| S3 Storage Classes?                                              | Standard, IA, One Zone-IA, Intelligent-Tiering, Glacier Instant/Flexible/Deep.      |
| S3 Durability vs Availability?                                   | Durability = 11 nines (data loss protection); Availability = uptime (99.9%-99.99%). |
| S3 Versioning?                                                   | Keeps multiple versions of objects (protects from deletions).                       |
| Lifecycle Policies?                                              | Automates transitions between storage classes and expiration.                       |
| Cross-Region Replication (CRR) vs Same-Region Replication (SRR)? | CRR = DR/compliance; SRR = log aggregation/compliance.                              |
| S3 Event Notification?                                           | Triggers Lambda, SQS, SNS on object changes (PUT/DELETE).                           |
| Pre-Signed URL?                                                  | Grants temporary access to private objects via signed URL.                          |
| S3 Consistency Model?                                            | Strong read-after-write consistency for all operations.                             |
| S3 Transfer Acceleration?                                        | Uses CloudFront edge locations to speed up uploads.                                 |
| Multipart Upload?                                                | Uploads large files in parts (required for >5GB objects).                           |
| S3 vs EBS vs EFS?                                                | S3 = object; EBS = block (single instance); EFS = shared file system.               |

|                                       |                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------|
| S3 Intelligent-Tiering?               | Auto-moves objects between access tiers based on usage.                       |
| S3 Object Lock vs Glacier Vault Lock? | Object Lock = WORM (immutable); Vault Lock = compliance policy lock.          |
| How to optimize S3 costs?             | Lifecycle policies, Intelligent-Tiering, delete incomplete multipart uploads. |

---

#### 4. Networking (VPC, Subnets, Gateways)

| Question                             | Answer                                                                  |
|--------------------------------------|-------------------------------------------------------------------------|
| What is VPC?                         | Logically isolated virtual network in AWS.                              |
| Public vs Private Subnet?            | Public has IGW route (internet access); private does not.               |
| Internet Gateway vs NAT Gateway?     | IGW = bidirectional internet; NAT = outbound only from private subnets. |
| NAT Gateway vs NAT Instance?         | NAT Gateway = managed, HA; NAT Instance = EC2-based, manual HA.         |
| Route Table?                         | Defines routing rules for traffic within VPC.                           |
| Security Group vs NACL?              | SG = stateful, instance-level; NACL = stateless, subnet-level.          |
| VPC Peering?                         | Private connection between two VPCs (no transitive routing).            |
| Transit Gateway?                     | Hub connecting multiple VPCs, VPNs, Direct Connect.                     |
| VPC Endpoint (Gateway vs Interface)? | Gateway = S3/DynamoDB; Interface = PrivateLink for other services.      |

|                                  |                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------|
| PrivateLink?                     | Securely expose services to other VPCs without internet.                        |
| Direct Connect?                  | Dedicated network connection from on-prem to AWS.                               |
| VPN vs Direct Connect?           | VPN = encrypted over internet; Direct Connect = dedicated, higher bandwidth.    |
| Bastion Host vs Session Manager? | Bastion = jump server via SSH; Session Manager = secure shell without SSH keys. |
| VPC Flow Logs?                   | Captures traffic metadata for debugging (accepted/rejected flows).              |
| Elastic IP?                      | Static public IPv4 address for instances.                                       |
| CIDR Block Sizing?               | /16 = largest VPC (65,536 IPs); /28 = smallest subnet (16 IPs).                 |

---

## 5. IAM (Identity & Access Management)

| Question                                 | Answer                                                                    |
|------------------------------------------|---------------------------------------------------------------------------|
| What is IAM?                             | Manages users, roles, and permissions in AWS.                             |
| User vs Group vs Role?                   | User = person; Group = collection of users; Role = temporary credentials. |
| IAM Policy Structure?                    | JSON with Effect, Action, Resource, Condition.                            |
| Least Privilege Principle?               | Grant minimum permissions required for tasks.                             |
| Identity-based vs Resource-based Policy? | Identity = who can act; Resource = what allows access.                    |
| STS AssumeRole?                          | Temporary credentials for cross-account or service access.                |



|                                       |                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------|
| IAM Roles vs Resource-based Policies? | Roles = assume credentials; Resource = direct access (e.g., S3 bucket policy). |
| MFA?                                  | Two-factor authentication for enhanced security.                               |
| AWS Organizations?                    | Centralized multi-account management with SCPs.                                |
| SCP (Service Control Policy)?         | Restricts actions across organization accounts (guardrails).                   |
| Permission Boundaries?                | Max permissions a user/role can have (even with explicit allow).               |
| IAM Access Analyzer?                  | Identifies resources shared with external entities.                            |
| Cross-Account Access?                 | Use IAM roles with trust policies.                                             |
| IAM Policy Evaluation Logic?          | Explicit Deny > Explicit Allow > Default Deny.                                 |

---

## 6. Storage & Databases

| Question               | Answer                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------|
| EBS vs Instance Store? | EBS = persistent; Instance Store = ephemeral (lost on stop).                               |
| EBS Volume Types?      | gp3/gp2 = SSD (general); io2/io1 = provisioned IOPS; st1 = HDD throughput; sc1 = cold HDD. |
| EBS Snapshots?         | Incremental backups stored in S3 (restore to any AZ).                                      |
| EBS vs EFS?            | EBS = block (single instance); EFS = shared NFS file system.                               |

|                                    |                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------|
| EFS vs FSx?                        | EFS = Linux NFS; FSx = Windows (FSx for Windows) or high-perf (FSx for Lustre). |
| What is RDS?                       | Managed relational database (MySQL, PostgreSQL, Oracle, SQL Server, MariaDB).   |
| Multi-AZ vs Read Replica?          | Multi-AZ = HA (sync failover); Read Replica = read scaling (async).             |
| RDS Backup vs Snapshot?            | Backup = automated daily; Snapshot = manual point-in-time.                      |
| Aurora vs RDS?                     | Aurora = 5x MySQL, 3x PostgreSQL performance, auto-scaling storage.             |
| Aurora Global Database?            | Cross-region read replicas with <1s replication lag.                            |
| Aurora Serverless?                 | Auto-scaling database with pay-per-request pricing.                             |
| DynamoDB?                          | Fully managed NoSQL database with single-digit ms latency.                      |
| Partition Key vs Sort Key?         | Partition Key = distribution; Sort Key = ordering within partition.             |
| DynamoDB Streams?                  | Captures item changes for event-driven processing.                              |
| DAX (DynamoDB Accelerator)?        | In-memory cache for microsecond latency.                                        |
| DynamoDB On-Demand vs Provisioned? | On-Demand = auto-scaling; Provisioned = pre-defined capacity.                   |
| Global Tables?                     | Multi-region, active-active DynamoDB replication.                               |

|                                   |                                                                        |
|-----------------------------------|------------------------------------------------------------------------|
| DynamoDB vs RDS?                  | DynamoDB = NoSQL, horizontal scaling; RDS = SQL, vertical scaling.     |
| Redshift?                         | Data warehouse for OLAP analytics (petabyte-scale).                    |
| ElastiCache (Redis vs Memcached)? | Redis = persistence, replication, pub/sub; Memcached = simple caching. |

---

## 7. Serverless (Lambda, SQS, SNS, API Gateway)

| Question                   | Answer                                                                   |
|----------------------------|--------------------------------------------------------------------------|
| What is Lambda?            | Run code without managing servers (event-driven, auto-scaling).          |
| Cold Start?                | Initial delay when Lambda execution environment is created.              |
| How to reduce cold starts? | Provisioned concurrency, smaller packages, compiled languages (Go/Java). |
| Lambda Timeout Limit?      | 15 minutes maximum.                                                      |
| Lambda Memory & CPU?       | Memory allocates proportional CPU (128MB-10GB).                          |
| Lambda Layers?             | Shared libraries, dependencies, or custom runtimes.                      |
| Lambda Triggers?           | S3, SQS, SNS, API Gateway, EventBridge, DynamoDB Streams, Kinesis.       |
| SQS vs SNS?                | SQS = queue (pull, decoupling); SNS = pub/sub (push, fanout).            |
| SQS Standard vs FIFO?      | Standard = at-least-once, high throughput; FIFO = exactly-once, ordered. |

|                           |                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------|
| Dead Letter Queue (DLQ)?  | Stores failed messages for debugging.                                                |
| Visibility Timeout (SQS)? | Time a message is hidden after being received (prevents duplicate processing).       |
| SNS Fanout?               | Publish once, deliver to multiple subscribers (SQS, Lambda, HTTP).                   |
| API Gateway?              | Managed service to create, publish, and secure APIs.                                 |
| API Gateway Throttling?   | Default 10,000 RPS with burst; configurable per stage/method.                        |
| EventBridge?              | Serverless event bus for app integration (successor to CloudWatch Events).           |
| Step Functions?           | Orchestrates workflows with visual state machines (retry, parallel, error handling). |
| Lambda vs Fargate?        | Lambda = stateless, <15min; Fargate = containerized, long-running.                   |

---

## 8. Containers & DevOps

| Question             | Answer                                                            |
|----------------------|-------------------------------------------------------------------|
| ECS vs EKS?          | ECS = AWS-native containers; EKS = managed Kubernetes.            |
| Fargate?             | Serverless compute for ECS/EKS (no EC2 management).               |
| ECS Task vs Service? | Task = running container; Service = maintains desired task count. |
| Task Definition?     | Blueprint defining container image, CPU, memory, IAM role.        |

|                                   |                                                                               |
|-----------------------------------|-------------------------------------------------------------------------------|
| ALB vs NLB vs GWLB?               | ALB = HTTP/S (L7); NLB = TCP/UDP (L4); GWLB = firewall/IDS (L3).              |
| Target Groups?                    | Routes traffic to registered targets (instances, IPs, Lambda).                |
| Health Checks?                    | Determines target availability (HTTP/TCP probes).                             |
| CodePipeline?                     | Automates CI/CD workflows.                                                    |
| CodeBuild?                        | Compiles, tests, and packages code.                                           |
| CodeDeploy?                       | Automates deployments (EC2, Lambda, ECS) with rollback.                       |
| Deployment Strategies?            | Blue/Green, Rolling, Canary, All-at-once.                                     |
| CloudFormation?                   | Infrastructure-as-Code (IaC) using JSON/YAML templates.                       |
| CloudFormation Stack vs StackSet? | Stack = single region; StackSet = multi-region/account.                       |
| CDK vs CloudFormation?            | CDK = code-based (TypeScript/Python); CloudFormation = declarative templates. |
| Terraform vs CloudFormation?      | Terraform = multi-cloud; CloudFormation = AWS-native.                         |
| Systems Manager (SSM)?            | Manages EC2/on-prem servers (patching, configuration, Session Manager).       |

---

## 9. Security & Monitoring

| Question    | Answer                                      |
|-------------|---------------------------------------------|
| CloudWatch? | Monitors metrics, logs, and creates alarms. |

|                                     |                                                                         |
|-------------------------------------|-------------------------------------------------------------------------|
| CloudWatch vs CloudTrail?           | CloudWatch = performance monitoring; CloudTrail = API audit logs.       |
| CloudTrail?                         | Records all API calls for auditing and compliance.                      |
| AWS Config?                         | Tracks resource configuration changes and compliance.                   |
| AWS Inspector?                      | Automated security assessment for EC2, containers, Lambda.              |
| GuardDuty?                          | Threat detection using ML (monitors VPC Flow Logs, DNS, CloudTrail).    |
| Security Hub?                       | Centralizes security findings across AWS services.                      |
| Macie?                              | Discovers and protects sensitive data in S3 (PII, credentials).         |
| KMS (Key Management Service)?       | Manages encryption keys (symmetric/asymmetric).                         |
| KMS vs CloudHSM?                    | KMS = managed multi-tenant; CloudHSM = dedicated single-tenant HSM.     |
| Encryption at Rest vs Transit?      | Rest = storage encryption (KMS); Transit = TLS/SSL in motion.           |
| Secrets Manager vs Parameter Store? | Secrets = auto-rotation; Parameter Store = free config storage.         |
| IAM Access Analyzer?                | Identifies resources accessible from outside your account.              |
| WAF (Web Application Firewall)?     | Protects against SQL injection, XSS, rate-limiting.                     |
| Shield Standard vs Advanced?        | Standard = free DDoS protection; Advanced = enhanced + cost protection. |

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| Trusted Advisor? | Best practice recommendations (cost, security, performance, fault tolerance). |
|------------------|-------------------------------------------------------------------------------|

## 10. Architecture & Design

| Question                            | Answer                                                                                                    |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| High Availability?                  | Multi-AZ, Load Balancers, Auto Scaling, health checks.                                                    |
| Fault Tolerance?                    | System continues operating despite component failures (self-healing).                                     |
| Disaster Recovery Strategies?       | Backup/Restore < Pilot Light < Warm Standby < Multi-Site (increasing RTO/RPO).                            |
| RTO vs RPO?                         | RTO = recovery time; RPO = data loss tolerance.                                                           |
| Multi-AZ vs Multi-Region?           | AZ = within region HA; Region = geographic DR.                                                            |
| Auto Scaling Group?                 | Adjusts EC2 instance count based on demand (target tracking, step scaling).                               |
| Well-Architected Framework Pillars? | Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. |
| Elastic Beanstalk?                  | PaaS for deploying apps without managing infrastructure.                                                  |
| Decoupling Microservices?           | Use SQS, SNS, EventBridge, API Gateway.                                                                   |
| Caching Strategies?                 | CloudFront (edge), ElastiCache (app), DAX (DynamoDB), API Gateway cache.                                  |
| CDN in AWS?                         | CloudFront for low-latency content delivery.                                                              |

|                            |                                                                        |
|----------------------------|------------------------------------------------------------------------|
| Event-Driven Architecture? | Lambda + S3/SNS/SQS/EventBridge for asynchronous processing.           |
| Saga Pattern?              | Distributed transactions using Step Functions or compensating actions. |
| Strangler Fig Pattern?     | Gradually replace legacy systems by routing traffic to new services.   |

## 11. Advanced / Tricky

| Question                     | Answer                                                                                  |
|------------------------------|-----------------------------------------------------------------------------------------|
| S3 Consistency?              | Strong read-after-write consistency for all operations.                                 |
| Lambda Concurrency Limits?   | 1,000 concurrent executions per region (soft limit, can increase).                      |
| Cross-Account Access?        | IAM roles with trust policies or resource-based policies.                               |
| VPC Flow Logs Use Cases?     | Troubleshoot connectivity, detect security threats, audit traffic.                      |
| Throttling in DynamoDB?      | Exceeding provisioned throughput (use on-demand or exponential backoff).                |
| Handling traffic spikes?     | ALB + Auto Scaling + CloudFront + caching.                                              |
| Data migration to AWS?       | Snowball (TB-PB), DataSync (online sync), Transfer Family (SFTP).                       |
| RDS performance issues?      | Slow queries (Performance Insights), read replicas, parameter tuning, upgrade instance. |
| Global latency optimization? | CloudFront, Global Accelerator, Route 53 latency routing.                               |



|                                   |                                                                     |
|-----------------------------------|---------------------------------------------------------------------|
| Lambda vs Step Functions?         | Lambda = single function; Step Functions = orchestrate workflows.   |
| How to debug Lambda failures?     | CloudWatch Logs, X-Ray tracing, DLQs.                               |
| DynamoDB hot partition?           | Use composite partition keys, burst capacity, or on-demand mode.    |
| S3 eventual consistency (legacy)? | Old model (before Dec 2020) had eventual for overwrites—now strong. |
| ECS vs EKS trade-offs?            | ECS = simpler, AWS-native; EKS = Kubernetes portability.            |
| CloudFormation Drift Detection?   | Identifies manual changes to stack resources.                       |

---

## 12. Scenario-Based (Senior-Level)

| Scenario                        | Answer                                               |
|---------------------------------|------------------------------------------------------|
| Sudden traffic spike?           | ALB + Auto Scaling + CloudFront caching.             |
| Large file uploads (GB-TB)?     | S3 Transfer Acceleration + multipart upload.         |
| Real-time analytics?            | Kinesis Data Streams → Lambda → DynamoDB/Redshift.   |
| Process millions of messages?   | SQS + Lambda with batch processing or ECS workers.   |
| Cost optimization for dev/test? | Stop/start schedules, Spot Instances, Savings Plans. |
| 1TB on-prem migration?          | Snowball (offline) or DataSync (online).             |
| Accidental EC2 deletion?        | Restore from AMI or termination protection.          |

|                                      |                                                                                              |
|--------------------------------------|----------------------------------------------------------------------------------------------|
| API rate limiting?                   | API Gateway throttling + usage plans.                                                        |
| Detect unusual IAM activity?         | CloudTrail + GuardDuty + EventBridge alerts.                                                 |
| Multi-account governance?            | AWS Organizations + SCPs + CloudFormation StackSets.                                         |
| Lambda running out of memory?        | Increase memory (also increases CPU), optimize code, use Lambda layers.                      |
| RDS running out of storage?          | Enable storage auto-scaling or upgrade instance.                                             |
| S3 costs too high?                   | Lifecycle policies, Intelligent-Tiering, delete incomplete uploads, analyze S3 Storage Lens. |
| Application slow during peak hours?  | Add read replicas, ElastiCache, CloudFront, optimize queries.                                |
| Need zero downtime deployment?       | Blue/Green with ALB target groups or Canary with CodeDeploy.                                 |
| Compliance requires audit logs?      | CloudTrail + S3 bucket with Object Lock.                                                     |
| Database connection pool exhaustion? | Use RDS Proxy for connection pooling.                                                        |
| Lambda cold starts affecting UX?     | Provisioned concurrency or use Fargate for long-running.                                     |
| Need to process data across regions? | S3 CRR + Lambda + EventBridge or Step Functions.                                             |

---

### 13. Cost Optimization (Critical for Senior Role)

| Question | Answer |
|----------|--------|
|----------|--------|

|                                      |                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------|
| Top cost optimization strategies?    | Reserved/Spot/Savings Plans, right-sizing, S3 lifecycle, auto-scaling, delete unused resources. |
| Cost Allocation Tags?                | Track costs by project, team, environment using tags.                                           |
| AWS Cost Explorer?                   | Visualizes spending patterns and forecasts.                                                     |
| AWS Budgets?                         | Sets spending alerts and thresholds.                                                            |
| Savings Plans vs Reserved Instances? | Savings Plans = flexible (compute/EC2/Fargate); Reserved = specific instance type.              |
| Spot Instances best practices?       | Use for stateless, fault-tolerant workloads with Spot Fleet or ASG.                             |
| S3 Storage Lens?                     | Organization-wide visibility into S3 usage and cost optimization.                               |
| Compute Optimizer?                   | Recommends optimal EC2/Lambda/EBS configurations.                                               |
| Right-sizing?                        | Match instance type to workload needs (CPU/memory/network).                                     |

---

#### 14. Troubleshooting (Must-Know for Senior Role)

| Problem                   | Solution                                                                            |
|---------------------------|-------------------------------------------------------------------------------------|
| EC2 instance won't start? | Check instance limits, subnet capacity, AMI availability, EBS snapshot status.      |
| Cannot connect to RDS?    | Verify security groups, NACLs, route tables, DB subnet group, public accessibility. |
| S3 403 Access Denied?     | Check bucket policy, IAM policy, ACLs, Block Public Access settings.                |

|                                      |                                                                            |
|--------------------------------------|----------------------------------------------------------------------------|
| Lambda timing out?                   | Increase timeout, optimize code, check VPC NAT Gateway for outbound calls. |
| High DynamoDB read/write throttling? | Enable auto-scaling, use on-demand mode, or refactor partition key.        |
| ALB returning 503 errors?            | Check target health, security groups, ASG capacity, connection draining.   |
| VPC peering not working?             | Verify route tables, no CIDR overlap, security groups allow traffic.       |
| CloudFormation stack stuck?          | Check resource dependencies, IAM permissions, service limits.              |

-----

GCP

(Note: Used default font, to preserve text style | Refactored with GPT)

## Table of Contents

---

1. GCP Philosophy & Overview
  2. Global Infrastructure — Regions, Zones, Tenancy
  3. Billing & Cost Management
  4. IAM & Security
  5. Networking
  6. Compute — VMs, Containers, Serverless
  7. Storage
  8. Databases
  9. Messaging & Async Processing
  10. Data & Analytics Platform
  11. DevOps — CI/CD, Artifact Registry
  12. Frontend & Mobile — Firebase
  13. API Management
  14. AI & Machine Learning — Vertex AI
  15. Logging — Where Logs Go
  16. Monitoring, Dashboards & Alerts
  17. Latency Considerations
  18. Running Trino on GCP
  19. AWS → GCP Service Map
- 

## 1. GCP Philosophy & Overview

---

Google Cloud Platform (GCP) is Google's public cloud. It runs the same infrastructure that powers Google Search, YouTube, and Gmail. Three things distinguish GCP from AWS:

**Global VPC:** a single VPC spans all regions. In AWS a VPC is regional; in GCP it is global. This simplifies multi-region architecture significantly.

**Best-in-class data services:** BigQuery (serverless warehouse), Bigtable (wide-column NoSQL at extreme scale), Dataflow (managed Apache Beam) are unmatched by AWS equivalents in managed simplicity.

**Kubernetes origin:** Google invented Kubernetes and GKE is the most mature managed Kubernetes offering.

### GCP Console & CLI

| Tool | Purpose |
|------|---------|
|------|---------|

|                                                                         |                                                                                    |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <a href="https://console.cloud.google.com">console.cloud.google.com</a> | Web UI for all services                                                            |
| <code>gcloud</code> CLI                                                 | Most operations — VMs, GKE, IAM, networking                                        |
| <code>gsutil</code> / <code>gcloud storage</code>                       | GCS bucket operations ( <code>gcloud storage</code> is the modern replacement)     |
| <code>bq</code> CLI                                                     | BigQuery operations                                                                |
| <code>kubectl</code>                                                    | Kubernetes (configured by <code>gcloud container clusters get-credentials</code> ) |
| Terraform                                                               | Infrastructure-as-code; GCP provider is first-class                                |

## Project Hierarchy

Everything in GCP lives inside a **Project**. Projects are the primary billing and access boundary.

Organisation (companyABC.com)

└─ Folder: Engineering

└─ Folder: Data Platform

└─ Project: companyABC-platform-prod

└─ Project: companyABC-platform-staging

└─ Project: companyABC-platform-dev

A resource (VM, bucket, database) always belongs to exactly one project. IAM policies, billing, and quotas are all scoped to the project.

## 2. Global Infrastructure — Regions, Zones, Tenancy

### Regions

A **region** is a geographic area containing multiple data centres. Pick a region close to your users and services. All resources in a project should be in the same region unless there is a specific multi-region requirement, to minimise latency and egress cost.

Common regions for companyABC:

- `asia-south1` — Mumbai (lowest latency for regional services)
- `us-central1` — Iowa (default for many GCP products)
- `eu-west1` — Belgium

### Zones

A **zone** is an isolated data centre inside a region. Zones are named `<region>-a`, `<region>-b`, `<region>-c`.

| Deployment                       | Zone strategy               | Trade-off                      |
|----------------------------------|-----------------------------|--------------------------------|
| Dev / test                       | Single zone                 | Cheaper, simpler               |
| Production stateless (GKE, APIs) | Multi-zone                  | HA, no single point of failure |
| Production stateful (DBs, disks) | Multi-zone with replication | Higher cost, required for SLA  |

### Tenancy

| Mode                     | Description                                                                            | Use case                             |
|--------------------------|----------------------------------------------------------------------------------------|--------------------------------------|
| <b>Shared (default)</b>  | VMs run on hardware shared with other GCP customers; hardware-level isolation enforced | All standard workloads               |
| <b>Sole-tenant nodes</b> | Dedicated physical server; your VMs only                                               | Compliance, Windows BYOL licensing   |
| <b>Confidential VMs</b>  | RAM encrypted in-use via AMD SEV                                                       | Regulated data, sensitive processing |

## 3. Billing & Cost Management

### How GCP Bills You

- Resources are billed to a **Billing Account** linked to a payment method.
- Each **Project** is associated with one Billing Account. Cost reports are per project.
- Billing is **per second** for most compute resources (after a 1-minute minimum).

### Key Cost Drivers

| Service            | Billed on                   |
|--------------------|-----------------------------|
| Compute Engine VMs | Per second, by machine type |



|                |                                                                    |
|----------------|--------------------------------------------------------------------|
| GKE            | No control plane charge; node VMs billed as Compute Engine         |
| Cloud Storage  | GB stored/month + per-operation + egress                           |
| BigQuery       | TB scanned (on-demand) or slot-hours (flat-rate)                   |
| Cloud SQL      | Instance uptime + storage                                          |
| Bigtable       | Node-hours + storage GB                                            |
| Pub/Sub        | Data volume published and delivered                                |
| Cloud Logging  | First 50 GB/month free, then per GB                                |
| Network egress | Intra-zone: free. Inter-zone: ~\$0.01/GB. Internet: \$0.08–0.12/GB |

### Discount Types

| Discount                      | How it works                                                                                              | Action needed                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Sustained use</b>          | Automatic — the longer a VM runs in a billing month, the more GCP discounts it (up to 30% for full-month) | None                                              |
| <b>Committed use (CUD)</b>    | 1-year or 3-year commitment for a specific machine family; up to 57% off                                  | Purchase in Console under Committed Use Discounts |
| <b>Spot / Preemptible VMs</b> | Up to 91% off; GCP can reclaim with 30s notice                                                            | Use for stateless/batch workloads                 |
| <b>BigQuery flat-rate</b>     | Buy dedicated slots; predictable cost regardless of bytes scanned                                         | Purchase slot reservations                        |

### Cost Controls

**Budgets & Alerts:** Go to **Billing** → **Budgets & Alerts** → **Create Budget**. Set thresholds at 50%, 90%, 100%. GCP emails/Pub/Sub notifies at each threshold. Set these up before any resources are created.

**Labels:** Tag every resource with labels. Labels flow into billing exports.

```
gcloud compute instances create my-vm \
```

```
--labels=env=prod,team=platform,service=trino
```

**Billing Export to BigQuery:** enables SQL analysis of all spend. Go to **Billing** → **Billing Export** → **BigQuery Export** → **Enable**. Creates a dataset with a detailed cost table you can query.

```
-- Top 10 most expensive services last 30 days
```

```
SELECT service.description, ROUND(SUM(cost), 2) AS total_cost
FROM `billing_dataset.gcp_billing_export_v1_*.`
WHERE DATE(_PARTITIONTIME) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30
DAY) GROUP BY 1 ORDER BY 2 DESC LIMIT 10;
```

---

## 4. IAM & Security

### IAM Fundamentals

IAM controls *who* can do *what* on *which* resource.

Principal + Role = Binding applied to Resource (or hierarchy level)

How IAM inheritance flows down the hierarchy:

Organisation (companyABC.com)

| roles/billing.viewer → applies to ALL projects below

||

| — Folder: Engineering

| roles/viewer → applies to all projects in this folder

||

| — Project: companyABC-platform-prod

| roles/bigquery.dataEditor → only this project

||

| — Resource: gs://companyABC-data-lake (GCS bucket)

| roles/storage.objectViewer → only this bucket

**Key rule:** A binding at a higher level is inherited downward. A binding at the resource level applies only to that resource. There is no DENY — any matching ALLOW anywhere in the hierarchy grants access.

### Principals (Who)

| Type            | Description                                                              |
|-----------------|--------------------------------------------------------------------------|
| Google Account  | Individual user — <code>user_abcd@companyABC.com</code>                  |
| Service Account | Non-human identity for apps, VMs, pipelines — equivalent to AWS IAM Role |

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| <b>Google Group</b>     | Collection of users; grant a role once, all members inherit |
| <b>Workspace Domain</b> | Everyone at <code>@companyABC.com</code>                    |
| <b>allUsers</b>         | Public internet (avoid unless intentionally public)         |

## Roles (What)

| Type                                                                                | Description                  | When to use                  |
|-------------------------------------------------------------------------------------|------------------------------|------------------------------|
| <b>Primitive</b> ( <code>owner</code> , <code>editor</code> , <code>viewer</code> ) | Broad legacy roles           | Never in production          |
| <b>Predefined</b>                                                                   | Per-service curated roles    | Always prefer these          |
| <b>Custom</b>                                                                       | You define exact permissions | Minimal-privilege automation |

**Hierarchy inheritance:** a role binding at the Organisation level applies to all Folders, Projects, and resources beneath it. Always assign the minimum needed role at the lowest level.

## IAM Access Logic — Critical Nuances

### 1. Implicit Deny (default state)

Every principal starts with zero access to every resource. GCP denies all requests unless an explicit ALLOW binding exists somewhere in the hierarchy that covers the action. You do not need to write a deny rule — the absence of an allow rule IS the deny.

`No binding exists for principal X on resource Y`

`→ X is DENIED by default, even if X is a project owner on a different project`

### 2. Allowlist Approach — you must explicitly grant

Access is additive. You add roles to grant access. You never "remove deny" — instead, you simply don't add the allow.

`To grant: add a role binding`

`To revoke: remove the role binding`

`No matching allow anywhere → implicit deny kicks in automatically`

### 3. Deny Policies — override allows (GCP IAM Deny API)

GCP introduced explicit Deny Policies (in addition to the default implicit deny). A deny policy explicitly blocks a principal from performing an action even if an allow binding exists. Evaluation order:

Request arrives

↓

Check Deny Policies first → if DENY matches → blocked immediately

↓ (no deny matched)

Check Allow bindings → if ALLOW matches → permitted

↓ (no allow matched)

Implicit Deny → blocked

Use deny policies for: blocking an entire team from a sensitive operation (e.g., deny `compute.instances.delete` to all engineers except ops), mandatory guard rails that should survive even if someone accidentally grants a broader role.

#### 4. Most permissive allow wins — no conflict between allows

If a principal has three different allow bindings that all apply to the same resource, ALL grants are additive. The broadest one wins. There is no "specific beats general" — unlike AWS, GCP does not have resource-based allow/deny conflict resolution between allows.

#### 5. IAM propagation delay

IAM changes are eventually consistent. After `gcloud projects add-iam-policy-binding`, the change may take **60 seconds to 7 minutes** to propagate globally. Do not test permissions immediately after granting — wait or build a retry loop.

#### 6. Conditional IAM bindings

You can add conditions to a binding so it only applies in certain contexts:

```
# Grant storage access only during business hours
```

```
gcloud projects add-iam-policy-binding my-project \
```

```
--member="user:contractor@example.com" \
```

```
--role="roles/storage.objectViewer" \
```

```
--condition='expression=request.time.getHours("Asia/Kolkata") >= 9  
&& request.time.getHours("Asia/Kolkata") < 18,title=business-hours'
```

```
# Conditions can also restrict by resource name, IP, device security  
level
```

#### 7. allUsers / allAuthenticatedUsers — public access principals

`allUsers` = the entire internet (no authentication required).

`allAuthenticatedUsers` = anyone with a Google account. Never grant these on non-public resources. GCP's org policy

`constraints/iam.allowedPolicyMemberDomains` can block this at the organisation level.

**Important:** GCP IAM — the old statement "no explicit DENY, only ALLOW" was true before IAM Deny Policies launched. Today: implicit deny is the default, explicit deny policies exist and override allows, and allows are still additive among themselves.

### Service Accounts

Every application, VM, and pipeline should run as a dedicated service account with minimal permissions.

#### # Create

```
gcloud iam service-accounts create api-sa \
  --display-name="Data Platform API Service Account"
```

#### # Grant a role

```
gcloud projects add-iam-policy-binding companyABC-platform-prod \
  --member="serviceAccount:api-sa@companyABC-platform-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

#### # Attach to a VM

```
gcloud compute instances create my-vm --service-account=api-sa@...
```

**Workload Identity (GKE):** link a Kubernetes ServiceAccount to a GCP ServiceAccount — pods get credentials automatically, no key files needed. Always use this over downloading JSON keys.

### Production Use Cases — Service Accounts & IAM

#### Scenario 1: Trino worker pods reading from GCS

Without Workload Identity, you'd download a JSON key file, put it in the pod, and hope it never rotates or leaks. With Workload Identity:

1. Create a GCP service account `trino-gsa@project.iam.gserviceaccount.com` and grant it `roles/storage.objectViewer` on the data bucket.
2. Create a Kubernetes service account `trino-sa` in the `trino` namespace.
3. Link the two: `gcloud iam service-accounts add-iam-policy-binding trino-gsa@... --member="serviceAccount:project.svc.id.goog[trino/trino-sa]" --role="roles/iam.workloadIdentityUser"`
4. Annotate the K8s SA: `iam.gke.io/gcp-service-account: trino-gsa@project.iam.gserviceaccount.com`

Now any pod using `trino-sa` can read from GCS using the GCP SDK — no credentials in code, no JSON files, token auto-rotated every hour.

### Scenario 2: VM running a batch job writing to GCS

A Compute Engine VM running a nightly export job needs to write Parquet files to `gs://companyABC-data/raw/`. Instead of putting credentials in the startup script:

1. Create a dedicated SA:  
`batch-export-sa@project.iam.gserviceaccount.com`
2. Grant `roles/storage.objectCreator` on the specific bucket only — not the whole project.
3. Launch the VM with `--service-account=batch-export-sa@...`

The VM's metadata server provides the token. Python's `google-cloud-storage` library picks it up automatically — no `GOOGLE_APPLICATION_CREDENTIALS` env var needed.

### Scenario 3: CI/CD pipeline (Cloud Build) deploying to GKE

Cloud Build runs as the Cloud Build service account by default. Grant it only what it needs:

- `roles/container.developer` — to deploy to GKE
- `roles/artifactregistry.reader` — to pull images

Never grant `roles/owner` to a CI service account. Principle of least privilege: if the build system is compromised, the blast radius stays small.

### Secret Manager

Managed secrets store for API keys, passwords, certificates, connection strings. Equivalent to AWS Secrets Manager.

# Store a secret

```
echo -n "my-db-password" | gcloud secrets create db-password  
--data-file=-
```

# Add a new version

```
echo -n "new-password" | gcloud secrets versions add db-password  
--data-file=-
```

# Read in application (or in a startup script)

```
gcloud secrets versions access latest --secret=db-password
```

# Automatic rotation can be configured via Pub/Sub notifications

Access in code (Python):

```
from google.cloud import secretmanager  
client = secretmanager.SecretManagerServiceClient()
```

```
response =  
client.access_secret_version(name="projects/my-project/secrets/db-password/versions/latest")  
password = response.payload.data.decode("UTF-8")
```

**Secret versioning:** every update creates a new version. Old versions are retained until you disable/destroy them. Always reference `latest` in non-pinned environments; pin to a version number for production deployments.

## Production Use Cases — Secret Manager

### Scenario 1: GKE pod reading a database password at startup

Store the Cloud SQL password in Secret Manager. Use the External Secrets Operator (ESO) to sync it into a Kubernetes Secret automatically — so you never manually run `kubectl create secret` and nothing sensitive ever lives in your Helm values or Git.

```
# ExternalSecret CRD – ESO watches this and creates a K8s Secret  
apiVersion: external-secrets.io/v1beta1  
kind: ExternalSecret  
metadata:  
  name: db-password  
  namespace: backend  
spec:  
  refreshInterval: 1h  
  secretStoreRef:  
    name: gcp-secret-store # points to your GCP project  
    kind: ClusterSecretStore  
  target:  
    name: db-password # name of the K8s Secret created  
  data:  
    - secretKey: password  
      remoteRef:  
        key: prod-db-password # Secret Manager secret name
```

The pod mounts the K8s Secret as an env var. When you rotate the secret in GCP, ESO re-syncs within the refresh interval — no pod restart needed unless the app requires it.

### Scenario 2: Cloud Run service reading an API key

Cloud Run services can reference Secret Manager secrets directly in the console — no code needed to fetch them. GCP injects the secret value as an environment variable at container startup:

```
gcloud run deploy my-api \
```

```
--set-secrets="STRIPE_API_KEY=stripe-key:latest"
# At runtime: os.environ["STRIPE_API_KEY"] is populated
automatically
```

### Scenario 3: Automatic rotation with Pub/Sub notification

Configure a Secret Manager rotation schedule. When a secret version is added, Secret Manager publishes to a Pub/Sub topic. A Cloud Function subscribes and updates the downstream system (e.g., updates a Cloud SQL user password). The application reads `latest` — it gets the new value on next restart without any manual intervention.

### Cloud KMS (Key Management Service)

Manage encryption keys for data at rest. GCP encrypts all data at rest by default with Google-managed keys. Use KMS when you need to control the keys yourself (CMEK — Customer-Managed Encryption Keys).

```
gcloud kms keyrings create companyABC-keyring --location=us-central1
gcloud kms keys create companyABC-data-key \
  --location=us-central1 \
  --keyring=companyABC-keyring \
  --purpose=encryption
```

Use CMEK with GCS, BigQuery, Cloud SQL, GKE secrets. Required for some compliance frameworks (ISO 27001, SOC 2).

### Cloud Armor

Web Application Firewall (WAF) and DDoS protection. Sits in front of HTTP(S) Load Balancers. Equivalent to AWS WAF + AWS Shield.

```
gcloud compute security-policies create companyABC-waf-policy \
  --description="WAF policy for data platform APIs"
```

```
# Block a specific IP range
gcloud compute security-policies rules create 1000 \
  --security-policy=companyABC-waf-policy \
  --expression="inIpRange(origin.ip, '198.51.100.0/24')" \
  --action=deny-403
```

```
# Attach to backend service
gcloud compute backend-services update companyABC-api-backend \
  --security-policy=companyABC-waf-policy --global
```

Preconfigured WAF rules: OWASP Top 10, SQLi, XSS, LFI — enable with a single flag.

### Identity-Aware Proxy (IAP)



IAP lets you control access to web applications and VMs based on Google identity — without a VPN. A user must authenticate with their Google account and be in an allowed group before reaching the app. Equivalent to adding authentication in front of an internal app.

Use cases:

- Secure internal dashboards (Grafana, Airflow UI, Trino UI) without exposing them to the internet
- SSH to VMs without opening port 22 to the internet (`gcloud compute ssh` via IAP tunnel)

```
# Enable IAP for a backend service
```

```
gcloud iap web enable --resource-type=backend-services  
--service=my-service
```

```
# SSH to a VM via IAP (no external IP, no firewall rule needed)
```

```
gcloud compute ssh my-vm --tunnel-through-iap
```

### VPC Service Controls

Creates a security perimeter around GCP services. Even if a user has IAM permission to access a BigQuery dataset, if they are outside the perimeter (wrong network, wrong project) they are blocked. Prevents data exfiltration. Use for regulated production environments.

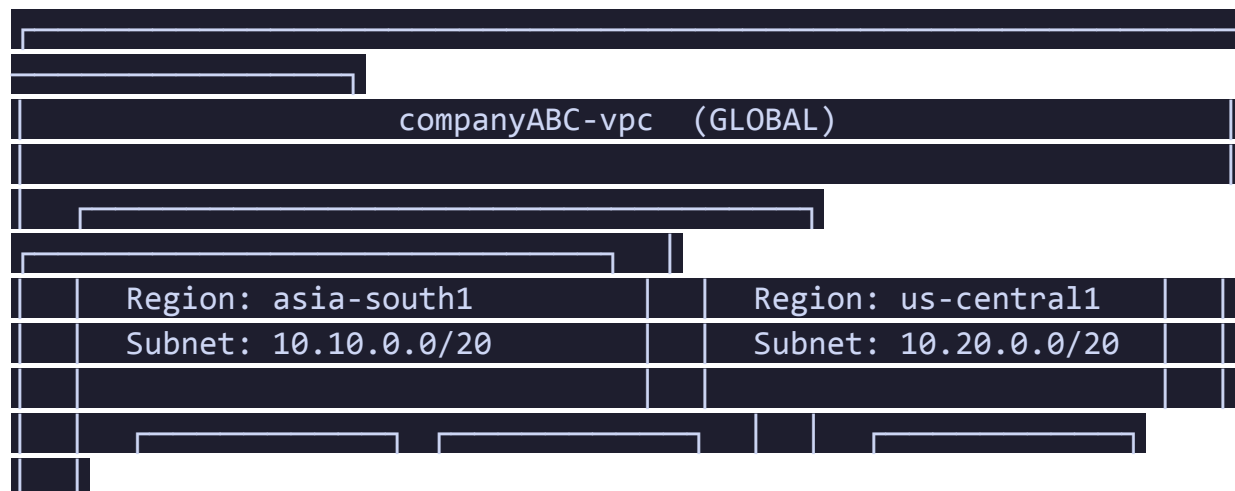
---

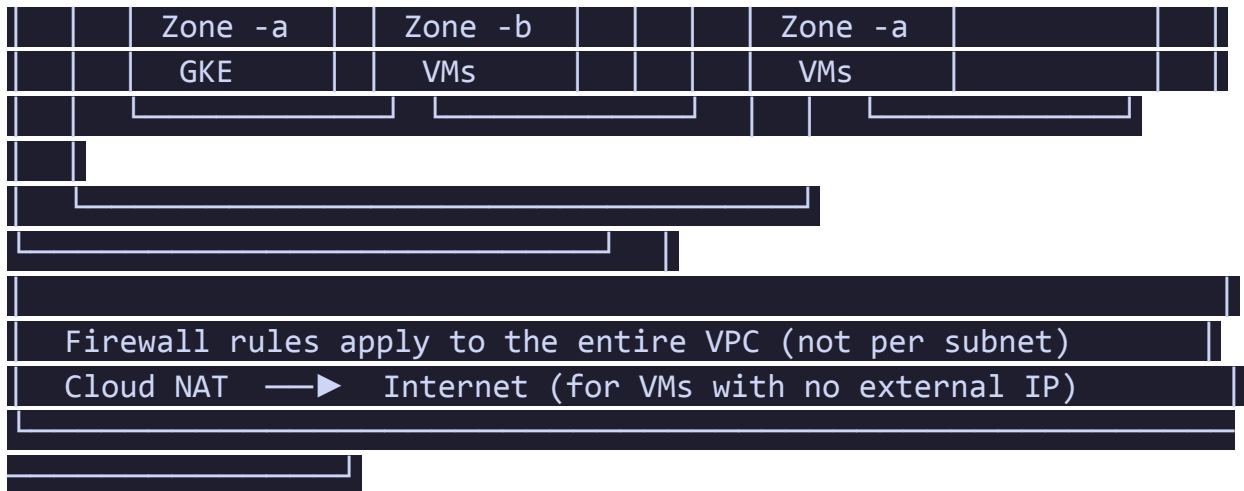
## 5. Networking

### VPC (Virtual Private Cloud)

GCP's VPC is **global** — one VPC spans all regions. This is the biggest networking difference from AWS (where VPCs are regional).

**GCP Networking topology — one VPC, subnets per region, zones within each subnet:**





VPC Peering: companyABC-vpc  $\longleftrightarrow$  platform-vpc (same or different project)

(private IPs only, non-transitive)

```
gcloud compute networks create companyABC-vpc \
```

```
--subnet-mode=custom \
```

```
--bgp-routing-mode=regional
```

## Subnets

Subnets are **regional** (not zonal). A subnet in **asia-south1** covers all zones in that region.

```
gcloud compute networks subnets create subnet-gke-primary \
```

```
--network=companyABC-vpc \
```

```
--region=asia-south1 \
```

```
--range=10.10.0.0/20 \
```

```
--enable-private-ip-google-access # allows VMs without external  
IPs to reach GCP APIs
```

**Secondary IP ranges** are required for GKE — one for Pods, one for Services:

```
gcloud compute networks subnets update subnet-gke-primary \
```

```
--region=asia-south1 \
```

```
--add-secondary-ranges=pod-range=10.20.0.0/14,svc-range=10.24.0.0/20
```

## Firewall Rules

Rules are VPC-wide and target VMs via **network tags** or **service accounts**.

# Allow internal traffic within VPC

```
gcloud compute firewall-rules create allow-internal \
```

```
--network=companyABC-vpc \
```

```
--allow=tcp,udp,icmp \
```

```
--source-ranges=10.0.0.0/8
```

```
# Allow HTTPS from anywhere (for public-facing load balancers)
gcloud compute firewall-rules create allow-https \
  --network=companyABC-vpc \
  --allow=tcp:443 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=https-server
```

```
# Allow SSH via IAP only (no public SSH)
gcloud compute firewall-rules create allow-iap-ssh \
  --network=companyABC-vpc \
  --allow=tcp:22 \
  --source-ranges=35.235.240.0/20 \ # Google's IAP IP range
  --target-tags=ssh-allowed
```

### VPC Peering

Connects two VPCs (in the same or different projects) so resources communicate via internal IPs. No internet traffic. Not transitive — if  $A \leftrightarrow B$  and  $B \leftrightarrow C$ , then A cannot reach C.

```
# From project A
gcloud compute networks peerings create peer-to-platform \
  --network=companyABC-vpc \
  --peer-project=companyABC-platform \
  --peer-network=platform-vpc
```

```
# Project B must create the reverse peering
```

### Shared VPC

One project is the **host** — it owns the VPC and subnets. Other **service projects** attach their resources to the host's subnets. Centralises network management across teams. Recommended for organisations with multiple project teams sharing a network.

### Cloud NAT

VMs without external IPs (recommended for security) need Cloud NAT to reach the internet (package installs, external API calls, pulling Docker images).

```
gcloud compute routers create nat-router --network=companyABC-vpc
--region=asia-south1
gcloud compute routers nats create nat-gateway \
  --router=nat-router \
  --region=asia-south1 \
```

```
--auto-allocate-nat-external-ips \
--nat-all-subnet-ip-ranges
```

## Load Balancing

GCP offers several load balancer types. Choose based on traffic type and whether you need global or regional distribution.

| Load Balancer                       | Layer | Scope    | Use case                                                            |
|-------------------------------------|-------|----------|---------------------------------------------------------------------|
| <b>Global External HTTP(S) LB</b>   | L7    | Global   | Public APIs, web apps — routes to nearest healthy backend worldwide |
| <b>Regional External HTTP(S) LB</b> | L7    | Regional | Same but single region; lower cost                                  |
| <b>Internal HTTP(S) LB</b>          | L7    | Regional | Service-to-service within VPC                                       |
| <b>External TCP/UDP LB</b>          | L4    | Regional | Non-HTTP protocols; gaming, streaming                               |
| <b>Internal TCP/UDP LB</b>          | L4    | Regional | Internal services, databases                                        |
| <b>SSL Proxy LB</b>                 | L4    | Global   | TLS termination for non-HTTP                                        |

**Global HTTP(S) LB** is the standard choice for public APIs and web frontends. It provides: SSL termination, URL-based routing, Cloud CDN integration, Cloud Armor (WAF), and Anycast IP (one IP, globally routed).

### Global HTTP(S) LB — request flow:

User (anywhere in world)

HTTPS request



Anycast IP (Google's edge PoP nearest to user — ~150 locations)

TLS terminated here



Cloud Armor (WAF / DDoS — block malicious requests)



Cloud CDN (cache hit? → return response immediately)

cache miss → continue

URL Map (route by host / path)

api.companyABC.com/v1/\* → backend-v1

api.companyABC.com/v2/\* → backend-v2

Backend Service (health checks, load balancing policy)

NEG / Instance Group (GKE Pod IPs or VM IPs)

Your application Pod / VM

# High-level: create a backend service pointing to a GKE NEG or instance group

```
gcloud compute backend-services create api-backend \
```

```
--protocol=HTTP \
```

```
--health-checks=api-health-check \
```

```
--global
```

```
gcloud compute url-maps create api-url-map
```

```
--default-service=api-backend
```

```
gcloud compute target-https-proxies create api-https-proxy \
```

```
--url-map=api-url-map \
```

```
--ssl-certificates=my-cert
```

```
gcloud compute forwarding-rules create api-forwarding-rule \
```

```
--global \
```

```
--target-https-proxy=api-https-proxy \
```

```
--ports=443
```

**NEG (Network Endpoint Group):** connects the load balancer directly to individual pod IPs in GKE. This is how you expose GKE services via a GCP load balancer instead of a Kubernetes LoadBalancer service.

## Cloud DNS

Managed authoritative DNS. Equivalent to AWS Route 53.

# Create a managed zone

```
gcloud dns managed-zones create companyABC-internal \
  --dns-name=internal.companyABC.com. \
  --visibility=private \
  --networks=companyABC-vpc
```

# Add an A record

```
gcloud dns record-sets create api.internal.companyABC.com. \
  --zone=companyABC-internal \
  --type=A \
  --ttl=300 \
  --rrdatas=10.10.0.5
```

Public and private zones supported. Private zones are only resolvable within the VPC — use for internal service discovery.

### Cloud CDN

CDN caches responses from HTTP(S) Load Balancers at Google's global edge nodes (~150+ locations). Equivalent to AWS CloudFront.

Enable on a backend service:

```
gcloud compute backend-services update my-backend \
  --enable-cdn \
  --global
```

Cache-Control headers from your origin control what is cached and for how long. Use for: static assets (JS, CSS, images), API responses that are safe to cache, frontend builds.

### Cloud Interconnect & VPN

For connecting on-premise infrastructure to GCP:

| Option                        | Bandwidth               | Latency                | Cost | Use case                          |
|-------------------------------|-------------------------|------------------------|------|-----------------------------------|
| <b>Cloud VPN</b>              | Up to 3 Gbps per tunnel | Higher (internet path) | Low  | Dev/test, small data volumes      |
| <b>Dedicated Interconnect</b> | 10/100 Gbps             | Low (private circuit)  | High | Large data transfers, prod hybrid |

|                             |                 |     |      |                                              |
|-----------------------------|-----------------|-----|------|----------------------------------------------|
| <b>Partner Interconnect</b> | 50 Mbps–10 Gbps | Low | M id | Via a network provider, no colocation needed |
|-----------------------------|-----------------|-----|------|----------------------------------------------|

## 6. Compute — VMs, Containers, Serverless

How to choose the right compute option:

Do you need to run containers?

└─ NO → Do you need full OS control / persistent processes?

└─ YES → Compute Engine (VM)

└─ NO → Cloud Functions (event-driven, < 60 min, single purpose)

||

└─ YES → Is it a stateless HTTP service or short job?

└─ YES → Does it need complex networking / sidecars / stateful sets?

└─ YES → GKE

└─ NO → Cloud Run (simpler, scales to zero, no cluster)

└─ NO (needs persistent storage, ordered startup, inter-pod DNS)

└─ GKE with StatefulSet

Need to host a web app / API with zero infra management?

└─ App Engine (push code, GCP handles everything)

|                    | VM   | GKE    | Cloud Run | Cloud Functions | App Engine     |
|--------------------|------|--------|-----------|-----------------|----------------|
| Unit of work       | VM   | Pod    | Container | Function        | App version    |
| Scales to zero     | No   | No     | Yes       | Yes             | Yes (Standard) |
| Min viable effort  | High | Medium | Low       | Very low        | Very low       |
| Custom OS / kernel | Yes  | No     | No        | No              | No             |

|                    |     |                   |    |    |    |
|--------------------|-----|-------------------|----|----|----|
| Stateful workloads | Yes | Yes (StatefulSet) | No | No | No |
|--------------------|-----|-------------------|----|----|----|

### Compute Engine (VMs)

Equivalent to AWS EC2. Full control over the machine.

#### Machine families:

| Family                             | Optimised for              | Examples                                   |
|------------------------------------|----------------------------|--------------------------------------------|
| <code>e2</code>                    | Cost — general purpose     | <code>e2-standard-4</code> (4 vCPU, 16 GB) |
| <code>n2</code> / <code>n2d</code> | Balanced performance       | <code>n2-standard-8</code> (8 vCPU, 32 GB) |
| <code>c2</code> / <code>c3</code>  | Compute — high clock speed | <code>c2-standard-16</code>                |
| <code>m2</code> / <code>m3</code>  | Memory — very large RAM    | <code>m2-ultramem-208</code> (up to 12 TB) |
| <code>a2</code> / <code>g2</code>  | GPU attached               | <code>a2-highgpu-1g</code> (NVIDIA A100)   |
| <code>t2d</code>                   | AMD EPYC — best price/perf | <code>t2d-standard-4</code>                |

#### Disk types attached to VMs:

| Type                           | IOPS               | Use case                          |
|--------------------------------|--------------------|-----------------------------------|
| <code>pd-standard</code> (HDD) | ~0.75 IOPS/GB      | Sequential reads, cold data, logs |
| <code>pd-balanced</code> (SSD) | 6 IOPS/GB          | General purpose — good default    |
| <code>pd-ssd</code> (SSD)      | 30 IOPS/GB         | Databases, high-throughput        |
| <code>pd-extreme</code> (SSD)  | Up to 120k IOPS    | Latency-sensitive databases       |
| Local SSD                      | Highest; ephemeral | Trino/Spark shuffle, temp sort    |

**Preemptible / Spot VMs:** up to 91% cheaper; GCP can reclaim with 30s notice. Use for stateless batch workloads. Not for coordinators, metadata stores, or stateful services.



**Managed Instance Groups (MIGs):** auto-healing, auto-scaling groups of identical VMs — equivalent to AWS Auto Scaling Groups.

```
gcloud compute instances create api-vm-1 \
  --zone=asia-south1-a \
  --machine-type=n2-standard-4 \
  --boot-disk-type=pd-balanced \
  --boot-disk-size=50GB \
  --subnet=subnet-api \
  --no-address \
  --service-account=api-sa@...
```

### Google Kubernetes Engine (GKE)

Managed Kubernetes. Best choice for containerised backend services, data platform components (Trino, Airflow), and anything you want to scale independently.

#### Cluster modes:

| Mode             | Description                             | Use case                              |
|------------------|-----------------------------------------|---------------------------------------|
| <b>Autopilot</b> | GCP manages nodes entirely; pay per Pod | Variable workloads, simpler ops       |
| <b>Standard</b>  | You manage node pools; full control     | Production — precise resource control |

**Node pools:** groups of identical VMs in the cluster. A node is literally a GCP VM — for example, an **n2-standard-8** (8 vCPU, 32 GB RAM, 100 GB SSD). All nodes in a pool are the same machine type. You can have multiple pools with different types — general-purpose for APIs, high-memory for Trino, Spot VMs for batch jobs — and use **nodeSelector** / **nodeAffinity** to schedule workloads to the right pool.

#### Example cluster for the data platform:

```
Pool: general (n2-standard-4, 4 vCPU / 16 GB) → APIs, Airflow, misc services
```

```
Pool: trino (n2-highmem-16, 16 vCPU / 128 GB) → Trino coordinator + workers
```

```
Pool: batch (n2-standard-8 Spot, 8 vCPU / 32 GB) → pipeline jobs, cheaper
```

```
gcloud container clusters create companyABC-cluster \
  --region=asia-south1 \
  --network=companyABC-vpc \
  --subnetwork=subnet-gke-primary \
  --cluster-secondary-range-name=pod-range \
```

```
--services-secondary-range-name=svc-range \
--workload-pool=my-project.svc.id.goog \
--enable-private-nodes \
--master-ipv4-cidr=172.16.0.0/28 \
--enable-autoscaling --min-nodes=1 --max-nodes=10
```

**Workload Identity:** lets GKE pods call GCP APIs (GCS, BigQuery, Pub/Sub) without key files. Link a Kubernetes ServiceAccount to a GCP ServiceAccount and annotate the Kubernetes SA. This is the standard and recommended approach.

### Useful GKE features:

| Feature                         | What it does                                                        |
|---------------------------------|---------------------------------------------------------------------|
| Cluster Autoscaler              | Scales node count up/down based on pending Pods                     |
| Horizontal Pod Autoscaler (HPA) | Scales Pod replicas based on CPU/memory/custom metrics              |
| Vertical Pod Autoscaler (VPA)   | Automatically adjusts Pod resource requests                         |
| Private cluster                 | Nodes have no external IP; API server on private endpoint           |
| GKE Gateway API                 | Modern ingress; integrates with GCP HTTP(S) LB natively             |
| Config Connector                | Manage GCP resources (GCS, Pub/Sub, etc.) from Kubernetes manifests |
| Binary Authorization            | Enforce only signed container images are deployed                   |

## GKE — Critical Nuances

### 1. Node auto-upgrade runs on your nodes by default — set a maintenance window

GKE automatically upgrades node pools to stay within the support window of the cluster version. This involves draining and replacing nodes, which can disrupt running Pods if you have no PodDisruptionBudgets. Set a maintenance window to off-peak hours:

```
gcloud container clusters update companyABC-cluster \
--maintenance-window-start="2026-01-01T20:00:00Z" \ # 1:30am IST
--maintenance-window-end="2026-01-01T23:00:00Z" \ # 4:30am IST
```

```
--maintenance-window-recurrence="FREQ=WEEKLY;BYDAY=SA,SU"
```

## 2. Spot/Preemptible nodes get 30 seconds warning before termination

GKE sends a SIGTERM to Pods on a preemptible node being reclaimed. Your Pod has 30 seconds to finish in-flight work and shut down cleanly. After 30s, it gets SIGKILL. Set `terminationGracePeriodSeconds` appropriately and use `preStop` hooks to drain connections. Never run Trino coordinator, databases, or Airflow scheduler on Spot nodes.

## 3. Cluster Autoscaler has a ~10-minute cooldown before scale-down

CA waits 10 minutes of idle before removing a node. This is to avoid thrashing. You can tune it but it means your cluster may over-provision for up to 10 minutes after a traffic spike ends. Account for this in cost estimates.

## 4. Private cluster API server — configure authorized networks

In a private GKE cluster, the API server has no public IP. To run `kubectl` from your laptop or from CI/CD, you must either:

- Add your IP to authorized networks
- Use Cloud Shell (runs inside GCP's network)
- Use IAP tunnel: `gcloud container clusters get-credentials ... --internal-ip`

## 5. Workload Identity has a small latency on first token fetch

The GKE Metadata Server issues tokens for Workload Identity. The first token fetch after Pod startup can take 1-3 seconds as the metadata server resolves the K8s SA → GCP SA mapping. This is usually invisible but can cause issues in apps that try to access GCP APIs in the first few milliseconds of startup.

## 6. Pod resource requests ≠ actual CPU usage — set them carefully

The scheduler places Pods based on `requests`, not `limits` or actual usage. If your Pods request 4 vCPU but only use 500m, you're wasting ~7 node slots. Use VPA in recommendation mode for 1-2 weeks to get actual usage data before setting final requests.

## Production Use Cases — GKE

### Scenario 1: Data platform stack (Trino + Airflow + monitoring on one cluster)

One GKE cluster hosts the entire data platform using separate node pools and namespaces:

```
Cluster: companyABC-data-cluster (asia-south1)
```

```
||
```

```
| — Node pool: general (n2-standard-4, 4 vCPU 16 GB, 3-10 nodes, autoscaling)
```

```
|
```

```
| Namespace: airflow → Airflow scheduler, workers, webserver
```

```
|
```

```
| Namespace: monitoring → Prometheus, Grafana, Alertmanager
```

```
||
```

```

└─ Node pool: trino (n2-highmem-16, 16 vCPU 128 GB, 2-8 nodes,
autoscaling)
  Namespace: trino → coordinator (1 pod) + workers (3-24
pods)
  Taint: dedicated=trino:NoSchedule (only Trino pods land here)
└─
└─ Node pool: batch (n2-standard-8 Spot, 8 vCPU 32 GB, 0-20
nodes)
  Namespace: pipelines → Spark/dbt jobs, temporary pipeline
pods
  Spot VMs: ~80% cheaper for batch that can handle preemption

```

All pods use Workload Identity – no JSON key files anywhere.  
 Trino workers scale via HPA on CPU; Cluster Autoscaler adds highmem nodes when needed.

### Scenario 2: Zero-downtime deployment of a backend API

A new version of the orders API is ready. The Deployment is configured for rolling updates: `maxUnavailable: 0`, `maxSurge: 1`. Cloud Build pushes the new image to Artifact Registry, triggers Cloud Deploy, which runs `helm upgrade`. Kubernetes starts one new Pod with the new image, waits for readiness probe to pass, then terminates one old Pod — repeated until all old Pods are replaced. Users see zero downtime. If the new version has an issue, `kubectl rollout undo deployment/orders-api` rolls back in ~30 seconds.

### Scenario 3: KEDA scaling pipeline workers based on Pub/Sub backlog

A batch pipeline worker processes application events from Pub/Sub. At 2am the backlog spikes (overnight events accumulate). KEDA watches the Pub/Sub subscription's unacked message count:

```

triggers:
- type: gcp-pubsub
  metadata:
    subscriptionName: app-events-processing
    value: "500" # scale up when >500 unacked messages

```

At 2am: 10,000 messages → KEDA scales workers from 0 to 20. By 3am: 0 messages → workers scale back to 0. You pay nothing for idle workers overnight.

### Cloud Run

Fully managed serverless container runtime. You deploy a container image; GCP handles servers, scaling (including scale-to-zero), and load balancing. Equivalent to AWS Fargate + ALB, or Heroku for containers.

**When to use over GKE:** stateless APIs, webhooks, event-driven workers, anything that can tolerate cold starts and doesn't need persistent connections or complex networking.

```
gcloud run deploy my-api \
```

```
--image=asia-south1-docker.pkg.dev/my-project/my-repo/my-api:latest \
```

```
--region=asia-south1 \
```

```
--allow-unauthenticated \ # public endpoint
```

```
--min-instances=0 \ # scale to zero
```

```
--max-instances=20 \
```

```
--memory=512Mi \
```

```
--cpu=1 \
```

```
--set-env-vars=ENV=prod \
```

```
--service-account=api-sa@...
```

**Cloud Run Jobs:** for batch/one-off tasks that run to completion, not HTTP servers. Equivalent to AWS Batch or a one-shot ECS task.

```
gcloud run jobs create data-import-job \
```

```
--image=.../importer:latest \
```

```
--region=asia-south1 \
```

```
--task-timeout=3600 \
```

```
--max-retries=3
```

**VPC connector:** Cloud Run is serverless but can connect to your VPC for database access.

```
gcloud run services update my-api \
```

```
--vpc-connector=my-connector \
```

```
--vpc-egress=private-ranges-only
```

## Cloud Run — Critical Nuances

### 1. Cold start — the first request after idle is slower

When all instances scale to zero (default), the next request starts a new container — this takes 200ms to 2s depending on image size and runtime. Mitigate with:

- `--min-instances=1` (or more): keep N instances always warm. Incurs idle cost but eliminates cold starts.
- Smaller images: fewer layers = faster pull = shorter cold start.
- Avoid heavy init code (DB migrations, large model loads) in the startup path.

### 2. Concurrency per instance defaults to 80

By default, one Cloud Run instance handles up to 80 concurrent requests simultaneously. Unlike Lambda (1 request/instance), Cloud Run supports concurrency. Set `--concurrency` based on your app's thread-safety and memory per request. For CPU-bound code, lower it (e.g., 10). For I/O-bound (DB queries, API calls), 80-200 is fine.

### 3. Request timeout: default 5 minutes, max 60 minutes

If a request runs longer than the timeout, Cloud Run cancels it with a 503. For long-running jobs (data exports, ML inference), use Cloud Run Jobs (no timeout limit) not Cloud Run Services.

### 4. Cloud Run is stateless by design — local disk is ephemeral

Any file written to the container's filesystem is lost when the instance shuts down. Use GCS for persistent output, Cloud SQL for state. The exception is in-memory tmp (up to 32 GB) which persists for the container's lifetime but not across restarts.

### 5. VPC connector adds latency and cost

A VPC connector adds ~1ms latency per request and has its own per-hour cost. Only use it when Cloud Run needs to reach private resources (Cloud SQL private IP, Redis, internal GKE services). Public-IP Cloud SQL via Auth Proxy does not require a VPC connector.

## Production Use Cases — Cloud Run

### Scenario 1: REST API for warehouse management (stateless, auto-scales)

The order management system exposes a REST API consumed by mobile clients. Traffic pattern: low during the night, spike during shift changes (6am, 2pm, 10pm). Cloud Run handles this naturally — scales to zero at 3am, scales to 50 instances in seconds when a shift starts:

Mobile scanner → HTTPS → Cloud Load Balancer → Cloud Run  
(orders-api)



Cost: you pay only for the request handling time. Zero cost at 3am.

Cold start: ~200-400ms for a Go/Java app. Acceptable for non-latency-critical APIs.

min-instances: 2 ← keep 2 warm to eliminate cold starts during business hours

### Scenario 2: Cloud Run Job for nightly data export

Every night at 1am, export the previous day's application events from BigQuery to a partner SFTP server. This is a batch job — not a server. Cloud Run Jobs handles this perfectly:

```
gcloud run jobs create nightly-export \
  --image=asia-south1-docker.pkg.dev/my-project/repo/exporter:latest \
  --region=asia-south1 \
  --task-timeout=1800 \
  --set-env-vars="DATE=yesterday"
```

# Triggered by Cloud Scheduler at 1am:

```
gcloud scheduler jobs create http nightly-export-trigger \
  --schedule="0 1 * * *" \
```

```
--uri="https://run.googleapis.com/v2/projects/.../jobs/nightly-export:run" \
  --message-body="{}"
```

### Scenario 3: Pub/Sub push subscriber

A Pub/Sub topic receives partner webhook events. Instead of managing a long-running subscriber in GKE, configure Pub/Sub to push directly to a Cloud Run service URL. Cloud Run scales up as messages arrive, scales down when done:

Partner system → POST webhook → Pub/Sub topic

push subscription

▼

Cloud Run: /process-webhook

(validates, normalises, writes to GCS)

ACK = 200 response. NACK = non-200 →

retry.

### Cloud Functions

Event-driven, serverless functions. Equivalent to AWS Lambda. You write a function; GCP invokes it in response to triggers.

#### Triggers:

- HTTP (direct invocation via URL)
- Pub/Sub message
- Cloud Storage object created/deleted
- Firestore document change
- Cloud Scheduler (cron)
- Eventarc (generic event router)

## Generations:

|                   | 1st gen                 | 2nd gen             |
|-------------------|-------------------------|---------------------|
| Max runtime       | 9 min                   | 60 min              |
| Concurrency       | 1 request/instance      | Up to 1000/instance |
| Minimum instances | 0                       | 0 (configurable)    |
| Under the hood    | Cloud Functions runtime | Cloud Run           |

Use 2nd gen for all new functions.

```
gcloud functions deploy process-event \
  --gen2 \
  --runtime=python311 \
  --region=asia-south1 \
  --source=. \
  --entry-point=handle_event \
  --trigger-topic=app-events \
  --service-account=functions-sa@... \
  --max-instances=100
```

## When to use Cloud Functions vs Cloud Run:

- Cloud Functions: small, event-triggered code (< a few hundred lines), tight integration with GCP events.
- Cloud Run: full containerised service, longer-running tasks, custom runtimes, more control.

## App Engine

Fully managed application hosting. You push code; GCP handles servers, scaling, load balancing, and OS patches. Best for teams that want zero infrastructure management.

|               | Standard                                      | Flexible             |
|---------------|-----------------------------------------------|----------------------|
| Runtime       | Sandboxed (Python, Node, Go, Java, PHP, Ruby) | Any Docker container |
| Scale to zero | Yes — free when idle                          | No — min 1 instance  |



|            |                               |                 |
|------------|-------------------------------|-----------------|
| VPC access | Via Serverless VPC Connector  | Native          |
| Use case   | Internal APIs, cron, webhooks | Custom runtimes |

App Engine has a built-in **cron.yaml** for scheduled tasks — simpler than Cloud Scheduler for App Engine apps.

## 7. Storage

### Cloud Storage (GCS)

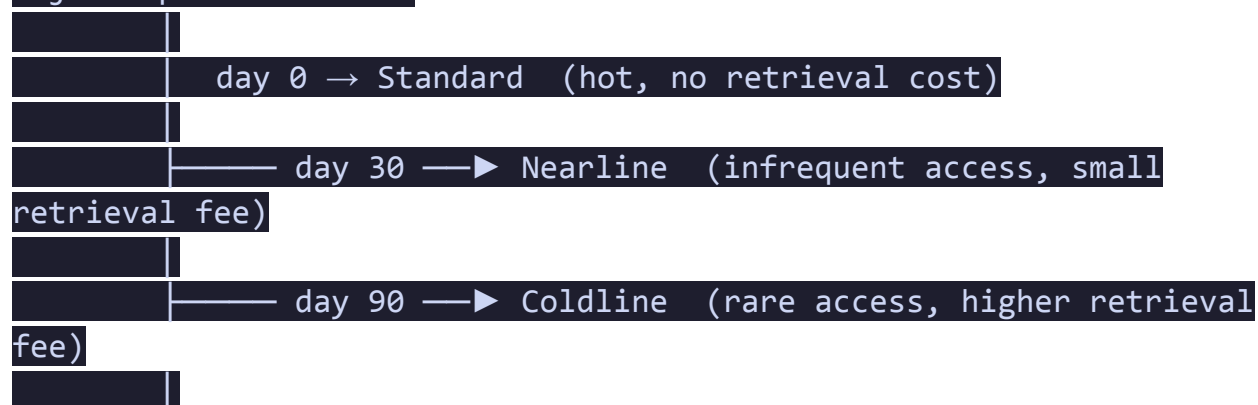
Object storage — equivalent to AWS S3. The primary data lake layer for data platforms. Store anything: raw files, Parquet, logs, backups, ML model artifacts.

**Storage classes — choose based on access frequency:**

| Class           | Min storage duration | Typical use                             | Retrieval cost       |
|-----------------|----------------------|-----------------------------------------|----------------------|
| <b>Standard</b> | None                 | Active data, hot pipelines, recent logs | None                 |
| <b>Nearline</b> | 30 days              | Monthly reports, infrequent data        | Per-GB fee           |
| <b>Coldline</b> | 90 days              | Quarterly archives, DR snapshots        | Per-GB fee (higher)  |
| <b>Archive</b>  | 365 days             | Compliance retention, audit logs        | Per-GB fee (highest) |

**Lifecycle transition flow — how data moves through classes automatically:**

Object uploaded to GCS



└─ day 365 ─► DELETE (or Archive before deletion for compliance)

Non-current versions (after overwrite/delete):

keep latest 2 → delete the rest (lifecycle rule: numNewerVersions=2)

**Versioning:** when enabled, overwriting or deleting creates a non-current version rather than permanently removing data. Always pair with a lifecycle rule to cap version count.

`gcloud storage buckets update gs://my-bucket --versioning`

**Object Lifecycle Management:** automatically transitions or deletes objects based on age, class, or version count. The primary cost-control mechanism.

```
{
  "lifecycle": {
    "rule": [
      {
        "action": { "type": "SetStorageClass", "storageClass":
"NEARLINE" },
        "condition": { "age": 30, "matchesStorageClass":
["STANDARD"] }
      },
      {
        "action": { "type": "SetStorageClass", "storageClass":
"COLDLINE" },
        "condition": { "age": 90, "matchesStorageClass":
["NEARLINE"] }
      },
      {
        "action": { "type": "Delete" },
        "condition": { "numNewerVersions": 2, "isLive": false }
      },
      {
        "action": { "type": "Delete" },
        "condition": { "age": 365 }
      }
    ]
  }
}
```

Apply:

```
gcloud storage buckets update gs://my-bucket  
--lifecycle-file=lifecycle.json
```

**Retention Policy:** a compliance lock — objects cannot be deleted or modified until they have been stored for the specified duration. Even admins cannot override.

```
gcloud storage buckets update gs://audit-logs  
--retention-period=7776000s # 90 days
```

**Access:** use **Uniform bucket-level access** (IAM only, no per-object ACLs) for all new buckets.

```
gcloud storage buckets create gs://my-bucket \  
--location=asia-south1 \  
--storage-class=STANDARD \  
--uniform-bucket-level-access
```

## GCS — Critical Nuances

### 1. Uniform bucket-level access is irreversible after 90 days

Once you enable uniform bucket-level access and 90 days pass, you cannot revert to fine-grained (per-object ACLs). Enable it from day 1 — it simplifies IAM and avoids accidental public objects.

### 2. Retention policy LOCK cannot be undone

You can set a retention policy (e.g., 90 days) and then *lock* it:

```
gcloud storage buckets update gs://audit-logs  
--lock-retention-policy
```

After locking, **no one — not even Project Owners — can remove or shorten the policy or delete the bucket** until all objects have lived past the retention period.

This is intentional for compliance (WORM). Never lock unless you're sure.

### 3. Soft delete — 7-day recovery window (enabled by default)

As of 2024, GCS enables soft delete by default. Deleted objects are retained for 7 days and can be restored. This incurs storage cost for those 7 days. You can disable it or change the window:

```
gcloud storage buckets update gs://my-bucket  
--soft-delete-duration=0 # disable
```

Soft delete and versioning are separate features. Versioning keeps all historical versions on every overwrite. Soft delete only applies to deletes, not overwrites.

### 4. Versioning + lifecycle interaction

When versioning is enabled, a deleted object becomes a *noncurrent* version (not truly deleted). The lifecycle rule `numNewerVersions: 2 + isLive: false` deletes noncurrent versions with 3 or more newer versions — but the current live version is never touched by that rule. You need a separate rule with `age` to delete live objects.

### 5. Egress costs are real — design to minimize

Reading data from GCS within the same region as the reader (e.g., Trino in asia-south1 reading from a asia-south1 bucket) is free. Cross-region reads cost ~\$0.08/GB. Always co-locate your compute (GKE cluster, Dataproc) with your GCS bucket in the same region.

## **6. GCS is eventually consistent for listing, strongly consistent for reads/writes**

Object reads and writes are strongly consistent (read-after-write is guaranteed). But listing objects after a bulk delete may momentarily return stale results. This rarely matters in practice but can affect pipeline code that lists then processes files.

### **Production Use Cases — Cloud Storage (GCS)**

#### **Scenario 1: Data platform raw → processed → curated bucket layout**

Use separate buckets (not folders in one bucket) per layer so you can apply different IAM, lifecycle, and retention rules independently:

```
gs://companyABC-data-raw/          ← Pub/Sub consumers write raw  
JSON/Avro here
```

```
gs://companyABC-data-processed/    ← Spark/Dataflow writes cleaned  
Parquet/Iceberg
```

```
gs://companyABC-data-curated/      ← dbt writes aggregated,  
business-ready tables
```

#### **IAM:**

```
raw:      pipeline-sa → objectCreator; Trino SA → objectViewer
```

```
processed: spark-sa → objectAdmin; Trino SA → objectViewer
```

```
curated:  dbt-sa → objectAdmin; BI tools SA → objectViewer
```

#### **Lifecycle on raw bucket:**

```
Standard → Nearline after 30 days (raw data rarely re-read after  
processing)
```

```
Delete after 365 days (keep 1 year for audit, then gone)
```

#### **Scenario 2: Triggering processing when a file lands**

A sensor uploads a CSV report to `gs://companyABC-data-raw/sensor-reports/`. You want to parse it and insert into BigQuery automatically — no polling.

```
1. GCS triggers Pub/Sub on object finalise (Eventarc)
```

```
2. Cloud Run service (or Cloud Function) subscribes to the Pub/Sub  
topic
```

```
3. Cloud Run: downloads the file, parses CSV, inserts rows into  
BigQuery
```

```
4. Writes a Parquet version to gs://companyABC-data-processed/
```

Cost: near-zero – you only pay when a file arrives (scale to zero)

### Scenario 3: GCS as Trino/Iceberg data store

Iceberg tables live entirely in GCS. The table metadata (snapshot chain, file manifests) is in `gs://companyABC-data-processed/metadata/` and the actual Parquet data files in `gs://companyABC-data-processed/data/`. Trino reads both using the Iceberg connector — it never needs a database server. When you `ALTER TABLE events ADD COLUMN region`, Iceberg writes a new metadata file; the old Parquet files are untouched.

### Filestore

Managed NFS file server. Use when multiple VMs or pods need to share a filesystem (read-write-many). Equivalent to AWS EFS.

```
gcloud filestore instances create shared-nfs \
  --zone=asia-south1-a \
  --tier=STANDARD \
  --file-share=name=data,capacity=1TB \
  --network=name=companyABC-vpc
```

Mount on a VM:

```
sudo mount -t nfs <filestore-ip>:/data /mnt/shared
```

### Persistent Disk

Block storage attached to Compute Engine VMs and GKE nodes. (Covered in detail in Section 6 Compute Engine.)

---

## 8. Databases

### Cloud SQL

Managed PostgreSQL, MySQL, or SQL Server. Equivalent to AWS RDS.

Use for:

- Application transactional databases (backend services)
- Trino / Hive Metastore (PostgreSQL)
- Airflow/Composer metadata database
- Any workload that needs SQL with relational integrity

```
gcloud sql instances create companyABC-postgres \
  --database-version=POSTGRES_15 \
  --region=asia-south1 \
  --tier=db-n1-standard-4 \           # machine type
  --storage-type=SSD \
  --storage-size=100GB \
  --availability-type=REGIONAL \     # multi-zone HA
  --no-assign-ip \                   # private IP only
```

```
--network=companyABC-vpc
```

Key features: automated backups, point-in-time recovery (PITR), read replicas, failover replicas, Cloud SQL Auth Proxy (secure connection without IP whitelisting).

**Cloud SQL Auth Proxy:** recommended connection method. Runs as a sidecar on your VM/pod; handles SSL and IAM authentication.

```
# Run proxy (as a sidecar)
```

```
./cloud-sql-proxy --port=5432
```

```
my-project:asia-south1:companyABC-postgres
```

```
# App connects to localhost:5432
```

## Cloud SQL — Critical Nuances

### 1. Connection limits are per-instance and low for small tiers

Cloud SQL has a hard cap on simultaneous database connections, determined by the machine tier:

| Tier             | Max connections |
|------------------|-----------------|
| db-f1-micro      | 25              |
| db-g1-small      | 100             |
| db-n1-standard-1 | 4,000           |
| db-n1-standard-4 | 4,000           |

Each connection consumes ~5-10 MB of RAM on the DB server. If your app opens a new connection per request (no pooling), you'll hit the limit fast. Always use a connection pool (e.g., PgBouncer or the app framework's built-in pool). Cloud SQL Auth Proxy does not pool — it multiplexes TCP but passes through connections to the DB.

### 2. Storage autogrow — you can grow but never shrink

If you enable storage auto-growth (recommended), Cloud SQL increases disk size when it reaches 95% capacity. But disk can **never be shrunk** — once at 500 GB, it stays at 500 GB even if you delete data. Size disk conservatively and let autogrow handle peaks, but don't let it spiral out of control. Set an alert at 80% disk usage.

### 3. PITR requires binlog/WAL archiving (on by default — don't disable)

Point-in-time recovery relies on continuous archiving of write-ahead logs (PostgreSQL) or binary logs (MySQL). This is enabled by default when you create an instance with backups on. If you disable automated backups to save cost, you also lose PITR. Never disable backups on a production instance.

### 4. Read replicas are NOT highly available by default

A standard read replica is a single VM. If that VM fails, the replica is gone. You need a **High Availability replica** (more expensive) for a fault-tolerant read replica. Also, read replicas have slight replication lag — they are not guaranteed to have the latest writes visible immediately.

### 5. Private IP only — no peering across projects without Shared VPC

If Cloud SQL is in project A with private IP, a VM in project B cannot connect to it unless you've set up Shared VPC or VPC peering between the two projects. The Cloud SQL Auth Proxy still requires network connectivity to the instance's private IP.

### Production Use Cases — Cloud SQL

#### Scenario 1: GKE service connecting to Cloud SQL via sidecar proxy

Your backend API runs in GKE. Instead of opening a public IP on Cloud SQL and IP-whitelisting the cluster, run the Cloud SQL Auth Proxy as a sidecar container in the same Pod:

```
containers:
  - name: api
    image: my-api:1.2.3
    env:
      - name: DB_HOST
        value: "127.0.0.1"    # talks to sidecar on localhost
      - name: DB_PORT
        value: "5432"

  - name: cloud-sql-proxy
    image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2
    args:
      - "--port=5432"
      - "my-project:asia-south1:companyABC-postgres"
    resources:
      requests: {cpu: "100m", memory: "64Mi"}
```

The proxy handles SSL and authenticates via the Pod's Workload Identity. No DB credentials in environment variables, no port exposed outside the Pod.

#### Scenario 2: Read replica for analytics queries

Your OLTP database (Cloud SQL) handles data management writes. Analysts run heavy **SELECT** queries that compete with writes. Solution: add a read replica and point analytics tools (Metabase, Superset) at the replica's private IP. Writes still go to the primary; reads from the replica don't impact production throughput.

```
gcloud sql instances create companyABC-postgres-replica \
  --master-instance-name=companyABC-postgres \
```

```
--region=asia-south1
```

```
# Read replica has its own private IP – point BI tools here
```

### Scenario 3: Point-in-time recovery after bad migration

A database migration script accidentally deletes 50,000 rows. With PITR enabled (default for Cloud SQL), you can restore to any second within the backup retention window:

```
gcloud sql instances clone companyABC-postgres  
companyABC-postgres-recovered \
```

```
--point-in-time="2026-06-08T14:25:00Z"
```

```
# Inspect the recovered instance, extract the lost rows, re-insert  
into prod
```

### Cloud Spanner

Globally distributed, horizontally scalable relational database. Fully consistent, no downtime scaling. Very high cost — justified only when you need global consistency at massive scale with SQL.

Use cases: global inventory systems, financial ledgers, anything with globally distributed writes that must be strongly consistent.

```
gcloud spanner instances create companyABC-spanner \
```

```
--config=regional-asia-south1 \
```

```
--processing-units=100 \ # 100 = 0.1 node; min billing  
unit
```

```
--description="Data Platform Spanner"
```

### AlloyDB

GCP's PostgreSQL-compatible database — faster than Cloud SQL for analytics (up to 4×) and comparable for OLTP. Columnar engine built in for mixed workloads.

Use when Cloud SQL isn't fast enough for analytical queries but BigQuery is overkill for transactional workloads.

### Cloud Bigtable

Managed wide-column NoSQL database — the same system powering Google Search and Maps. HBase API compatible. Equivalent to AWS DynamoDB at extreme scale.

### When to use:

| Good fit                                | Poor fit                   |
|-----------------------------------------|----------------------------|
| Time-series data, IoT telemetry, events | SQL joins, complex queries |
| Writes > 1 million/sec                  | Datasets < 1 TB            |



|                                 |                                 |
|---------------------------------|---------------------------------|
| Single-millisecond read latency | Ad-hoc analytics (use BigQuery) |
| Key-based lookups at scale      | Multi-row transactions          |

**Row key design is critical** — poor keys cause hotspots (all writes to one node).

```
gcloud bigtable instances create companyABC-bigtable \
  --display-name="Data Bigtable" \
  --cluster=data-bt-cluster \
  --cluster-zone=asia-south1-a \
  --cluster-num-nodes=3 \
  --cluster-storage-type=SSD
```

## Firestore

Serverless document database. Equivalent to AWS DynamoDB for document/key-value patterns, or MongoDB in the GCP ecosystem. Scales automatically; no servers to manage.

Two modes:

- **Native mode** (recommended): real-time listeners, offline support. Used for mobile/web apps.
- **Datastore mode**: legacy; compatible with older Datastore API.

Use for: user profiles, product catalogues, config stores, feature flags, session data.  
Not for bulk analytical data.

```
gcloud firestore databases create --location=asia-south1
```

In code:

```
from google.cloud import firestore
db = firestore.Client()
db.collection("locations").document("SITE-001").set({"name":
"Seattle DC", "active": True})
```

## Memorystore

Managed Redis or Memcached. Equivalent to AWS ElastiCache.

```
gcloud redis instances create companyABC-redis \
  --size=1 \                               # 1 GB
  --region=asia-south1 \
  --redis-version=redis_7_0 \
  --network=companyABC-vpc \
  --tier=STANDARD_HA                       # HA with replica
```

Use for: caching API responses, rate limiting, session storage, Pub/Sub job deduplication, leaderboards.

---

## 9. Messaging & Async Processing

---

## Pub/Sub

Fully managed message bus. Equivalent to AWS Kinesis (streaming) + SQS (queue) combined. No shard management — scales automatically.

### Core concepts:

- **Topic:** where producers publish messages.
- **Subscription:** where consumers receive messages.
  - **Pull:** consumer calls API to pull messages (default).
  - **Push:** Pub/Sub sends HTTP POST to a webhook URL.
- Messages retained up to 7 days. At-least-once delivery (design consumers to be idempotent).

```
gcloud pubsub topics create app-events
```

```
gcloud pubsub subscriptions create app-events-pipeline-sub \
```

```
--topic=app-events \
```

```
--ack-deadline=60 \
```

```
--message-retention-duration=1d
```

Publish from code:

```
from google.cloud import pubsub_v1
```

```
publisher = pubsub_v1.PublisherClient()
```

```
publisher.publish("projects/my-project/topics/app-events",
```

```
data=b'{"site":"SITE-001","event":"order_placed"}')
```

**Dead-letter topics:** messages that fail to be acknowledged after N attempts are forwarded to a dead-letter topic. Always configure for production subscriptions.

### Pub/Sub — Critical Nuances

#### 1. At-least-once delivery — duplicates will happen

Pub/Sub guarantees every message is delivered *at least once*. Under retries, network glitches, or ack deadline expiry, the same message can be delivered 2 or more times. Your consumer **must be idempotent** — processing the same message twice should produce the same result as processing it once.

```
# Idempotent consumer example: use message ID for deduplication
```

```
def process_message(message):
```

```
    msg_id = message.message_id
```

```
    if redis.exists(f"processed:{msg_id}"): 
```

```
        message.ack()
```

```
        return
```

```
    # ... do work ...
```

```
    redis.setex(f"processed:{msg_id}", 86400, 1) # remember for 24h
```

```
    message.ack()
```

#### 2. Ack deadline must be longer than your processing time

The default ack deadline is 10 seconds. If your consumer takes 30 seconds to process a message, Pub/Sub will redeliver it before you ack. Set `--ack-deadline` to at least 2× your typical processing time. The subscriber can also call `modifyAckDeadline()` to extend it dynamically.

### 3. Message ordering requires same ordering key AND single region

`enableMessageOrdering` only guarantees order within the same ordering key AND same Pub/Sub region. Cross-region or mixed-key ordering is not guaranteed. If you need strict global ordering, use a single ordering key and accept that throughput is limited (one partition per key).

### 4. Dead-letter topic needs its own IAM grant

When Pub/Sub forwards to a dead-letter topic, it does so as the Pub/Sub service account. That service account needs `roles/pubsub.publisher` on the dead-letter topic — otherwise the forwarding silently fails. This is a common setup mistake.

```
gcloud pubsub topics add-iam-policy-binding dead-letter-topic \
--member="serviceAccount:service-PROJECT_NUMBER@gcp-sa-pubsub.iam.gserviceaccount.com" --role="roles/pubsub.publisher"
```

### 5. Pull vs Push — choose based on consumer type

|                   | Pull                                           | Push                                            |
|-------------------|------------------------------------------------|-------------------------------------------------|
| Consumer location | Anywhere (behind firewall OK)                  | Must have public HTTPS URL                      |
| Scaling           | Consumer controls rate                         | Pub/Sub controls rate                           |
| Best for          | GKE workers, Dataflow, long-running processing | Cloud Run, Cloud Functions, webhooks            |
| Backpressure      | Consumer stops pulling when slow               | Pub/Sub sends regardless — consumer must handle |

### 6. Subscription-level message retention vs topic-level

Messages are retained on the *topic* (default 7 days). If you create a subscription *after* messages were published, the subscription will not receive those old messages (they are retained on the topic but not replayed to new subscriptions unless you use snapshot replay). Create subscriptions before you start publishing if you need guaranteed delivery from the start.

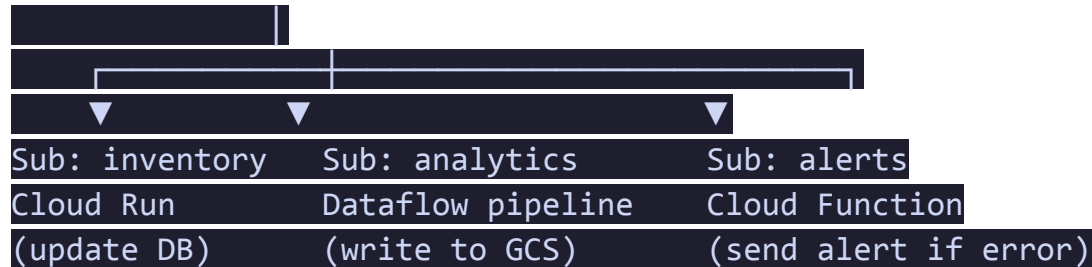
### Production Use Cases — Pub/Sub

#### Scenario 1: Warehouse sensor event pipeline (fan-out to multiple consumers)

Each IoT device publishes an event when a order fulfillment operation happens. Multiple downstream systems need the same event — the inventory service, the

analytics pipeline, and the alerting system. With Pub/Sub you publish once to a topic and each consumer has its own subscription:

Sensor → Pub/Sub topic: app-events



Each subscriber gets every message independently.

If analytics is slow, it doesn't affect inventory processing.

### Scenario 2: Decoupling API from heavy background processing

An API endpoint receives a file upload (data report). Parsing it takes 30 seconds — too long to hold the HTTP connection open. Solution:

1. API receives upload → saves file to GCS → publishes message to Pub/Sub

Response to client: 202 Accepted (immediate)

2. Cloud Run worker subscribes to Pub/Sub pull subscription

Downloads the file → parses → inserts to BigQuery → ACK the message

3. If parsing fails → message goes to dead-letter topic after 5 retries

→ Cloud Function on DLT → Slack alert + save to error GCS bucket

The API is always fast. Processing failures don't lose data (message retried up to 5× then DLT).

### Scenario 3: Real-time streaming into Dataflow

Pub/Sub is the standard source for Apache Beam / Dataflow streaming pipelines.

Flink and Spark Structured Streaming can also consume Pub/Sub directly:

# Dataflow (Apache Beam) reading from Pub/Sub

```
p | ReadFromPubSub(topic='projects/my-project/topics/app-events') \
  | ParseJSON() \
  |
```

```
WriteToParquet(gcs_path='gs://companyABC-data-processed/events/')
```

Pub/Sub handles the backpressure — if Dataflow is slow, messages queue up (up to 7 days by default). Dataflow auto-scales workers based on Pub/Sub backlog.

### Cloud Tasks

Managed task queue for reliable, scalable asynchronous task execution. Equivalent to AWS SQS (as a task queue, not a message bus). A producer enqueues a task; Cloud Tasks dispatches it to a worker (HTTP endpoint or App Engine handler) with retry and rate limiting.

Use for: sending emails, processing uploads, triggering long-running background jobs from an API without blocking the request.

```
gcloud tasks queues create email-queue \
  --location=asia-south1 \
  --max-dispatches-per-second=100 \
  --max-attempts=5
```

### Cloud Scheduler

Managed cron service. Equivalent to AWS EventBridge Scheduler. Triggers HTTP, Pub/Sub, or App Engine endpoints on a cron schedule.

```
gcloud scheduler jobs create http daily-pipeline-trigger \
  --schedule="0 2 * * *" \
```

```
--uri="https://asia-south1-run.googleapis.com/v2/projects/.../jobs/d
ata-import-job:run" \
  --http-method=POST \
  --oauth-service-account-email=scheduler-sa@... \
  --location=asia-south1
```

---

## 10. Data & Analytics Platform

### BigQuery

Serverless, fully managed analytical data warehouse. No cluster to size or manage. Equivalent to AWS Redshift but truly serverless — no idle cost on on-demand pricing.

#### Pricing models:

- **On-demand:** ~\$6.25 per TB of data scanned. Pay only when you run queries.
- **Flat-rate (slot reservations):** buy a fixed number of query compute slots; predictable cost for high-query-volume environments.

#### Core concepts:

- **Dataset:** namespace for tables (like a schema in SQL).

- **Table:** can be native (stored in BigQuery) or external (Parquet/CSV in GCS, queried in place via BigLake).
- **View / Materialized View:** logical or pre-computed query results.
- **Routine:** stored procedures and user-defined functions (UDFs).

**Always partition and cluster production tables — it is the single biggest cost-reduction action:**

```
CREATE TABLE my_dataset.payment_transactions (
  transaction_id STRING,
  transaction_time TIMESTAMP,
  user_id STRING,
  amount NUMERIC,
  status STRING,
  payment_method STRING
)
PARTITION BY DATE(transaction_time)      -- only scan the relevant
day's partition
CLUSTER BY user_id, status                -- within partition, skip
irrelevant blocks
OPTIONS (
  partition_expiration_days = 365,
  require_partition_filter = TRUE      -- prevent accidental full
scans
);
```

**External tables (BigLake)** — query GCS data without loading it:

```
CREATE EXTERNAL TABLE my_dataset.raw_logs
WITH PARTITION COLUMNS
OPTIONS (
  uris = ['gs://my-data-lake/raw/logs/*.parquet'],
  format = 'PARQUET',
  hive_partition_uri_prefix = 'gs://my-data-lake/raw/logs'
);
```

**Row-level and column-level security** via policy tags and row access policies — controls what a user can see without duplicating tables.

**Scheduled queries:** run a SQL query on a cron schedule and write results to a table — equivalent to a simple ETL without Dataflow.

**BigQuery — Critical Nuances**

**1. You are billed for bytes scanned, not rows returned or result size**

```
SELECT COUNT(*) FROM my_dataset.payment_transactions WHERE  
transaction_date = '2024-01-01'  
-- Scans ALL columns for that partition → expensive  
-- Even though result is a single number
```

```
SELECT COUNT(*) FROM my_dataset.payment_transactions WHERE  
transaction_date = '2024-01-01'  
-- With PARTITION BY transaction_date → scans only that partition  
→ cheap
```

```
-- BEST: query only the columns you need (BigQuery is columnar)  
SELECT transaction_id, user_id FROM my_dataset.payment_transactions  
WHERE transaction_date = '2024-01-01'  
-- Scans only 2 columns instead of all → even cheaper
```

## 2. Require partition filter to prevent accidental full table scans

```
ALTER TABLE analytics.transactions  
SET OPTIONS (require_partition_filter = TRUE);  
-- Now any query without WHERE transaction_date = ... fails with an  
error  
-- Protects against expensive queries like: SELECT * FROM  
analytics.transactions
```

Enable this on every large table. It blocks queries that would scan the whole table.

## 3. Streaming inserts cost money; batch loads (load jobs) are free

| Method                           | Cost          | Latency | Use for              |
|----------------------------------|---------------|---------|----------------------|
| Streaming inserts (BigQuery API) | ~\$0.01/200MB | Seconds | Real-time dashboards |
| Load jobs (GCS → BigQuery)       | Free          | Minutes | Batch ETL, backfills |
| <b>LOAD DATA</b> / Dataflow      | Free          | Minutes | Structured batch     |

If you do not need sub-minute freshness, always batch load. Streaming inserts are expensive at scale — 1 TB/day streamed costs ~\$50/day vs free via load jobs.

## 4. Table expiration will silently delete your data

If you set `expiration_timestamp` on a table (or dataset-level default), BigQuery will delete it automatically without warning. Never set expiration on production tables.

Audit with:

```
SELECT table_name, expiration_time FROM
my_dataset.INFORMATION_SCHEMA.TABLES
WHERE expiration_time IS NOT NULL;
```

## 5. On-demand slots are shared and can be throttled

On-demand pricing gives you up to 2,000 concurrent slots. Heavy concurrent queries compete for the same slot pool. If everyone on the team runs large queries at the same time, some will be queued or slow. For predictable performance, buy flat-rate slot reservations. Monitor slot utilization in the BigQuery Admin console.

## 6. Views are re-executed on every query; Materialized Views are pre-computed

A regular view is just saved SQL — it runs the full query every time someone queries the view. A materialized view stores the pre-computed result and refreshes incrementally:

```
CREATE MATERIALIZED VIEW analytics.daily_payment_summary
OPTIONS (enable_refresh = TRUE, refresh_interval_minutes = 60)
AS SELECT DATE(transaction_time) AS dt, region_id, COUNT(*) AS
total_transactions, SUM(amount) AS total_volume
FROM analytics.transactions GROUP BY 1, 2;
```

Use materialized views for frequently-queried aggregations accessed by dashboards.

## Production Use Cases — BigQuery

### Scenario 1: Operational analytics on application data (partitioned + clustered)

Every credit card transaction event is streamed into BigQuery for payment reporting. The transactions table is partitioned by `transaction_date` and clustered by `region_id`:

```
CREATE TABLE analytics.transactions (
  transaction_id STRING,
  region_id      STRING,
  merchant_id   STRING,
  amount        NUMERIC,
  transaction_date DATE,
  created_at    TIMESTAMP
)
PARTITION BY transaction_date
CLUSTER BY region_id;
```



```
-- Query: yesterday's payments for REGION-US-EAST – scans only 1
partition
SELECT merchant_id, SUM(amount) as total_volume
FROM analytics.transactions
WHERE transaction_date = CURRENT_DATE - 1
  AND region_id = 'REGION-US-EAST'
GROUP BY merchant_id
ORDER BY total_volume DESC;
```

Without partitioning: scans the full multi-TB table. With partitioning + clustering: scans only that day's data for that region — 100× cheaper and faster.

### Scenario 2: Federated query — BigQuery + GCS Parquet via BigLake

Your curated Iceberg tables live in GCS. Product managers want to query them from BigQuery (they know SQL, not Trino). Create a BigLake external table — BigQuery queries the GCS files directly without copying data:

```
CREATE EXTERNAL TABLE analytics.iceberg_events
WITH CONNECTION `asia-south1.biglake-conn`
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://companyABC-data-processed/events/*.parquet']
);
-- Analysts now query GCS Parquet from BigQuery UI
-- Trino still queries the same files from its side
-- Single source of truth, two query engines
```

### Scenario 3: Scheduled summary table for Looker dashboard

Instead of running an expensive aggregation query every time someone opens the Looker dashboard, run it once nightly as a BigQuery scheduled query and materialise the results:

```
-- Scheduled query: runs at 3am every day
-- Destination: analytics.daily_region_summary
SELECT
  transaction_date,
  region_id,
  COUNT(*) AS total_transactions,
  SUM(amount) AS volume,
  COUNTIF(status = 'FAILED') AS failures
FROM analytics.transactions
WHERE transaction_date = CURRENT_DATE - 1
GROUP BY 1, 2;
```

Looker reads from the pre-aggregated summary table for payment metrics — sub-second dashboard loads. The heavy scan runs once, not on every dashboard open.

## Dataproc

Managed Spark, Hadoop, Flink, Hive, and Trino clusters. Equivalent to AWS EMR.

```
gcloud dataproc clusters create companyABC-spark \
  --region=asia-south1 \
  --master-machine-type=n2-standard-8 \
  --worker-machine-type=n2-standard-8 \
  --num-workers=4 \
  --optional-components=TRINO,JUPYTER \
  --enable-component-gateway \           # exposes web UIs via proxy
  --autoscaling-policy=companyABC-autoscale-policy
```

**Dataproc Serverless (Spark):** submit a Spark job directly without provisioning a cluster. GCP allocates and releases workers automatically. No cluster management at all.

```
gcloud dataproc batches submit spark \
  --region=asia-south1 \
  --class=com.companyABC.pipeline.Main \
  --jars=gs://my-bucket/pipeline.jar \
  -- --input=gs://my-bucket/raw/ --output=gs://my-bucket/processed/
```

**Dataproc Metastore:** managed Hive Metastore compatible with Trino, Spark, and BigQuery. Central schema registry for the data lake — define a table once, query from any engine.

```
gcloud metastore services create companyABC-metastore \
  --location=asia-south1 \
  --hive-metastore-version=3.1.2 \
  --network=companyABC-vpc
```

## Cloud Dataflow

Managed Apache Beam runner for streaming and batch ETL. Equivalent to AWS Glue (ETL) + Kinesis Data Analytics (streaming). Fully autoscaled — no workers to provision.

Use for:

- Pub/Sub → BigQuery real-time streaming
- GCS → BigQuery batch transforms
- Change Data Capture (CDC) event processing
- Enrichment and aggregation pipelines

Google provides ready-made Dataflow templates for common patterns (Pub/Sub to BigQuery, GCS to BigQuery, etc.) so you often don't write any Beam code.

## Cloud Composer

Managed Apache Airflow. Equivalent to AWS MWAA. Runs on GKE under the hood.

```
gcloud composer environments create companyABC-airflow \
  --location=asia-south1 \
  --image-version=composer-2-airflow-2
```

Access the Airflow UI via the environment's web server URL (IAP-protected). DAGs are synced from a GCS bucket — upload a `.py` DAG file and it appears in Airflow within seconds.

## Datastream

Managed Change Data Capture (CDC) service. Replicates data from Cloud SQL, AlloyDB, MySQL, PostgreSQL, or Oracle to BigQuery or GCS in near real-time.

Use for: keeping a BigQuery copy of your operational database current for analytics, without writing custom pipelines.

## Looker & Looker Studio

| Tool                           | What it is                                             | Use case                                               |
|--------------------------------|--------------------------------------------------------|--------------------------------------------------------|
| <b>Looker Studio</b><br>(free) | Google Data Studio rebranded; drag-and-drop dashboards | Quick dashboards from BigQuery, GCS, Sheets            |
| <b>Looker</b><br>(paid)        | Enterprise BI platform with semantic layer (LookML)    | Company-wide metrics definitions, self-serve analytics |

Looker Studio connects directly to BigQuery — zero setup. Good for internal reporting dashboards.

---

## 11. DevOps — CI/CD, Artifact Registry

### Artifact Registry

Managed container image and package registry. Stores Docker images, Helm charts, Maven, npm, Python packages. Equivalent to AWS ECR + CodeArtifact.

```
# Create a Docker repository
gcloud artifacts repositories create companyABC-images \
  --repository-format=docker \
  --location=asia-south1
```

```
# Authenticate Docker
gcloud auth configure-docker asia-south1-docker.pkg.dev
```

```
# Push an image
docker tag my-api:latest
asia-south1-docker.pkg.dev/my-project/companyABC-images/my-api:lates
t
docker push
asia-south1-docker.pkg.dev/my-project/companyABC-images/my-api:lates
t
```

All GKE clusters should pull images from Artifact Registry in the same project — no authentication needed when Workload Identity is configured.

**Vulnerability scanning:** Artifact Registry automatically scans images for OS CVEs on push. View results in **Security** → **Vulnerability Scanning**.

### Cloud Build

Managed CI/CD build system. Equivalent to AWS CodeBuild. Define a `cloudbuild.yaml`; triggers run on GitHub/GitLab push, tag, or PR.

```
# cloudbuild.yaml
steps:
  - name: 'gcr.io/cloud-builders/docker'
    args: ['build', '-t',
'asia-south1-docker.pkg.dev/$PROJECT_ID/companyABC-images/my-api:$COMMIT_SHA', '.']

  - name: 'gcr.io/cloud-builders/docker'
    args: ['push',
'asia-south1-docker.pkg.dev/$PROJECT_ID/companyABC-images/my-api:$COMMIT_SHA']

  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    args: ['gcloud', 'run', 'deploy', 'my-api',
'--image',
'asia-south1-docker.pkg.dev/$PROJECT_ID/companyABC-images/my-api:$COMMIT_SHA',
'--region', 'asia-south1']
```

```
options:
  logging: CLOUD_LOGGING_ONLY
```

Connect a GitHub repo:

```
gcloud builds triggers create github \
  --repo-name=my-repo \
```

```
--repo-owner=companyABC \
--branch-pattern="^main$" \
--build-config=cloudbuild.yaml
```

## Cloud Deploy

Managed continuous delivery pipeline for deploying to GKE, Cloud Run, or Compute Engine. Sits on top of Cloud Build. Defines promotion sequences (dev → staging → prod) with manual approval gates.

```
gcloud deploy delivery-pipelines create my-api-pipeline \
--region=asia-south1 \
--file=delivery-pipeline.yaml
```

## GitOps — ArgoCD and Flux (brief overview)

GitOps is a pattern where Git is the single source of truth for what should be running in a cluster. A controller in the cluster watches a Git repo; when you push a manifest change, it automatically applies it. No `kubectl apply` from CI.

### Two main tools:

| Tool          | Style                                    | When to use                          |
|---------------|------------------------------------------|--------------------------------------|
| <b>ArgoCD</b> | UI-first, per-app sync, great visibility | You want a dashboard, multiple teams |
| <b>Flux</b>   | CLI/GitOps-native, lightweight, no UI    | Simpler, GitOps-only teams           |

### Rough flow with ArgoCD:

Developer pushes `values-prod.yaml` to Git



ArgoCD controller detects change (polls every 3 min or via webhook)



Compares desired state (Git) vs live state (cluster)



Syncs – applies the diff to the cluster automatically (or with manual approval)

**GCP-native alternative:** Cloud Deploy + Cloud Build handles most of this without GitOps tools — good enough if you're starting out. Move to ArgoCD/Flux when you have multiple teams deploying to the same cluster or want full Git-driven drift detection.

See [ArgoCD docs](#) and [Flux docs](#) for setup on GKE.

## Production Use Cases — CI/CD Pipeline

### Scenario 1: Full build → test → deploy pipeline for a GKE service

Every push to `main` on the `orders-api` repo kicks off this pipeline automatically:

GitHub push to main

||

▼ Cloud Build trigger fires

Step 1: go test ./... (unit tests, ~30s)

Step 2: docker build → tag with \$COMMIT\_SHA

Step 3: docker push →

asia-south1-docker.pkg.dev/.../orders-api:abc1234

Step 4: Vulnerability scan (automatic in Artifact Registry)

Step 5: cloud deploy releases create (creates a release)

||

▼ Cloud Deploy promotion pipeline

dev → auto-deploy (helm upgrade in dev namespace, ~1 min)

staging → auto-deploy after integration tests pass

prod → MANUAL APPROVAL required (team lead approves in console)

||

▼ helm upgrade --install orders-api ./chart

--values values.yaml --values values-prod.yaml

--set image.tag=abc1234

Total time: ~5 min from push to dev. ~10 min to staging. Prod: whenever approved.

### Scenario 2: Image vulnerability blocking

Cloud Build step after push checks the CVE scan result. If a HIGH or CRITICAL vulnerability is found in the image, the build fails — the image never reaches any environment:

steps:

- name: 'gcr.io/cloud-builders/docker'

args: ['push', '...']

- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'

entrypoint: bash

args:

- -c

- |

# Wait for scan, fail if HIGH/CRITICAL found

```
gcloud artifacts docker images describe
.../orders-api:$COMMIT_SHA \
--show-package-vulnerability \
--format=json | python3 check_vulns.py
```

### Scenario 3: Ephemeral preview environments for PRs

When a PR is opened, Cloud Build deploys the branch to a temporary GKE namespace `pr-{number}`. When the PR is merged or closed, Cloud Build tears it down. The PR URL is posted as a GitHub comment automatically.

---

## 12. Frontend & Mobile — Firebase

Firebase is Google's platform for frontend and mobile development. It integrates tightly with GCP — Firebase projects ARE GCP projects, sharing the same IAM, billing, and APIs.

### Firebase Hosting

Static web hosting with global CDN. Deploy React, Vue, Angular, or any static site in seconds. Equivalent to AWS Amplify Hosting or Netlify.

```
npm install -g firebase-tools
firebase login
firebase init hosting # configure which directory to deploy
firebase deploy
```

Features: automatic SSL, custom domains, preview channels (deploy a PR to a temporary URL for review), instant rollbacks.

### Firebase Authentication (Identity Platform)

Managed user authentication. Supports Google, GitHub, Apple, Facebook, email/password, phone OTP, SAML, and OIDC. Equivalent to AWS Cognito.

```
import { getAuth, signInWithPopup, GoogleAuthProvider } from
"firebase/auth";
const auth = getAuth();
signInWithPopup(auth, new GoogleAuthProvider()).then(result => {
  const user = result.user;
  const idToken = await user.getIdToken(); // send to your backend
  for verification
});
```

Backend verification:

```
from firebase_admin import auth
decoded_token = auth.verify_id_token(id_token)
uid = decoded_token["uid"]
email = decoded_token["email"]
```

### Firestore (Frontend / Mobile)

Real-time NoSQL document database with offline support and live listeners. Mobile and web SDKs included. (Same Firestore as covered in databases — two usage patterns.)

```
import { getFirestore, collection, onSnapshot } from
"firebase/firestore";
const db = getFirestore();
// Real-time listener – updates UI when data changes
onSnapshot(collection(db, "orders"), snapshot => {
  snapshot.docs.forEach(doc => console.log(doc.data()));
});
```

### Firebase Realtime Database

Original Firebase NoSQL database — JSON tree, real-time sync, offline support. Simpler than Firestore but less flexible. Use Firestore for new projects; Realtime Database only if you have an existing dependency.

### Firebase Cloud Messaging (FCM)

Push notification service for iOS, Android, and web. Equivalent to AWS SNS mobile push. Free.

```
from firebase_admin import messaging
message = messaging.Message(
    notification=messaging.Notification(title="Order ready",
body="ITEM-123 arrived at store"),
    token="<device-fcm-token>"
)
messaging.send(message)
```

### Firebase Remote Config

Feature flags and A/B testing for mobile/web apps. Define parameters with default values; override per-user, per-device, or per-country without an app update.

---

## 13. API Management

### Cloud Endpoints

Managed API gateway for APIs running on GCP (Cloud Run, GKE, Compute Engine). Provides authentication (API key, JWT), monitoring, rate limiting, and a developer portal.

### API Gateway

Fully managed, serverless API gateway — sits in front of Cloud Functions or Cloud Run backends. Lower-overhead than Endpoints for simple use cases.

```
gcloud api-gateway gateways create my-api-gateway \
```



```
--api=my-api \
--api-config=my-api-config \
--location=asia-south1
```

### Apigee

Enterprise-grade API management platform — full lifecycle management, developer portal, advanced analytics, monetisation. GCP's equivalent to AWS API Gateway at enterprise scale.

Use Apigee when you are exposing APIs to external partners, need advanced rate limiting and quota management per developer, or need a public API developer portal.

---

## 14. AI & Machine Learning — Vertex AI

---

Vertex AI is GCP's unified ML platform. Equivalent to AWS SageMaker.

### Key Capabilities

| Capability                    | Description                                                            |
|-------------------------------|------------------------------------------------------------------------|
| <b>Vertex AI Workbench</b>    | Managed Jupyter notebooks with GCP integrations                        |
| <b>AutoML</b>                 | Train ML models without writing model code; provide labelled data      |
| <b>Custom training</b>        | Train your own models on managed GPU/TPU clusters                      |
| <b>Model Registry</b>         | Version and track trained models                                       |
| <b>Vertex AI Pipelines</b>    | Managed Kubeflow Pipelines for ML workflow orchestration               |
| <b>Feature Store</b>          | Managed store for ML features — consistent across training and serving |
| <b>Prediction / Endpoints</b> | Deploy models for online (real-time) or batch prediction               |
| <b>Gemini API</b>             | GCP-hosted access to Google's Gemini models                            |

### Gemini on Vertex AI

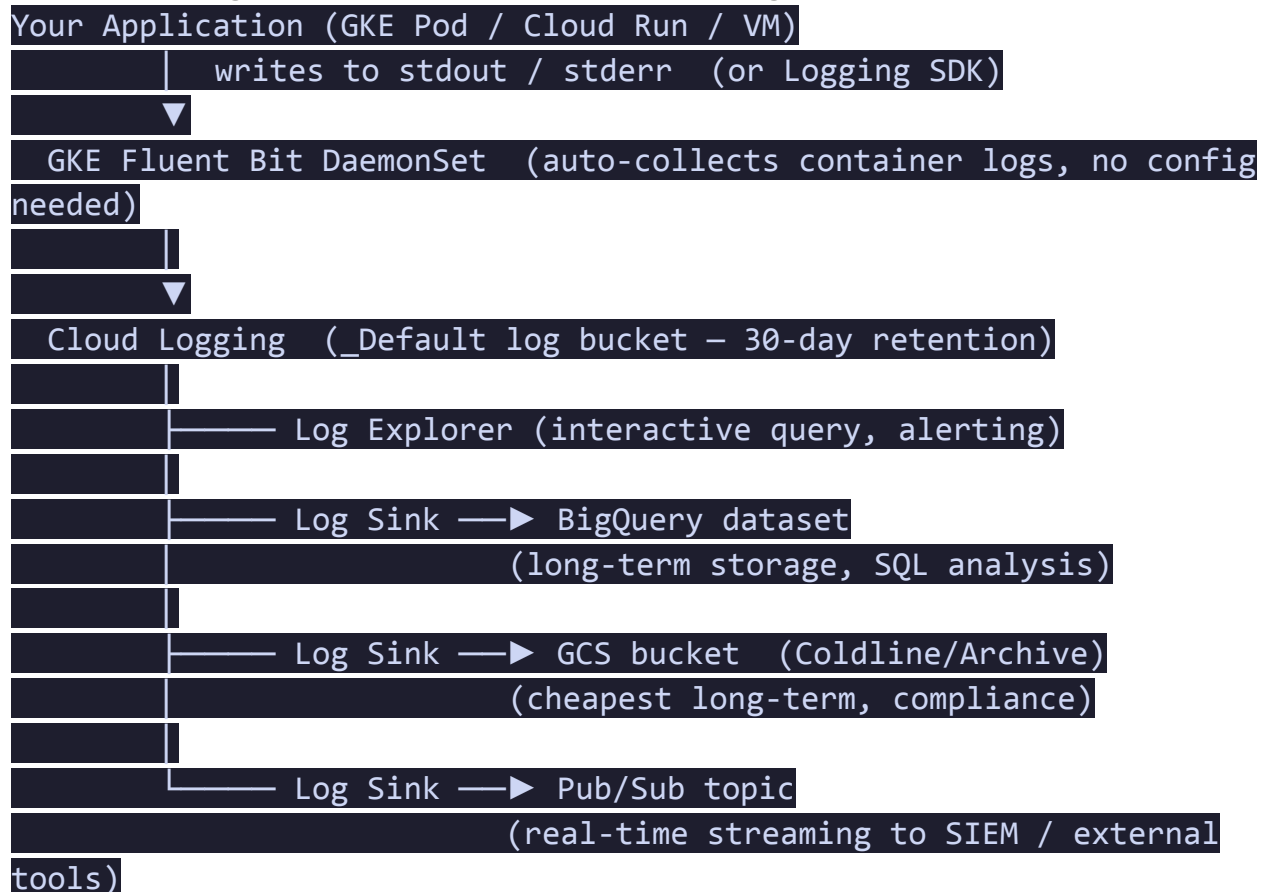
Access Google's foundation models (Gemini Pro, Gemini Flash) via API for LLM features in applications.

```
import vertexai
from vertexai.generative_models import GenerativeModel

vertexai.init(project="my-project", location="asia-south1")
model = GenerativeModel("gemini-1.5-pro")
response = model.generate_content("Summarise this data report: ...")
print(response.text)
```

## 15. Logging — Where Logs Go

### End-to-end log flow — from application to storage:



### Cloud Logging

All GCP services write logs here automatically. Your applications write to it via stdout/stderr (GKE/Cloud Run collect automatically) or the Cloud Logging SDK.

#### Log types:

| Log type      | Example                                                  |
|---------------|----------------------------------------------------------|
| Platform logs | GKE cluster events, Cloud SQL slow query, LB access logs |

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| Audit logs       | Who did what on which GCP resource (Admin Activity, Data Access) |
| Application logs | Anything your code writes to stdout/stderr or the SDK            |

**Structured logs** (JSON to stdout) give you filterable fields in Log Explorer:

```
import json, sys
print(json.dumps({
    "severity": "ERROR",
    "message": "Pipeline failed",
    "site_id": "SITE-001",
    "stage": "transform",
    "error": str(e)
})))
```

**Log Explorer** — GCP Console → Logging → Log Explorer. Query syntax:

```
resource.type="k8s_container"
resource.labels.cluster_name="companyABC-cluster"
severity>=ERROR
jsonPayload.site_id="SITE-001"
timestamp>="2026-06-01T00:00:00Z"
```

## Log Sinks

Route logs to a destination for long-term storage or analysis:

| Destination             | Use case                                         |
|-------------------------|--------------------------------------------------|
| Cloud Logging (default) | Interactive queries; 30-day retention            |
| BigQuery                | Long-term; SQL analysis of logs                  |
| GCS                     | Archive; cheapest long-term retention            |
| Pub/Sub                 | Real-time streaming to external SIEM or alerting |

```
gcloud logging sinks create error-to-bigquery \
    bigquery.googleapis.com/projects/my-project/datasets/logs_dataset \
    --log-filter='severity>=ERROR' \
    --description="ERROR+ logs to BigQuery for analysis"
```

## Log Retention

Default: 30 days in the `_Default` log bucket. Extend up to 3,650 days:

```
gcloud logging buckets update _Default \  
  --location=global \  
  --retention-days=90
```

For longer or cheaper retention, sink to GCS Coldline.

## Cloud Audit Logs

Three types:

- **Admin Activity**: always on; who created/modified GCP resources (free).
- **Data Access**: who read data (e.g. who queried a BigQuery table). Off by default — enable per service.
- **System Event**: automated GCP actions (VM migrations, auto-scaling).

Enable Data Access audit logs:

```
gcloud projects get-iam-policy my-project --format=json >  
policy.json  
# Add auditConfigs section for bigquery.googleapis.com with  
DATA_READ, DATA_WRITE  
gcloud projects set-iam-policy my-project policy.json
```

---

## 16. Monitoring, Dashboards & Alerts

### Cloud Monitoring

Metrics from all GCP services flow here automatically. Custom application metrics can be pushed via the Cloud Monitoring client library or by exposing Prometheus metrics (collected by Google Managed Prometheus on GKE).

### Google Managed Prometheus (GKE)

Enable on the cluster:

```
gcloud container clusters update companyABC-cluster \  
  --enable-managed-prometheus \  
  --region=asia-south1
```

Scrape your application's Prometheus endpoint:

```
apiVersion: monitoring.googleapis.com/v1  
kind: PodMonitoring  
metadata:  
  name: my-service-monitoring  
  namespace: default  
spec:  
  selector:  
    matchLabels:  
      app: my-service  
  endpoints:
```

```
- port: 8080
  path: /metrics
  interval: 30s
```

Metrics land in Cloud Monitoring and are queryable via PromQL.

## Dashboards

Create at **GCP Console** → **Monitoring** → **Dashboards** → **Create Dashboard**.

### Recommended dashboards to set up:

#### Infrastructure — GKE Health

- CPU and memory utilisation per node pool
- Pod restart count (alert if > 5 in 10 min)
- Pending pods count (alert if > 0 for > 5 min — scaling issue)
- Node count per pool (autoscaler activity)

#### Application — Backend API

- HTTP request rate (rpm)
- HTTP error rate (5xx %)
- Request latency (p50, p95, p99)
- Active connections

#### Data Platform — Trino

- Active / queued query count
- Failed queries per minute
- Worker memory utilisation
- Query latency p95

#### Data Platform — Pipelines

- Dataflow job backlog (unprocessed elements)
- Pub/Sub subscription oldest unacked message age
- BigQuery job failures per hour
- GCS bytes written per pipeline

#### Cost

- Daily spend by service (from Billing Export in BigQuery + Looker Studio dashboard)

## Alerting

Create at **Monitoring** → **Alerting** → **Create Policy**.

### Required alerts for every production deployment:

| Alert             | Condition                             | Channel           |
|-------------------|---------------------------------------|-------------------|
| Pod crash-looping | Container restart count > 5 in 10 min | Slack + PagerDuty |

|                            |                                    |                   |
|----------------------------|------------------------------------|-------------------|
| Node OOM / memory pressure | Node memory > 90%                  | Slack             |
| High HTTP error rate       | 5xx rate > 5% over 5 min           | Slack + PagerDuty |
| Pub/Sub lag                | Oldest unacked message age > 5 min | Slack             |
| BigQuery job failures      | Failed jobs > 10/hour              | Slack             |
| Budget threshold           | Spend > 90% of monthly budget      | Email             |
| Disk near full             | Persistent disk utilisation > 85%  | Slack             |

**Notification channels:** set up Slack and email first. PagerDuty or OpsGenie for critical (page on-call).

**# Create a Slack notification channel**

```
gcloud alpha monitoring channels create \
```

```
--channel-content='{ "type": "slack", "displayName": "#alerts-data-platform", "labels": { "channel_name": "#alerts-data-platform", "auth_token": "<token>" } }'
```

## Cloud Trace

Distributed tracing. Automatically captures latency data from Cloud Run and App Engine. Instrument backend services with the OpenTelemetry SDK for end-to-end trace visibility across services. Equivalent to AWS X-Ray.

## Error Reporting

Automatically groups and counts application errors from Cloud Logging, Cloud Run, and App Engine. Sends notifications for new error types. View at **GCP Console** →

## Error Reporting.

## Cloud Profiler

Continuous CPU and memory profiling for production applications. Low overhead (< 1% CPU). Add the Profiler agent to your app:

```
import googlecloudprofiler
```

```
googlecloudprofiler.start(service="my-api", service_version="1.0.0")
```

## Production Use Cases — Monitoring & Observability

### Scenario 1: Trino query failure alert via Cloud Monitoring + Slack

Trino exposes JMX metrics scraped by Google Managed Prometheus. You want a Slack alert when the failed query count spikes (e.g., OOM errors during peak load):

```
1. PodMonitoring CRD scrapes /metrics from Trino coordinator port 8081
```

```
2. Prometheus metric: trino_query_manager_failed_queries_total
```

```
3. Cloud Monitoring alert policy:
```

```
Condition: trino_failed_queries rate > 5/min for 3 consecutive minutes
```

```
Notification: Slack channel #alerts-data-platform + PagerDuty
```

Result: on-call is paged within 3 minutes of a Trino issue, before analysts notice.

### Scenario 2: Tracking down a latency regression with Cloud Trace

A new deployment of the API causes P99 latency to jump from 200ms to 800ms.

Cloud Trace shows the full call tree for slow requests:

```
Incoming request: POST /api/v1/events → 820ms total
```

```
├── Auth middleware (JWT verify) → 12ms
```

```
├── DB query: SELECT FROM events → 650ms ← SLOW
```

```
│   (full table scan – missing index on user_status)
```

```
├── Pub/Sub publish → 45ms
```

```
└── Response serialisation → 8ms
```

```
Fix: add composite index on (user_status, event_date). Latency back to 210ms.
```

Without Trace, you'd only know the request is slow — not which step.

### Scenario 3: Log-based alerting for data pipeline failures

The nightly ETL pipeline logs `{"severity": "ERROR", "stage": "transform"}` when it fails. Instead of building a custom monitoring system, create a log-based metric and alert:

```
# Create a log-based metric counting pipeline errors
```

```
gcloud logging metrics create pipeline-errors \
```

```
--description="ETL pipeline error count" \
```

```
--log-filter='resource.type="k8s_container" AND
```

```
jsonPayload.stage="transform" AND severity="ERROR"'
```

```
# Create an alert: notify if > 0 errors in any 5-minute window
```

```
# (a single error is worth knowing about for a nightly batch)
```

This means: any pipeline failure sends a Slack message within 5 minutes — without writing a single line of alerting code.

## 17. Latency Considerations

### Network Latency by Traffic Type

| Traffic                  | Typical latency | Cost           |
|--------------------------|-----------------|----------------|
| Intra-zone (same zone)   | < 1 ms          | Free           |
| Inter-zone (same region) | 1–5 ms          | ~\$0.01/GB     |
| Inter-region             | 50–200 ms       | \$0.02–0.08/GB |
| Internet egress          | Variable        | \$0.08–0.12/GB |

**Design principle:** keep services that talk frequently in the same zone. GKE nodes, Bigtable cluster, Cloud SQL, and Trino workers should all be in the same zone to eliminate inter-zone latency and charges.

### Storage Latency

| Storage             | First-byte latency      | Throughput pattern                             |
|---------------------|-------------------------|------------------------------------------------|
| Local SSD           | < 1 ms                  | Best for random I/O                            |
| pd-ssd              | ~1 ms                   | Good for random I/O                            |
| pd-standard (HDD)   | ~5 ms                   | Good for sequential I/O                        |
| GCS                 | 100–200 ms (first byte) | Scale via parallelism; use files > 128 MB      |
| Memorystore (Redis) | < 1 ms                  | In-memory                                      |
| Bigtable            | 1–10 ms (p99)           | Scale via more nodes                           |
| BigQuery            | Seconds to minutes      | Columnar; partition and cluster to reduce scan |

### Network Tiers

| Tier | Routing | Cost | Use case |
|------|---------|------|----------|
|------|---------|------|----------|



|                 |                                  |        |                                 |
|-----------------|----------------------------------|--------|---------------------------------|
| <b>Premium</b>  | Google's private global backbone | Higher | All production services         |
| <b>Standard</b> | Public internet for egress       | Lower  | Dev/test, non-latency-sensitive |

Use Premium for all production data platform traffic.

### BigQuery Performance Optimisation

- **Partitioning + clustering:** always. Reduces bytes scanned → faster queries and lower cost.
- **BI Engine:** in-memory cache for sub-second dashboard queries on small result sets (< 100 GB working set).
- **Materialized views:** pre-compute expensive aggregations; BigQuery keeps them fresh automatically.
- **Flat-rate slots:** predictable query latency without queuing behind other on-demand users.

---

## 18. Running Trino on GCP

Trino is a distributed SQL query engine. It queries data across sources (GCS, BigQuery, Bigtable, PostgreSQL, Kafka) using SQL without moving data. It is a compute layer, not a database.

### Deployment Options

#### Option A — GKE (recommended for production)

Deploy using the official Helm chart. Benefits: rolling restarts, Workload Identity for secure GCS access, cluster autoscaler for workers, no VM management.

```
helm repo add trino https://trinodb.github.io/charts
helm install trino trino/trino \
  --namespace trino --create-namespace \
  --set coordinator.resources.requests.memory=8Gi \
  --set worker.replicas=5 \
  --set worker.resources.requests.memory=32Gi \
  --set worker.nodeSelector."workload"=trino
```

Use a dedicated high-memory node pool for Trino workers, spot instances for cost saving.

#### Option B — Dataproc (simpler to start)

```
gcloud dataproc clusters create companyABC-trino \
  --region=asia-south1 \
  --optional-components=TRINO \
  --enable-component-gateway \
```

```
--master-machine-type=n2-standard-8 \
--worker-machine-type=n2-standard-8 \
--num-workers=4
```

## Connectors Relevant to the Data Platform

| Connector  | What it queries                               |
|------------|-----------------------------------------------|
| Hive (GCS) | Parquet/ORC/CSV in GCS via Dataproc Metastore |
| BigQuery   | BigQuery tables directly                      |
| PostgreSQL | Cloud SQL PostgreSQL                          |
| Iceberg    | Apache Iceberg tables on GCS                  |
| Delta Lake | Delta tables on GCS                           |
| Kafka      | Self-managed Kafka or Confluent on GCP        |
| HTTP       | REST APIs                                     |

## Dataproc Metastore (Shared Schema Registry)

All query engines (Trino, Spark, BigQuery via federation) share schema definitions. Define a table once; query from anywhere.

```
gcloud metastore services create companyABC-metastore \
--location=asia-south1 \
--hive-metastore-version=3.1.2 \
--network=companyABC-vpc
```

## 19. AWS → GCP Service Map

| Category            | AWS           | GCP            | Notes                                          |
|---------------------|---------------|----------------|------------------------------------------------|
| <b>VM / Compute</b> | EC2           | Compute Engine | GCP has automatic sustained-use discounts      |
| <b>Containers</b>   | ECS / Fargate | Cloud Run      | Cloud Run is serverless; no cluster management |
| <b>Kubernetes</b>   | EKS           | GKE            | GKE is more mature; no control plane charge    |

|                             |                   |                      |                                                   |
|-----------------------------|-------------------|----------------------|---------------------------------------------------|
| <b>Serverless functions</b> | Lambda            | Cloud Functions      | Cloud Functions 2nd gen runs on Cloud Run         |
| <b>PaaS</b>                 | Elastic Beanstalk | App Engine           |                                                   |
| <b>Object storage</b>       | S3                | Cloud Storage (GCS)  | GCS lifecycle management is more granular         |
| <b>Block storage</b>        | EBS               | Persistent Disk      |                                                   |
| <b>File storage (NFS)</b>   | EFS               | Filestore            |                                                   |
| <b>Container registry</b>   | ECR               | Artifact Registry    | Artifact Registry also stores Helm, Maven, npm    |
| <b>CDN</b>                  | CloudFront        | Cloud CDN            | Integrated with GCP HTTP(S) LB                    |
| <b>DNS</b>                  | Route 53          | Cloud DNS            |                                                   |
| <b>Load balancer</b>        | ALB / NLB         | Cloud Load Balancing | GCP Global HTTP LB is Anycast — one IP, worldwide |
| <b>WAF / DDoS</b>           | WAF + Shield      | Cloud Armor          |                                                   |
| <b>VPN</b>                  | Site-to-Site VPN  | Cloud VPN            |                                                   |
| <b>Direct Connect</b>       | Direct Connect    | Cloud Interconnect   |                                                   |
| <b>SQL DB</b>               | RDS               | Cloud SQL            |                                                   |
| <b>Global SQL DB</b>        | Aurora Global     | Cloud Spanner        |                                                   |

|                               |                          |                      |                                                  |
|-------------------------------|--------------------------|----------------------|--------------------------------------------------|
| <b>NoSQL / Document</b>       | DynamoDB                 | Bigtable + Firestore | Bigtable for throughput; Firestore for documents |
| <b>In-memory cache</b>        | ElastiCache              | Memorystore          | Managed Redis / Memcached                        |
| <b>Data warehouse</b>         | Redshift                 | BigQuery             | BigQuery is serverless; no cluster to size       |
| <b>Managed Spark</b>          | EMR                      | Dataproc             | Dataproc also manages Trino natively             |
| <b>Stream ETL</b>             | Glue + Kinesis Analytics | Dataflow             | Apache Beam; fully autoscaled                    |
| <b>Workflow / ETL catalog</b> | Glue Data Catalog        | Dataproc Metastore   | Hive-compatible                                  |
| <b>Managed Airflow</b>        | MWAA                     | Cloud Composer       | Composer runs on GKE                             |
| <b>Messaging (stream)</b>     | Kinesis                  | Pub/Sub              | No shard management in Pub/Sub                   |
| <b>Messaging (queue)</b>      | SQS                      | Cloud Tasks          |                                                  |
| <b>Cron / Scheduler</b>       | EventBridge Scheduler    | Cloud Scheduler      |                                                  |
| <b>CI Build</b>               | CodeBuild                | Cloud Build          |                                                  |
| <b>CD Pipeline</b>            | CodePipeline             | Cloud Deploy         |                                                  |
| <b>Secrets</b>                | Secrets Manager          | Secret Manager       |                                                  |
| <b>KMS</b>                    | KMS                      | Cloud KMS            |                                                  |

|                                |                    |                           |                                               |
|--------------------------------|--------------------|---------------------------|-----------------------------------------------|
| <b>IAM</b>                     | IAM                | IAM                       | GCP has no explicit DENY; only ALLOW          |
| <b>Auth / Identity</b>         | Cognito            | Firebase Authentication   | Firebase also has mobile SDKs                 |
| <b>Static hosting</b>          | Amplify / S3 + CF  | Firebase Hosting          |                                               |
| <b>Mobile / Realtime DB</b>    | DynamoDB + AppSync | Firestore + Firebase      | Firebase has offline-first mobile SDKs        |
| <b>Push notifications</b>      | SNS Mobile Push    | Firebase Cloud Messaging  |                                               |
| <b>API gateway</b>             | API Gateway        | API Gateway / Apigee      | Apigee for enterprise; API Gateway for simple |
| <b>ML platform</b>             | SageMaker          | Vertex AI                 |                                               |
| <b>LLM / Foundation models</b> | Bedrock            | Vertex AI (Gemini API)    |                                               |
| <b>Observability — metrics</b> | CloudWatch         | Cloud Monitoring          |                                               |
| <b>Observability — logs</b>    | CloudWatch Logs    | Cloud Logging             |                                               |
| <b>Observability — tracing</b> | X-Ray              | Cloud Trace               |                                               |
| <b>Error tracking</b>          | CloudWatch         | Error Reporting           |                                               |
| <b>Billing export</b>          | Cost Explorer      | Billing Export → BigQuery | GCP export is a raw SQL table; more flexible  |

|                            |     |            |                                         |
|----------------------------|-----|------------|-----------------------------------------|
| <b>CDC<br/>replication</b> | DMS | Datastream | Datastream targets<br>BigQuery natively |
|----------------------------|-----|------------|-----------------------------------------|

---

# K8s-Helm-Terraform

(Note: Used default font, to preserve text style | Refactored with GPT)

## Table of Contents

---

1. What is Kubernetes & Why
2. Architecture — Control Plane & Nodes
3. Core Concepts & Objects
4. Namespaces
5. Pods
6. Workload Controllers
7. Services & Networking
8. Ingress & Gateway API
9. Configuration — ConfigMaps & Secrets
10. Storage — Volumes, PVCs, StorageClasses
11. Resource Management — Requests, Limits, QoS
12. Scheduling — Node Selection, Affinity, Taints
13. Autoscaling — HPA, VPA, KEDA, Cluster Autoscaler
14. Health Checks — Probes
15. RBAC — Role-Based Access Control
16. CRDs — Custom Resource Definitions
17. Helm — Package Manager
18. GKE — Google Kubernetes Engine Setup
19. Workload Identity (GKE + GCP IAM)
20. kubectl — Essential Commands
21. Observability — Logs, Metrics, Tracing
22. Production Patterns & Best Practices
23. Troubleshooting Guide
24. CI/CD — GitHub Actions
25. GitOps with ArgoCD
26. Terraform — Infrastructure as Code

---

## 1. What is Kubernetes & Why

Kubernetes (K8s) is an open-source container orchestration system. You describe the desired state of your application (how many replicas, what image, how much memory) in YAML, and Kubernetes makes it so — and keeps it that way even if containers crash, nodes fail, or load spikes.

### What it solves:

- Running containers reliably in production (restarts on crash, health checks)



- Scaling services up and down automatically based on load
- Rolling deployments with zero downtime
- Service discovery — containers find each other by name, not IP
- Efficient bin-packing of workloads onto servers
- Secrets and configuration management

### What Kubernetes is NOT:

- It is not a PaaS — you still manage the application lifecycle
- It is not a CI/CD system — it runs what you deploy, not how you deploy it
- It does not manage data inside your containers — storage is external

### Key terminology:

- **Cluster:** the entire Kubernetes system — a control plane + worker nodes
- **Node:** a worker machine (VM) that runs containers
- **Pod:** the smallest deployable unit — one or more containers running together
- **Manifest:** a YAML file describing a Kubernetes object

---

## 2. Architecture — Control Plane & Nodes

---

### What is a Node?

A **node** is simply a machine — a VM in GCP's case. When you create a GKE cluster, GCP spins up VMs (nodes) that run your Pods. You pick the machine type; GCP manages the rest.

### Real example — a 5-node data platform cluster:

Node pool: n2-standard-8 → 8 vCPUs, 32 GB RAM, 100 GB SSD boot disk

(this is one VM, one machine)

Cluster with 5 such nodes:

Total compute available to Pods ≈ 40 vCPUs, ~150 GB RAM usable  
(~6 GB per node is used by Kubernetes system components, so ~26 GB usable per node)

Node pool: n2-highmem-16 → 16 vCPUs, 128 GB RAM

Used for memory-heavy workloads like Trino workers

3 nodes = 48 vCPUs, ~360 GB RAM for Trino

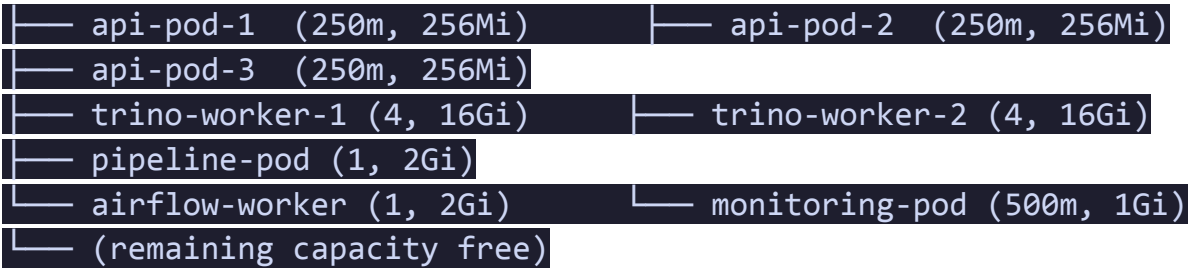
You choose machine types per **node pool** — one pool of cheap general-purpose nodes for APIs and pipelines, another pool of high-memory nodes for Trino, optionally a pool of Spot VMs (cheaper, preemptible) for batch jobs.

### How Pods are spread across nodes:

Cluster (3 × n2-standard-8 nodes, 8 vCPU / 32 GB each)

Node 1 (8 vCPU, 32 GB)                      Node 2 (8 vCPU, 32 GB)

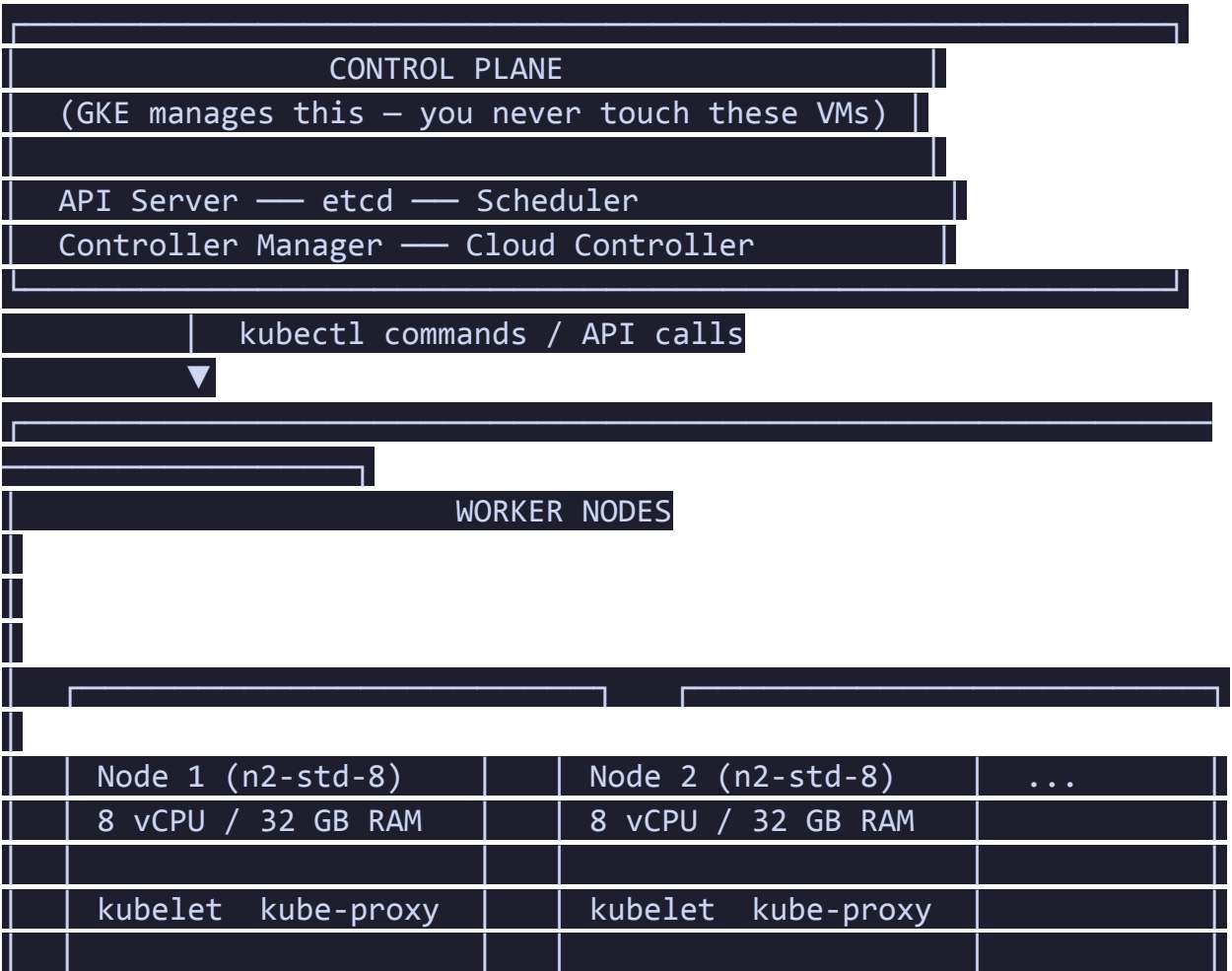
Node 3 (8 vCPU, 32 GB)



The scheduler placed each Pod on the node that had enough free CPU + memory.

If all nodes are full and a new Pod is requested → Cluster Autoscaler adds Node 4.

Full Architecture Diagram





Kubernetes is explicitly designed to be a multi-tenant system. A single control plane can manage, track, and orchestrate thousands of containers across a massive pool of worker machines, regardless of how many different services you deploy.

We'll learn more about these (upcoming):

- Central Registry: The control plane uses etcd as its single source of truth. Every single pod, container, and machine (node) registers its status here.
- Continuous Monitoring: The control plane constantly talks to a tiny agent called the kubelet running on every single worker machine.
- Global Visibility: Because of this constant loop, the control plane knows exactly which machine is running which container, how much memory they are using, and whether they are healthy.
- When You Would Actually Want Separate Clusters: Separating Environments:
  - Keeping Production completely isolated from Development so a developer mistake cannot take down the live website
  - Compliance: If one specific service handles credit cards (PCI-DSS compliance) and requires strict security auditing that other services do not need, etc.

Control Plane Components

| Component                                     | Role                                                                                                                                                          |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Server<br>( <code>kube-apiserver</code> ) | The front door — all <code>kubectl</code> commands, internal components, and external clients talk to the API server. Validates and persists objects to etcd. |

|                                                                 |                                                                                                                                                                                                            |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>etcd</b>                                                     | Distributed key-value store — the source of truth for all cluster state. Always run with an odd number of nodes (3 or 5) for quorum.                                                                       |
| <b>Scheduler</b><br>( <b>kube-scheduler</b> )                   | Watches for unscheduled Pods; assigns them to nodes based on resource availability, affinity rules, and taints/tolerations.                                                                                |
| <b>Controller Manager</b><br>( <b>kube-controller-manager</b> ) | Runs reconciliation loops for built-in resources — ReplicaSet controller, Deployment controller, Job controller, etc. Each controller watches desired state vs actual state and takes action to reconcile. |
| <b>Cloud Controller Manager</b>                                 | On GKE — integrates with GCP APIs to manage load balancers, persistent disks, and node lifecycle.                                                                                                          |

### Node Components

| Component                | Role                                                                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kubelet</b>           | Agent on every node. Watches the API server for Pods assigned to its node; starts/stops containers via the container runtime. Reports node and pod status back. |
| <b>kube-proxy</b>        | Maintains network rules (iptables/IPVS) on the node for Service routing. Routes traffic to the correct Pod IP.                                                  |
| <b>Container runtime</b> | Runs containers. GKE uses <b>containerd</b> . (Docker was deprecated as a runtime in K8s 1.24.)                                                                 |

### GKE — Managed Control Plane

On GKE, Google manages the control plane (API server, etcd, scheduler, controller manager). You never SSH into control plane nodes. You only manage worker nodes via node pools.

## 3. Core Concepts & Objects

Every Kubernetes object is described by a manifest with four top-level fields:

```
apiVersion: apps/v1      # which API group and version
kind: Deployment          # what type of object
```

```
metadata:
  name: my-app
  namespace: default
  labels:
    app: my-app
    env: prod
spec:                                # desired state – different per kind
  ...
```

Apply any manifest:

```
kubectl apply -f my-manifest.yaml
```

### Labels & Selectors

Labels are key-value pairs on any object. Selectors filter objects by labels.

```
metadata:
  labels:
    app: trino
    component: worker
    env: prod
    version: "428"
```

```
kubectl get pods -l app=trino,env=prod
```

```
kubectl get pods -l 'env in (prod,staging)'
```

Labels are how Services find Pods, how Deployments manage ReplicaSets, and how monitoring systems discover scrape targets.

### Annotations

Key-value metadata not used for selection — for tooling, documentation, or operators.

```
metadata:
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
    deployment.kubernetes.io/revision: "3"
```

---

## 4. Namespaces

Namespaces are virtual clusters within a physical cluster. They provide:

- Scope for names (same name can exist in different namespaces)
- Resource quota isolation
- RBAC scope

```
kubectl get namespaces
```

```
# Create
```

```
kubectl create namespace trino
```

```
kubectl create namespace airflow
```

```
kubectl create namespace monitoring
```

```
# Work in a namespace
```

```
kubectl get pods -n trino
```

```
kubectl get all -n trino
```

```
# Set default namespace for your session
```

```
kubectl config set-context --current --namespace=trino
```

**Recommended namespace structure:**

| Namespace     | Purpose                                                                      |
|---------------|------------------------------------------------------------------------------|
| default       | Avoid using — easy to accidentally pollute                                   |
| kube-system   | Kubernetes system components (DNS, metrics-server). Do not deploy apps here. |
| trino         | Trino coordinator + workers                                                  |
| airflow       | Airflow scheduler, webserver, workers                                        |
| monitoring    | Prometheus, Grafana, alertmanager                                            |
| data-platform | Custom data platform services                                                |
| ingress-nginx | Ingress controller                                                           |

---

## 5. Pods

### Container vs Pod — The Relationship

A **container** is a single isolated process running from a Docker image. It has its own filesystem and process space, but no networking or storage of its own — those come from the Pod.

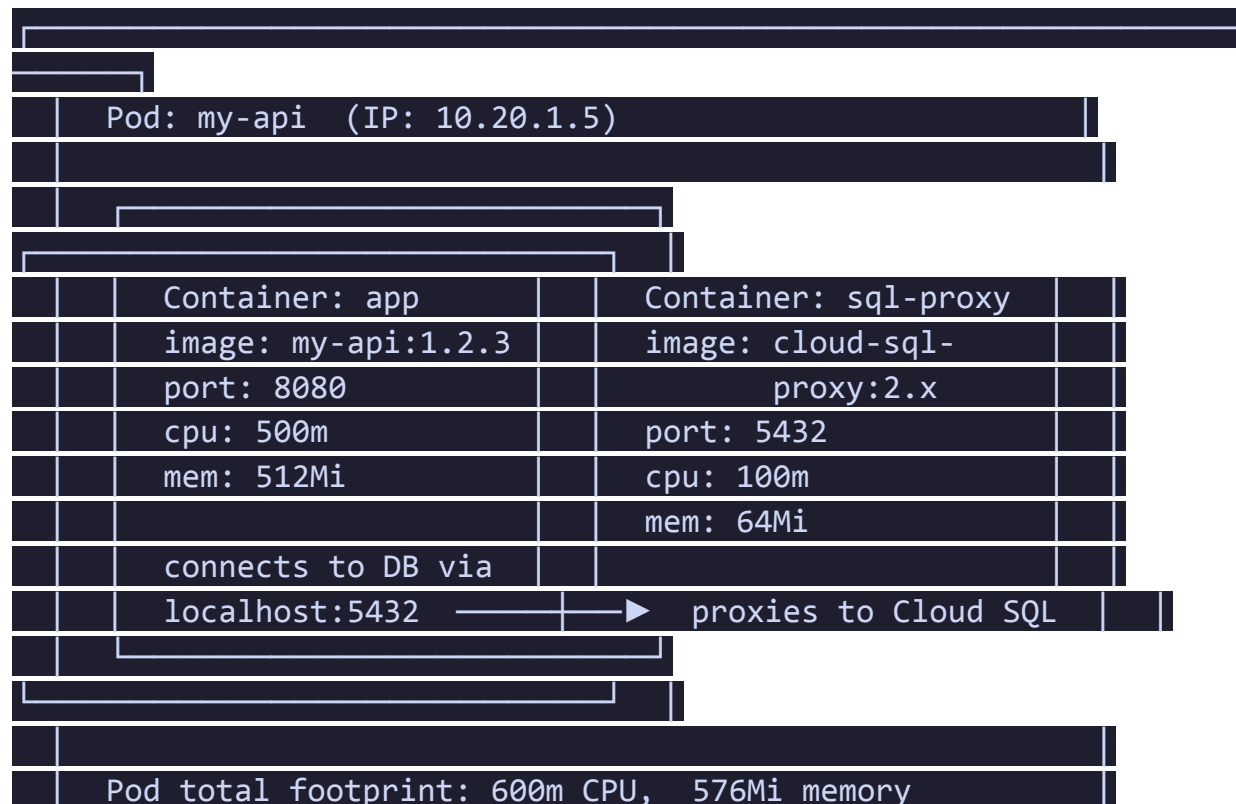
A **Pod** is Kubernetes' wrapper around one or more containers. It provides:

- A shared **network namespace** — all containers in the Pod share one IP; they reach each other via `localhost`

- Shared **storage volumes** — any volume mounted in the Pod spec can be accessed by any container in that Pod
- A shared **lifecycle** — all containers start and stop together as a unit

### Real example — an API backend Pod with a Cloud SQL proxy sidecar:

Node: n2-standard-4 (4 vCPU, 16 GB RAM) ← the actual VM in GCP



The app container talks to **localhost:5432** — it doesn't know or care that there is a proxy container. The proxy container handles the secure tunnel to Cloud SQL. This is the sidecar pattern: one container per concern.

### Key rules:

- All containers in a Pod share one IP. They communicate via **localhost:<port>**.
- Resources (CPU, memory) are set **per container**. The Pod's total is the sum — the scheduler uses this sum to decide which node fits the Pod.
- You can have 1, 2, or 3+ containers in a Pod. Most Pods have 1 (main app) or 2 (main + sidecar). More than 3 is unusual and worth questioning.

- Containers inside a Pod **cannot be individually scaled**. To scale, you scale the Pod count.

## Scaling Pods vs Scaling Containers — Critical Distinction

This is one of the most important concepts to understand correctly.

### Scaling containers (NOT how Kubernetes works):

"My app is busy — I'll add more app containers to this Pod."

This is not a pattern in Kubernetes. You cannot add more instances of a container inside a running Pod to handle more load. A container inside a Pod is a fixed part of that Pod's definition.

### Scaling Pods (how Kubernetes works):

"My app is busy — I'll run more Pods, each containing the same set of containers."

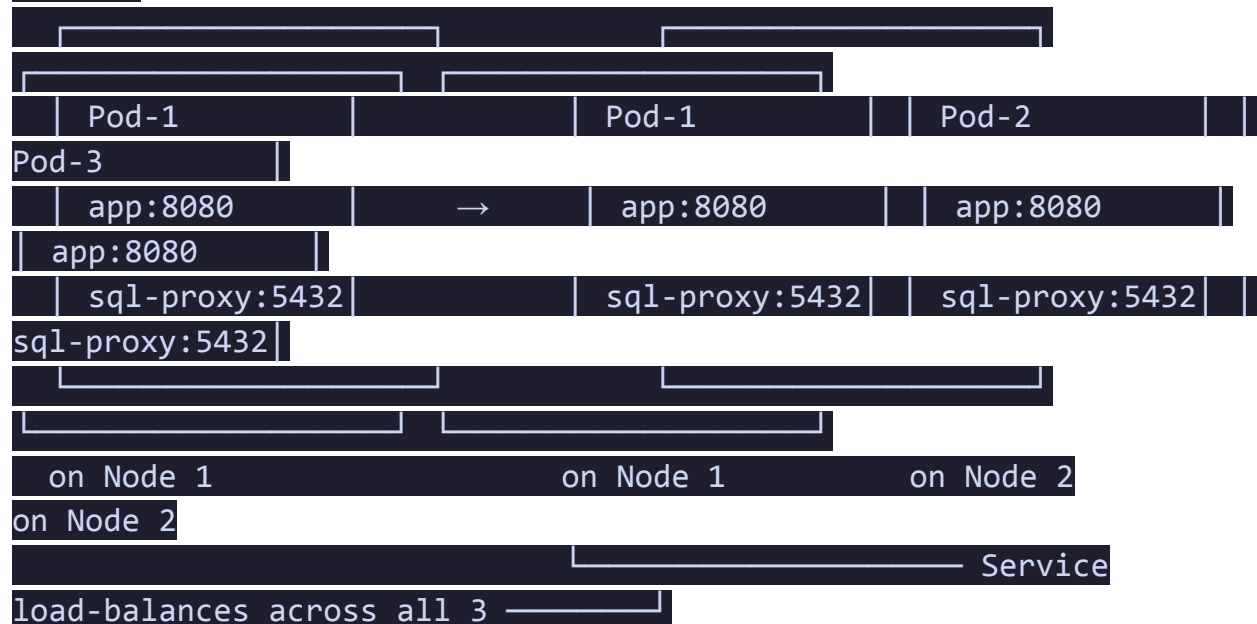
You scale the **number of Pod replicas**. Each replica is a full, independent copy of the Pod (with all its containers). Load balancing distributes traffic across them.

WRONG mental model — "scale the container":

Pod → Pod (now with 3 app containers inside?) ← K8s has no such concept

CORRECT mental model — "scale the Pod" (each Pod is an identical copy):

Before (1 replica):                      After (3 replicas, e.g. due to traffic spike):



Concrete example with real numbers:



You have a data platform API. Each Pod uses 500m CPU and 512Mi memory. Nodes are `n2-standard-4` (4 vCPU, 16 GB RAM, ~13 GB usable after system overhead).

| Replicas | CPU used | Memory used | Nodes needed                  |
|----------|----------|-------------|-------------------------------|
| 1 Pod    | 500m     | 512Mi       | 1 node (13 GB free remaining) |
| 5 Pods   | 2.5 vCPU | 2.5 GB      | 1 node (still fits)           |
| 10 Pods  | 5 vCPU   | 5 GB        | 2 nodes                       |
| 30 Pods  | 15 vCPU  | 15 GB       | 4 nodes                       |

The Cluster Autoscaler adds nodes as Pods grow. HPA decides the Pod count based on load.

**What each scaler actually does — in plain terms:**

| Scaler                                  | Analogy                             | What it changes               |
|-----------------------------------------|-------------------------------------|-------------------------------|
| <code>kubectl scale --replicas=5</code> | Hiring 5 workers                    | Pod count                     |
| HPA                                     | Auto-hire/fire based on queue depth | Pod count                     |
| VPA                                     | Give each worker a bigger desk      | Container CPU/memory requests |
| Cluster Autoscaler                      | Rent a new office floor             | Node (VM) count               |

Adding a second container to a Pod = hiring a support role alongside a worker (sidecar), not cloning the worker. To handle more load, you clone the whole Pod.

**The Kubernetes mental model — one sentence each:**

| Unit       | What it is                                               |
|------------|----------------------------------------------------------|
| Container  | One process running from an image                        |
| Pod        | The deployable unit — always runs on exactly ONE node    |
| Deployment | The scalable unit — controls how many Pod replicas exist |

---

**Pod Placement — One Pod, One Node (Always)**

A Pod is **always scheduled to exactly one node**. Kubernetes never splits a Pod across machines.

```
Cluster: Node1 (8 CPU, 32 GB) Node2 (8 CPU, 32 GB) Node3 (8 CPU, 32 GB)
```

```
Pod requesting 16 CPU, 64 GB:
```

```
Node1 ❌ (only 8 CPU available)
```

```
Node2 ❌ (only 8 CPU available)
```

```
Node3 ❌ (only 8 CPU available)
```

```
Kubernetes does NOT do:
```

```
CPU: 8 from Node1 + 8 from Node2 ← impossible
```

```
RAM: 32 GB from Node1 + 32 GB from Node2 ← impossible
```

```
Result: Pod Status = Pending (forever, until a node large enough exists)
```

**What to do when your Pod is bigger than any node:** use the Cluster Autoscaler — it provisions a larger VM (node) to fit the Pod. Without that, the Pod stays Pending.

This constraint also explains why you can't scale by adding containers inside one Pod. If three identical app containers are inside one Pod, they all live and die on the same machine together — you get no fault tolerance and no multi-node spreading.

---

### Same-Type Containers in a Pod — Why It Doesn't Work

Suppose you add two copies of the same app container to handle more traffic:

```
Pod
```

```
|— app-1 (port 8080)
```

```
|— app-2 (port 8080) ← IMPOSSIBLE – port conflict
```

**Problem 1 — Port conflicts.** Both containers share the Pod's single IP address and network namespace. Two containers cannot listen on the same port. One would have to use a different port (e.g. 8081), which means clients must know which port to use — defeating the purpose.

**Problem 2 — No automatic load balancing.** A Kubernetes Service routes traffic to a Pod IP, not to individual containers inside it. There is no built-in mechanism to distribute requests across app-1 and app-2 inside the same Pod.

```
Service
```

```
↓
```

```
Pod IP (10.0.0.5) ← Service sends traffic HERE
├── app-1 :8080
└── app-2 :8081 ← Kubernetes has no idea which container
should get each request
```

**Problem 3 — Shared fate, no redundancy.** If the node running this Pod crashes, both app-1 and app-2 die simultaneously. There is no redundancy.

**Problem 4 — Rolling updates break.** With multiple replicas of the same app inside one Pod, you cannot roll one copy to v2 while keeping another at v1. The whole Pod upgrades or restarts together.

The correct solution for all four problems is separate Pods behind a Service:

```
Service
├── Pod-1 app:8080 (Node1)
├── Pod-2 app:8080 (Node2) ← Service load-balances across
all three
└── Pod-3 app:8080 (Node2)
```

Each Pod can fail independently, be upgraded independently, and run on a different node.

---

## Pod Lifecycle — Phases

A Pod goes through a sequence of phases:

```
API receives Pod creation
├──
└── ▼
    PENDING ← scheduler is finding a node, or images are being
pulled
├──
└── ▼ (all containers started)
    RUNNING ← at least one container is running (others may still
be starting)
├──
└── ─▶ SUCCEEDED ← all containers exited with code 0
(Job)
├──
└── ─▶ FAILED ← at least one container exited non-zero
and won't restart
OR pod was evicted / OOMKilled
```

UNKNOWN ← node lost contact with API server; pod state cannot be determined

kubectl get pods # shows Phase

kubectl describe pod <name> # shows Phase + Conditions + Events

**Pod Conditions** (more granular than Phase):

| Condition       | Meaning                                                    |
|-----------------|------------------------------------------------------------|
| PodScheduled    | A node has been assigned                                   |
| Initialized     | All init containers completed successfully                 |
| ContainersReady | All containers passing their readiness probes              |
| Ready           | Pod is ready to receive traffic (Service will route to it) |

## Container States

Inside a running Pod, each container has its own state independently:

| State      | Meaning                                                                                 |
|------------|-----------------------------------------------------------------------------------------|
| Waiting    | Container is not yet running — pulling image, waiting on init containers, backoff delay |
| Running    | Container started successfully and is executing                                         |
| Terminated | Container finished (exit code 0 = success, non-zero = error)                            |

kubectl describe pod <name> # "Containers:" section shows each container's State

kubectl get pod <name> -o jsonpath='{.status.containerStatuses[\*].state}'

**CrashLoopBackOff** — what it actually means:

Container starts → crashes (non-zero exit) → kubelet restarts it  
restart delay: 10s → 20s → 40s → 80s → 160s → capped at 300s

Each restart is a new container process inside the same Pod.  
The Pod itself is still "Running" — it is the CONTAINER that is crashing.

```
kubectl logs --previous <pod-name> ← logs from the crashed
(previous) container
```

---

## restartPolicy

Controls what kubelet does when a container in the Pod exits:

| Policy                     | Behaviour                                | Use case                            |
|----------------------------|------------------------------------------|-------------------------------------|
| <b>Always</b><br>(default) | Restart on any exit — success or failure | Long-running services (Deployments) |
| <b>OnFailure</b>           | Restart only on non-zero exit            | Jobs that may fail and need retry   |
| <b>Never</b>               | Never restart                            | One-shot Jobs; debugging            |

```
spec:
  restartPolicy: OnFailure # set at Pod level, applies to all
containers
```

Note: **restartPolicy** is for the containers inside the Pod. The Pod object itself is managed by the Deployment/ReplicaSet controller — if the Pod is deleted or the node dies, the controller creates a new Pod (not a restart of the old one).

---

## Multi-Container Pod — Resource Isolation

When a Pod has multiple containers, each container has its own resource requests and limits. They do NOT share a resource pool.

```
spec:
  containers:
    - name: app
      resources:
        requests: { cpu: "500m", memory: "512Mi" }
        limits:   { cpu: "1",    memory: "1Gi" }
    - name: log-shipper
      resources:
        requests: { cpu: "100m", memory: "64Mi" }
        limits:   { cpu: "200m", memory: "128Mi" }

# Pod's total footprint seen by the scheduler:
# requests: cpu=600m, memory=576Mi
```

```
# limits:  cpu=1.2,  memory=1.1Gi
```

The scheduler places the Pod on a node that has enough capacity for the **total** requests of all containers. Each container is then individually throttled (CPU) or killed (memory) against its own limit.

---

### Full Pod Spec — Reference

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: my-app
```

```
  namespace: default
```

```
  labels:
```

```
    app: my-app
```

```
spec:
```

```
  restartPolicy: Always
```

```
  # Init containers run sequentially before main containers start
```

```
  initContainers:
```

```
    - name: db-migrate
```

```
      image: ../my-app:1.2.3
```

```
      command: ["python", "manage.py", "migrate"]
```

```
  containers:
```

```
    # Main application container
```

```
    - name: app
```

```
      image: asia-south1-docker.pkg.dev/my-project/repo/my-app:1.2.3
```

```
      ports:
```

```
        - containerPort: 8080
```

```
      env:
```

```
        - name: ENV
```

```
          value: "prod"
```

```
        - name: DB_PASSWORD
```

```
          valueFrom:
```

```
            secretKeyRef:
```

```
              name: db-secret
```

```
              key: password
```

```
      resources:
```

```
        requests:
```

```

      cpu: "500m"
      memory: "512Mi"
      limits:
        cpu: "1"
        memory: "1Gi"
      readinessProbe:
        httpGet: { path: /health, port: 8080 }
        initialDelaySeconds: 10
        periodSeconds: 5
      livenessProbe:
        httpGet: { path: /health, port: 8080 }
        initialDelaySeconds: 30
        periodSeconds: 10
      volumeMounts:
        - name: app-logs
          mountPath: /var/log/app

```

```

# Sidecar container – supports the main app, NOT a scale-out of
it

```

```

  - name: log-shipper
    image: fluent/fluent-bit:2.2
    resources:
      requests: { cpu: "100m", memory: "64Mi" }
      limits: { cpu: "200m", memory: "128Mi" }
    volumeMounts:
      - name: app-logs
        mountPath: /var/log/app
        readOnly: true

```

```

volumes:
  - name: app-logs
    emptyDir: {} # shared scratch space; deleted when
Pod is deleted

```

**Pods are ephemeral.** Never create naked Pods for production — use a Deployment so Kubernetes can reschedule them if a node fails.

### Container Fields — Quick Reference

The fields below cover ~95% of what you'll use day to day. For the complete spec (e.g. [lifecycle hooks](#), [stdin](#), [tty](#), [securityContext](#) sub-fields), see the official

reference:

<https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/pod-v1/#Container>

Every container definition in a Pod spec can use these fields:

```
containers:
  - name: app                                # required; unique within
the Pod
    image: my-api:1.2.3                      # required;
registry/image:tag
    imagePullPolicy: IfNotPresent           # Always | IfNotPresent |
Never

    command: ["python"]                     # overrides Docker
ENTRYPOINT
    args: ["server.py", "--port=8080"]      # overrides Docker CMD

  ports:
    - containerPort: 8080                   # informational only;
doesn't open firewall
      protocol: TCP

  env:                                       # environment variables
    - name: ENV
      value: "prod"
    - name: DB_PASS
      valueFrom:
        secretKeyRef: { name: db-secret, key: password }
    - name: MY_POD_NAME
      valueFrom:
        fieldRef: { fieldPath: metadata.name } # downward API

  envFrom:                                  # load all keys from a
ConfigMap/Secret
    - configMapRef: { name: app-config }
    - secretRef:    { name: app-secrets }

  resources:
    requests: { cpu: "250m", memory: "256Mi" }
```



```
    limits: { cpu: "1", memory: "512Mi" }
```

```
    volumeMounts:  
      - name: config-vol  
        mountPath: /etc/app  
        readOnly: true  
      - name: data-vol  
        mountPath: /data  
        subPath: myapp/ # mount only a  
subdirectory of the volume
```

```
    livenessProbe: { ... }  
    readinessProbe: { ... }  
    startupProbe: { ... }
```

```
    lifecycle:  
      postStart: # runs immediately after  
container starts  
      exec:  
        command: ["/bin/sh", "-c", "echo started >>  
/var/log/startup.log"]  
      preStop: # runs before container is  
terminated  
      exec:  
        command: ["sleep", "5"] # give load balancer time  
to drain
```

```
    securityContext:  
      runAsNonRoot: true  
      runAsUser: 1001  
      readOnlyRootFilesystem: true  
      allowPrivilegeEscalation: false  
      capabilities:  
        drop: ["ALL"]
```

```
    # terminationMessagePolicy: FallbackToLogsOnError  
    # ^ if the app doesn't write to /dev/termination-log, K8s will  
use the last
```

```
# few lines of stderr as the termination message shown in  
kubectl describe.
```

```
# Useful for debugging crash reasons. Leave default (File) if  
unsure.
```

**Note:** There are many more rarely-used container fields (`stdin`, `tty`, `volumeDevices`, etc.). The above covers what you'll actually need. For the complete API reference:

<https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/pod-v1/#Container>

## Init Containers

Run to completion **sequentially** before any main containers start. If an init container fails, the Pod stays in `Init:Error` and kubelet retries according to `restartPolicy`. Use for: database migrations, config generation, waiting for a dependency (`until curl -s http://postgres:5432; do sleep 2; done`).

## Sidecar Containers

Run **alongside** the main container in the same Pod, for the lifetime of the Pod. They support the main container — they do not add capacity to it. Use for: log forwarding to Cloud Logging, Envoy/Istio service mesh proxy, secret rotation agents, Cloud SQL Auth Proxy.

**Kubernetes 1.29+ native sidecar:** mark a container as a sidecar with `restartPolicy: Always` inside `initContainers` — it starts before main containers and is kept running for the Pod's lifetime, unlike a regular init container that exits.

```
initContainers:
```

```
- name: cloud-sql-proxy
```

```
  image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2
```

```
  restartPolicy: Always           # native sidecar – stays running
```

```
  args: ["--port=5432", "my-project:asia-south1:my-db"]
```

---

## 6. Workload Controllers

### Deployment

The standard way to run stateless applications. Manages a `ReplicaSet`, which manages Pods. Supports rolling updates and rollback.

### Object ownership hierarchy:

```
Deployment (desired state – "run 3 replicas of my-api:1.2.3")
```

```
  | owns / manages
```

```
  | — ReplicaSet (current template snapshot – "3 pods of  
my-api:1.2.3")
```

|   |                                        |
|---|----------------------------------------|
|   | owns / manages                         |
| — | Pod (my-api:1.2.3 – running on node-1) |
| — | Pod (my-api:1.2.3 – running on node-2) |
| — | Pod (my-api:1.2.3 – running on node-3) |

On image update (my-api:1.2.3 → 1.2.4):

| Old ReplicaSet (1.2.3)                                                                        |   | New ReplicaSet (1.2.4)                      |
|-----------------------------------------------------------------------------------------------|---|---------------------------------------------|
| 3 pods running                                                                                | → | 0 pods initially                            |
| <div> <div></div> <div></div> <div></div> </div>                                              |   |                                             |
| <div> <div></div> <div>Rolling update (maxUnavailable=0, maxSurge=1)</div> <div></div> </div> |   |                                             |
| <div> <div></div> <div></div> <div></div> </div>                                              |   |                                             |
| 2 pods running                                                                                |   | 1 pod running ← new pod becomes Ready first |
| 1 pod running                                                                                 |   | 2 pods running                              |
| 0 pods running                                                                                |   | 3 pods running ✓ done                       |
| <div> <div></div> <div></div> </div>                                                          |   |                                             |
| (kept for rollback, scaled to 0)                                                              |   |                                             |

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-api
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # can temporarily have 4 pods during
update
      maxUnavailable: 0  # never go below 3 healthy pods
  template:
    metadata:
      labels:
        app: my-api

```

```

spec:
  containers:
    - name: api
      image: my-api:1.2.3
      ports:
        - containerPort: 8080
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
# Apply
kubectl apply -f deployment.yaml

# Check rollout
kubectl rollout status deployment/my-api

# History
kubectl rollout history deployment/my-api

# Rollback to previous version
kubectl rollout undo deployment/my-api

# Rollback to specific revision
kubectl rollout undo deployment/my-api --to-revision=3

# Scale manually
kubectl scale deployment my-api --replicas=5

# Update image (triggers rolling update)
kubectl set image deployment/my-api api=my-api:1.2.4

```

## StatefulSet

Like a Deployment but for stateful workloads. Pods get:

- Stable, predictable names: `my-db-0`, `my-db-1`, `my-db-2`
- Stable network identities (DNS:  
`my-db-0.my-db.namespace.svc.cluster.local`)

- Ordered startup and shutdown (0 before 1 before 2)
- Persistent volume claims per Pod (each Pod gets its own PVC)

Use for: databases (PostgreSQL, Kafka, Zookeeper, Redis), Trino if using local storage.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres # must match a headless Service
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
      volumeClaimTemplates: # each Pod gets its own PVC
        - metadata:
            name: data
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: premium-rwo
            resources:
              requests:
                storage: 100Gi
```

## DaemonSet

Runs exactly one Pod on every node (or a subset of nodes). Used for node-level agents.

Use for: log collectors (Fluent Bit), monitoring agents (node exporter), network plugins, security agents.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: fluent-bit
  template:
    metadata:
      labels:
        app: fluent-bit
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:2.2
```

## Job

Runs a Pod to completion — when the container exits with code 0, the Job is done.  
For one-off tasks.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-backfill
spec:
  parallelism: 4      # run 4 pods simultaneously
  completions: 20     # total successful completions needed
  backoffLimit: 3     # retry failed pods up to 3 times
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: backfill
```

```

    image: my-pipeline:1.0
    command: ["python", "backfill.py"]
CronJob
Runs a Job on a cron schedule.
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-aggregation
spec:
  schedule: "0 2 * * *"          # 2 AM daily
  concurrencyPolicy: Forbid      # don't run if previous is still
running
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: aggregator
              image: my-pipeline:1.0
              command: ["python", "aggregate.py"]

```

---

## 7. Services & Networking

---

### How Pod Networking Works

Every Pod gets a unique IP address within the cluster. Pods can reach each other directly by IP — but Pod IPs change when Pods are restarted. **Services** provide a stable IP and DNS name that routes to healthy Pods.

#### How traffic reaches a Pod through a Service:

```

External / Internal caller
└─┬─┘
   │ DNS lookup: "my-api-svc.default.svc.cluster.local"
   ▼
CoreDNS → returns ClusterIP (e.g. 10.96.45.12) – stable,
never changes
└─┬─┘
   ▼

```

```

kube-proxy (iptables rules on each node)
└─ load balances across healthy Pod IPs
    └─► Pod A (10.20.1.5:8080) ← only Pods passing
readinessProbe
    └─► Pod B (10.20.2.7:8080)
    └─► Pod C (10.20.3.2:8080)

```

If a Pod is deleted → kube-proxy removes it from iptables immediately

If a new Pod starts → added only after readinessProbe passes

**ClusterIP vs NodePort vs LoadBalancer at a glance:**

```

ClusterIP — internal only —► Pod
              (cluster DNS)

```

```

NodePort — NodeIP:30080 —► Service —► Pod
              (any node IP)

```

```

LoadBalancer— GCP External IP —► GCP LB —► NodePort —►
Service —► Pod
              (public internet)

```

## Service Types

### ClusterIP (default)

Internal service — reachable only within the cluster. Kubernetes assigns a stable virtual IP; kube-proxy routes traffic to a healthy Pod.

```

apiVersion: v1
kind: Service
metadata:
  name: my-api-svc
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: my-api          # routes to Pods with this label
  ports:
    - port: 80           # port on the Service
      targetPort: 8080   # port on the Pod

```

DNS within cluster: `my-api-svc.default.svc.cluster.local` or just `my-api-svc` from the same namespace.



## NodePort

Exposes the Service on a port on every node's IP (30000–32767 range). Mostly used for debugging. Not recommended for production.

```
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080    # optional; auto-assigned if omitted
```

## LoadBalancer

Provisions a GCP load balancer (external by default). The load balancer gets a public IP that routes to the Service.

```
spec:
  type: LoadBalancer
  ports:
    - port: 443
      targetPort: 8080
  annotations:
    cloud.google.com/load-balancer-type: "Internal" # internal LB only
```

## Headless Service

No ClusterIP assigned. DNS returns Pod IPs directly. Required for StatefulSets — lets clients address individual pods by DNS name.

```
spec:
  clusterIP: None
  selector:
    app: postgres
```

DNS: `postgres-0.postgres.namespace.svc.cluster.local` → Pod 0's IP.

## DNS in Kubernetes

CoreDNS runs in the cluster and resolves:

- `<service>` — same namespace
- `<service>.<namespace>` — any namespace
- `<service>.<namespace>.svc.cluster.local` — fully qualified
- `<pod-ip-dashes>.<namespace>.pod.cluster.local` — Pod DNS (rarely used)

## NetworkPolicy

Firewall rules for Pod-to-Pod traffic. By default all Pods can reach all other Pods. NetworkPolicy restricts this.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: trino-worker-policy
  namespace: trino
spec:
  podSelector:
    matchLabels:
      component: worker
  ingress:
    - from:
        - podSelector:
            matchLabels:
              component: coordinator # only coordinator can reach
workers
      ports:
        - port: 8080
    egress:
      - to:
          - namespaceSelector:
              matchLabels:
                name: monitoring # allow metrics scraping
      ports:
        - port: 8080

```

---

## 8. Ingress & Gateway API

---

### Ingress

Ingress routes external HTTP/HTTPS traffic to Services. Requires an **Ingress Controller** (nginx, GKE Ingress, Traefik, etc.) to implement.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    kubernetes.io/ingress.class: "gce" # GKE managed
LB
    kubernetes.io/ingress.global-static-ip-name: "my-static-ip"
    networking.gke.io/managed-certificates: "my-cert"

```

```
spec:
  rules:
    - host: api.companyABC.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api-v1-svc
                port:
                  number: 80
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: api-v2-svc
                port:
                  number: 80
    - host: dashboard.companyABC.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: dashboard-svc
                port:
                  number: 80
```

### **GKE Managed Certificate**

```
apiVersion: networking.gke.io/v1
kind: ManagedCertificate
metadata:
  name: my-cert
spec:
  domains:
    - api.companyABC.com
    - dashboard.companyABC.com
```

GKE provisions and auto-renews the SSL certificate via Google Certificate Manager.

### Gateway API (modern replacement for Ingress)

Gateway API is the successor to Ingress — more expressive, supports multi-team usage, and integrates with GCP's load balancer natively.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
spec:
  parentRefs:
    - name: my-gateway
  hostnames:
    - api.companyABC.com
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /v1
      backendRefs:
        - name: api-v1-svc
          port: 80
```

---

## 9. Configuration — ConfigMaps & Secrets

### ConfigMap

Store non-sensitive configuration — environment variables, config files, command-line arguments.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  config.yaml: |
    server:
      port: 8080
      timeout: 30s
    database:
```

```

    host: postgres-svc
    port: 5432
Use in a Pod:
spec:
  containers:
  - name: app
    envFrom:
    - configMapRef:
        name: app-config          # all keys become env vars
    volumeMounts:
    - name: config-vol
      mountPath: /etc/app
  volumes:
  - name: config-vol
    configMap:
      name: app-config          # mount as files

```

## Secret

Store sensitive data — passwords, API keys, TLS certs. Values are base64-encoded (not encrypted at rest by default — enable Envelope Encryption with KMS for production).

```

apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:          # plain text; auto-encoded to
base64
  password: "super-secret-password"
  connection-string: "postgres://user:password@postgres:5432/db"

```

Use in a Pod:

```

env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password

```

**Production approach:** do not store Secrets in Git. Use GCP Secret Manager and the **External Secrets Operator** to sync secrets from Secret Manager into Kubernetes Secrets automatically.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-secret
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: gcp-secretstore
    kind: ClusterSecretStore
  target:
    name: db-secret
  data:
    - secretKey: password
      remoteRef:
        key: projects/my-project/secrets/db-password
        version: latest
```

---

## 10. Storage — Volumes, PVCs, StorageClasses

---

### Volume Types

| Type                                         | Description                                               | Lifecycle                                     |
|----------------------------------------------|-----------------------------------------------------------|-----------------------------------------------|
| <code>emptyDir</code>                        | Temporary scratch space, shared between containers in Pod | Deleted when Pod is deleted                   |
| <code>hostPath</code>                        | Mount a directory from the node's filesystem              | Tied to specific node — avoid for portability |
| <code>configMap</code> / <code>secret</code> | Mount ConfigMap or Secret as files                        | Managed by Kubernetes                         |
| <code>persistentVolumeClaim</code>           | Attach a persistent volume (GCP Persistent Disk)          | Independent of Pod lifecycle                  |
| <code>nfs</code>                             | Mount an NFS share (Filestore)                            | Independent of Pod lifecycle                  |

|           |                                         |  |
|-----------|-----------------------------------------|--|
| projected | Combine multiple sources into one mount |  |
|-----------|-----------------------------------------|--|

## PersistentVolume (PV) & PersistentVolumeClaim (PVC)

**PersistentVolume:** a piece of storage provisioned by an admin or dynamically by a StorageClass. **PersistentVolumeClaim:** a request for storage by a Pod — specifies size and access mode.

With dynamic provisioning (StorageClass), you only need to create a PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: trino-data
spec:
  accessModes:
    - ReadWriteOnce      # RWO: one node at a time (block storage)
  storageClassName: premium-rwo
  resources:
    requests:
      storage: 200Gi
```

Access modes:

- **ReadWriteOnce (RWO)**: single node read-write — GCP Persistent Disk
- **ReadOnlyMany (ROX)**: multiple nodes read-only
- **ReadWriteMany (RWX)**: multiple nodes read-write — requires Filestore (NFS)

## StorageClasses on GKE

GKE provides built-in StorageClasses for GCP Persistent Disks:

| StorageClass | Disk type         | IOPS      | Use case                   |
|--------------|-------------------|-----------|----------------------------|
| standard     | pd-standard (HDD) | Low       | Cheap sequential storage   |
| standard-rwo | pd-balanced (SSD) | Medium    | General purpose            |
| premium-rwo  | pd-ssd (SSD)      | High      | Databases, high-throughput |
| pd-extreme   | pd-extreme (SSD)  | Very high | Latency-critical           |

```
kubectl get storageclasses
```

Create a custom StorageClass:

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
```

```
metadata:
```

```
  name: fast-ssd
```

```
provisioner: pd.csi.storage.gke.io
```

```
parameters:
```

```
  type: pd-ssd
```

```
  replication-type: regional-pd    # replicate across 2 zones
```

```
reclaimPolicy: Retain                # don't delete disk when PVC is  
deleted
```

```
allowVolumeExpansion: true          # allow resizing
```

```
volumeBindingMode: WaitForFirstConsumer # provision in same zone as  
Pod
```

---

## 11. Resource Management — Requests, Limits, QoS

---

### Requests vs Limits

```
resources:
```

```
  requests:
```

```
    cpu: "500m"    # 500 millicores = 0.5 vCPU – guaranteed; used  
for scheduling
```

```
    memory: "512Mi" # guaranteed memory
```

```
  limits:
```

```
    cpu: "2"        # 2 vCPU – container is throttled if it  
exceeds this
```

```
    memory: "2Gi"   # container is OOMKilled if it exceeds this
```

- **Requests:** what the container is *guaranteed*. The scheduler uses requests to decide where to place Pods.
- **Limits:** the container is throttled (CPU) or killed (memory) if it exceeds this.

**CPU units:** 1 = 1 vCPU core. 500m = 0.5 vCPU. 100m = 0.1 vCPU. **Memory units:** Ki, Mi, Gi (powers of 1024). K, M, G (powers of 1000).

### Quality of Service (QoS) Classes

Kubernetes assigns a QoS class based on requests/limits, which determines eviction priority when a node is under memory pressure:

| QoS Class | Condition | Eviction priority |
|-----------|-----------|-------------------|
|-----------|-----------|-------------------|



|                   |                                       |                     |
|-------------------|---------------------------------------|---------------------|
| <b>Guaranteed</b> | requests == limits for ALL containers | Last to be evicted  |
| <b>Burstable</b>  | requests < limits (or limits not set) | Middle              |
| <b>BestEffort</b> | No requests or limits set             | First to be evicted |

For production: set requests and limits on all containers. For system-critical Pods (monitoring, networking), use Guaranteed QoS.

### LimitRange

Default requests/limits for a namespace — prevents Pods without resources from being scheduled.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: data-platform
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      max:
        cpu: "8"
        memory: "16Gi"
```

### ResourceQuota

Limit total resources consumed by a namespace:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: data-platform-quota
  namespace: data-platform
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
```

```
limits.cpu: "40"
limits.memory: 80Gi
pods: "50"
persistentvolumeclaims: "20"
```

---

## 12. Scheduling — Node Selection, Affinity, Taints

---

### nodeSelector

Simple label-based node selection:

```
spec:
  nodeSelector:
    workload: trino
    cloud.google.com/gke-spot: "true"
```

### Node Affinity

More expressive than nodeSelector — supports preferred (soft) rules and operators.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution: # hard rule
        nodeSelectorTerms:
          - matchExpressions:
              - key: workload
                operator: In
                values: [trino, data]
        preferredDuringSchedulingIgnoredDuringExecution: # soft rule
          - weight: 100
            preference:
              matchExpressions:
                - key: cloud.google.com/gke-spot
                  operator: In
                  values: ["true"]
```

### Pod Affinity & Anti-Affinity

Schedule Pods relative to other Pods (co-locate or spread):

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
```

```
app: my-api
```

```
topologyKey: kubernetes.io/hostname # one pod per node
```

This ensures each replica is on a different node — important for HA.

## Taints & Tolerations

Taints mark a node to repel Pods. Tolerations allow specific Pods to be scheduled on tainted nodes.

```
# Taint a node pool for dedicated Trino workloads
```

```
kubectl taint nodes -l workload=trino dedicated=trino:NoSchedule
```

Pods that should run on tainted nodes:

```
spec:
```

```
  tolerations:
```

```
    - key: dedicated
```

```
      operator: Equal
```

```
      value: trino
```

```
      effect: NoSchedule
```

### Taint effects:

- **NoSchedule**: new Pods without toleration won't be scheduled here.
- **PreferNoSchedule**: scheduler tries to avoid the node but isn't forced.
- **NoExecute**: existing Pods without toleration are evicted.

## Topology Spread Constraints

Distribute Pods evenly across zones or nodes:

```
spec:
```

```
  topologySpreadConstraints:
```

```
    - maxSkew: 1 # max difference in
```

```
Pod count across zones
```

```
      topologyKey: topology.kubernetes.io/zone
```

```
      whenUnsatisfiable: DoNotSchedule
```

```
      labelSelector:
```

```
        matchLabels:
```

```
          app: my-api
```

## PriorityClass — Pod Scheduling Priority

When the cluster is under pressure (not enough CPU/memory), the scheduler needs to decide which Pods to evict. **PriorityClass** controls that order.

**Data platform use case:** Trino worker Pods should never be killed to make room for batch pipeline jobs. Assign a high priority to Trino, low to batch.

```
# High priority – for production services and Trino workers
```

```
apiVersion: scheduling.k8s.io/v1
```

```
kind: PriorityClass
```

```
metadata:
  name: high-priority
value: 1000
globalDefault: false
description: "Production services and Trino workers"
```

```
---
# Low priority – for batch pipeline jobs (can be preempted)
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 100
globalDefault: false
description: "Batch jobs and non-critical workers"
```

Use in a Pod spec:

```
spec:
  priorityClassName: high-priority
  containers:
    - name: trino-worker
    ...
```

**How preemption works:** if a high-priority Pod is Pending and the node is full, the scheduler evicts lower-priority Pods on that node to make room. Those evicted Pods go back to Pending and get rescheduled elsewhere.

GKE built-in values for reference: `system-node-critical` (2,000,001,000) and `system-cluster-critical` (2,000,000,000) are reserved for Kubernetes system Pods.

---

### 13. Autoscaling — HPA, VPA, KEDA, Cluster Autoscaler

Horizontal vs Vertical scaling — what each actually means:

| Scaling type | What changes                                        | K8s mechanism                       |
|--------------|-----------------------------------------------------|-------------------------------------|
| Horizontal   | More Pod replicas — same resource size, more copies | HPA /<br><code>kubectl scale</code> |

|                                   |                                                                |     |
|-----------------------------------|----------------------------------------------------------------|-----|
| <b>Vertical</b>                   | Bigger Pod — same number of copies, more CPU/memory per Pod    | VPA |
| <b>Adding containers to a Pod</b> | Neither — this is adding sidecars, not scaling the application | N/A |

KEDA (optional, event-driven):

Replaces HPA for Pub/Sub queue depth, Kafka lag, HTTP req rate, etc.

Can scale to ZERO replicas when no events – HPA cannot.

**Typical scaling sequence under sudden traffic spike:**

High load arrives



HPA detects CPU > 70%

adds more Pod replicas



Cluster Autoscaler detects Pods are Pending (nodes full)

provisions new node from node pool (GCP VM starts ~60-90s)



New Pods scheduled on new node, readiness probe passes

traffic distributed across more Pods



Load subsides → HPA scales Pods down → Cluster Autoscaler removes idle nodes

**Horizontal Pod Autoscaler (HPA)**

Scales Pod replicas based on CPU, memory, or custom metrics.

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: my-api-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: my-api

minReplicas: 2

maxReplicas: 20

metrics:

- type: Resource

resource:

name: cpu

target:

```

      type: Utilization
      averageUtilization: 70    # scale when avg CPU > 70%
    - type: Resource
      resource:
        name: memory
      target:
        type: Utilization
        averageUtilization: 80

```

```
kubectl get hpa
```

```
kubectl describe hpa my-api-hpa
```

### Vertical Pod Autoscaler (VPA)

Automatically adjusts Pod CPU and memory **requests** based on historical usage.

Does not change replica count.

```
apiVersion: autoscaling.k8s.io/v1
```

```
kind: VerticalPodAutoscaler
```

```
metadata:
```

```
  name: my-api-vpa
```

```
spec:
```

```
  targetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: my-api
```

```
  updatePolicy:
```

```
    updateMode: "Auto"    # Off: recommend only. Initial: set on
                           creation. Auto: restart pods with new values.
```

```
  resourcePolicy:
```

```
    containerPolicies:
```

```
      - containerName: api
```

```
        minAllowed:
```

```
          cpu: 100m
```

```
          memory: 128Mi
```

```
        maxAllowed:
```

```
          cpu: 4
```

```
          memory: 8Gi
```

**Note:** VPA and HPA cannot both scale on CPU/memory at the same time. Use HPA for replica scaling + VPA in "Recommendation" mode to tune requests.

### KEDA (Kubernetes Event-Driven Autoscaling)

Scales Deployments based on external event sources — Pub/Sub queue depth, Kafka lag, HTTP request rate, etc. More powerful than HPA for event-driven workloads.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: pipeline-worker-scaler
spec:
  scaleTargetRef:
    name: pipeline-worker
  minReplicaCount: 0          # scale to zero when idle
  maxReplicaCount: 50
  triggers:
    - type: gcp-pubsub
      metadata:
        subscriptionName: app-events-sub
        value: "100"          # scale up when > 100 unprocessed
      messages
```

### Cluster Autoscaler

Scales the number of **nodes** in a node pool. When Pods are pending (not enough node capacity), Cluster Autoscaler adds nodes. When nodes are underutilised, it removes them.

Enable on GKE node pool:

```
gcloud container node-pools update default-pool \
  --cluster=companyABC-cluster \
  --region=asia-south1 \
  --enable-autoscaling \
  --min-nodes=1 \
  --max-nodes=20
```

---

## 14. Health Checks — Probes

Kubernetes uses three types of probes to monitor container health:

| Probe           | What it does            | On failure                        |
|-----------------|-------------------------|-----------------------------------|
| <b>Liveness</b> | Is the container alive? | Container is killed and restarted |



|                  |                                            |                                                       |
|------------------|--------------------------------------------|-------------------------------------------------------|
| <b>Readiness</b> | Is the container ready to receive traffic? | Pod is removed from Service endpoints (traffic stops) |
| <b>Startup</b>   | Has the container finished starting?       | Container is killed if startup takes too long         |

### Probe Methods

# HTTP GET – most common

livenessProbe:

httpGet:

path: /healthz

port: 8080

initialDelaySeconds: 30 # wait before first check

periodSeconds: 10 # check every 10s

failureThreshold: 3 # restart after 3 failures

successThreshold: 1

timeoutSeconds: 5

# TCP Socket – for non-HTTP services

readinessProbe:

tcpSocket:

port: 5432

initialDelaySeconds: 5

periodSeconds: 5

# Exec – run a command

livenessProbe:

exec:

command: ["pg\_isready", "-U", "postgres"]

periodSeconds: 10

# gRPC

livenessProbe:

grpc:

port: 50051

periodSeconds: 10

**Startup probe** — use for slow-starting containers so liveness probe doesn't kill them during startup:

startupProbe:

```
httpGet:
  path: /healthz
  port: 8080
failureThreshold: 30    # allow up to 300s (30 × 10s) for startup
periodSeconds: 10
```

---

## 15. RBAC — Role-Based Access Control

---

RBAC controls who can do what in the Kubernetes API.

**Four objects:**

- **Role**: permissions in a specific namespace
- **ClusterRole**: permissions cluster-wide (or for non-namespaced resources)
- **RoleBinding**: grant a Role to a subject in a namespace
- **ClusterRoleBinding**: grant a ClusterRole to a subject cluster-wide

```
# Role: allow reading pods and logs in the trino namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: trino-reader
  namespace: trino
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log", "services", "configmaps"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch"]
```

```
---
# Bind the role to a user
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: trino-reader-binding
  namespace: trino
subjects:
- kind: User
  name: userX@companyABC.com
```

```

    apiGroup: rbac.authorization.k8s.io
  - kind: ServiceAccount
    name: ci-service-account
    namespace: ci-cd
  roleRef:
    kind: Role
    name: trino-reader
    apiGroup: rbac.authorization.k8s.io
# Check what a user can do
kubectl auth can-i list pods --namespace trino
--as=userX@companyABC.com

# List all bindings in a namespace
kubectl get rolebindings -n trino
kubectl get clusterrolebindings

# Describe permissions
kubectl describe clusterrole cluster-admin

```

---

## 16. CRDs — Custom Resource Definitions

Kubernetes ships with built-in resource types: Pod, Deployment, Service, etc. **CRDs let you add your own resource types** so that Kubernetes can manage custom concepts in the same way it manages built-in ones.

Think of it like this: once someone installs a CRD, you can `kubectl get <that-resource>` / `kubectl apply -f <that-resource>.yaml` just like any built-in object.

**Your data platform already uses CRDs — you just might not have noticed:**

| CRD            | Installed by              | Example resource                          |
|----------------|---------------------------|-------------------------------------------|
| ScaledObject   | KEDA                      | Scales Deployments based on Pub/Sub depth |
| ExternalSecret | External Secrets Operator | Syncs GCP Secret Manager → K8s Secret     |
| PodMonitoring  | Google Managed Prometheus | Tells Prometheus how to scrape a Pod      |

|                      |              |                                                      |
|----------------------|--------------|------------------------------------------------------|
| <b>Certificate</b>   | cert-manager | Provisions and rotates TLS certs                     |
| <b>BackendConfig</b> | GKE          | Configures Cloud Armor, session affinity on a GKE LB |

You don't write CRDs unless you're building a Kubernetes Operator. You just install charts that bring their CRDs with them, then use those resources.

**What an operator is (brief):** A controller + a CRD. The operator watches for instances of its CRD and takes action. For example, the External Secrets Operator watches for **ExternalSecret** objects and creates **Secret** objects from GCP Secret Manager. Under the hood it's just a Go program running in a Pod.

```
# List all CRDs in the cluster
```

```
kubectl get crds
```

```
# See all ExternalSecret objects across namespaces
```

```
kubectl get externalsecrets -A
```

```
# See all ScaledObjects
```

```
kubectl get scaledobjects -A
```

For writing your own CRDs and operators, see the [Kubernetes operator pattern docs](#) and the [Operator SDK](#).

---

## 17. Helm — Package Manager

Helm is the package manager for Kubernetes. A **Chart** is a packaged set of Kubernetes manifests with templating. Equivalent to apt/npm for Kubernetes.

### Concepts

- **Chart:** the package — a directory of templates + **values.yaml** + **Chart.yaml**
- **Release:** an installed instance of a chart in the cluster
- **Repository:** a collection of charts (like an npm registry)
- **values.yaml:** default configuration; override at install time

### Essential Commands

```
# Add a chart repository
```

```
helm repo add trino https://trinodb.github.io/charts
```

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
helm repo update
```

```
# Search for charts
```

```
helm search repo trino
```

```
helm search hub airflow
```

```
# Install a chart
```

```
helm install my-release bitnami/postgresql \
  --namespace postgres \
  --create-namespace \
  --values my-values.yaml \
  --set auth.password=mysecretpassword
```

```
# Upgrade (update config or chart version)
```

```
helm upgrade my-release bitnami/postgresql \
  --namespace postgres \
  --values my-values.yaml \
  --reuse-values          # keep values not specified in --values
```

```
# Upgrade or install (idempotent)
```

```
helm upgrade --install my-release bitnami/postgresql \
  --namespace postgres \
  --create-namespace \
  --values my-values.yaml
```

```
# Check status
```

```
helm list -n postgres
helm status my-release -n postgres
```

```
# View what will change (dry run + diff with helm-diff plugin)
```

```
helm diff upgrade my-release bitnami/postgresql --values
my-values.yaml
```

```
# Rollback
```

```
helm rollback my-release 1 -n postgres # rollback to revision 1
```

```
# Uninstall
```

```
helm uninstall my-release -n postgres
```

```
# Show default values
```

```
helm show values bitnami/postgresql
```

```
# Render templates locally (debugging)
helm template my-release bitnami/postgresql --values my-values.yaml
```

## Writing a Chart

my-chart/

```
├── Chart.yaml          # chart metadata
├── values.yaml         # default values
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── _helpers.tpl    # reusable template snippets
│   └── NOTES.txt       # printed after install
└── charts/             # chart dependencies
```

Chart.yaml:

```
apiVersion: v2
name: my-api
description: companyABC Data Platform API
version: 1.2.3
appVersion: "1.2.3"
```

values.yaml:

```
replicaCount: 2
image:
  repository: asia-south1-docker.pkg.dev/my-project/repo/my-api
  tag: "1.2.3"
  pullPolicy: IfNotPresent
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 1
    memory: 512Mi
```

templates/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-api.fullname" . }}
  labels:
```

```

    {{- include "my-api.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{
.Values.image.tag }}"
          resources:
            {{- toYaml .Values.resources | nindent 12 }}

```

## Helm Templating — Essential Syntax

Helm templates use Go's **text/template** syntax. The `{{- ... -}}` dashes trim whitespace.

### Variables:

```

# .Values = your values.yaml content
# .Chart = Chart.yaml content
# .Release = release metadata (name, namespace, etc.)

```

```

name: {{ .Values.image.tag }}
release: {{ .Release.Name }}
chart: {{ .Chart.Name }}-{{ .Chart.Version }}
namespace: {{ .Release.Namespace }}

```

### Conditionals:

```

{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}

```

```

# if / else
{{- if eq .Values.env "prod" }}
  replicas: 3
{{- else }}
  replicas: 1
{{- end }}

```

### Default values and required:

```

# Use a default if value is not set

```

```
image: {{ .Values.image.tag | default "latest" }}
```

```
# Fail at render time if value is missing (good for required secrets)
```

```
password: {{ required "database.password must be set" .Values.database.password }}
```

### Loops (range):

```
# Loop over a list
```

```
env:
```

```
{{- range .Values.extraEnv }}
```

```
  - name: {{ .name }}
```

```
    value: {{ .value | quote }}
```

```
{{- end }}
```

```
# Loop over a map
```

```
annotations:
```

```
{{- range $key, $value := .Values.annotations }}
```

```
  {{ $key }}: {{ $value | quote }}
```

```
{{- end }}
```

### String functions:

```
# quote wraps in double quotes (use for string env vars)
```

```
value: {{ .Values.someString | quote }}
```

```
# nindent = newline + indent – used after toYaml
```

```
resources:
```

```
  {{- toYaml .Values.resources | nindent 2 }}
```

```
# upper, lower, trimAll, replace
```

```
name: {{ .Values.name | lower | replace " " "-" }}
```

### Named templates (**\_helpers.tpl**):

```
# _helpers.tpl – define reusable snippets
```

```
{{- define "my-api.fullname" -}}
```

```
{{ .Release.Name }}-{{ .Chart.Name }}
```

```
{{- end }}
```

```
{{- define "my-api.labels" -}}
```

```
app: {{ include "my-api.fullname" . }}
```

```
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
```



```
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

```
# Use in any template
```

```
metadata:
```

```
  name: {{ include "my-api.fullname" . }}
```

```
  labels:
```

```
    {{- include "my-api.labels" . | nindent 4 }}
```

## Multi-Environment Values Pattern

The most common real-world pattern: one base `values.yaml` with defaults, then environment-specific override files. Only the differences go in the override file.

```
my-chart/
```

```
|— values.yaml          # shared defaults (all environments
start here)
```

```
|— values-dev.yaml      # dev overrides
```

```
|— values-staging.yaml  # staging overrides
```

```
|— values-prod.yaml     # prod overrides
```

`values.yaml` (shared defaults):

```
replicaCount: 1
```

```
image:
```

```
  repository: asia-south1-docker.pkg.dev/my-project/repo/my-api
```

```
  tag: "latest"
```

```
resources:
```

```
  requests:
```

```
    cpu: 250m
```

```
    memory: 256Mi
```

```
  limits:
```

```
    cpu: 500m
```

```
    memory: 512Mi
```

```
ingress:
```

```
  enabled: false
```

```
  host: ""
```

`values-prod.yaml` (only what changes):

```
replicaCount: 3
```

```
image:
```

```
  tag: "1.4.2"          # pin to a specific version in prod
```

```
resources:
```

```
  requests:
```

```
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: 2
    memory: 1Gi
ingress:
  enabled: true
  host: api.companyABC.com
```

Deploy (Helm merges both files; right-most wins):

```
helm upgrade --install my-api ./my-chart \
  --namespace prod \
  --values values.yaml \
  --values values-prod.yaml
```

### Validate and Debug

# Lint a chart – catch syntax errors and missing required fields

```
helm lint ./my-chart
```

```
helm lint ./my-chart --values values-prod.yaml
```

# Render templates locally without installing – useful for reviewing output

```
helm template my-api ./my-chart --values values-prod.yaml
```

# Dry run – validate against the cluster API server without applying

```
helm upgrade --install my-api ./my-chart --values values-prod.yaml
--dry-run
```

# See what changed between current and new version

```
helm diff upgrade my-api ./my-chart --values values-prod.yaml
```

### Plugins

# Install helm-diff – shows a git-diff-style view of what will change before upgrade

```
helm plugin install https://github.com/databus23/helm-diff
```

# Install helm-secrets – encrypts values files using SOPS (for secrets in Git)

```
helm plugin install https://github.com/jkroepke/helm-secrets
```

# List installed plugins

```
helm plugin list
```

For chart dependencies (sub-charts), chart testing (`helm test`), and publishing to Artifact Registry as a Helm repo, see the [Helm documentation](#).

---

## 18. GKE — Google Kubernetes Engine Setup

---

### Cluster Creation

```
# Full production cluster
gcloud container clusters create companyABC-cluster \
  --region=asia-south1 \
  --release-channel=regular \           # auto-upgrades on regular
cadence
  --network=companyABC-vpc \
  --subnetwork=subnet-gke-primary \
  --cluster-secondary-range-name=pod-range \
  --services-secondary-range-name=svc-range \
  --workload-pool=my-project.svc.id.goog \   # enable Workload
```

### Identity

```
--enable-private-nodes \           # nodes have no
external IP
  --master-ipv4-cidr=172.16.0.0/28 \
  --master-authorized-networks=10.0.0.0/8 \   # restrict API server
access
  --enable-ip-alias \           # VPC-native;
required for private nodes
  --enable-network-policy \       # enable
```

### NetworkPolicy enforcement

```
--enable-managed-prometheus \       # Google Managed
```

### Prometheus

```
--enable-shielded-nodes \
--logging=SYSTEM,WORKLOAD \
--monitoring=SYSTEM,POD,DEPLOYMENT
```

### Node Pools

```
# General-purpose pool
gcloud container node-pools create general-pool \
  --cluster=companyABC-cluster \
  --region=asia-south1 \
  --machine-type=n2-standard-4 \
  --num-nodes=2 \
```

```
--enable-autoscaling --min-nodes=1 --max-nodes=10 \  
--disk-type=pd-balanced \  
--disk-size=100GB \  
--enable-autorepair \  
--enable-autoupgrade \  
--workload-metadata=GKE_METADATA # required for Workload  
Identity
```

```
# High-memory pool for Trino  
gcloud container node-pools create trino-pool \  
--cluster=companyABC-cluster \  
--region=asia-south1 \  
--machine-type=n2-highmem-16 \  
--num-nodes=3 \  
--enable-autoscaling --min-nodes=0 --max-nodes=20 \  
--node-labels=workload=trino \  
--node-taints=dedicated=trino:NoSchedule \  
--spot # spot instances for  
cost  
--workload-metadata=GKE_METADATA
```

```
# Connect kubectl  
gcloud container clusters get-credentials companyABC-cluster  
--region=asia-south1
```

### Upgrade Strategy

GKE auto-upgrades nodes on the **regular** release channel. Configure a **maintenance window** so upgrades don't happen during business hours:

```
gcloud container clusters update companyABC-cluster \  
--maintenance-window-start=2026-06-07T22:00:00Z \  
--maintenance-window-end=2026-06-07T06:00:00Z \  
--maintenance-window-recurrence="FREQ=WEEKLY;BYDAY=SA,SU"
```

### GKE Autopilot

Alternative to Standard — GCP manages nodes entirely. You deploy Pods; GCP provisions nodes automatically and charges per Pod CPU/memory, not per node.

```
gcloud container clusters create-auto companyABC-autopilot \  
--region=asia-south1 \  
--network=companyABC-vpc \  
--subnetwork=subnet-gke-primary
```

Autopilot enforces security policies (no privileged containers, required resource requests) automatically.

---

## 19. Workload Identity (GKE + GCP IAM)

---

Workload Identity is the recommended way for GKE Pods to authenticate to GCP APIs (GCS, BigQuery, Pub/Sub, Secret Manager, etc.) without key files.

### How it Works

A Kubernetes ServiceAccount is linked to a GCP ServiceAccount. When a Pod uses the Kubernetes SA, it automatically gets credentials for the GCP SA — issued by the GKE metadata server, rotated automatically.

### Workload Identity authentication flow:

Pod (annotated with K8s ServiceAccount "trino-sa")

↓

calls GCS / BigQuery / Pub/Sub API

↓

GKE Metadata Server (runs on each node, intercepts metadata requests)

↓

"which GCP SA is trino-sa mapped to?"

↓

GKE Workload Identity Pool (my-project.svc.id.goog)

↓

verifies K8s SA → GCP SA binding exists

↓

Google IAM STS (issues short-lived OAuth token for the GCP SA)

↓

↓

GCP API call succeeds as

"trino-gcp-sa@my-project.iam.gserviceaccount.com"

No key files. No secrets. Token rotated automatically every hour.

### Setup

# 1. Create GCP Service Account

```
gcloud iam service-accounts create trino-gcp-sa \
  --display-name="Trino GCP Service Account"
```

# 2. Grant it GCP permissions

```
gcloud projects add-iam-policy-binding my-project \
```

```
--member="serviceAccount:trino-gcp-sa@my-project.iam.gserviceaccount.com" \
--role="roles/storage.objectViewer"
```

```
# 3. Allow the Kubernetes SA to impersonate the GCP SA
gcloud iam service-accounts add-iam-policy-binding \
trino-gcp-sa@my-project.iam.gserviceaccount.com \
--role=roles/iam.workloadIdentityUser \
--member="serviceAccount:my-project.svc.id.goog[trino/trino-sa]"
# namespace^ ^K8s
SA name
```

```
# 4. Create and annotate the Kubernetes ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: trino-sa
  namespace: trino
  annotations:
    iam.gke.io/gcp-service-account:
trino-gcp-sa@my-project.iam.gserviceaccount.com
# 5. Use the SA in your Pod/Deployment
spec:
  serviceAccountName: trino-sa
  containers:
    - name: trino
    ...
```

The Pod can now call GCS, BigQuery, etc. as `trino-gcp-sa` without any key files or env vars.

---

## 20. kubectl — Essential Commands

---

### Context & Cluster

```
# List contexts (clusters you have access to)
kubectl config get-contexts
```

```
# Switch context
kubectl config use-context
gke_my-project_asia-south1_companyABC-cluster
```

```
# View current context
kubectl config current-context
```

```
# Set default namespace
kubectl config set-context --current --namespace=trino
```

## Getting Resources

```
# General pattern
kubectl get <resource> [-n <namespace>] [-l <selector>] [-o
<format>]
```

```
# Common examples
kubectl get pods
kubectl get pods -n trino
kubectl get pods -A # all namespaces
kubectl get pods -l app=trino,component=worker
kubectl get pods -o wide # show node and IP
kubectl get pods -o yaml # full YAML output
kubectl get pods --field-selector=status.phase=Running
```

```
kubectl get all -n trino # pods, services,
deployments, etc.
kubectl get events -n trino --sort-by='.lastTimestamp'
kubectl get nodes
kubectl get nodes -o
custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu,MEM:.sta
tus.capacity.memory
```

```
# Watch (live updates)
kubectl get pods -w
kubectl get pods -w -n trino
```

## Describing Resources

```
kubectl describe pod <pod-name> -n trino
kubectl describe deployment my-api
kubectl describe node <node-name>
kubectl describe service my-api-svc
kubectl describe hpa my-api-hpa
```

**describe** is the first place to look when something is wrong — it shows Events at the bottom which explain why a Pod is failing to start.

## Logs

```
# Logs from a pod (most recent container)
```

```
kubectl logs <pod-name>
```

```
kubectl logs <pod-name> -n trino
```

```
# Follow (tail -f equivalent)
```

```
kubectl logs -f <pod-name>
```

```
# Previous container (useful after a crash/restart)
```

```
kubectl logs --previous <pod-name>
```

```
# Specific container in a multi-container pod
```

```
kubectl logs <pod-name> -c <container-name>
```

```
# Last N lines
```

```
kubectl logs --tail=100 <pod-name>
```

```
# Since a time
```

```
kubectl logs --since=1h <pod-name>
```

```
# All pods matching a label (aggregated)
```

```
kubectl logs -l app=trino -f --prefix=true
```

## Exec into a Container

```
# Open a shell
```

```
kubectl exec -it <pod-name> -- /bin/bash
```

```
kubectl exec -it <pod-name> -n trino -- /bin/sh
```

```
# Run a single command
```

```
kubectl exec <pod-name> -- cat /etc/config/app.yaml
```

```
# In a specific container
```

```
kubectl exec -it <pod-name> -c <container-name> -- /bin/bash
```

## Applying & Managing Resources

```
# Apply (create or update)
```

```
kubectl apply -f manifest.yaml
```



```
kubectl apply -f directory/          # apply all YAMLs in a directory
```

```
kubectl apply -k kustomization/      # apply via Kustomize
```

**# Delete**

```
kubectl delete -f manifest.yaml
```

```
kubectl delete pod <pod-name>
```

```
kubectl delete pod <pod-name> --grace-period=0 # force delete
```

**# Edit in-place (opens \$EDITOR)**

```
kubectl edit deployment my-api
```

**# Patch (JSON or strategic merge)**

```
kubectl patch deployment my-api -p '{"spec":{"replicas":5}}'
```

**# Replace (delete then create – use sparingly)**

```
kubectl replace -f manifest.yaml
```

## Deployments

```
kubectl rollout status deployment/my-api
```

```
kubectl rollout history deployment/my-api
```

```
kubectl rollout undo deployment/my-api
```

```
kubectl rollout undo deployment/my-api --to-revision=2
```

```
kubectl scale deployment my-api --replicas=10
```

```
kubectl set image deployment/my-api app=my-api:1.2.5
```

## Port Forwarding

Expose a Pod or Service port locally for debugging without creating an Ingress.

```
kubectl port-forward pod/<pod-name> 8080:8080
```

```
kubectl port-forward svc/trino 8080:8080 -n trino
```

```
kubectl port-forward deployment/my-api 8080:8080
```

## Copying Files

```
kubectl cp <pod-name>:/path/in/container ./local-path
```

```
kubectl cp ./local-file <pod-name>:/path/in/container
```

## Resource Usage

```
kubectl top nodes
```

```
kubectl top pods
```

```
kubectl top pods -n trino --sort-by=memory
```

```
kubectl top pods -A
```

## Debugging — Ephemeral Debug Container

Run a debug container alongside a running Pod without modifying it:

```
kubectl debug -it <pod-name> --image=busybox  
--target=<container-name>
```

## Kustomize

Kustomize is built into kubectl. Manage environment-specific config without Helm:

```
# Directory structure  
base/  
  deployment.yaml  
  service.yaml  
  kustomization.yaml  
overlays/  
  prod/  
    kustomization.yaml    # patches for prod  
  staging/  
    kustomization.yaml
```

```
# Apply  
kubectl apply -k overlays/prod/
```

```
# Preview  
kubectl kustomize overlays/prod/
```

---

## 21. Observability — Logs, Metrics, Tracing

---

### Logs on GKE

All container stdout/stderr is automatically collected by GKE's Fluent Bit DaemonSet and shipped to Cloud Logging. No configuration needed for basic log collection.

**Structured logging** (JSON to stdout) — enables rich filtering in Cloud Logging:

```
import json, sys  
  
def log(severity, message, **kwargs):  
    entry = {"severity": severity, "message": message, **kwargs}  
    print(json.dumps(entry), flush=True)  
  
log("ERROR", "Query failed", query_id="abc123", site="SITE-001",  
duration_ms=5432)
```

**View logs:**

```
# Using gcloud
```

```
gcloud logging read 'resource.type="k8s_container" AND
resource.labels.namespace_name="trino" ' \
--limit=50 \
--format="table(timestamp, jsonPayload.severity,
jsonPayload.message)"
```

```
# Using kubectl (direct from container)
kubectl logs -f -l app=trino -n trino
```

## Metrics — Google Managed Prometheus

Enable on cluster (already shown in GKE setup). Metrics are stored in Cloud Monitoring and queryable via PromQL.

### Instrument your application:

```
from prometheus_client import Counter, Histogram, start_http_server
```

```
REQUEST_COUNT = Counter('requests_total', 'Total requests',
['method', 'endpoint', 'status'])
REQUEST_LATENCY = Histogram('request_duration_seconds', 'Request
latency', ['endpoint'])
```

```
start_http_server(8080) # /metrics endpoint
```

### Scrape configuration:

```
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: my-api-monitoring
  namespace: data-platform
spec:
  selector:
    matchLabels:
      app: my-api
  endpoints:
    - port: 8080
      path: /metrics
      interval: 30s
```

## Distributed Tracing — Cloud Trace

Instrument with OpenTelemetry:

```
from opentelemetry import trace
```

```

from opentelemetry.exporter.cloud_trace import
CloudTraceSpanExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(CloudTraceSpanExporte
r()))
trace.set_tracer_provider(provider)

tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("process-event"):
    with tracer.start_as_current_span("fetch-from-gcs"):
        data = fetch_data()
        with tracer.start_as_current_span("write-to-bigquery"):
            write_data(data)

```

---

## 22. Production Patterns & Best Practices

---

### Pod Disruption Budgets (PDB)

Guarantee minimum availability during voluntary disruptions (node drains, upgrades):

```

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-api-pdb
spec:
  minAvailable: 2      # always keep at least 2 Pods running
  # OR maxUnavailable: 1
  selector:
    matchLabels:
      app: my-api

```

Without a PDB, a node drain can kill all your Pods simultaneously.

### Image Pull Policy

**imagePullPolicy** controls when kubelet pulls the container image from the registry:

| Policy | When image is pulled | Default behaviour |
|--------|----------------------|-------------------|
|--------|----------------------|-------------------|

|                     |                                              |                                                           |
|---------------------|----------------------------------------------|-----------------------------------------------------------|
| <b>Always</b>       | Every time the Pod starts                    | Default when tag is <b>latest</b> or tag is omitted       |
| <b>IfNotPresent</b> | Only if image is not already on the node     | Default when a specific tag is given (e.g. <b>1.2.3</b> ) |
| <b>Never</b>        | Never — image must already exist on the node | Use only for local development or pre-loaded images       |

containers:

```
- name: app
  image: my-api:1.2.3
  imagePullPolicy: IfNotPresent # don't re-pull if node already has it
```

**Production rule:** always use a specific version tag (never **latest**) and **IfNotPresent**. Using **latest** with **Always** means every Pod restart pulls the image — slow, and you lose reproducibility. If **latest** was updated between restarts, two replicas may run different code.

Pod restart with `imagePullPolicy: Always + tag: latest`

- kubelet pulls image every time
- possible different image between replicas if registry was updated
- slow startup (registry round-trip)

Pod restart with `imagePullPolicy: IfNotPresent + tag: 1.2.4`

- kubelet checks: "do I have my-api:1.2.4?" → yes → start immediately
- reproducible – every node runs exactly 1.2.4

## Image Best Practices

# Use specific digest, not mutable tags (most reproducible)  
FROM python:3.11.9-slim@sha256:abc123...

# Run as non-root

```
RUN addgroup --gid 1001 appgroup && adduser --uid 1001 --gid 1001 appuser
USER appuser
```

# Don't run as root in pod spec either  
securityContext:

```
runAsNonRoot: true
runAsUser: 1001
readOnlyRootFilesystem: true # prevent writes to container
filesystem
```

## Security Context

```
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
```

## Rolling Deployments with Zero Downtime

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0 # never drop below desired replicas
  template:
    spec:
      terminationGracePeriodSeconds: 60 # allow in-flight requests
      to complete
      containers:
        - lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"] # give LB time to
                de-register the pod
```

## Graceful Shutdown

Configure your app to handle SIGTERM — finish in-flight requests, close DB connections, flush buffers.

```
import signal, sys
```

```
def handle_sigterm(sig, frame):
    # finish current requests, close connections
    app.shutdown()
    sys.exit(0)
```

```
signal.signal(signal.SIGTERM, handle_sigterm)
```

### Labels Convention (companyABC Standard)

```
metadata:
```

```
  labels:
```

```
    app.kubernetes.io/name: trino
```

```
    app.kubernetes.io/instance: trino-prod
```

```
    app.kubernetes.io/component: worker
```

```
    app.kubernetes.io/version: "428"
```

```
    app.kubernetes.io/managed-by: helm
```

```
    team: data-platform
```

```
    env: prod
```

### Resource Sizing Guide

Start with these and adjust based on VPA recommendations:

| Workload          | CPU request | Memory request | Notes                               |
|-------------------|-------------|----------------|-------------------------------------|
| Lightweight API   | 100m        | 128Mi          | Stateless, < 10 req/s               |
| Medium API        | 250m        | 512Mi          | Standard backend service            |
| Trino coordinator | 2           | 8Gi            | Needs headroom for query planning   |
| Trino worker      | 4           | 16–32Gi        | Size based on expected query memory |
| Airflow scheduler | 500m        | 2Gi            |                                     |
| Airflow worker    | 1           | 2Gi            |                                     |

## 23. Troubleshooting Guide

### Pod Not Starting

# Step 1: describe the pod – look at Events section at the bottom

```
kubectl describe pod <pod-name> -n <namespace>
```

# Step 2: check logs (even failed containers log before dying)

```
kubectl logs <pod-name> --previous -n <namespace>
```

# Step 3: check node conditions

```
kubectl describe node <node-name>
```

### Common errors and fixes:

| Status                     | Reason                                    | Fix                                                                               |
|----------------------------|-------------------------------------------|-----------------------------------------------------------------------------------|
| Pending                    | No node has enough CPU/memory             | Check <code>kubectl describe pod</code> events; scale up nodes or reduce requests |
| Pending                    | Unschedulable — taint/toleration mismatch | Check nodeSelector and tolerations                                                |
| ImagePullBackOff           | Can't pull image                          | Check image name, tag, registry auth; check Artifact Registry permissions         |
| CrashLoopBackOff           | Container starts then crashes             | Check <code>kubectl logs --previous</code> ; fix app crash                        |
| OOMKilled                  | Out of memory                             | Increase memory limit; check for memory leaks                                     |
| Evicted                    | Node out of resources                     | Check node memory pressure; increase node size or add nodes                       |
| CreateContainerConfigError | Missing Secret or ConfigMap               | Check referenced Secret/ConfigMap exists in same namespace                        |



|                              |                            |                                                                          |
|------------------------------|----------------------------|--------------------------------------------------------------------------|
| <b>RunContainerErr</b><br>or | Container runtime<br>issue | Check security context; privileged<br>container on Autopilot not allowed |
|------------------------------|----------------------------|--------------------------------------------------------------------------|

### Service Not Routing Traffic

# 1. Check endpoints – are pods being selected?

```
kubectl get endpoints my-svc -n <namespace>
```

# If ENDPOINTS shows <none>, the selector doesn't match any pods

# 2. Check labels on pods vs service selector

```
kubectl get pods -l app=my-api -n <namespace>
```

```
kubectl describe service my-svc -n <namespace>
```

# 3. Test connectivity from inside the cluster

```
kubectl run debug --image=curlimages/curl -it --rm -- /bin/sh
```

```
curl http://my-svc.namespace.svc.cluster.local:80/health
```

### DNS Not Resolving

# Test DNS from inside cluster

```
kubectl run debug --image=busybox -it --rm -- /bin/sh
```

```
nslookup my-svc.namespace.svc.cluster.local
```

```
nslookup kubernetes.default.svc.cluster.local # should always  
resolve
```

# Check CoreDNS is running

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

```
kubectl logs -n kube-system -l k8s-app=kube-dns
```

### Node Issues

# List node conditions

```
kubectl get nodes
```

```
kubectl describe node <node-name> # look for MemoryPressure,  
DiskPressure, PIDPressure
```

# Drain a node (for maintenance)

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

# Uncordon after maintenance

```
kubectl uncordon <node-name>
```

# Cordon (prevent new pods, don't evict existing)

```
kubectl cordon <node-name>
```

### Resource Usage Problems

```
# Which pods are using most CPU/memory
```

```
kubectl top pods -A --sort-by=memory | head -20
```

```
kubectl top nodes
```

```
# Check if HPA is working
```

```
kubectl describe hpa my-api-hpa
```

```
# Look for: "Conditions" and "Events" – explains why it is/isn't scaling
```

```
# Check resource quota usage
```

```
kubectl describe resourcequota -n <namespace>
```

### etcd / API Server Unreachable

On GKE this is Google's responsibility — check the GCP console for cluster incidents. If the cluster is healthy but kubectl times out:

```
# Refresh credentials
```

```
gcloud container clusters get-credentials companyABC-cluster
```

```
--region=asia-south1
```

```
# Check cluster status
```

```
gcloud container clusters describe companyABC-cluster
```

```
--region=asia-south1 | grep status
```

---

## 24. CI/CD — GitHub Actions

This section covers how to wire a GitHub Actions pipeline to automatically build, push, and deploy a service to GKE. Two deployment variants are covered: raw kubectl (simple) and Helm (recommended for production).

### Architecture Overview

GitHub (source code)

└──

└── git push → main

└── ▼

GitHub Actions

└──

└── — Run tests

└──

```

    — Build Docker image (tagged with Git SHA for
traceability)
    — Push to Artifact Registry
(asia-south1-docker.pkg.dev/companyABC-platform-dev/...:abc1234)
    — Deploy — Variant A: kubectl apply -f k8s/
(replace image tag via sed)
    — Variant B: helm upgrade --install
(--set image.tag=abc1234)

```

### Prerequisites — GitHub Secrets

Go to your repo → **Settings** → **Secrets and variables** → **Actions**. Add:

| Secret name      | Value                                        |
|------------------|----------------------------------------------|
| GCP_PROJECT_ID   | companyABC-platform-dev                      |
| GCP_SA_KEY       | JSON key of a CI service account (see below) |
| GKE_CLUSTER_NAME | companyABC-dev                               |
| GKE_REGION       | asia-south1                                  |

### Create the CI service account (one-time):

```
# Create SA
```

```
gcloud iam service-accounts create github-actions-sa \
  --display-name="GitHub Actions CI/CD"
```

```
# Grant Artifact Registry write access (push images)
```

```
gcloud projects add-iam-policy-binding companyABC-platform-dev \
```

```
--member="serviceAccount:github-actions-sa@companyABC-platform-dev.i
am.gserviceaccount.com" \
```

```
--role="roles/artifactregistry.writer"
```

```
# Grant GKE deploy access
```

```
gcloud projects add-iam-policy-binding companyABC-platform-dev \
```

```
--member="serviceAccount:github-actions-sa@companyABC-platform-dev.iam.gserviceaccount.com" \
--role="roles/container.developer"
```

```
# Download key and paste into GitHub secret GCP_SA_KEY
gcloud iam service-accounts keys create github-actions-key.json \
```

```
--iam-account=github-actions-sa@companyABC-platform-dev.iam.gserviceaccount.com
```

```
cat github-actions-key.json # copy entire JSON into the GitHub secret
```

```
rm github-actions-key.json # delete locally after copying
```

**Production note:** prefer [Workload Identity Federation](#) over JSON keys. Keys are long-lived credentials; WIF issues short-lived tokens per pipeline run and requires no key management.

#### Variant A — Raw kubectl Deploy

[.github/workflows/deploy.yaml](#)

```
name: Build and Deploy (Raw YAML)
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

```
env:
```

```
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
```

```
  REGION: asia-south1
```

```
  REPO: companyABC-images
```

```
  IMAGE: fastapi-service
```

```
  CLUSTER: ${ secrets.GKE_CLUSTER_NAME }
```

```
  CLUSTER_REGION: ${ secrets.GKE_REGION }
```

```
  NAMESPACE: fastapi-app
```

```
jobs:
```

```
# — 1. Test
```

```
test:
```

```
  runs-on: ubuntu-latest
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-python@v5
```

```
      with:
```

```
        python-version: "3.12"
```

```
    - run: pip install -r requirements.txt pytest httpx
```

```
    - run: pytest tests/ -v
```

```
# — 2. Build & Push
```

```
build-push:
```

```
  runs-on: ubuntu-latest
```

```
  needs: test
```

```
  outputs:
```

```
    image_tag: ${{ steps.meta.outputs.tag }}
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - name: Authenticate to GCP
```

```
      uses: google-github-actions/auth@v2
```

```
      with:
```

```
        credentials_json: ${{ secrets.GCP_SA_KEY }}
```

```
    - uses: google-github-actions/setup-gcloud@v2
```

```
    - name: Configure Docker
```

```
      run: gcloud auth configure-docker ${{ env.REGION }}-docker.pkg.dev --quiet
```

```
    - name: Set image tag (first 8 chars of Git SHA)
```

```
      id: meta
```

```

    run: |
        SHA=$(echo ${github.sha} | cut -c1-8)
        echo "tag=${SHA}" >> $GITHUB_OUTPUT
        echo "full_image=${env.REGION}-docker.pkg.dev/${env.PROJECT_ID}/${env.REPO}/${env.IMAGE}:${SHA}" >>
        $GITHUB_OUTPUT

- name: Build Docker image
  run: docker build -t ${steps.meta.outputs.full_image} .

- name: Push to Artifact Registry
  run: docker push ${steps.meta.outputs.full_image}

- name: Also tag as latest (main branch only)
  if: github.ref == 'refs/heads/main'
  run: |
      docker tag ${steps.meta.outputs.full_image} \
          ${env.REGION}-docker.pkg.dev/${env.PROJECT_ID}/${env.REPO}/${env.IMAGE}:latest
      docker push ${env.REGION}-docker.pkg.dev/${env.PROJECT_ID}/${env.REPO}/${env.IMAGE}:latest

# — 3. Deploy (raw kubectl)

deploy:
  runs-on: ubuntu-latest
  needs: build-push
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v4

    - name: Authenticate to GCP
      uses: google-github-actions/auth@v2
      with:
        credentials_json: ${secrets.GCP_SA_KEY}

    - name: Get GKE credentials

```

```
    uses: google-github-actions/get-gke-credentials@v2
  with:
    cluster_name: ${ env.CLUSTER }
    location: ${ env.CLUSTER_REGION }
```

```
  - name: Substitute image tag
    run: |
      sed -i "s|:IMAGE_TAG|:${{
needs.build-push.outputs.image_tag }}|g" k8s/deployment.yaml
```

```
  - name: Apply manifests
    run: kubectl apply -f k8s/ -n ${ env.NAMESPACE }
```

```
  - name: Wait for rollout
    run: kubectl rollout status deployment/fastapi-service -n
${ env.NAMESPACE } --timeout=300s
```

```
  - name: Verify
    run: |
      kubectl get pods -n ${ env.NAMESPACE }
      kubectl get svc -n ${ env.NAMESPACE }
```

### Variant B — Helm Deploy (Recommended)

Swaps the deploy step for a `helm upgrade --install`. The `--atomic` flag auto-rolls back if the deploy fails.

[.github/workflows/deploy.yaml](#)

name: Build and Deploy (Helm)

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```
env:
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
  REGION: asia-south1
  REPO: companyABC-images
  IMAGE: fastapi-service
```

```
CLUSTER: ${ secrets.GKE_CLUSTER_NAME }
CLUSTER_REGION: ${ secrets.GKE_REGION }
NAMESPACE: fastapi-app
HELM_RELEASE: fastapi-dev
```

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
      - run: pip install -r requirements.txt pytest httpx
      - run: pytest tests/ -v
```

```
build-push:
  runs-on: ubuntu-latest
  needs: test
  outputs:
    image_tag: ${ steps.meta.outputs.tag }
```

```
steps:
  - uses: actions/checkout@v4
  - name: Authenticate to GCP
    uses: google-github-actions/auth@v2
    with:
      credentials_json: ${ secrets.GCP_SA_KEY }
  - uses: google-github-actions/setup-gcloud@v2
  - run: gcloud auth configure-docker ${ env.REGION }
    id: meta
  - run: echo "tag=$(echo ${ github.sha } | cut -c1-8)" >>
    $GITHUB_OUTPUT
  - run: |
      FULL=${ env.REGION }-docker.pkg.dev/${ env.PROJECT_ID }
      ${ env.REPO }/${ env.IMAGE }:${ steps.meta.outputs.tag }
      docker build -t $FULL .
```



```
docker push $FULL
```

```
deploy:
```

```
  runs-on: ubuntu-latest
```

```
  needs: build-push
```

```
  if: github.ref == 'refs/heads/main'
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - name: Authenticate to GCP
```

```
      uses: google-github-actions/auth@v2
```

```
      with:
```

```
        credentials_json: ${ secrets.GCP_SA_KEY }
```

```
    - name: Get GKE credentials
```

```
      uses: google-github-actions/get-gke-credentials@v2
```

```
      with:
```

```
        cluster_name: ${ env.CLUSTER }
```

```
        location: ${ env.CLUSTER_REGION }
```

```
    - name: Install Helm
```

```
      uses: azure/setup-helm@v4
```

```
      with:
```

```
        version: "3.15.0"
```

```
    - name: Helm upgrade / install
```

```
      run: |
```

```
        helm upgrade --install ${ env.HELM_RELEASE }
```

```
./helm/fastapi-service \
```

```
  --namespace ${ env.NAMESPACE } \
```

```
  --create-namespace \
```

```
  --values helm/fastapi-service/values.yaml \
```

```
  --values helm/fastapi-service/values-dev.yaml \
```

```
  --set image.tag=${ needs.build-push.outputs.image_tag
```

```
}} \
```

```
  --wait \
```

```
  --timeout 5m \
```

```
  --atomic      # rolls back automatically if deploy fails
```

```
    - run: |
```

```
      helm status ${ env.HELM_RELEASE } -n ${ env.NAMESPACE
```

```
}}
```

```
kubectl get pods -n ${ env.NAMESPACE }
```

## Pipeline Stages Summary

| Stage      | Trigger                              | What it does                                                      |
|------------|--------------------------------------|-------------------------------------------------------------------|
| test       | every push                           | Run unit/integration tests                                        |
| build-push | after test passes                    | Build Docker image tagged with Git SHA, push to Artifact Registry |
| deploy     | after build, <b>main branch only</b> | Deploy to GKE via kubectl or Helm                                 |

**Pull requests:** the pipeline runs **test** + **build-push** on every PR (so you can catch build failures early), but skips **deploy** — only merges to **main** deploy.

## 25. GitOps with ArgoCD

GitOps is the deployment model used by mature engineering teams. **GitHub Actions never touches the cluster directly** — it only builds an image and updates a config file. ArgoCD, running inside the cluster, watches that config file and keeps the cluster in sync with it.

### Why GitOps?

|                 | Push-based (raw kubectl / Helm)   | GitOps / ArgoCD                                |
|-----------------|-----------------------------------|------------------------------------------------|
| Who deploys     | GitHub Actions (outside cluster)  | ArgoCD (inside cluster, pulls)                 |
| Rollback        | Re-run pipeline                   | <b>git revert</b> → auto-sync                  |
| Drift detection | None                              | ArgoCD alerts + auto-heals                     |
| Audit trail     | CI logs                           | Git history <b>is</b> the deployment history   |
| Secrets in CI   | SA key with cluster access needed | Not needed — ArgoCD already has cluster access |
| Multi-cluster   | Separate pipeline per cluster     | One ArgoCD, multiple targets                   |

### Repository Structure with GitOps

Maintain two separate repos so they have independent history:

```
app-repo (github.com/companyABC/fastapi-service)
├── app/
├── Dockerfile
├── .github/workflows/ci.yaml ← builds + pushes image only; no
kubectl
```

```
config-repo (github.com/companyABC/k8s-config)
├── services/
│   └── fastapi-service/
│       ├── Chart.yaml
│       ├── values.yaml
│       └── values-prod.yaml ← CI updates image.tag here
```

### Step 1 — Install ArgoCD in the Cluster

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f \
```

```
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
install.yaml
```

```
# Wait for ArgoCD pods
kubectl wait --for=condition=Ready pod -l
app.kubernetes.io/name=argocd-server \
-n argocd --timeout=120s
```

```
# Port-forward the UI
kubectl port-forward svc/argocd-server -n argocd 8080:443
# Open: https://localhost:8080
```

```
# Get initial admin password
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d
```

### Step 2 — Create an ArgoCD Application

```
# argocd-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
```

```

name: fastapi-service
namespace: argocd
spec:
  project: default

  source:
    repoURL: https://github.com/companyABC/k8s-config.git
    targetRevision: HEAD
    path: services/fastapi-service
    helm:
      valueFiles:
        - values.yaml
        - values-prod.yaml

  destination:
    server: https://kubernetes.default.svc # cluster ArgoCD is
running in
    namespace: fastapi-app

  syncPolicy:
    automated:
      prune: true # delete resources removed from config
      selfHeal: true # re-apply if someone manually edits the
cluster
    syncOptions:
      - CreateNamespace=true
kubectl apply -f argocd-app.yaml

```

# Check sync status

```
kubectl get application fastapi-service -n argocd
```

ArgoCD polls the config repo every 3 minutes (default). Any change to `values-prod.yaml` automatically triggers a deployment. You can also configure a webhook for instant triggers.

### Step 3 — GitHub Actions CI (image only — no kubectl)

In GitOps, CI builds and pushes the image, then commits the updated image tag to the config repo. ArgoCD does the rest.

```
# .github/workflows/ci.yaml (in the APP repo)
```

```
name: CI – Build and Update Config
```

```

on:
  push:
    branches: [main]

jobs:
  build-and-update:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Authenticate to GCP
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${ secrets.GCP_SA_KEY }

      - uses: google-github-actions/setup-gcloud@v2
      - run: gcloud auth configure-docker asia-south1-docker.pkg.dev
      --quiet

      - name: Set image tag
        id: meta
        run: echo "tag=$(echo ${ github.sha } | cut -c1-8)" >>
$GITHUB_OUTPUT

      - name: Build and push image
        run: |
          IMAGE=asia-south1-docker.pkg.dev/${
secrets.GCP_PROJECT_ID }/companyABC-images/fastapi-service:${
steps.meta.outputs.tag }
          docker build -t $IMAGE .
          docker push $IMAGE

      # — Update image tag in config repo

      - name: Checkout config repo
        uses: actions/checkout@v4

```

```

      with:
        repository: companyABC/k8s-config
        token: ${ secrets.CONFIG_REPO_TOKEN } # PAT with write
access
        path: k8s-config

    - name: Update image tag
      run: |
        cd k8s-config
        sed -i "s|tag: .*|tag: \"${ steps.meta.outputs.tag }|\"|\"
        \
        services/fastapi-service/values-prod.yaml

    - name: Commit and push config change
      run: |
        cd k8s-config
        git config user.name "github-actions[bot]"
        git config user.email
        "github-actions[bot]@users.noreply.github.com"
        git add services/fastapi-service/values-prod.yaml
        git commit -m "deploy: fastapi-service → ${
steps.meta.outputs.tag }"
        git push

# ArgoCD detects the change and deploys automatically.
# No kubectl or helm commands needed in this pipeline.

```

## End-to-End Deployment Flow

1. Developer pushes code to app-repo main

2. GitHub Actions (ci.yaml):

- runs tests
- builds image: fastapi-service:abc1234
- pushes to Artifact Registry
- checks out k8s-config repo
- edits values-prod.yaml: tag: "abc1234"
- commits + pushes to k8s-config

3. ArgoCD (running in GKE, polls every 3 min or on webhook):

- detects k8s-config changed
- computes diff: current cluster vs desired state in config repo
- applies diff: updates Deployment image tag
- monitors rollout until healthy
- marks sync as "Synced"

#### 4. If new Pods fail health checks:

- ArgoCD can auto-rollback (sync to previous commit)
- Or: `git revert <commit>` → push → ArgoCD re-syncs to old tag

### Push-based vs GitOps Security Model

#### Push-based:

GitHub Actions —kubect/helm—► GKE

CI needs cluster credentials. If pipeline is compromised, cluster is exposed.

#### GitOps (pull-based):

GitHub Actions —————► Config Repo



ArgoCD pulls

GKE

ArgoCD lives INSIDE the cluster. CI never touches the cluster.

Blast radius of a compromised pipeline: limited to the config repo.

### Approach Comparison — Raw YAML vs Helm vs GitOps

|                       | Raw YAML                          | Helm                       | GitOps / ArgoCD                     |
|-----------------------|-----------------------------------|----------------------------|-------------------------------------|
| Complexity            | Lowest                            | Medium                     | Highest — extra infra               |
| Multiple environments | Copy-paste files                  | One chart + values files   | One config repo, ArgoCD app per env |
| Rollback              | <code>kubectl rollout undo</code> | <code>helm rollback</code> | <code>git revert</code> → auto-sync |
| Drift detection       | None                              | None                       | ArgoCD alerts + auto-heals          |

|                       |                            |                            |                                       |
|-----------------------|----------------------------|----------------------------|---------------------------------------|
| Audit trail           | CI logs                    | CI logs + Helm history     | Git history IS the audit trail        |
| CI credentials needed | SA key with cluster access | SA key with cluster access | SA key for Artifact Registry only     |
| Best for              | Learning, PoCs             | Most production teams      | Multi-cluster, regulated environments |

#### Which to pick:

- **Learning / first service** → Raw YAML. Understand every object before abstracting.
- **Production, 2+ environments** → Helm. `--atomic` auto-rollback alone justifies it.
- **Multiple clusters / teams / compliance** → ArgoCD + Helm. Git is the single source of truth; no deploy without a code review.

## 26. Terraform — Infrastructure as Code

Terraform is the tool used to provision and manage GCP infrastructure (VPCs, GKE clusters, IAM, Artifact Registry, etc.) as code. Instead of running `gcloud` commands by hand, you write `.tf` files that describe the desired state, and Terraform creates or updates resources to match.

#### Why Terraform?

|                    | Manual ( <code>gcloud</code> commands) | Terraform                                          |
|--------------------|----------------------------------------|----------------------------------------------------|
| Reproducibility    | One-off commands; hard to replay       | Declarative — apply the same config repeatedly     |
| Drift detection    | None                                   | <code>terraform plan</code> shows what will change |
| Version control    | Commands in a doc nobody updates       | <code>.tf</code> files live in Git — full history  |
| Destroy / recreate | Error-prone; easy to forget a resource | <code>terraform destroy</code> removes everything  |



|                   |                                |                                                   |
|-------------------|--------------------------------|---------------------------------------------------|
| Multi-environment | Re-run commands per env        | One config, different <code>tfvars</code> per env |
| Review process    | No way to review infra changes | PR review on <code>.tf</code> file changes        |

## Core Concepts

| Concept            | What it is                                                                               |
|--------------------|------------------------------------------------------------------------------------------|
| <b>Provider</b>    | Plugin that talks to a cloud API (e.g. <code>hashicorp/google</code> for GCP)            |
| <b>Resource</b>    | A piece of infrastructure to create/manage (e.g. <code>google_container_cluster</code> ) |
| <b>Data source</b> | Read-only lookup of existing infrastructure (e.g. look up a VPC you didn't create)       |
| <b>Module</b>      | Reusable group of resources — like a function in code                                    |
| <b>State</b>       | Terraform's record of what it has created (stored in <code>.tfstate</code> )             |
| <b>Plan</b>        | Diff between current state and desired config — what will change                         |
| <b>Apply</b>       | Execute the plan — create/update/delete resources                                        |

## Installation

# Mac

```
brew install terraform
```

# Verify

```
terraform version
```

## Recommended Directory Structure

```
infra/
├── environments/
│   ├── dev/
│   │   ├── main.tf          # calls modules with dev config
│   │   ├── variables.tf
│   │   ├── terraform.tfvars # dev-specific values (not in Git)
│   │   └── backend.tf       # GCS state backend config
│   └── prod/
│       └── main.tf
```

```

├── variables.tf
├── terraform.tfvars
├── backend.tf
├── modules/
│   ├── networking/          # VPC, subnets, Cloud NAT
│   ├── gke/                 # GKE cluster + node pools
│   └── iam/                  # Service accounts + bindings

```

## Provider Configuration

```

# versions.tf
terraform {
  required_version = ">= 1.7.0"

  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

```

```

provider "google" {
  project = var.project_id
  region  = var.region
}

```

## Remote State Backend (GCS)

Never leave state local. Store it in GCS so the whole team shares the same state and concurrent runs are safe.

```

# backend.tf
terraform {
  backend "gcs" {
    bucket = "companyABC-terraform-state" # create this bucket
    manually first
    prefix = "gke/dev"                   # path within the bucket
  }
}

# Create the bucket (one-time, before first terraform init)
gsutil mb -p companyABC-platform-dev \
-l asia-south1 \

```

```
gs://companyABC-terraform-state
```

```
# Enable versioning (lets you recover from accidental state corruption)
```

```
gsutil versioning set on gs://companyABC-terraform-state
```

### Variables and Outputs

```
# variables.tf
```

```
variable "project_id" {  
  description = "GCP project ID"  
  type        = string  
}
```

```
variable "region" {  
  description = "GCP region"  
  type        = string  
  default     = "asia-south1"  
}
```

```
variable "cluster_name" {  
  description = "GKE cluster name"  
  type        = string  
}
```

```
variable "min_nodes" {  
  description = "Minimum nodes per zone"  
  type        = number  
  default     = 1  
}
```

```
variable "max_nodes" {  
  description = "Maximum nodes per zone"  
  type        = number  
  default     = 5  
}
```

```
# terraform.tfvars (dev – do NOT commit passwords/keys here)  
project_id = "companyABC-platform-dev"  
cluster_name = "companyABC-dev"  
min_nodes   = 1
```

```
max_nodes      = 5
# outputs.tf
output "cluster_name" {
  description = "GKE cluster name"
  value       = google_container_cluster.primary.name
}
```

```
output "cluster_endpoint" {
  description = "GKE cluster endpoint"
  value       = google_container_cluster.primary.endpoint
  sensitive   = true
}
```

```
output "kubeconfig_command" {
  description = "Command to configure kubectl"
  value       = "gcloud container clusters get-credentials
${google_container_cluster.primary.name} --region=${var.region}"
}
```

## Networking Module

```
# modules/networking/main.tf
```

```
resource "google_compute_network" "vpc" {
  name                  = "${var.name_prefix}-vpc"
  auto_create_subnetworks = false
  routing_mode          = "REGIONAL"
}
```

```
resource "google_compute_subnetwork" "gke_subnet" {
  name          = "${var.name_prefix}-subnet-gke"
  network       = google_compute_network.vpc.id
  region        = var.region
  ip_cidr_range = "10.10.0.0/20"
```

```
  secondary_ip_range {
    range_name      = "pod-range"
    ip_cidr_range   = "10.20.0.0/14"
  }
}
```

```
    secondary_ip_range {  
      range_name      = "svc-range"  
      ip_cidr_range = "10.24.0.0/20"  
    }  
  }  
}
```

```
  private_ip_google_access = true  
}
```

```
# Cloud Router + NAT so private nodes can pull images from the  
internet
```

```
resource "google_compute_router" "nat_router" {  
  name      = "${var.name_prefix}-nat-router"  
  network   = google_compute_network.vpc.id  
  region    = var.region  
}
```

```
resource "google_compute_router_nat" "nat_gateway" {  
  name                      =  
  "${var.name_prefix}-nat-gateway"  
  router                    =  
  google_compute_router.nat_router.name  
  region                    = var.region  
  nat_ip_allocate_option   = "AUTO_ONLY"  
  source_subnetwork_ip_ranges_to_nat =  
  "ALL_SUBNETWORKS_ALL_IP_RANGES"  
}
```

## **GKE Cluster Module**

```
# modules/gke/main.tf
```

```
resource "google_container_cluster" "primary" {  
  name      = var.cluster_name  
  location  = var.region    # regional cluster – 3 control planes  
  across_zones
```

```
  # We manage node pools separately – remove the default node pool  
  remove_default_node_pool = true  
  initial_node_count       = 1
```

```
network      = var.network_id
subnetwork   = var.subnet_id
```

```
# VPC-native (required for private nodes)
ip_allocation_policy {
  cluster_secondary_range_name = "pod-range"
  services_secondary_range_name = "svc-range"
}
```

```
# Private cluster – nodes have no external IPs
private_cluster_config {
  enable_private_nodes      = true
  enable_private_endpoint   = false           # keep public
  API_endpoint              =
  master_ipv4_cidr_block    = "172.16.0.0/28"
}
```

```
# Workload Identity – allows Pods to use GCP IAM without key files
workload_identity_config {
  workload_pool = "${var.project_id}.svc.id.goog"
}
```

```
# Auto-upgrades on the regular release channel
release_channel {
  channel = "REGULAR"
}
```

```
# Google Managed Prometheus
monitoring_config {
  enable_components = ["SYSTEM_COMPONENTS", "POD", "DEPLOYMENT"]
  managed_prometheus {
    enabled = true
  }
}
```

```
# Cloud Logging
logging_config {
  enable_components = ["SYSTEM_COMPONENTS", "WORKLOADS"]
}
```

```
}  
}
```

```
# General-purpose node pool  
resource "google_container_node_pool" "general" {  
  name      = "general-pool"  
  cluster   = google_container_cluster.primary.id  
  location  = var.region
```

```
  autoscaling {  
    min_node_count = var.min_nodes  
    max_node_count = var.max_nodes  
  }
```

```
  management {  
    auto_repair = true  
    auto_upgrade = true  
  }
```

```
  node_config {  
    machine_type = "n2-standard-4" # 4 vCPU, 16 GB  
    disk_type    = "pd-balanced"  
    disk_size_gb = 100
```

```
  # Required for Workload Identity  
  workload_metadata_config {  
    mode = "GKE_METADATA"  
  }
```

```
  oauth_scopes = [  
    "https://www.googleapis.com/auth/cloud-platform"  
  ]  
}
```

```
# High-memory node pool for Trino  
resource "google_container_node_pool" "trino" {  
  name      = "trino-pool"
```

```
cluster = google_container_cluster.primary.id
location = var.region
```

```
autoscaling {
  min_node_count = 0
  max_node_count = 20
}
```

```
management {
  auto_repair = true
  auto_upgrade = true
}
```

```
node_config {
  machine_type = "n2-highmem-16" # 16 vCPU, 128 GB
  disk_type    = "pd-balanced"
  disk_size_gb = 100
  spot         = true           # use Spot VMs to reduce cost
}
```

```
labels = {
  workload = "trino"
}
```

```
taint {
  key    = "dedicated"
  value  = "trino"
  effect = "NO_SCHEDULE"
}
```

```
workload_metadata_config {
  mode = "GKE_METADATA"
}
```

```
oauth_scopes = [
  "https://www.googleapis.com/auth/cloud-platform"
]
}
```



## IAM Module — Workload Identity Setup

```
# modules/iam/main.tf
```

```
# GCP Service Account for a workload (e.g. Trino)
resource "google_service_account" "workload_sa" {
  account_id   = var.sa_name
  display_name = var.sa_display_name
  project      = var.project_id
}
```

```
# Grant GCP permissions to the SA
resource "google_project_iam_member" "workload_sa_roles" {
  for_each = toset(var.roles)
```

```
  project = var.project_id
  role    = each.value
  member  =
    "serviceAccount:${google_service_account.workload_sa.email}"
}
```

```
# Allow the Kubernetes SA to impersonate the GCP SA (Workload
Identity binding)
resource "google_service_account_iam_member"
"workload_identity_binding" {
  service_account_id = google_service_account.workload_sa.name
  role                = "roles/iam.workloadIdentityUser"
  member              =
    "serviceAccount:${var.project_id}.svc.id.goog[${var.k8s_namespace}/${
var.k8s_sa_name}]"
}
```

## Artifact Registry

```
# artifact_registry.tf
resource "google_artifact_registry_repository" "images" {
  repository_id = "companyABC-images"
  location      = var.region
  format        = "DOCKER"
  description   = "Docker images for companyABC services"
}
```

```
# Allow the CI service account to push
resource "google_artifact_registry_repository_iam_member"
"ci_writer" {
  repository = google_artifact_registry_repository.images.name
  location   = var.region
  role       = "roles/artifactregistry.writer"
  member     =
"serviceAccount:${google_service_account.github_actions_sa.email}"
}
```

## Root Module — Composing Everything

```
# environments/dev/main.tf
```

```
module "networking" {
  source      = "../modules/networking"
  name_prefix = "companyABC-dev"
  region      = var.region
}
```

```
module "gke" {
  source      = "../modules/gke"
  cluster_name = var.cluster_name
  project_id   = var.project_id
  region       = var.region
  network_id   = module.networking.vpc_id
  subnet_id    = module.networking.subnet_id
  min_nodes    = var.min_nodes
  max_nodes    = var.max_nodes
}
```

```
module "trino_iam" {
  source      = "../modules/iam"
  sa_name      = "trino-gcp-sa"
  sa_display_name = "Trino GCP Service Account"
  project_id   = var.project_id
  roles        = [
    "roles/storage.objectViewer",
    "roles/bigquery.dataViewer",

```

```
    "roles/bigquery.jobUser",  
  ]  
  k8s_namespace    = "trino"  
  k8s_sa_name      = "trino-sa"  
}
```

## Essential Terraform Commands

```
# Initialise – download providers, configure backend  
terraform init
```

```
# Format code (run before committing)  
terraform fmt -recursive
```

```
# Validate syntax  
terraform validate
```

```
# Preview changes – what will be created/updated/destroyed  
terraform plan
```

```
# Preview with specific var file  
terraform plan -var-file=terraform.tfvars
```

```
# Apply changes (prompts for confirmation)  
terraform apply
```

```
# Apply without confirmation (CI/CD)  
terraform apply -auto-approve
```

```
# Destroy all managed resources  
terraform destroy
```

```
# Show current state  
terraform show
```

```
# List resources in state  
terraform state list
```

```
# Import an existing resource into state
```

```
# (use when a resource was created outside Terraform and you want to manage it)
```

```
terraform import google_container_cluster.primary  
projects/my-project/locations/asia-south1/clusters/companyABC-dev
```

```
# Remove a resource from state WITHOUT destroying it  
terraform state rm google_container_cluster.primary
```

```
# Refresh state from real infrastructure  
terraform refresh
```

```
# Target a specific resource (useful for debugging)  
terraform apply -target=module.gke.google_container_cluster.primary
```

### **Terraform in CI/CD**

Run Terraform in GitHub Actions so infrastructure changes go through PR review.

```
# .github/workflows/terraform.yaml  
name: Terraform Plan / Apply
```

```
on:  
  pull_request:  
    paths: ["infra/**"]  
  push:  
    branches: [main]  
    paths: ["infra/**"]
```

```
jobs:  
  terraform:  
    runs-on: ubuntu-latest  
    defaults:  
      run:  
        working-directory: infra/environments/dev
```

```
    steps:  
      - uses: actions/checkout@v4  
  
      - name: Authenticate to GCP  
        uses: google-github-actions/auth@v2  
        with:
```

```

    credentials_json: ${ secrets.GCP_SA_KEY }}

- name: Set up Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: "1.7.0"

- name: Terraform Init
  run: terraform init

- name: Terraform Fmt Check
  run: terraform fmt -check -recursive

- name: Terraform Validate
  run: terraform validate

- name: Terraform Plan
  run: terraform plan -var-file=terraform.tfvars -out=tfplan
  env:
    TF_VAR_project_id: ${ secrets.GCP_PROJECT_ID }}

# On PR: post plan output as a comment
- name: Post plan to PR
  if: github.event_name == 'pull_request'
  uses: actions/github-script@v7
  with:
    script: |
      // post terraform plan output as PR comment

# On merge to main: apply
- name: Terraform Apply
  if: github.ref == 'refs/heads/main' && github.event_name ==
'push'
  run: terraform apply -auto-approve tfplan

```

## Best Practices

### State management

- Always use a remote backend (GCS). Never commit `.tfstate` files to Git.
- Enable state locking — GCS backend locks automatically.

- Enable versioning on the GCS bucket to recover from state corruption.

## Code organisation

- One root module per environment (`dev/`, `staging/`, `prod/`). Don't use Terraform workspaces for environment separation — separate directories are clearer.
- Keep modules small and single-purpose. A `gke` module creates a cluster and node pools. An `iam` module creates service accounts and bindings. Don't mix them.

## Safety

- Always run `terraform plan` and review before `apply`. In CI, post the plan as a PR comment.
- Use `lifecycle { prevent_destroy = true }` on critical resources (the GKE cluster, the state bucket) to prevent accidental deletion.
- Use `-target` only for debugging, not in normal workflows — it can leave state inconsistent.

```
# Prevent accidental destruction of the cluster
resource "google_container_cluster" "primary" {
  # ...
  lifecycle {
    prevent_destroy = true
  }
}
```

## Variables vs hardcoding

- Parameterise anything that differs between environments (project IDs, replica counts, machine types).
- Hardcode things that are truly global constants (GCP region, naming conventions).
- Never put secrets (DB passwords, API keys) in `.tfvars`. Use Secret Manager and `data "google_secret_manager_secret_version"` to read them at apply time.

## Drift

- Run `terraform plan` in CI on a schedule (e.g. nightly) to detect drift — infra changed outside Terraform shows up as a diff.

## Terraform vs gcloud — When to Use Each

| Task                                                      | Use       |
|-----------------------------------------------------------|-----------|
| Provision long-lived infrastructure (clusters, VPCs, IAM) | Terraform |

|                                                                        |                                                      |
|------------------------------------------------------------------------|------------------------------------------------------|
| One-off debugging (check cluster status, get credentials)              | gcloud                                               |
| Authenticate kubectl locally                                           | gcloud ( <b>get-credentials</b> )                    |
| Create a resource to test something quickly                            | gcloud (then import to Terraform later if it sticks) |
| Manage infrastructure that needs to be version-controlled and reviewed | Terraform                                            |

---

React/Node



## NodeJS

- Node.js is an open-source, cross-platform JavaScript runtime environment that executes code outside of a web browser. It is built on V8, the same JavaScript engine within Chrome, and optimized for high performance.
  - Node.js employs event-driven architecture and non-blocking I/O for efficiency. While it operates off a single main thread, it can harness the full power of multi-core systems by launching child threads for specific tasks, such as file compression or image processing. To manage these child threads, Node.js uses a combination of:
    - A thread pool, powered by the libuv library.
    - Worker threads for dedicated, offloaded computation.
  - Node.js Event Loop: When a task in Node.js is designated to operate on a child thread, the main event loop hands it over to the thread pool. This setup allows Node.js to stay responsive to incoming requests, benefiting from asynchronous I/O. The main event loop regains control once the task on the child thread is completed, and results are ready.
  - <https://github.com/Devinterview-io/node-interview-questions>
- 

## ReactJS

- Component-based architecture: UI is broken into reusable, self-contained components
  - Declarative: You describe what the UI should look like, React handles the DOM updates
  - Unidirectional data flow: Data flows down from parent to child components via props
  - Virtual DOM: React maintains a lightweight copy of the actual DOM for efficient updates.
    - It is a programming concept where a virtual representation of the UI is kept in memory and synced with the real DOM through a process called reconciliation. When state changes:

- React creates a new Virtual DOM tree
  - Compares it with the previous tree (diffing algorithm)
  - Calculates the minimum number of changes needed
  - Updates only those parts in the real DOM (batching updates)
  - This makes updates more efficient than directly manipulating the DOM.
- JSX is a syntax extension for JavaScript that looks similar to HTML. It allows you to write markup inside JavaScript: Eg: JSX is a syntax extension for JavaScript that looks similar to HTML. It allows you to write markup inside JavaScript.
  - A React Element is a plain JavaScript object that describes what you want to see on the UI. It represents a DOM node or a component at a specific point in time. Elements are immutable: once created, you cannot change their properties. Instead, you create new elements to reflect updates.
  - A Component is a function or class that returns an element (or a tree of elements) to describe part of the UI. Components can accept inputs (called props) and manage their own state (in case of class or function components with hooks). Components allow you to split the UI into independent, reusable pieces, each isolated and composable. You can define a component using a function or a class
  - When several components need to share the same changing data then it is recommended to lift the shared state up to their closest common ancestor.
  - State of a component is an object that holds some information that may change over the lifetime of the component. The important point is whenever the state object changes, the component re-renders. It is always recommended to make our state as simple as possible and minimize the number of stateful components.
  - Props are inputs to components. Eg: `<Element reactProp={"1"} />`
  - <https://github.com/sudheerj/reactjs-interview-questions>
-

# Languages

## Languages

- A language is:
  - Category 1: Static vs Dynamic:
    - Statically typed if the type of a variable is known at compile time, eg: `int a = 5`. Means that you must specify what type each variable is. Eg: C, C++, Java, Rust, Go, Scala. Faster.
    - Dynamically typed if the type is known during run-time. Eg: Perl, Ruby, Python, PHP, JavaScript, Erlang. Slower.
  - Category 2: Compiled vs Interpreted:
    - Your CPU runs instructions in 0s and 1s. So every instruction defined via a language must eventually become machine code. The difference is WHEN and HOW that translation happens.
    - Compiled languages Flow: Source code → Compiler → Machine code executable → Run the executable. Compiler: Reads the entire program; Does static analysis; Optimizes globally. Interpreted languages Flow: Source code → Interpreter → Execute line by line.
    - Interpreter: Reads one statement, Executes it immediately, Moves to the next. Compiled languages came first. Note that today almost all modern “interpreted” languages compile internally.
    - What did interpreted languages solve?: Early computers were painful; Compilation took minutes to hours hence debugging meant: Write code, Compile, Run, Crash, Repeat. Interpreters solved this: Immediate feedback, Interactive programming (REPL), Dynamic behavior.
- Python:
  - Python code goes through both compilation and interpretation.
    - Bytecode Compilation: High-level Python code is transformed into low-level bytecode by the Python interpreter with the help of a compiler. Bytecode is a set of instructions that Python's virtual machine (PVM) can understand and execute.
    - On-the-fly Interpretation: The PVM reads and executes bytecode instructions in a step-by-step manner.

- While some programming languages compile directly to machine code, Python compiles to bytecode. This bytecode is then executed by the Python virtual machine. This extra step of bytecode execution can make Python slower in certain use-cases when compared to languages that compile directly to machine code. The advantage, however, is that bytecode is platform-independent. A Python program can be run on any machine with a compatible PVM, ensuring cross-platform support.
- Source Code to Bytecode steps: Lexical Analysis & Token generation, Syntax Parsing, Semantic Analysis, Bytecode Generation
- While Python typically uses a combination of interpretation and compilation, JIT (Just-In-Time) boosts efficiency by selectively compiling parts of the program that are frequently used or could benefit from optimization. JIT compiles sections of the program to machine code on-the-fly. This direct machine code generation for frequently executed parts can significantly speed up those segments, blurring the line between traditional interpreters and compilers.
- PEP 8 (Python Enhancement Proposal 8) is a style guide for Python code that promotes code consistency, readability, and maintainability.
- Memory Allocation
  - The "heap" is the pool of memory for storing objects. The Python memory manager allocates and deallocates this space as needed.
  - In latest Python versions, the obmalloc system is responsible for small object allocations. This system preallocates small and medium-sized memory blocks to manage frequently created small objects.
  - The allocator abstracts the system-level memory management, employing memory management libraries like Glibc to interact with the operating system.

- Larger blocks of memory are primarily obtained directly from the operating system.
- Stack and Heap separation is joined by "Pool Allocator" for internal use.
- Garbage Collection
  - Python employs a method called reference counting along with a cycle-detecting garbage collector.
  - Reference Counting
    - Every object has a reference count. When an object's count drops to zero, it is immediately deallocated.
    - This mechanism is swift, often releasing objects instantly without the need for garbage collection.
    - However, it can be insufficient in handling circular references.
  - Cycle-Detecting Garbage Collector
    - Python has a separate garbage collector that periodically identifies and deals with circular references.
    - This is, however, a more time-consuming process and is invoked less frequently than reference counting.
- Python handles memory management quite differently from languages like C or C++:
  - In Python, the developer isn't directly responsible for memory allocations or deallocations, reducing the likelihood of memory-related bugs.
  - The memory manager in Python is what's known as a "general-purpose memory manager" that can be slower than the dedicated memory managers of C or C++ in certain contexts.
  - Python, especially due to the existence of a garbage collector, might have memory overhead compared to C or C++ where manual memory management often results in minimal overhead is one of the factors that might contribute to Python's sometimes slower performance.

- The level of memory efficiency isn't as high as that of C or C++. This is because Python is designed to be convenient and easy to use, often at the expense of some performance optimization.

- Datatypes: (Apart from int, float, bool, etc).

| Type              | Mutable | Ordered         | Allows Duplicates | Example          |
|-------------------|---------|-----------------|-------------------|------------------|
| List              | ✓ Yes   | ✓ Yes           | ✓ Yes             | [1, 2, 3]        |
| Tuple             | ✗ No    | ✓ Yes           | ✓ Yes             | (1, 2, 3)        |
| Set               | ✓ Yes   | ✗ No            | ✗ No              | {1, 2, 3}        |
| Dictionary (dict) | ✓ Yes   | ✓ (Python 3.7+) | ✗ (unique keys)   | {"a": 1, "b": 2} |

- A frozenset is an immutable version of a set.
- Additional Data Structures (from `collections` module): deque, Counter, defaultdict, OrderedDict, namedtuple
- List comprehension vs Generator: List builds memory; generator is lazy: `[x*x for x in range(3)]` ; `(x*x for x in range(3))`
- `@staticmethod` no self; `@classmethod` gets cls.
- Inheritance & `super()`: MRO defines lookup order; `super()` calls parent.
  - MRO = the order in which Python looks for a method or attribute when it's called on an object. MRO exists to resolve ambiguity in multiple inheritance. Prevents “diamond problem” (method being inherited twice). Python uses the C3 linearization to compute MRO. Rule summary:
    - Child appears before its parents.
    - Maintain the order of base classes as declared.
    - Each parent appears only once.

```
class A:
    def show(self): print("A")
class B(A):
```

```

    def show(self): print("B")
class C(A):
    def show(self): print("C")
class D(B, C):
    pass

d = D()
d.show()
print(D.__mro__)

## Output
B
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,
<class '__main__.A'>, <class 'object'>)
## Python searched for show() in this order: D → B → C → A →
object
## ClassName.__mro__

```

- Exception handling:

```

try:
    risky_operation()
except IndexError: # Handle specific exception types first.
    handle_index_error()
except Exception as e: # More general exception must come last.
    handle_generic_error()
finally:
    cleanup()
## raise Exception("Some generic error occurred")

```

- The with keyword provides a more efficient and clean way to handle resources, like files, ensuring their proper closure when operations are complete or in case of any exceptions. The resource should implement a context manager, typically by having `__enter__` and `__exit__` methods.

```

with open("example.txt", "r") as file:
    data = file.read()
# File is automatically closed when the block is exited.

```



- There are times when not raising an exception is appropriate. You can use pass or continue in an exception block when you want to essentially ignore an exception and proceed with the rest of your code.
- zip() Function: This creates a dictionary by zipping two lists, where the first list corresponds to the keys, and the second to the values.

```
keys = ["a", "b", "c"]
values = [1, 2, 3]
zipped = zip(keys, values)
dict_from_zip = dict(zipped) # Result: {"a": 1, "b": 2, "c": 3}
```

- Both the == and is operators in Python are used for comparison, but they function differently.
  - The == operator checks for value equality.
  - The is operator, on the other hand, validates object identity,

```
# Example 1: same values but different objects
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True -- same values
print(a is b)    # False -- different objects (different memory locations)

# Example 2: same object reference
c = a
print(a == c)    # True -- same values
print(a is c)    # True -- both refer to the same object
```

- \*args collects a variable number of positional arguments into a tuple, while \*\*kwargs does the same for keyword arguments into a dictionary.

```
Eg:
Ex1: args
def sum_all(*args):
    result = 0
    for num in args:
        result += num
    return result
print(sum_all(1, 2, 3, 4)) # Output: 10
```

```

Ex2: kwargs
def print_values(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
# Keyword arguments are captured as a dictionary
print_values(name="John", age=30, city="New York")
# Output:
# name: John
# age: 30
# city: New York

```

- In Python, a decorator is a design pattern and a feature that allows you to modify functions and methods dynamically. Decorators wrap a target function, allowing you to execute custom code before and after that function. They are typically higher-order functions that take a function as an argument and return a new function. This paradigm of "functions that modify functions" is often referred to as metaprogramming. Use cases: logging, caching, counting/profiling, etc.

```

def greet_decorator(func):
    def wrapper():
        print("Before fun")
        func()
        print("After fun")
    return wrapper

@greet_decorator
def say_hello():
    print("Hii")

say_hello()
## Output:
# Before fun
# Hii
# After fun

```

- Create module in python adding `__init__.py` file.
- `self` represents the current instance of a class which is used to access the properties and methods of a class.
- Copy in Py:
  - Shallow Copy: Creates a new container object (like a new list or dict). But elements inside are references to the same inner objects. So changes to inner (nested) objects affect both copies.
  - Deep Copy: Creates a completely independent clone — both outer and inner objects are copied recursively. So modifying one does not affect the other.
- GIL = Global Interpreter Lock
  - A mutex (mutual exclusion lock) that allows only one thread to execute Python bytecode at a time within one process — even on multi-core CPUs.
  - CPython (the main Python implementation) manages memory dynamically.
    - Python objects are stored in heap memory.
    - CPython uses reference counting to track when objects are no longer needed.
    - If multiple threads modify the reference count of the same object at once, that can cause race conditions and memory corruption.
    - To prevent that, the GIL ensures that only one thread manipulates Python objects at a time.
  - If we removed GIL, CPython would need to:
    - Add fine-grained locks around every object.
    - That would make all operations slower, even for single-threaded programs.
    - So, GIL is a performance tradeoff — simplicity + safety at the cost of true parallelism.
  - Languages like Java, C++, Go, or Rust don't need a GIL because:
    - They have thread-safe memory management (e.g., GC with atomic operations).

- Or they don't use reference counting (e.g., Java's mark-and-sweep GC).
- Or they use compile-time safety guarantees (like Rust's borrow checker).
- That's why they can run multiple threads truly in parallel on multiple cores.
- We can't say, Python is a dynamically-typed lang, so it's handled like this, but rather about CPython's memory model and implementation choice (reference counting).

Some ex., related to Cores/Process/Threads:

Lets say you have 4 cores.

Case 1: One Process, 4 Threads (under GIL)

Threads run one at a time, taking turns (time slicing). Even on 4 cores, only one executes Python code. However, I/O operations (file, network, DB) release the GIL, so I/O-bound tasks benefit.

✅ Good for: I/O-bound tasks (web requests, API calls).

❌ Bad for: CPU-bound tasks (math, computation).

-----  
Case 2: 4 Processes (Each Single Thread)

Each process has its own interpreter and GIL. So, true parallelism across cores. Memory is separate, so need inter-process communication (IPC) to share data.

✅ Good for: CPU-bound tasks (ML, computation).

❌ Consumes more memory; communication overhead.

- Java:
  - Java is a high-level, object-oriented programming language designed to be platform-independent. Its core philosophy is encapsulated in the concept of "Write Once, Run Anywhere" (WORA). This is achieved through several key components:
    - Java Virtual Machine (JVM): The JVM acts as an abstraction layer between Java code and the underlying hardware or operating system. It interprets and executes Java bytecode, ensuring consistent behavior across different platforms.

- Bytecode: Java source code is compiled into platform-independent bytecode, which can be executed by any JVM, regardless of the underlying system architecture.
- Standard Library: Java provides a comprehensive standard library that offers cross-platform capabilities for common tasks like file handling and networking.
- Java Achieves WORA via:
  - Platform Independence: Java bytecode can run on any device with a compatible JVM, from smartphones to supercomputers.
  - JVM Customization: Each operating system has a tailored JVM version, ensuring WORA functionality across diverse environments.
  - Garbage Collection: Automatic memory management reduces the risk of memory leaks and simplifies development across platforms.
  - Security: Java's design excludes direct memory manipulation through pointers, enhancing security across different systems.
- Static members belong to the class rather than instances, allowing for utility functions and shared data.
- (OOPS related stuff - Java - covered in LLD section of this doc)
- JVM (Java Virtual Machine) is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. JRE (Java Runtime Environment) is the minimum environment required to execute a Java application. It consists of the JVM, core libraries, and other supporting files. JDK (Java Development Kit) is a superset of the JRE, providing everything needed for Java application development.
- ClassLoader is a crucial component in Java's runtime environment, responsible for loading class files into memory. It has different types.
- Classpath vs Path: Classpath is a parameter that tells the Java Virtual Machine (JVM) where to find compiled Java classes (.class files) and packages during runtime. Path is a system environment variable

that specifies directories where executable programs are located. It's used by the operating system to find executables when you run commands in the terminal or command prompt.

- Wrapper classes in Java allow you to work with primitive data types as objects. They are particularly useful when working with generic collections or when using features that require objects, such as Java Bean properties. Eg: Primitive: boolean; Wrapper Class: Boolean; Conversion Methods: `.valueOf()`, `.parseBoolean()`, `.booleanValue()`; Primitive Example: `true`; Wrapper Example: `Boolean.TRUE`
- Garbage Collection:
  - In Java, the Virtual Machine (JVM) manages memory through automatic garbage collection (GC). This process identifies and reclaims objects that are no longer in use. Objects are considered "alive" if they are reachable from the root object, which can be a Thread, Stack, or Static reference. Unreachable objects are eligible for garbage collection. There are different reference types that play a role in determining an object's reachability and GC eligibility.
    - Strong Reference: Default reference — object not eligible for GC until explicitly set to null. No default collection until no references exist. Eg: `MyClass obj = new MyClass();`
    - Weak Reference: Object eligible for GC once no strong refs exist; useful for caches. Collection eagerly on next GC. Eg: `WeakReference<MyClass> ref = new WeakReference<>(obj);`
    - Soft Reference: Object kept until JVM needs memory; used for memory-sensitive caches. Collection only in low-memory situations. Eg: `SoftReference<MyClass> ref = new SoftReference<>(obj);`
    - Phantom Reference: Doesn't prevent GC; object enqueued in ReferenceQueue after finalization. Used for cleanup tracking. Collection happens after `finalize()` before actual memory reclaim. Eg: `new PhantomReference<>(obj, refQueue);`

- GC Algos: Java uses different garbage collection algorithms, like Mark and Sweep algorithm. In Mark phase it detects all the unreachable objects and Sweep phase it reclaim the heap space used by the garbage objects and make the space available again to the program. Types:
  - Serial GC: Single-threaded collector. Suitable for small applications with limited memory
  - Parallel GC: Uses multiple threads for minor garbage collection. Default for most applications.
  - Concurrent Mark Sweep (CMS) GC: Minimizes pauses by doing most of its work concurrently with the application threads.
  - G1 (Garbage First) GC: Designed for applications with large amounts of memory. Divides the heap into regions for more efficient collection.
- Java divides heap into Young, Old, and (pre-8) PermGen to optimize GC — most short-lived objects are collected quickly, reducing pause times. GC can pause application threads (called Stop-The-World pauses), because objects may reference each other in complex ways, the JVM must ensure no thread is modifying memory while GC identifies and removes unused objects — otherwise, memory corruption could occur.
  - Young Generation: Newly created objects start here. Most objects die young → frequent Minor GCs. Divided into:
    - Eden Space – new objects allocated here.
    - Survivor Spaces (S0, S1) – objects that survive GC move between these.
    - If an object survives multiple Minor GCs → promoted to Old Gen.
  - Old (Tenured) Generation: Holds long-lived objects that survived multiple GCs. Collected less often via Major/Full GC. Slower but happens less frequently.

- Permanent Generation (PermGen): (Java 7 and earlier)  
Stores metadata, class info, static variables, method definitions, etc. Fixed size — can throw `OutOfMemoryError: PermGen space`. \
  - Replaced in Java 8+ by Metaspace, which grows dynamically in native memory.
- The `this` keyword in Java is a reference to the current instance of a class.

```
public class Person {
    private String name;

    public Person(String name) {
        this.name = name; // 'this.name' refers to the instance
variable
    }
}
```

- In Java, the `final` keyword offers restrictions and benefits. It primarily maintains the immutability of different entities.
- The Just-In-Time (JIT) compiler is a component of the runtime environment that improves the performance of Java applications by compiling bytecodes to native machine code at run time.
- Class loading:
  - Static Class Loading: Creating objects and instance using `new` keyword is known as static class loading. The retrieval of class definition and instantiation of the object is done at compile time.
  - Dynamic Class Loading: Loading classes use `Class.forName()` method. Dynamic class loading is done when the name of the class is not known at compile time.
- Transient: The transient modifier tells the Java object serialization subsystem to exclude the field when serializing an instance of the class. When the object is then deserialized, the field will be initialized to the default value; i.e. null for a reference type, and zero or false for a primitive type.



- Serialization is a mechanism of converting the state of an object into a byte stream. Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory.

```
public transient int limit = 55;    // will not persist
public int b;    // will persist
```

- Volatile: The volatile modifier tells the JVM that writes to the field should always be synchronously flushed to memory, and that reads of the field should always read from memory. This means that fields marked as volatile can be safely accessed and updated in a multi-thread application without using native or standard library-based synchronization.

```
public class MyRunnable implements Runnable {
    private volatile boolean active;
    public void run() {
        active = true;
        while (active) {
        }
    }
    public void stop() {
        active = false;
    }
}
```

- Types of Memory areas allocated by the JVM:
  - 1. Classloader: Classloader is a subsystem of JVM that is used to load class files.
  - 2. Class(Method) Area: Class(Method) Area stores per-class structures such as the runtime constant pool, field and method data, the code for methods.
  - 3. Heap: It is the runtime data area in which objects are allocated.
  - 4. Stack: Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and

return. Each thread has a private JVM stack, created at the same time as thread.

- 5. Program Counter Register: PC (program counter) register. It contains the address of the Java virtual machine instruction currently being executed.
- 6. Native Method Stack: It contains all the native methods used in the application.
- Java constructor chaining is a method of calling one constructor with the help of another while considering the present object. It can be done in 2 ways –
  - Within same class: It can be done using this() keyword for constructors in the same class.
  - From base class: By using super() keyword to call a constructor from the base class.

```
class Temp
{
    // default constructor 1
    // default constructor will call another constructor
    // using this keyword from same class
    Temp() {
        // calls constructor 2
        this(5);
        System.out.println("The Default constructor");
    }

    // parameterized constructor 2
    Temp(int x) {
        // calls constructor 3
        this(10, 20);
        System.out.println(x);
    }

    // parameterized constructor 3
    Temp(int x, int y) {
        System.out.println(10 + 20);
    }
}
```

```

public static void main(String args[]) {
    // invokes default constructor first
    new Temp();
}
}

```

- Ruby/Rails:

- A gem is just ruby code. It is installed on a machine and it's available for all ruby applications running on that machine. Rails, rake, json, rspec — are all examples of gems. Plugin is also ruby code but it is installed in the application folder and only available for that specific application.
- Active Record: Simplifies data handling from databases, treating tables as classes and rows as objects.
- Action View: Manages user interface elements like forms and templates.
- Action Controller: Handles requests, processes data, and manages flow between models and views.

#### # MVC Code in Ruby:

##### # Controller

```

class ArticlesController < ApplicationController
  def index
    @articles = Article.all
  end
end

```

##### # Model

```

class Article < ApplicationRecord
end

```

##### # View (ERB)

```

<% @articles.each do |article| %>
  <li><%= article.title %></li>

```

```
<% end %>
```

- The Gemfile in a Ruby on Rails application acts as a manifest of all required gems for the project. It serves as the foundation for the Bundler tool, which automates the process of managing and versioning gem dependencies.
  - Convention over Configuration is a foundational principle in Ruby on Rails.
  - Routes in Ruby: Eg: `match '/articles/:id'`, to: `'articles#show'`, via: `'get'`
- JavaScript:
    - `var/let/const`:

| Behavior                     | <code>var</code>       | <code>let</code>            | <code>const</code>          |
|------------------------------|------------------------|-----------------------------|-----------------------------|
| Scope                        | Function or Global     | Block                       | Block                       |
| Initialization               | Optional               | Optional                    | Required                    |
| Redeclaration                | Yes                    | No                          | No                          |
| Reassignment                 | Yes                    | Yes                         | No                          |
| Accessing before declaration | <code>undefined</code> | <code>ReferenceError</code> | <code>ReferenceError</code> |

- `==` is the abstract equality operator while `===` is the strict equality operator.
- Event Loop in JavaScript enables non-blocking asynchronous execution in a single-threaded environment. How It Works

- Call Stack → Executes synchronous code line by line.
- Web APIs / Node APIs → Handle async tasks (like setTimeout, fetch, I/O).
- Queues → When async tasks finish, callbacks go to:
  - Microtask Queue → Promises (then, catch, finally), queueMicrotask()
  - Macrotask Queue → setTimeout, setInterval, fetch, DOM events
- Event Loop → Keeps checking:
  - If Call Stack is empty → run all Microtasks first
  - Then run one Macrotask, and repeat
- Event delegation is a technique in JavaScript where a single event listener is attached to a parent element instead of attaching event listeners to multiple child elements. When an event occurs on a child element, the event bubbles up the DOM tree, and the parent element's event listener handles the event based on the target element.
- cookie, sessionStorage and localStorage in browsers: All of the following are mechanisms of storing data on the client, the user's browser in this case. localStorage and sessionStorage both implement the Web Storage API interface.
  - Cookies: Suitable for server-client communication, small storage capacity, can be persistent or session-based, domain-specific. Sent to the server on every request.
  - localStorage: Suitable for long-term storage, data persists even after the browser is closed, accessible across all tabs and windows of the same origin, highest storage capacity among the three.
  - sessionStorage: Suitable for temporary data within a single page session, data is cleared when the tab or window is closed, has a higher storage capacity compared to cookies.
- Garbage collection in JavaScript is an automatic memory management mechanism that reclaims memory occupied by objects and variables that are no longer in use by the program. The two

most common algorithms are mark-and-sweep and generational garbage collection.

- Mark-and-sweep: Algorithm effectively identifies and removes objects that have become unreachable, freeing up memory for new allocations.
- Generational garbage collection: Leveraged by modern JavaScript engines, objects are divided into different generations based on their age and usage patterns. Frequently accessed objects are moved to younger generations, while less frequently used objects are promoted to older generations. This optimization reduces the overhead of garbage collection by focusing on the younger generations, where most objects are short-lived.
- The splice() method in JavaScript is used to add, remove, or replace elements within an array. Modifies original array.
- Promises are used to handle asynchronous operations, especially in languages like JavaScript, which often work with non-blocking operations such as network requests, file I/O, and timers. When an operation is asynchronous, it doesn't immediately return a result; instead, it works in the background and provides the result later. Handling this in a clean, organized way can be difficult without a structured approach.
- Prototypal inheritance = Objects inherit from other objects.
- A callback function is a function passed into another function as an argument - higher-order function. The callbacks are needed because javascript is an event driven language. That means instead of waiting for a response, javascript will keep executing while listening for other events.

```
function firstFunction() {  
  // Simulate a code delay  
  setTimeout(function () {  
    console.log("First function called");  
  }, 1000);  
}  
function secondFunction() {
```

```

    console.log("Second function called");
}
firstFunction();
secondFunction();
// Output:
// Second function called
// First function called

## As observed from the output, javascript didn't wait for the
response of the first function and the remaining code block got
executed. So callbacks are used in a way to make sure that certain
code doesn't execute until the other code finishes execution.
#####
## Function as argument eg:
function greet(name) {
    return "Hello, " + name;
}
function processUserInput(callback) {
    const name = "Alice";
    console.log(callback(name));
}

processUserInput(greet);

```

- GoLang:
  - I have written a detailed article in my [personal blog](#).
  - Go (Golang) is a statically typed, compiled programming language designed at Google for simplicity, concurrency, scalability, and high-performance backend systems.
  - Packages, Imports, and Project Structure: Every Go file belongs to a package. Packages provide modularity and namespace separation. The main package represents the executable entry point. Imports are explicit, and unused imports fail compilation to prevent dead code accumulation. Go modules (go.mod) manage dependencies and versioning with deterministic builds. Production projects

commonly separate handlers/controllers, services/business logic, repositories/data layer, models, configs, middleware, and shared/internal packages. Go communities generally prefer relatively simple and flatter package structures compared to deeply nested enterprise Java layouts.

- Variables, Constants, and Types:
  - `var` defines mutable variables with automatic zero initialization. `:=` is shorthand declaration syntax used inside functions.
  - `const` defines immutable compile-time constants.
  - `type` defines structs, interfaces, aliases, and custom primitive types.
  - Go avoids classical inheritance; composition and interfaces are preferred.
  - Struct embedding enables composition and inheritance-like behavior without deep class hierarchies.
- Functions and Methods:
  - Functions are first-class citizens and can be passed, returned, and stored. Go encourages small focused functions. Multiple return values are heavily used, especially for error handling: `func getUser() (User, error)`
  - Methods are functions attached to structs.
  - Value receivers copy structs; pointer receivers avoid copies and allow mutation.
  - Pointer receivers are preferred for large structs or mutable state.
- Interfaces: Go interfaces are satisfied implicitly; no explicit `implements` keyword exists. A type implements an interface automatically by defining required methods. Interfaces promote loose coupling, mocking, testing, and composition-oriented design. Small focused interfaces are preferred. Common interfaces: `io.Reader`, `io.Writer`, `http.Handler`. Common Go philosophy: “accept interfaces, return structs.”
- Control Flow:



- Go intentionally keeps control flow minimalistic. it supports inline initialization: `if err := process(); err != nil { return err }`.
- Go only has one looping construct: `for`.
- `switch` statements automatically break unless explicit `fallthrough` is used.
- `select` is critical for concurrent programming and waits on multiple channel operations simultaneously.
- Error Handling:
  - Go avoids exceptions for normal application errors. Errors are explicit return values:

```
result, err := do()
if err != nil { return err }
```

- Errors are expected states; panics are exceptional states. Modern error handling commonly uses: `fmt.Errorf`, `errors.Is`, `errors.As`. These support wrapping, propagation, and classification of errors.
- `panic`, `defer`, `recover`:
  - `panic` triggers stack unwinding and abnormal control flow within the current goroutine. Used for unrecoverable programmer mistakes, corrupted state, impossible invariants, and startup failures.
  - `panic` should NOT replace normal error handling.
  - `defer` delays execution until function return: `defer file.Close()`. Deferred calls execute in LIFO order. Common `defer` use cases: file closing, mutex unlocking, DB transaction rollback, tracing/metrics cleanup, `recover` catches panics inside deferred functions and prevents process crashes.
- Goroutines:
  - Goroutines are lightweight concurrent execution units managed by the Go runtime. Eg: `go process()`
  - Goroutines are much cheaper than OS threads because of: small initial stacks (~2KB), dynamically growing stacks, user-space scheduling, cheaper context switching.

- Production systems may run hundreds of thousands of goroutines simultaneously.
  - Goroutines are not directly mapped 1:1 to OS threads.
- M:N Scheduler and GMP Model:
  - Go uses an M:N scheduler where many goroutines are multiplexed onto fewer OS threads.
  - Core components:
    - $G \rightarrow$  Goroutine
    - $M \rightarrow$  Machine (OS thread)
    - $P \rightarrow$  Processor (logical scheduler)
  - Goroutines are placed into runnable queues. Processors schedule runnable goroutines onto worker threads.
  - Scheduler responsibilities: work stealing, asynchronous preemption, syscall handling, runtime-managed scheduling
  - Blocking syscalls may temporarily tie up OS threads, after which the scheduler remaps goroutines efficiently.
  - The runtime integrates scheduling with a network poller (epoll/kqueue/IOCP), enabling massive concurrent network connections without thread-per-request architectures.
- GOMAXPROCS:
  - Controls how many OS threads can execute Go code simultaneously. Defaults to the number of CPU cores.
  - Important distinction: goroutines provide concurrency; GOMAXPROCS controls parallelism.
  - A system may have 1 million goroutines but only 8 actively executing on 8 CPU cores.
- 
- Context Switching:
  - OS thread context switching is expensive because the kernel must save registers, switch memory contexts, and transition kernel/user mode.
  - Goroutine switching is cheaper because scheduling mostly occurs in user space and stacks are smaller.
  - Go runtime significantly reduces concurrency overhead compared to thread-per-request architectures.

- Channels:
  - Channels are Go's core communication primitive implementing CSP (Communicating Sequential Processes). Go philosophy: "Do not communicate by sharing memory; share memory by communicating."
  - Channels provide synchronization, communication, and coordination between goroutines.
  - Unbuffered channels synchronize sender and receiver directly.
  - Buffered channels temporarily decouple producers and consumers using internal queues. Uses: worker pools, batching, rate limiting, event queues. Improper usage may cause: deadlocks, goroutine leaks, blocked pipelines. Only senders should close channels.
- select Statement:
  - select waits on multiple channel operations simultaneously.
  - Critical for: cancellation, timeouts, fan-in/fan-out systems, graceful shutdown, async orchestration.
  - default makes select non-blocking.
  - select + context.Context form the backbone of cancellation-aware systems.
- WaitGroup:
  - sync.WaitGroup coordinates goroutine completion.
  - Core methods: Add(), Done(), Wait().
  - Common use cases: parallel API calls, worker synchronization, graceful shutdown coordination
  - WaitGroup only waits; it does not propagate errors or cancellations.
  - errgroup + context are commonly used for advanced coordination.
- Mutexes and Synchronization:
  - sync.Mutex provides exclusive locking.
  - sync.RWMutex supports multiple readers and one writer. RWMutex helps in read-heavy workloads but increases complexity and contention overhead.

- Mutexes protect shared memory and establish happens-before guarantees.
- Go Memory Model:
  - Defines visibility guarantees between goroutines.
  - Without synchronization, compiler/runtime/CPU reordering may make writes invisible across goroutines.
  - Synchronization primitives like mutexes, channels, atomics, and coordination mechanisms like WaitGroups establish ordering guarantees.
  - Race conditions occur when multiple goroutines access shared memory concurrently and at least one operation is a write.
- Race Detector: Built-in race detector: Eg: `go test -race`.
- Slices:
  - Slices are lightweight descriptors over arrays.
  - Internally consist of: pointer to backing array, length, capacity
  - Appending beyond capacity allocates a new backing array and copies elements.
  - Passing slices copies only the slice descriptor, not underlying data.
  - Multiple slices may share the same backing array.
- Maps:
  - Go maps are runtime-managed hash tables.
  - Maps automatically resize and rehash as entries grow.
  - Maps are NOT safe for concurrent writes.
  - Concurrent map writes may cause runtime panics.
  - Synchronization options:
    - mutexes
    - `sync.Map`
    - Sharding techniques
- `new` vs `make`:
  - `New`: allocates memory, zero initializes, returns pointer.
  - `Make`: initializes runtime-backed structures, used only for slices, maps, and channels.

- Important distinction: `new(map)` → pointer to nil map;  
`make(map)` → usable initialized map
- Escape Analysis:
  - The compiler decides whether variables are stack allocated or heap allocated. Variables escaping function scope are allocated on the heap. Heap allocations increase GC pressure. Escape analysis significantly impacts performance. Tooling:  
`go build -gcflags="-m"`
- Garbage Collection:
  - Go uses a concurrent tri-color mark-and-sweep garbage collector optimized for low pause times.
  - Goals: low latency, predictable pauses, minimal tuning
  - GC is mostly concurrent with very small stop-the-world phases.
  - Major phases: mark reachable objects, sweep unreachable objects, reclaim memory
  - Go prioritizes latency and operational simplicity over maximum throughput.
- Go GC vs Java GC vs Python GC:
  - Go GC:
    - concurrent tri-color mark-and-sweep
    - low latency
    - predictable pauses
    - minimal tuning
    - optimized for scalable backend systems
  - Java GC:
    - advanced generational collectors (G1GC, ZGC, Shenandoah)
    - heap generations (young/old)
    - highly optimized for throughput and large heaps
    - significantly more tuning complexity
  - Python GC:
    - primarily reference counting
    - cyclic GC for circular references
    - GIL limits multicore concurrency

- higher runtime overhead compared to Go
- Overall:
  - Go prioritizes simplicity and scalable concurrency.
  - Java prioritizes throughput and GC flexibility.
  - Python prioritizes developer productivity.
- sync.Pool:
  - Used for temporary object reuse to reduce allocations and GC pressure.
  - Common use cases: buffers, serialization objects, reusable request objects, Objects inside sync.Pool may disappear anytime during GC. sync.Pool should not be treated as a permanent cache.
- unsafe Package: unsafe bypasses Go's type safety and memory guarantees. Used for: runtime internals, zero-copy optimizations, Interoperability, advanced performance tuning. Risks: memory corruption, invalid pointers, portability issues, Most business applications should avoid unsafe.
- Context Package: Used for: cancellation propagation, deadlines/timeouts, request-scoped metadata. Eg: HTTP servers, gRPC, DB calls, etc.
- Benchmarking and Profiling:
  - Benchmarking: go test -bench=.
  - Profiling uses pprof.
- gRPC and Protocol Buffers:
  - gRPC is a high-performance RPC framework built on HTTP/2 and Protocol Buffers.
  - Advantages over REST:
    - binary serialization
    - smaller payloads
    - faster communication
    - bidirectional streaming
    - strong typing
    - schema contracts
  - Protocol Buffers define schemas in .proto files and generate strongly typed Go code automatically.

- Widely used in: microservices, distributed systems, internal infrastructure.
  - Production Concurrency Patterns:
    - Worker Pool Pattern: Jobs → Channel → Worker Goroutines → Results
    - Fan-out distributes work across workers.
    - Fan-in aggregates results back into a single stream.
    - Backpressure becomes important when producers generate data faster than consumers process data.
    - Common overload-control mechanisms:
      - buffered channels
      - batching
      - queue sizing
      - rate limiting
    - Goroutine leaks are major production issues and commonly occur because of:
      - blocked channels
      - forgotten receivers
      - missing context cancellation
      - infinite waits
    - Proper cancellation and cleanup are critical in long-running systems.
-

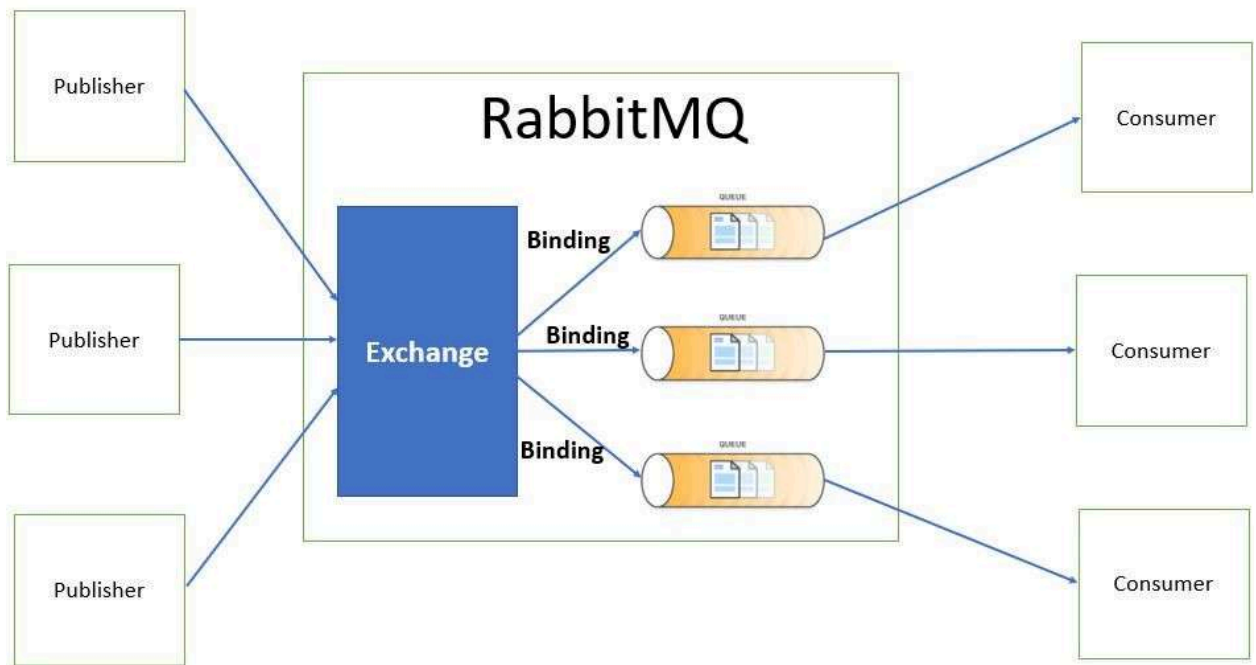
\_Misc\_Tech



## Misc Tech:

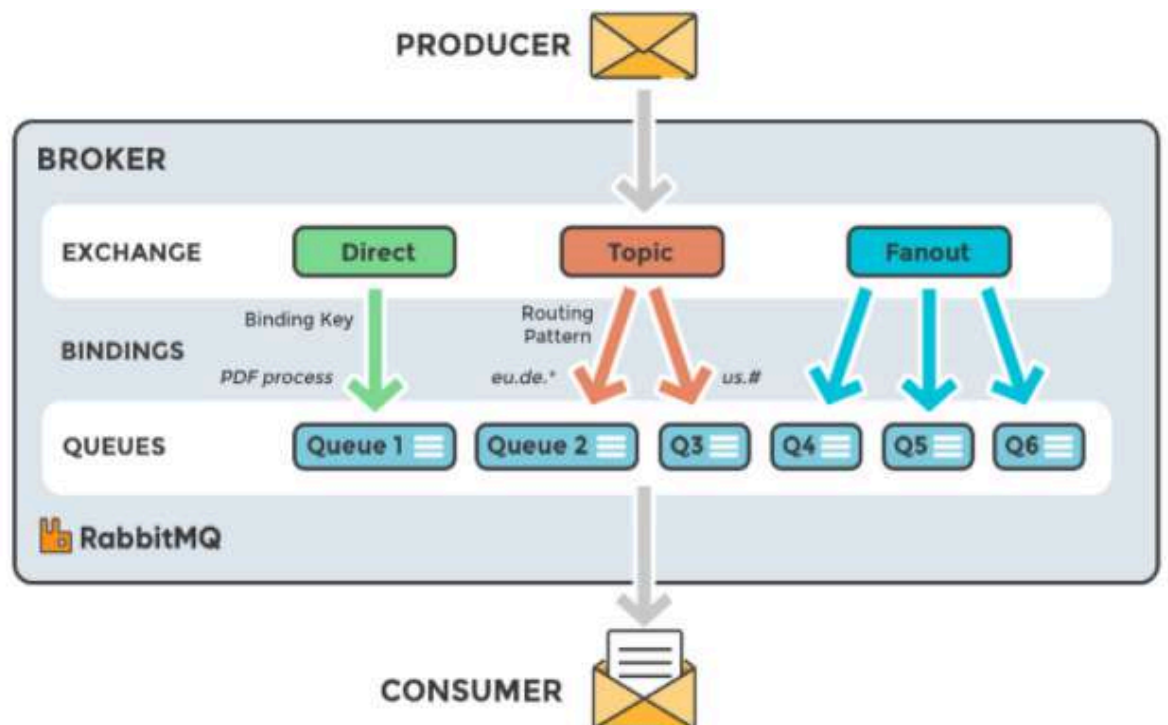
### RabbitMQ

- RabbitMQ is a message broker or queue manager that implements the AMQP (Advanced Message Queuing) protocol. It takes messages from a queue and sends them to consumers.
  - Kafka uses the binary protocol over TCP to stream messages across real-time data pipelines
  - Protocol: A standardised way for systems to talk to each other.



- RabbitMQ does not publish the messages directly to a queue but via an exchange. Exchange ensures that the message is published to the right queue, it receives message and routing info. Each exchange is linked to a queue via binding. There are 4 types of exchanges:
  - Direct exchange delivers messages to queues based on a message routing key. The routing key is a message attribute added to the message header by the producer. Think of the routing key as an "address" that the exchange is using to decide how to route the message. A message goes to the queue(s) with the binding key that exactly matches the routing key of the message.

- Fanout exchange copies and routes a received message to all queues that are bound to it regardless of routing keys or pattern matching as with direct and topic exchanges. The keys provided will simply be ignored. Fanout exchanges can be useful when the same message needs to be sent to one or more queues with consumers who may process the same message in different ways
- Topic exchanges route messages to queues based on wildcard matches between the routing key and the routing pattern, which is specified by the queue binding. Messages are routed to one or many queues based on a matching between a message routing key and this pattern. Eg: The routing patterns may contain an asterisk ("") or ("#") symbol like agreements.ru.berlin.#.
- Headers exchange routes messages based on arguments containing headers and optional values, similar to Topic exchange.



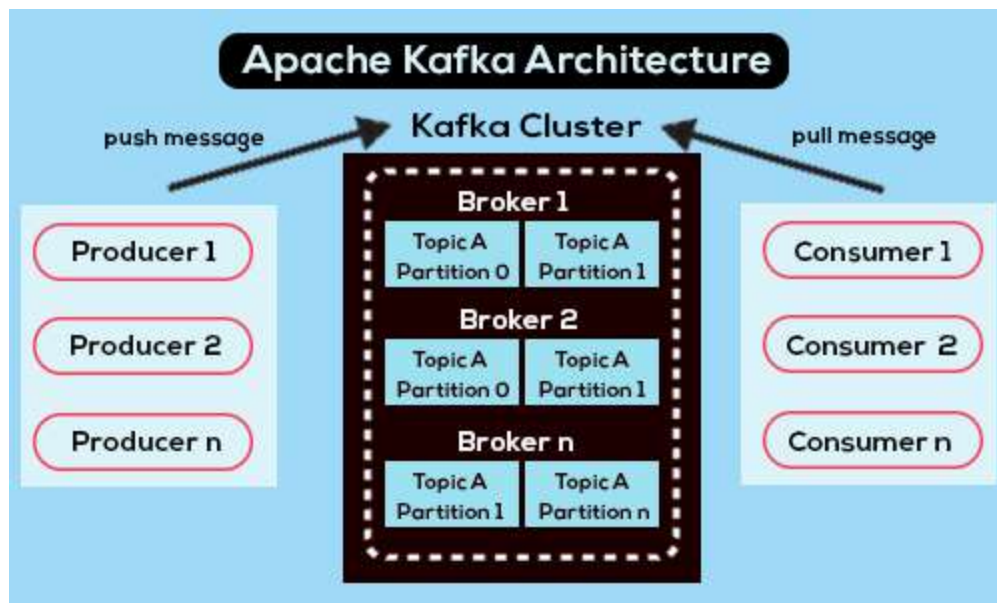
- We can define the queue's priority range at the time the queue is created.

- Dead-letter queue (DLQ), sometimes which is also known as an undelivered-message queue, is a holding queue for messages that cannot be delivered to their destinations due to some reason or other. Few failure criteria:
  - Message that is sent to a queue that does not exist.
  - Queue length limit exceeded.
  - Message length limit exceeded.
  - Message is rejected by another queue exchange.
  - Message reaches a threshold read counter number, because it is not consumed. Sometimes this is called a "back out queue".
- RabbitMQ default behavior:
  - If you didn't define the time-to-live argument, RabbitMQ will try to re-queue your message forever.
  - If you defined a TTL but didn't define a dlx, after TTL, RabbitMQ will remove your message to the queue and you will lose the message.
  - If you defined a TTL and a dlx, after the TTL, RabbitMQ will send the message to the exchange defined in the dlx.
- Although you can have a strategy to retry messages in the RMQ.
- Pika is a Python client library that implements the AMQP (Advanced Message Queuing Protocol) to communicate with RabbitMQ.

## Data Ecosystem

- Ingestion (Kafka, NiFi) - Topics, partitions, processors, data flow patterns
- Storage (S3, Hudi, Iceberg, Delta Lake) - Storage classes, table formats, ACID operations
- Compute (Spark, Flink) - Execution models, transformations, state management
- Query (Trino, Hive) - MPP architecture, query optimization, execution engines
- Catalog (Hive Metastore, AWS Glue) - Metadata management, crawlers, schema registry
- Orchestration (Airflow, Dagster) - DAGs, operators, asset-based orchestration
- Governance (Atlas, Ranger) - Lineage, classification, access control

## Kafka



- Core:
  - Topic: Logical category/feed where messages are published.
  - Partition: Unit of parallelism within a topic. Each partition is an ordered, immutable log of records.
  - Message order is guaranteed only within a single partition.
  - Broker: A Kafka server that stores partitions and handles client requests. A cluster usually has multiple brokers (e.g., broker1, broker2, ...).
    - The partition leader on that broker will be re-elected from followers (if in-sync replicas exist).
  - Consumer group:
    - A set of consumers working together to consume a topic's partitions without overlap.
    - One partition can be consumed by only one consumer (within the same group). But one consumer can consume multiple partitions.
      - If more consumers than partitions → some consumers stay idle
      - If fewer consumers than partitions → consumers take multiple partitions

- Multiple Consumer Groups → Each group gets its own copy of the topic stream. Kafka topic does not care how many consumer groups exist. Kafka enforces ordering & exclusivity only within a consumer group, not across groups. Adding more consumer groups does NOT affect each other. Across groups: The same partition can be consumed by many different groups independently.
- If new consumer joins: Rebalance occurs — partitions redistributed among consumers.
- Rebalances can cause temporary unavailability, so production systems often use static membership or incremental rebalancing.
  - Incremental Rebalancing: Instead of revoking all partitions from every consumer during a rebalance, Kafka only revokes or assigns partitions that actually need to move, i.e some partitions from existing consumer to new consumer, rest flow runs intact.
  - Static Membership: Each consumer gets a stable member ID (group.instance.id) that persists across restarts. So if a consumer disconnects and reconnects quickly, Kafka knows it's the same member, not a new one.
- Offsets: Sequential IDs assigned to messages within a partition. In case of failure, consumer re-reads from last committed offset.

Types of offset commit:

- Auto Commit: Commits the latest offset returned by poll() every auto.commit.interval.ms (default: 5000 ms). Con: Can cause message loss/duplicate processing.
- Manual Synchronous Commit: Explicitly commit offsets after successful processing. Con: Can reduce throughput.
- Manual Asynchronous Commit: Sends commit request and returns immediately. Con: No automatic retry, possible duplicates.

- Under the hood, Kafka topics = folders, and partitions = append-only log files stored on disk. Each message is written sequentially at the end of a partition with a unique offset (like a line number). Once written, messages are immutable — never updated or deleted (until retention expires).
  - The producer simply appends data to the end of a partition. No routing logic, no locks. This makes Kafka extremely fast (thanks to sequential disk writes and OS page cache).
  - Consumers explicitly ask for “messages from offset X onward.” Kafka just serves bytes from disk, like a range query in a database.
  - The commit operation only records, *separately*, the offset up to which the consumer has processed data safely. This is stored in a special Kafka internal topic: `__consumer_offsets`. So when your consumer calls `commitSync()` or `commitAsync()`, Kafka does not touch your topic’s data — instead, it writes a metadata record to `__consumer_offsets` like: `(group.id = analytics-consumer, topic = orders, partition = 0, offset = 1000)`
- Each partition has one leader and multiple followers, i.e: partition is replicated across multiple brokers for fault tolerance. Producers send data only to the leader. Consumers read data only from the leader. Followers just copy (replicate) the data in the background. How do I decide the number of partitions?:  $\text{Number of partitions} = \text{desired parallelism} = \text{target throughput} \div \text{per-partition throughput}$ .
  - Throughput-based: Example: You need 50 MB/s ingestion. One partition gives ~5 MB/s. → You need ~10 partitions.
  - Consumer parallelism: If a consumer group has 20 instances → Topic should have  $\geq 20$  partitions.
  - Key-based ordering: If you want ordering by a key (say `userId`), that key must land in one partition. More partitions = better spread = better parallelism.
- Replication Process: Producer writes to the leader. Leader writes data to its local log. Followers fetch data from the leader and

append it to their logs. Once a message is replicated to the required number of followers (defined by acks), Kafka acknowledges success to the producer. ISR (In-Sync Replicas) = Leader + followers that are fully caught up with the leader. If leader fails, one of the followers becomes leader.

- Schema Registry: Ensures data consistency between producers and consumers. Validates message schemas (Avro, Protobuf, JSON Schema). If producer sends incompatible schema the registry rejects it.
- Kafka used Zookeeper for metadata info - which topics in partitions, broker leader for which partition, configurations, etc. Now a days, KRaft used.
- Kafka - At-least-once delivery semantics by default (others, have to handle).
  - At-most-once: Messages may be lost, but never redelivered
  - At-least-once (default): No data loss, but possible duplicates\
  - Exactly-once: No data loss and no duplicates
- What makes Kafka high throughput?
  - Sequential disk writes (append-only log): Kafka writes to disk sequentially, not randomly. Sequential disk writes are as fast as RAM due to OS page cache.
  - Zero-copy architecture: Kafka uses: sendfile(), OS page cache, Bypasses user-space copying. This avoids: CPU overhead, Multiple memory copies, Syscalls.
  - Partitioned parallelism: Every partition is: independent, append-only, consumed independently. More partitions → more consumers → more throughput.
  - Dumb broker, smart consumer: Kafka brokers do almost no processing, do not re-encode messages, simply store bytes and serve them. Consumers handle: offset tracking, ordering, semantics. This "log-based" architecture makes Kafka extremely scalable.
- Schema registry in Kafka: It is an application that resides outside of your Kafka cluster and handles the distribution of schemas to the producer and consumer by storing a copy of schema in its local

cache. With the schema registry in place, the producer, before sending the data to Kafka, talks to the schema registry first and checks if the schema is available. If it doesn't find the schema then it registers and caches it in the schema registry. Once the producer gets the schema, it will serialize the data with the schema and send it to Kafka in binary format prepended with a unique schema ID. When the consumer processes this message, it will communicate with the schema registry using the schema ID it got from the producer and deserialize it using the same schema. If there is a schema mismatch, the schema registry will throw an error letting the producer know that it's breaking the schema agreement.

## Dask

- Dask is a parallel computing framework for Python that scales from a single machine to a cluster.
- Components: Client, Scheduler, Workers, Task graph. Dask builds a task graph of operations. Scheduler assigns tasks to workers. Workers execute tasks in parallel, report results. Results are aggregated and returned to client.
- Dask DataFrame: It is a collection of many smaller Pandas DataFrames, partitioned along an index.
- Dask Array: Parallel, chunked numpy array for numerical computations.
- Dask Bag: Handles unstructured/semi-structured data (JSON, logs, text files). Similar to PySpark RDD. Can convert to DataFrame using `.to_dataframe()`.
- Dask builds a task graph first, then executes when `.compute()` or `.persist()` is called. (Lazy evaluation)

```
A Dask graph is a dictionary mapping keys to computations:
{'x': 1,
 'y': 2,
 'z': (add, 'x', 'y'), // Computation function
 'w': (sum, ['x', 'y', 'z']), // Some computation being performed
 'v': [(sum, ['w', 'z']), 2]}
```

## Ansible

- It's an automation engine for configuration management, application deployment, and orchestration.



- Consider command: `ansible-playbook -vv -i inventories/${INSTALL_TO_ENV}/hosts.py airflow.yml --vault-password-file=~/.vault_pass_no_prompt`
  - Ansible playbooks are a fundamental component of Ansible, serving as the primary means to automate tasks across multiple systems. They are written in YAML format.
  - When you execute a playbook using the command `ansible-playbook <playbook.yml>`, Ansible parses the YAML file and prepares to execute the defined plays. Ansible connects to each host specified in the inventory file and executes the tasks in order. If a task fails on one host, Ansible continues executing tasks on other hosts unless configured otherwise. During execution, Ansible provides real-time feedback on the status of each task, including successes and failures. After completing all tasks, Ansible summarizes the execution results, detailing which tasks succeeded or failed.
- Dissecting command:
  - `ansible-playbook`: This is the command used to run Ansible playbooks. It initiates the execution of the specified playbook file.
  - `-vv`: This option increases the verbosity level of the output. The `-v` flag can be repeated (up to four times) to provide more detailed logging information during execution. This is useful for debugging and understanding what Ansible is doing behind the scenes.
  - `-i inventories/prod/hosts.py`: This specifies the inventory file that contains the list of hosts on which the playbook will run. In this case, `hosts.py` is likely a dynamic inventory script, which means it generates the list of hosts dynamically rather than being a static file.
  - `airflow.yml`: This is the name of the playbook file that will be executed. The playbook contains tasks that define what actions should be performed on the specified hosts.
  - `--vault-password-file=~/.vault_pass_no_prompt`: This option specifies a file that contains the password used to decrypt any sensitive data stored in Ansible Vault within the playbook or any included files.
- Ansible Vault is a feature within Ansible that provides a secure mechanism for managing sensitive data, such as passwords, API keys, and other

confidential information. It allows users to encrypt values and data structures within Ansible projects, ensuring that sensitive information is not stored in plaintext, which could pose a security risk.

- Encryption and Decryption: Ansible Vault enables users to encrypt files containing sensitive data. This encryption uses symmetric encryption with the AES256 algorithm, meaning the same password is used for both encryption and decryption. Users can encrypt existing files or create new encrypted files directly.
- File-Level Granularity: Ansible Vault operates at the file level, meaning that entire files can be encrypted or left unencrypted. This allows for flexibility in managing sensitive information alongside regular configuration data.
- Integration with Playbooks: Encrypted content can be seamlessly integrated into Ansible playbooks. When executing a playbook, Ansible automatically decrypts any vault-encrypted content at runtime if the correct password is provided.
- Secure Storage: By using Ansible Vault, users can securely store sensitive data within their playbooks and configuration files, preventing unauthorized access even if someone gains access to the files themselves.
- In above case we provide this flag in command:  
`--vault-password-file=~/.vault_pass_no_prompt`. The vault password is crucial when working with Ansible Vault because it:
  - Decrypts Encrypted Data: If your playbook or any referenced files (like variables or configuration files) contain encrypted content (e.g., passwords, API keys), Ansible needs to decrypt this data before executing tasks that require it. The `--vault-password-file` option tells Ansible where to find the password needed for decryption.
  - Automates Password Handling: By using a password file (in this case, `~/.vault_pass_no_prompt`), you can automate the process of providing the vault password without being prompted during execution. This is particularly useful for automated deployments or CI/CD pipelines where manual input would be impractical.

## Debezium

- Open-source distributed platform for Change Data Capture (CDC). Captures row-level changes in databases and streams them as events.
- Arch: Database → Transaction Log → Debezium Connector → Kafka → Consumers

## Docker

- Components:
  - Dockerfile: A text document containing commands that assemble a container image.
  - Image: A standalone, executable package containing everything required to run a piece of software.
  - Container: A runtime instance of an image.
- Virtual Machines virtualize the hardware layer, meaning each VM runs its own operating system on top of a hypervisor. This gives strong isolation but causes heavy resource consumption and slow startup times.
- Docker containers, on the other hand, virtualize at the OS level, sharing the host kernel. This makes them extremely lightweight, fast to start, and ideal for microservices or CI/CD environments. However, container isolation isn't as strict as VMs since they share the underlying OS.
- Images are stored in Docker image registries like Docker Hub, which provides a central location for image management and sharing. You can download existing images, modify them, and upload the modified versions also.
- COPY vs ADD in Docker

| Aspect         | COPY                                   | ADD                                                                  |
|----------------|----------------------------------------|----------------------------------------------------------------------|
| Purpose        | Simple, direct file and directory copy | Includes advanced features like URL fetch and TAR extraction         |
| URL Support    | ✗ Not supported                        | ✓ Can copy from remote URLs (e.g., <a href="#">ADD https://...</a> ) |
| TAR Extraction | ✗ No extraction                        | ✓ Automatically extracts local <a href="#">.tar</a> archives         |

|         |                                 |                                                |
|---------|---------------------------------|------------------------------------------------|
| Caching | ✅ Deterministic, respects cache | ⚠️ May bypass cache if source differs slightly |
|---------|---------------------------------|------------------------------------------------|

- A Docker volume is a directory or file within a Docker Host's writable layer that isn't tied to a specific container. This decoupling allows data persistence, even after containers have been stopped or removed. Containers are ephemeral — data is lost when they stop. Volumes persist data outside containers. Volume Types:
  - Host-Mounted Volumes: These link a directory on the host machine to the container.
  - Named Volumes: They have a specific name and are managed by Docker.
  - Anonymous Volumes: These are generated by Docker and not tied to a specific container or its data.
- Docker provides isolated virtual networks for containers to communicate: bridge (default), host (shares host network), none (no network), overlay (for Swarm multi-host), macvlan (assign MAC address)
- Docker compose, docker commands, etc...

## Nginx

- Nginx = High-performance web server, reverse proxy, load balancer, and cache.
- Event-driven & asynchronous → handles thousands of connections with low memory.
- Process model:
  - Master process starts and creates worker processes.
  - Workers wait for events (client requests) via event loop.
  - When a request comes in:
    - OS (via epoll) notifies an available worker.
    - Worker handles the full request (read → process → send response).
  - Master doesn't handle traffic — it only supervises.
  - On reload, the master spawns new workers and gracefully shuts down old ones (no dropped connections).

- Common roles: Serve static files, Reverse proxy to app servers (e.g., Gunicorn, Node, Spring Boot), Load balance traffic, Terminate SSL/TLS, API Gateway. Eg:

```
# /etc/nginx/nginx.conf
events {
    worker_connections 1024;
}
http {
    upstream backend {
        server 127.0.0.1:8080;
        server 127.0.0.1:8081;
    }
    server {
        listen 80;
        server_name example.com;
        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
        location /static/ {
            root /var/www/html;
        }
    }
}
```

## Redis

- Execution model
  - Single-threaded event loop using epoll (Linux) / kqueue (BSD/macOS). Handles thousands of clients using non-blocking I/O and I/O multiplexing. No context switching → predictable low latency.
  - Redis 6+ introduces optional I/O threads (for read/write only, not command execution).
  - Simplicity is the performance secret — no threads, no locks, no coordination overhead. Each command runs atomically because only one executes at a time.

- Memory Model
  - Entire dataset lives in RAM → no disk seek latency. Uses jemalloc / mimalloc for efficient memory allocation.
  - Keys and values stored as SDS (Simple Dynamic Strings) – safer and resizable C strings. Values use specialized data structures (Lists, Sets, Hashes, ZSets, Bitmaps, HyperLogLogs, Streams).
  - When memory hits limit (maxmemory), eviction policy applies based on TTL, etc.
- Persistence: Redis is not purely volatile – you choose durability level. Like:
  - RDB (Snapshotting): Saves full in-memory data to .rdb file periodically.
  - AOF (Append Only File): Logs every write command.
  - Hybrid Mode: Combines RDB + AOF (since Redis 4.0).
- Replication & HA: Primary–Replica model, asynchronous replication.

## PostgreSQL

- Multi-Version Concurrency Control (MVCC) is PostgreSQL's approach to handling concurrent transactions without locking reads.  
 How it works: Each transaction sees a snapshot of the database at a specific point in time → Updates create new row versions rather than overwriting existing data → Old versions are kept until no transaction needs them (handled by VACUUM) → Readers don't block writers, and writers don't block readers → Each row has hidden system columns: xmin (transaction ID that created it) and xmax (transaction ID that deleted/updated it)
  - Benefits: High concurrency with minimal locking, Consistent reads without blocking, Point-in-time recovery capabilities
  - Drawbacks: Requires VACUUM to clean up old row versions, Can lead to table bloat if not managed properly, Increased disk I/O for updates
- Components in Postgres:
  - Postmaster: Main daemon process that handles client connections
  - Backend Processes: One per client connection, handles queries
  - Background Processes:
    - Background Writer: Writes dirty buffers to disk periodically
    - WAL Writer: Writes WAL (Write-Ahead Log) records to disk

- Checkpointer: Performs checkpoints to ensure data durability
  - Autovacuum Launcher/Workers: Automatic VACUUM and ANALYZE
  - Stats Collector: Collects usage statistics
  - Logical Replication Workers: Handle logical replication
  - Archiver: Archives WAL files for PITR
- By default, PostgreSQL is a single-node OLTP database with one primary (writable) instance and no built-in read replicas. In production, it's often extended using streaming replication or managed services like Amazon RDS / Aurora to support readers and writers.
- `shared_buffers` is the amount of memory PostgreSQL uses for caching data.
- PostgreSQL supports multiple index types: BTree, Hash, Generalized Search Tree, Block Range Index, etc.
- Execution plan of query in psql: `EXPLAIN (ANALYZE, BUFFERS, VERBOSE) SELECT * FROM orders WHERE user_id = 123;`
  - Find slow queries: `SELECT query, mean_exec_time, calls FROM pg_stat_statements ORDER BY mean_exec_time DESC LIMIT 10;`
- EXPLAIN vs EXPLAIN ANALYZE:
  - Explain: Shows the query plan PostgreSQL will use, Estimated costs and row counts, Does NOT execute the query, Safe to run on production
  - Explain Analyze: Actually executes the query, Shows actual execution time and row counts, Shows actual vs estimated comparisons, Includes I/O statistics with BUFFERS option, Can impact production systems
- N+1 query problems in PostgreSQL: N+1 occurs when you fetch a list and then query for related data for each item.

Eg:

```
-- Bad: N+1 in application code
SELECT * FROM users;
-- Then for each user: SELECT * FROM orders WHERE user_id = ?
-- Good: Single query
SELECT u.*, o.*
FROM users u
```

```
LEFT JOIN orders o ON u.id = o.user_id;
```

- A transaction = a group of SQL operations that run as one logical unit. Either all succeed (COMMIT). Or none take effect (ROLLBACK-restores the database to its previous committed state). A transaction is committed when its changes are permanently written to the database and become visible to other transactions. Until you COMMIT, your changes are private to your session. Isolation defines how much one transaction can "see" of another transaction's changes while both run concurrently.
  - Read Uncommitted (acts like Read Committed in Postgres): Reads only committed data (no dirty reads).
  - Read Committed (Default): Each SQL statement sees data committed before that statement starts. If another transaction commits after your query, your next query can see the new data.
  - Repeatable Read: Sees a snapshot of the database from the moment the transaction started. You will see consistent results across queries, even if others commit changes.
  - Serializable (Stricest): Transactions behave as if they were run one after another (fully isolated). PostgreSQL enforces this using predicate locking. Safest, but may cause more rollbacks due to conflicts.
- A deadlock occurs when two or more transactions hold locks that the other needs, causing all of them to wait forever. PostgreSQL detects deadlocks automatically. It kills one of the transactions (throws an error) so the other can continue. The killed transaction must be retried. Avoid deadlock: Lock ordering, Keep transactions short, Use appropriate isolation levels, Use SELECT ... FOR UPDATE or NOWAIT/SKIP LOCKED carefully, Tune deadlock\_timeout, Monitor logs / stats.
  - Advisory locks are application-level locks not tied to database objects. Types:
    - Session-level: Last until explicitly released or session ends
    - Transaction-level: Released at transaction end
- Reduce write latency and increase throughput by optimizing:



- WAL (Write-Ahead Logging): WAL ensures durability — but too frequent writes slow performance.
- Checkpoints: Checkpoints write dirty buffers to disk — too frequent = I/O spikes.
- Background writes: Keeps cache (shared\_buffers) clean proactively.
- Application-level behavior: Batch Inserts for less WAL overhead, Disable triggers, COPY for bulk loads etc.
- Table Bloat: When rows are updated or deleted, PostgreSQL doesn't immediately remove old row versions — it marks them as dead tuples. Over time, these accumulate, causing table bloat (larger disk size without increase in live rows).
  - Internal Mechanism (MVCC):
    - UPDATE → creates a new tuple, marks old one as dead.
    - DELETE → marks tuple as dead, not physically removed.
    - Dead tuples still consume space until vacuumed.
  - Impact:
    - Slower sequential scans and index lookups
    - Increased I/O due to bloated pages
    - Noticeable performance degradation on frequently updated tables
- Auto-Vacuum: It is PostgreSQL's background process that automatically removes dead tuples to control bloat and maintain visibility maps. Usually takes an ACCESS SHARE lock (doesn't block reads/writes). On large, busy tables, it can compete for CPU/I/O. Poor tuning may cause lag → excessive bloat or transaction wraparound risk. How It Works:
  - Tracks row updates/deletes per table.
  - When thresholds are crossed (autovacuum\_vacuum\_threshold + scale factor), it triggers vacuuming.
  - Removes dead tuples invisible to active transactions.
  - Updates the Free Space Map (FSM) and Visibility Map (VM).
  - Performs aggressive freezing near vacuum\_freeze\_max\_age to prevent transaction ID wraparound.
- Manual Vacuum:
  - VACUUM → Reclaims dead tuples' space; lightweight and non-blocking.

- VACUUM ANALYZE → Cleans up and refreshes optimizer statistics.
  - VACUUM FULL → Fully rewrites the table to reclaim disk space (blocks access during operation).
- TOAST (The Oversized-Attribute Storage Technique): TOAST manages large column values (e.g., text, byte, jsonb) that exceed ~2 KB. Internal Behavior:
  - Compresses large values when possible.
  - If still too big, stores them in a per-table TOAST table in 2 KB chunks.
  - The main table only holds a pointer to the TOAST data.
  - Cleanup Considerations:
    - VACUUM and autovacuum must also maintain TOAST tables.
    - Large updates/deletes can leave dead TOAST tuples → hidden bloat if not cleaned.
- pg\_repack: It reclaims bloat without blocking most operations, but requires a brief lock during the final table swap. Process Overview:
  - Creates a new compacted copy of the table.
  - Uses triggers to capture ongoing DML changes.
  - Applies captured changes to the new copy.
  - Atomically swaps old and new tables.
  - Not ideal for high-write tables due to the short final lock and trigger overhead.
- pg\_partman: It automates time-based or serial-based partitioning in PostgreSQL.
  - It can auto-create future partitions and drop old ones as per retention policy.
  - Use cron or pg\_cron to schedule periodic maintenance via: `SELECT partman.run_maintenance();`
- Replication means copying data from primary master node → standby slave node in real time.
  - Asynchronous: Primary doesn't wait for standby → faster, may lose last few transactions if crash.
  - Synchronous: Primary waits for standby confirmation → no data loss, but slower.

- Standby node can be used for read-only queries (hot standby). Tools like Patroni or repmgr handle automatic failover.
- Physical replication copies: Binary WAL (whole DB); But Logical rep., copies data changes (rows) - may cause partial sync, migrations, cross-version.
- Failovers (Automatically switching from a primary master db to a standby replica when the primary fails)
  - Manual: run `pg_ctl promote` on standby.
  - Automatic: tools like Patroni, repmgr, `pg_auto_failover` detect failure and promote standby.
- Partitioning: Range, Hash, List Partitioning.
- JSON vs JSONB
  - JSON: Stores exact input text, Faster to insert, Preserves whitespace and key order, No processing overhead on insert, Slower to query
  - JSONB (Binary JSON): Stores decomposed binary format, Slower to insert (decomposition cost), Removes whitespace, no key ordering, Much faster to query and process, Supports indexing
- RLS (row-level-security) restricts which rows users can access. Can enable RLS & add policy via a query.
- Backup Strategies:
  - Logical Backup (`pg_dump` / `pg_dumpall`): Exports SQL statements to recreate data. Use for migrations, small-medium DBs. Supports parallel dumps for speed. Restore with `pg_restore`.
  - Physical Backup (`pg_basebackup`): Copies data files + WALs — exact binary snapshot. Use for large databases, replication setup, or PITR. Faster restore, but must match PG version.
  - Continuous Archiving (PITR): Archives WAL logs continuously. Allows Point-In-Time Recovery (restore to any moment). Needs: Base backup + archived WALs.
    - Point-in-Time Recovery (PITR): Take base backup (`pg_basebackup`). Restore base backup to data directory. Set recovery configs in `postgresql.auto.conf`. Create `recovery.signal` file. Start PostgreSQL → replays WALs until target.

- Connection Pooling:
  - PostgreSQL uses a new OS process per connection → expensive setup & memory cost.
  - Default limit: max\_connections = 100. Frequent connects/disconnects cause connection storms and latency spikes.
  - Pooling reuses connections, reducing overhead and improving throughput.
  - PgBouncer is a lightweight connection pooler for PostgreSQL. It sits between your application and the database, managing and reusing PostgreSQL connections efficiently. Pooling modes:

| Mode                                                                                                    | Behavior                                   | Use Case                      |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------|
| Session Pooling                                                                                         | Connection held for the entire session     | Compatible but limited reuse  |
| Transaction Pooling  | Connection released after each transaction | Best balance for performance  |
| Statement Pooling                                                                                       | Released after every statement             | Very restrictive, rarely used |

- Use pg\_stat\_\* views to monitor runtime activity and performance, and pg\_\* catalogs to inspect metadata and configuration inside PostgreSQL. Eg: pg\_stat\_activity, pg\_stat\_database, pg\_stat\_replication, pg\_lock, pg\_database, etc.

## Timescale (Postgres)

- TimescaleDB is a PostgreSQL extension optimized for time-series data. Since it is built on top of PostgreSQL, it supports full SQL, joins, indexes, transactions, ACID guarantees, extensions, and the broader PostgreSQL ecosystem. On top of PostgreSQL capabilities, TimescaleDB adds hypertables, automatic chunking, compression, continuous aggregates (CAGGs), retention policies, and time-series optimizations. Common use cases include metrics storage, IoT telemetry, observability platforms, monitoring systems, event streams, sensor data, and financial tick data.

- Core Idea of TimescaleDB: Instead of storing all time-series data in one massive table, TimescaleDB automatically partitions data into smaller chunks based on time intervals. This improves insert performance, query pruning, retention management, compression efficiency, and overall scalability for large datasets.
- Hypertables: A hypertable is a logical abstraction over multiple chunk tables. Applications interact with it like a normal PostgreSQL table, while TimescaleDB manages partitioning internally.

Example:

```
CREATE TABLE metrics(
  time TIMESTAMPTZ,
  device_id INT,
  cpu DOUBLE PRECISION
);
```

```
SELECT create_hypertable('metrics', 'time');
```

- Chunks: Chunks are physical partitions created automatically by TimescaleDB. They are typically partitioned by time and optionally by an additional space partition such as device\_id or region. Benefits include partition pruning, improved parallelism, better compression handling, easier retention management, and faster query execution on time-bounded datasets.
- Chunk Interval: Chunk interval defines the time range covered by each chunk, for example 1 day or 7 days. Smaller chunks improve pruning efficiency and query locality but increase metadata and planning overhead. Larger chunks reduce partition count but may hurt pruning and increase scan sizes. A common rule is to size chunks such that actively queried chunks comfortably fit into memory.
- Compression: TimescaleDB supports automatic compression of older chunks. Compression provides major storage savings and can sometimes improve analytical query performance because fewer blocks are scanned. Compression is columnar internally, making it efficient for repetitive time-series data. Typically, recent hot data remains uncompressed for fast writes and updates, while older cold historical data is compressed.

- Retention Policies: Retention policies automatically drop old chunks after a configured duration, for example deleting data older than 90 days. This is especially useful for monitoring and observability systems where raw metrics lose value after some retention window.
- Continuous Aggregates (CAGGs): CAGGs are one of the most important TimescaleDB features. The problem with raw time-series data is that repeatedly aggregating billions of rows is expensive.

Example:

```
SELECT time_bucket('1 hour', time),
       AVG(cpu)
FROM metrics
GROUP BY 1;
```

- As data volume grows, such queries become increasingly expensive. Continuous Aggregates solve this by maintaining incrementally refreshed materialized aggregations over time-series data. Unlike traditional materialized views, CAGGs refresh incrementally instead of recomputing everything.

Example:

```
CREATE MATERIALIZED VIEW cpu_hourly
WITH (timescaledb.continuous)
AS
SELECT time_bucket('1 hour', time) AS bucket,
       device_id, AVG(cpu) avg_cpu
FROM metrics
GROUP BY bucket, device_id;
```

- How CAGGs Work: Raw data remains stored in hypertables, aggregate results are stored separately, and background jobs refresh only modified time buckets instead of recomputing the entire dataset. This provides major performance improvements for dashboards and analytical workloads.
- Real-time Aggregates: CAGGs can combine materialized historical aggregates with fresh recent raw data, allowing near real-time query results while still avoiding expensive full-table aggregations.
- Refresh Policies: Background jobs automatically refresh aggregates at configured intervals, for example every 5 minutes. Refresh windows are

important because late-arriving data may require recomputation of recent buckets. A common strategy is continuously refreshing the last few hours of data.

- Time Buckets: `time_bucket()` is TimescaleDB's core time-grouping function. Example: `time_bucket('5 minutes', time)`
  - It is conceptually similar to `DATE_TRUNC` but optimized specifically for time-series aggregation and heavily used with CAGGs.
- Late Arriving Data: Late-arriving events are common in IoT and distributed systems, for example an event generated at 10:00 arriving at 10:10. Continuous aggregates handle this using refresh windows that periodically recompute recent buckets. The refresh window must be large enough to account for expected lateness.
- Data Retention with CAGGs: A common architecture is retaining raw data for a short duration while keeping aggregated data much longer. For example, raw metrics may be retained for 7 days while hourly aggregates are retained for 1 year.
- Compression + CAGGs Pattern: A very common production setup is: recent data remains raw and uncompressed for fast ingestion, older historical data is compressed for storage efficiency, and analytical queries primarily hit continuous aggregates. This significantly reduces storage cost and query latency.
- Why TimescaleDB over Plain PostgreSQL Partitioning?: TimescaleDB provides automatic chunk management, optimized query planning, built-in compression, continuous aggregates, automated retention handling, and simpler operational management compared to manually managing PostgreSQL partitions.
- Why not Cassandra or InfluxDB?: TimescaleDB benefits from the PostgreSQL ecosystem, meaning full SQL support, joins, relational modeling, strong consistency, mature tooling, and transactional guarantees. However, extremely large-scale write-heavy workloads may scale better horizontally in systems like Cassandra.
- Important Internals: During inserts, rows are automatically routed into the correct chunks. During query execution, the planner prunes irrelevant

chunks using time filters. Without time predicates, queries may scan many chunks and become slow.

- Optimization Rule: Always query with a time filter. Eg: Bad: `SELECT * FROM metrics WHERE device_id = 1;`  
Better: `SELECT * FROM metrics WHERE time > now() - interval '1 day' AND device_id = 1;`
- Indexes in TimescaleDB: Common indexes are: (time DESC) or (device\_id, time DESC). This is because most workloads are recent-first and time-range bounded. Compression on old chunks may reduce the usefulness of indexes for historical analytical queries since compressed scans are often more efficient than indexed lookups.

## Clickhouse

- ClickHouse is an open-source, high-performance columnar OLAP database designed for real-time analytics on massive datasets. It is optimized for read-heavy analytical workloads rather than transactional systems.
- ClickHouse uses a columnar storage model, where data is stored column-by-column instead of row-by-row. This enables highly efficient compression, faster scans, and reading only required columns during query execution. It is designed for OLAP workloads, meaning it excels at aggregation-heavy queries such as GROUP BY, COUNT, SUM, and time-series analytics over billions of rows.
- A key concept in ClickHouse is the MergeTree engine family, which is the foundation of most production tables. Data inserted into ClickHouse is written as immutable “parts”, and background processes continuously merge these parts into larger optimized segments. This is similar to LSM-tree compaction in concept. Because of this design, batch inserts are highly recommended, while frequent small inserts degrade performance due to excessive part creation and merge overhead.
- ClickHouse uses a sparse primary index, meaning it does not index every row like B-tree databases. Instead, it stores index marks at intervals (granules), typically every 8192 rows. This allows ClickHouse to quickly skip large portions of data during scans while keeping index size very small. A granule is the smallest unit of data that can be read during query execution.



- Query execution in ClickHouse follows a highly parallel and vectorized model. Data is read in columnar batches, processed using SIMD-style operations, and executed across multiple CPU cores, partitions, and shards simultaneously. This results in very high throughput for aggregation-heavy workloads.
- ClickHouse achieves high performance through multiple factors: columnar storage, compression (LZ4, ZSTD, delta encoding), sparse indexing, partition pruning, vectorized execution, and parallel query execution. These optimizations together enable ClickHouse to process billions of rows per second on commodity hardware.
- Replication in ClickHouse is handled using ReplicatedMergeTree with coordination via ZooKeeper or ClickHouse Keeper. Replication is asynchronous and ensures high availability across nodes. Sharding is used for horizontal scaling, where data is distributed across multiple nodes and queries are executed in parallel across shards using Distributed tables.
- Materialized Views are a major optimization feature in ClickHouse. They allow pre-aggregation or transformation of data during ingestion itself. For example, raw event data can be automatically aggregated into hourly or daily summaries, significantly improving dashboard query performance.
- ClickHouse supports TTL (Time-To-Live) policies for automatic data expiration or movement to cheaper storage. This is heavily used in observability and log systems where data retention is limited to a specific time window.
- Instead of traditional indexes, ClickHouse uses data-skipping indexes such as minmax, bloom filters, and set indexes. These indexes help skip irrelevant data blocks instead of pinpoint row-level access, making them efficient for analytical queries.
- ClickHouse is especially powerful for event analytics, logs, metrics, security analytics, and real-time dashboards. It is commonly integrated with Kafka, where data flows from Kafka into ClickHouse via Kafka engine tables and materialized views into MergeTree tables.
- However, ClickHouse is not suitable for heavy transactional systems. Updates and deletes are expensive because data is immutable, and mutations require rewriting data parts asynchronously. While updates are

supported (e.g., via `ReplacingMergeTree`), they are not instantaneous and should not be used for frequent OLTP-style operations.

- A key design best practice is choosing the correct ORDER BY key. It should be based on common filter conditions and time-series patterns to maximize data skipping. Poor ORDER BY design leads to full scans and degraded performance.
- Memory management is important in ClickHouse, especially for aggregations and joins. Large GROUP BY operations may spill to disk if memory limits are exceeded. Although joins are supported, ClickHouse performs best with denormalized data models or pre-aggregated structures.
- Common table engines include `MergeTree` (base engine), `ReplacingMergeTree` (deduplication), `SummingMergeTree` (aggregation), `AggregatingMergeTree` (state-based aggregation), and `ReplicatedMergeTree` (replication-enabled storage). Each engine is optimized for specific analytical patterns.

## NATS/Jetstream

- NATS (Neural Autonomic Transport System) is a messaging system that allows different parts of your software to communicate with each other. It delivers messages between services without those services needing to know each other's location or availability.
- At the heart of NATS is subject-based messaging. In NATS, you don't create queues or topics; instead, you publish messages to subjects, which are simple strings describing what the message is about, for example "orders.created" or "users.login.success". These subjects don't need to be created beforehand; they exist simply because you use them. Think of subjects like email addresses. You don't register them before sending an email; you just send to them, and if someone is listening they receive it, otherwise the message goes nowhere.
- Subjects can also be hierarchical, such as "orders.us.california.store1.created" or "orders.europe.uk.london.updated". This hierarchy is not just organizational; it enables powerful pattern matching.
- NATS supports two wildcards. The asterisk (\*) *wildcard matches exactly one token in the subject hierarchy*. So "orders..created" matches

“orders.us.created” or “orders.europe.created”, but not “orders.us.california.created” because it has more tokens. The greater-than (>) wildcard matches one or more tokens at the end, so “orders.us.>” matches “orders.us.created”, “orders.us.california.store1.created”, and anything starting with “orders.us.”.

- (Publish-Subscribe Model): The core publish-subscribe model works like this: a publisher sends a message to a subject, and subscribers receive messages that match that subject. In core NATS, messages exist only at the moment of publication. If subscribers are present, they receive copies immediately; if not, the message is dropped. This fire-and-forget nature makes NATS extremely fast, but it also means there is no persistence.
- Queue groups allow load balancing across multiple instances of the same service. When multiple subscribers join the same queue group for a subject, NATS delivers each message to only one member of the group, chosen randomly. This makes horizontal scaling of services very simple.
- A NATS cluster is a group of NATS servers working together as a unified system. From the client’s perspective, it behaves like a single server. The client connects to one node, and if it fails, it automatically reconnects to another server in the cluster. Inside the cluster, servers communicate using a gossip protocol. When a message is published, the server forwards it to other servers that have subscribers interested in that subject, ensuring efficient distribution across the system.
- Core NATS provides at-most-once delivery because it does not store messages. Once a message is sent, it is not retained.
- JetStream is NATS’s distributed persistence layer. It adds storage, replay, and guaranteed delivery while remaining integrated into the same NATS server, so no separate system is required.
- A stream in JetStream is an append-only log of messages. It is configured to capture messages published to specific subjects, and every matching message is stored according to defined retention policies. Streams can store data in memory or on disk depending on configuration.
- Consumers define how messages are read from streams.
  - Push consumers behave like traditional pub-sub, where JetStream actively pushes messages to the application as they arrive.

- Pull consumers work differently: the application explicitly requests messages when ready, for example asking for the next batch, which gives precise control over flow and backpressure.
  - Ordered consumers are a special case designed for strict ordering guarantees without requiring acknowledgments, simplifying use cases where ordering is critical.
- JetStream supports acknowledgments to track message processing. While it helps reliability, exactly-once processing is not automatic and must be handled carefully at the application level.
- Message deduplication is supported using message IDs. If a message with the same ID is published within a deduplication window, JetStream discards it, preventing accidental duplicates.
- Consumers can also filter messages based on subject or headers, allowing fine-grained control over what each consumer receives.
- Durable consumers maintain state, including acknowledged messages. If a consumer disconnects and reconnects, it can resume from where it left off using the same durable name.
- Ephemeral consumers exist only as long as there's an active subscription to them. When the last subscription is closed, the consumer is automatically deleted along with all its state. Ephemeral consumers are useful for temporary operations like debugging or for real-time processors that don't need to maintain state across restarts.
- When you create a stream, you specify how many replicas you want. A replication factor of one means the stream is stored on a single server with no replication. A replication factor of three means the stream is stored on three different servers. JetStream uses the Raft consensus algorithm for replication. For each stream, one node acts as leader and handles all writes, which are then replicated to followers. A write is confirmed only when a majority of replicas acknowledge it.
- A key-value store in JetStream is implemented on top of streams. Each key operation is stored as a message, and the latest message for a key represents its current value. This gives full history, auditability, and the ability to watch changes in real time.
- Similarly, the object store is built on streams and is used for large data such as files or machine learning models. Large objects are automatically

chunked into messages and reassembled when retrieved, making storage and retrieval transparent while still benefiting from JetStream's durability and replication guarantees.

## CSE Fundamentals

- A Web Server receives HTTP requests, serves static content, and forwards dynamic ones to application frameworks (FastAPI, Spring, etc) through interfaces like WSGI (Web Server Gateway Interface, synchronous) or ASGI (Asynchronous Server Gateway Interface, async). In Python, Gunicorn + UvicornWorker runs ASGI apps like FastAPI, similar to how Tomcat hosts Spring apps via the Servlet API in Java.
  - WSGI apps block per request (single thread per request).
  - ASGI apps are event-driven — ideal for concurrent tasks like WebSockets, real-time APIs, or heavy I/O workloads.

Consider environment:

You have FastAPI app (async-based).

Running under Gunicorn, with `--worker-class uvicorn.workers.UvicornWorker` (ASGI worker).

You configured 4 Gunicorn workers.

Each worker handles requests to 3 DynamoDB tables, via aioboto3 async clients.

Machine: 4 cores, 16 GB RAM.

Deployed as 2 ECS tasks, behind a load balancer.

How the System Handles a Request (Step by Step)

A client sends an HTTP request → LB (like ALB) forwards it to one of the ECS tasks.

You have 2 ECS tasks, each running 4 Gunicorn workers.

→ total of 8 worker processes across your cluster.

Each worker is a separate process, each having its own GIL and own asyncio event loop.

Gunicorn manages process-level parallelism.

Each Gunicorn worker:

Runs UvicornWorker, which spins up an ASGI server and event loop (asyncio).

Each worker runs in its own process (→ independent GIL per

worker).

Each worker can handle many concurrent async requests (not just one).

Concurrency model is multi-process (Gunicorn) + async inside each process (Uvicorn).

When a request hits FastAPI, it's handled by an async endpoint.

Python is not truly multi-threaded for CPU-bound tasks (due to GIL),

but async I/O is cooperatively multitasked within the same thread.

That means: Only one coroutine runs on the CPU at a time,

But when it hits an I/O wait (like network call to DynamoDB),

→ it awaits the operation,

→ releases control back to the event loop,

→ and allows another coroutine (request handler) to run.

So while your coroutine is waiting for Dynamo to respond, the same thread can process other requests.

The Global Interpreter Lock (GIL) only affects Python bytecode execution (CPU work).

It does not block I/O waits.

Since your workload is I/O-bound (network to DynamoDB), GIL doesn't degrade performance much.

So total concurrency capacity =

ECS tasks \* Gunicorn workers \* concurrent coroutines per worker.

E.g. if each worker handles 500 concurrent connections efficiently,

→ you can handle  $2 \times 4 \times 500 = 4000$  concurrent connections.

More about these connections:

When a client (e.g., browser, mobile app, or another backend) makes an HTTP request:

It opens or reuses a TCP connection (possibly HTTP/1.1 keep-alive

or HTTP/2 stream).

The ASGI server (Uvicorn running inside each Gunicorn worker) accepts this socket.

It passes control of that connection to your FastAPI app's coroutine.

In async mode:

Each connection corresponds to one coroutine (async task).

The event loop can hold hundreds or thousands of coroutines concurrently, because each coroutine is paused most of the time (waiting for I/O like DynamoDB responses).

Unlike traditional synchronous servers (where one thread handles one request),

async servers don't block threads during I/O waits.

Even though it's all on one thread, you can have hundreds of outstanding requests.

What limits concurrency?

File descriptors (ulimit -n): Max open sockets

ASGI server max\_connections: Max concurrent connections per worker

Memory per connection: Each request has context data & response buffer, Eg: 100KB per req.

Network I/O speed: Bandwidth, latency to DynamoDB

Event loop scheduling overhead: Async tasks per loop

If multiple clients create more simultaneous requests and if all workers are full, Gunicorn starts queueing or rejecting new connections (HTTP 503 or timeout).

A single client (like a browser) usually reuses a few persistent TCP connections (keep-alive).

Also, we have two layers of async in your setup:

Gunicorn + UvicornWorker → runs FastAPI as an ASGI async web server

aioboto3 → makes async network I/O calls to DynamoDB

Both are async, but they're not "separate async systems" – they're

running inside the same event loop.

If inner logic been sync - it would block thread during API call, impacting latency.

- Core is an individual physical processing unit, while threads are logical/virtual sequences of instructions. If a CPU has more threads than cores, say 4 cores, 8 threads so 2 threads per core, in such a case, the core would be context switching between each thread to process the task. 1 thread per core will get you the best performance. Adding more threads usually helps, but after some point, they cause some performance degradation.
- CPU-intensive vs IO-intensive tasks:
  - CPU-intensive tasks involve complex calculations, mathematical operations, data processing, or simulations that consume a substantial amount of CPU time.
  - Thread-Intensive (I/O-Bound) processes are characterized by tasks that spend a significant amount of time waiting for I/O operations to complete, such as reading/writing to files, making network requests, or interacting with databases. Many I/O operations are inherently blocking, which means they can cause a thread to wait for external resources (e.g., network data, disk I/O) to become available. During this waiting period, the thread is effectively idle, and the system may benefit from switching to another thread to perform useful work.
- CPU vs GPU:
  - CPU: Has few cores (say 4–16) but each is very powerful and complex. It focus on low latency per task – doing one task really fast. Each core has:
    - Large control logic and branch prediction units (for “if/else”, loops).
    - Large caches (L1, L2, L3) for low-latency memory access.
    - Out-of-order execution and speculative execution for efficiency.
  - GPU: Has hundreds or thousands of simpler cores (Streaming Multiprocessors in NVIDIA). Each “core” can run thousands of



threads simultaneously. Very small control logic – most transistor area is used for arithmetic units (ALUs). High memory bandwidth (GDDR / HBM) but less cache per core. Executes code in SIMD / SIMT (Single Instruction, Multiple Data / Thread) fashion: All threads in a “warp” (group of ~32) execute the same instruction in parallel on different data.

- Concourse:

- A pipeline defines your workflow – how code moves from build → test → deploy.
- Each job represents a unit of work – e.g., build, test, deploy. Has steps: get, task, put.
- Resources are external inputs/outputs that Concourse interacts with. Eg: Git repo, S3, Docker, etc.
- Task: The actual work done – e.g., run tests, build code. A task runs inside a container, defined by a task.yml.
- Workers are machines (VMs or containers) that run tasks. Each task in a pipeline runs in a container on a worker.
- Web UI → visual view of pipelines, jobs, and status. Fly CLI → used to interact with Concourse (login, set pipelines, trigger builds).
- Internally: Web Node (UI), Worker Node (Runs tasks), Postgres DB (Pipeline/resource details), ATC (Air Traffic Controller - Concourse scheduler), Garden / Containerd (Container runtime engine to isolate tasks).

- Tables:

- Use **external tables** when data lives in **S3 / external systems** (so you don't lose it).
  - `MSCK REPAIR TABLE my_table`: Automatically discovers new partitions by scanning the S3 directory structure of a Hive/Glue table and adds them to the metastore. Adds new/missing partitions. Slow for large datasets, scans entire S3 folder tree. Works only for Hive-style partitioning (e.g., key=value directories).
  - `ALTER TABLE my_table ADD PARTITION (date='2025-10-24', country='IN') LOCATION`: Adds the exact partitions you mention & not full scan, faster than msck command

- Use **internal tables** when the warehouse manages lifecycle fully.
- Concurrency vs Parallelism:
  - Concurrency: Doing many things at once (logically) – tasks start, run, and finish in overlapping time, but not necessarily simultaneously. Eg: A single-core CPU switching between tasks A and B so quickly that both seem to run together.
  - Parallelism: Doing many things literally at the same time, using multiple cores or processors. Eg: Task A runs on Core 1, Task B runs on Core 2, truly at the same instant.
- Explode vs Normalize: JSON explode creates more rows by expanding a column with lists, while JSON normalize creates more columns by flattening nested JSON objects within a column.

Consider the df:

|   | id | name  | scores       |
|---|----|-------|--------------|
| 0 | 1  | Alice | [90, 95, 88] |
| 1 | 2  | Bob   | [85, 92]     |

```
df_exploded = df.explode('scores')
```

The df\_exploded DataFrame after applying JSON explode:

|   | id | name  | scores |
|---|----|-------|--------|
| 0 | 1  | Alice | 90     |
| 0 | 1  | Alice | 95     |
| 0 | 1  | Alice | 88     |
| 1 | 2  | Bob   | 85     |
| 1 | 2  | Bob   | 92     |

```
df_normalized = pd.json_normalize(df, 'scores', ['id', 'name'],
sep='_')
```

The df\_normalized DataFrame after applying JSON normalize:

|   | id | name  | scores_0 | scores_1 | scores_2 |
|---|----|-------|----------|----------|----------|
| 0 | 1  | Alice | 90       | 95       | 88       |
| 1 | 2  | Bob   | 85       | 92       | NaN      |

- Database:
  - Schemas:

- Database Schema: It refers to the overall blueprint or logical structure of a database including tables, views, relationships, constraints, indexes, and other database objects
  - Table Schema: Refers to the structure of an individual table within the database. It defines the columns (fields), data types, constraints, and other attributes.
- Processing:
  - Online analytical processing (OLAP) is to analyze aggregated data.
    - Data Storage: Column-by-column (each column stored separately). Eg: `SELECT AVG(balance) FROM customer;`
      - Database reads only the Balance column, which is stored contiguously on disk and compressed efficiently.
  - Online transaction processing (OLTP) is to process database transactions.
    - Data Storage: Row-by-row (all fields of a record together)
- Normalization: It is to avoid complexities, eliminate duplicates, and organize data in a consistent way. 1NF - no duplicated rows or columns, 2NF - should be 1NF + no partial dependency, i.e, all non-key attributes are fully dependent on a primary key, 3NF - be in 2NF + no transitive partial dependency, etc.
- Denormalization attempts to improve read performance at the expense of some write performance. Redundant copies of the data are written in multiple tables to avoid expensive joins.
- Cardinality: Uniqueness of data contained in a column. Eg: id - high cardinality.
- ACID is a set of properties of relational database transactions.
  - Atomicity - Each transaction is all or nothing
  - Consistency - Any transaction will bring the database from one valid state to another
  - Isolation - Executing transactions concurrently has the same results as if the transactions were executed serially
  - Durability - Once a transaction has been committed, it will remain so

- Federation sharding: Federation (or functional partitioning) splits up databases by function. For example, instead of a single, monolithic database, you could have three databases: forums, users, and products, resulting in less read and write traffic to each database and therefore less replication lag.
- Reasons for choosing:
  - SQL db: Structured data; Strict schema; Relational data; Need for complex joins; Transactions; Clear patterns for scaling; More established: developers, community, code, tools, etc; Lookups by index are very fast
  - NoSQL db: Semi-structured data; Dynamic or flexible schema; Non-relational data; No need for complex joins; Store many TB (or PB) of data; Very data intensive workload; Very high throughput for IOPS
- Database Indexing: Index is just a data structure that makes the searching faster for a specific column in a database. Usually a b-tree or a hash table structure. Creating an index on a field in a table creates another data structure which holds the field value, and a pointer to the record it relates to. This index structure is then sorted, allowing Binary Searches to be performed on it. Eg: Books - Pages - Table of Contents, etc.
  - Data on Disk: Data is stored in blocks (pages) — the smallest unit of disk I/O. Each block holds multiple records and includes pointers to other blocks (like a linked list). Blocks need not be contiguous in storage.
  - Without Index: Searching requires scanning blocks sequentially (linear search). Average cost:  $(N+1)/2$  block accesses (where  $N$  = number of blocks). For non-key fields (duplicates allowed), entire table may need to be scanned ( $\approx N$  accesses).
  - With Sorted Field / Index: Enables binary search, reducing cost to  $\log_2 N$  block accesses. Indexes store sorted mappings of key  $\rightarrow$  block pointer. Allows direct lookup instead of full-table scan.

- Trade-off: Indexes speed up reads/searches, but slow down writes (INSERT/UPDATE/DELETE must also update the index). They also consume extra disk space.
- View is a virtual table whose contents are defined by a query. You can think of a view as a "saved select statement" that you can repeat.
  - Materialized VIEW: It is a special type of view that stores the result of a query in a physical table. Unlike a regular view, a materialized view does not update automatically when the underlying tables or views change. Instead, you have to refresh it manually or on a schedule using the REFRESH MATERIALIZED VIEW statement.
- Query Federation in Databricks: Query federation allows Databricks to execute queries against data served by other Databricks metastores as well as many third-party database management systems (DBMS) such as PostgreSQL, MySQL, AWS Redshift and Snowflake. To query data from another system you must: Create a foreign connection., etc. Simply put, from databricks UI itself you can run queries over external/third-party db post necessary connections are set up.
- Thread/Process-Core Model Py
  - Thread-per-core model: Single process contains multiple threads, with one thread assigned to each CPU core. Shared address space means all threads can access the same memory. Lightweight synchronization through mutexes, semaphores, etc. Good for: I/O-bound workloads with shared state, Applications where memory efficiency is critical, Languages with efficient threading models (Java, C++)
  - Process-per-core model: Multiple independent processes, with one process assigned to each CPU core. Separate memory spaces with no direct access between processes. More robust isolation prevents cascading failures. Good for: CPU-bound workloads, Languages with threading limitations (Python with GIL), Applications requiring strong fault isolation, Avoiding complex thread synchronization issues.

- If I run 1 process with 4 threads for CPU-heavy work, due to GIL only one thread runs/executes python bytecode at a time on a core (time-sliced), remaining cores are idle. But if I run 4 processes, each on one core, they can run in parallel and be faster. Note that each process has its own Python interpreter and GIL.
- A thread is the unit of execution within a process. A process has at least one thread. It is called the main thread. A process can have many threads. Each thread has its own stack memory, registers, program counters, etc.
- Threads within a process share a memory address space. It is possible to communicate between threads using that shared memory space. However, one misbehaving thread could bring down the entire process. The operating system runs a thread or process on a CPU by context switching.
- Context switching is expensive. It involves saving and loading of registers, switching out memory pages, and updating various kernel data structures. It is generally faster to switch context between threads than between processes. There are fewer states to track, and more importantly, since threads share the same memory address space, there is no need to switch out virtual memory pages, which is very expensive.
- In case of Python:
  - Python 3.13+ introduced an experimental "free-threaded" mode. In this mode, the Global Interpreter Lock is removed, allowing multiple threads within a single process to execute Python bytecode in true parallel across multiple CPU cores.
    - Past (GIL-bound Python): Even on a 4-core machine, Python behaved like a single-lane road on a four-lane highway. Only one thread could execute Python bytecode at a time due to the GIL, so one core was busy while the other three cores sat idle, even within the same process.
    - Present (Python 3.13 – Free-threaded mode option): The highway is fully open. In a single process, 4 threads

can truly run in parallel, each executing on its own core, allowing all 4 cores to be utilized simultaneously.

- You must explicitly run `--disable-gil` / free-threaded build.
  - True parallelism applies to CPU-bound Python code. Many C extensions are not yet thread-safe - Hence as a note while the interpreter handles internal safety, remember that your own code still needs to be "thread-safe" using mutexes if multiple threads are modifying the same global variable at once.
- Generators vs Co-routines in Py
    - Generators:
      - Lazy iteration — produce a sequence of values on demand, without storing them in memory.
      - Each call to `next()` resumes from the last yield point. Execution pauses and resumes automatically.
      - Defined using: `def + yield`. Use: Streaming large datasets, pipelines, infinite sequences, reading large files, etc.

A generator is just a special kind of function that can pause and resume. Normally, when you call a function: It runs from start to end; Returns a value once (via `return`); After that, it's done -- memory is freed. But with a generator, when it hits a `yield`, it pauses instead of finishing. It remembers its position, local variables, etc. The next time you ask for a value, it resumes from where it left off.

Eg1:

```
def countdown(n):
    while n > 0:
        yield n      # pause here and give back n
        n -= 1       # resume from here next time

gen = countdown(3)
print(next(gen))    # prints 3
print(next(gen))    # prints 2
print(next(gen))    # prints 1
print(next(gen))    # raises StopIteration
```



```
## We use it as it doesn't store all values in memory.
```

Eg2:

```
with open("huge_file.txt") as f:
    lines = f.readlines() # ✗ Loads entire file into memory
    for line in lines:
        process(line)
```

Instead (below): The generator reads one line from disk at a time. After processing it, the memory is freed. Your app can handle huge files with constant memory usage (a few KBs).

```
def read_large_file(file_path):
    with open(file_path) as f:
        for line in f:
            yield line.strip() # ⚡ yield one line at a time
for log_line in read_large_file("huge_file.txt"):
    process(log_line)
```

- Coroutines:
  - Asynchronous I/O — allow tasks to pause while waiting for I/O, freeing the event loop to run others.
  - await marks points where coroutine yields control to the event loop. The event loop schedules and resumes them when I/O completes. They can run concurrently, though not in parallel (within one thread).
  - Defined using: `async def` + `await`. Use: Network calls, DB queries, file I/O — any I/O-bound workload.
- HTTP:
  - HTTP/1.1 (1999 — Traditional Web)
    - Transport: TCP
    - Text-based protocol — requests and responses are human-readable.
    - Each request-response pair uses a separate TCP connection (in HTTP/1.0).
    - HTTP/1.1 added persistent connections (keep-alive), so multiple requests reuse one connection sequentially.
    - Still head-of-line (HOL) blocking at the HTTP layer — only one request in flight per connection; others must wait.



- Head of Line blocking in HTTP terms is often referring to the fact that each browser/client has a limited number of connections to a server and doing a new request over one of those connections has to wait for the ones to complete before it can fire it off. The head of line requests block the subsequent ones. HTTP/2 solves this by introducing multiplexing so that you can issue new requests over the same connection without having to wait for the previous ones to complete. In theory, the pipelining of HTTP/1.1 also offered a way around HOL, but it was tricky and very error-prone to implement in practice. That has made it not get widely enabled on the web even up till today.
  - Workarounds: browsers opened 6–8 parallel TCP connections per domain to simulate concurrency.
  - Problem: Latency from TCP slow start + blocking + redundant headers (same headers repeated each time).
- HTTP/2 (2015 – Modern Web)
  - Transport: Still TCP
  - Binary protocol – efficient, compact framing (not text-based).
  - Multiplexing: multiple requests and responses can share a single TCP connection concurrently – removes HTTP-level HOL blocking.
  - Header compression (HPACK): reduces redundancy in repeated headers (e.g., cookies).
  - Server Push: server can proactively send resources client will likely need.
  - Remaining limitation: Still suffers from TCP-level head-of-line blocking – if one packet is lost, all streams in that connection stall until it's retransmitted.
- HTTP/3 (2022 – Future of Web)
  - Transport: QUIC (built on UDP)
  - QUIC provides built-in encryption (TLS 1.3) and multiplexing at transport layer – no TCP involved.

- No TCP-level HOL blocking: each stream runs independently, so packet loss on one stream doesn't stall others.
  - Faster connection setup: 0-RTT or 1-RTT handshake vs 3-way TCP + TLS handshake in HTTP/1/2.
  - Better mobility: connection identified by connection ID (not IP), so if you switch networks (e.g., Wi-Fi → mobile), connection stays alive.
  - Result: HTTP/3 is faster for real-world web browsing, especially over unreliable or mobile networks.
- Race condition: Occurs when system attempts to perform two or more operations at the same time, could lead to issues if due to nature of the system, the operations must be done in the proper sequence.
  - [Order of the loops affect performance when iterating over a 2D array](#) al:

Version 1 (slow)

```
main () {
    int i, j;
    static int x[4000][4000];
    for (j = 0; j < 4000; j++) {
        for (i = 0; i < 4000; i++) {
            x[i][j] = i + j; }
    }
}
```

Version 2 (fast)

```
main () {
    int i, j;
    static int x[4000][4000];
    for (i = 0; i < 4000; i++) {
        for (j = 0; j < 4000; j++) {
            x[i][j] = i + j; }
    }
}
```

The issue is the store to the memory location in the array: `x[i][j]`. You have a 2-dimensional array, but memory in the computer is inherently 1-dimensional. So while you imagine your

```
array like this:
0,0 | 0,1 | 0,2 | 0,3
----+-----+-----+----
1,0 | 1,1 | 1,2 | 1,3
----+-----+-----+----
2,0 | 2,1 | 2,2 | 2,3
```

Your computer stores it in memory as a single line:

```
0,0 | 0,1 | 0,2 | 0,3 | 1,0 | 1,1 | 1,2 | 1,3 | 2,0 | 2,1 | 2,2...
```

In the 2nd version (fast), you access the array by looping over the 2nd number first, i.e.: `x[0][0]` `x[0][1]` `x[0][2]` `x[0][3]`

`x[1][0]` etc... ...Meaning that you're hitting them all in order.

In first case, it's jumping here and there.

Data from your memory gets brought over to the CPU in little chunks (called 'cache lines'), typically 64 bytes. If you have 4-byte integers, that means you're getting 16 consecutive integers in a neat little bundle. It's actually fairly slow to fetch these chunks of memory; your CPU can do a lot of work in the time it takes for a single cache line to load.

Now look back at the order of accesses:

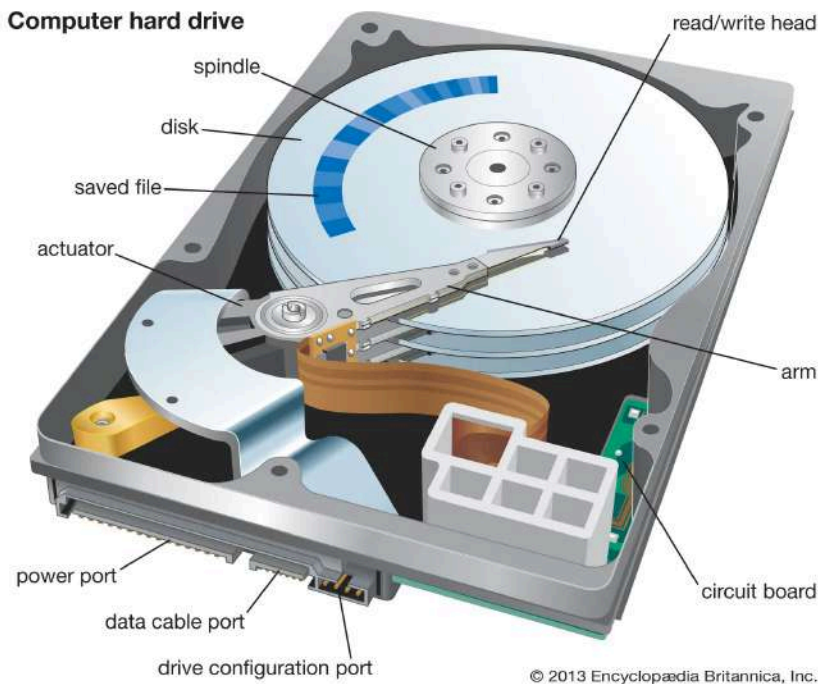
- The second "fast" version is (1) grabbing a chunk of 16 ints, (2) modifying all of them, (3) repeat  $4000 \times 4000 / 16$  times. That's nice and fast, and the CPU always has something to work on.
- The first "slow" version is (1) grab a chunk of 16 ints, (2) modify only one of them, (3) repeat  $4000 \times 4000$  times. That's going to require 16 times the number of "fetches" from memory. Your CPU will actually have to spend time sitting around waiting for that memory to show up, and while it's sitting around you're wasting valuable time.

- Medallion Architecture is a data design pattern used to logically organize data in a lakehouse, with the goal of incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture (from Bronze  $\Rightarrow$  Silver  $\Rightarrow$  Gold layer tables).
- Namespace: It is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it.

- [Why is processing a sorted array faster than processing an unsorted array](#)  
[al](#): It involves concept of Branch prediction. Branch prediction attempts to guess whether a conditional jump will be taken or not. Branch target prediction attempts to guess the target of a taken conditional or unconditional jump before it is computed by decoding and executing the instruction itself. It is a technique to predict the outcome of a conditional operation.
- Ephemeral storage is a temporary, non-persistent storage space for a compute instance that is erased when the instance is terminated.
- A bastion host is a publicly facing server that acts as an entry-point to the system which is protected from the high-end firewall or located in a private server.
- C++ vs Java Code execution differences:

| Aspect      | C/C++                                                                                                            | Java                                                                                                                                                                                               |
|-------------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Compilation | Source code → compiled by a native compiler → machine code (binary)                                              | Source code → compiled by <code>javac</code> → bytecode (.class)                                                                                                                                   |
| Execution   | Executed directly by CPU (hardware-dependent).<br>Compiled binary is not portable across architectures or OSes.  | Executed by JVM (Java Virtual Machine). “Write Once, Run Anywhere” – JVM handles platform differences.                                                                                             |
| Target      | Architecture-specific (x86, ARM, etc.)                                                                           | Platform-independent (JVM provides abstraction)                                                                                                                                                    |
| Other       | Precompiled machine code → near-native performance.<br>Manual memory management using malloc/free or new/delete. | JVM uses JIT (Just-In-Time) Compilation → dynamically compiles frequently executed (“hot”) code to native machine code. Can perform runtime optimizations. Garbage collector takes care of memory. |

- P99 latency: 99th percentile latency; It tells you that 99% of requests are faster than this number, and only the slowest 1% take longer. In distributed systems, tail latencies dominate user experience.
- Storage Evolution: From CHS to LBA to ZNS



- Disk components:
  - Platters: Several circular disks stacked vertically. Each coated with magnetic material
  - Spindle: Rotates all platters at high speed
  - Tracks: Concentric circles on platter
  - Head = Which platter surface
  - Sectors: Each track divided into 512B / 4KB sectors
  - Cylinder: Stack of matching tracks across platters
- Early Era — CHS (Cylinder, Head, Sector) Addressing:
  - OS accessed drives via physical geometry: (cylinder, head, sector) → exact physical location.
  - Problem: Tight coupling between OS and hardware. Any change in drive geometry required OS and driver updates.
- Abstraction Layer — LBA (Logical Block Addressing)

- Replaced CHS with a logical view: drive = flat array of blocks (LBA 0..N). The drive firmware (controller) internally maps LBAs → physical locations.
  - Pros: Platform independence; Cons - Added indirection overhead in mapping table, etc.
- Modern Revisit — ZNS (Zoned Namespace) NVMe
  - Idea: Reintroduce partial geometry awareness back to the OS. Drive divided into Zones — groups of LBAs guaranteed to be physically contiguous.
  - Rules: Writes must be sequential within a zone. A zone must be erased entirely before reuse. Drive still does mapping, but only zone-level, not block-level (smaller mapping table).
- Eg: RocksDB, LevelDB (LSM-based stores) benefit because their design is append-only / sequential-write heavy, matching ZNS semantics.
- Docker Image Optimization: Faster CI/CD pipelines, reduced network transfer, quicker scaling on K8s, lower storage costs across registries (ECR), better security, etc. Some tips:
  - Base Image Choice — Minimal Alpine: Eg: Switch from python:3.9 (~900 MB) → python:3.9-alpine (~40 MB). ~95% size reduction.
  - Layer Minimization: Every RUN, COPY, ADD → new filesystem layer. Group related commands: RUN apk update && apk add --no-cache git && rm -rf /var/cache/apk/\*... etc.
  - Use .dockerignore: Avoid copying unnecessary files (like .git/, venv/, \_\_pycache\_\_/).
  - Place static layers (like requirements.txt) before dynamic ones (COPY . .) to reuse cache efficiently. Eg:

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
# This way, dependency install layer is reused unless
requirements.txt changes.
```

- Single-stage images include build tools (gcc, gfortran, build-base, musl-dev, etc.) → bloat image size. Hence do multi-stage builds.

- Stage 1: Builder: Installs heavy build dependencies -> Builds and installs Python deps (pip install) -> Optionally removes large libs after wheels are built.
  - Stage 2: Production: Copies only built artifacts and runtime files -> No compiler or headers → ultra-lean runtime.
- Decorators vs Annotations:
  - Decorators: Used to modify or enhance the behavior of code elements (functions, methods, classes) by wrapping them with additional functionality. Eg:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Something before the function.")
        result = func(*args, **kwargs)
        print("Something after the function.")
        return result
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# In this example, @my_decorator adds "Something before..." and
"Something after..." messages to the say_hello function's
execution without directly modifying say_hello itself.
```

- Annotations: Primarily used to add metadata to code elements (classes, methods, fields, parameters) without altering their core logic or behavior. Can be imagined as passive markers or tags. Eg:

```
@Override
public String toString() {
    return "MyObject";
}

// Other ex:
@RestController
class UserController {
```

```
@GetMapping("/users")  
public List<User> getUsers() { ... }  
}
```



# AI Usage

# How I Use AI (Claude Code-centric workflow)

(Apart from Claude, I also use Gemini, WhisperAi, etc.)

## 1. Project Setup & Context Management

- I use `/init` to generate a `claude.md` file that acts as **project memory and guidance**.
  - Maintain structured context:
    - `claude.md` → shared project rules and context
    - `claude.local.md` → local/personal overrides
    - `.claude/claude.md` → user-level persistent memory
  - I explicitly define:
    - coding standards
    - boundaries (what AI should/shouldn't do)
  - I also let Claude **update its own rules** over time to improve consistency.
- 

## 2. Efficient Context Usage

- I avoid overloading context using:
    - `/compact` to reduce token usage
    - `/context` to inspect what's consuming tokens (e.g., MCP, skills, history)
  - I keep prompts **focused and minimal**, adding context only when needed.
  - Claude also performs **auto-compaction**, which helps maintain efficiency.
- 

## 3. Automation via MCP (Model Context Protocol)

- I use MCP servers to let AI interact with external systems/APIs.
  - Example:
    - **Context7 MCP server** → fetches relevant documentation
  - I explicitly instruct:
    - “use context7” when I need accurate docs
  - I'm mindful that MCP calls can be **token-heavy**, so I use them selectively.
- 

## 4. Agents & Tooling

- I create custom agents using `/agent`:
  - Assign specific tools (read/write files, APIs, etc.)
  - Scope their capabilities for safety and modularity

- Useful for:
    - code edits
    - debugging
    - workflow automation
  - Sometimes requires restarting Claude to activate new agents.
- 

## 5. Memory & Learning

- Claude supports **Auto-Memory (Automemory)**:
    - Learns coding patterns, corrections, and preferences
    - Stores them in `~/.claude/`
  - I also manually manage memory using `/memory` for better control.
- 

## 6. Safe Automation & Permissions

- I avoid unsafe commands like skipping permissions blindly.
  - Use:
    - `/permissions` to explicitly control access
    - `--no-ask` mode carefully to reduce interruptions while staying safe
  - I define **clear boundaries** in `claude.md` to prevent risky actions.
- 

## 7. Planning Before Execution

- I use `/plan` mode:
    - Claude analyzes the codebase
    - Suggests step-by-step changes **without modifying anything**
  - Helps in:
    - large refactors
    - architectural decisions
  - I approve the plan before execution.
- 

## 8. Reusable Workflows (Skills)

- I create **custom skills** for repetitive tasks:
    - e.g., fetching data, running scripts, reviewing code
  - This avoids rewriting prompts and improves consistency.
- 

## 9. Debugging & Verification

- I include:

- debug logs
  - verification commands (in something like `commands.md`)
  - Ensures generated code is **testable and reliable**.
- 

## 10. GitHub & Collaboration

- I integrate with GitHub:
    - Use `gh` CLI (`brew install gh`, `gh auth login`)
  - Use `@claude` in:
    - PR reviews
    - Issues → Claude can generate PRs to fix them
  - Improves productivity in team workflows.
- 

## 11. UI & Multimodal Work

- Use tools like:
    - **Figma MCP** for UI-related tasks
  - I can also:
    - drag & drop screenshots for context-aware help
- 

## 12. General Workflow Philosophy

- Keep prompts **clear, scoped, and intentional**
  - Treat AI as:
    - a **coding assistant + planner**, not just autocomplete
  - Focus on:
    - correctness
    - maintainability
    - iterative improvement
- 
-

DSA

## DSA Patterns:

<https://leetcode.com/discuss/post/5886397/dsa-patterns-you-need-to-know-by-anubhav-x7og/>

### 1. Fast and Slow Pointer

**Description:** This technique uses two pointers moving at different speeds to solve problems involving cycles, such as finding the middle of a list, detecting loops, or checking for palindromes.

- [Linked List Cycle II](#)
- [Remove nth Node from the End of List](#)
- [Find the Duplicate Number](#)
- [Palindrome Linked List](#)
- [\[Self Added v2\] Intersection of two Linkedlists](#)

---

### 2. Overlapping Intervals

**Description:** Intervals are often manipulated through sorting and merging based on their start and end times.

- [Basic Merge: Merge Intervals](#)
- [Interval Insertion: Insert Interval](#)
- [My Calendar ii](#)
- [Minimum Number of Arrows to Burst Balloons](#)
- [Non-overlapping Intervals](#)
- [\[Self Added v2\] Merge Sorted Array](#)
- [\[Self Added v2\] Meeting Rooms II](#)

---

### 3. Prefix Sum

**Description:** Prefix Sums/Products are techniques that store cumulative sums or products up to each index, allowing for quick subarray range queries.

- [Find the middle index in array](#)
- [Product of array except self](#)
- [Maximum product subarray](#)
- [Number of ways to split array](#)
- [Range Sum Query 2D](#)

---

## 4. Sliding Window

**Description:** A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Fixed Size

- [Maximum Sum Subarray of Size K](#)
- [Number of Subarrays having Average Greater or Equal to Threshold](#)
- [Repeated DNA sequences](#)
- [Permutation in String](#)
- [Sliding Subarray Beauty](#)
- [Sliding Window Maximum](#)

### Variable Size

- [Longest Substring Without Repeating Characters](#)
- [Minimum Size Subarray Sum](#)
- [Subarray Product Less Than K](#)
- [Max Consecutive Ones](#)
- [Fruits Into Baskets](#)
- [Count Number of Nice Subarrays](#)
- [Minimum Window Substring: Minimum Window Substring](#)

---

## 5. Two Pointers

**Description:** The two pointers technique involves having two different indices move through the input at different speeds to solve various array or linked list problems.

- [Two Sum II - Input Array is Sorted](#)
- [Dutch National Flag: Sort Colors](#)
- [Next Permutation](#)
- [Bag of Tokens](#)
- [Container with most water](#)
- [Trapping Rain Water](#)

---

## 6. Cyclic Sort (Index-Based)

**Description:** Cyclic sort is an efficient approach to solve problems where numbers are consecutively ordered and must be placed in the correct index.

- [Missing Number](#)
- [Set Mismatch](#)
- [Find Missing Numbers](#)
- [First Missing Positive](#)

---

## 7. Reversal & Manipulation of Linked List (In-place)

**Description:** Reversing a linked list in place without using extra space is key for problems that require in-place list manipulations.

- [Reverse Linked List](#)
- [Reverse Nodes in k-Group](#)
- [Swap Nodes in Pairs](#)
- [\[Self added v2\] Flatten LinkedList](#)

---

## 8. Matrix Manipulation

**Description:** Problems involving 2D arrays (matrices) are often solved using row-column traversal or manipulation based on matrix properties.

- [Rotate Image](#)
- [Spiral Matrix](#)
- [Set Matrix Zeroes](#)
- [Game of Life](#)

---

## 9. Breadth First Search (BFS)

**Description:** BFS explores nodes level by level using a queue. It is particularly useful for shortest path problems.

- [Shortest Path in Binary Matrix](#)
- [Rotten Oranges](#)
- [Word Ladder: Word Ladder](#)
- [As Far From Land as Possible](#)



---

## 10. Depth First Search (DFS)

**Description:** DFS explores as far as possible along a branch before backtracking. It's useful for graph traversal, pathfinding, and connected components.

- [Number of Closed Islands](#)
- [Coloring a Border](#)
- [DFS from boundary: Number of Enclaves](#)
- [Shortest time: Time Needed to Inform all Employees](#)
- [Cyclic Find: Find Eventual Safe States](#)

---

## 11. Backtracking

**Description:** Backtracking helps in problems where you need to explore all potential solutions, such as solving puzzles, generating combinations, or finding paths.

- [Permutation ii](#)
- [Combination Sum](#)
- [Generate Parenthesis](#)
- [N-Queens](#)
- [Sudoku Solver](#)
- [Palindrome Partitioning](#)
- [Word Search: Word Search](#)

---

## 12. Modified Binary Search

**Description:** A modified version of binary search that applies to rotated arrays, unsorted arrays, or specialized conditions.

- [Search in Rotated Sorted Array](#)
- [Find Minimum in Rotated Sorted Array](#)
- [Find Peak Element](#)
- [Single element in a sorted array](#)
- [Minimum Time to Arrive on Time](#)
- [Capacity to Ship Packages within 'd' Days](#)
- [Koko Eating Bananas](#)

- [Find in Mountain Array](#)
  - [Median of Two Sorted Arrays](#)
  - [\[Self added v2\] Magnetic force between two balls](#)
  - [\[Self added v2\] Split Array Largest Sum](#)
- 

### 13. Bitwise XOR

**Description:** XOR is a powerful bitwise operator that can solve problems like finding single numbers or efficiently pairing elements.

- [Single Number II](#)
  - [Single Number III](#)
  - [Find the Original array of Prefix XOR](#)
  - [XOR Queries of a Subarray](#)
- 

### 14. Top 'K' Elements

**Description:** This pattern uses heaps or quickselect to efficiently find the top 'K' largest/smallest elements from a dataset.

- [Top K Frequent Elements](#)
  - [Kth Largest Element](#)
  - [Ugly Number II](#)
  - [K Closest Points to Origin](#)
- 

### 15. K-way Merge

**Description:** The K-way merge technique uses a heap to efficiently merge multiple sorted lists or arrays.

- [Find K Pairs with Smallest Sums](#)
  - [Kth Smallest Element in a Sorted Matrix](#)
  - [Merge K Sorted Lists](#)
  - [Smallest Range: Smallest Range Covering Elements from K Lists](#)
- 

### 16. Two Heaps

**Description:** This pattern uses two heaps (max heap and min heap) to solve problems involving tracking medians and efficiently managing dynamic data.

- [Find Median from Data Stream](#)
- [Sliding Window Median](#)
- [IPO](#)

---

## 17. Monotonic Stack

**Description:** A monotonic stack helps solve range queries by maintaining a stack of elements in increasing or decreasing order.

- [\[Self Added\] Next Greater Element I](#)
- [Next Greater Element II](#)
- [Next Greater Node in Linked List](#)
- [Daily Temperatures](#)
- [Online Stock Span](#)
- [Maximum Width Ramp](#)
- [Largest Rectangle in Histogram](#)
- [\[Self added v2\] Find Celebrity](#)

---

## 18. Trees

Level Order Traversal (BFS in Binary Tree)

- [Level order Traversal](#)
- [Zigzag Level order Traversal](#)
- [Even Odd Tree](#)
- [Reverse odd Levels](#)
- [Deepest Leaves Sum](#)
- [Add one row to Tree](#)
- [Maximum width of Binary Tree](#)
- [All Nodes Distance K in Binary tree](#)

[Self added] Tree Traversals

- [\[Self added v2\] Left, Right, Top View of Binary Tree](#)

## Tree Construction

- [Construct BT from Preorder and Inorder](#)
- [Construct BT from Postorder and Inorder](#)
- [Maximum Binary Tree](#)
- [Construct BST from Preorder](#)

## Height related Problems

- [Maximum Depth of BT](#)
- [Balanced Binary Tree](#)
- [Diameter of Binary Tree](#)
- [Minimum Depth of BT](#)

## Root to leaf path problems

- [Binary Tree Paths](#)
- [Path Sum ii](#)
- [Sum Root to Leaf numbers](#)
- [Smallest string starting from Leaf](#)
- [Insufficient nodes in root to Leaf](#)
- [Pseudo-Palindromic Paths in a Binary Tree](#)
- [Binary Tree Maximum Path Sum](#)

## Ancestor problem

- [LCA of Binary Tree](#)
- [Maximum difference between node and ancestor](#)
- [LCA of deepest leaves](#)
- [Kth Ancestor of a Tree Node](#)

## Binary Search Tree

- [Validate BST](#)
- [Range Sum of BST](#)
- [Minimum Absolute Difference in BST](#)
- [Insert into a BST](#)
- [LCA of BST](#)

### Take / Not take (DP)

**Description:** Solve optimization problems like selecting items with the max/min value under certain constraints.

- [\[Self Added\] House Robber i](#)
- [House Robber ii](#)
- [Target Sum](#)
- [Partition Equal Subset Sum](#)
- [Ones and Zeroes](#)
- [Last Stone Weight ii](#)

### Infinite Supply (DP)

**Description:** Similar to the 0/1 knapsack, but items can be chosen multiple times.

- [Coin Change](#)
- [Coin Change II](#)
- [Perfect Squares](#)
- [Minimum Cost For Tickets](#)
- [\[Self added v2\] Rod Cutting Problem](#)

### Longest Increasing subsequence

**Description:** It involves finding the longest subsequence of a given sequence where the elements are in ascending order

- [Longest Increasing Subsequence](#)
- [Largest Divisible Subset](#)
- [Maximum Length of Pair Chain](#)
- [Number of LIS](#)
- [Longest String Chain](#)

### DP on Grids

**Description:** Dynamic Programming on matrices involves solving problems that can be broken down into smaller overlapping subproblems within a matrix.

- [\[Self Added\] Unique Paths i](#)
- [Unique Paths ii](#)
- [Minimum Path Sum](#)

- [Triangle](#)
- [Minimum Falling Path Sum](#)
- [Maximal Square](#)
- [Cherry Pickup](#)
- [Dungeon Game: Dungeon Game](#)

## DP on Strings

**Description:** It Involves 2 strings, whenever you are considering two substrings/subsequence from given two strings, concentrate on what happens when the last characters of the two substrings are same, i.e, matching.

- [Longest Common Subsequence](#)
- [Longest Palindromic Subsequence](#)
- [Palindromic Substrings](#)
- [Longest Palindromic Substrings](#)
- [Edit Distance](#)
- [Minimum ASCII Delete Sum for Two Strings](#)
- [Distinct Subsequences](#)
- [Shortest Common Supersequence](#)
- [Wildcard Matching](#)

## DP on Stocks

**Description:** It focuses on maximizing profit from buying and selling stocks over time while considering constraints.

- [\[Self Added\] Buy and Sell Stocks i](#)
- [Buy and Sell Stocks ii](#)
- [Buy and Sell Stocks iii](#)
- [Buy and Sell Stocks iv](#)
- [Buy and Sell Stocks with Cooldown](#)
- [Buy and Sell Stocks with Transaction fee](#)

## Partition DP (MCM)

**Description:** It Involves a sequence that needs to be divided into partitions in an optimal way. The goal is often to minimize or maximize a cost function, such as computation time, multiplications, or some other metric, by exploring all possible partitions and combining results from subproblems.

- [Partition array for Maximum Sum](#)
  - [Minimum Cost to Cut a Stick](#)
  - [Burst Balloons](#)
  - [Palindrome Partitioning ii](#)
  - [\[Self added v2\] Egg Dropping](#)
- 

## 20. Graphs

[Self added] Basic Questions

- [\[Self added v2\] Clone graph](#)
- [\[Self added v2\] Number of islands](#)

Topological Sort

**Description:** Topological sorting is useful for tasks that require dependency resolution (InDegree) in directed acyclic graphs (DAGs).

- [Course Schedule](#)
- [Course Schedule II](#)
- [Strange Printer ii](#)
- [Sequence Reconstruction](#)
- [Alien Dictionary](#)
- [\[Self added v2\] Cycle detection in Directed & Undirected graph](#)

Union Find (Disjoint Set)

**Description:** Union-Find (or Disjoint Set) is used to solve problems involving connectivity or grouping, often in graphs.

- [Number of Operations to Make Network Connected](#)
- [Redundant Connection](#)
- [Accounts Merge](#)
- [Satisfiability of Equality Equations](#)

Graph Algorithms

**Description:** Advanced graph algorithms are used to solve complex problems involving shortest paths, minimum spanning trees, and graph cycles.

- [Kruskal's Algorithm: Minimum Cost to connect all Points](#)
- [Dijkstra's Algorithm: Cheapest Flights Within K Stops](#)
- [Floyd-Warshall: Find the City with Smallest Number of Neighbours at a Threshold Distance](#)
- [Bellman Ford: Network Delay time](#)

---

## 21. Greedy

**Description:** Greedy algorithms make local optimal choices at each step, which lead to a global optimal solution for problems like scheduling and resource allocation.

- [Jump Game ii](#)
- [Gas Station](#)
- [Bag of Tokens](#)
- [Boats to Save People](#)
- [Wiggle Subsequence](#)
- [Car Pooling](#)
- [Candy](#)

---

## 22. Design Data Structure

**Description:** It involves building custom data structures to efficiently handle specific operations, like managing data access, updates, and memory usage. Focusing on optimizing performance and resource management.

- [Design Twitter](#)
- [Design Browser History](#)
- [Design Circular Deque](#)
- [Snapshot Array](#)
- [LRU Cache](#)
- [LFU Cache](#)

---

Some Useful Articles on LeetCode for Better Understanding!



## Two Pointers

- [Solved all Two Pointers problems in 100 days](#)

## Sliding Window

- [Sliding Window Technique and Question Bank](#)
- [C++ Maximum Sliding Window Cheatsheet Template!](#)

## Greedy

- [Greedy for Beginners: Problems & Sample Solutions](#)
- [Top Greedy Questions](#)

## Linked List

- [Become Master In Linked List](#)
- [Must-Do LinkedList Problems on LeetCode](#)

## Trees

- [Tree Question Pattern | 2021 Placement](#)
- [Master Tree Patterns](#)

## Binary Search

- [5 Variations of Binary Search](#)
- [Binary Search for Beginners: Problems & Patterns](#)

## Dynamic Programming (DP)

- [Dynamic Programming Patterns](#)
- [DP for Beginners: Problems & Patterns](#)

## Graphs

- [Graph For Beginners](#)
- [Become Master In Graph](#)
- [Graph algorithms + problems to practice](#)

## Bit Manipulation

- [Bit Manipulation Problem solving](#)
- [All Types of Patterns for Bits Manipulations and How to use it](#)

\_\_Patterns1

-----  
DS

**Vector** - pb, pop\_back, insert/erase(v.begin()+i, x), size(), empty, front->index0, back->indexN-1, sort/reverse-vBegin|vEnd.., clear

**String** - pb/pop\_back, insert/erase/replace, s.find("x")  
sort/reverse, stoi(x)/to\_string(x)

**Set** - insert(x)/ erase, find -> .end() to compare, size, empty, count(x) -> 1 or 0

**Map** - insert/ erase, find -> .end(), size, empty

**Queue** - push(x), pop, front, back, empty, size

**Stack** - push(x), pop, top, empty, size

**Heap** - priority\_queue<int> pq / priority\_queue<int, vector<int>, greater<int>>, push, pop, top, empty, size

**Deque** (double ended) - push-pop\_front-back, erase, empty, size

**List** - similar functions as deque + list1.splice(pos, list2, first, last) - moves range  
-----

## Pointers

| Context                             | Meaning                       |
|-------------------------------------|-------------------------------|
| Declaration ( <code>int* p</code> ) | "p is a pointer"              |
| Usage ( <code>*p</code> )           | "go to address and get value" |

(`*head`).next OR using `head->next`; Both are exactly the same.  
-----

## General bfs

Shortest path in matrix

queue<pair<int,int>> q, push 0,0 starting node, have a dirs[8][2] for directions, while() then for(), pair<int,int> p check in while loop, dist=grid[nr][nc] store the distance in cell - which you use in for loop for 8 directions to push new pair in queue if condition met and store the dist+1 value in the grid cell.

Rotten oranges

Same queue pair, increment fresh++ else store the rotten oranges, then while() then while(size--) here i also do minutes++, then for() in 4 dirs

```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int,int>> q; int fresh = 0;
    for (int i=0;i<m;i++) {
        for (int j=0;j<n;j++) {
            if (grid[i][j] == 2) q.push(make_pair(i,j));
            else if (grid[i][j] == 1) fresh++;
        }
    }
    if (fresh == 0) return 0;
    int minutes = -1;
    int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!q.empty()) {
        int size = q.size();
        minutes++;
        while (size--) {
            pair<int,int> p = q.front();
            q.pop();
            int r = p.first, c = p.second;
            for (int i=0;i<4;i++) {
                int nr = r + dirs[i][0];
                int nc = c + dirs[i][1];
                if (nr>=0&&nr<m&&nc>=0&&nc<n&&grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    q.push(make_pair(nr,nc));
                }
            }
        }
    }
    return (fresh == 0 ? minutes : -1);
}
```

Word ladder

Use set string to store the wordlist begin/end, queue string int pair, string has the word, int has the counter, while() then double for() -> to iterate over char 'a' to 'z' char++, if the word i get from queue front(), match then erase() from set and push in queue<tempword, steps+1>.

Far from land

Same story, store pairs of i,j in queue where value in grid=1, then while() dist++ -> Inner while() -> for () for directions, and making pairs if(nr range [0,m) and nc range [0,n) and gridij==0...

-----

General dfs

Number of closed islands

for()->for() to iterate over grid, and if gridij==0 then we do a dfs - if dfs true then ++res else skip, coming in dfs, i do bool check in all 4 directions dfs(grid, i+-1, j+-1) and d1&&d2&&d3&&d4, base conditions are if i,j<0 or i,j>grid.size, then return false, or if gridij visited with 2 then false, and for every visit i modify cell value of grid to 2.

```
int closedIsland(vector<vector<int>>& grid) {
    int res = 0;
    for (int i = 0; i < grid.size(); i++){
        for (int j = 0; j < grid[0].size(); j++){
            if (grid[i][j] == 0){
                res += dfs(grid, i, j) ? 1 : 0;
            }
        }
    }
    return res;
}

bool dfs(vector<vector<int>>& g, int i, int j){
    if (i < 0 || j < 0 || i >= g.size() || j >= g[0].size()){
        return false;
    }
    if (g[i][j] == 1 || g[i][j] == 2){
        return true;
    }
    g[i][j] = 2;
```

```

    bool down = dfs(g, i+1, j);
    bool right = dfs(g, i, j+1);
    bool up = dfs(g, i-1, j);
    bool left = dfs(g, i, j-1);
    return up && down && left && right;
}

```

Coloring border - skip

Number of enclaves

Same like above, let dfs keep coloring the graph, in main(), call dfs for boundaries first to color them -> j[0, m) for dfs(0,j,grid) & dfs(n-1, j, grid) later i[0,n) for dfs(i,0,grid) & dfs(i,m-1,grid), later for()->for() and if gridij==1 ans++.

Time needed to inform employees

Create the graph from [manager[i],i], return dfs which will iterate over adj in for() to get maxtime=max(maxtime, dfs()), and return informtime[u] + maxtime, maxtime is initialized as 0 before the for().

Find eventual safe states

Same as finding cycle in graph, if(!vis[i]) call dfs, in dfs, if no cycle then safenode[node]=1, which will be ans.

-----

Design DS

Twitter - skip

Browser history - skip

Circular Deque - skip

Snapshot array - skip

LFU cache - skip

LRU cache

Have list<pii> dq, u\_mp<int, pii:iterator> mp, set cap in constructor, getFun -> check key in mp[key]->second & move dq.splice(dq.begin(), dq, mp[key]), a condition for mp.end() -1, putFun -> first same logic to getFun to move keys, else if dq.size()==cap then mp.erase(dq.back().first) and dq.pop\_back(), else normal dq.push\_front({key,val}) & mp[key]=dq.begin()

-----

Tree-LevelOrderTraversal

Level order traversal

[while(!q.empty())->for(q.size)] queue<TreeNode\*> q, push root, while(!q.empty()) - run for loop based on q size, q.front()->pop(), inside loop, push x->left/right in queue and x->val in temp\_ans, outside for() push temp\_ans in ans

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> ans; queue<TreeNode*> q;
    if(root==NULL) return ans;
    q.push(root);
    while(!q.empty()){
        int size = q.size(); vector<int> l;
        for(int i=0; i<size; i++){
            TreeNode* x = q.front();
            q.pop();
            l.push_back(x->val);
            if(x->left) q.push(x->left);
            if(x->right) q.push(x->right);
        }
        ans.push_back(l);
    }
    return ans;
}
```

Zigzag level order traversal

[while(!q.empty())->for(q.size)] Same as above, keep bool direction flag after for loop, l[i]=x->val else l[size-i-1]=x->val

Even-odd tree

[while(!q.empty())->for(q.size)] Same story, just v=cur->val, if level%2 even -> if v%2 even|| v<=prev then false, same for odd,

false cases take, and then `prev=v`, if `curr->left/right` then push in queue - all inside for loop, `level++` after for loop

Reverse odd levels

[`while(!q.empty())->for(q.size)`] Same story, after `for()` if(`level%2 odd`), then `l=0,r=ans.size()-1`, while(`l<r`)  
`swap(ans[l]->value,ans[r]->value)` `l++,r--` to swap at odd levels.  
**Swapping in extreme ends, reverses things**

Deepest leaves sum

[`while(!q.empty())->for(q.size)`] Same story, just initialize sum before `while()` and let `sum+=cur->val` in `for()`, in last level, it will anyways reset to 0 and store last level sum

Add one row tree

[`while(!q.empty())->for(q.size)`] Same story, only when `d==depth-1` (`depth++` after `for()`), `cur=q.front`, `l` is `cur->left`/`r` is `right` temporarily, new `cur->left/right=new Node(val)` & `cur->left->left=l` & `right->right=r`

Max width of binary tree

[`while(!q.empty())->for(q.size)`], here `queue<pair<Node*, int>>`, before `while()` get `leftInd = q.front().second`, then in `for()` get the `pair<Node*,int> p.first/second`, make `rightInd` or `tempInd=p.second`, if(`cur->left/right`) push {`cur->left`, `tempInd*2` or `(tempInd*2)+1` depending on node left/right child}, after `for()`  
`ans=max(ans,rightInd-leftInd+1)`

All nodes distance K in BT

Have `unordered_map<Node*,Node*> parent`, `unordered_set<Node*> vis`, build parent map in separate `fun()` will do level order traversal in single `while()` no inner `for()` using `queue<Node*>` and insert `parent[curr->left/right]=curr`, coming back, new `queue<Node*>`, push node there and even in `vis` set to mark visited, then `while()` has a `dist` variable to mark distance, if `dist==k` then break then and `dist++` -> nested `for()` -> inside have `arr[3]=curr->left/right`, `parent[cur]`, 3 nodes, and another nested `for(j,3)->` take the `Node*` in `arr` in `nxt` var, if(`nxt && !vis.count(nxt)`) means `nxt` exists and



its not visited, then q.push(nxt) & vis.insert(nxt). So loop runs, checks current dist breaks if it has reached k, and below code then iterates via BFS

-----

Tree traversals

Left view

Dfs call to cur->left then right, level+1 for both, and pass &maxLevel by reference, if level>maxLevel, [ans.pb\(data\)](#), maxLevel=level

Right view

Same as above, sequence of dfs call would be right, then left

Top view

All rests on this: if (mp.count(dist)==0 || mp[dist].second>level) then mp[dist] = {curr->data, level}, send level+1,dist+-1 to recursive calls

```
void topViewDFS(Node* curr, int dist, int level,
                map<int, pair<int,int>> &mp) {
    if (curr == NULL) return;
    if (mp.count(dist) == 0 || mp[dist].second > level) {
        mp[dist] = {curr->data, level};
    }
    topViewDFS(curr->left, dist - 1, level + 1, mp);
    topViewDFS(curr->right, dist + 1, level + 1, mp);
}
```

-----

Tree construction

Construct BT from Preorder and Inorder - skip

Construct BT from Postorder and Inorder - skip

Construct BST from Preorder - skip

Maximum Binary Tree

Get l,r (0,size), find index of max nums from l to r and using value in that create tree, in recursive calls call from l to index-1 & index+1 to r

```

TreeNode* build(vector<int>& nums, int l, int r) {
    if(l > r) return NULL;
    int idx = l;
    for(int i = l; i <= r; i++)
        if(nums[i] > nums[idx]) idx = i;
    TreeNode* root = new TreeNode(nums[idx]);
    root->left = build(nums, l, idx - 1);
    root->right = build(nums, idx + 1, r);
    return root;
}

```

-----  
Tree height: Return-Based DFS (Bottom-Up), Parent needs computed value from children

Maximum Depth of BT

root==null - return 0, lh=fun(node->l), rh=fun(node->r), return 1+max(lh,rh)

Balanced Binary Tree

Similar as above, extra condition - if(abs(lh - rh) > 1) return -1

Diameter of Binary Tree

Same as depth of BT, just a global ans, ans = max(ans, lh + rh);

Minimum Depth of BT

Similar to max depth of BT, but return 1 + min(lh, rh) and also, if tree one half not available handle cases, i.e add before return  
if(root->left == NULL) return 1 + right - as we want min height, if 1 half null then will give null height which is not true

Binary Tree Maximum Path Sum

Same as max depth of BT, just that, left=max(0, dfs(root->left), similarly right, and then global ans stores max() of ans & root->val+left+right, return root->val + max(left, right);

LCA of Binary Tree

Same story for left/right, base condition - if (!root || root == p || root == q) return root, if left&&right return root, else return left?left:right

Maximum difference between node and ancestor

Same left,right story, but base condition - if (!node) return mx - mn, where mn is min & mx is max in each iteration, return max(left,right)

```
int dfs(TreeNode* node, int mn, int mx) {
    if (!node) return mx - mn;
    mn = min(mn, node->val);
    mx = max(mx, node->val);
    int left = dfs(node->left, mn, mx);
    int right = dfs(node->right, mn, mx);
    return max(left, right);
}
int maxAncestorDiff(TreeNode* root) {
    return dfs(root, root->val, root->val);
}
```

LCA of deepest leaves - skip

-----

Tree root: Leaf path problems

Section: Backtracking DFS (Top-Down, push-recursive\_call-pop)

Binary Tree Paths

Simple recursive call to left/right node, before them, if path str empty then push node->value as to\_string, else add '-' and then value, only case is if both left&right null then push\_back path you constructed into the ans string vector

Path Sum ii

Similar like above, but here apart from push back we also pop back if sum not matches after 2 recursive calls, if sum matches then push in another res vector

```
vector<vector<int>> res;
vector<int> curr;
vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
    dfs(root, targetSum);
    return res;
}
void dfs(TreeNode* root, int sum) {
```

```

    if(root == NULL) return;
    curr.push_back(root->val);
    if(root->left==NULL && root->right==NULL && sum==root->val)
        res.push_back(curr);
    dfs(root->left, sum - root->val);
    dfs(root->right, sum - root->val);
    curr.pop_back();
}

```

Smallest string starting from Leaf

Same as binary tree paths, just that, `char c = 'a' + root->val` (to increase the chars, like `'a'+2 = 'c'`), `curr = c + curr` to prepend value, then same if `node->left/right` null and `ans=min(ans,curr)`, `ans` could initially be `'~'` as its lexicographically high

Insufficient nodes in root to Leaf

```

sum += root->val;
// store original leaf status
bool isLeaf = (!root->left && !root->right);
root->left = dfs(root->left, sum, limit);
root->right = dfs(root->right, sum, limit);
if (isLeaf) {
    return (sum >= limit) ? root : NULL;
}
// internal node: keep only if at least one child survives
return (root->left || root->right) ? root : NULL;

```

Pseudo-Palindromic Paths in a Binary Tree

Section: DFS with State Passing (No pop needed)

Sum Root to Leaf numbers

-----

Kth Ancestor of a Tree Node

-----

## Validate BST

## Range Sum of BST

## Minimum Absolute Difference in BST

## Insert into a BST

## LCA of BST

This image shows a full page of white paper with horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

\_DS-Cpp-SyntaxBasics

## Basic STL DS

---

**Vector:** Dynamic array – contiguous memory, supports random access.

| Operation / Usage   | Syntax                                        | Complexity       | Notes                       |
|---------------------|-----------------------------------------------|------------------|-----------------------------|
| Empty               | <code>vector&lt;int&gt; v;</code>             | $O(1)$           | Empty vector                |
| Fixed size          | <code>vector&lt;int&gt; v(5);</code>          | $O(n)$           | 5 elements initialized to 0 |
| Fixed size + value  | <code>vector&lt;int&gt; v(5, 1);</code>       | $O(n)$           | {1,1,1,1,1}                 |
| From array          | <code>vector&lt;int&gt; v(arr, arr+n);</code> | $O(n)$           | Copies elements             |
| Copy another vector | <code>vector&lt;int&gt; v2(v1);</code>        | $O(n)$           |                             |
| Add element         | <code>v.push_back(x);</code>                  | Amortized $O(1)$ | Append                      |
| Remove last         | <code>v.pop_back();</code>                    | $O(1)$           | Remove last                 |
| Insert              | <code>v.insert(v.begin()+i, x);</code>        | $O(n)$           | Shift elements              |
| Erase               | <code>v.erase(v.begin()+i);</code>            | $O(n)$           | Shift elements              |
| Erase range         | <code>v.erase(v.begin(), v.begin()+k);</code> | $O(n)$           | Remove range                |
| Access              | <code>v[i] / v.at(i)</code>                   | $O(1)$           | Random access               |
| Size                | <code>v.size();</code>                        | $O(1)$           | Count                       |
| Empty               | <code>v.empty();</code>                       | $O(1)$           | Check empty                 |

|              |                                           |               |                      |
|--------------|-------------------------------------------|---------------|----------------------|
| Front / Back | <code>v.front(); v.back();</code>         | $O(1)$        | Ends                 |
| Iterate      | <code>for (auto &amp;x : v)</code>        | $O(n)$        | Range-based          |
| Sort         | <code>sort(v.begin(), v.end());</code>    | $O(n \log n)$ | Needs<br><algorithm> |
| Reverse      | <code>reverse(v.begin(), v.end());</code> | $O(n)$        |                      |
| Clear        | <code>v.clear();</code>                   | $O(n)$        | Remove all           |

**Note:**

- 2D Vector: `vector<vector<int>> v;`
  - `vector<vector<int>> v(n, vector<int>(m, 0));`
- 1D Pair Vector: `vector<pair<int,int>> vp;`
  - `vp.push_back({x, y});`
  - `vp[i].first, vp[i].second`

-----

**Set:** Ordered, unique elements – balanced BST (red-black tree). For `std::map` and `std::set`:

- Insertion =  $O(\log n)$ , NOT  $O(n \log n)$
- Because each insertion involves finding one place to insert (using a balanced tree lookup) – not re-sorting or rebalancing all elements.

**Unordered\_set:** Hash set – unique, unordered.

| Operation / Usage | Syntax                                   | Complexity    | Notes              |
|-------------------|------------------------------------------|---------------|--------------------|
| Empty             | <code>set&lt;int&gt; s;</code>           | $O(1)$        |                    |
| Init list         | <code>set&lt;int&gt; s = {1,2,3};</code> | $O(n \log n)$ |                    |
| Insert            | <code>s.insert(x);</code>                | $O(\log n)$   | Ignores duplicates |



|             |                   |          |              |
|-------------|-------------------|----------|--------------|
| Erase value | s.erase(x);       | O(log n) |              |
| Find        | s.find(x);        | O(log n) | Iterator     |
| Count       | s.count(x);       | O(log n) | 1 or 0       |
| Size        | s.size();         | O(1)     |              |
| Empty       | s.empty();        | O(1)     |              |
| Iterate     | for (auto &x : s) | O(n)     | Sorted order |
| Lower bound | s.lower_bound(x); | O(log n) | $\geq x$     |
| Upper bound | s.upper_bound(x); | O(log n) | $> x$        |
| Clear       | s.clear();        | O(n)     |              |

**Note:**

- Pair Set: set<pair<int,int>> s;
  - s.insert({x,y}), s.find({x,y});
  - unordered\_set<pair<int,int>> ❌ not directly supported
- To find in set:

```
auto it = s.find("abc");
if (it != s.end()) {
    cout << "Found: " << *it;
}
```

-----

**Map: Sorted key-value pairs (unique keys, ordered by key).**

**unordered\_map: Hash map – key-value pairs, no order.**

| Operation     | Syntax                            | Complexity | Notes         |
|---------------|-----------------------------------|------------|---------------|
| Create        | map<int,int> mp;                  | O(1)       |               |
| Init list     | map<int,int> mp = {{1,2}, {3,4}}; | O(n log n) |               |
| Insert/update | mp[key] = val;                    | O(log n)   | Add or update |

|             |                                                                                           |             |              |
|-------------|-------------------------------------------------------------------------------------------|-------------|--------------|
| Insert pair | <code>mp.insert({k, v});</code>                                                           | $O(\log n)$ |              |
| Find        | <code>mp.find(key);</code>                                                                | $O(\log n)$ | Iterator     |
| Erase       | <code>mp.erase(key);</code>                                                               | $O(\log n)$ |              |
| Size        | <code>mp.size();</code>                                                                   | $O(1)$      |              |
| Empty       | <code>mp.empty();</code>                                                                  | $O(1)$      |              |
| Iterate     | <code>for (auto &amp;p : mp) cout &lt;&lt; p.first &lt;&lt; " " &lt;&lt; p.second;</code> | $O(n)$      | Sorted order |

**Note:**

```

- Using pair:
  map<int,int> = OK. map<int, pair<int,int>> mp = OK.
- To search:
  auto it = mp.find(key);
  if (it != mp.end()) {
      cout << it->second << endl; // Safe access
  }

```

-----

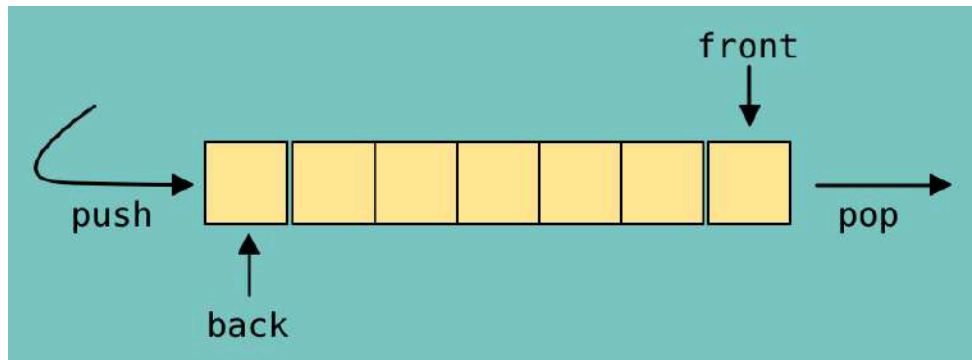
**Queue: FIFO – first in, first out.**

| Operation | Syntax                           | Complexity | Notes        |
|-----------|----------------------------------|------------|--------------|
| Create    | <code>queue&lt;int&gt; q;</code> | $O(1)$     |              |
| Push      | <code>q.push(x);</code>          | $O(1)$     | Enqueue      |
| Pop       | <code>q.pop();</code>            | $O(1)$     | Dequeue      |
| Front     | <code>q.front();</code>          | $O(1)$     | Peek         |
| Back      | <code>q.back();</code>           | $O(1)$     | Last element |

|       |                         |        |  |
|-------|-------------------------|--------|--|
| Empty | <code>q.empty();</code> | $O(1)$ |  |
| Size  | <code>q.size();</code>  | $O(1)$ |  |

**Note:**

- `queue<pair<int,int>> q; q.push({x,y});`



- Also - Queue pops from the front element, vector pops\_back from the back.

-----

**Stack: LIFO – last in, first out.**

| Operation | Syntax                            | Complexity | Notes |
|-----------|-----------------------------------|------------|-------|
| Create    | <code>stack&lt;int&gt; st;</code> | $O(1)$     |       |
| Push      | <code>st.push(x);</code>          | $O(1)$     |       |
| Pop       | <code>st.pop();</code>            | $O(1)$     |       |
| Top       | <code>st.top();</code>            | $O(1)$     | Peek  |
| Empty     | <code>st.empty();</code>          | $O(1)$     |       |
| Size      | <code>st.size();</code>           | $O(1)$     |       |

**Note:**

- `stack<pair<int,int>> st; st.push({x,y});`
-

**Priority\_queue: Max-heap by default.**

| Operation | Syntax                                                                            | Complexity  | Notes                 |
|-----------|-----------------------------------------------------------------------------------|-------------|-----------------------|
| Create    | <code>priority_queue&lt;int&gt; pq;</code>                                        | $O(1)$      | Max-heap              |
| Min-heap  | <code>priority_queue&lt;int, vector&lt;int&gt;, greater&lt;int&gt;&gt; pq;</code> | $O(1)$      | Min-heap              |
| Push      | <code>pq.push(x);</code>                                                          | $O(\log n)$ |                       |
| Pop       | <code>pq.pop();</code>                                                            | $O(\log n)$ |                       |
| Top       | <code>pq.top();</code>                                                            | $O(1)$      | Peek largest/smallest |
| Size      | <code>pq.size();</code>                                                           | $O(1)$      |                       |
| Empty     | <code>pq.empty();</code>                                                          | $O(1)$      |                       |

**Note:**

- Pair Heaps:
  - **MaxHeap:** `priority_queue<pair<int,int>> pq`
  - **MinHeap:** `priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq`

-----

**Deque: Double-ended queue – push/pop from both ends.**

| Operation       | Syntax                                                          | Complexity | Notes       |
|-----------------|-----------------------------------------------------------------|------------|-------------|
| Create          | <code>deque&lt;int&gt; dq;</code>                               | $O(1)$     |             |
| Init            | <code>deque&lt;int&gt; dq(5, 1);</code>                         | $O(n)$     | {1,1,1,1,1} |
| Push front/back | <code>dq.push_front(x);</code><br><code>dq.push_back(x);</code> | $O(1)$     |             |
| Pop front/back  | <code>dq.pop_front();</code><br><code>dq.pop_back();</code>     | $O(1)$     |             |

|         |                                                |        |  |
|---------|------------------------------------------------|--------|--|
| Erase   | <code>dq.erase(dq.begin()+i)</code>            | $O(n)$ |  |
| Access  | <code>dq[i]; dq.front();<br/>dq.back();</code> | $O(1)$ |  |
| Size    | <code>dq.size();</code>                        | $O(1)$ |  |
| Empty   | <code>dq.empty();</code>                       | $O(1)$ |  |
| Iterate | <code>for (auto &amp;x : dq)</code>            | $O(n)$ |  |

**Note:**

- `deque<pair<int,int>> dq; dq.push_back({x,y});  
dq.push_front({x,y});`

-----

**pair & tuple: Used to store multiple values together.**

- `map<int,int>` and `pair<int,int>` DO share the same access syntax: `.first` and `.second`. But they are not the same DS.

```
pair<int, string> p = {1, "one"};
cout << p.first << " " << p.second;

tuple<int, string, double> t = {1, "A", 3.14};
int a; string b; double c;
tie(a, b, c) = t;
```

-----

**Common Operations Across STL Containers**

| Operation             | Syntax Example                                                      | Used In                     | Meaning           |
|-----------------------|---------------------------------------------------------------------|-----------------------------|-------------------|
| <code>.push(x)</code> | <code>st.push(x); q.push(x);<br/>pq.push(x); v.push_back(x);</code> | stack, queue,<br>pq, vector | Add<br>element    |
| <code>.pop()</code>   | <code>st.pop(); q.pop(); pq.pop();<br/>v.pop_back();</code>         | stack, queue,<br>pq, vector | Remove<br>element |

|                                |                                                |                              |                |
|--------------------------------|------------------------------------------------|------------------------------|----------------|
| <code>.top()</code>            | <code>st.top(); pq.top();</code>               | stack, priority_queue        | Peek top       |
| <code>.front()</code>          | <code>q.front(); dq.front(); v.front();</code> | queue, deque, vector         | Peek first     |
| <code>.back()</code>           | <code>v.back(); dq.back();</code>              | vector, deque                | Peek last      |
| <code>.size()</code>           | <code>v.size(); s.size(); mp.size();</code>    | All                          | Count elements |
| <code>.empty()</code>          | <code>v.empty(); q.empty();</code>             | All                          | Check if empty |
| <code>.clear()</code>          | <code>v.clear(); s.clear(); mp.clear();</code> | Most                         | Remove all     |
| <code>.begin() / .end()</code> | <code>v.begin(); s.end();</code>               | All iterable                 | Iteration      |
| <code>.find(x)</code>          | <code>s.find(x); mp.find(key);</code>          | set, map, unordered variants | Search element |
| <code>.insert(x)</code>        | <code>s.insert(x); v.insert(it,x);</code>      | Many                         | Insert element |
| <code>.erase(x)</code>         | <code>s.erase(x); mp.erase(key);</code>        | Many                         | Remove element |

-----

### Quick Big-O Summary

| Container | Insert           | Erase  | Find   | Access | Ordered | Notes              |
|-----------|------------------|--------|--------|--------|---------|--------------------|
| vector    | Amortized $O(1)$ | $O(n)$ | $O(n)$ | $O(1)$ | ✗       | Fast random access |

|                |             |             |             |        |      |                  |
|----------------|-------------|-------------|-------------|--------|------|------------------|
| set            | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | –      | ✓    | Sorted unique    |
| unordered_set  | $O(1)$      | $O(1)$      | $O(1)$      | –      | ✗    | Hash-based       |
| map            | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | –      | ✓    | Key-value sorted |
| unordered_map  | $O(1)$      | $O(1)$      | $O(1)$      | –      | ✗    | Key-value hash   |
| queue          | $O(1)$      | $O(1)$      | –           | –      | FIFO |                  |
| stack          | $O(1)$      | $O(1)$      | –           | –      | LIFO |                  |
| priority_queue | $O(\log n)$ | $O(\log n)$ | –           | $O(1)$ | Heap | Max/min element  |

-----

## String

| Operation    | Syntax                                      | Complexity       | Notes                          |
|--------------|---------------------------------------------|------------------|--------------------------------|
| From literal | <code>string s="hello";</code>              | $O(n)$           | Copy from literal              |
| Repeat char  | <code>string s(5, 'a');</code>              | $O(n)$           | "aaaaa"                        |
| Concatenate  | <code>s1+s2;</code> or <code>s1+=s2;</code> | $O(n)$           | Creates new string             |
| Append       | <code>s.append("abc");</code>               | $O(k)$           | Append another string          |
| Push back    | <code>s.push_back('x');</code>              | Amortized $O(1)$ | Add char at end                |
| Pop back     | <code>s.pop_back();</code>                  | $O(1)$           | Remove last char               |
| Front/Back   | <code>s.front()/s.back();</code>            | $O(1)$           | Eg: In 'hello';<br>front: 'h', |

|                        |                                 |          |                           |
|------------------------|---------------------------------|----------|---------------------------|
|                        |                                 |          | back: 'o'.                |
| Access                 | s[i] or s.at(i)                 | O(1)     | Random access             |
| Size                   | s.size(); /<br>s.length();      | O(1)     | Same function             |
| Substring              | s.substr(pos,<br>len);          | O(len)   | Returns substring         |
| Find substring         | s.find("abc");                  | O(n)     | Returns index or<br>npos. |
| Replace                | s.replace(pos,<br>len, "xyz");  | O(n)     | Modify in place           |
| Insert                 | s.insert(pos,<br>"abc");        | O(n)     | Insert substring          |
| Erase                  | s.erase(pos, len);              | O(n)     | Remove substring          |
| Clear                  | s.clear();                      | O(n)     | Empty string              |
| Compare                | s1==s2, s1<s2                   | O(n)     | Lexicographic             |
| Iterate                | for(auto &c:s)                  | O(n)     | Char-wise                 |
| Convert → int          | stoi(s)                         | O(n)     | Throws if invalid         |
| Convert → long<br>long | stoll(s)                        | O(n)     |                           |
| Convert →<br>double    | stod(s)                         | O(n)     |                           |
| Convert ← int          | to_string(x)                    | O(n)     |                           |
| Reverse                | reverse(s.begin(),<br>s.end()); | O(n)     | From <algorithm>          |
| Sort                   | sort(s.begin(),<br>s.end());    | O(nlogn) | Alphabetical order        |

**Note:**



```
- To search string: Eg:  
auto pos = s.find("abc"); //OK  
if(pos != string::npos){ // OK  
    cout << pos;  
}
```

---

## Other DS

- `list<T>` (Doubly Linked List - has prev/next nodes)
    - `push_back`, `push_front`, `pop_back`, `pop_front`, `erase(it)`, `remove(value)`, `splice()`.
    - For `splice`:
      - `list1.splice(pos, list2, it)`: Moves element at `it` in `list2` to `list1` before `pos`
      - `list1.splice(pos, list2, first, last)`: Moves range `[first, last)` from `list2` into `list1`
  - `forward_list<T>` (Singly Linked List)
    - `push_front`, `pop_front`, `erase_after(it)`, `splice_after()`
  - `multiset<T>` / `unordered_multiset<T>` (Set which allows duplicates)
    - `insert`, `erase(value or it)`, `find`, `count`, `equal_range`
  - `multimap<K,V>` / `unordered_multimap<K,V>` (One key → many values)
    - `insert`, `erase(key or it)`, `find`, `equal_range`
  - `bitset<N>` (Fixed size bit array)
    - `set`, `reset`, `flip`, `test`, `count`
-

Construct-DS-Algos

## Linkedlist

```
class ListNode {
public:
    int val; ListNode* next;
    ListNode(int x) {
        val = x;
        next = nullptr;
    }
};

// Insert at end
ListNode* insertEnd(ListNode* head, int val) {
    ListNode* node = new ListNode(val);
    if (!head) return node;
    ListNode* temp = head;
    while (temp->next != nullptr) temp = temp->next;
    temp->next = node;
    return head;
}

// Insert at beginning
ListNode* insertBegin(ListNode* head, int val) {
    ListNode* node = new ListNode(val);
    node->next = head;
    return node;
}

void printList(ListNode* head) {
    while (head != nullptr) {
        cout << head->val << " -> ";
        head = head->next;
    }
}

void linkedListExample() {
    ListNode* head = nullptr;
    for (int i = 1; i <= 5; i++) head = insertEnd(head, i);
    head = insertBegin(head, 0);
    printList(head);
}
```

---

## Graph - Unweighted

```
// Add edge (undirected by default)
void addEdge(int u, int v, vector<vector<int>>& adj, bool directed =
false) {
    adj[u].push_back(v);
    if (!directed) adj[v].push_back(u);
}

// DFS traversal
void dfs(int u, vector<vector<int>>& adj, vector<int>& vis,
vector<int>& ans) {
    vis[u] = 1;
    ans.push_back(u);
    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs(v, adj, vis, ans);
        }
    }
}

// BFS traversal
void bfs(int start, vector<vector<int>>& adj, vector<int>& vis,
vector<int>& ans) {
    queue<int> q;
    q.push(start);
    vis[start] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);
        for (int v : adj[u]) {
            if (!vis[v]) {
                vis[v] = 1;
                q.push(v);
            }
        }
    }
}
```

```
// Example usage
void graphExample() {
    int n = 6;
    vector<vector<int>> adj(n); // standard adjacency list
    addEdge(0, 1, adj);
    addEdge(0, 2, adj);
    addEdge(1, 3, adj);
    addEdge(1, 4, adj);
    addEdge(2, 5, adj);
    vector<int> vis(n, 0), ans;
    dfs(0, adj, vis, ans);
    cout << "DFS: ";
    for (int x : ans) cout << x << " ";
    cout << "\n";
    ans.clear();
    fill(vis.begin(), vis.end(), 0);
    bfs(0, adj, vis, ans);
    cout << "BFS: ";
    for (int x : ans) cout << x << " ";
    cout << "\n";
}
-----
```

## Graph - Weighted

```
// Add weighted edge (directed by default)
void addWeightedEdge(
    int u, int v, int w,
    vector<vector<pair<int,int>>>& adj,
    bool directed = true
) {
    adj[u].push_back({v, w});
    if (!directed) adj[v].push_back({u, w});
}
```

```

// DFS traversal (weights ignored for traversal order)
void dfsWeighted(
    int u,
    vector<vector<pair<int,int>>>& adj,
    vector<int>& vis, vector<int>& ans
) {
    vis[u] = 1;
    ans.push_back(u);
    for (auto& [v, w] : adj[u]) {
        if (!vis[v]) {
            dfsWeighted(v, adj, vis, ans);
        }
    }
}

// BFS traversal (weights ignored)
void bfsWeighted(
    int start,
    vector<vector<pair<int,int>>>& adj,
    vector<int>& vis, vector<int>& ans
) {
    queue<int> q;
    q.push(start);
    vis[start] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        ans.push_back(u);
        for (auto& [v, w] : adj[u]) {
            if (!vis[v]) {
                vis[v] = 1;
                q.push(v);
            }
        }
    }
}

// Example usage
void weightedGraphExample() {
    int n = 6;
    vector<vector<pair<int,int>>> adj(n); // standard adj list

```

```

addWeightedEdge(0, 1, 10, adj);
addWeightedEdge(0, 2, 5, adj);
addWeightedEdge(1, 3, 1, adj);
addWeightedEdge(1, 4, 2, adj);
addWeightedEdge(2, 5, 9, adj);
vector<int> vis(n, 0), ans;
dfsWeighted(0, adj, vis, ans);
cout << "DFS (weighted, ignores weights): ";
for (int x : ans) cout << x << " \n";
ans.clear();
fill(vis.begin(), vis.end(), 0);
bfsWeighted(0, adj, vis, ans);
cout << "BFS (weighted, ignores weights): ";
for (int x : ans) cout << x << " \n";
cout << "Adjacency List:\n";
for (int i = 0; i < n; i++) {
    cout << i << ": ";
    for (auto& [v, w] : adj[i]) {
        cout << "(" << v << ", " << w << ") ";
    }
    cout << "\n";
}
}

```

---

## Stack & Queue

```

void stackQueueExample() {
    stack<int> st;
    st.push(1); st.push(2); st.push(3);
    cout << "Stack top: " << st.top() << "\n";
    st.pop();
    cout << "Stack top after pop: " << st.top() << "\n";
    queue<int> q;
    q.push(10); q.push(20); q.push(30);
    cout << "Queue front: " << q.front() << "\n";
    q.pop();
    cout << "Queue front after pop: " << q.front() << "\n";
}

```

-----

## Tree

```
class TreeNode {
public:
    int data; TreeNode* left; TreeNode* right;
    TreeNode(int val) {
        data = val;
        left = nullptr; right = nullptr;
    }
};

TreeNode* addNode(int val) { // Insert node manually or level-order
    TreeNode* node = new TreeNode(val);
    return node;
}

void inorder(TreeNode* root, vector<int>& ans) {
    if (!root) return; // Inorder traversal
    inorder(root->left, ans);
    ans.push_back(root->data);
    inorder(root->right, ans);
}

void preorder(TreeNode* root, vector<int>& ans) {
    if (!root) return; // Preorder traversal
    ans.push_back(root->data);
    preorder(root->left, ans);
    preorder(root->right, ans);
}

void levelOrder(TreeNode* root, vector<int>& ans) {
    if (!root) return; // Level order traversal
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* node = q.front(); q.pop();
        ans.push_back(node->data);
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}
```



```
// Example usage
void treeExample() {
    TreeNode* root = addNode(10);
    root->left = addNode(2);
    root->right = addNode(3);
    root->left->left = addNode(28);
    root->left->right = addNode(25);
    root->right->left = addNode(33);
    root->right->right = addNode(31);
    vector<int> ans;
    inorder(root, ans);
    cout << "Inorder: ";
    for (int x : ans) cout << x << " \n";
    ans.clear();
    preorder(root, ans);
    cout << "Preorder: ";
    for (int x : ans) cout << x << " \n";
    ans.clear();
    levelOrder(root, ans);
    cout << "Level order: ";
    for (int x : ans) cout << x << " ";
    cout << "\n";
}
```

-----

## **Others:**

### **Graph Traversal Algos**

#### **Kruskal's Algorithm (Minimum Spanning Tree)**

**Use when:** Graph is undirected + weighted, need minimum spanning tree. **Idea:** Sort edges → apply DSU to pick edges that connect different components. **Input graph edges:** (u, v, w):

(0,1,1), (0,2,4), (1,2,2), (1,3,7), (2,3,3)

**Sorted edges (by weight):**

1: (0,1), 2: (1,2), 3: (2,3), 4: (0,2), 7: (1,3)

**Pick without forming cycles** → MST cost = 1 + 2 + 3 = 6

struct DSU {

```

vector<int> parent, rankv;
DSU(int n) {
    parent.resize(n);
    rankv.assign(n, 0);
    for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
bool unite(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) return false;
    if (rankv[pa] < rankv[pb]) swap(pa, pb);
    parent[pb] = pa;
    if (rankv[pa] == rankv[pb]) rankv[pa]++;
    return true;
}
};
int kruskalMST(int n, vector<vector<int>>& edges) {
    // edges[i] = {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    int mstCost = 0;
    for (auto &e : edges) {
        int w = e[0], u = e[1], v = e[2];
        if (dsu.unite(u, v))
            mstCost += w;
    }
    return mstCost;
}

```

## Dijkstra's Algorithm (Shortest Path in Weighted Graph)

Use when: Graph has positive weights only.

Idea: Priority queue → always expand smallest distance.

Graph:

0 → 1 (4)

0 → 2 (1)

2 → 1 (2)

1 → 3 (1)

2 → 3 (5)

Running Dijkstra from 0:

Shortest distances:

0 → 0 = 0

0 → 2 = 1

0 → 1 = 1 + 2 = 3

0 → 3 = 3 + 1 = 4

```
int dijkstra(int n, vector<vector<pair<int,int>>>& adj, int src, int
dst) {
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;
    priority_queue<pair<int,int>,
                    vector<pair<int,int>>,
                    greater<pair<int,int>>> pq;
    pq.push({0, src}); // {distance, node}
    while (!pq.empty()) {
        auto cur = pq.top(); pq.pop();
        int d = cur.first, u = cur.second;
        if (u == dst) return d;
        if (d > dist[u]) continue;
        for (auto &p : adj[u]) {
            int v = p.first, w = p.second;
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist[dst];
}
```

### Floyd-Warshall (All-Pairs Shortest Path)

Use when: Need distance between every pair of nodes, graph max size ~200.

Works with: positive & negative weights (but no negative cycles)

Initial matrix (INF = 99999):

```
0  3  INF
2  0   5
INF INF 0
```

After Floyd-Warshall:

```
0  3   8
2  0   5
INF INF 0
```

```
void floydWarshall(vector<vector<int>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] == INT_MAX || dist[k][j] == INT_MAX)
                    continue;
                dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
            }
        }
    }
}
```

### Bellman-Ford (Shortest Path with Negative Weights)

Use when: Graph may have negative edges.

Also detects: Negative cycles.

Graph edges:

```
0 → 1 (4)
0 → 2 (5)
1 → 2 (-3)
2 → 3 (2)
```

Bellman-Ford from 0:

dist = [0, 4, 1, 3]

```

int bellmanFord(int n, vector<vector<int>>& edges, int src) {
    // edges[i] = {u, v, w}
    const int INF = 1e9;
    vector<int> dist(n, INF);
    dist[src] = 0;
    // Relax edges (n-1) times
    for (int i = 1; i < n; i++) {
        bool changed = false;
        for (auto &e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        }
        if (!changed) break;
    }
    // Check negative cycle
    for (auto &e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            return -1; // negative cycle detected
        }
    }
    return dist[src];    // or return dist[] array for all nodes
}

```

-----

## Binary Search Algo

```
int binarySearch(vector<int> &arr, int x) {
    int low = 0;
    int high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        // Check if x is present at mid
        if (arr[mid] == x)
            return mid;
        // If x greater, ignore left half
        if (arr[mid] < x)
            low = mid + 1;
        // If x is smaller, ignore right half
        else
            high = mid - 1;
    }
    // If we reach here, then element was not present
    return -1;
}
```

---

GenDS-FastSlowPointer

## General DS - Fast and Slow Pointer

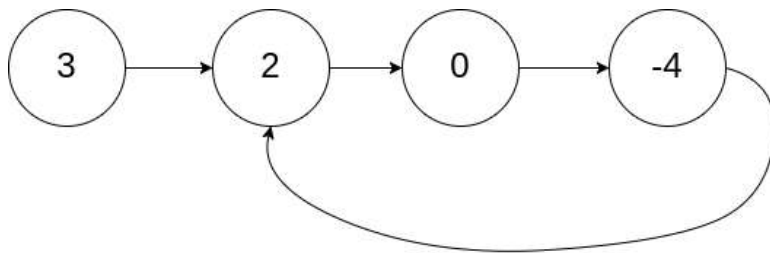
**Description:** This technique uses two pointers moving at different speeds to solve problems involving cycles, such as finding the middle of a list, detecting loops, or checking for palindromes.

- [Linked List Cycle](#)
- [Linked List Cycle II](#)
- [Remove nth Node from the End of List](#)
- [Palindrome Linked List](#)
- [Find the Duplicate Number](#)
- [\[Self Added v2\] Intersection of two Linkedlists](#)

Given head, the head of a linked list, determine if the linked list has a cycle in it.

Input: head = [3,2,0,-4], pos = 1. Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).



```
bool hasCycle(ListNode *head) {
    if (!head || !head->next) return false;
    ListNode *slow = head;
    ListNode *fast = head;
    while (fast && fast->next) { // doing fast & fast->next, so that
        fast->next->next don't crash. As fast->next->next is nullptr defined
        in constructor, so it's still accessible
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return true;
    }
    return false;
}
```

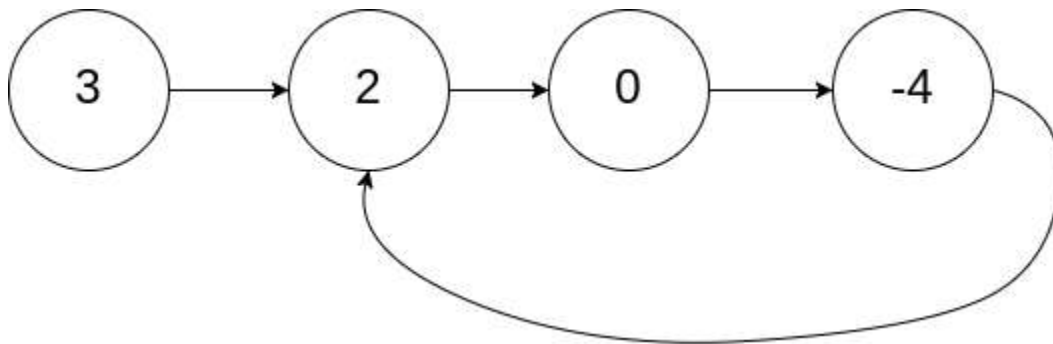


Given the **head** of a linked list, return the node where the cycle begins. If there is no cycle, return **null**.

Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

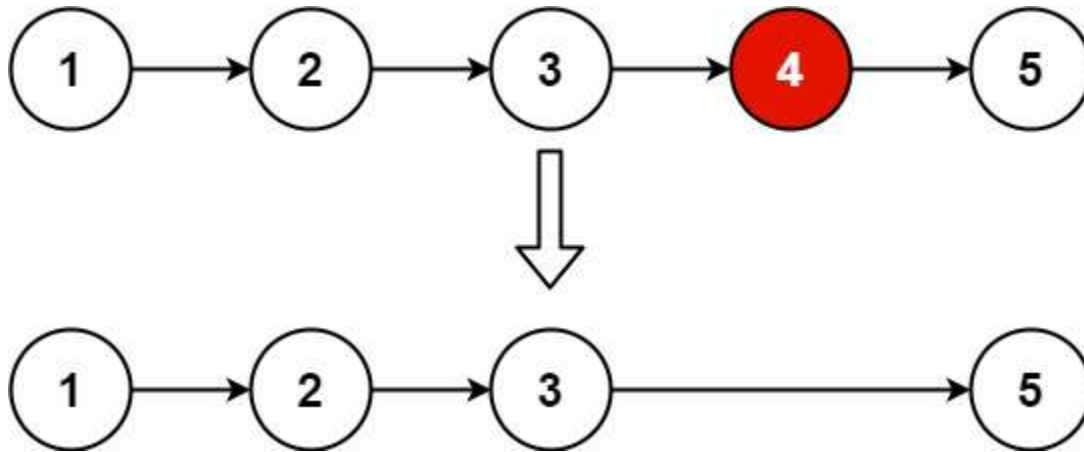


```
ListNode *detectCycle(ListNode *head) {  
    if (!head || !head->next) return nullptr;  
    ListNode *slow = head;  
    ListNode *fast = head;  
    while (fast && fast->next) { // Phase 1: Detect cycle  
        slow = slow->next;  
        fast = fast->next->next;  
        if (slow == fast) {  
            // Cycle found → move to phase 2  
            slow = head;  
            // Phase 2: find cycle start  
            while (slow != fast) {  
                slow = slow->next;  
                fast = fast->next;  
            }  
            return slow; // meeting point = cycle start  
        }  
    }  
    return nullptr; // no cycle  
}
```

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

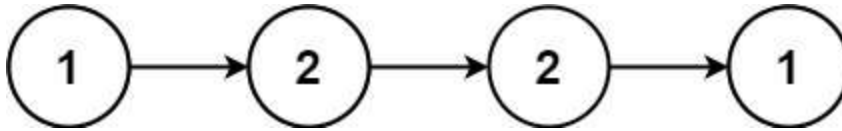


```
ListNode* removeNthFromEnd(ListNode* head, int gap) {  
    ListNode dummy(0);  
    dummy.next = head;  
    ListNode* fast = &dummy;  
    ListNode* slow = &dummy;  
    // Move fast gap+1 steps ahead  
    for (int i = 0; i <= gap; i++) {  
        fast = fast->next;  
    }  
    // Move both until fast reaches end  
    while (fast != nullptr) {  
        fast = fast->next;  
        slow = slow->next;  
    }  
    // Remove the nth node from end  
    slow->next = slow->next->next;  
    return dummy.next;  
}
```

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Input: head = [1,2,2,1]

Output: true



```

bool isPalindrome(ListNode* head) {
    vector<int> listVals;
    while (head) {
        listVals.push_back(head->val);
        head = head->next;
    }
    int left = 0, right = listVals.size() - 1;
    while (left < right && listVals[left] == listVals[right]) {
        left++;
        right--;
    }
    return left >= right;
// other approach could be to reverse linkedlist and compare
}

```

Given an array of integers `nums` containing  $n + 1$  integers where each integer is in the range  $[1, n]$  inclusive. There is only one repeated number in `nums`, return this repeated number. You must solve the problem without modifying the array `nums` and using only constant extra space.

**Input:** `nums = [1,3,4,2,2]`. **Output:** 2

```

int findDuplicate(vector<int>& nums) {
    int slow = nums[0];
    int fast = nums[0];
    do { // Step 1: Detect cycle - Floyd Cycle Detection
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);
    slow = nums[0]; // Step 2: Find entry point (duplicate)
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}

```

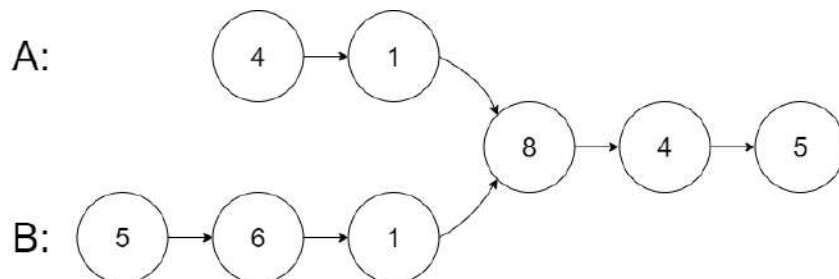
Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1: The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must retain their original structure after the function returns.

Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3. Output: Intersected at '8'

Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B. - Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.



```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
    if (!headA || !headB) return nullptr;
    ListNode *pA = headA;
    ListNode *pB = headB;
    // One pointer finishes its list, redirect to other list head.
    while (pA != pB) {
        pA = pA ? pA->next : headB;
        pB = pB ? pB->next : headA;
    }
    return pA; // either intersection node or nullptr
}
```

# GenDS-OverlappingIntervals

## General DS - Overlapping Intervals

Description: Intervals are often manipulated through sorting and merging based on their start and end times.

- [Basic Merge: Merge Intervals](#)
- [Interval Insertion: Insert Interval](#)
- [My Calendar ii](#)
- [Minimum Number of Arrows to Burst Balloons](#)
- [Non-overlapping Intervals](#)
- [\[Self Added v2\] Merge Sorted Array](#)
- [\[Self Added v2\] Meeting Rooms II](#)

Given an array of intervals where  $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ , merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Input:  $\text{intervals} = [[1,3],[2,6],[8,10],[15,18]]$

Output:  $[[1,6],[8,10],[15,18]]$ . Explanation: Since intervals  $[1,3]$  and  $[2,6]$  overlap, merge them into  $[1,6]$ .

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};
    // Step 1: Sort by start time, Step 2: Iterate and merge
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> merged;
    merged.push_back(intervals[0]);
    for (int i = 1; i < intervals.size(); ++i) {
        auto &last = merged.back(); // last interval in result
        // overlap check: next.start <= last.end
        if (intervals[i][0] <= last[1]) {
            // merge by updating end
            last[1] = max(last[1], intervals[i][1]);
        } else {
            // no overlap → push new interval
            merged.push_back(intervals[i]);
        }
    }
    return merged;
}
```

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the *i*th interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary). Return `intervals` after the insertion. Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Input: `intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval = [4,8]`. Output: `[[1,2],[3,10],[12,16]]`

Explanation: Because the new interval `[4,8]` overlaps with `[3,5]`, `[6,7]`, `[8,10]`.

```
vector<vector<int>> insert(vector<vector<int>>& intervals,
vector<int>& newInterval) {
    vector<vector<int>> res;
    int i = 0, n = intervals.size();
    // Add all intervals before newInterval
    while (i < n && intervals[i][1] < newInterval[0]) {
        res.push_back(intervals[i]);
        i++;
    }
    // Merge all overlapping intervals
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.push_back(newInterval);
    while (i < n) { // Add remaining intervals
        res.push_back(intervals[i]);
        i++;
    }
    return res;
}
```

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval  $[startTime, endTime)$ , the range of real numbers  $x$  such that  $startTime \leq x < endTime$ . Implement the `MyCalendarTwo` class:

`MyCalendarTwo()` Initializes the calendar object.

`boolean book(int startTime, int endTime)` Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Input

```
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]  
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
```

Output

```
[null, true, true, true, false, true, true]
```

Explanation

```
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
```

```
myCalendarTwo.book(10, 20); // return True, The event can be booked.
```

```
myCalendarTwo.book(50, 60); // return True, The event can be booked.
```

```
myCalendarTwo.book(10, 40); // return True, The event can be double  
booked.
```

```
myCalendarTwo.book(5, 15); // return False, The event cannot be  
booked, because it would result in a triple booking.
```

```
myCalendarTwo.book(5, 10); // return True, The event can be booked,  
as it does not use time 10 which is already double booked.
```

```
myCalendarTwo.book(25, 55); // return True, The event can be booked,  
as the time in [25, 40) will be double booked with the third event,  
the time [40, 50) will be single booked, and the time [50, 55) will  
be double booked with the second event.
```

```
vector<pair<int, int>> bookings; // all events
```

```
vector<pair<int, int>> overlaps; // double-booked regions
```



```

MyCalendarTwo() {}
bool book(int start, int end) {
    // Check against all double-booked intervals first (to prevent
    triple booking)
    for (int i = 0; i < overlaps.size(); i++) {
        int os = overlaps[i].first;
        int oe = overlaps[i].second;
        if (max(start, os) < min(end, oe)) {
            // If overlap exists with a double-booked range, triple
            booking would occur
            return false;
        }
    }
    // Add new overlaps (where this event overlaps existing single
    bookings)
    for (int i = 0; i < bookings.size(); i++) {
        int bs = bookings[i].first;
        int be = bookings[i].second;
        if (max(start, bs) < min(end, be)) {
            int overlapStart = max(start, bs);
            int overlapEnd = min(end, be);
            overlaps.push_back({overlapStart, overlapEnd});
        } // Check if overlap exists
    }
    // Finally add this booking
    bookings.push_back({start, end});
    return true;
}

```

/\*

Imagine:

Case1:

-----  
 \*-----

Case2:

-----  
 \*-----

Dry run:

| Step     | Input   | Result | bookings                                 |
|----------|---------|--------|------------------------------------------|
| overlaps |         |        |                                          |
| 1        | [10,20] | ✓      | [[10,20]]                                |
| 2        | [50,60] | ✓      | [[10,20],[50,60]]                        |
| 3        | [10,40] | ✓      | [[10,20],[50,60],[10,40]]                |
| 4        | [5,15]  | ✗      | same                                     |
| 5        | [5,10]  | ✓      | [[10,20],[50,60],[10,40],[5,10]]         |
| 6        | [25,55] | ✓      | [[10,20],[50,60],[10,40],[5,10],[25,55]] |

\*/

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose horizontal diameter stretches between `xstart` and `xend`. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with `xstart` and `xend` is burst by an arrow shot at `x` if `xstart <= x <= xend`. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path. Given the array `points`, return the minimum number of arrows that must be shot to burst all balloons.

Input: `points = [[10,16],[2,8],[1,6],[7,12]]`. Output: 2

Explanation: The balloons can be burst by 2 arrows:

- Shoot an arrow at `x = 6`, bursting the balloons `[2,8]` and `[1,6]`.
- Shoot an arrow at `x = 11`, bursting the balloons `[10,16]` and `[7,12]`.

```
int findMinArrowShots(vector<vector<int>>& points) {
    if (points.empty()) return 0;
```

```

// Step 1: Sort by end coordinate
sort(points.begin(), points.end(), [](auto &a, auto &b) {
    return a[1] < b[1];
});
int arrows = 1;
int end = points[0][1]; // arrow at the end of first balloon
// Step 2: Traverse balloons
for (auto &p : points) {
    if (p[0] > end) { // no overlap
        arrows++;
        end = p[1]; // new arrow at new balloon's end
    }
}
return arrows;
}

```

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

// It's conceptually similar to the Minimum Number of Arrows to Burst Balloons, but instead of counting arrows, we count how many intervals we must remove to eliminate overlaps.

`removals = total intervals - non-overlapping kept intervals`. All code remains same, just we return `removals`.

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of  $m + n$ , where the first  $m$  elements denote the elements that should be merged, and the last  $n$  elements are set to 0 and should be ignored. `nums2` has a length of  $n$ .

Input: `nums1 = [1,2,3,0,0,0]`,  $m = 3$ , `nums2 = [2,5,6]`,  $n = 3$

Output: `[1,2,2,3,5,6]`

Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.

The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

```
void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
    int i = m - 1;          // last index of nums1 actual values
    int j = n - 1;          // last index of nums2
    int k = m + n - 1;      // last index of nums1 total size
    while (j >= 0) {
        if (i >= 0 && nums1[i] > nums2[j]) {
            nums1[k--] = nums1[i--];
        } else {
            nums1[k--] = nums2[j--];
        }
    }
}
```

Given an array of meeting time intervals `intervals` where `intervals[i] = [start_i, end_i]`, return the minimum number of conference rooms required.

input: `intervals = [[0, 30], [5, 10], [15, 20]]`

output: 2

```
int minMeetingRooms(vector<vector<int>>& intervals) {
    int n = intervals.size();
    vector<int> starts, ends;
    for (auto &it : intervals) {
        starts.push_back(it[0]);
    }
}
```

```
        ends.push_back(it[1]);
    }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());
    int rooms = 0, endPtr = 0;
    for (int i = 0; i < n; i++) {
        if (starts[i] < ends[endPtr]) {
            rooms++;        // need new room
        } else {
            endPtr++;        // meeting ended → free a room
        }
    }
    return rooms;
}
```

GenDS-PrefixSum

## General DS - Prefix Sum

Description: Prefix Sums/Products are techniques that store cumulative sums or products up to each index, allowing for quick subarray range queries.

- [Find the middle index in array](#)
- [Product of array except self](#)
- [Maximum product subarray](#)
- [Number of ways to split array](#)
- [Range Sum Query 2D](#)

Given a 0-indexed integer array `nums`, find the leftmost `middleIndex` (i.e., the smallest amongst all the possible ones).

A `middleIndex` is an index where  $\text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[\text{middleIndex}-1] == \text{nums}[\text{middleIndex}+1] + \text{nums}[\text{middleIndex}+2] + \dots + \text{nums}[\text{nums.length}-1]$ .

If `middleIndex == 0`, the left side sum is considered to be 0.

Similarly, if `middleIndex == nums.length - 1`, the right side sum is considered to be 0.

Return the leftmost `middleIndex` that satisfies the condition, or -1 if there is no such index.

Input: `nums = [2,3,-1,8,4]`

Output: 3

Explanation: The sum of the numbers before index 3 is:  $2+3+-1 = 4$

The sum of the numbers after index 3 is:  $4 = 4$

```
int findMiddleIndex(vector<int>& nums) {
    int totalSum = accumulate(nums.begin(), nums.end(), 0);
    int leftSum = 0;
    for (int i = 0; i < nums.size(); i++) {
        int rightSum = totalSum - leftSum - nums[i];
        if (leftSum == rightSum)
            return i; // Found the middle index
        leftSum += nums[i];
    }
    return -1; // No valid index found
}
```

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in  $O(n)$  time and without using the division operation.

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, 1);
    int prefix = 1;
    for (int i = 0; i < n; i++) {
        answer[i] = prefix;           // product of everything to the
left of i
        prefix *= nums[i];           // update prefix for next index
    }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= suffix;         // multiply by product of
everything to the right of i
        suffix *= nums[i];           // update suffix for next index
    }
    return answer;
}
```

Given an integer array `nums`, find a subarray that has the largest product, and return the product. The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: `[2,3]` has the largest product 6.



```

int maxProduct(int A[], int n) {
    // store the result that is the max we have found so far
    int r = A[0];
    // imax/imin stores the max/min product of
    // subarray that ends with the current number A[i]
    for (int i = 1, imax = r, imin = r; i < n; i++) {
        // multiplied by a negative makes big number smaller, small
number bigger
        // so we redefine the extremums by swapping them
        if (A[i] < 0)
            swap(imax, imin);
        // max/min product for the current number is either the
current number itself or the max/min by the previous number times
the current one
        imax = max(A[i], imax * A[i]);
        imin = min(A[i], imin * A[i]);
        // the newly computed max value is a candidate for our
global result
        r = max(r, imax);
    }
    return r;
}

```

You are given a 0-indexed integer array `nums` of length `n`.

`nums` contains a valid split at index `i` if the following are true:

The sum of the first `i + 1` elements is greater than or equal to the sum of the last `n - i - 1` elements.

There is at least one element to the right of `i`. That is,  $0 \leq i < n - 1$ .

Return the number of valid splits in `nums`.

Input: `nums = [10,4,-8,7]`

Output: 2

Explanation:

There are three ways of splitting `nums` into two non-empty parts:

- Split `nums` at index 0. Then, the first part is `[10]`, and its sum is 10. The second part is `[4,-8,7]`, and its sum is 3. Since  $10 \geq 3$ ,  $i = 0$  is a valid split.

- Split nums at index 1. Then, the first part is [10,4], and its sum is 14. The second part is [-8,7], and its sum is -1. Since  $14 \geq -1$ ,  $i = 1$  is a valid split.
- Split nums at index 2. Then, the first part is [10,4,-8], and its sum is 6. The second part is [7], and its sum is 7. Since  $6 < 7$ ,  $i = 2$  is not a valid split.

Thus, the number of valid splits in nums is 2.

```
int waysToSplitArray(vector<int>& nums) {
    long long total = 0;
    for (int x : nums) total += x;
    long long prefix = 0;
    int count = 0;
    for (int i = 0; i < nums.size() - 1; i++) {
        prefix += nums[i];
        if (prefix * 2 >= total) count++;
        // prefixSum[i] >= totalSum - prefixSum[i] -> Equivalent to:
        2 * prefixSum[i] >= totalSum
    }
    return count;
}
```

Given a 2D matrix matrix, handle multiple queries of the following type:

Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.

int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

You must design an algorithm where sumRegion works on  $O(1)$  time complexity.

Input

```
["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
[[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [1, 1, 2, 2], [1, 2, 2, 4]]]
```

## Output

[null, 8, 11, 12]

Explanation: NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2],  
[5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);

numMatrix.sumRegion(2, 1, 4, 3); // return 8 (i.e sum of the red rectangle)

numMatrix.sumRegion(1, 1, 2, 2); // return 11 (i.e sum of the green rectangle)

numMatrix.sumRegion(1, 2, 2, 4); // return 12 (i.e sum of the blue rectangle)

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 0 | 1 | 4 | 2 |
| 5 | 6 | 3 | 2 | 1 |
| 1 | 2 | 0 | 1 | 5 |
| 4 | 1 | 0 | 1 | 7 |
| 1 | 0 | 3 | 0 | 5 |

```
NumMatrix(vector<vector<int>>& matrix) {
    int m = matrix.size();
    int n = matrix[0].size();
    prefix = vector<vector<int>>(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            prefix[i][j] = matrix[i-1][j-1]
                + prefix[i-1][j]
                + prefix[i][j-1]
                - prefix[i-1][j-1];
        }
    }
}

int sumRegion(int row1, int col1, int row2, int col2) {
    return prefix[row2+1][col2+1]
        - prefix[row1][col2+1]
        - prefix[row2+1][col1]
        + prefix[row1][col1];
}
```

GenDS-SlidingWindowFixSize

## General DS - Sliding Window

Description: A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Fixed Size

- [Maximum Sum Subarray of Size K](#)
- [Number of Subarrays having Average Greater or Equal to Threshold](#)
- [Repeated DNA sequences](#)
- [Permutation in String](#)
- [Sliding Subarray Beauty](#)
- [Sliding Window Maximum](#)

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

The length of the subarray is `k`, and

All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Input: `nums = [1,5,4,2,9,9,9]`, `k = 3`

Output: 15

Explanation: The subarrays of `nums` with length 3 are:

- `[1,5,4]` which meets the requirements and has a sum of 10.
- `[5,4,2]` which meets the requirements and has a sum of 11.
- `[4,2,9]` which meets the requirements and has a sum of 15.
- `[2,9,9]` which does not meet the requirements because the element 9 is repeated.
- `[9,9,9]` which does not meet the requirements because the element 9 is repeated.

We return 15 because it is the maximum subarray sum of all the subarrays that meet the conditions

```

int maximumSubarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> freq; // element -> frequency
    long long curSum = 0, maxSum = 0;
    int left = 0;
    for (int right = 0; right < nums.size(); right++) {
        freq[nums[right]]++;
        curSum += nums[right];
        // shrink window if duplicate or size > k
        while (freq[nums[right]] > 1 || right - left + 1 > k) {
            freq[nums[left]]--;
            curSum -= nums[left];
            left++;
            if (freq[nums[left]] == 0)
                freq.erase(nums[left]);
        }
        // valid window of size k with distinct elements
        if (right - left + 1 == k)
            maxSum = max(maxSum, curSum);
    }
    return (int)maxSum;
}

```

Given an array of integers arr and two integers k and threshold, return the number of sub-arrays of size k and average greater than or equal to threshold.

Input: arr = [2,2,2,2,5,5,5,8], k = 3, threshold = 4

Output: 3

Explanation: Sub-arrays [2,5,5],[5,5,5] and [5,5,8] have averages 4, 5 and 6 respectively. All other sub-arrays of size 3 have averages less than 4 (the threshold).

```

int numOfSubarrays(vector<int>& arr, int k, int threshold) {
    int n = arr.size();
    int target = k * threshold; // required minimum sum
    int count = 0;
    int windowSum = 0;
    // Step 1: initialize first window
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }
    if (windowSum >= target) count++;
    // Step 2: slide the window
    for (int i = k; i < n; i++) {
        windowSum += arr[i]; // add rightmost element
        windowSum -= arr[i - k]; // remove leftmost element
        if (windowSum >= target)
            count++;
    }
    return count;
}

```

The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'.

For example, "ACGAATTCCG" is a DNA sequence.

When studying DNA, it is useful to identify repeated sequences within the DNA.

Given a string *s* that represents a DNA sequence, return all the 10-letter-long sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in any order.

Input: *s* = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"

Output: ["AAAAACCCCC", "CCCCCAAAAA"]

```

vector<string> findRepeatedDnaSequences(string s) {
    int n = s.size(); // could also use a map<int,int> to store
sequence & count and get seqs with count >=2
    unordered_set<string> seen; // stores all seen substrings
    unordered_set<string> repeated; // substrings seen more times
    for (int i = 0; i + 10 <= n; i++) {
        string sub = s.substr(i, 10);
        if (seen.count(sub))
            repeated.insert(sub); // found again
        else
            seen.insert(sub); // first occurrence
    }
    return vector<string>(repeated.begin(), repeated.end());
}

```

Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false otherwise.  
In other words, return true if one of s1's permutations is the substring of s2.

**Input:** s1 = "ab", s2 = "eidbaooo"

**Output:** true

**Explanation:** s2 contains one permutation of s1 ("ba").

```

bool checkInclusion(string s1, string s2) {
    if (s1.size() > s2.size()) return false;
    vector<int> count1(26, 0), count2(26, 0);
    // build initial count for s1
    for (char c : s1) count1[c - 'a']++;
    int k = s1.size();
    // sliding window
    for (int i = 0; i < s2.size(); i++) {
        count2[s2[i] - 'a']++; // include current char
        // remove char left of window if window too large
        if (i >= k) count2[s2[i - k] - 'a']--;
        // check if match
        if (count1 == count2) return true;
    }
    return false;
}

```



Given an integer array `nums` containing `n` integers, find the beauty of each subarray of size `k`. The beauty of a subarray is the `x`th smallest integer in the subarray if it is negative, or `0` if there are fewer than `x` negative integers.

Return an integer array containing `n - k + 1` integers, which denote the beauty of the subarrays in order from the first index in the array.

A subarray is a contiguous non-empty sequence of elements within an array.

Input: `nums = [1,-1,-3,-2,3]`, `k = 3`, `x = 2`

Output: `[-1,-2,-2]`

Explanation: There are 3 subarrays with size `k = 3`.

The first subarray is `[1, -1, -3]` and the 2nd smallest negative integer is `-1`.

The second subarray is `[-1, -3, -2]` and the 2nd smallest negative integer is `-2`.

The third subarray is `[-3, -2, 3]` and the 2nd smallest negative integer is `-2`.

// If a positive integer, then beauty would've been `0`.

```

vector<int> getSubarrayBeauty(vector<int>& nums, int k, int x) {
    int n = nums.size();
    vector<int> res;
    vector<int> freq(51, 0); // store freq of -ve's from -50 to -1
    for (int i = 0; i < n; i++) {
        if (nums[i] < 0) // Add current number to frequency
            freq[nums[i] + 50]++;
        // Remove element that goes out of the window
        if (i >= k && nums[i - k] < 0)
            freq[nums[i - k] + 50]--;
        // When window size becomes k, calculate the beauty
        if (i >= k - 1) {
            int count = 0;
            int val = 0;
            // find the x-th smallest negative number
            for (int j = 0; j < 51; j++) {
                count += freq[j];
                if (count >= x) {
                    val = j - 50;
                    // convert index back to original number
                    break;
                }
            }
            res.push_back(val);
        }
    }
    return res;
}

```

**[hard]**

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

| Window position     | Max   |
|---------------------|-------|
| -----               | ----- |
| [1 3 -1] -3 5 3 6 7 | 3     |
| 1 [3 -1 -3] 5 3 6 7 | 3     |
| 1 3 [-1 -3 5] 3 6 7 | 5     |
| 1 3 -1 [-3 5 3] 6 7 | 5     |
| 1 3 -1 -3 [5 3 6] 7 | 6     |
| 1 3 -1 -3 5 [3 6 7] | 7     |

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // stores indices of useful elements
    vector<int> res;
    int n = nums.size();
    for (int i = 0; i < n; ++i) {
        // Remove indices that are out of this window
        if (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();
        // Remove smaller elements from back
        while (!dq.empty() && nums[dq.back()] <= nums[i])
            dq.pop_back();
        // Add current index
        dq.push_back(i);
        // Add current window's max to result (once window size ≥ k)
        if (i >= k - 1)
            res.push_back(nums[dq.front()]);
    }
    return res;
}
/*
```

We maintain a deque such that:

Indices are stored in decreasing order of their values ( $\text{nums}[i]$ )

The front of deque always holds index of current window's maximum

When window moves:

Remove elements from front if they go out of window ( $\text{index} < i - k + 1$ ). Remove elements from back if their value  $\leq$  current element ( $\text{nums}[i]$ ). Push current index  $i$  to deque. If window size  $\geq k$ , add  $\text{nums}[\text{dq.front()}]$  to result

\*/

GenDS-SlidingWindowVariableSize

## General DS - Sliding Window

Description: A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Variable Size

- [Longest Substring Without Repeating Characters](#)
- [Minimum Size Subarray Sum](#)
- [Subarray Product Less Than K](#)
- [Max Consecutive Ones](#)
- [Fruits Into Baskets](#)
- [Count Number of Nice Subarrays](#)
- [Minimum Window Substring: Minimum Window Substring](#)

Given a string *s*, find the length of the longest substring without duplicate characters.

Input: *s* = "pwwkew"

Output: 3.

Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

```
int lengthOfLongestSubstring(string s) {
    unordered_set<char> seen;
    int left = 0, right = 0, maxLen = 0;
    while (right < s.size()) {
        // if character not seen → expand window
        if (seen.find(s[right]) == seen.end()) {
            seen.insert(s[right]);
            maxLen = max(maxLen, right - left + 1);
            right++;
        }
        else { // if repeated → shrink window
            seen.erase(s[left]);
            left++;
        }
    }
    return maxLen;
}
```

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

Input: `target = 7, nums = [2,3,1,2,4,3]`

Output: 2

Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

```
int minSubArrayLen(int target, vector<int>& nums) {
    int n = nums.size();
    int left = 0, curSum = 0, minLen = INT_MAX;
    for (int right = 0; right < n; right++) {
        curSum += nums[right];
        while (curSum >= target) {
            minLen = min(minLen, right - left + 1);
            curSum -= nums[left];
            left++;
        }
    }
    return (minLen == INT_MAX) ? 0 : minLen;
}
```

Given an array of integers `nums` and an integer `k`, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than `k`.

Input: `nums = [10,5,2,6], k = 100`

Output: 8

Explanation: The 8 subarrays that have product less than 100 are: `[10]`, `[5]`, `[2]`, `[6]`, `[10, 5]`, `[5, 2]`, `[2, 6]`, `[5, 2, 6]`

Note that `[10, 5, 2]` is not included as the product of 100 is not strictly less than `k`.

```

int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    if (k <= 1) return 0; // since product of positives ≥1, no valid
subarray possible
    int left = 0;
    int count = 0;
    long long prod = 1;
    for (int right = 0; right < nums.size(); right++) {
        prod *= nums[right];
        while (prod >= k) { // shrink window if product ≥ k
            prod /= nums[left];
            left++;
        } // all subarrays ending at right and starting from
[left..right] are valid
        count += (right - left + 1);
    }
    return count;
}

```

Given a binary array **nums** and an integer **k**, return the maximum number of consecutive 1's in the array if you can flip at most **k** 0's.

**Input:** **nums** = [1,1,1,0,0,0,1,1,1,1,0], **k** = 2. **Output:** 6

**Explanation:** [1,1,1,0,0,1,1,1,1,1,1]. Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

```

int longestOnes(vector<int>& nums, int k) {
    int left = 0, zeroCount = 0, maxLen = 0;
    for (int right = 0; right < nums.size(); ++right) {
        if (nums[right] == 0) zeroCount++;
        // shrink window until valid (≤ k flips)
        while (zeroCount > k) {
            if (nums[left] == 0) zeroCount--;
            left++;
        }
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

```

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the type of fruit the *i*th tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.

Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.

Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return the maximum number of fruits you can pick.

Input: `fruits = [1,2,3,2,2]`

Output: 4

Explanation: We can pick from trees `[2,3,2,2]`.

If we had started at the first tree, we would only pick from trees `[1,2]`.

```
int totalFruit(vector<int>& fruits) {
    unordered_map<int, int> basket;
    int left = 0, maxFruits = 0;
    for (int right = 0; right < fruits.size(); ++right) {
        basket[fruits[right]]++; // pick fruit at right
        // if more than 2 fruit types, move left to shrink window
        while (basket.size() > 2) {
            basket[fruits[left]]--;
            if (basket[fruits[left]] == 0)
                basket.erase(fruits[left]);
            left++;
        }
        // update max window size
        maxFruits = max(maxFruits, right - left + 1);
    }
    return maxFruits;
}
```



Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Input: `nums = [1,1,2,1,1]`, `k = 3`

Output: 2

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

// Kind of hard to understand - logic dependent on `-> atMost(nums, k) - atMost(nums, k-1)`

```
int numberOfSubarrays(vector<int>& nums, int k) {
    return atMost(nums, k) - atMost(nums, k-1);
}

int atMost(vector<int>& nums, int k) {
    int left = 0, count = 0, odd = 0;
    for (int right = 0; right < nums.size(); right++) {
        if (nums[right] % 2 == 1) odd++;
        while (odd > k) {
            if (nums[left] % 2 == 1) odd--;
            left++;
        }
        count += right - left + 1;
    }
    return count;
}
```

[hard]

Given two strings `s` and `t` of lengths `m` and `n` respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

The testcases will be generated such that the answer is unique.

Input: `s = "ADOBECODEBANC"`, `t = "ABC"`

Output: "BANC"

Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string `t`.

```

string minWindow(string s, string t) {
    if (s.size() == 0 || t.size() == 0) return "";
    vector<int> need(128, 0);
    for (char c : t) need[c]++;    // required counts
    int required = 0;
    for (int x : need) if (x > 0) required++;
    vector<int> have(128, 0);
    int formed = 0;
    int left = 0, right = 0;
    int bestLen = INT_MAX;
    int bestStart = 0;
    while (right < s.size()) {
        char c = s[right];
        have[c]++;
        if (need[c] > 0 && have[c] == need[c])
            formed++;
        // When valid, start shrinking
        while (formed == required) {
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestStart = left;
            }
            char d = s[left];
            have[d]--;
            if (need[d] > 0 && have[d] < need[d])
                formed--;
            left++;
        }
        right++;
    }
    if (bestLen == INT_MAX) return "";
    return s.substr(bestStart, bestLen);
}

```

# GenDS-TwoPointers

## General DS - Two Pointers

**Description:** The two pointers technique involves having two different indices move through the input at different speeds to solve various array or linked list problems.

- [Two Sum II - Input Array is Sorted](#)
- [Dutch National Flag: Sort Colors](#)
- [Next Permutation](#)
- [Bag of Tokens](#)
- [Container with most water](#)
- [Trapping Rain Water](#)

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where  $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$ . Return the indices of the two numbers, index1 and index2, added by one as an integer array [index1, index2] of length 2. The tests are generated such that there is exactly one solution. You may not use the same element twice. Use only constant extra space.

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

```
vector<int> twoSum(vector<int>& numbers, int target) {
    int left = 0, right = numbers.size() - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target)
            return {left + 1, right + 1}; // +1 because 1-indexed
        else if (sum < target)
            left++; // need bigger sum
        else
            right--; // need smaller sum
    }
    return {}; // will never happen, as one solution is guaranteed
}
```

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

```
void sortColors(vector<int>& nums) {
    int start = 0;
    int mid = 0;
    int end = nums.size()-1;
    while(mid<=end){
        if(nums[mid]==0){
            swap(nums[mid], nums[start]);
            start++; mid++;
        }
        else if(nums[mid]==1){
            mid++;
        }
        else{
            swap(nums[mid], nums[end]);
            end--;
        }
    }
}
```

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2, 1, 3], [2, 3, 1], [3,1,2], [3,2,1]. The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order). For example, the next permutation of arr = [1,2,3] is [1,3,2].

Similarly, the next permutation of arr = [2,3,1] is [3,1,2].

While the next permutation of arr = [3,2,1] is [1,2,3] because [3,2,1] does not have a lexicographical larger rearrangement.

Given an array of integers nums, find the next permutation of nums. The replacement must be in place and use only constant extra memory.

Input: nums = [1,2,3]

Output: [1,3,2]

// Not quite intuitive, you may/maynot know ans, don't worry.

```
void nextPermutation(vector<int>& nums) {
    int i = nums.size() - 1;
    while (i > 0 && nums[i-1] >= nums[i]) {
        i--;
    }
    if (i == 0) {
        reverse(nums.begin(), nums.end());
        return;
    }
    int j = nums.size() - 1;
    while (j >= i && nums[j] <= nums[i-1]) {
        j--;
    }
    swap(nums[i-1], nums[j]);
    reverse(nums.begin() + i, nums.end());
}
```

You start with an initial power of `power`, an initial score of 0, and a bag of tokens given as an integer array `tokens`, where each `tokens[i]` denotes the value of token `i`. Your goal is to maximize the total score by strategically playing these tokens. In one move, you can play an unplayed token in one of the two ways (but not both for the same token):

- Face-up: If your current power is at least `tokens[i]`, you may play token `i`, losing `tokens[i]` power and gaining 1 score.
- Face-down: If your current score is at least 1, you may play token `i`, gaining `tokens[i]` power and losing 1 score.

Return the maximum possible score you can achieve after playing any number of tokens. Input: `tokens = [100,200,300,400]`, `power = 200`.

Output: 2. Explanation: Play the tokens in this order to get a score of 2: Play token0 (100) face-up, reducing power to 100 and increasing score to 1. Play token3 (400) face-down, increasing power to 500 and reducing score to 0. Play token1 (200) face-up, reducing power to 300 and increasing score to 1. Play token2 (300) face-up, reducing power to 0 and increasing score to 2. The maximum score achievable is 2.

```
int bagOfTokensScore(vector<int>& tokens, int power) {
    sort(tokens.begin(), tokens.end());
    int left = 0, right = tokens.size() - 1;
    int score = 0, maxScore = 0;
    while (left <= right) {
        if (power >= tokens[left]) {
            power -= tokens[left];
            score++; left++;
            maxScore = max(maxScore, score);
        } // If we can afford the smallest token → play face-up
        else if (score > 0) {
            power += tokens[right];
            score--; right--;
        } // Else, if possible, trade score for power (face-down)
        else {
            break; // If neither move possible → break
        }
    }
    return maxScore;
}
```

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are  $(i, 0)$  and  $(i, \text{height}[i])$ .

Find two lines that together with the x-axis form a container, such that the container contains the most water.

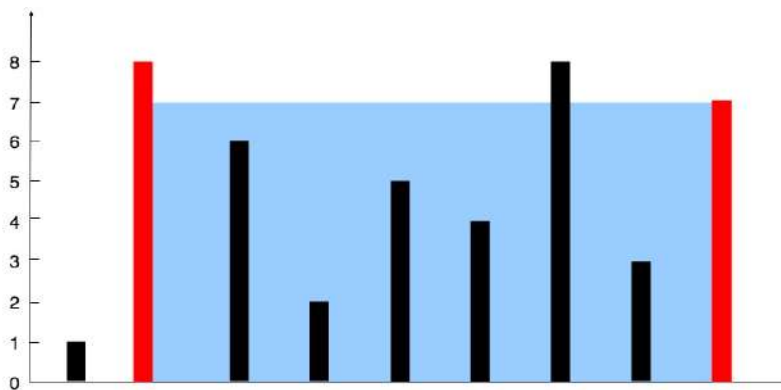
Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.



```
int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int maxArea = 0;
    while (left < right) {
        int h = min(height[left], height[right]);
        int width = right - left;
        maxArea = max(maxArea, h * width);
        // Move the smaller height inward
        if (height[left] < height[right])
            left++;
        else
            right--;
    }
    return maxArea;
}
```



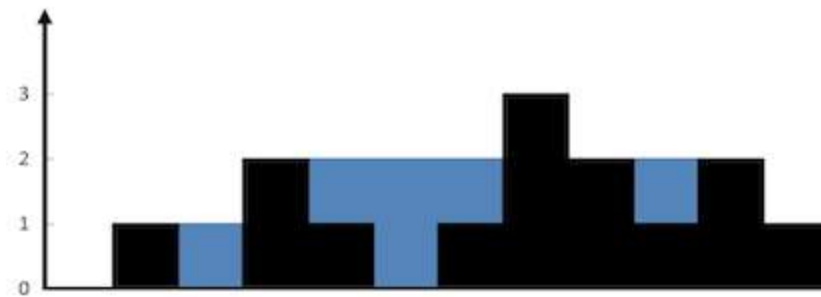
[hard]\*\*

Given  $n$  non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.



```
int trap(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax)
                leftMax = height[left];
            else
                water += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax)
                rightMax = height[right];
            else
                water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
```

# GenDS-CyclicSort

## General DS - Cyclic Sort (Index-Based)

**Description:** Cyclic sort is an efficient approach to solve problems where numbers are consecutively ordered and must be placed in the correct index.

- [Missing Number](#)
- [Set Mismatch](#)
- [Find Missing Numbers](#)
- [First Missing Positive](#)

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

Input: `nums = [3,0,1]`

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Ans -> Could also use: `Missing number = totalSum - actualSum`.

`totalSum = n(n+1)/2`. Also Bitwise XOR method there. Eg:

```
for(int i=0; i<nums.length; i++){
    res ^= i;
    res ^= nums[i];
}
return res...
```

You have a set of integers `s`, which originally contains all the numbers from 1 to `n`. Unfortunately, due to some error, one of the numbers in `s` got duplicated to another number in the set, which results in repetition of one number and loss of another number. You are given an integer array `nums` representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Input: `nums = [1,2,2,4]`

Output: `[2,3]`

```

vector<int> findErrorNums(vector<int>& nums) {
    int duplicate = -1, missing = -1;
    for (int i = 0; i < nums.size(); i++) {
        int idx = abs(nums[i]) - 1; // Step 1: find the duplicate
        if (nums[idx] < 0) { // already visited → duplicate
            duplicate = abs(nums[i]);
        } else {
            nums[idx] = -nums[idx]; // mark visited
        }
    }
    for (int i = 0; i < nums.size(); i++) {
        if (nums[i] > 0) {
            missing = i + 1;
            break;
        } // Step 2: find the missing (index still positive)
    }
    return {duplicate, missing};
}

```

**Given an array `nums` of `n` integers where `nums[i]` is in the range `[1, n]`, return an array of all the integers in the range `[1, n]` that do not appear in `nums`. Input: `nums = [4,3,2,7,8,2,3,1]`. Output: `[5,6]`**

```

vector<int> findDisappearedNumbers(vector<int>& nums) {
    int n = nums.size();
    // Mark presence by negating nums[x-1]
    for (int i = 0; i < n; i++) {
        int idx = abs(nums[i]) - 1; // target index
        if (nums[idx] > 0)
            nums[idx] = -nums[idx]; // mark as visited
    }
    vector<int> result;
    for (int i = 0; i < n; i++) {
        if (nums[i] > 0) {
            result.push_back(i + 1); // positive means missing
        }
    }
    return result;
}

```

[hard]

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in  $O(n)$  time and uses  $O(1)$  auxiliary space.

Input: `nums = [3,4,-1,1]`

Output: 2

Explanation: 1 is in the array but 2 is missing.

```
int firstMissingPositive(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n; i++) {
        while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] !=
nums[i]) {
            // place nums[i] at its correct position
            swap(nums[i], nums[nums[i] - 1]);
        }
    }
    // now find the first index where nums[i] != i+1
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1)
            return i + 1;
    }
    return n + 1;
}
```

# LList-ReversalManipulation

## LinkedList - Reversal & Manipulation of Linked List (In-place)

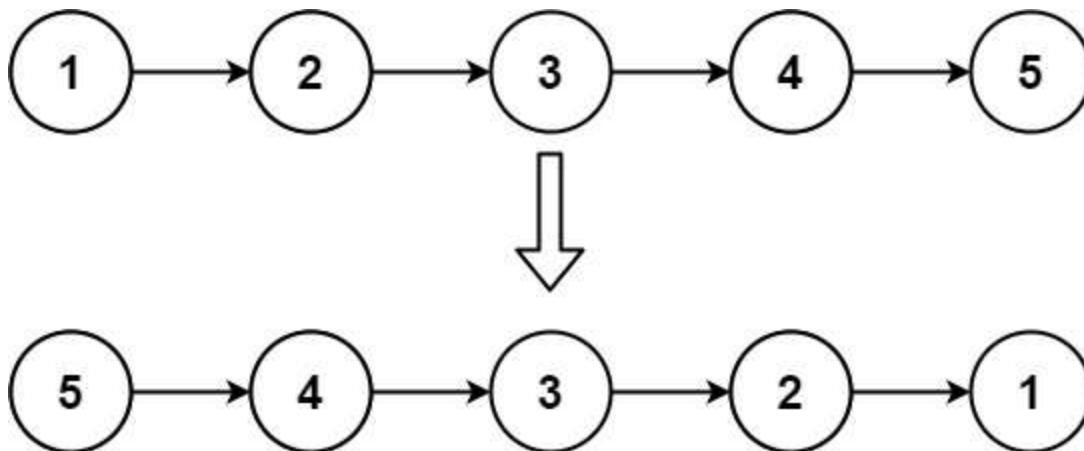
Description: Reversing a linked list in place without using extra space is key for problems that require in-place list manipulations.

- [Reverse Linked List](#)
- [Reverse Nodes in k-Group](#)
- [Swap Nodes in Pairs](#)
- [\[Self added v2\] Flatten LinkedList](#)

Given the head of a singly linked list, reverse the list, and return the reversed list.

Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

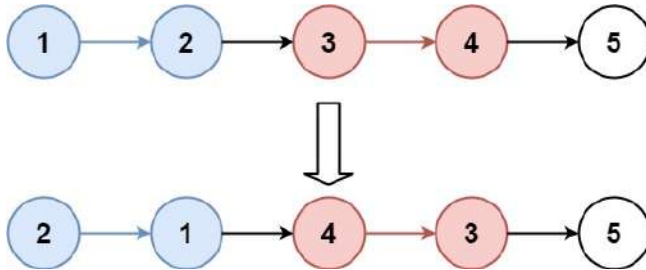


```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* next = curr->next; // store next
        curr->next = prev;           // reverse pointer
        prev = curr;                 // move prev
        curr = next;                 // move curr
    }
    return prev; // new head
}
```

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed. Input: head = [1,2,3,4,5], k = 2. Output: [2,1,4,3,5]



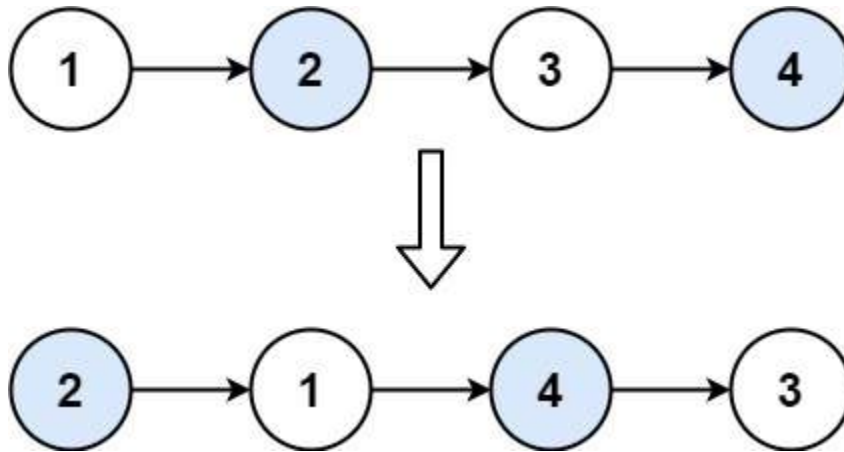
```
ListNode* reverseKGroup(ListNode* head, int k) {
    if (!head || k == 1) return head;
    ListNode dummy(0); dummy.next = head;
    ListNode* prevGroupEnd = &dummy;
    while (true) {
        ListNode* kth = prevGroupEnd;
        for (int i = 0; i < k; i++) {
            kth = kth->next;
            if (!kth) return dummy.next; // Not enough nodes left
        }
        ListNode* groupStart = prevGroupEnd->next;
        ListNode* nextGroupStart = kth->next;
        ListNode* prev = nextGroupStart;
        ListNode* curr = groupStart;
        while (curr != nextGroupStart) {
            ListNode* next = curr->next;
            curr->next = prev;
            prev = curr; curr = next;
        }
        prevGroupEnd->next = kth; // new head of reversed segment
        prevGroupEnd = groupStart; // becomes end of this segment
    }
    return dummy.next;
}
```



Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Input: head = [1,2,3,4]

Output: [2,1,4,3]



```
ListNode* swapPairs(ListNode* head) {  
    ListNode dummy(0);  
    dummy.next = head;  
    ListNode* prev = &dummy;  
    while (prev->next && prev->next->next) {  
        ListNode* first = prev->next;  
        ListNode* second = prev->next->next;  
        // Swapping  
        first->next = second->next;  
        second->next = first;  
        prev->next = second;  
        // Move prev two nodes ahead  
        prev = first;  
    }  
    return dummy.next;  
}
```

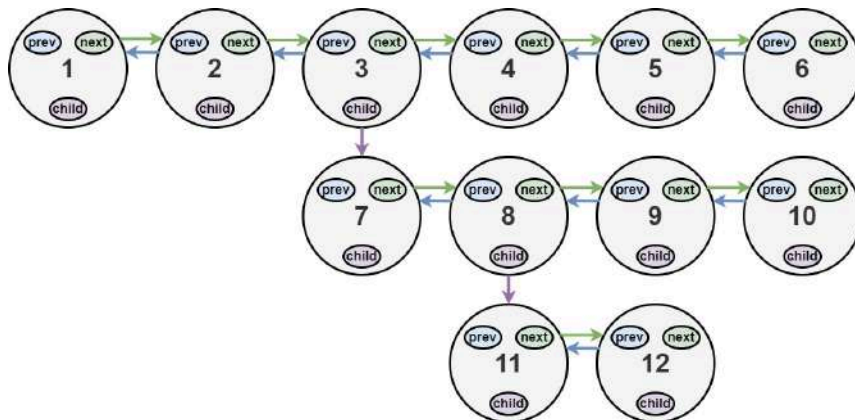
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

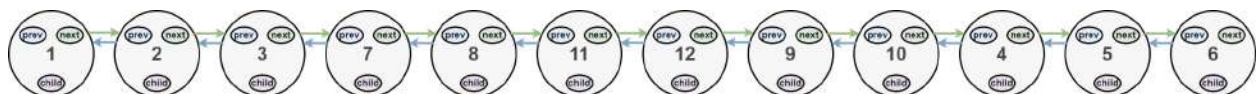
Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

Output: [1,2,3,7,8,11,12,9,10,4,5,6]



Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:



```
Node* flatten(Node* head) {
    if (!head) return nullptr;
```

```

        flattenDFS(head);
        return head;
    }
    // returns the tail after flattening
    Node* flattenDFS(Node* node) {
        Node* curr = node;
        Node* last = nullptr;
        while (curr) {
            Node* nextNode = curr->next;
            // If current node has a child
            if (curr->child) {
                // Flatten the child list
                Node* childTail = flattenDFS(curr->child);
                // Insert child between curr and curr->next
                curr->next = curr->child;
                curr->child->prev = curr;
                curr->child = nullptr;
                // Connect child's tail to nextNode
                if (nextNode) {
                    childTail->next = nextNode;
                    nextNode->prev = childTail;
                }
                last = childTail;
            } else {
                last = curr;
            }
            curr = nextNode;
        }
        return last;
    }
}

```

# GenDS-MatrixManipulation

## General DS - Matrix Manipulation

**Description:** Problems involving 2D arrays (matrices) are often solved using row-column traversal or manipulation based on matrix properties.

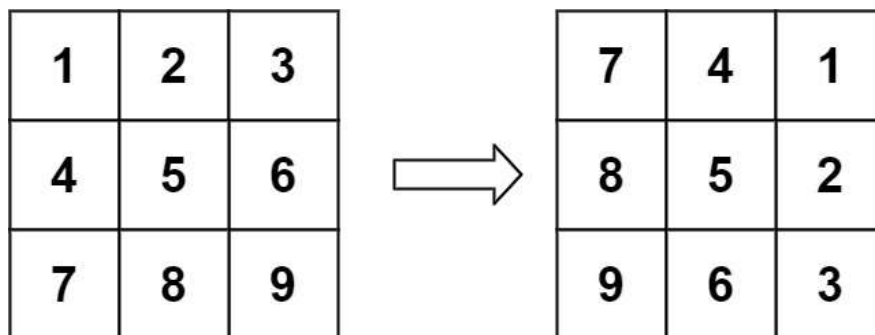
- [Rotate Image](#)
- [Spiral Matrix](#)
- [Set Matrix Zeroes](#)
- [Game of Life](#)

You are given an  $n \times n$  2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Input: matrix = `[[1,2,3],[4,5,6],[7,8,9]]`

Output: `[[7,4,1],[8,5,2],[9,6,3]]`



```
void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    // Step 1: transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            swap(matrix[i][j], matrix[j][i]);
        }
    }
    // Step 2: reverse each row
    for (int i = 0; i < n; i++) {
        reverse(matrix[i].begin(), matrix[i].end());
    }
}
```

Given an m x n matrix, return all elements of the matrix in spiral order.

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

|       |     |     |
|-------|-----|-----|
| 1 →   | 2 → | 3 ↓ |
| 4 →   | 5   | 6 ↓ |
| ↑ 7 ← | 8 ← | 9   |

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
    int top = 0, bottom = matrix.size() - 1;
    int left = 0, right = matrix[0].size() - 1;
    vector<int> ans;
    while (top <= bottom && left <= right) {
        for (int i = left; i <= right; i++)
            ans.push_back(matrix[top][i]);
        top++;
        for (int i = top; i <= bottom; i++)
            ans.push_back(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int i = right; i >= left; i--)
                ans.push_back(matrix[bottom][i]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--)
                ans.push_back(matrix[i][left]);
            left++;
        }
    }
    return ans;
}
```

Given an  $m \times n$  integer matrix `matrix`, if an element is 0, set its entire row and column to 0's. You must do it in place.

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`.

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 |   | 1 | 0 | 1 |
| 1 | 0 | 1 | → | 0 | 0 | 0 |
| 1 | 1 | 1 |   | 1 | 0 | 1 |

```
void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size(), n = matrix[0].size();
    bool firstColZero = false;
    for (int i = 0; i < m; i++) {
        if (matrix[i][0] == 0) firstColZero = true;
        for (int j = 1; j < n; j++) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
    if (matrix[0][0] == 0) {
        for (int j = 0; j < n; j++) matrix[0][j] = 0;
    }
    if (firstColZero) {
        for (int i = 0; i < m; i++) matrix[i][0] = 0;
    }
}
```

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an  $m \times n$  grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

Any live cell with fewer than two live neighbors dies as if caused by under-population.

Any live cell with two or three live neighbors lives on to the next generation.

Any live cell with more than three live neighbors dies, as if by over-population.

Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the  $m \times n$  grid board. In this process, births and deaths occur simultaneously. Given the current state of the board, update the board to reflect its next state.

Note that you do not need to return anything.

Input: board = `[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]`

Output: `[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]`

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 |   | 0 | 0 | 0 |
| 0 | 0 | 1 |   | 1 | 0 | 1 |
| 1 | 1 | 1 | → | 0 | 1 | 1 |
| 0 | 0 | 0 |   | 0 | 1 | 0 |

```
void gameOfLife(vector<vector<int>>& board) {
```



```

int m = board.size(); int n = board[0].size();
int dirs[8][2] = { // Directions (8 neighbors)
    {-1,-1}, {-1,0}, {-1,1}, {0,-1},
    {0,1}, {1,-1}, {1,0}, {1,1}
};
for (int i = 0; i < m; ++i) { // Step 1: Mark transitions
    for (int j = 0; j < n; ++j) {
        int liveNeighbors = 0;
        // Count live neighbors (1 or 2 means originally alive)
        for (auto &d : dirs) {
            int r = i + d[0];
            int c = j + d[1];
            if (r >= 0 && r < m && c >= 0 && c < n) {
                if (board[r][c] == 1 || board[r][c] == 2)
                    liveNeighbors++;
            }
        }
        // Rule applications with encoded marking
        if (board[i][j] == 1) {
            // live cell dies → mark 2
            if (liveNeighbors < 2 || liveNeighbors > 3)
                board[i][j] = 2;
        } else {
            // dead cell becomes live → mark 3
            if (liveNeighbors == 3)
                board[i][j] = 3;
        }
    }
}
// Step 2: Convert 2→0 and 3→1
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        if (board[i][j] == 2) board[i][j] = 0;
        if (board[i][j] == 3) board[i][j] = 1;
    }
}
}

```

GenDS-BFS

## General DS - Breadth First Search (BFS)

Description: BFS explores nodes level by level using a queue. It is particularly useful for shortest path problems.

- [Shortest Path in Binary Matrix](#)
- [Rotten Oranges](#)
- [Word Ladder: Word Ladder](#)
- [As Far From Land as Possible](#)

Given an  $n \times n$  binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e.,  $(0, 0)$ ) to the bottom-right cell (i.e.,  $(n - 1, n - 1)$ ) such that:

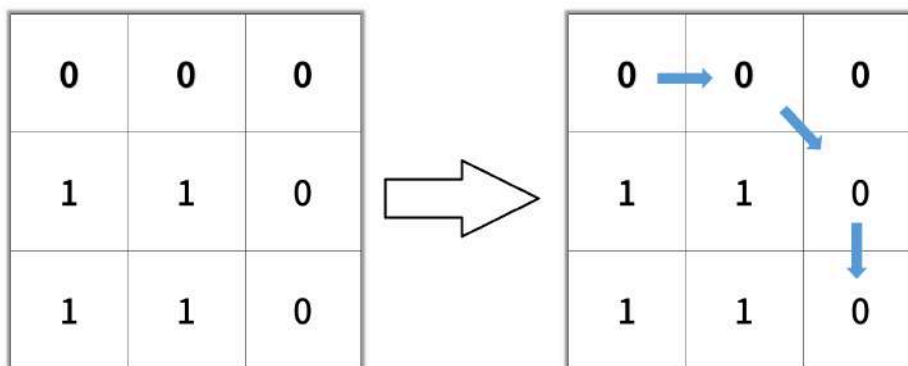
All the visited cells of the path are 0.

All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

Input: grid =  $[[0,0,0],[1,1,0],[1,1,0]]$

Output: 4



```
int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
```

```

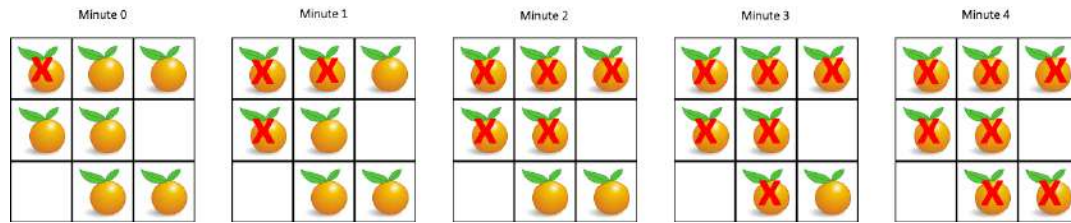
int n = grid.size();
if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
queue<pair<int,int>> q;
q.push(make_pair(0, 0));
grid[0][0] = 1; // distance
int dirs[8][2] = {
    {1,0}, {-1,0}, {0,1}, {0,-1},
    {1,1}, {1,-1}, {-1,1}, {-1,-1}
};
while (!q.empty()) {
    pair<int,int> p = q.front();
    q.pop();
    int r = p.first, c = p.second;
    int dist = grid[r][c];
    if (r == n-1 && c == n-1) return dist;
    for (int i=0;i<8;i++) {
        int nr = r + dirs[i][0];
        int nc = c + dirs[i][1];
        if (nr>=0 && nr<n && nc>=0 && nc<n && grid[nr][nc]==0) {
            grid[nr][nc] = dist + 1;
            q.push(make_pair(nr, nc));
        }
    }
}
return -1;
}

```

You are given an  $m \times n$  grid where each cell can have one of three values: 0 representing an empty cell, 1 representing a fresh orange, or 2 representing a rotten orange. Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Input: grid = [[2,1,1],[1,1,0],[0,1,1]]

Output: 4



```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int,int>> q; int fresh = 0;
    for (int i=0;i<m;i++) {
        for (int j=0;j<n;j++) {
            if (grid[i][j] == 2) q.push(make_pair(i,j));
            else if (grid[i][j] == 1) fresh++;
        }
    }
    if (fresh == 0) return 0;
    int minutes = -1;
    int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!q.empty()) {
        int size = q.size();
        minutes++;
        while (size-->0) {
            pair<int,int> p = q.front();
            q.pop();
            int r = p.first, c = p.second;
            for (int i=0;i<4;i++) {
                int nr = r + dirs[i][0];
                int nc = c + dirs[i][1];
                if (nr>=0 && nr<m && nc>=0 && nc<n && grid[nr][nc]
== 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    q.push(make_pair(nr,nc));
                }
            }
        }
    }
    return (fresh == 0 ? minutes : -1);
}
```

[hard]

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` -> `s1` -> `s2` -> ... -> `sk` such that: Every adjacent pair of words differs by a single letter. Every `si` for  $1 \leq i \leq k$  is in `wordList`. Note that `beginWord` does not need to be in `wordList`.

`sk == endWord`. Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists. Input: `beginWord` = "hit", `endWord` = "cog", `wordList` = ["hot","dot","dog","lot","log","cog"]. Output: 5. Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

```
int ladderLength(string beginWord, string endWord, vector<string>&
wordList) {
    unordered_set<string> dict(wordList.begin(), wordList.end());
    if (!dict.count(endWord)) return 0;
    queue<pair<string,int>> q;
    q.push(make_pair(beginWord, 1));
    while (!q.empty()) {
        pair<string,int> p = q.front();
        q.pop();
        string word = p.first; int steps = p.second;
        if (word == endWord) return steps;
        for (int i=0;i<word.size();i++) {
            string temp = word;
            for (char c='a'; c<='z'; c++) {
                temp[i] = c;
                if (dict.count(temp)) {
                    dict.erase(temp);
                    q.push(make_pair(temp, steps + 1));
                }
            }
        }
    }
    return 0;
}
```

Given an  $n \times n$  grid containing only values 0 and 1, where 0 represents water and 1 represents land, find a water cell such that its distance to the nearest land cell is maximized, and return the distance. If no land or water exists in the grid, return -1. The distance used in this problem is the Manhattan distance: the distance between two cells  $(x_0, y_0)$  and  $(x_1, y_1)$  is  $|x_0 - x_1| + |y_0 - y_1|$ . Input: `grid = [[1,0,1],[0,0,0],[1,0,1]]`. Output: 2  
Explanation: The cell (1, 1) is as far as possible from all the land with distance 2.

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 0 | 0 |
| 1 | 0 | 1 |

```
int maxDistance(vector<vector<int>>& grid) {
    int n = grid.size(); queue<pair<int,int>> q;
    for (int i=0;i<n;i++) {
        for (int j=0;j<n;j++) {
            if (grid[i][j] == 1) {
                q.push(make_pair(i,j));
            }
        }
    }
    if (q.empty() || q.size() == n*n) return -1;
    int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
    int dist = -1;
    while (!q.empty()) {
        int size = q.size(); dist++;
        while (size-->0) {
            pair<int,int> p = q.front();
            q.pop();
            int r = p.first, c = p.second;
            for (int i=0;i<4;i++) {
                int nr = r + dirs[i][0];
                int nc = c + dirs[i][1];
                if (nr<0 || nr>=n || nc<0 || nc>=n || grid[nr][nc]
== 0) {
                    grid[nr][nc] = 1; // visited
                    q.push(make_pair(nr,nc));
                }
            }
        }
    }
    return dist;
}
```

GenDS-DFS



## General DS - Depth First Search (DFS)

Description: DFS explores as far as possible along a branch before backtracking. It's useful for graph traversal, pathfinding, and connected components.

- [Number of Closed Islands](#)
- [Coloring a Border](#)
- [DFS from boundary: Number of Enclaves](#)
- [Shortest time: Time Needed to Inform all Employees](#)
- [Cyclic Find: Find Eventual Safe States](#)

Given a 2D grid consists of 0s (land) and 1s (water). An island is a maximal 4-directionally connected group of 0s and a closed island is an island totally (all left, top, right, bottom) surrounded by 1s.

Return the number of closed islands.

Input: grid =

```
[[1,1,1,1,1,1,1,0],[1,0,0,0,0,1,1,0],[1,0,1,0,1,1,1,0],[1,0,0,0,0,1,0,1],[1,1,1,1,1,1,1,0]]
```

Output: 2

Explanation:

Islands in gray are closed because they are completely surrounded by water (group of 1s).

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |

```

int closedIsland(vector<vector<int>>& grid) {
    int res = 0;
    for (int i = 0; i < grid.size(); i++){
        for (int j = 0; j < grid[0].size(); j++){
            if (grid[i][j] == 0){
                res += dfs(grid, i, j) ? 1 : 0;
            }
        }
    }
    return res;
}

bool dfs(vector<vector<int>>& g, int i, int j){
    if (i < 0 || j < 0 || i >= g.size() || j >= g[0].size()){
        return false;
    }
    if (g[i][j] == 1 || g[i][j] == 2){
        return true;
    }
    g[i][j] = 2;
    /* Note: Cannot use: `return dfs(g, i+1, j) && dfs(g, i, j+1) &&
    dfs(g, i-1, j) && dfs(g, i, j-1);` as if any first dfs() returns
    false, following ones will not execute, hence won't have chance to
    mark them as 0s/1s - Short circuit evaluation
    */
    bool down = dfs(g, i+1, j);
    bool right = dfs(g, i, j+1);
    bool up = dfs(g, i-1, j);
    bool left = dfs(g, i, j-1);
    return up && down && left && right;
}

```

You are given an  $m \times n$  integer matrix grid, and three integers row, col, and color. Each value in the grid represents the color of the grid square at that location.

Two squares are called adjacent if they are next to each other in any of the 4 directions.

Two squares belong to the same connected component if they have the same color and they are adjacent.

The border of a connected component is all the squares in the connected component that are either adjacent to (at least) a square not in the component, or on the boundary of the grid (the first or last row or column).

You should color the border of the connected component that contains the square grid[row][col] with color.

Return the final grid.

Input: grid = [[1,1,1],[1,1,1],[1,1,1]], row = 1, col = 1, color = 2. Output: [[2,2,2],[2,1,2],[2,2,2]]

```
void dfs(int row, int col, int color, int orig, vector<vector<int>>&
grid, vector<vector<int>> &vis){
    int m= grid.size(), n = grid[0].size();
    vis[row][col]=1;
    int a[] = {0, -1,0,1}; int b[] = {1, 0, -1, 0};
    for(int k=0; k<4 ; k++){
        int newr = row + a[k];
        int newc = col + b[k];
        if(newr<0 || newc<0 || newr>=m || newc>=n){
            grid[row][col] = color;
            continue;
        }
        if(vis[newr][newc]) continue;
        if(grid[newr][newc] == orig){ // if it's connected
            dfs(newr, newc, color, orig, grid, vis);
        }
        else{
            grid[row][col] = color;
        }
    }
    return;
}
```

```

vector<vector<int>> colorBorder(vector<vector<int>>& grid, int row,
int col, int color) {
    int m= grid.size(), n = grid[0].size();
    vector<vector<int>> vis(m , vector<int>(n,0));
    dfs(row, col, color, grid[row][col], grid, vis);
    return grid;
}

```

You are given an  $m \times n$  binary matrix `grid`, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid. Return the number of land cells in `grid` for which we cannot walk off the boundary of the grid in any number of moves.

Input: `grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]`

Output: 3

Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |

```

void dfs(int i, int j, vector<vector<int>>& grid) {
    int n = grid.size(), m = grid[0].size();
    if (i < 0 || j < 0 || i >= n || j >= m) return;
    if (grid[i][j] != 1) return; // water or already visited
    grid[i][j] = 2; // Mark land as visited (escaped land)
    dfs(i - 1, j, grid);
    dfs(i + 1, j, grid);
    dfs(i, j - 1, grid);
    dfs(i, j + 1, grid);
}

int numEnclaves(vector<vector<int>>& grid) {

```

```

int n = grid.size(), m = grid[0].size();
// Traverse boundary
for (int j = 0; j < m; j++) {
    dfs(0, j, grid);
    dfs(n - 1, j, grid);
}
for (int i = 0; i < n; i++) {
    dfs(i, 0, grid);
    dfs(i, m - 1, grid);
}
// Count land that never got visited
int ans = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == 1)
            ans++;
    }
}
return ans;
}

```

A company has  $n$  employees with a unique ID for each employee from 0 to  $n - 1$ . The head of the company is the one with headID.

Each employee has one direct manager given in the manager array where  $manager[i]$  is the direct manager of the  $i$ -th employee,  $manager[headID] = -1$ . Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

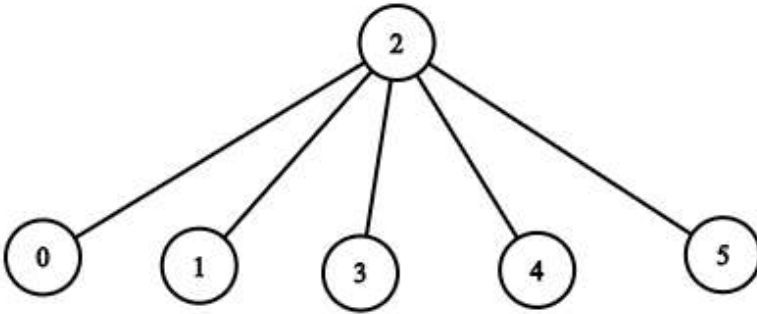
The  $i$ -th employee needs  $informTime[i]$  minutes to inform all of his direct subordinates (i.e., After  $informTime[i]$  minutes, all his direct subordinates can start spreading the news).

Return the number of minutes needed to inform all the employees about the urgent news.

Input:  $n = 6$ ,  $headID = 2$ ,  $manager = [2,2,-1,2,2,2]$ ,  $informTime = [0,0,1,0,0,0]$

Output: 1

Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all. The tree structure of the employees in the company is shown.



Other eg:

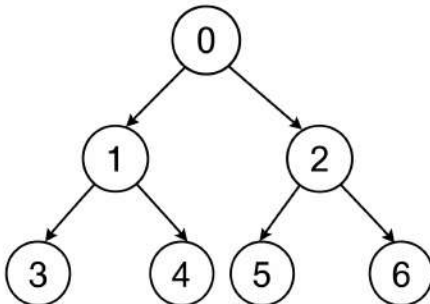
Example 1:

Input:  $n = 1$ ,  $headID = 0$ ,  $manager = [-1]$ ,  $informTime = [0]$   
Output: 0  
Explanation: The head of the company is the only employee in the company.

Example 2:

Input:  $n = 6$ ,  $headID = 2$ ,  $manager = [2, 2, -1, 2, 2, 2]$ ,  $informTime = [0, 0, 1, 0, 0, 0]$   
Output: 1  
Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all.  
The tree structure of the employees in the company is

Other eg 2: For  $n=7$ ,  $headID=0$ ,  $manager=[-1,0,0,1,1,2,2]$ ,  $informTime=[2,3,1,0,0,0,0]$



```
int dfs(int u, vector<vector<int>>& adj, vector<int>& informTime) {  
    int maxTime = 0;
```

```

    for (int v : adj[u]) {
        maxTime = max(maxTime, dfs(v, adj, informTime));
    }
    return informTime[u] + maxTime;
}
int numOfMinutes(int n, int headID, vector<int>& manager,
vector<int>& informTime) {
    vector<vector<int>> adj(n);
    for (int i = 0; i < n; i++) {
        if (manager[i] != -1)
            adj[manager[i]].push_back(i); // Build tree
    }
    return dfs(headID, adj, informTime);
}

```

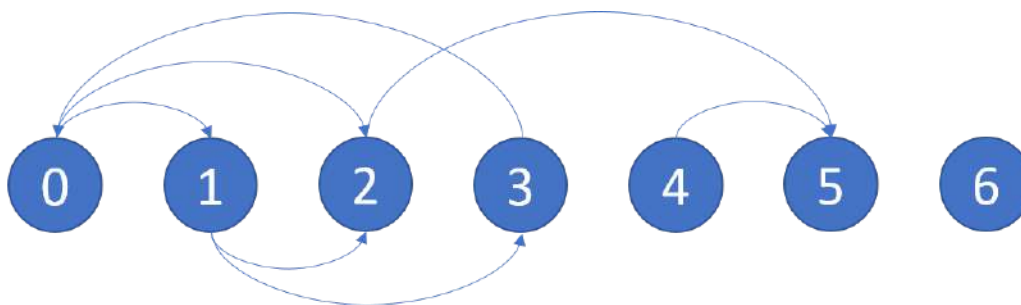
There is a directed graph of  $n$  nodes with each node labeled from  $0$  to  $n - 1$ . The graph is represented by a  $0$ -indexed  $2D$  integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node  $i$ , meaning there is an edge from node  $i$  to each node in `graph[i]`. A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[[]]`. Output: `[2,4,5,6]`

Explanation: The given graph is shown above.

Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them. Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.



```

bool dfs(int node, vector<vector<int>>& graph, vector<int>& vis,
vector<int>& dfsVis, vector<int>& safe) {

```

```

vis[node] = 1;
dfsVis[node] = 1;
for (int nxt : graph[node]) {
    if (!vis[nxt]) {
        bool hasCycle = dfs(nxt, graph, vis, dfsVis, safe);
        if (hasCycle) {
            return true; // cycle reachable → unsafe
        }
    }
    else if (dfsVis[nxt]) {
        return true; // back-edge → cycle
    }
}
dfsVis[node] = 0; // backtrack
safe[node] = 1; // no cycle found from this node
return false; // this path is safe
}

vector<int> eventualSafeNodes(vector<vector<int>>& graph) {
    // Similar logic to finding cycle in a graph
    int n = graph.size();
    vector<int> vis(n, 0), dfsVis(n, 0), safe(n, 0);
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            dfs(i, graph, vis, dfsVis, safe);
        }
    }
    vector<int> ans;
    for (int i = 0; i < n; i++) {
        if (safe[i]) ans.push_back(i);
    }
    return ans;
}

```



# GenDS-Backtracking

## General DS - Backtracking

Description: Backtracking helps in problems where you need to explore all potential solutions, such as solving puzzles, generating combinations, or finding paths.

- [Permutation ii](#)
- [Combination Sum](#)
- [Generate Parenthesis](#)
- [N-Queens](#)
- [Sudoku Solver](#)
- [Palindrome Partitioning](#)
- [Word Search: Word Search](#)

Given a collection of numbers, `nums`, that might contain duplicates, return all possible unique permutations in any order.

Input: `nums = [1,1,2]`

Output:

```
[[1,1,2],  
 [1,2,1],  
 [2,1,1]]
```

```
vector<vector<int>> ans; vector<int> temp; vector<bool> used;  
void backtrack(vector<int>& nums) {  
    if (temp.size() == nums.size()) {  
        ans.push_back(temp);  
        return;  
    }  
    for (int i = 0; i < nums.size(); i++) {  
        if (used[i]) continue; // Skip duplicates  
        if (i > 0 && nums[i] == nums[i-1] && !used[i-1])  
            continue;  
        used[i] = true;  
        temp.push_back(nums[i]);  
        backtrack(nums);  
        used[i] = false;  
        temp.pop_back();  
    }  
}
```

```

vector<vector<int>> permuteUnique(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    used.assign(nums.size(), false);
    backtrack(nums);
    return ans;
}

```

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input. Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]. Explanation: 2 and 3 are candidates, and  $2 + 2 + 3 = 7$ . Note that 2 can be used multiple times. 7 is a candidate, and  $7 = 7$ . These are the only two combinations.

```

vector<vector<int>> res; vector<int> curr;
void dfs(vector<int>& c, int target, int idx) {
    if (target == 0) {
        res.push_back(curr);
        return;
    }
    if (idx == c.size() || target < 0) return;
    // include
    curr.push_back(c[idx]);
    dfs(c, target - c[idx], idx);
    curr.pop_back();
    // skip
    dfs(c, target, idx + 1);
}
vector<vector<int>> combinationSum(vector<int>& c, int target) {
    dfs(c, target, 0);
    return res;
}

```

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Input: n = 3

Output: ["((()))","(()())","(())()","()(())","()()()"]

```
vector<string> ans;
void backtrack(int open, int close, string s, int n) {
    if (s.size() == 2*n) {
        ans.push_back(s);
        return;
    }
    if (open < n) backtrack(open + 1, close, s + "(", n);
    if (close < open) backtrack(open, close + 1, s + ")", n);
}
vector<string> generateParenthesis(int n) {
    backtrack(0, 0, "", n);
    return ans;
}
```

[hard]

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

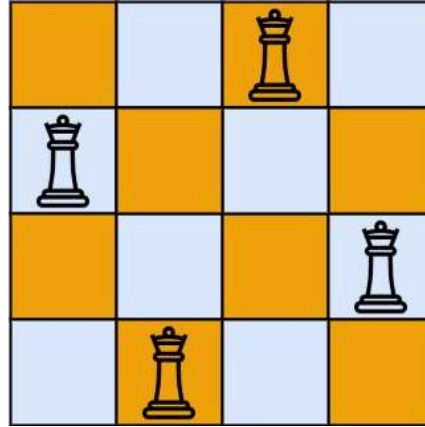
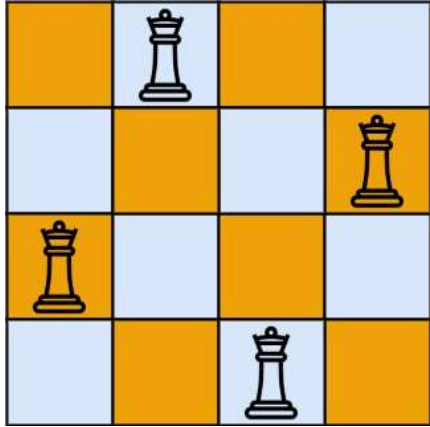
Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Input: n = 4

Output:

[[".Q...", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above



```
vector<vector<string>> res;
vector<string> board;
vector<bool> col, diag1, diag2;
void dfs(int r, int n) {
    if (r == n) {
        res.push_back(board);
        return;
    }
    for (int c = 0; c < n; c++) {
        if (col[c] || diag1[r+c] || diag2[r-c+n-1]) continue;
        board[r][c] = 'Q';
        col[c] = diag1[r+c] = diag2[r-c+n-1] = true;
        dfs(r + 1, n);
        board[r][c] = '.';
        col[c] = diag1[r+c] = diag2[r-c+n-1] = false;
    }
}
vector<vector<string>> solveNQueens(int n) {
    board.assign(n, string(n, '.')); /*Equivalent to: board.clear();
    then for (int i = 0; i < n; i++) board.push_back(string(n,
    '.'));...*/
    col.assign(n, false);
    diag1.assign(2*n, false);
    diag2.assign(2*n, false);
    dfs(0, n);
    return res;
}
```

[hard]

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

Each of the digits 1-9 must occur exactly once in each row.

Each of the digits 1-9 must occur exactly once in each column.

Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Input: board =

```
[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".",
".","."],[".","9","8",".",".",".",".","6","."],["8",".",".",".","6",
".",".",".",3"],["4",".",".","8",".","3",".",".",1"],["7",".",".",
".","2",".",".",6"],[".","6",".",".",",",",2","8","."],[".",
".",".",4","1","9",".",",",5"],[".",",",",8",".",",7","9
"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3",
"4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6",
"1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3",
"9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2",
"8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9
"]]
```

Explanation: The input board is shown above and the only valid solution is shown below:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 4 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 2 | 1 | 9 | 5 | 3 | 4 | 8 |
| 1 | 9 | 8 | 3 | 4 | 2 | 5 | 6 | 7 |
| 8 | 5 | 9 | 7 | 6 | 1 | 4 | 2 | 3 |
| 4 | 2 | 6 | 8 | 5 | 3 | 7 | 9 | 1 |
| 7 | 1 | 3 | 9 | 2 | 4 | 8 | 5 | 6 |
| 9 | 6 | 1 | 5 | 3 | 7 | 2 | 8 | 4 |
| 2 | 8 | 7 | 4 | 1 | 9 | 6 | 3 | 5 |
| 3 | 4 | 5 | 2 | 8 | 6 | 1 | 7 | 9 |

```
bool isValid(vector<vector<char>>& b, int r, int c, char ch) {
```

```

        for (int i = 0; i < 9; i++) {
            if (b[r][i] == ch) return false;
            if (b[i][c] == ch) return false;
            int br = 3*(r/3) + i/3;
            int bc = 3*(c/3) + i%3;
            if (b[br][bc] == ch) return false;
        }
        return true;
    }
    bool solve(vector<vector<char>>& b) {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (b[r][c] == '.') {
                    for (char ch = '1'; ch <= '9'; ch++) {
                        if (isValid(b, r, c, ch)) {
                            b[r][c] = ch;
                            if (solve(b)) return true;
                            b[r][c] = '.';
                        }
                    }
                    return false;
                }
            }
        }
        return true;
    }
}

void solveSudoku(vector<vector<char>>& board) {
    solve(board);
}

```

**Given a string *s*, partition *s* such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of *s*.**

**Input:** *s* = "aab"

**Output:** `[["a","a","b"],["aa","b"]]`

`vector<vector<string>> ans;`

```

vector<string> curr;
bool isPal(string &s, int l, int r) {
    while (l < r) {
        if (s[l] != s[r]) return false;
        l++; r--;
    }
    return true;
}
void dfs(string &s, int idx) {
    if (idx == s.size()) {
        ans.push_back(curr);
        return;
    }
    for (int end = idx; end < s.size(); end++) {
        if (isPal(s, idx, end)) {
            curr.push_back(s.substr(idx, end - idx + 1));
            dfs(s, end + 1);
            curr.pop_back();
        }
    }
}
vector<vector<string>> partition(string s) {
    dfs(s, 0);
    return ans;
}

```

Given an  $m \times n$  grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Input: board =

[[ "A","B","C","E"],[ "S","F","C","S"],[ "A","D","E","E"]], word = "ABCCED"

Output: true



|   |   |   |   |
|---|---|---|---|
| A | B | C | E |
| S | F | C | S |
| A | D | E | E |

```

bool dfs(vector<vector<char>>& b, string& w, int i, int j, int idx)
{
    if (idx == w.size()) return true;
    if (i < 0 || j < 0 || i >= b.size() || j >= b[0].size()) return
false;
    if (b[i][j] != w[idx]) return false;
    char temp = b[i][j];
    b[i][j] = '#'; // mark visited
    bool found = dfs(b, w, i+1, j, idx+1)
                || dfs(b, w, i-1, j, idx+1)
                || dfs(b, w, i, j+1, idx+1)
                || dfs(b, w, i, j-1, idx+1);
    b[i][j] = temp; // unmark
    return found;
}

bool exist(vector<vector<char>>& board, string word) {
    int m = board.size(), n = board[0].size();
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (dfs(board, word, i, j, 0))
                return true;
        }
    }
    return false;
}

```

# GenDS-BinarySearch

## General DS - Modified Binary Search

Description: A modified version of binary search that applies to rotated arrays, unsorted arrays, or specialized conditions.

- [Search in Rotated Sorted Array](#)
- [Find Minimum in Rotated Sorted Array](#)
- [Find Peak Element](#)
- [Single element in a sorted array](#)
- [Minimum Time to Arrive on Time](#)
- [Capacity to Ship Packages within 'd' Days](#)
- [Koko Eating Bananas](#)
- [Find in Mountain Array](#)
- [Median of Two Sorted Arrays](#)
- [\[Self added v2\] Magnetic force between two balls](#)
- [\[Self added v2\] Split Array Largest Sum](#)

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, `nums` is rotated at an unknown pivot index `k` ( $0 \leq k < \text{nums.length}$ ) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index 5 and become `[4,5,6,6,7,0,1,2,4,4]`.

Given the array `nums` after the rotation and an integer `target`, return `true` if `target` is in `nums`, or `false` if it is not in `nums`.

You must decrease the overall operation steps as much as possible.

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0`

Output: `true`

```

bool search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return true;
        // Handle duplicates
        if (nums[l] == nums[mid] && nums[mid] == nums[r]) {
            l++;
            r--;
        }
        // Left is sorted
        else if (nums[l] <= nums[mid]) {
            if (nums[l] <= target && target < nums[mid])
                r = mid - 1;
            else
                l = mid + 1;
        }
        // Right is sorted
        else {
            if (nums[mid] < target && target <= nums[r])
                l = mid + 1;
            else
                r = mid - 1;
        }
    }
    return false;
}

```

Suppose an array of length  $n$  sorted in ascending order is rotated between 1 and  $n$  times. For example, the array `nums = [0,1,2,4,5,6,7]` might become: `[4,5,6,7,0,1,2]` if it was rotated 4 times.

`[0,1,2,4,5,6,7]` if it was rotated 7 times. Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`. Given the sorted rotated array `nums` of unique elements, return the minimum element of this array. You must write an algorithm that runs in  $O(\log n)$  time.

Input: `nums = [3,4,5,1,2]`. Output: 1.

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

```

int findMin(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    int ans = INT_MAX;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        // If left part is sorted
        if (nums[l] <= nums[mid]) {
            // the minimum of this sorted region is nums[l]
            ans = min(ans, nums[l]);
            l = mid + 1;
        }
        // Else right part is sorted
        else {
            // the minimum of this sorted region is nums[mid]
            ans = min(ans, nums[mid]);
            r = mid - 1;
        }
    }
    return ans;
}

```

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in  $O(\log n)$  time.

Input: `nums = [1,2,1,3,5,6,4]`. Output: 5. Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

```

int findPeakElement(vector<int> &arr) {
    int n = arr.size();
    if(n == 1) return 0;
    if(arr[0] > arr[1]) return 0;
    if(arr[n-1] > arr[n-2]) return n - 1;
    int low = 1, high = n - 2;
    while(low <= high) {
        int mid = (low + high) / 2;
        // If mid is peak
        if(arr[mid] > arr[mid - 1] && arr[mid] > arr[mid + 1]) {
            return mid;
        }
        // Move to the side that has the bigger slope
        else if(arr[mid] > arr[mid - 1]) {
            low = mid + 1; // you're in an increasing slope
        }
        else {
            high = mid - 1; // you're in a decreasing slope
        }
    }
    return -1;
}

```

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once. Return the single element that appears only once. Your solution must run in  $O(\log n)$  time and  $O(1)$  space.

Input: nums = [1,1,2,3,3,4,4,8,8]. Output: 2

```

int singleNonDuplicate(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (mid % 2 == 1) mid--; // make mid even
        if (nums[mid] == nums[mid + 1]) l = mid + 2;
        else r = mid;
    }
    return nums[l];
}

```

You are given a floating-point number `hour`, representing the amount of time you have to reach the office. To commute to the office, you must take `n` trains in sequential order. You are also given an integer array `dist` of length `n`, where `dist[i]` describes the distance (in kilometers) of the `i`th train ride.

Each train can only depart at an integer hour, so you may need to wait in between each train ride.

For example, if the 1st train ride takes 1.5 hours, you must wait for an additional 0.5 hours before you can depart on the 2nd train ride at the 2 hour mark.

Return the minimum positive integer speed (in kilometers per hour) that all the trains must travel at for you to reach the office on time, or -1 if it is impossible to be on time.

Tests are generated such that the answer will not exceed  $10^7$  and hour will have at most two digits after the decimal point.

Input: `dist = [1,3,2]`, `hour = 6`

Output: 1

Explanation: At speed 1:

- The first train ride takes  $1/1 = 1$  hour.
- Since we are already at an integer hour, we depart immediately at the 1 hour mark. The second train takes  $3/1 = 3$  hours.
- Since we are already at an integer hour, we depart immediately at the 4 hour mark. The third train takes  $2/1 = 2$  hours.
- You will arrive at exactly the 6 hour mark.

```

double calc(vector<int>& dist, int speed) {
    double t = 0.0;
    int n = dist.size();
    for (int i = 0; i < n - 1; i++) {
        t += ceil((double)dist[i] / speed);
    }
    t += (double)dist[n-1] / speed;
    return t;
}

int minSpeedOnTime(vector<int>& dist, double hour) {
    int l = 1, r = INT_MAX/2;
    int ans = -1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (calc(dist, m) <= hour) {
            ans = m;
            r = m - 1;
        } else {
            l = m + 1;
        }
    }
    return ans;
}

```

A conveyor belt has packages that must be shipped from one port to another within  $days$  days. The  $i$ th package on the conveyor belt has a weight of  $weights[i]$ . Each day, we load the ship with packages on the conveyor belt (in the order given by  $weights$ ). We may not load more weight than the maximum weight capacity of the ship. Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within  $days$  days.

Input:  $weights = [1,2,3,4,5,6,7,8,9,10]$ ,  $days = 5$ . Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this: 1st day: 1, 2, 3, 4, 5; 2nd day: 6, 7; 3rd day: 8; 4th day: 9; 5th day: 10. Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.



```

bool canShip(vector<int>& w, int cap, int days) {
    int need = 1, cur = 0;
    for (int x : w) {
        if (x > cap) return false;
        if (cur + x > cap) {
            need++; cur = 0;
        }
        cur += x;
    }
    return need <= days;
}

int shipWithinDays(vector<int>& weights, int days) {
    int l = 1, r = 0;
    for (int x : weights) r += x;
    int ans = r;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (canShip(weights, mid, days)) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    return ans;
}

```

Koko loves to eat bananas. There are  $n$  piles of bananas, the  $i$ th pile has  $\text{piles}[i]$  bananas. The guards have gone and will come back in  $h$  hours. Koko can decide her bananas-per-hour eating speed of  $k$ . Each hour, she chooses some pile of bananas and eats  $k$  bananas from that pile. If the pile has less than  $k$  bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer  $k$  such that she can eat all the bananas within  $h$  hours.

Input:  $\text{piles} = [3,6,7,11]$ ,  $h = 8$ . Output: 4

```

bool canEat(vector<int>& p, int h, int k) {
    long long t = 0;
    for (int x : p) {
        t += (x + k - 1) / k;
    }
    return t <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
    int l = 1, r = INT_MAX/2;
    int ans = r;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (canEat(piles, h, mid)) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    return ans;
}

```

[hard]

You may recall that an array `arr` is a mountain array if and only if:  
`arr.length >= 3`

There exists some `i` with  $0 < i < arr.length - 1$  such that:

`arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`

`arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

Given a mountain array `mountainArr`, return the minimum index such that `mountainArr.get(index) == target`. If such an index does not exist, return -1. You cannot access the mountain array directly. You may only access the array using a `MountainArray` interface:

`MountainArray.get(k)` returns the element of the array at index `k` (0-indexed). `MountainArray.length()` returns the length of the array. Submissions making more than 100 calls to `MountainArray.get` will be judged Wrong Answer. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Input: `mountainArr = [1,2,3,4,5,3,1]`, `target = 3`. Output: 2

**Explanation:** 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.

```
int findPeak(MountainArray &m) {
    int l = 0, r = m.length() - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (m.get(mid) < m.get(mid + 1))
            l = mid + 1;
        else
            r = mid;
    }
    return l;
}

int bs(MountainArray &m, int l, int r, int t, bool asc) {
    while (l <= r) {
        int mid = l + (r - l) / 2;
        int v = m.get(mid);
        if (v == t) return mid;
        if (asc) {
            if (v < t) l = mid + 1;
            else r = mid - 1;
        } else {
            if (v > t) l = mid + 1;
            else r = mid - 1;
        }
    }
    return -1;
}

int findInMountainArray(int target, MountainArray &m) {
    int peak = findPeak(m);
    int left = bs(m, 0, peak, target, true);
    if (left != -1) return left;
    return bs(m, peak + 1, m.length() - 1, target, false);
}
```

**[hard]**

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays. The overall run time complexity should be  $O(\log(m+n))$ .

Input: nums1 = [1,3], nums2 = [2]

Output: 2.00000

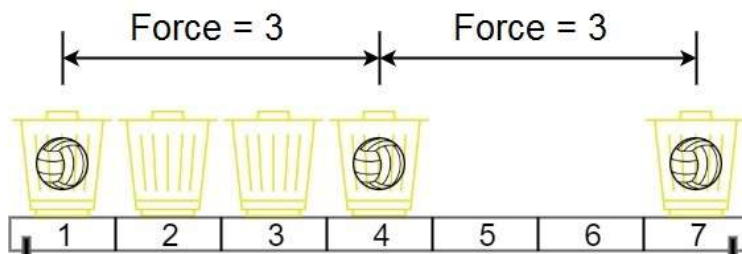
Explanation: merged array = [1,2,3] and median is 2.

```
double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
    if (A.size() > B.size()) return findMedianSortedArrays(B, A);
    int m = A.size(), n = B.size();
    int l = 0, r = m;
    while (l <= r) {
        int i = (l + r) / 2;
        int j = (m + n + 1) / 2 - i;
        int Aleft = (i == 0 ? INT_MIN : A[i-1]);
        int Aright = (i == m ? INT_MAX : A[i]);
        int Bleft = (j == 0 ? INT_MIN : B[j-1]);
        int Bright = (j == n ? INT_MAX : B[j]);
        if (Aleft <= Bright && Bleft <= Aright) {
            if ((m + n) % 2 == 0)
                return (max(Aleft, Bleft) + min(Aright, Bright))/2.0;
            else
                return max(Aleft, Bleft);
        }
        else if (Aleft > Bright)
            r = i - 1;
        else
            l = i + 1;
    }
    return 0.0;
}
```

In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has  $n$  empty baskets, the  $i$ th basket is at `position[i]`, Morty has  $m$  balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum. Rick stated that magnetic force between two different balls at positions  $x$  and  $y$  is  $|x - y|$ . Given the integer array `position` and the integer  $m$ . Return the required force.

Input: `position = [1,2,3,4,7]`,  $m = 3$ . Output: 3

Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs  $[3, 3, 6]$ . The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.



```
bool canPlace(vector<int>& pos, int m, int dist) {
```

```

    int count = 1;
    int last = pos[0];
    for (int i = 1; i < pos.size(); i++) {
        if (pos[i] - last >= dist) {
            count++;
            last = pos[i];
            if (count >= m) return true;
        }
    }
    return false;
}

int maxDistance(vector<int>& position, int m) {
    sort(position.begin(), position.end());
    int low = 1; int high = position.back() - position[0];
    int ans = 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (canPlace(position, m, mid)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

```

**[hard]**

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized. Return the minimized largest sum of the split.

A subarray is a contiguous part of the array.

Input: `nums = [7,2,5,10,8]`, `k = 2`. Output: 18

Explanation: There are four ways to split `nums` into two subarrays. The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

```
bool canSplit(vector<int>& nums, int k, int limit) {
```

```

    int parts = 1;
    int curr = 0;
    for (int x : nums) {
        if (curr + x <= limit) {
            curr += x;
        } else {
            parts++;
            curr = x;
            if (parts > k) return false;
        }
    }
    return true;
}

int splitArray(vector<int>& nums, int k) {
    int low = *max_element(nums.begin(), nums.end());
    int high = accumulate(nums.begin(), nums.end(), 0);
    int ans = high;
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (canSplit(nums, k, mid)) {
            ans = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return ans;
}

```

GenDS-BitwiseXOR



## General DS - Bitwise XOR

Description: XOR is a powerful bitwise operator that can solve problems like finding single numbers or efficiently pairing elements.

- [Single Number II](#)
- [Single Number III](#)
- [Find the Original array of Prefix XOR](#)
- [XOR Queries of a Subarray](#)

Given an integer array `nums` where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Input: `nums = [2,2,3,2]`

Output: 3

```
int singleNumber(vector<int>& nums) {
    // For each bit (0-31), count how many numbers have that bit
    // set. If a bit count % 3 == 1 → that bit belongs to the single
    // number.
    int result = 0;
    for (int bit = 0; bit < 32; bit++) {
        int count = 0;
        for (int x: nums) {
            if (x & (1 << bit)) count++;
        }
        if (count % 3 != 0) {
            result |= (1 << bit);
        }
    }
    return result;
}
```

Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Input: `nums = [1,2,1,3,2,5]`

Output: `[3,5]`

Explanation: `[5, 3]` is also a valid answer.

```
vector<int> singleNumber(vector<int>& nums) {  
    // Let xorAll = a ^ b (a & b are the unique numbers). Find a  
    rightmost set bit (mask). Split nums into 2 groups based on this bit  
    and XOR separately.  
    int xr = 0;  
    for (int x : nums) xr ^= x;  
    int mask = xr & (-xr); // rightmost set bit  
    int a = 0, b = 0;  
    for (int x : nums) {  
        if (x & mask) a ^= x;  
        else b ^= x;  
    }  
    return {a, b};  
}
```

You are given an integer array `pref` of size `n`. Find and return the array `arr` of size `n` that satisfies:

`pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i]`.

Note that `^` denotes the bitwise-xor operation.

It can be proven that the answer is unique.

Input: `pref = [5,2,0,3,1]`

Output: `[5,7,2,3,2]`

Explanation: From the array `[5,7,2,3,2]` we have the following:

- `pref[0] = 5.`
- `pref[1] = 5 ^ 7 = 2.`
- `pref[2] = 5 ^ 7 ^ 2 = 0.`
- `pref[3] = 5 ^ 7 ^ 2 ^ 3 = 3.`
- `pref[4] = 5 ^ 7 ^ 2 ^ 3 ^ 2 = 1.`

```

vector<int> findArray(vector<int>& pref) {
    // Observe: pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i], arr[i] =
    pref[i] ^ pref[i-1]
    int n = pref.size();
    vector<int> arr(n);
    arr[0] = pref[0];
    for (int i = 1; i < n; i++) {
        arr[i] = pref[i] ^ pref[i-1];
    }
    return arr;
}

```

You are given an array `arr` of positive integers. You are also given the array `queries` where `queries[i] = [lefti, righti]`.

For each query `i` compute the XOR of elements from `lefti` to `righti` (that is, `arr[lefti] XOR arr[lefti + 1] XOR ... XOR arr[righti]` ).

Return an array `answer` where `answer[i]` is the answer to the `i`th query. Input: `arr = [1,3,4,8]`, `queries = [[0,1],[1,2],[0,3],[3,3]]`

Output: `[2,7,14,8]`. Explanation:

The binary representation of the elements in the array are:

`1 = 0001`, `3 = 0011`, `4 = 0100`, `8 = 1000`. The XOR values for queries are: `[0,1] = 1 xor 3 = 2`, `[1,2] = 3 xor 4 = 7`, `[0,3] = 1 xor 3 xor 4 xor 8 = 14`, `[3,3] = 8`.

```

vector<int> xorQueries(vector<int>& arr, vector<vector<int>>&
queries) {
    int n = arr.size();
    vector<int> px(n);
    px[0] = arr[0];
    for (int i = 1; i < n; i++) px[i] = px[i-1] ^ arr[i];
    vector<int> ans;
    for (auto &q : queries) {
        int l = q[0], r = q[1];
        if (l == 0) ans.push_back(px[r]);
        else ans.push_back(px[r] ^ px[l-1]);
    }
    return ans;
}

```

# Heaps-TopKElements

## Heaps - Top 'K' Elements

**Description:** This pattern uses heaps or quickselect to efficiently find the top 'K' largest/smallest elements from a dataset.

- [Top K Frequent Elements](#)
- [Kth Largest Element](#)
- [Ugly Number ii](#)
- [K Closest Points to Origin](#)

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`. Output: `[1,2]`

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for (int x : nums) freq[x]++;
    // min-heap of {frequency, value}
    priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;
    for (auto &p : freq) {
        pq.push({p.second, p.first});
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> ans;
    while (!pq.empty()) {
        ans.push_back(pq.top().second);
        pq.pop();
    }
    return ans;
}
```

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

```

int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int x : nums) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    return pq.top();
}

```

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Given an integer n, return the nth ugly number.

Input: n = 10

Output: 12

Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.

```

int nthUglyNumber(int n) {
    vector<int> ugly(n);
    ugly[0] = 1;
    int i2 = 0, i3 = 0, i5 = 0;
    int n2 = 2, n3 = 3, n5 = 5;
    for (int i = 1; i < n; i++) {
        int next = min(n2, min(n3, n5));
        ugly[i] = next;
        if (next == n2) n2 = ugly[++i2] * 2;
        if (next == n3) n3 = ugly[++i3] * 3;
        if (next == n5) n5 = ugly[++i5] * 5;
    }
    return ugly[n - 1];
}

```

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0). The distance between two points on the X-Y plane is the Euclidean distance (i.e.,  $\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$ ). You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Input: points = [[1,3],[-2,2]], k = 1. Output: [[-2,2]]

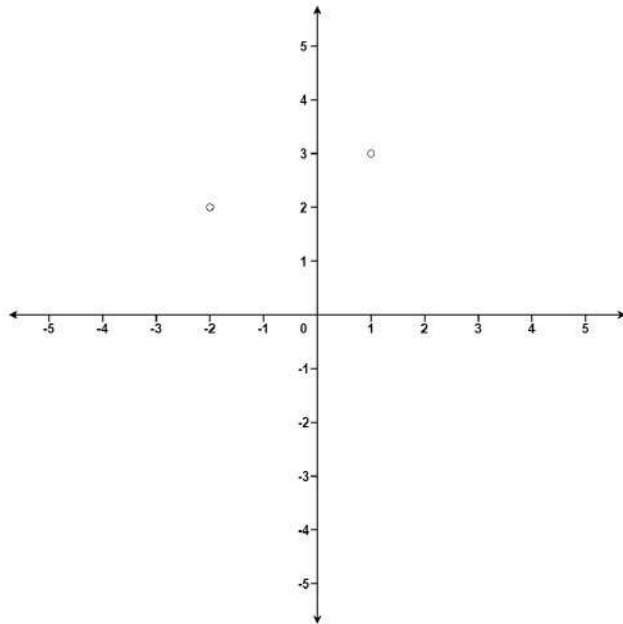
### Explanation:

The distance between (1, 3) and the origin is  $\sqrt{10}$ .

The distance between (-2, 2) and the origin is  $\sqrt{8}$ .

Since  $\sqrt{8} < \sqrt{10}$ , (-2, 2) is closer to the origin.

We only want the closest  $k = 1$  points from the origin, so the answer is just  $[[-2, 2]]$ .



```
vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {  
    // max-heap: {distance, index}  
    priority_queue<pair<long long, int>> pq;  
    for (int i = 0; i < points.size(); i++) {  
        long long x = points[i][0];  
        long long y = points[i][1];  
        long long dist = x*x + y*y;  
        pq.push({dist, i});  
        if ((int)pq.size() > k) pq.pop();  
    }  
    vector<vector<int>> ans;  
    while (!pq.empty()) {  
        int idx = pq.top().second;  
        pq.pop();  
        ans.push_back(points[idx]);  
    }  
    return ans;  
}
```

# Heaps-KWayMerge



## Heaps - K-way Merge

Description: The K-way merge technique uses a heap to efficiently merge multiple sorted lists or arrays.

- [Find K Pairs with Smallest Sums](#)
- [Kth Smallest Element in a Sorted Matrix](#)
- [Merge K Sorted Lists](#)
- [Smallest Range: Smallest Range Covering Elements from K Lists](#)

You are given two integer arrays `nums1` and `nums2` sorted in non-decreasing order and an integer `k`. Define a pair  $(u, v)$  which consists of one element from the first array and one element from the second array. Return the `k` pairs  $(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)$  with the smallest sums. Input: `nums1 = [1,7,11]`, `nums2 = [2,4,6]`, `k = 3`. Output: `[[1,2],[1,4],[1,6]]`. Explanation: The first 3 pairs are returned from the sequence:

```
[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>&
nums2, int k) {
    vector<vector<int>> ans;
    if (nums1.empty() || nums2.empty() || k == 0) return ans;
    priority_queue< vector<int>, vector<vector<int>>,
greater<vector<int>> > pq; // min heap: {sum, i, j}
    // start with (nums1[i], nums2[0])
    for (int i = 0; i < nums1.size() && i < k; i++) {
        pq.push({nums1[i] + nums2[0], i, 0});
    }
    while (!pq.empty() && ans.size() < k) {
        vector<int> top = pq.top(); pq.pop();
        int i = top[1], j = top[2];
        ans.push_back({nums1[i], nums2[j]});
        // push next pair (i, j+1)
        if (j + 1 < nums2.size()) {
            pq.push({nums1[i] + nums2[j+1], i, j+1});
        }
    }
    return ans;
}
```

Given an  $n \times n$  matrix where each of the rows and columns is sorted in ascending order, return the  $k$ th smallest element in the matrix. Note that it is the  $k$ th smallest element in the sorted order, not the  $k$ th distinct element.

You must find a solution with a memory complexity better than  $O(n^2)$ .

Input: matrix = `[[1,5,9],[10,11,13],[12,13,15]]`,  $k = 8$

Output: 13

Explanation: The elements in the matrix are

`[1,5,9,10,11,12,13,13,15]`, and the 8th smallest number is 13

```
int kthSmallest(vector<vector<int>>& matrix, int k) {
    int n = matrix.size();
    // min heap: {value, row, col}
    priority_queue< vector<int>, vector<vector<int>>,
greater<vector<int>> > pq;
    // push the first element of each row
    for (int r = 0; r < n; r++) {
        pq.push({matrix[r][0], r, 0});
    }
    int val = 0;
    while (k--) {
        vector<int> top = pq.top(); pq.pop();
        val = top[0];
        int r = top[1], c = top[2];
        if (c + 1 < n) {
            pq.push({matrix[r][c+1], r, c+1});
        }
    }
    return val;
}
```

[hard]

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Input: lists = [[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Explanation: The linked-lists are:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

merging them into one sorted linked list:

1->1->2->3->4->4->5->6

```
struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val; // min-heap
    }
};

ListNode* mergeKLists(vector<ListNode*>& lists) {
    priority_queue<ListNode*, vector<ListNode*>, Compare> pq;
    for (int i = 0; i < lists.size(); i++) {
        if (lists[i]) pq.push(lists[i]);
    }
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (!pq.empty()) {
        ListNode* node = pq.top(); pq.pop();
        tail->next = node;
        tail = tail->next;
        if (node->next) pq.push(node->next);
    }
    return dummy.next;
}
```

[hard]

You have  $k$  lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the  $k$  lists. We define the range  $[a, b]$  is smaller than range  $[c, d]$  if  $b - a < d - c$  or  $a < c$  if  $b - a == d - c$ .

Input: `nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]`. Output: `[20,24]`. Explanation:

List 1: `[4, 10, 15, 24,26]`, 24 is in range `[20,24]`.

List 2: `[0, 9, 12, 20]`, 20 is in range `[20,24]`.

List 3: `[5, 18, 22, 30]`, 22 is in range `[20,24]`.

```
vector<int> smallestRange(vector<vector<int>>& nums) {
    int k = nums.size();
    // min-heap: {value, listIndex, elementIndex}
    priority_queue<vector<int>, vector<vector<int>>,
greater<vector<int>>> pq;
    int currentMax = INT_MIN;
    // Push first element from each list
    for (int i = 0; i < k; i++) {
        pq.push({nums[i][0], i, 0});
        currentMax = max(currentMax, nums[i][0]);
    }
    int bestL = 0, bestR = INT_MAX;
    while (true) {
        vector<int> top = pq.top(); pq.pop();
        int val = top[0], listIdx = top[1], elemIdx = top[2];
        if (currentMax - val < bestR - bestL) {
            bestL = val; // Update the best range
            bestR = currentMax;
        }
        // Move to next element in this list
        if (elemIdx + 1 == nums[listIdx].size())
            break; // can't expand this list → end
        int nextVal = nums[listIdx][elemIdx + 1];
        pq.push({nextVal, listIdx, elemIdx + 1});
        currentMax = max(currentMax, nextVal);
    }
    return {bestL, bestR};
}
```

# Heaps-TwoHeaps

## Heaps - Two Heaps

**Description:** This pattern uses two heaps (max heap and min heap) to solve problems involving tracking medians and efficiently managing dynamic data.

- [Find Median from Data Stream](#)
- [Sliding Window Median](#)
- [IPO](#)

[hard]

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

For example, for arr = [2,3,4], the median is 3.

For example, for arr = [2,3], the median is  $(2 + 3) / 2 = 2.5$ .

Implement the MedianFinder class:

MedianFinder() initializes the MedianFinder object.

void addNum(int num) adds the integer num from the data stream to the data structure.

double findMedian() returns the median of all elements so far.

Answers within  $10^{-5}$  of the actual answer will be accepted.

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum",  
"findMedian"]
```

```
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
```

```
medianFinder.addNum(1);    // arr = [1]
```

```
medianFinder.addNum(2);    // arr = [1, 2]
```

```
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
```

```
medianFinder.addNum(3);    // arr[1, 2, 3]
```

```
medianFinder.findMedian(); // return 2.0
```

```

priority_queue<int> leftMax; // max-heap
priority_queue<int, vector<int>, greater<int>> rightMin; // min-heap
MedianFinder() {}
void addNum(int num) {
    if (leftMax.empty() || num <= leftMax.top())
        leftMax.push(num);
    else
        rightMin.push(num);
    // balance
    if (leftMax.size() > rightMin.size() + 1) {
        rightMin.push(leftMax.top());
        leftMax.pop();
    }
    else if (rightMin.size() > leftMax.size()) {
        leftMax.push(rightMin.top());
        rightMin.pop();
    }
}
double findMedian() {
    if (leftMax.size() == rightMin.size())
        return (leftMax.top() + rightMin.top()) / 2.0;
    return leftMax.top();
}

```

**[hard]**

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values. For examples, if arr = [2,3,4], the median is 3. For examples, if arr = [1,2,3,4], the median is  $(2 + 3) / 2 = 2.5$ . You are given an integer array nums and an integer k. There is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position. Return the median array for each window in the original array. Answers within  $10^{-5}$  of the actual value will be accepted.

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]

Explanation:

| Window position     | Median |
|---------------------|--------|
| -----               | -----  |
| [1 3 -1] -3 5 3 6 7 | 1      |
| 1 [3 -1 -3] 5 3 6 7 | -1     |
| 1 3 [-1 -3 5] 3 6 7 | -1     |
| 1 3 -1 [-3 5 3] 6 7 | 3      |
| 1 3 -1 -3 [5 3 6] 7 | 5      |
| 1 3 -1 -3 5 [3 6 7] | 6      |

```

class DualHeap {
public:
    priority_queue<long> maxHeap;
    priority_queue<long, vector<long>, greater<long>> minHeap;
    unordered_map<long, int> delayed;
    int small = 0, large = 0;
    int k;
    DualHeap(int _k) : k(_k) {}
    void prune(priority_queue<long> &heap) {
        while (!heap.empty()) {
            long x = heap.top();
            if (delayed.count(x)) {
                if (--delayed[x] == 0) delayed.erase(x);
                heap.pop();
            } else break;
        }
    }
    void pruneMin(priority_queue<long, vector<long>, greater<long>>
&heap) {
        while (!heap.empty()) {
            long x = heap.top();
            if (delayed.count(x)) {
                if (--delayed[x] == 0) delayed.erase(x);
                heap.pop();
            } else break;
        }
    }
    void balance() {

```



```

        if (small > large + 1) {
            long x = maxHeap.top(); maxHeap.pop();
            minHeap.push(x);
            small--; large++;
            prune(maxHeap);
        } else if (large > small) {
            long x = minHeap.top(); minHeap.pop();
            maxHeap.push(x);
            large--; small++;
            pruneMin(minHeap);
        }
    }

    void insert(long num) {
        if (maxHeap.empty() || num <= maxHeap.top()) {
            maxHeap.push(num); small++;
        } else {
            minHeap.push(num); large++;
        }
        balance();
    }

    void erase(long num) {
        delayed[num]++;
        if (!maxHeap.empty() && num <= maxHeap.top()) {
            small--;
            if (num == maxHeap.top()) prune(maxHeap);
        } else {
            large--;
            if (!minHeap.empty() && num == minHeap.top())
pruneMin(minHeap);
        }
        balance();
    }

    double getMedian() {
        if (k % 2 == 1) return maxHeap.top();
        return (maxHeap.top() + minHeap.top()) / 2.0;
    }
};

vector<double> medianSlidingWindow(vector<int>& nums, int k) {

```

```

DualHeap dh(k);
vector<double> ans;

for (int i = 0; i < nums.size(); i++) {
    dh.insert(nums[i]);
    if (i >= k - 1) {
        ans.push_back(dh.getMedian());
        dh.erase(nums[i - k + 1]);
    }
}
return ans;
}

```

[hard]

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most  $k$  distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most  $k$  distinct projects. You are given  $n$  projects where the  $i$ th project has a pure profit  $profits[i]$  and a minimum capital of  $capital[i]$  is needed to start it. Initially, you have  $w$  capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital. Pick a list of at most  $k$  distinct projects from given projects to maximize your final capital, and return the final maximized capital. The answer is guaranteed to fit in a 32-bit signed integer. Input:  $k = 2$ ,  $w = 0$ ,  $profits = [1,2,3]$ ,  $capital = [0,1,1]$ . Output: 4

Explanation: Since your initial capital is 0, you can only start the project indexed 0. After finishing it you will obtain profit 1 and your capital becomes 1. With capital 1, you can either start the project indexed 1 or the project indexed 2.

Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.

Therefore, output the final maximized capital, which is  $0 + 1 + 3 = 4$ .

```

int findMaximizedCapital(int k, int w, vector<int>& profits,
vector<int>& capital) {
    int n = profits.size();
    vector<pair<int,int>> projects; // (capital, profit)
    for (int i = 0; i < n; i++)
        projects.push_back({capital[i], profits[i]});
    sort(projects.begin(), projects.end());
    priority_queue<int> maxProfit;
    int idx = 0;
    while (k--) {
        while (idx < n && projects[idx].first <= w) {
            maxProfit.push(projects[idx].second);
            idx++;
        }
        if (maxProfit.empty()) break;
        w += maxProfit.top();
        maxProfit.pop();
    }
    return w;
}

```

Stack

## Stack - Monotonic Stack

Description: A monotonic stack helps solve range queries by maintaining a stack of elements in increasing or decreasing order.

- [\[Self Added\] Next Greater Element I](#)
- [Next Greater Element II](#)
- [Next Greater Node in Linked List](#)
- [Daily Temperatures](#)
- [Online Stock Span](#)
- [Maximum Width Ramp](#)
- [Largest Rectangle in Histogram](#)
- [\[Self added v2\] Find Celebrity](#)

The next greater element of some element  $x$  in an array is the first greater element that is to the right of  $x$  in the same array.

You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each  $0 \leq i < \text{nums1.length}$ , find the index  $j$  such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above.

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.

```

vector<int> nextGreaterElement(vector<int>& nums1, vector<int>&
nums2) {
    stack<int> st;
    unordered_map<int,int> nge;
    for (int i = 0; i < nums2.size(); i++) {
        while (!st.empty() && nums2[st.top()] < nums2[i]) {
            nge[nums2[st.top()]] = nums2[i];
            st.pop();
        }
        st.push(i);
    }
    while (!st.empty()) {
        nge[nums2[st.top()]] = -1;
        st.pop();
    }
    vector<int> ans;
    for (int i = 0; i < nums1.size(); i++)
        ans.push_back(nge[nums1[i]]);
    return ans;
}

```

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the next greater number for every element in `nums`.

The next greater number of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Input: `nums = [1,2,1]`

Output: `[2,-1,2]`

Explanation: The first `1`'s next greater number is `2`;

The number `2` can't find next greater number.

The second `1`'s next greater number needs to search circularly, which is also `2`.

```

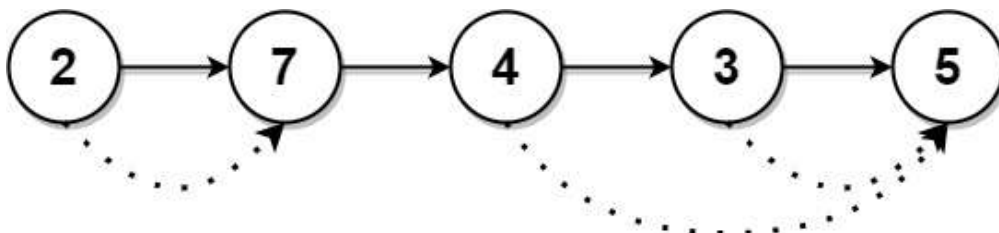
vector<int> nextGreaterElements(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for (int i = 0; i < 2*n; i++) {
        int idx = i % n;
        while (!st.empty() && nums[st.top()] < nums[idx]) {
            ans[st.top()] = nums[idx];
            st.pop();
        }
        if (i < n) st.push(idx);
    }
    return ans;
}

```

You are given the head of a linked list with  $n$  nodes.  
 For each node in the list, find the value of the next greater node.  
 That is, for each node, find the value of the first node that is  
 next to it and has a strictly larger value than it.  
 Return an integer array `answer` where `answer[i]` is the value of the  
 next greater node of the  $i$ th node (1-indexed). If the  $i$ th node does  
 not have a next greater node, set `answer[i] = 0`.

Input: head = [2,7,4,3,5]

Output: [7,0,5,5,0]



```

struct ListNode {
    int val; ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};

vector<int> nextLargerNodes(ListNode* head) {
    vector<int> arr;
    while (head) {
        arr.push_back(head->val);
        head = head->next;
    }
    int n = arr.size(); vector<int> ans(n, 0); stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] < arr[i]) {
            ans[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }
    return ans;
}

```

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead. Input: temperatures = [73,74,75,71,69,72,76,73]. Output: [1,1,4,2,1,1,0,0]

```

vector<int> dailyTemperatures(vector<int>& t) {
    int n = t.size(); vector<int> ans(n, 0); stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && t[st.top()] < t[i]) {
            int idx = st.top(); st.pop();
            ans[idx] = i - idx;
        }
        st.push(i);
    }
    return ans;
}

```



Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day. The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.

Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

StockSpanner() Initializes the object of the class.

int next(int price) Returns the span of the stock's price given that today's price is price.

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
```

```
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();
```

```
stockSpanner.next(100); // return 1
```

```
stockSpanner.next(80); // return 1
```

```
stockSpanner.next(60); // return 1
```

```
stockSpanner.next(70); // return 2
```

```
stockSpanner.next(60); // return 1
```

```
stockSpanner.next(75); // return 4, because the last 4 prices  
(including today's price of 75) were less than or equal to today's  
price.
```

```
stockSpanner.next(85); // return 6
```

```

class StockSpanner {
public:
    stack<pair<int,int> > st; // {price, span}
    StockSpanner() {}
    int next(int price) {
        int span = 1;
        while (!st.empty() && st.top().first <= price) {
            span += st.top().second;
            st.pop();
        }
        st.push(make_pair(price, span));
        return span;
    }
};

```

A ramp in an integer array `nums` is a pair  $(i, j)$  for which  $i < j$  and `nums[i] <= nums[j]`. The width of such a ramp is  $j - i$ .

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Input: `nums = [6,0,8,2,1,5]`

Output: 4

Explanation: The maximum width ramp is achieved at  $(i, j) = (1, 5)$ :  
`nums[1] = 0` and `nums[5] = 5`.

```

int maxWidthRamp(vector<int>& nums) {
    int n = nums.size(); stack<int> st;
    for (int i = 0; i < n; i++)
        if (st.empty() || nums[i] < nums[st.top()])
            st.push(i);
    int ans = 0;
    for (int j = n - 1; j >= 0; j--) {
        while (!st.empty() && nums[st.top()] <= nums[j]) {
            ans = max(ans, j - st.top());
            st.pop();
        }
    }
    return ans;
}

```

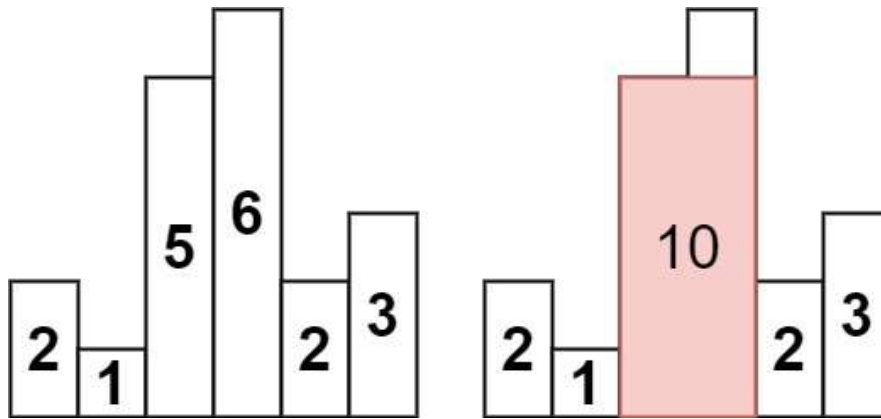
[hard]

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1. The largest rectangle is shown in the red area, which has an area = 10 units.



```
int largestRectangleArea(vector<int>& h) {  
    int n = h.size();  
    stack<int> st;  
    int maxArea = 0;  
    for (int i = 0; i <= n; i++) {  
        int height = (i == n ? 0 : h[i]);  
        while (!st.empty() && h[st.top()] > height) {  
            int ht = h[st.top()]; st.pop();  
            int left = st.empty() ? -1 : st.top();  
            int width = i - left - 1;  
            maxArea = max(maxArea, ht * width);  
        }  
        st.push(i);  
    }  
    return maxArea;  
}
```

A celebrity is a person who is known to all but does not know anyone at a party. If you go to a party of N people, find if there is a celebrity in the party or not.

A square NxN matrix M[][] is used to represent people at the party such that if an element of row i and column j is set to 1 it means ith person knows jth person. Here M[i][i] will always be 0.

Note: Follow 0 based indexing. Input: N = 3

```
M[][] = {{0 1 0},
          {0 0 0},
          {0 1 0}}
```

Output: 1

```
bool knows(int x, int y, vector<vector<int> >& M) {
    return M[x][y];
    // it itself send if celebrity know a person or not...
}

int celebrity(vector<vector<int> >& M, int n)
{
    stack<int> st;
    for (int i = 0 ; i < n ; i++) {
        st.push(i);
    }
    while (st.size() > 1) {
        int x = st.top(); st.pop();
        int y = st.top(); st.pop();
        if (knows(x, y, M)) st.push(y); // we push back y as its a
potential celebrity..
        else st.push(x);
    }
    int pc = st.top(); // potential celebrity... pc...
    if (st.empty()) return -1;
    for (int i = 0 ; i < n ; i++) {
        if (i != pc and (knows(pc, i, M) or !knows(i, pc, M))) {
            return -1;
        }
    }

    return pc;
}
```

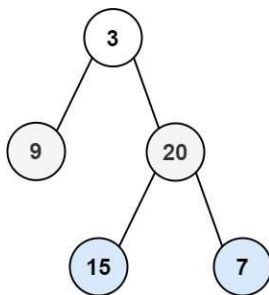
# Tree-LevelOrderTraversal

## Trees - Level Order Traversal (BFS in Binary Tree)

- [Level order Traversal](#)
- [Zigzag Level order Traversal](#)
- [Even Odd Tree](#)
- [Reverse odd Levels](#)
- [Deepest Leaves Sum](#)
- [Add one row to Tree](#)
- [Maximum width of Binary Tree](#)
- [All Nodes Distance K in Binary tree](#)

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Input: root = [3,9,20,null,null,15,7]. Output: [[3],[9,20],[15,7]]

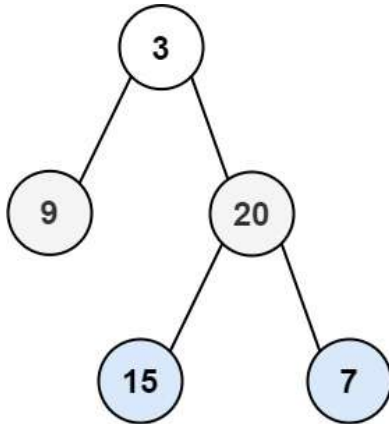


```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> ans; queue<TreeNode*> q;
    if(root==NULL) return ans;
    q.push(root);
    while(!q.empty()){
        int size = q.size(); vector<int> l;
        for(int i=0; i<size; i++){
            TreeNode* x = q.front();
            q.pop();
            l.push_back(x->val);
            if(x->left) q.push(x->left);
            if(x->right) q.push(x->right);
        }
        ans.push_back(l);
    }
    return ans;
}
```

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Input: root = [3,9,20,null,null,15,7]

Output: [[3],[20,9],[15,7]]



```
vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
    vector<vector<int>> ans;
    if(root==NULL) return ans;
    queue<TreeNode*> q;
    q.push(root);
    bool direction = true;
    while(!q.empty()){
        int size = q.size();
        vector<int> l(size);
        for(int i = 0; i<size; i++){
            TreeNode* x = q.front();
            q.pop();
            if(direction)l[i]=x->val;
            else l[size-i-1]=x->val;
            if(x->left) q.push(x->left);
            if(x->right) q.push(x->right);
        }
        direction=!direction;
        ans.push_back(l);
    }
    return ans;
}
```

A binary tree is named Even-Odd if it meets the following conditions:

The root of the binary tree is at level index 0, its children are at level index 1, their children are at level index 2, etc.

For every even-indexed level, all nodes at the level have odd integer values in strictly increasing order (from left to right).

For every odd-indexed level, all nodes at the level have even integer values in strictly decreasing order (from left to right).

Given the root of a binary tree, return true if the binary tree is Even-Odd, otherwise return false.

Input: root = [1,10,4,3,null,7,9,12,8,6,null,null,2]

Output: true

Explanation: The node values on each level are:

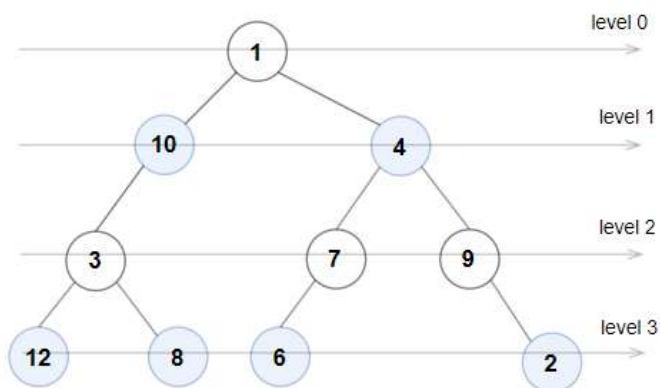
Level 0: [1]

Level 1: [10,4]

Level 2: [3,7,9]

Level 3: [12,8,6,2]

Since levels 0 and 2 are all odd and increasing and levels 1 and 3 are all even and decreasing, the tree is Even-Odd.





```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isEvenOddTree(TreeNode* root) {
    queue<TreeNode*> q;
    q.push(root);
    int level = 0;
    while (!q.empty()) {
        int sz = q.size();
        int prev = (level % 2 == 0 ? INT_MIN : INT_MAX);
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front(); q.pop();
            int v = cur->val;
            if (level % 2 == 0) { // even level
                if (v % 2 == 0 || v <= prev) return false;
            } else { // odd level
                if (v % 2 == 1 || v >= prev) return false;
            }
            prev = v;
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        level++;
    }
    return true;
}

```

Given the root of a perfect binary tree, reverse the node values at each odd level of the tree.

For example, suppose the node values at level 3 are [2,1,3,4,7,11,29,18], then it should become [18,29,11,7,4,3,1,2]. Return the root of the reversed tree.

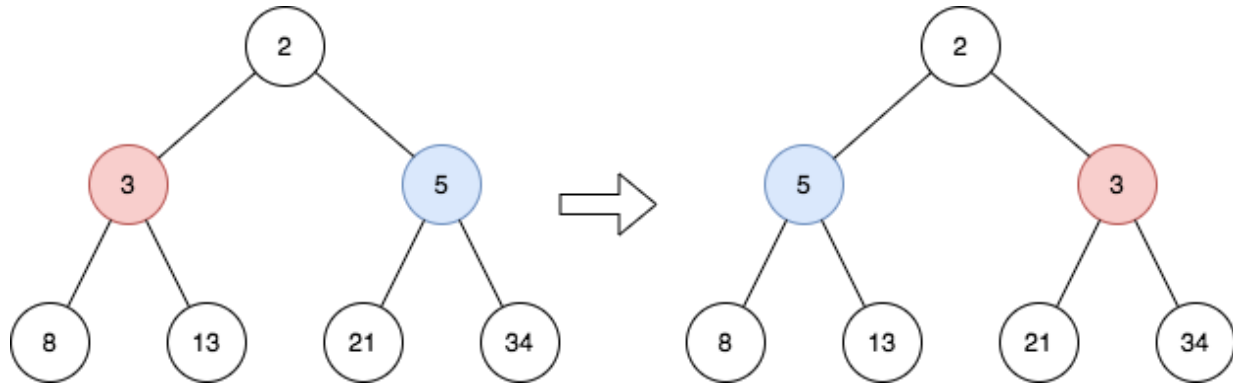
A binary tree is perfect if all parent nodes have two children and all leaves are on the same level.

The level of a node is the number of edges along the path between it and the root node.

Input: root = [2,3,5,8,13,21,34]. Output: [2,5,3,8,13,21,34]

Explanation: The tree has only one odd level.

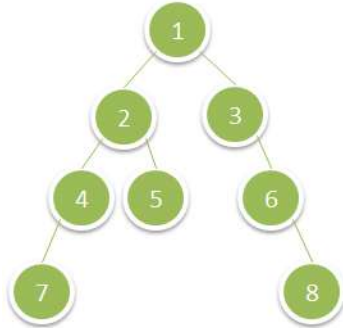
The nodes at level 1 are 3, 5 respectively, which are reversed and become 5, 3.



```
TreeNode* reverseOddLevels(TreeNode* root) {
    if (!root) return root;
    queue<TreeNode*> q; q.push(root);
    int level = 0;
    while (!q.empty()) {
        int sz = q.size(); vector<TreeNode*> nodes;
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front(); q.pop();
            nodes.push_back(cur);
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        // Reverse values at odd level
        if (level % 2 == 1) {
            int l = 0, r = nodes.size() - 1;
            while (l < r) {
                swap(nodes[l]->val, nodes[r]->val);
                l++; r--;
            }
        }
        level++;
    }
    return root;
}
```

Given the root of a binary tree, return the sum of values of its deepest leaves. Input: root =

[1,2,3,4,5,null,6,7,null,null,null,null,8]. Output: 15



```
int deepestLeavesSum(TreeNode* root) {
    queue<TreeNode*> q; q.push(root);
    int sum = 0;
    while (!q.empty()) {
        int sz = q.size(); sum = 0;
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front(); q.pop();
            sum += cur->val;
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
    }
    return sum;
}
```

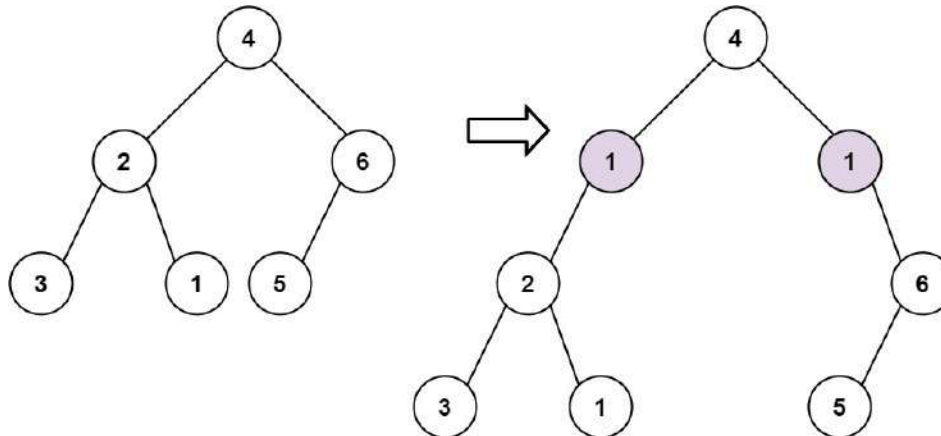
Given the root of a binary tree and two integers val and depth, add a row of nodes with value val at the given depth depth. Note that the root node is at depth 1.

The adding rule is: Given the integer depth, for each not null tree node cur at the depth depth - 1, create two tree nodes with value val as cur's left subtree root and right subtree root. cur's original left subtree should be the left subtree of the new left subtree root. cur's original right subtree should be the right subtree of the new right subtree root.

If  $\text{depth} == 1$  that means there is no depth  $\text{depth} - 1$  at all, then create a tree node with value  $\text{val}$  as the new root of the whole original tree, and the original tree is the new root's left subtree.

Input:  $\text{root} = [4, 2, 6, 3, 1, 5]$ ,  $\text{val} = 1$ ,  $\text{depth} = 2$ .

Output:  $[4, 1, 1, 2, \text{null}, \text{null}, 6, 3, 1, 5]$



Other ex:

Level 1:

10

Level 2:

/ \

5 15

Level 3:

/ \ / \

3 7 12 18

Level 4:

/ \ \

1 4 13

Level 5:

/

0

Case: Row added at depth = 3,  $\text{val} = 99$

Level 1:

10

Level 2:

/ \

5 15

Level 3:

/ \ / \

99 99 99 99

Level 4:

/ \ / \

3 7 12 18

Level 5:

/ \ \

1 4 13

Level 6:

/

0

TreeNode\* addOneRow(TreeNode\* root, int val, int depth) {

```

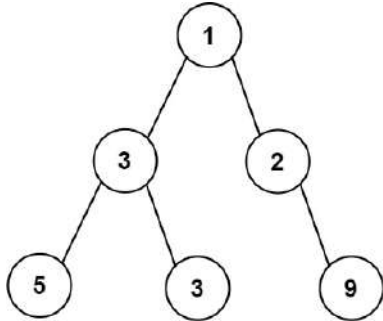
if (depth == 1) {
    TreeNode* newRoot = new TreeNode(val);
    newRoot->left = root;
    return newRoot;
}
queue<TreeNode*> q;
q.push(root);
int d = 1;
while (!q.empty()) {
    int sz = q.size();
    if (d == depth - 1) {
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front();
            q.pop();
            TreeNode* l = cur->left;
            TreeNode* r = cur->right;
            cur->left = new TreeNode(val);
            cur->left->left = l;
            cur->right = new TreeNode(val);
            cur->right->right = r;
        }
        break;
    }
    for (int i = 0; i < sz; i++) {
        TreeNode* cur = q.front();
        q.pop();
        if (cur->left) q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
    d++;
}
return root;
}

```

**Given the root of a binary tree, return the maximum width of the given tree. The maximum width of a tree is the maximum width among**

all levels. The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation. It is guaranteed that the answer will in the range of a 32-bit signed integer.

Input: root = [1,3,2,5,3,null,9]. Output: 4. Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).



```

long long widthOfBinaryTree(TreeNode* root) {
    if (!root) return 0;
    queue<pair<TreeNode*, long long>> q;
    q.push(make_pair(root, 1)); long long ans = 0;
    while (!q.empty()) {
        int sz = q.size();
        long long left = q.front().second, right = left;
        for (int i = 0; i < sz; i++) {
            pair<TreeNode*, long long> p = q.front(); q.pop();
            TreeNode* cur = p.first;
            long long idx = p.second;
            right = idx;
            if (cur->left) q.push(make_pair(cur->left, idx*2));
            if (cur->right) q.push(make_pair(cur->right, idx*2+1));
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
  
```

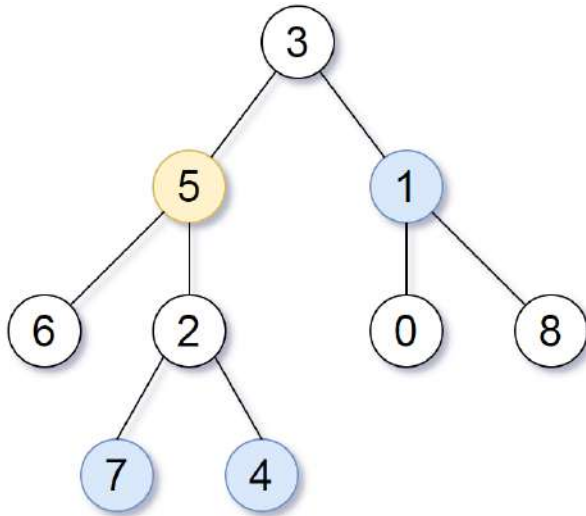
Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.



```
void buildParents(TreeNode* root, unordered_map<TreeNode*,
TreeNode*>& parent) {
    queue<TreeNode*> q;
    q.push(root);
    parent[root] = NULL;
    while (!q.empty()) {
        TreeNode* cur = q.front(); q.pop();
        if (cur->left) {
            parent[cur->left] = cur;
            q.push(cur->left);
        }
        if (cur->right) {
            parent[cur->right] = cur;
            q.push(cur->right);
        }
    }
}

vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
```

```

unordered_map<TreeNode*, TreeNode*> parent;
buildParents(root, parent);
unordered_set<TreeNode*> vis;
queue<TreeNode*> q;
q.push(target);
vis.insert(target);
int dist = 0;
while (!q.empty()) {
    int sz = q.size();
    if (dist == k) break;
    dist++;
    for (int i = 0; i < sz; i++) {
        TreeNode* cur= q.front(); q.pop();
        TreeNode* arr[3]= {cur->left, cur->right, parent[cur]};
        for (int j = 0; j < 3; j++) {
            TreeNode* nxt = arr[j];
            if (nxt && !vis.count(nxt)) {
                vis.insert(nxt);
                q.push(nxt);
            }
        }
    }
}
vector<int> ans;
while (!q.empty()) {
    ans.push_back(q.front()->val);
    q.pop();
}
return ans;
}

```



# Tree-Traversals

[Self added] Tree Traversals

- [\[Self added v2\] Left, Right, Top View of Binary Tree](#)

**// Left View**

// recursive

```
void leftViewDFS(Node* curr, int level, int &maxLevel, vector<int>
&ans) {
```

```
    if (curr == NULL) return;
```

```
    if (level > maxLevel) {
```

```
        ans.push_back(curr->data);
```

```
        maxLevel = level;
```

```
    }
```

```
    leftViewDFS(curr->left, level + 1, maxLevel, ans);
```

```
    leftViewDFS(curr->right, level + 1, maxLevel, ans);
```

```
}
```

```
vector<int> leftView(Node *root) {
```

```
    if (root == NULL) return {};
```

```
    vector<int> ans;
```

```
    int maxLevel = -1;
```

```
    leftViewDFS(root, 0, maxLevel, ans);
```

```
    return ans;
```

```
}
```

// iterative bfs

```
vector<int> leftViewIterative(Node* root) {
```

```
    if (root == NULL) return {};
```

```
    queue<Node*> q; q.push(root); vector<int> ans;
```

```
    while (!q.empty()) {
```

```
        int sz = q.size();
```

```
        ans.push_back(q.front()->data); // leftmost element
```

```
        for (int i = 0; i < sz; i++) {
```

```
            Node* curr = q.front(); q.pop();
```

```
            if (curr->left) q.push(curr->left);
```

```
            if (curr->right) q.push(curr->right);
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

```

// Right View
// recursive
void rightViewDFS(Node* curr, int level, int &maxLevel, vector<int>
&ans) {
    if (curr == NULL) return;
    if (level > maxLevel) {
        ans.push_back(curr->data);
        maxLevel = level;
    }
    rightViewDFS(curr->right, level + 1, maxLevel, ans);
    rightViewDFS(curr->left, level + 1, maxLevel, ans);
}
vector<int> rightView(Node *root) {
    if (root == NULL) return {};
    vector<int> ans;
    int maxLevel = -1;
    rightViewDFS(root, 0, maxLevel, ans);
    return ans;
}
// iterative bfs
vector<int> rightViewIterative(Node* root) {
    if (root == NULL) return {};
    queue<Node*> q;
    q.push(root);
    vector<int> ans;
    while (!q.empty()) {
        int sz = q.size();
        ans.push_back(q.front()->data); // rightmost now!
        for (int i = 0; i < sz; i++) {
            Node* curr = q.front(); q.pop();
            if (curr->right) q.push(curr->right);
            if (curr->left) q.push(curr->left);
        }
    }
    return ans;
}

```

```

// Top View
// recursive
void topViewDFS(Node* curr, int dist, int level,
                map<int, pair<int,int>> &mp) {
    if (curr == NULL) return;
    if (mp.count(dist) == 0 || mp[dist].second > level) {
        mp[dist] = {curr->data, level};
    }
    topViewDFS(curr->left, dist - 1, level + 1, mp);
    topViewDFS(curr->right, dist + 1, level + 1, mp);
}

vector<int> topView(Node* root) {
    if (root == NULL) return {};
    map<int, pair<int,int>> mp; // dist -> {val, level}
    topViewDFS(root, 0, 0, mp);
    vector<int> ans;
    for (auto &p : mp) ans.push_back(p.second.first);
    return ans;
}

// iterative bfs
vector<int> topViewIterative(Node* root) {
    if (root == NULL) return {};
    map<int, int> mp; // dist -> node value
    queue<pair<Node*, int>> q; // node, horizontal dist
    q.push({root, 0}); vector<int> ans;
    while (!q.empty()) {
        auto front = q.front(); q.pop();
        Node* curr = front.first; int dist = front.second;
        if (mp.count(dist) == 0) {
            mp[dist] = curr->data;
        }
        if (curr->left) q.push({curr->left, dist - 1});
        if (curr->right) q.push({curr->right, dist + 1});
    }
    for (auto &p : mp) ans.push_back(p.second);
    return ans;
}

```

# Tree-Construction

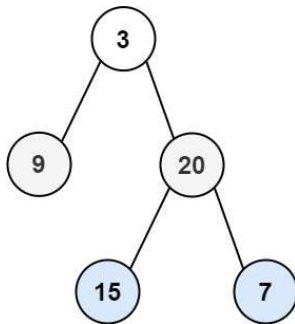
## Trees - Tree Construction

- [Construct BT from Preorder and Inorder](#)
- [Construct BT from Postorder and Inorder](#)
- [Maximum Binary Tree](#)
- [Construct BST from Preorder](#)

(Preorder-PLR, Inorder-LPR, Postorder-LRP)

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Input: root = [3,9,20,null,null,15,7]. Output: [[3],[9,20],[15,7]]



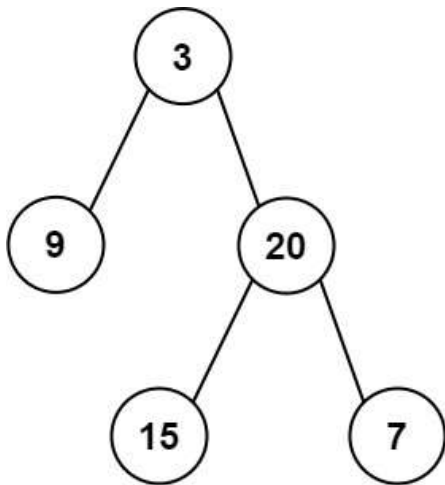
```
TreeNode* solve(vector<int>& preorder, vector<int>& inorder,
                unordered_map<int, int>& inomap, int& preo,
                int start, int end) {
    if (start > end) return nullptr;
    int data = preorder[preo++];
    TreeNode* root = new TreeNode(data);
    int mid = inomap[data];
    root->left = solve(preorder, inorder, inomap, preo, start, mid - 1);
    root->right = solve(preorder, inorder, inomap, preo, mid + 1, end);
    return root;
}

TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
    unordered_map<int, int> inomap;
    for (int i = 0; i < inorder.size(); i++)
        inomap[inorder[i]] = i;
    int preo = 0;
    return solve(preorder, inorder, inomap, preo, 0, inorder.size() - 1);
}
```

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return the binary tree.

Input: `inorder = [9,3,15,20,7]`, `postorder = [9,15,7,20,3]`

Output: `[3,9,20,null,null,15,7]`



```
TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
    unordered_map<int,int> mp;
    for(int i = 0; i < inorder.size(); i++) mp[inorder[i]] = i;
    int postIdx = postorder.size() - 1;
    return build(inorder, postorder, mp, 0, inorder.size() - 1,
postIdx);
}

TreeNode* build(vector<int>& inorder, vector<int>& postorder,
                unordered_map<int,int>& mp,
                int inL, int inR, int &postIdx) {
    if(inL > inR) return NULL;
    int rootVal = postorder[postIdx--];
    TreeNode* root = new TreeNode(rootVal);
    int k = mp[rootVal];
    // Important: build right subtree first
    root->right = build(inorder, postorder, mp, k+1, inR, postIdx);
    root->left = build(inorder, postorder, mp, inL, k-1, postIdx);
    return root;
}
```

You are given an integer array `nums` with no duplicates. A maximum binary tree can be built recursively from `nums` using the following algorithm:

Create a root node whose value is the maximum value in `nums`.

Recursively build the left subtree on the subarray prefix to the left of the maximum value. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

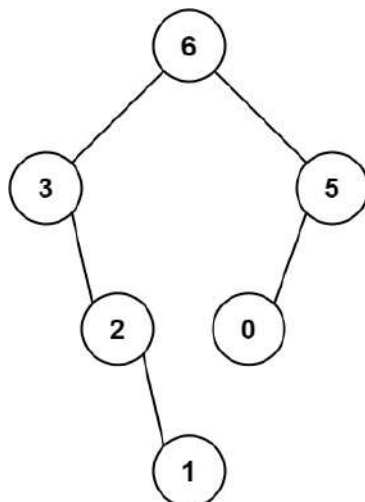
Return the maximum binary tree built from `nums`.

Input: `nums = [3,2,1,6,0,5]`

Output: `[6,3,5,null,2,0,null,null,1]`

Explanation: The recursive calls are as follow:

- The largest value in `[3,2,1,6,0,5]` is 6. Left prefix is `[3,2,1]` and right suffix is `[0,5]`.
  - The largest value in `[3,2,1]` is 3. Left prefix is `[]` and right suffix is `[2,1]`.
    - Empty array, so no child.
    - The largest value in `[2,1]` is 2. Left prefix is `[]` and right suffix is `[1]`.
      - Empty array, so no child.
      - Only one element, so child is a node with value 1.
  - The largest value in `[0,5]` is 5. Left prefix is `[0]` and right suffix is `[]`.
    - Only one element, so child is a node with value 0.
    - Empty array, so no child.





```

TreeNode* constructMaximumBinaryTree(vector<int>& nums) {
    return build(nums, 0, nums.size() - 1);
}
TreeNode* build(vector<int>& nums, int l, int r) {
    if(l > r) return NULL;
    int idx = l;
    for(int i = l; i <= r; i++)
        if(nums[i] > nums[idx]) idx = i;
    TreeNode* root = new TreeNode(nums[idx]);
    root->left = build(nums, l, idx - 1);
    root->right = build(nums, idx + 1, r);
    return root;
}

```

Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

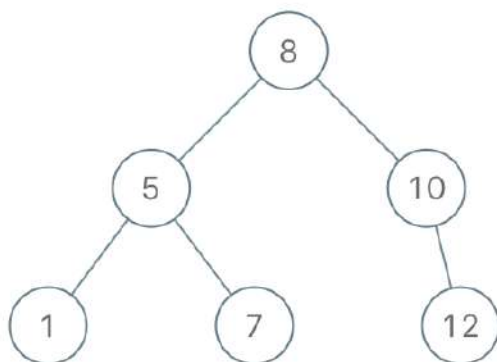
It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

A preorder traversal of a binary tree displays the value of the node first, then traverses Node.left, then traverses Node.right.

Input: preorder = [8,5,1,7,10,12]

Output: [8,5,10,1,7,null,12]



```
int idx;
TreeNode* bstFromPreorder(vector<int>& preorder) {
    idx = 0;
    return build(preorder, INT_MAX);
}
TreeNode* build(vector<int>& preorder, int bound) {
    if(idx == preorder.size() || preorder[idx] > bound)
        return NULL;
    int rootVal = preorder[idx++];
    TreeNode* root = new TreeNode(rootVal);
    root->left = build(preorder, rootVal);
    root->right = build(preorder, bound);
    return root;
}
```

# Tree-Height

## Trees - Height related Problems

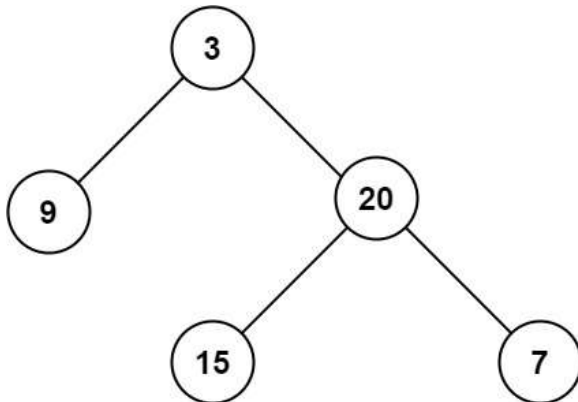
- [Maximum Depth of BT](#)
- [Balanced Binary Tree](#)
- [Diameter of Binary Tree](#)
- [Minimum Depth of BT](#)

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Input: root = [3,9,20,null,null,15,7]

Output: 3

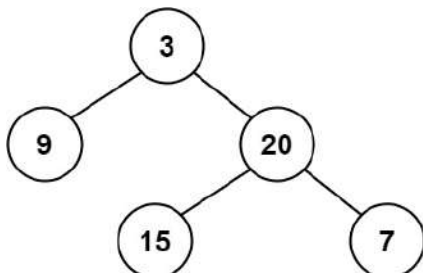


```
int maxDepth(TreeNode* root) {  
    if(root == NULL) return 0;  
    return 1 + max(maxDepth(root->left), maxDepth(root->right));  
}
```

Given a binary tree, determine if it is height-balanced.

Input: root = [3,9,20,null,null,15,7]

Output: true



```

bool isBalanced(TreeNode* root) {
    return height(root) != -1;
}
int height(TreeNode* root) {
    if(root == NULL) return 0;
    int lh = height(root->left);
    if(lh == -1) return -1;
    int rh = height(root->right);
    if(rh == -1) return -1;
    if(abs(lh - rh) > 1) return -1;
    return 1 + max(lh, rh);
}

```

Given the root of a binary tree, return the length of the diameter of the tree.

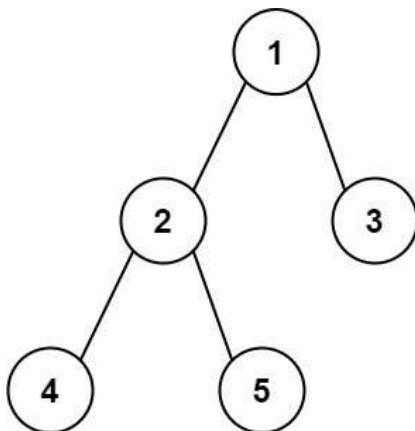
The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].



```

int ans = 0;
int diameterOfBinaryTree(TreeNode* root) {
    height(root);
    return ans;
}
int height(TreeNode* root) {
    if(root == NULL) return 0;
    int lh = height(root->left);
    int rh = height(root->right);
    ans = max(ans, lh + rh);    // edges = nodes - 1
    return 1 + max(lh, rh);
}

```

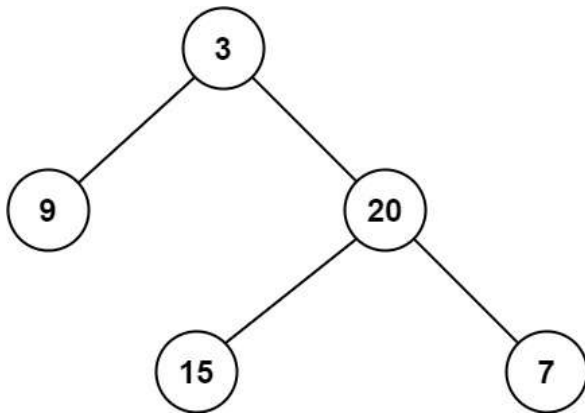
**Given a binary tree, find its minimum depth.**

The minimum depth is the number of nodes along the shortest path from the root node down to the nearest leaf node.

**Note:** A leaf is a node with no children.

**Input:** root = [3,9,20,null,null,15,7]

**Output:** 2



```

int minDepth(TreeNode* root) {
    if(root == NULL) return 0;
    int left = minDepth(root->left);
    int right = minDepth(root->right);
    if(root->left == NULL) return 1 + right;
    if(root->right == NULL) return 1 + left;
    return 1 + min(left, right);
}

```

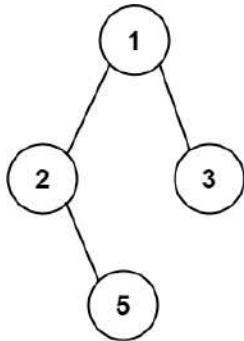
Tree-RootToLeaf

## Trees - Root to leaf path problems

- [Binary Tree Paths](#)
- [Path Sum ii](#)
- [Sum Root to Leaf numbers](#)
- [Smallest string starting from Leaf](#)
- [Insufficient nodes in root to Leaf](#)
- [Pseudo-Palindromic Paths in a Binary Tree](#)
- [Binary Tree Maximum Path Sum](#)

Given the root of a binary tree, return all root-to-leaf paths in any order. A leaf is a node with no children.

Input: root = [1,2,3,null,5]. Output: ["1->2->5","1->3"]



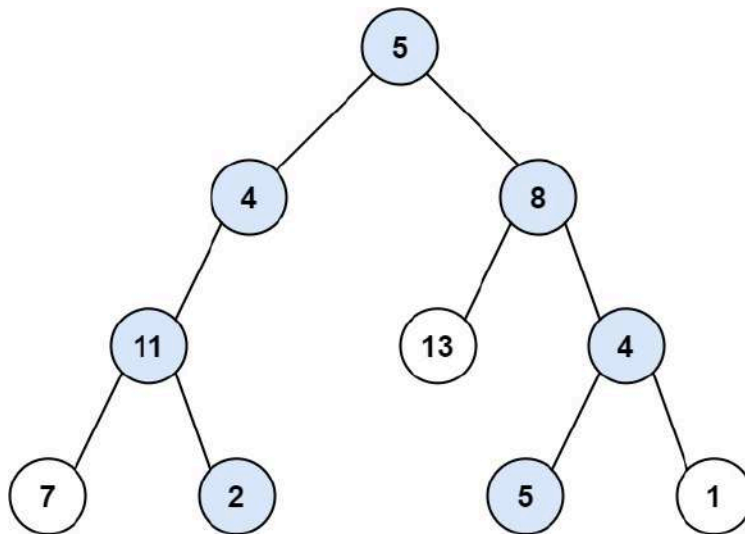
```
vector<string> ans;
vector<string> binaryTreePaths(TreeNode* root) {
    if(root == NULL) return ans;
    dfs(root, "");
    return ans;
}

void dfs(TreeNode* node, string path) {
    if(node == NULL) return;
    if(path.size() == 0) path = to_string(node->val);
    else path += "->" + to_string(node->val);
    if(node->left == NULL && node->right == NULL) {
        ans.push_back(path);
        return;
    }
    dfs(node->left, path);
    dfs(node->right, path);
}
```



Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references. A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22. Output: [[5,4,11,2],[5,8,4,5]]. Explanation: There are two paths whose sum equals targetSum: 5+4+11+2=22, 5+8+4+5=22



```
vector<vector<int>> res;
vector<int> curr;
vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
    dfs(root, targetSum);
    return res;
}
void dfs(TreeNode* root, int sum) {
    if(root == NULL) return;
    curr.push_back(root->val);
    if(root->left==NULL && root->right==NULL && sum==root->val)
        res.push_back(curr);
    dfs(root->left, sum - root->val);
    dfs(root->right, sum - root->val);
    curr.pop_back();
}
```

You are given the root of a binary tree containing digits from 0 to 9 only.

Each root-to-leaf path in the tree represents a number.

For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

A leaf node is a node with no children.

Input: root = [4,9,0,5,1]

Output: 1026

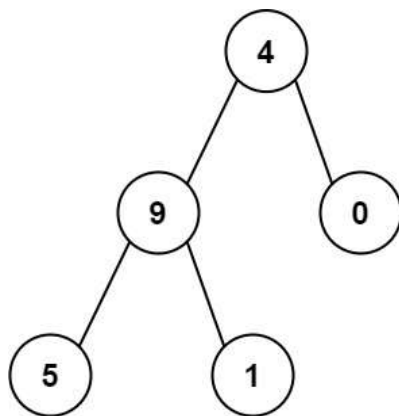
Explanation:

The root-to-leaf path 4->9->5 represents the number 495.

The root-to-leaf path 4->9->1 represents the number 491.

The root-to-leaf path 4->0 represents the number 40.

Therefore, sum = 495 + 491 + 40 = 1026.



```
int sumNumbers(TreeNode* root) {  
    return dfs(root, 0);  
}  
  
int dfs(TreeNode* root, int num) {  
    if(root == NULL) return 0;  
    num = num * 10 + root->val;  
    if(root->left == NULL && root->right == NULL)  
        return num;  
    return dfs(root->left, num) + dfs(root->right, num);  
}
```

You are given the root of a binary tree where each node has a value in the range [0, 25] representing the letters 'a' to 'z'.

Return the lexicographically smallest string that starts at a leaf of this tree and ends at the root.

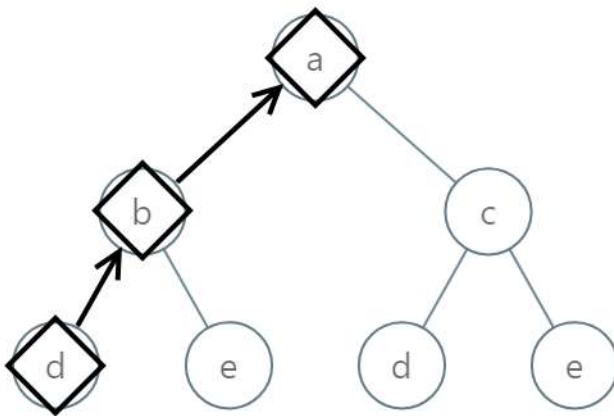
As a reminder, any shorter prefix of a string is lexicographically smaller.

For example, "ab" is lexicographically smaller than "aba".

A leaf of a node is a node that has no children.

Input: root = [0,1,2,3,4,3,4]

Output: "dba"



```
string ans = "~"; // '~' is lexicographically larger than any lowercase
```

```
string smallestFromLeaf(TreeNode* root) {
    string curr = "";
    dfs(root, curr);
    return ans;
}

void dfs(TreeNode* root, string curr) {
    if(root == NULL) return;
    char c = 'a' + root->val;
    curr = c + curr;    // prepend
    if(root->left == NULL && root->right == NULL) {
        if(curr < ans) ans = curr;
        return;
    }
    dfs(root->left, curr);
    dfs(root->right, curr);
}
```

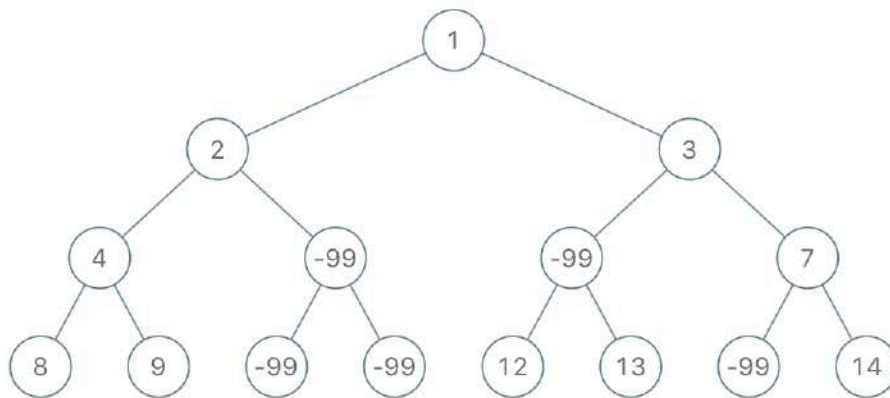
Given the root of a binary tree and an integer limit, delete all insufficient nodes in the tree simultaneously, and return the root of the resulting binary tree.

A node is insufficient if every root to leaf path intersecting this node has a sum strictly less than limit.

A leaf is a node with no children.

Input: root = [1,2,3,4,-99,-99,7,8,9,-99,-99,12,13,-99,14], limit = 1

Output: [1,2,3,4,null,null,7,8,9,null,14]



```

TreeNode* sufficientSubset(TreeNode* root, int limit) {
    return dfs(root, 0, limit);
}

TreeNode* dfs(TreeNode* root, int sum, int limit) {
    if (!root) return NULL;
    sum += root->val;
    // store original leaf status
    bool isLeaf = (!root->left && !root->right);
    root->left = dfs(root->left, sum, limit);
    root->right = dfs(root->right, sum, limit);
    if (isLeaf) {
        return (sum >= limit) ? root : NULL;
    }
    // internal node: keep only if at least one child survives
    return (root->left || root->right) ? root : NULL;
}

```

Given a binary tree where node values are digits from 1 to 9. A path in the binary tree is said to be pseudo-palindromic if at least one permutation of the node values in the path is a palindrome.

Return the number of pseudo-palindromic paths going from the root node to leaf nodes.

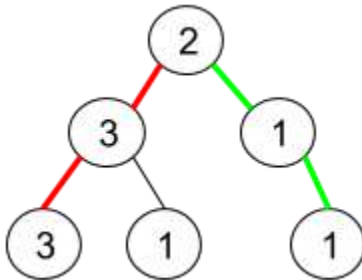
Input: root = [2,3,1,3,1,null,1]

Output: 2

Explanation: The figure above represents the given binary tree.

There are three paths going from the root node to leaf nodes: the red path [2,3,3], the green path [2,1,1], and the path [2,3,1].

Among these paths only red path and green path are pseudo-palindromic paths since the red path [2,3,3] can be rearranged in [3,2,3] (palindrome) and the green path [2,1,1] can be rearranged in [1,2,1] (palindrome).



```
int count = 0;
int pseudoPalindromicPaths (TreeNode* root) {
    dfs(root, 0);
    return count;
}

void dfs(TreeNode* root, int mask) {
    if(root == NULL) return;
    mask ^= (1 << root->val);
    if(root->left == NULL && root->right == NULL) {
        if((mask & (mask - 1)) == 0)
            count++;
        return;
    }
    dfs(root->left, mask);
    dfs(root->right, mask);
}
```

[hard]

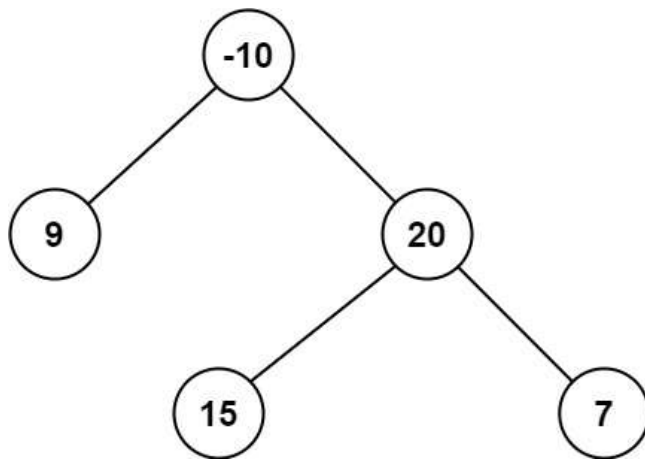
A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path. Given the root of a binary tree, return the maximum path sum of any non-empty path.

Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of  $15 + 20 + 7 = 42$ .



```
int ans;
int maxPathSum(TreeNode* root) {
    ans = INT_MIN;
    dfs(root);
    return ans;
}
int dfs(TreeNode* root) {
    if(root == NULL) return 0;
    int left = max(0, dfs(root->left));
    int right = max(0, dfs(root->right));
    ans = max(ans, root->val + left + right);
    return root->val + max(left, right);
}
```

Tree-Ancestor

## Trees - Ancestor problem

- [LCA of Binary Tree](#)
- [Maximum difference between node and ancestor](#)
- [LCA of deepest leaves](#)
- [Kth Ancestor of a Tree Node](#)

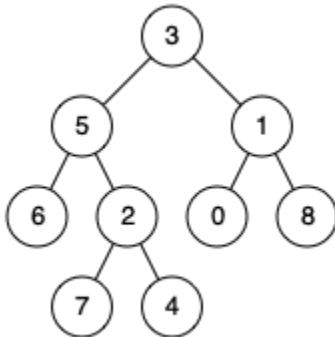
Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.



```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;
    return left ? left : right;
}
```



Given the root of a binary tree, find the maximum value  $v$  for which there exist different nodes  $a$  and  $b$  where  $v = |a.val - b.val|$  and  $a$  is an ancestor of  $b$ .

A node  $a$  is an ancestor of  $b$  if either: any child of  $a$  is equal to  $b$  or any child of  $a$  is an ancestor of  $b$ .

Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]

Output: 7

Explanation: We have various ancestor-node differences, some of which are given below :

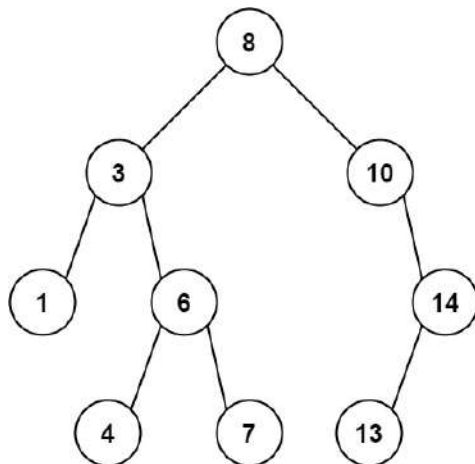
$$|8 - 3| = 5$$

$$|3 - 7| = 4$$

$$|8 - 1| = 7$$

$$|10 - 13| = 3$$

Among all possible differences, the maximum value of 7 is obtained by  $|8 - 1| = 7$ .



```
int dfs(TreeNode* node, int mn, int mx) {
    if (!node) return mx - mn;
    mn = min(mn, node->val);
    mx = max(mx, node->val);
    int left = dfs(node->left, mn, mx);
    int right = dfs(node->right, mn, mx);
    return max(left, right);
}

int maxAncestorDiff(TreeNode* root) {
    return dfs(root, root->val, root->val);
}
```

Given the root of a binary tree, return the lowest common ancestor of its deepest leaves.

Recall that:

The node of a binary tree is a leaf if and only if it has no children

The depth of the root of the tree is 0. if the depth of a node is  $d$ , the depth of each of its children is  $d + 1$ .

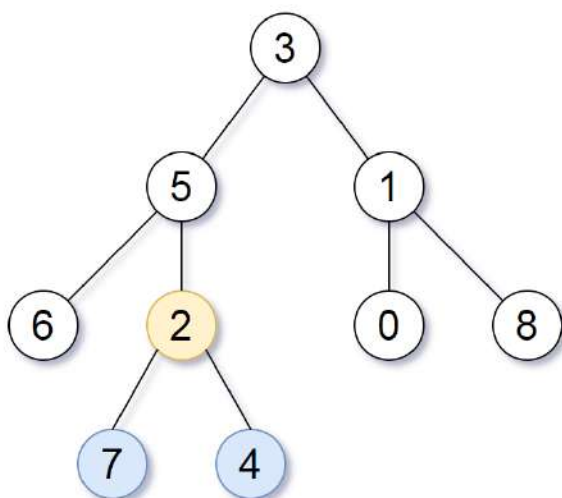
The lowest common ancestor of a set  $S$  of nodes, is the node  $A$  with the largest depth such that every node in  $S$  is in the subtree with root  $A$ .

Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest leaf-nodes of the tree. Note that nodes 6, 0, and 8 are also leaf nodes, but the depth of them is 2, but the depth of nodes 7 and 4 is 3.



```

struct Info {
    int depth;
    TreeNode* lca;
};

Info solve(TreeNode* root) {
    if (!root) {
        Info base;
        base.depth = 0;
        base.lca = NULL;
        return base;
    }
    Info left = solve(root->left);
    Info right = solve(root->right);
    Info result;
    if (left.depth > right.depth) {
        result.depth = left.depth + 1;
        result.lca = left.lca;
    }
    else if (right.depth > left.depth) {
        result.depth = right.depth + 1;
        result.lca = right.lca;
    }
    else {
        // equal depth → current root is LCA
        result.depth = left.depth + 1;
        result.lca = root;
    }
    return result;
}

TreeNode* lcaDeepestLeaves(TreeNode* root) {
    return solve(root).lca;
}

```

[hard]

You are given a tree with  $n$  nodes numbered from  $0$  to  $n - 1$  in the form of a parent array `parent` where `parent[i]` is the parent of  $i$ th node. The root of the tree is node  $0$ . Find the  $k$ th ancestor of a given node.

The  $k$ th ancestor of a tree node is the  $k$ th node in the path from that node to the root node.

Implement the `TreeAncestor` class:

`TreeAncestor(int n, int[] parent)` Initializes the object with the number of nodes in the tree and the parent array.

`int getKthAncestor(int node, int k)` return the  $k$ th ancestor of the given node `node`. If there is no such ancestor, return `-1`.

Input

```
["TreeAncestor", "getKthAncestor", "getKthAncestor",  
"getKthAncestor"]
```

```
[[7, [-1, 0, 0, 1, 1, 2, 2]], [3, 1], [5, 2], [6, 3]]
```

Output

```
[null, 1, 0, -1]
```

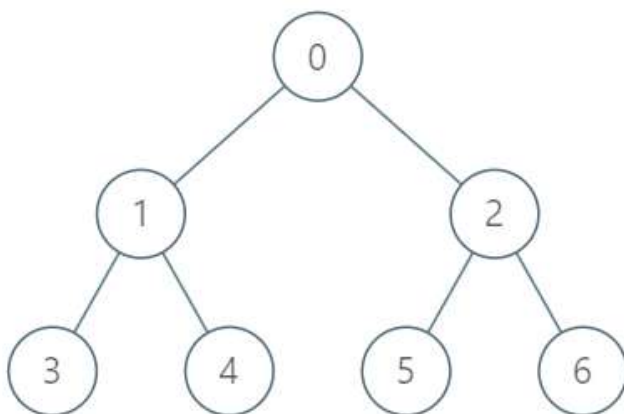
Explanation

```
TreeAncestor treeAncestor = new TreeAncestor(7, [-1, 0, 0, 1, 1, 2, 2]);
```

```
treeAncestor.getKthAncestor(3, 1); // returns 1 which is the parent  
of 3
```

```
treeAncestor.getKthAncestor(5, 2); // returns 0 which is the  
grandparent of 5
```

```
treeAncestor.getKthAncestor(6, 3); // returns -1 because there is no  
such ancestor
```



```

class TreeAncestor {
/*
Do binary lifting
Constraints
 $n \leq 5e5$  often  $\rightarrow$  binary lifting needed.
Preprocessing
 $up[i][j]$  =  $2^j$ -th ancestor of node  $i$ .
Binary lifting table size
 $LOG = 20$  (since  $2^{19} \approx 5e5$ )
*/
public:
    vector<vector<int>> up;
    int LOG;
    TreeAncestor(int n, vector<int>& parent) {
        LOG = 20;
        up.assign(n, vector<int>(LOG, -1));
        for (int i = 0; i < n; i++)
            up[i][0] = parent[i];
        for (int j = 1; j < LOG; j++) {
            for (int i = 0; i < n; i++) {
                int prev = up[i][j-1];
                up[i][j] = (prev == -1) ? -1 : up[prev][j-1];
            }
        }
    }
    int getKthAncestor(int node, int k) {
        for (int j = 0; j < LOG; j++) {
            if (k & (1 << j)) {
                node = up[node][j];
                if (node == -1) return -1;
            }
        }
        return node;
    }
};

```

# Tree-BST

## Trees - Binary Search Tree

- [Validate BST](#)
- [Range Sum of BST](#)
- [Minimum Absolute Difference in BST](#)
- [Insert into a BST](#)
- [LCA of BST](#)

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

The left subtree of a node contains only nodes with keys strictly less than the node's key.

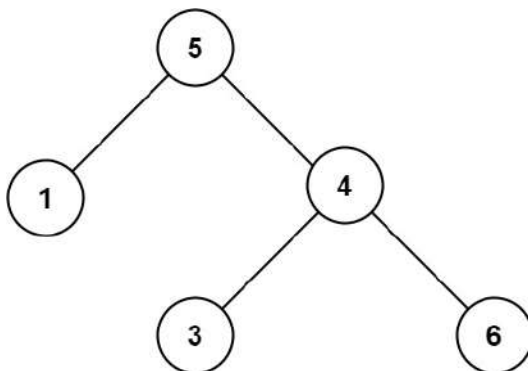
The right subtree of a node contains only nodes with keys strictly greater than the node's key.

Both the left and right subtrees must also be binary search trees.

Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.



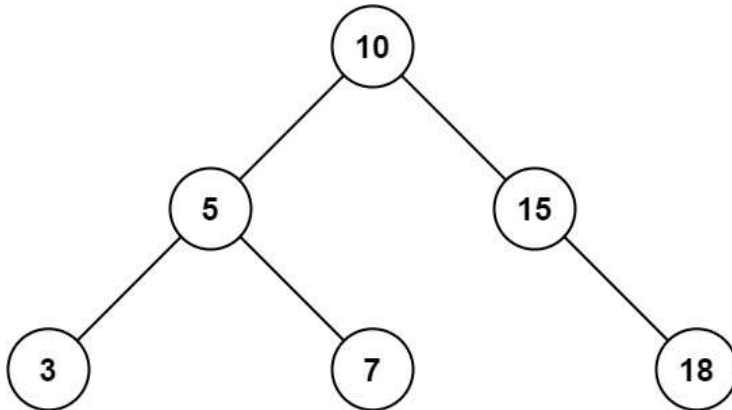
```
bool isValidBST(TreeNode* root) {  
    return validate(root, LONG_MIN, LONG_MAX);  
}  
  
bool validate(TreeNode* node, long low, long high) {  
    if (!node) return true;  
    if (node->val <= low || node->val >= high) return false;  
    return validate(node->left, low, node->val) &&  
        validate(node->right, node->val, high);  
}
```

Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Input: root = [10,5,15,3,7,null,18], low = 7, high = 15

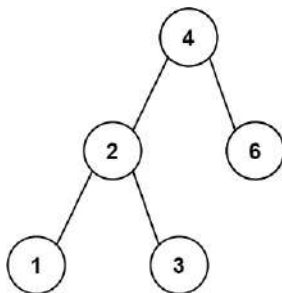
Output: 32

Explanation: Nodes 7, 10, and 15 are in the range [7, 15].  $7 + 10 + 15 = 32$ .



```
int rangeSumBST(TreeNode* root, int low, int high) {  
    if (!root) return 0;  
    if (root->val < low) return rangeSumBST(root->right, low, high);  
    if (root->val > high) return rangeSumBST(root->left, low, high);  
    return root->val +  
           rangeSumBST(root->left, low, high) +  
           rangeSumBST(root->right, low, high);  
}
```

Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree. Input: root = [4,2,6,1,3]. Output: 1



```
int res = INT_MAX;  
TreeNode* prev = NULL;
```



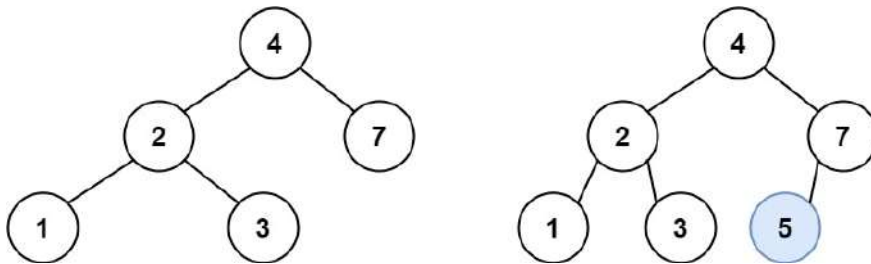
```

int getMinimumDifference(TreeNode* root) {
    inorder(root);
    return res;
}

void inorder(TreeNode* node) {
    if (!node) return;
    inorder(node->left);
    if (prev != NULL) {
        res = min(res, node->val - prev->val);
    }
    prev = node;
    inorder(node->right);
}

```

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST. Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them. Input: root = [4,2,7,1,3], val = 5. Output: [4,2,7,1,3,5]. Explanation: Another accepted tree is: [5,2,7,1,3,null,null,null,null,4].



```

TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val)
        root->left = insertIntoBST(root->left, val);
    else
        root->right = insertIntoBST(root->right, val);
    return root;
}

```

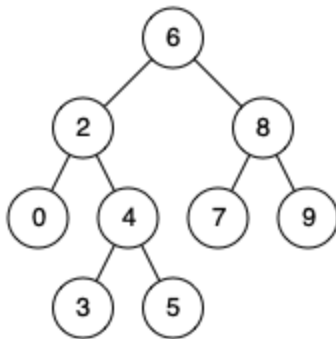
Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6. And for 2 & 4 is 2.



```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)
            root = root->left;
        else if (p->val > root->val && q->val > root->val)
            root = root->right;
        else
            return root;
    }
    return NULL;
}
```

DP-Take/Not Take

## DP - Take / Not take (DP)

Description: Solve optimization problems like selecting items with the max/min value under certain constraints.

- [\[Self Added\] House Robber i](#)
- [House Robber ii](#)
- [Target Sum](#)
- [Partition Equal Subset Sum](#)
- [Ones and Zeroes](#)
- [Last Stone Weight ii](#)

Given an integer array `nums` where each element represents the amount of money in a house arranged in a line – You cannot rob two adjacent houses. Find the maximum money you can rob.

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = 2 + 9 + 1 = 12.

```
int solve(int i, vector<int>& nums, vector<int>& dp) {
    int n = nums.size();
    if (i >= n) return 0; // past last house
    if (dp[i] != -1) return dp[i]; // memoized
    int rob = nums[i] + solve(i + 2, nums, dp);
    int skip = solve(i + 1, nums, dp);
    return dp[i] = max(rob, skip);
}

int rob(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, -1);
    return solve(0, nums, dp);
}
```

```

/*
solve(0)
├─ rob = 2 + solve(2)
│   └─ rob = 9 + solve(4)
│       └─ rob = 1 + solve(6=0)
│           └─ skip = solve(5=0)
│               → max = 1
│       └─ skip = solve(3)
│           └─ rob = 3 + solve(5=0)
│               └─ skip = solve(4)=1
│                   → max = 3
│       → max = 9 + 1 = 10
├─ skip = solve(1)
│   └─ rob = 7 + solve(3)
│       → 7 + 3 = 10
│   └─ skip = solve(2)=10
│       → max = 10
→ max = 2 + 10 = 12
*/

```

You're given an array `nums` where: Each element = amount of money in a house. Houses are arranged in a circle → first and last houses are adjacent. You cannot rob two adjacent houses. Return the maximum amount of money you can rob.

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

```

int solve(int i, vector<int>& nums, vector<int>& dp) {
    if (i >= nums.size()) return 0; // beyond last index
    if (dp[i] != -1) return dp[i]; // memoized result
    int rob = nums[i] + solve(i + 2, nums, dp); // rob this house
    int skip = solve(i + 1, nums, dp); // skip this house
    return dp[i] = max(rob, skip);
}

int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) return nums[0];
    vector<int> nums1(nums.begin(), nums.end() - 1); // exclude last
    vector<int> nums2(nums.begin() + 1, nums.end()); // exclude first
    vector<int> dp1(nums1.size(), -1);
    vector<int> dp2(nums2.size(), -1);
    return max(solve(0, nums1, dp1), solve(0, nums2, dp2));
}

```

You are given an integer array `nums` and an integer `target`. You must assign either '+' or '-' before each number and evaluate the expression. Return the number of different expressions that evaluate to target. Input: `nums = [1,1,1,1,1]`, `target = 3`.

Output: 5. Explanation: There are 5 ways to assign symbols to make the sum of `nums` be target 3. Eg:  $-1 + 1 + 1 + 1 + 1 = 3$ ,  $+1 - 1 + 1 + 1 + 1 = 3$ ,  $+1 + 1 - 1 + 1 + 1 = 3$ ,  $+1 + 1 + 1 - 1 + 1 = 3$ ,  $+1 + 1 + 1 + 1 - 1 = 3$

```

unordered_map<string, int> dp; // global variable
int solve(vector<int>& nums, int i, int currSum, int target) {
    if (i == nums.size())
        return currSum == target ? 1 : 0;
    string key = to_string(i) + "," + to_string(currSum);
    if (dp.count(key)) return dp[key];
    int plus = solve(nums, i + 1, currSum + nums[i], target);
    int minus = solve(nums, i + 1, currSum - nums[i], target);
    return dp[key] = plus + minus;
}

int findTargetSumWays(vector<int>& nums, int target) {
    return solve(nums, 0, 0, target);
}

```

```

/*
                (i=0, sum=0)
                /      \
            +1          -1
        (i=1,1)      (i=1,-1)
        /      \      /      \
    +1          -1    +1          -1
(i=2,2)      (i=2,0) (i=2,0)      (i=2,-2)
    /\          /\      /\          /\
+1  -1      +1  -1    +1  -1      +1  -1
(3,3)(3,1) (3,1)(3,-1)(3,1)(3,-1) (3,-1)(3,-3)
... continue 2 more levels ...
*/

```

You are given an integer array `nums`. Return `true` if you can partition it into two subsets with equal sum, otherwise `false`.

Input: `nums = [1,5,11,5]`. Output: `true`. Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

// Let total sum =  $S$ . For equal partition, both subsets must sum to  $S/2$ . So, the problem becomes: Can we find a subset of `nums` whose sum =  $S/2$ ? If yes  $\rightarrow$  the remaining elements also sum to  $S/2$ .

```

bool solve(int i, int currSum, int target, vector<int>& nums,
vector<vector<int>>& dp) {
    if (currSum == target) return true; // found a subset
    if (i == nums.size() || currSum > target) return false;
    if (dp[i][currSum] != -1) return dp[i][currSum];
    // Option 1: Take the current element
    bool take = solve(i + 1, currSum + nums[i], target, nums, dp);
    // Option 2: Skip the current element
    bool notTake = solve(i + 1, currSum, target, nums, dp);
    return dp[i][currSum] = (take || notTake);
}

bool canPartition(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (total % 2 != 0) return false; // odd sum  $\rightarrow$  cannot partition
    int target = total / 2; int n = nums.size();
    vector<vector<int>> dp(n, vector<int>(target + 1, -1));
    return solve(0, 0, target, nums, dp);
}

```

```

/*
(0, 0)
├── take 1 → (1, 1)
│   ├── take 5 → (2, 6)
│   │   ├── take 11 → (3, 17) ✗
│   │   └── skip → (3, 6)
│   └── skip 5 → (2, 1)
│       ├── take 11 → (3, 12) ✗
│       └── take 5 → (3, 6)
└── skip 1 → (1, 0)
    ├── take 5 → (2, 5)
    │   ├── take 11 → (3, 16) ✗
    │   └── take 5 → (3, 10)
    └── take 11 → (2, 11) ✓ FOUND!
*/

```

You are given an array of binary strings `strs` and two integers `m` and `n`. Return the size of the largest subset of `strs` such that there are at most `m` 0's and `n` 1's in the subset.

A set `x` is a subset of a set `y` if all elements of `x` are also elements of `y`.

Input: `strs = ["10","0001","111001","1","0"]`, `m = 5`, `n = 3`

Output: 4

Explanation: The largest subset with at most 5 0's and 3 1's is `{"10", "0001", "1", "0"}`, so the answer is 4.

Other valid but smaller subsets include `{"0001", "1"}` and `{"10", "1", "0"}`.

`{"111001"}` is an invalid subset because it contains 4 1's, greater than the maximum of 3.



```

vector<vector<int>> cnt;
vector<vector<vector<int>>> dp;
int solve(int i, int m, int n, const vector<string>& strs) {
    if (i == strs.size()) return 0;
    int &res = dp[i][m][n];
    if (res != -1) return res;
    int zeros = cnt[i][0];
    int ones  = cnt[i][1];
    // skip
    int notTake = solve(i + 1, m, n, strs);
    // take (if fits)
    int take = 0;
    if (m >= zeros && n >= ones)
        take = 1 + solve(i + 1, m - zeros, n - ones, strs);
    return res = max(take, notTake);
}
int findMaxForm(vector<string>& strs, int m, int n) {
    int N = strs.size();
    cnt.assign(N, vector<int>(2, 0)); // Same as: cnt = [[0, 0], [0,
0], [0, 0],...N times]
    for (int i = 0; i < N; i++) {
        int z = 0, o = 0;
        for (char c : strs[i])
            (c == '0') ? z++ : o++;
        cnt[i][0] = z;
        cnt[i][1] = o;
    }
    dp.assign(N+1, vector<vector<int>>(m+1, vector<int>(n+1,-1)));
    return solve(0, m, n, strs);
}

```

You are given an array of integers stones where stones[i] is the weight of the ith stone. We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights x and y with  $x \leq y$ . The result of this smash is: If  $x == y$ , both stones are destroyed, and If  $x \neq y$ , the stone of weight x is destroyed, and the stone of weight y has new weight  $y - x$ . At the end of the game, there is at most one stone left. Return the smallest possible weight of the left stone. If there are no stones left, return 0. Input: stones = [2,7,4,1,8,1]. Output: 1 Explanation: We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then, we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then, we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then, we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.

```
bool solve(int i, int sum, vector<int>& stones, vector<vector<int>>& dp){
    if (sum == 0) return true;
    if (i == stones.size()) return false;
    if (dp[i][sum] != -1) return dp[i][sum];
    bool notTake = solve(i+1, sum, stones, dp);
    bool take = false;
    if (sum >= stones[i])
        take = solve(i+1, sum - stones[i], stones, dp);
    return dp[i][sum] = (take || notTake);
}

int lastStoneWeightII(vector<int>& stones) {
    int total = accumulate(stones.begin(), stones.end(), 0);
    int target = total / 2; int n = stones.size();
    vector<vector<int>> dp(n, vector<int>(target + 1, -1));
    int best = 0;
    for (int s = target; s >= 0; s--) {
        if (solve(0, s, stones, dp)) {
            best = s;
            break;
        }
    }
    return total - 2 * best;}
```

DP-InfiniteSupply

## DP - Infinite Supply (DP)

Description: Similar to the 0/1 knapsack, but items can be chosen multiple times.

- [Coin Change](#)
- [Coin Change II](#)
- [Perfect Squares](#)
- [Minimum Cost For Tickets](#)
- [\[Self added v2\] Rod Cutting Problem](#)

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Input: `coins = [1,2,5]`, `amount = 11`

Output: 3

Explanation:  $11 = 5 + 5 + 1$

```
vector<int> dp;    // dp[x] = min coins for amount x
int INF = INT_MAX/2;
int solve(vector<int>& coins, int amount) {
    if (amount == 0) return 0;
    if (amount < 0) return INF;
    if (dp[amount] != -1) return dp[amount];
    int best = INF;
    for (int c : coins) {
        best = min(best, 1 + solve(coins, amount - c));
    }
    return dp[amount] = best;
}
int coinChange(vector<int>& coins, int amount) {
    dp.assign(amount + 1, -1);
    int ans = solve(coins, amount);
    return (ans >= INF ? -1 : ans);
}
```

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

Input: `amount = 5, coins = [1,2,5]`

Output: 4

Explanation: there are four ways to make up the amount:

5=5

5=2+2+1

5=2+1+1+1

5=1+1+1+1+1

```
vector<vector<int>> dp;
int solve(int i, int amount, vector<int>& coins) {
    if (amount == 0) return 1;
    if (i == coins.size()) return 0;
    if (dp[i][amount] != -1) return dp[i][amount];
    long long ways = 0;
    // take unlimited
    if (amount >= coins[i])
        ways += solve(i, amount - coins[i], coins);
    // skip
    ways += solve(i + 1, amount, coins);
    return dp[i][amount] = (int)ways;
}
int change(int amount, vector<int>& coins) {
    dp.assign(coins.size(), vector<int>(amount + 1, -1));
    return solve(0, amount, coins);
}
```

Given an integer  $n$ , return the least number of perfect square numbers that sum to  $n$ .

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Input:  $n = 12$ . Output: 3

Explanation:  $12 = 4 + 4 + 4$ .

```
vector<int> dp;
int INF = INT_MAX/2;
int solve(int n, vector<int>& sq) {
    if (n == 0) return 0;
    if (n < 0) return INF;
    if (dp[n] != -1) return dp[n];
    int best = INF;
    for (int s : sq)
        best = min(best, 1 + solve(n - s, sq));
    return dp[n] = best;
}
int numSquares(int n) {
    vector<int> sq;
    for (int i = 1; i * i <= n; i++)
        sq.push_back(i * i);
    dp.assign(n + 1, -1);
    return solve(n, sq);
}
```

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array `days`. Each day is an integer from 1 to 365.

Train tickets are sold in three different ways:

a 1-day pass is sold for `costs[0]` dollars, a 7-day pass is sold for `costs[1]` dollars, and a 30-day pass is sold for `costs[2]` dollars.

The passes allow that many days of consecutive travel.

For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return the minimum number of dollars you need to travel every day in the given list of days.

Input: `days = [1,4,6,7,8,20]`, `costs = [2,7,15]`

**Output: 11**

**Explanation:** For example, here is one way to buy passes that lets you travel your travel plan:

On day 1, you bought a 1-day pass for  $\text{costs}[0] = \$2$ , which covered day 1.

On day 3, you bought a 7-day pass for  $\text{costs}[1] = \$7$ , which covered days 3, 4, ..., 9.

On day 20, you bought a 1-day pass for  $\text{costs}[0] = \$2$ , which covered day 20.

In total, you spent \$11 and covered all the days of your travel.

```
vector<int> dp;
vector<int> d, cost;
int solve(int i) {
    if (i == d.size()) return 0;
    if (dp[i] != -1) return dp[i];
    // cost for 1-day
    int c1 = cost[0] + solve(i + 1);
    // cost for 7-day
    int j = i;
    while (j < d.size() && d[j] <= d[i] + 6) j++;
    int c7 = cost[1] + solve(j);
    // cost for 30-day
    j = i;
    while (j < d.size() && d[j] <= d[i] + 29) j++;
    int c30 = cost[2] + solve(j);
    return dp[i] = min(c1, min(c7, c30));
}
int mincostTickets(vector<int>& days, vector<int>& costs) {
    d = days;
    cost = costs;
    dp.assign(d.size(), -1);
    return solve(0);
}
```

**You are given:**

A rod of length N

length[] array → available cut lengths

price[] array → price of each cut size

You can cut the rod into pieces such that sum of lengths = N and you maximize the profit. (Unbounded Knapsack)

N = 8

length = [1,2,3,4]

price = [3,7,9,10]

Explain: If say: (8 cuts of size 1 →  $8 * 3 = 24$  profit). But (2 + 2 + 2 + 2 = 4 cuts of size 2 →  $4 * 7 = 28$ )

Hence best profit: 28.

```
int solve(int idx, int rem, vector<int> &len, vector<int> &price,
          vector<vector<int>> &dp) {
    // no more indices left
    if (idx == len.size()) return 0;
    if (dp[idx][rem] != -1) return dp[idx][rem];
    // Option 1: Do NOT take this cut size
    int notTake = solve(idx + 1, rem, len, price, dp);
    // Option 2: TAKE this cut size if it fits
    int take = INT_MIN;
    if (len[idx] <= rem) {
        take = price[idx] + solve(idx, rem-len[idx], len, price, dp);
    }
    return dp[idx][rem] = max(take, notTake);
}
```

```
int cutRod(vector<int> &price, int N) {
    vector<int> len(N);
    for (int i = 0; i < N; i++) len[i] = i + 1; // lengths = 1..N
    vector<vector<int>> dp(N, vector<int>(N + 1, -1));
    return solve(0, N, len, price, dp);
}
```



DP-LIS

## DP - Longest Increasing subsequence

Description: It involves finding the longest subsequence of a given sequence where the elements are in ascending order

- [Longest Increasing Subsequence](#)
- [Largest Divisible Subset](#)
- [Maximum Length of Pair Chain](#)
- [Number of LIS](#)
- [Longest String Chain](#)

Given an integer array nums, return the length of the longest strictly increasing subsequence

Input: nums = [10,9,2,5,3,7,101,18]

Output: 4

Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<int>& nums) {
    if (i == nums.size()) return 0;
    if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
    int skip = solve(i + 1, prev, nums);
    int take = 0;
    if (prev == -1 || nums[i] > nums[prev])
        take = 1 + solve(i + 1, i, nums);
    return dp[i][prev + 1] = max(skip, take);
}
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
    return solve(0, -1, nums);
}
```

Given a set of distinct positive integers `nums`, return the largest subset `answer` such that every pair `(answer[i], answer[j])` of elements in this subset satisfies:

`answer[i] % answer[j] == 0`, or `answer[j] % answer[i] == 0`

If there are multiple solutions, return any of them.

Input: `nums = [1,2,3]`

Output: `[1,2]`

Explanation: `[1,3]` is also accepted.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<int>& nums) {
    if (i == nums.size()) return 0;
    if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
    int skip = solve(i + 1, prev, nums);
    int take = 0;
    if (prev == -1 || nums[i] % nums[prev] == 0)
        take = 1 + solve(i + 1, i, nums);
    return dp[i][prev + 1] = max(skip, take);
}
vector<int> largestDivisibleSubset(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    int n = nums.size();
    dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
    solve(0, -1, nums);
    vector<int> ans; int i = 0, prev = -1;
    while (i < n) {
        int best = dp[i][prev + 1];
        int skip = solve(i + 1, prev, nums);
        if (best == skip) {
            i++;
        } else {
            ans.push_back(nums[i]);
            prev = i;
            i++;
        }
    }
    return ans;
}
```

You are given an array of  $n$  pairs `pairs` where `pairs[i] = [lefti, righti]` and `lefti < righti`.

A pair `p2 = [c, d]` follows a pair `p1 = [a, b]` if `b < c`. A chain of pairs can be formed in this fashion.

Return the length longest chain which can be formed.

You do not need to use up all the given intervals. You can select pairs in any order.

Input: `pairs = [[1,2],[2,3],[3,4]]`

Output: 2

Explanation: The longest chain is `[1,2] -> [3,4]`.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<vector<int>>& p) {
    if (i == p.size()) return 0;
    if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
    int skip = solve(i + 1, prev, p);
    int take = 0;
    if (prev == -1 || p[prev][1] < p[i][0])
        take = 1 + solve(i + 1, i, p);
    return dp[i][prev + 1] = max(skip, take);
}
int findLongestChain(vector<vector<int>>& pairs) {
    sort(pairs.begin(), pairs.end());
    int n = pairs.size();
    dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
    return solve(0, -1, pairs);
}
```

Given an integer array `nums`, return the number of longest increasing subsequences.

Notice that the sequence has to be strictly increasing.

Input: nums = [1,3,5,4,7]

Output: 2

Explanation: The two longest increasing subsequences are [1, 3, 4, 7] and [1, 3, 5, 7].

```
vector<vector<int>> lenDP, cntDP;
pair<int,int> solve(int i, int prev, vector<int>& nums) {
    if (i == nums.size()) return {0, 1};
    if (lenDP[i][prev+1] != -1)
        return {lenDP[i][prev+1], cntDP[i][prev+1]};
    // skip
    auto skip = solve(i + 1, prev, nums);
    // take
    pair<int,int> take = {0, 0};
    if (prev == -1 || nums[i] > nums[prev]) {
        auto r = solve(i + 1, i, nums);
        take.first = 1 + r.first;
        take.second = r.second;
    }
    // compare
    int L = max(skip.first, take.first);
    int C = 0;
    if (skip.first == L) C += skip.second;
    if (take.first == L) C += take.second;
    lenDP[i][prev+1] = L;
    cntDP[i][prev+1] = C;
    return {L, C};
}
int findNumberOfLIS(vector<int>& nums) {
    int n = nums.size();
    lenDP = cntDP = vector<vector<int>>(n, vector<int>(n + 1, -1));
    return solve(0, -1, nums).second;
}
```

You are given an array of words where each word consists of lowercase English letters. wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in wordA without changing the order of the other characters to make it equal to

wordB. For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad". A word chain is a sequence of words [word1, word2, ..., wordk] with  $k \geq 1$ , where word1 is a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is trivially a word chain with  $k = 1$ . Return the length of the longest possible word chain with words chosen from the given list of words. Input: words = ["a","b","ba","bca","bda","bdca"]. Output: 4. Explanation: One of the longest word chains is ["a","ba","bda","bdca"].

```
vector<vector<int>> dp;
bool isPred(string& a, string& b) {
    if (b.size() != a.size() + 1) return false;
    int i = 0, j = 0;
    while (j < b.size()) {
        if (i < a.size() && a[i] == b[j]) i++;
        j++;
    }
    return i == a.size();
}
int solve(int i, int prev, vector<string>& w) {
    if (i == w.size()) return 0;
    if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
    int skip = solve(i + 1, prev, w);
    int take = 0;
    if (prev == -1 || isPred(w[prev], w[i])) take = 1 + solve(i + 1, i, w);
    return dp[i][prev + 1] = max(skip, take);
}
int longestStrChain(vector<string>& words) {
    sort(words.begin(), words.end(),
        [](string& a, string& b) { return a.size() < b.size(); });
    int n = words.size();
    dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
    return solve(0, -1, words);
}
```

# DP-Grids

## DP - DP on Grids

Description: Dynamic Programming on matrices involves solving problems that can be broken down into smaller overlapping subproblems within a matrix.

- [\[Self Added\] Unique Paths i](#)
- [Unique Paths ii](#)
- [Minimum Path Sum](#)
- [Triangle](#)
- [Minimum Falling Path Sum](#)
- [Maximal Square](#)
- [Cherry Pickup](#)
- [Dungeon Game: Dungeon Game](#)

There is a robot on an  $m \times n$  grid. The robot is initially located at the top-left corner (i.e.,  $\text{grid}[0][0]$ ). The robot tries to move to the bottom-right corner (i.e.,  $\text{grid}[m - 1][n - 1]$ ). The robot can only move either down or right at any point in time.

Given the two integers  $m$  and  $n$ , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to  $2 * 10^9$ .

Input:  $m = 3, n = 7$

Output: 28



```
vector<vector<int>> dp;
```



```

int solve(int i, int j, int m, int n) {
    if (i == m - 1 && j == n - 1) return 1;
    if (i >= m || j >= n) return 0;
    int &res = dp[i][j];
    if (res != -1) return res;
    return res = solve(i + 1, j, m, n) + solve(i, j + 1, m, n);
}

int uniquePaths(int m, int n) {
    dp.assign(m, vector<int>(n, -1));
    return solve(0, 0, m, n);
}

```

You are given an  $m \times n$  integer array grid. There is a robot initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The testcases are generated so that the answer will be less than or equal to  $2 * 10^9$ .

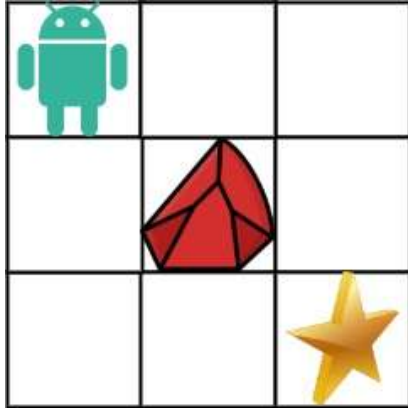
Input: `obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]`

Output: 2

Explanation: There is one obstacle in the middle of the 3x3 grid above.

There are two ways to reach the bottom-right corner:

1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right



```
vector<vector<long long>> dp;
long long solve(int i, int j, vector<vector<int>>& g) {
    int m = g.size(), n = g[0].size();
    if (i >= m || j >= n) return 0;
    if (g[i][j] == 1) return 0;
    if (i == m - 1 && j == n - 1) return 1;
    long long &res = dp[i][j];
    if (res != -1) return res;
    return res = solve(i + 1, j, g) + solve(i, j + 1, g);
}
int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
    int m = obstacleGrid.size(), n = obstacleGrid[0].size();
    dp.assign(m, vector<long long>(n, -1));
    return (int)solve(0, 0, obstacleGrid);
}
```

Given a  $m \times n$  grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path  $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$  minimizes the sum.

|   |   |   |
|---|---|---|
| 1 | 3 | 1 |
| 1 | 5 | 1 |
| 4 | 2 | 1 |

```

vector<vector<int>> dp;
int solve(int i, int j, vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    if (i >= m || j >= n) return 1e9;
    if (i == m - 1 && j == n - 1) return grid[i][j];
    int &res = dp[i][j];
    if (res != -1) return res;
    int down = solve(i + 1, j, grid);
    int right = solve(i, j + 1, grid);
    return res = grid[i][j] + min(down, right);
}
int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    dp.assign(m, vector<int>(n, -1));
    return solve(0, 0, grid);
}

```

Given a triangle array, return the minimum path sum from top to bottom. For each step, you may move to an adjacent number of the row below. More formally, if you are on index  $i$  on the current row, you may move to either index  $i$  or index  $i + 1$  on the next row.

Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]

Output: 11

Explanation: The triangle looks like:

```

  2
 3 4
6 5 7
4 1 8 3

```

The minimum path sum from top to bottom is  $2 + 3 + 5 + 1 = 11$  (underlined above).

```

int minimumTotal(vector<vector<int>>& triangle) {
    // top down gives TLE, hence using bottom up dp
    int n = triangle.size();
    vector<int> dp = triangle.back();    // start with last row
    for (int row = n - 2; row >= 0; row--) {
        for (int col = 0; col <= row; col++) {
            dp[col] = triangle[row][col] + min(dp[col], dp[col+1]);
        }
    }
    return dp[0];
}

```

Given an  $n \times n$  array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]

Output: 13

Explanation: There are two falling paths with a minimum sum as shown.

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

```

int minFallingPathSum(vector<vector<int>>& matrix) {
    // top down TLE, hence bottom up
    int n = matrix.size();
    vector<vector<int>> dp = matrix;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int best = dp[i-1][j];
            if (j > 0) best = min(best, dp[i-1][j-1]);
            if (j+1 < n) best = min(best, dp[i-1][j+1]);

            dp[i][j] = matrix[i][j] + best;
        }
    }

    return *min_element(dp[n-1].begin(), dp[n-1].end());
}

```

Given an  $m \times n$  binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Input: matrix =

```

[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],
["1","0","0","1","0"]]

```

Output: 4

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 |

```

int maximalSquare(vector<vector<char>>& matrix) {
    // top down TLE, hence bottom up
    int m = matrix.size(), n = matrix[0].size();
    vector<vector<int>> dp(m, vector<int>(n, 0));
    int best = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == '1') {
                if (i == 0 || j == 0) dp[i][j] = 1;
                else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1],
dp[i-1][j-1]});
                best = max(best, dp[i][j]);
            }
        }
    }
    return best * best;    // return area
}

```

[hard]

You are given an  $n \times n$  grid representing a field of cherries, each cell is one of three possible integers.

0 means the cell is empty, so you can pass through,

1 means the cell contains a cherry that you can pick up and pass through, or

-1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

Starting at the position  $(0, 0)$  and reaching  $(n - 1, n - 1)$  by moving right or down through valid path cells (cells with value 0 or 1).

After reaching  $(n - 1, n - 1)$ , returning to  $(0, 0)$  by moving left or up through valid path cells.

When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.

If there is no valid path between  $(0, 0)$  and  $(n - 1, n - 1)$ , then no cherries can be collected.

Input: `grid = [[0,1,-1],[1,0,-1],[1,1,1]]`

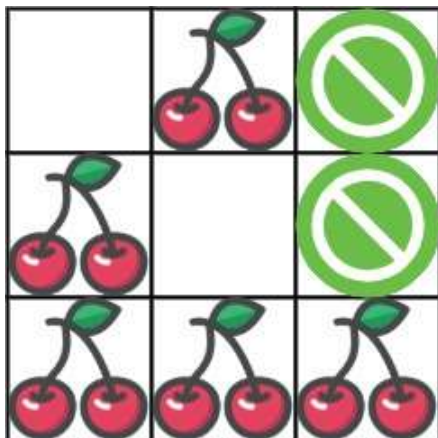
Output: 5

Explanation: The player started at  $(0, 0)$  and went down, down, right right to reach  $(2, 2)$ .

4 cherries were picked up during this single trip, and the matrix becomes `[[0,1,-1],[0,0,-1],[0,0,0]]`.

Then, the player went left, up, up, left to return home, picking up one more cherry.

The total number of cherries picked up is 5, and this is the maximum possible.



```

int n;
vector<vector<int>> g;
int dp[51][51][51];
int solve(int r1, int c1, int r2) {
    // top down TLE, hence bottom up
    int c2 = r1 + c1 - r2;
    if (r1 >= n || c1 >= n || r2 >= n || c2 >= n) return -1e9;
    if (g[r1][c1] == -1 || g[r2][c2] == -1) return -1e9;
    if (r1 == n-1 && c1 == n-1) return g[r1][c1];
    int &res = dp[r1][c1][r2];
    if (res != -1) return res;
    int best = max({
        solve(r1+1, c1, r2+1),
        solve(r1, c1+1, r2),
        solve(r1+1, c1, r2),
        solve(r1, c1+1, r2+1)
    });
    if (best < 0) return res = -1e9;
    int cherries = g[r1][c1];
    if (r1 != r2 || c1 != c2) cherries += g[r2][c2];
    return res = cherries + best;
}
int cherryPickup(vector<vector<int>>& grid) {
    n = grid.size();
    g = grid;
    memset(dp, -1, sizeof(dp));
    int ans = solve(0,0,0);
    return max(0, ans);
}

```



[hard]

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of  $m \times n$  rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Input: `dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]`

Output: 7

Explanation: The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.

|    |     |    |
|----|-----|----|
| -2 | -3  | 3  |
| -5 | -10 | 1  |
| 10 | 30  | -5 |

```

vector<vector<int>> dp;
int solve(int i, int j, vector<vector<int>>& d) {
    int m = d.size(), n = d[0].size();
    if (i >= m || j >= n) return 1e9;
    if (i == m - 1 && j == n - 1)
        return max(1, 1 - d[i][j]);
    int &res = dp[i][j];
    if (res != -1) return res;
    int down = solve(i + 1, j, d);
    int right = solve(i, j + 1, d);
    int need = min(down, right) - d[i][j];
    return res = max(1, need);
}
int calculateMinimumHP(vector<vector<int>>& dungeon) {
    int m = dungeon.size(), n = dungeon[0].size();
    dp.assign(m, vector<int>(n, -1));
    return solve(0, 0, dungeon);
}

```

# DP-Strings

## DP - DP on Strings

Description: It Involves 2 strings, whenever you are considering two substrings/subsequence from given two strings, concentrate on what happens when the last characters of the two substrings are same, i.e, matching.

- [Longest Common Subsequence](#)
- [Longest Palindromic Subsequence](#)
- [Palindromic Substrings](#)
- [Longest Palindromic Substrings](#)
- [Edit Distance](#)
- [Minimum ASCII Delete Sum for Two Strings](#)
- [Distinct Subsequences](#)
- [Shortest Common Supersequence](#)
- [Wildcard Matching](#)

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Input: text1 = "abcde", text2 = "ace"

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

```

string a, b;
vector<vector<int>> dp;
int solve(int i, int j) {
    if (i == a.size() || j == b.size()) return 0;
    int &res = dp[i][j];
    if (res != -1) return res;
    if (a[i] == b[j])
        return res = 1 + solve(i+1, j+1);
    return res = max(solve(i+1, j), solve(i, j+1));
}
int longestCommonSubsequence(string s1, string s2) {
    a = s1; b = s2;
    dp.assign(a.size(), vector<int>(b.size(), -1));
    return solve(0, 0);
}

```

**Given a string s, find the longest palindromic subsequence's length in s. A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements. Input: s = "bbbab". Output: 4.**

**Explanation: One possible longest palindromic subsequence is "bbbb".**

```

string s; vector<vector<int>> dp;
int solve(int i, int j) {
    if (i > j) return 0;
    if (i == j) return 1;
    int &res = dp[i][j];
    if (res != -1) return res;
    if (s[i] == s[j])
        return res = 2 + solve(i+1, j-1);
    return res = max(solve(i+1, j), solve(i, j-1));
}
int longestPalindromeSubseq(string str) {
    s = str; int n = s.size();
    dp.assign(n, vector<int>(n, -1));
    return solve(0, n-1);
}

```

Given a string *s*, return the number of palindromic substrings in it. A string is a palindrome when it reads the same backward as forward. A substring is a contiguous sequence of characters within the string.

Input: *s* = "aaa"

Output: 6

Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

```
int countSubstrings(string s) {
    // bottom up better than top down
    int n = s.size();
    vector<vector<bool>> dp(n, vector<bool>(n, false));
    int ans = 0;
    for (int len = 1; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (s[i] == s[j] && (len <= 2 || dp[i+1][j-1])) {
                dp[i][j] = true;
                ans++;
            }
        }
    }
    return ans;
}
```

Given a string *s*, return the longest palindromic substring in *s*.

Input: *s* = "babad"

Output: "bab"

Explanation: "aba" is also a valid answer.

```
bool isPalin(const string &s, int i, int j) {
    while (i < j) {
```

```

        if (s[i] != s[j]) return false;
        i++;
        j--;
    }
    return true;
}
string longestPalindrome(string s) {
    int n = s.size();
    int maxLen = 0, start = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (isPalin(s, i, j)) {
                int len = j - i + 1;
                if (len > maxLen) {
                    maxLen = len;
                    start = i;
                }
            }
        }
    }
    return s.substr(start, maxLen);
}

```

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

Insert a character

Delete a character

Replace a character

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse -> rorse (replace 'h' with 'r')

rorse -> rose (remove 'r')

rose -> ros (remove 'e')

string a, b;

vector<vector<int>> dp;

```

int solve(int i, int j) {
    if (i == a.size()) return b.size() - j; // insert all remaining
    if (j == b.size()) return a.size() - i; // delete all remaining
    int &res = dp[i][j];
    if (res != -1) return res;
    if (a[i] == b[j])
        return res = solve(i+1, j+1);
    int insertOp = 1 + solve(i, j+1);
    int deleteOp = 1 + solve(i+1, j);
    int replaceOp = 1 + solve(i+1, j+1);
    return res = min({insertOp, deleteOp, replaceOp});
}

int minDistance(string s1, string s2) {
    a = s1; b = s2;
    dp.assign(a.size(), vector<int>(b.size(), -1));
    return solve(0, 0);
}

```

Given two strings s1 and s2, return the lowest ASCII sum of deleted characters to make two strings equal.

Input: s1 = "sea", s2 = "eat"

Output: 231

Explanation: Deleting "s" from "sea" adds the ASCII value of "s" (115) to the sum.

Deleting "t" from "eat" adds 116 to the sum.

At the end, both strings are equal, and  $115 + 116 = 231$  is the minimum sum possible to achieve this.

string a, b;

vector<vector<int>> dp;



```

int solve(int i, int j) {
    if (i == a.size()) {
        int sum = 0;
        for (int k = j; k < b.size(); k++) sum += b[k];
        return sum;
    }
    if (j == b.size()) {
        int sum = 0;
        for (int k = i; k < a.size(); k++) sum += a[k];
        return sum;
    }
    int &res = dp[i][j];
    if (res != -1) return res;
    if (a[i] == b[j])
        return res = solve(i+1, j+1);
    int deleteA = a[i] + solve(i+1, j);
    int deleteB = b[j] + solve(i, j+1);
    return res = min(deleteA, deleteB);
}

int minimumDeleteSum(string s1, string s2) {
    a = s1; b = s2;
    dp.assign(a.size(), vector<int>(b.size(), -1));
    return solve(0, 0);
}

```

**[hard]**

Given two strings *s* and *t*, return the number of distinct subsequences of *s* which equals *t*.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Input: *s* = "rabbbit", *t* = "rabbit"

Output: 3

Explanation:

As shown below, there are 3 ways you can generate "rabbit" from *s*.  
rabbbit, rabbbit, rabbbit

string *s*, *t*;

vector<vector<long long>> dp;

```

long long solve(int i, int j) {
    if (j == t.size()) return 1; // matched full t
    if (i == s.size()) return 0; // s ended but t not finished
    long long &res = dp[i][j];
    if (res != -1) return res;
    long long ways = 0;
    if (s[i] == t[j])
        ways += solve(i+1, j+1);
    ways += solve(i+1, j); // skip char in s
    return res = ways;
}

int numDistinct(string S, string T) {
    s = S; t = T;
    dp.assign(s.size(), vector<long long>(t.size(), -1));
    return solve(0, 0);
}

```

**[hard]**

Given two strings `str1` and `str2`, return the shortest string that has both `str1` and `str2` as subsequences. If there are multiple valid strings, return any of them.

A string `s` is a subsequence of string `t` if deleting some number of characters from `t` (possibly 0) results in the string `s`.

Input: `str1 = "abac"`, `str2 = "cab"`

Output: `"cabac"`

Explanation:

`str1 = "abac"` is a subsequence of `"cabac"` because we can delete the first `"c"`.

`str2 = "cab"` is a subsequence of `"cabac"` because we can delete the last `"ac"`.

The answer provided is the shortest such string that satisfies these properties.

```

string shortestCommonSupersequence(string a, string b) {
    int n = a.size(), m = b.size();

```

```

vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
// LCS table
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a[i-1] == b[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
}
// Construct SCS
string res = "";
int i = n, j = m;
while (i > 0 && j > 0) {
    if (a[i-1] == b[j-1]) {
        res.push_back(a[i-1]);
        i--, j--;
    } else if (dp[i-1][j] >= dp[i][j-1]) {
        res.push_back(a[i-1]);
        i--;
    } else {
        res.push_back(b[j-1]);
        j--;
    }
}
while (i > 0) res.push_back(a[i-1]), i--;
while (j > 0) res.push_back(b[j-1]), j--;
reverse(res.begin(), res.end());
return res;
}

```

**[hard]**

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '\*' where:

'?' Matches any single character.

'\*' Matches any sequence of characters (including the empty sequence). The matching should cover the entire input string (not partial). Input: s = "aa", p = "\*". Output: true

Explanation: '\*' matches any sequence.

Input: s = "cb", p = "?a"

Output: false

Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.

```
string s, p;
vector<vector<int>> dp;
bool solve(int i, int j) {
    if (j == p.size()) return i == s.size();
    if (i == s.size()) {
        // s ended → only valid if remaining p is all '*'
        for (int k = j; k < p.size(); k++)
            if (p[k] != '*') return false;
        return true;
    }
    int &res = dp[i][j];
    if (res != -1) return res;
    bool ans = false;
    if (p[j] == s[i] || p[j] == '?')
        ans = solve(i+1, j+1);
    else if (p[j] == '*') {
        ans = solve(i, j+1)      // '*' = empty
            || solve(i+1, j);    // '*' consumes s[i]
    }
    return res = ans;
}
bool isMatch(string S, string P) {
    s = S; p = P;
    dp.assign(s.size(), vector<int>(p.size(), -1));
    return solve(0, 0);
}
```

DP-Stocks

## DP - DP on Stocks

**Description:** It focuses on maximizing profit from buying and selling stocks over time while considering constraints.

- [\[Self Added\] Buy and Sell Stocks i](#)
- [Buy and Sell Stocks ii](#)
- [Buy and Sell Stocks iii](#)
- [Buy and Sell Stocks iv](#)
- [Buy and Sell Stocks with Cooldown](#)
- [Buy and Sell Stocks with Transaction fee](#)

You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Input: prices = [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

```
int maxProfit(vector<int>& prices) {
    int mn = prices[0];
    int ans = 0;
    for (int i = 1; i < prices.size(); i++) {
        ans = max(ans, prices[i] - mn);
        mn = min(mn, prices[i]);
    }
    return ans;
}
```

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

Input: prices = [7,1,5,3,6,4]

Output: 7

Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit =  $5 - 1 = 4$ .

Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit =  $6 - 3 = 3$ .

Total profit is  $4 + 3 = 7$ .

```
int maxProfit(vector<int>& prices) {
    int ans = 0;
    for (int i = 1; i < prices.size(); i++) {
        if (prices[i] > prices[i-1])
            ans += prices[i] - prices[i-1];
    }
    return ans;
}
```

[hard]

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Input: prices = [3,3,5,0,0,3,1,4]

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit =  $3 - 0 = 3$ .

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit =  $4 - 1 = 3$ .

```
vector<vector<vector<int>>> dp;
vector<int> prices;
```

```

int solve(int i, int buy, int t) {
    if (i == prices.size() || t == 0) return 0;
    int &res = dp[i][buy][t];
    if (res != -1) return res;
    if (buy) {
        int take = -prices[i] + solve(i+1, 0, t);
        int skip = solve(i+1, 1, t);
        return res = max(take, skip);
    } else {
        int sell = prices[i] + solve(i+1, 1, t-1);
        int skip = solve(i+1, 0, t);
        return res = max(sell, skip);
    }
}

int maxProfit(vector<int>& p) {
    prices = p;
    int n = prices.size();
    dp.assign(n, vector<vector<int>>(2, vector<int>(3, -1)));
    return solve(0, 1, 2);
}

```

[hard]

You are given an integer array prices where prices[i] is the price of a given stock on the ith day, and an integer k. Find the maximum profit you can achieve. You may complete at most k transactions: i.e. you may buy at most k times and sell at most k times.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Input: k = 2, prices = [3,2,6,5,0,3]

Output: 7

Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

```

vector<vector<vector<int>>> dp;
vector<int> prices;

```



```

int K;
int solve(int i, int buy, int t) {
    if (i == prices.size() || t == 0) return 0;
    int &res = dp[i][buy][t];
    if (res != -1) return res;
    if (buy) {
        int take = -prices[i] + solve(i+1, 0, t);
        int skip = solve(i+1, 1, t);
        return res = max(take, skip);
    } else {
        int sell = prices[i] + solve(i+1, 1, t-1);
        int skip = solve(i+1, 0, t);
        return res = max(sell, skip);
    }
}

int maxProfit(int k, vector<int>& p) {
    prices = p;
    K = k;
    int n = prices.size();
    dp.assign(n, vector<vector<int>>(2, vector<int>(k+1, -1)));
    return solve(0, 1, k);
}

```

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

**Note:** You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

**Input:** prices = [1,2,3,0,2]

**Output:** 3

**Explanation:** transactions = [buy, sell, cooldown, buy, sell]

vector<vector<int>> dp;

vector<int> prices;

```

int solve(int i, int state) {
    if (i >= prices.size()) return 0;
    int &res = dp[i][state];
    if (res != -1) return res;
    if (state == 0) { // can buy
        int buy = -prices[i] + solve(i+1, 1);
        int skip = solve(i+1, 0);
        return res = max(buy, skip);
    }
    else if (state == 1) { // have stock
        int sell = prices[i] + solve(i+1, 2);
        int skip = solve(i+1, 1);
        return res = max(sell, skip);
    }
    else { // cooldown
        return res = solve(i+1, 0);
    }
}

int maxProfit(vector<int>& p) {
    prices = p;
    int n = prices.size();
    dp.assign(n, vector<int>(3, -1));
    return solve(0, 0);
}

```

You are given an array prices where prices[i] is the price of a given stock on the ith day, and an integer fee representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

**Note:**

You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

The transaction fee is only charged once for each stock purchase and sale.

Input: prices = [1,3,2,8,4,9], fee = 2

Output: 8

**Explanation:** The maximum profit can be achieved by:

- Buying at prices[0] = 1
- Selling at prices[3] = 8
- Buying at prices[4] = 4
- Selling at prices[5] = 9

The total profit is  $((8 - 1) - 2) + ((9 - 4) - 2) = 8$ .

```
vector<vector<int>> dp;
```

```
vector<int> prices;
```

```
int fee;
```

```
int solve(int i, int buy) {
    if (i == prices.size()) return 0;
    int &res = dp[i][buy];
    if (res != -1) return res;
    if (buy) {
        int take = -prices[i] + solve(i+1, 0);
        int skip = solve(i+1, 1);
        return res = max(take, skip);
    } else {
        int sell = prices[i] - fee + solve(i+1, 1);
        int skip = solve(i+1, 0);
        return res = max(sell, skip);
    }
}

int maxProfit(vector<int>& p, int f) {
    prices = p;
    fee = f;
    int n = prices.size();
    dp.assign(n, vector<int>(2, -1));
    return solve(0, 1);
}
```

DP-MCM

## DP - Partition DP (MCM)

**Description:** It involves a sequence that needs to be divided into partitions in an optimal way. The goal is often to minimize or maximize a cost function, such as computation time, multiplications, or some other metric, by exploring all possible partitions and combining results from subproblems.

- [Partition array for Maximum Sum](#)
- [Minimum Cost to Cut a Stick](#)
- [Burst Balloons](#)
- [Palindrome Partitioning ii](#)
- [\[Self added v2\] Egg Dropping](#)

Given an integer array `arr`, partition the array into (contiguous) subarrays of length at most `k`. After partitioning, each subarray has their values changed to become the maximum value of that subarray. Return the largest sum of the given array after partitioning. Test cases are generated so that the answer fits in a 32-bit integer.

Input: `arr = [1,15,7,9,2,5,10]`, `k = 3`

Output: 84

Explanation: `arr` becomes `[15,15,15,9,10,10,10]`

```
int n;
vector<int> arr;
vector<int> memo;
int solve(int i, int k) {
    if (i >= n) return 0;
    int &res = memo[i];
    if (res != -1) return res;
    int mx = 0;
    res = 0;
    for (int len = 1; len <= k && i + len <= n; ++len) {
        mx = max(mx, arr[i + len - 1]);
        res = max(res, mx * len + solve(i + len, k));
    }
    return res;
}
int maxSumAfterPartitioning(vector<int>& A, int K) {
```

```

    arr = A;
    n = arr.size();
    memo.assign(n, -1);
    return solve(0, K);
}

```

[hard]

Given a wooden stick of length  $n$  units. The stick is labelled from  $0$  to  $n$ . For example, a stick of length  $6$  is labelled as follows:

Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at.

You should perform the cuts in order, you can change the order of the cuts as you wish.

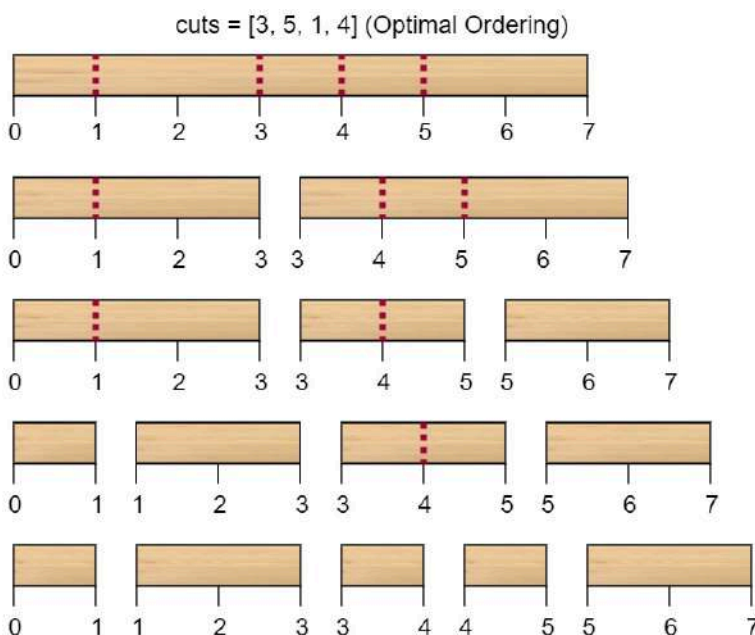
The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Input:  $n = 7$ , `cuts = [1,3,4,5]`

Output: 16

Explanation: Using cuts order = `[1, 3, 4, 5]` as in the input leads to the following scenario:



```
vector<int> cuts2;
```

```

vector<vector<int>> memo;
int solve(int l, int r) {
    if (l + 1 == r) return 0; // no cuts between
    int &res = memo[l][r];
    if (res != -1) return res;
    res = INT_MAX/4;
    for (int k = l + 1; k <= r - 1; ++k) {
        int cost = cuts2[r] - cuts2[l] + solve(l, k) + solve(k, r);
        res = min(res, cost);
    }
    return res;
}
int minCost(int n, vector<int>& cuts) {
    sort(cuts.begin(), cuts.end());
    cuts2.clear();
    cuts2.push_back(0);
    for (int c : cuts) cuts2.push_back(c);
    cuts2.push_back(n);
    int m = cuts2.size();
    memo.assign(m, vector<int>(m, -1));
    return solve(0, m-1);
}

```

[hard]

You are given  $n$  balloons, indexed from 0 to  $n - 1$ . Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the  $i$ th balloon, you will get  $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$  coins. If  $i - 1$  or  $i + 1$  goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it. Return the maximum coins you can collect by bursting the balloons wisely.

Input: `nums = [3,1,5,8]`. Output: 167

Explanation:

`nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []`

`coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167`

`vector<int> a; // padded`

`vector<vector<int>> memo;`

```

int m;
int solve(int l, int r) {
    if (l + 1 == r) return 0; // no balloons between
    int &res = memo[l][r];
    if (res != -1) return res;
    res = 0;
    for (int k = l + 1; k <= r - 1; ++k) {
        int gain = a[l] * a[k] * a[r];
        res = max(res, gain + solve(l, k) + solve(k, r));
    }
    return res;
}
int maxCoins(vector<int>& nums) {
    int n = nums.size();
    a.clear();
    a.push_back(1);
    for (int x : nums) a.push_back(x);
    a.push_back(1);
    m = a.size();
    memo.assign(m, vector<int>(m, -1));
    return solve(0, m-1);
}

```

[hard]

Given a string *s*, partition *s* such that every substring of the partition is a palindrome.

Return the minimum cuts needed for a palindrome partitioning of *s*.

Input: *s* = "aab"

Output: 1

Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.

```
string s;
```

```
int n;
```



```

vector<vector<char>> isPal;
vector<int> memo;
int solve(int i) {
    if (i >= n) return 0; // no cuts needed for empty suffix
    int &res = memo[i];
    if (res != -1) return res;
    int best = INT_MAX/4;
    for (int j = i; j < n; ++j) {
        if (isPal[i][j]) {
            // if we cut after j, we add 1 cut, except when j is at
end => 0 cuts needed
            int cost = (j == n-1) ? 0 : 1 + solve(j+1);
            best = min(best, cost);
        }
    }
    return res = best;
}

int minCut(string str) {
    s = str;
    n = s.size();
    isPal.assign(n, vector<char>(n, 0));
    for (int i = n-1; i >= 0; --i) {
        for (int j = i; j < n; ++j) {
            if (s[i] == s[j] && (j - i < 2 || isPal[i+1][j-1]))
                isPal[i][j] = 1;
        }
    }
    memo.assign(n, -1);
    return solve(0);
}

```

**[hard]**

You are given  $k$  identical eggs and you have access to a building with  $n$  floors labeled from 1 to  $n$ .

You know that there exists a floor  $f$  where  $0 \leq f \leq n$  such that any egg dropped at a floor higher than  $f$  will break, and any egg dropped at or below floor  $f$  will not break.

Each move, you may take an unbroken egg and drop it from any floor  $x$  (where  $1 \leq x \leq n$ ). If the egg breaks, you can no longer use it.

However, if the egg does not break, you may reuse it in future moves. Return the minimum number of moves that you need to determine with certainty what the value of  $f$  is.

Input:  $k = 1$ ,  $n = 2$ . Output: 2. Explanation: Drop the egg from floor 1. If it breaks, we know that  $f = 0$ . Otherwise, drop the egg from floor 2. If it breaks, we know that  $f = 1$ . If it does not break, then we know  $f = 2$ . Hence, we need at minimum 2 moves to determine with certainty what the value of  $f$  is.

Input:  $k = 3$ ,  $n = 14$ . Output: 4

```
int superEggDrop(int k, int n) {
    // Top down may give TLE
    if (n == 0 || n == 1) return n;
    if (k == 1) return n;
    // dp[e] will represent dp[m][e] for current m in the loop
    vector<long long> dp(k + 1, 0);
    int moves = 0;
    while (dp[k] < n) {
        moves++;
        // update dp[e] for new moves value from high e to low e
        for (int e = k; e >= 1; --e) {
            dp[e] = dp[e] + dp[e - 1] + 1;
        }
        // dp[0] stays 0
    }
    return moves;
}
```

# Graphs-BasicQs

[Self added] Basic Questions

- [\[Self added v2\] Clone graph](#)
- [\[Self added v2\] Number of islands](#)

Given a reference of a node in a connected undirected graph.  
Return a deep copy (clone) of the graph.

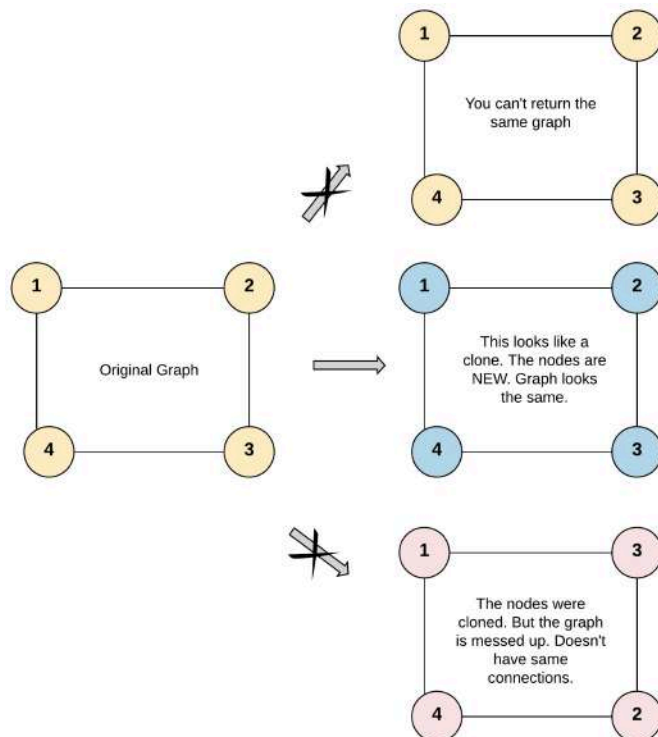
Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

Input: adjList = [[2,4],[1,3],[2,4],[1,3]]. Output:

[[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph. 1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4). 2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3). 3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4). 4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).



```

// DFS
unordered_map<Node*, Node*> mp; // maps original → cloned
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    // If node is already created, return it
    if (mp.count(node)) return mp[node];
    // Create clone node
    Node* clone = new Node(node->val);
    mp[node] = clone;
    // Clone all neighbors
    for (auto neigh : node->neighbors) {
        clone->neighbors.push_back(cloneGraph(neigh));
    }
    return clone;
}

// BFS
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    unordered_map<Node*, Node*> mp;
    queue<Node*> q;
    mp[node] = new Node(node->val);
    q.push(node);
    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();
        for (auto next : curr->neighbors) {
            if (!mp.count(next)) {
                mp[next] = new Node(next->val);
                q.push(next);
            }
            mp[curr]->neighbors.push_back(mp[next]);
        }
    }
    return mp[node];
}

```

Given an  $m \times n$  2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

Output: 1

```
void dfs(vector<vector<char>>& grid, int i, int j) {
    int n = grid.size();
    int m = grid[0].size();
    if (i < 0 || j < 0 || i >= n || j >= m || grid[i][j] == '0')
        return;
    grid[i][j] = '0'; // mark visited
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}

int numIslands(vector<vector<char>>& grid) {
    int n = grid.size();
    int m = grid[0].size();
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j); // flood-fill this island
            }
        }
    }
    return count;
}
```

# Graphs-TopologicalSort

## Graphs - Topological Sort

Description: Topological sorting is useful for tasks that require dependency resolution (InDegree) in directed acyclic graphs (DAGs).

- [Course Schedule](#)
- [Course Schedule II](#)
- [Strange Printer ii](#)
- [Sequence Reconstruction](#)
- [Alien Dictionary](#)
- [\[Self added v2\] Cycle detection in Directed & Undirected graph](#)

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
```



```

// Kahn's Topological Sort
vector<vector<int>> adj(numCourses);
vector<int> indeg(numCourses, 0);
for (auto &p : prerequisites) {
    int a = p[0], b = p[1];
    adj[b].push_back(a);
    indeg[a]++;
}
queue<int> q;
for (int i = 0; i < numCourses; i++)
    if (indeg[i] == 0) q.push(i);
int count = 0;
while (!q.empty()) {
    int u = q.front(); q.pop();
    count++;
    for (int v : adj[u]) {
        indeg[v]--;
        if (indeg[v] == 0) q.push(v);
    }
}
return count == numCourses;
}

```

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

**So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].**

```
vector<int> findOrder(int numCourses, vector<vector<int>>&
prerequisites) {
    vector<vector<int>> adj(numCourses);
    vector<int> indeg(numCourses, 0);
    for (auto &p : prerequisites) {
        adj[p[1]].push_back(p[0]);
        indeg[p[0]]++;
    }
    queue<int> q;
    for (int i = 0; i < numCourses; i++)
        if (indeg[i] == 0) q.push(i);
    vector<int> topo;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    if (topo.size() != numCourses) return {};
    return topo;
}
```

[hard]

There is a strange printer with the following two special requirements:

On each turn, the printer will print a solid rectangular pattern of a single color on the grid. This will cover up the existing colors in the rectangle.

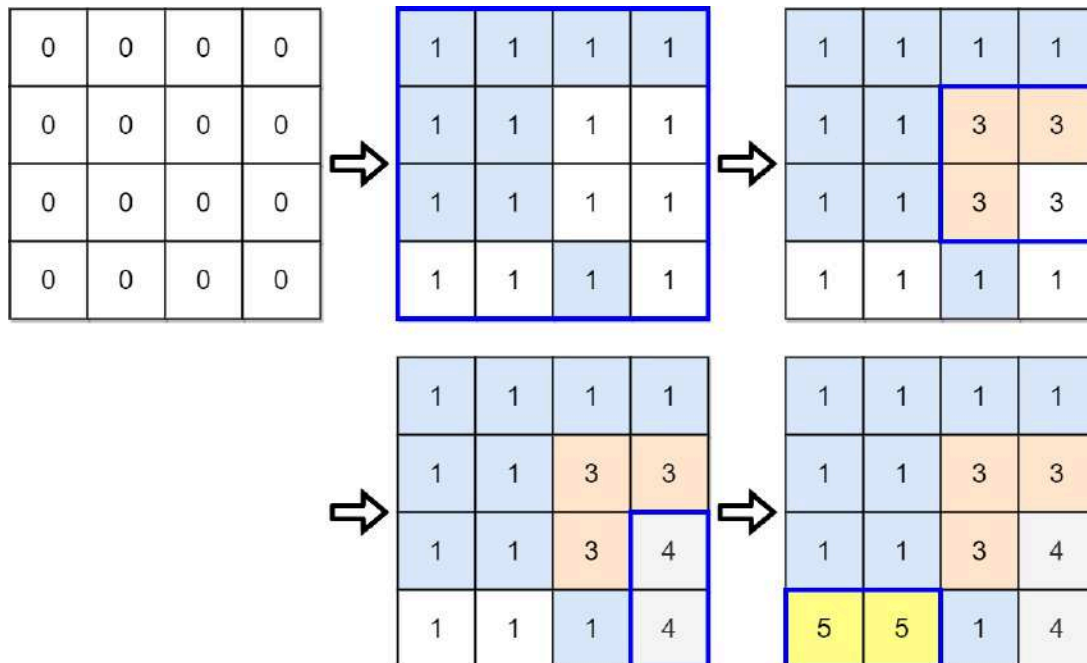
Once the printer has used a color for the above operation, the same color cannot be used again.

You are given a  $m \times n$  matrix `targetGrid`, where `targetGrid[row][col]` is the color in the position (row, col) of the grid.

Return true if it is possible to print the matrix `targetGrid`, otherwise, return false.

Input: `targetGrid = [[1,1,1,1],[1,1,3,3],[1,1,3,4],[5,5,1,4]]`

Output: true



```
bool isPrintable(vector<vector<int>>& g) {  
    int m = g.size(), n = g[0].size();  
    // Find actual max color
```

```

int maxColor = 0;
for (auto &row : g)
    for (int c : row)
        maxColor = max(maxColor, c);
maxColor++; // colors are 1..maxColor-1
vector<int> minr(maxColor, INT_MAX), minc(maxColor, INT_MAX);
vector<int> maxr(maxColor, INT_MIN), maxc(maxColor, INT_MIN);
vector<bool> present(maxColor, false);
// bounding boxes
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        int col = g[r][c];
        present[col] = true;
        minr[col] = min(minr[col], r);
        minc[col] = min(minc[col], c);
        maxr[col] = max(maxr[col], r);
        maxc[col] = max(maxc[col], c);
    }
}
// Build dependency graph
vector<vector<int>> adj(maxColor);
vector<int> indeg(maxColor, 0);
for (int col = 1; col < maxColor; col++) {
    if (!present[col]) continue;

    for (int r = minr[col]; r <= maxr[col]; r++) {
        for (int c = minc[col]; c <= maxc[col]; c++) {
            int inside = g[r][c];
            if (inside != col) {
                adj[inside].push_back(col);
                indeg[col]++;
            }
        }
    }
}
// topo-sort
queue<int> q;
for (int col = 1; col < maxColor; col++)

```

```

        if (present[col] && indeg[col] == 0)
            q.push(col);
    int printed = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        printed++;
        for (int v : adj[u]) {
            if (--indeg[v] == 0)
                q.push(v);
        }
    }
    int total = 0;
    for (int i = 1; i < maxColor; i++)
        if (present[i]) total++;
    return printed == total;
}

```

(premium)

### Sequence reconstruction

You are given an integer array `nums` which is the original sequence, and an array of sequences `seqs` where each `seqs[i]` is a subsequence of `org`. Check whether the `org` sequence can be uniquely reconstructed from the sequences in `seqs`.

Reconstruction means:

The sequence is the shortest common supersequence of all `seqs[i]`.

Return:

true if `org` is the only sequence that can be reconstructed.

false otherwise.

Input:

`org = [1,2,3]`

`seqs = [[1,2],[1,3]]`

Output: false

Explanation: Possible sequences: [1,2,3] or [1,3,2]

Both satisfy `seqs`  $\Rightarrow$  reconstruction not unique.

```

bool sequenceReconstruction(vector<int>& org, vector<vector<int>>&
seqs) {

```

```

    int n = org.size();

```

```

    if (seqs.empty()) return false;

```

```

unordered_map<int, vector<int>> adj;
unordered_map<int, int> indeg; unordered_set<int> seen;
for (auto& s : seqs) {
    for (int x : s) {
        indeg[x] = 0;
        adj[x] = {};
        seen.insert(x);
    }
}
if (seen.size() != n) return false; // missing elements
// build graph from seqs
for (auto& s : seqs) {
    for (int i = 1; i < (int)s.size(); i++) {
        int u = s[i-1], v = s[i];
        adj[u].push_back(v);
        indeg[v]++;
    }
}
queue<int> q;
for (auto& p : indeg)
    if (p.second == 0)
        q.push(p.first);
vector<int> result;
while (!q.empty()) {
    if (q.size() > 1) return false; // not unique
    int u = q.front(); q.pop();
    result.push_back(u);
    for (int v : adj[u]) {
        if (--indeg[v] == 0)
            q.push(v);
    }
}
return result == org;
}

```

### Alien Dictionary - Premium

There is a new alien language and you are given a list of words sorted lexicographically according to this new language.

You need to derive the order of letters in this alien language.

Return:

A string representing the unique character order

"" if no valid order exists or if multiple orders are possible

Rules:

Words are sorted according to the alien alphabet.

If w1 is a prefix of w2, then w2 must not come before w1.

Input:

["wrt","wrf","er","ett","rftt"]

Output:

"wertf"

```
string alienOrder(vector<string>& words) {
    unordered_map<char, vector<char>>> adj;
    unordered_map<char, int> indeg;
    // Register all characters
    for (auto& w : words)
        for (char c : w)
            indeg[c] = 0;
    // Build graph by comparing adjacent words
    for (int i = 0; i + 1 < words.size(); i++) {
        string& w1 = words[i];
        string& w2 = words[i+1];
        int len = min(w1.size(), w2.size());
```

```

    bool found = false;
    for (int j = 0; j < len; j++) {
        if (w1[j] != w2[j]) {
            adj[w1[j]].push_back(w2[j]);
            indeg[w2[j]]++;
            found = true;
            break;
        }
    }
    // prefix rule: invalid
    if (!found && w1.size() > w2.size())
        return "";
}
queue<char> q;
for (auto& p : indeg)
    if (p.second == 0)
        q.push(p.first);
string order;
while (!q.empty()) {
    if (q.size() > 1) {
        // Multiple valid orders → return any? No → return all
        // deterministic?
        // LeetCode expects ANY valid deterministic topo.
    }
    char c = q.front(); q.pop();
    order.push_back(c);
    for (char nxt : adj[c]) {
        if (--indeg[nxt] == 0)
            q.push(nxt);
    }
}
if (order.size() != indeg.size()) return ""; // cycle found
return order;
}

```

### Cycle detection in directed/undirected graph

// Undirected - DFS

```
bool dfsCycle(int node, int parent, vector<vector<int>>& adj,
```



```

        vector<int>& vis) {
vis[node] = 1;
for (int nxt : adj[node]) {
    if (!vis[nxt]) {
        bool hasCycle = dfsCycle(nxt, node, adj, vis);
        if (hasCycle) {
            return true;
        }
    }
    else if (nxt != parent) {
        return true; // visited neighbor that's not parent
    }
}
return false;
}

bool isCycle(int V, vector<vector<int>>& adj) {
    vector<int> vis(V, 0);
    for (int i = 0; i < V; i++) {
        if (!vis[i]) {
            if (dfsCycle(i, -1, adj, vis))
                return true;
        }
    }
    return false;
}

// Undirected - BFS
bool bfsCycle(int start, vector<int> adj[], vector<int> &vis) {
    // Use - vector<vector<int>>& adj preferably
    queue<pair<int,int>> q;
    q.push({start, -1});
    vis[start] = 1;
    while (!q.empty()) {
        int node = q.front().first;
        int parent = q.front().second;
        q.pop();
        for (int nxt : adj[node]) {
            if (!vis[nxt]) {
                vis[nxt] = 1;

```

```

        q.push({nxt, node});
    }
    else if (nxt != parent) {
        return true;
    }
}
}
return false;
}
bool isCycle(int V, vector<int> adj[]) {
    vector<int> vis(V, 0);
    for (int i = 0; i < V; i++) {
        if (!vis[i]) {
            if (bfsCycle(i, adj, vis))
                return true;
        }
    }
    return false;
}
// Directed - DFS
bool dfsDirected(int node, vector<vector<int>>& adj, vector<int>& vis, vector<int>& dfsVis) {
    vis[node] = 1;
    dfsVis[node] = 1;
    for (int nxt : adj[node]) {
        if (!vis[nxt]) {
            bool hasCycle = dfsDirected(nxt, adj, vis, dfsVis);
            if (hasCycle) {
                return true;
            }
        }
        else if (dfsVis[nxt]) {
            return true; // back edge
        }
    }
    dfsVis[node] = 0; // backtrack
    return false;
}

```

```

bool isCyclic(int V, vector<vector<int>>& adj) {
    vector<int> vis(V, 0), dfsVis(V, 0);
    for (int i = 0; i < V; i++) {
        if (!vis[i]) {
            if (dfsDirected(i, adj, vis, dfsVis))
                return true;
        }
    }
    return false;
}

// Directed - BFS + Indegree (Kahn's Algorithm)
// A Directed Acyclic Graph (DAG) always has a valid topological
// ordering. If graph has a cycle, Kahn's algo won't be able to remove
// all nodes. If no. of removed nodes (count) < V → cycle exists.
bool isCyclicKahn(int V, vector<int> adj[]) {
    // Use - vector<vector<int>>& adj preferably
    vector<int> indegree(V, 0);
    for (int i = 0; i < V; i++) {
        for (int nxt : adj[i]) {
            indegree[nxt]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < V; i++) {
        if (indegree[i] == 0) q.push(i);
    }
    int count = 0; // number of nodes processed
    while (!q.empty()) {
        int node = q.front(); q.pop();
        count++;
        for (int nxt : adj[node]) {
            indegree[nxt]--;
            if (indegree[nxt] == 0)
                q.push(nxt);
        }
    }
    // If topological sort cannot include all nodes → cycle exists
    return (count != V);
}

```

}

# Graphs-UnionFind

## Graphs - Union Find (Disjoint Set)

Description: Union-Find (or Disjoint Set) is used to solve problems involving connectivity or grouping, often in graphs.

- [Number of Operations to Make Network Connected](#)
- [Redundant Connection](#)
- [Accounts Merge](#)
- [Satisfiability of Equality Equations](#)

There are  $n$  computers numbered from  $0$  to  $n - 1$  connected by ethernet cables connections forming a network where  $\text{connections}[i] = [a_i, b_i]$  represents a connection between computers  $a_i$  and  $b_i$ . Any computer can reach any other computer directly or indirectly through the network.

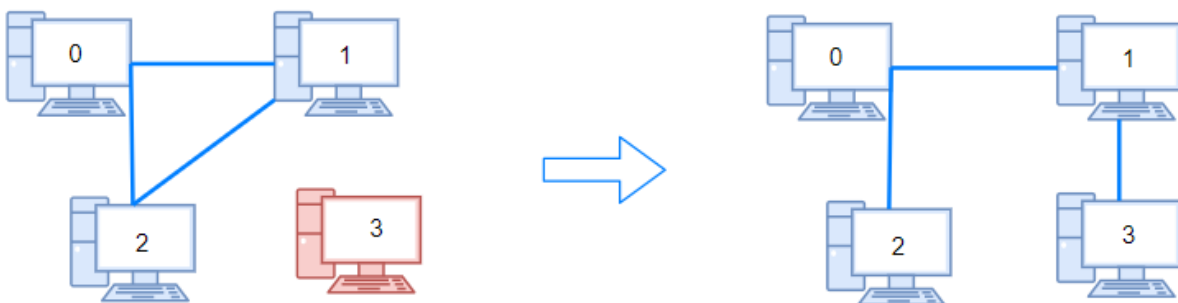
You are given an initial computer network connections. You can extract certain cables between two directly connected computers, and place them between any pair of disconnected computers to make them directly connected.

Return the minimum number of times you need to do this in order to make all the computers connected. If it is not possible, return  $-1$ .

Input:  $n = 4$ ,  $\text{connections} = [[0,1],[0,2],[1,2]]$

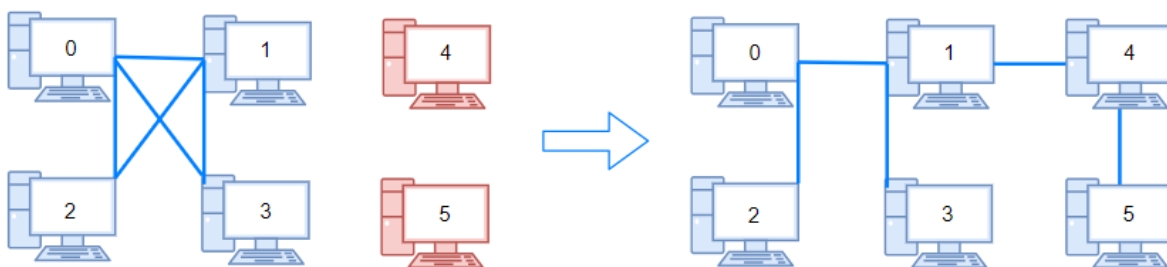
Output: 1

Explanation: Remove cable between computer 1 and 2 and place between computers 1 and 3.



Input:  $n = 6$ ,  $\text{connections} = [[0,1],[0,2],[0,3],[1,2],[1,3]]$

Output: 2



```

struct DSU {
    vector<int> parent, rankv;
    DSU(int n) {
        parent.resize(n);
        rankv.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) return false;
        if (rankv[pa] < rankv[pb]) swap(pa, pb);
        parent[pb] = pa;
        if (rankv[pa] == rankv[pb]) rankv[pa]++;
        return true;
    }
};

int makeConnected(int n, vector<vector<int>>& connections) {
    if (connections.size() < n - 1)
        return -1;
    DSU dsu(n);
    int components = n;
    for (auto &e : connections)
        if (dsu.unite(e[0], e[1]))
            components--;
    return components - 1;
}

```

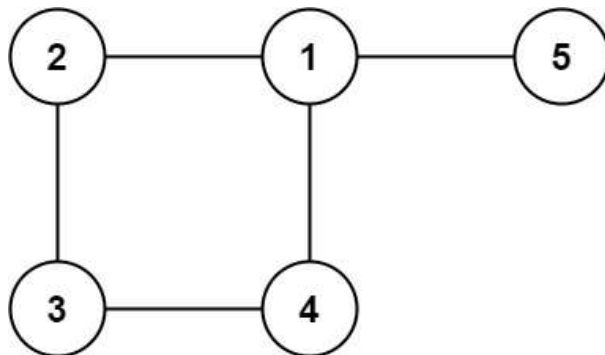
In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with  $n$  nodes labeled from 1 to  $n$ , with one additional edge added. The added edge has two different vertices chosen from 1 to  $n$ , and was not an edge that already existed. The graph is represented as an array `edges` of length  $n$  where `edges[i] = [ai, bi]` indicates that there is an edge between nodes  $ai$  and  $bi$  in the graph.

Return an edge that can be removed so that the resulting graph is a tree of  $n$  nodes. If there are multiple answers, return the answer that occurs last in the input.

Input: `edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]`

Output: `[1,4]`



```
struct DSU {
vector<int> parent, rankv;
DSU(int n) {
    parent.resize(n);
    rankv.resize(n, 0);
    for (int i = 0; i < n; i++)
        parent[i] = i;
}
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]); // path compression
    return parent[x];
}
```



```

bool unite(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) return false; // already connected → cycle
    if (rankv[pa] < rankv[pb]) swap(pa, pb);
    parent[pb] = pa;
    if (rankv[pa] == rankv[pb]) rankv[pa]++;
    return true;
}
};

vector<int> findRedundantConnection(vector<vector<int>>& edges) {
    int n = edges.size();
    DSU dsu(n + 1);
    for (auto &e : edges) {
        int u = e[0], v = e[1];
        if (!dsu.unite(u, v))
            return e; // this edge forms the cycle
    }
    return {};
}

```

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account. Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Input: `accounts =`

`[["John", "johnsmith@mail.com", "john_newyork@mail.com"], ["John", "john`

```
smith@mail.com", "john00@mail.com"], ["Mary", "mary@mail.com"], ["John",  
"johnnybravo@mail.com"]]
```

Output:

```
[["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.c  
om"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]
```

Explanation:

The first and second John's are the same person as they have the common email "johnsmith@mail.com".

The third John and Mary are different people as none of their email addresses are used by other accounts.

We could return these lists in any order, for example the answer

```
[['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'],  
['John', 'john00@mail.com', 'john_newyork@mail.com',  
'johnsmith@mail.com']] would still be accepted.
```

```
struct DSU {  
    vector<int> parent, rankv;  
    DSU(int n) {  
        parent.resize(n);  
        rankv.resize(n, 0);  
        for (int i = 0; i < n; i++)  
            parent[i] = i;  
    }  
    int find(int x) {  
        if (parent[x] != x)  
            parent[x] = find(parent[x]); // path compression  
        return parent[x];  
    }  
    bool unite(int a, int b) {  
        int pa = find(a), pb = find(b);  
        if (pa == pb) return false; // already connected → cycle  
        if (rankv[pa] < rankv[pb]) swap(pa, pb);  
        parent[pb] = pa;  
        if (rankv[pa] == rankv[pb]) rankv[pa]++;  
        return true;  
    }  
};  
vector<vector<string>> accountsMerge(vector<vector<string>>&  
accounts) {
```

```

unordered_map<string, int> emailID;
int id = 0;
for (auto &acc : accounts)
    for (int i = 1; i < acc.size(); i++)
        if (!emailID.count(acc[i]))
            emailID[acc[i]] = id++; // assign id to each email
DSU dsu(id);
// union all emails in each account
for (auto &acc : accounts) {
    string rootEmail = acc[1];
    int rootID = emailID[rootEmail];
    for (int i = 2; i < acc.size(); i++)
        dsu.unite(rootID, emailID[acc[i]]);
}
// group emails by DSU parent
unordered_map<int, vector<string>> groups;
for (auto &p : emailID) {
    int parent = dsu.find(p.second);
    groups[parent].push_back(p.first);
}
vector<vector<string>> ans;
for (auto &acc : accounts)
    acc[0]; // name used below
unordered_map<string, string> emailToName;
for (auto &acc : accounts)
    for (int i = 1; i < acc.size(); i++)
        emailToName[acc[i]] = acc[0];
for (auto &g : groups) {
    vector<string> emails = g.second;
    sort(emails.begin(), emails.end());
    string name = emailToName[emails[0]];
    emails.insert(emails.begin(), name);
    ans.push_back(emails);
}
return ans;
}

```

You are given an array of strings equations that represent relationships between variables where each string equations[i] is of

length 4 and takes one of two different forms: "xi==yi" or "xi!=yi". Here, xi and yi are lowercase letters (not necessarily different) that represent one-letter variable names.

Return true if it is possible to assign integers to variable names so as to satisfy all the given equations, or false otherwise.

Input: equations = ["a==b","b!=a"]

Output: false

Explanation: If we assign say, a = 1 and b = 1, then the first equation is satisfied, but not the second.

There is no way to assign the variables to satisfy both equations.

Input: equations = ["b==a","a==b"]

Output: true

Explanation: We could assign a = 1 and b = 1 to satisfy both equations.

```
struct DSU {
    vector<int> parent, rankv;
    DSU(int n) {
        parent.resize(n);
        rankv.resize(n, 0);
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    bool unite(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) return false; // already connected → cycle
        if (rankv[pa] < rankv[pb]) swap(pa, pb);
        parent[pb] = pa;
        if (rankv[pa] == rankv[pb]) rankv[pa]++;
        return true;
    }
};

bool equationsPossible(vector<string>& equations) {
    DSU dsu(26);
```

```
// 1. union all equalities
for (auto &eq : equations)
    if (eq[1] == '=')
        dsu.unite(eq[0] - 'a', eq[3] - 'a');
// 2. check inequalities
for (auto &eq : equations)
    if (eq[1] == '!=')
        if (dsu.find(eq[0] - 'a') == dsu.find(eq[3] - 'a'))
            return false;
return true;
}
```

# Graphs-Algos

## Graphs - Graph Algorithms

Description: Advanced graph algorithms are used to solve complex problems involving shortest paths, minimum spanning trees, and graph cycles.

- [Kruskal's Algorithm: Minimum Cost to connect all Points](#)
- [Dijkstra's Algorithm: Cheapest Flights Within K Stops](#)
- [Floyd-Warshall: Find the City with Smallest Number of Neighbours at a Threshold Distance](#)
- [Bellman Ford: Network Delay time](#)

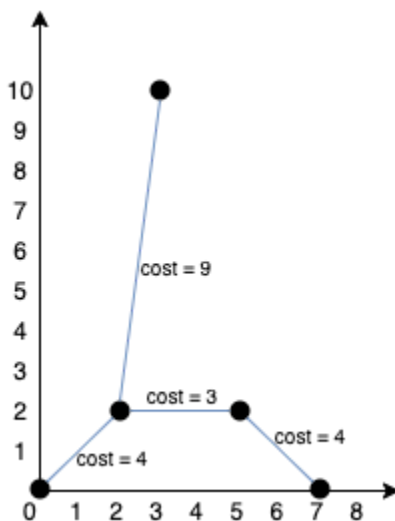
You are given an array `points` representing integer coordinates of some points on a 2D-plane, where `points[i] = [xi, yi]`.

The cost of connecting two points `[xi, yi]` and `[xj, yj]` is the manhattan distance between them:  $|xi - xj| + |yi - yj|$ , where  $|val|$  denotes the absolute value of  $val$ .

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Input: `points = [[0,0],[2,2],[3,10],[5,2],[7,0]]`

Output: 20



```

struct DSU {
    vector<int> parent, rankv;
    DSU(int n) {
        parent.resize(n);
        rankv.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) return false;
        if (rankv[pa] < rankv[pb]) swap(pa, pb);
        parent[pb] = pa;
        if (rankv[pa] == rankv[pb]) rankv[pa]++;
        return true;
    }
};

int minCostConnectPoints(vector<vector<int>>& points) {
    int n = points.size();
    vector<vector<int>> edges;
    // Build all edges: O(n^2)
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int dist = abs(points[i][0] - points[j][0]) +
                      abs(points[i][1] - points[j][1]);
            edges.push_back({dist, i, j});
        }
    }
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    int cost = 0, used = 0;

```



```

    for (auto &e : edges) {
        int w = e[0], a = e[1], b = e[2];
        if (dsu.unite(a, b)) {
            cost += w;
            used++;
            if (used == n - 1) break;
        }
    }
    return cost;
}

```

There are  $n$  cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Input: `n = 4`, `flights =`

`[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

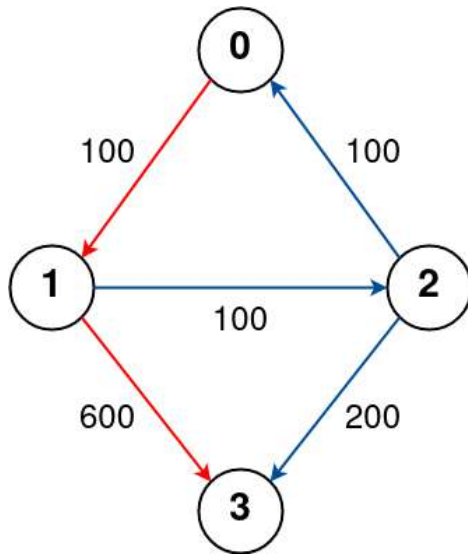
Output: `700`

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost  $100 + 600 = 700$ .

Note that the path through cities `[0,1,2,3]` is cheaper but is invalid because it uses 2 stops.



```

int findCheapestPrice(int n, vector<vector<int>>& flights,
                    int src, int dst, int K) {
    vector<vector<pair<int,int>>> adj(n);
    for (auto &f : flights)
        adj[f[0]].push_back({f[1], f[2]});
    vector<vector<int>> dist(n, vector<int>(K+2, INT_MAX));
    dist[src][0] = 0;
    queue<array<int,3>> q;
    q.push({src, 0, 0}); // node, cost, stops
    int ans = INT_MAX;
    while (!q.empty()) {
        auto cur = q.front(); q.pop();
        int u = cur[0], cost = cur[1], stops = cur[2];
        if (u == dst) ans = min(ans, cost);
        if (stops > K) continue;
        for (auto &nx : adj[u]) {
            int v = nx.first, w = nx.second;
            if (cost + w < dist[v][stops+1]) {
                dist[v][stops+1] = cost + w;
                q.push({v, cost + w, stops+1});
            }
        }
    }
    return ans == INT_MAX ? -1 : ans;
}

```

There are  $n$  cities numbered from 0 to  $n-1$ . Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities  $i$  and  $j$  is equal to the sum of the edges' weights along that path.

Input:  $n = 4$ , `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`,  
`distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

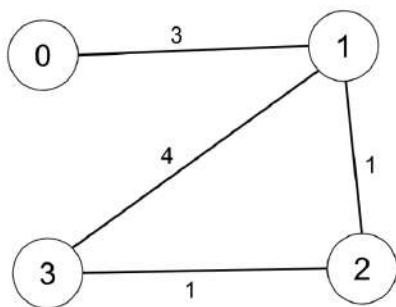
City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.



```
int findTheCity(int n, vector<vector<int>>& edges, int threshold) {
```

```

const int INF = 1e9;
vector<vector<int>> dist(n, vector<int>(n, INF));
for (int i = 0; i < n; i++) dist[i][i] = 0;
for (auto &e : edges) {
    int u = e[0], v = e[1], w = e[2];
    dist[u][v] = dist[v][u] = min(dist[u][v], w);
}
// Floyd-Warshall
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
int bestCity = -1, bestCnt = 1e9;
for (int i = 0; i < n; i++) {
    int cnt = 0;
    for (int j = 0; j < n; j++)
        if (dist[i][j] <= threshold) cnt++;
    if (cnt <= bestCnt) {
        bestCnt = cnt;
        bestCity = i; // choose largest i on tie
    }
}
return bestCity;
}

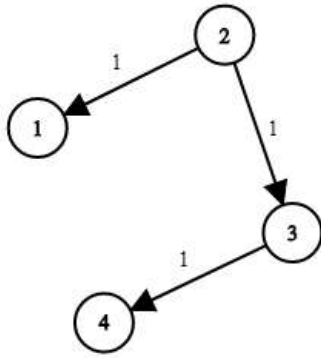
```

You are given a network of  $n$  nodes, labeled from 1 to  $n$ . You are also given times, a list of travel times as directed edges  $\text{times}[i] = (u_i, v_i, w_i)$ , where  $u_i$  is the source node,  $v_i$  is the target node, and  $w_i$  is the time it takes for a signal to travel from source to target.

We will send a signal from a given node  $k$ . Return the minimum time it takes for all the  $n$  nodes to receive the signal. If it is impossible for all the  $n$  nodes to receive the signal, return -1.

Input:  $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$ ,  $n = 4$ ,  $k = 2$

Output: 2



```

int networkDelayTime(vector<vector<int>>& times, int n, int k) {
    const int INF = 1e9;
    vector<int> dist(n + 1, INF);
    dist[k] = 0;
    // Relax edges n-1 times
    for (int i = 1; i < n; i++) {
        bool updated = false;
        for (auto &e : times) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                updated = true;
            }
        }
        if (!updated) break;
    }
    int ans = 0;
    for (int i = 1; i <= n; i++)
        ans = max(ans, dist[i]);
    return ans == INF ? -1 : ans;
}

```

GenDS-Greedy

## General DS - Greedy

Description: Greedy algorithms make local optimal choices at each step, which lead to a global optimal solution for problems like scheduling and resource allocation.

- [\[Can add\] Jump Game](#)
- [Jump Game ii](#)
- [Gas Station](#)
- [Bag of Tokens](#)
- [Boats to Save People](#)
- [Wiggle Subsequence](#)
- [Car Pooling](#)
- [Candy](#)

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

```
bool canJump(vector<int>& nums) {
    int n = nums.size();
    int maxReach = 0;
    for (int i = 0; i < n; i++) {
        if (i > maxReach) return false; // can't reach this
position
        maxReach = max(maxReach, i + nums[i]);
        if (maxReach >= n - 1) return true; // can reach the end
    }
    return true;
}
```

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

$0 \leq j \leq \text{nums}[i]$  and  $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

```
int jump(vector<int>& nums) {
    int jumps = 0, curEnd = 0, curFarthest = 0;
    int n = nums.size();
    for (int i = 0; i < n - 1; i++) { // no need to jump from last
index
        curFarthest = max(curFarthest, i + nums[i]);

        if (i == curEnd) { // end of current jump range
            jumps++;
            curEnd = curFarthest;
        }
    }
    return jumps;
}
```

There are `n` gas stations along a circular route, where the amount of gas at the `i`th station is `gas[i]`.

You have a car with an unlimited gas tank and it costs `cost[i]` of gas to travel from the `i`th station to its next `(i + 1)`th station.

You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays `gas` and `cost`, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1. If there exists a solution, it is guaranteed to be unique.

Input: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]`



Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank =  $0 + 4 = 4$

Travel to station 4. Your tank =  $4 - 1 + 5 = 8$

Travel to station 0. Your tank =  $8 - 2 + 1 = 7$

Travel to station 1. Your tank =  $7 - 3 + 2 = 6$

Travel to station 2. Your tank =  $6 - 4 + 3 = 5$

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int totalTank = 0, currTank = 0, start = 0;
    int n = gas.size();
    for (int i = 0; i < n; i++) {
        int diff = gas[i] - cost[i];
        totalTank += diff;
        currTank += diff;
        if (currTank < 0) {
            start = i + 1;
            currTank = 0;
        }
    }
    return (totalTank >= 0) ? start : -1;
}
```

You start with an initial power of power, an initial score of 0, and a bag of tokens given as an integer array tokens, where each tokens[i] denotes the value of tokeni. Your goal is to maximize the total score by strategically playing these tokens. In one move, you can play an unplayed token in one of the two ways (but not both for the same token): Face-up: If your current power is at least tokens[i], you may play tokeni, losing tokens[i] power and gaining 1 score; Face-down: If your current score is at least 1, you may play tokeni, gaining tokens[i] power and losing 1 score. Return the maximum possible score you can achieve after playing any number of tokens.

Input: tokens = [100], power = 50

**Output: 0**

**Explanation:** Since your score is 0 initially, you cannot play the token face-down. You also cannot play it face-up since your power (50) is less than tokens[0] (100).

```
int bagOfTokensScore(vector<int>& tokens, int power) {
    sort(tokens.begin(), tokens.end());
    int i = 0, j = tokens.size() - 1;
    int score = 0, maxScore = 0;
    while (i <= j) {
        if (power >= tokens[i]) {
            power -= tokens[i++];
            score++;
            maxScore = max(maxScore, score);
        } else if (score > 0) {
            power += tokens[j--];
            score--;
        } else {
            break;
        }
    }
    return maxScore;
}
```

You are given an array people where people[i] is the weight of the ith person, and an infinite number of boats where each boat can carry a maximum weight of limit. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most limit.

Return the minimum number of boats to carry every given person.

**Input:** people = [3,5,3,4], limit = 5

**Output:** 4

**Explanation:** 4 boats (3), (3), (4), (5)

```
int numRescueBoats(vector<int>& people, int limit) {
```

```

    sort(people.begin(), people.end());
    int i = 0, j = people.size() - 1, boats = 0;
    while (i <= j) {
        if (people[i] + people[j] <= limit) i++;
        j--;
        boats++;
    }
    return boats;
}

```

A wiggle sequence is a sequence where the differences between successive numbers strictly alternate between positive and negative. The first difference (if one exists) may be either positive or negative. A sequence with one element and a sequence with two non-equal elements are trivially wiggle sequences.

For example, [1, 7, 4, 9, 2, 5] is a wiggle sequence because the differences (6, -3, 5, -7, 3) alternate between positive and negative.

In contrast, [1, 4, 7, 2, 5] and [1, 7, 4, 5, 5] are not wiggle sequences. The first is not because its first two differences are positive, and the second is not because its last difference is zero. A subsequence is obtained by deleting some elements (possibly zero) from the original sequence, leaving the remaining elements in their original order.

Given an integer array nums, return the length of the longest wiggle subsequence of nums.

Input: nums = [1,7,4,9,2,5]

Output: 6

Explanation: The entire sequence is a wiggle sequence with differences (6, -3, 5, -7, 3).

```

int wiggleMaxLength(vector<int>& nums) {
    if (nums.size() < 2) return nums.size();

```

```

    int up = 1, down = 1;
    for (int i = 1; i < nums.size(); i++) {
        if (nums[i] > nums[i-1]) up = down + 1;
        else if (nums[i] < nums[i-1]) down = up + 1;
    }
    return max(up, down);
}

```

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer capacity and an array trips where trips[i] = [numPassengersi, fromi, toi] indicates that the ith trip has numPassengersi passengers and the locations to pick them up and drop them off are fromi and toi respectively. The locations are given as the number of kilometers due east from the car's initial location. Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Input: trips = [[2,1,5],[3,3,7]], capacity = 5

Output: true

```

bool carPooling(vector<vector<int>>& trips, int capacity) {
    vector<int> changes(1001, 0); // constraints: locations <= 1000
    for (auto &t : trips) {
        changes[t[1]] += t[0]; // pickup
        changes[t[2]] -= t[0]; // drop
    }
    int curr = 0;
    for (int p : changes) {
        curr += p;
        if (curr > capacity) return false;
    }
    return true;
}

```

[hard]

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

Each child must have at least one candy.

Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Input: ratings = [1,0,2]

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

```
int candy(vector<int>& ratings) {
    int n = ratings.size();
    vector<int> candies(n,1);
    for(int i=1;i<n;i++){
        if(ratings[i]>ratings[i-1]) candies[i] = candies[i-1]+1;
    }
    for(int i=n-2;i>=0;i--){
        if(ratings[i]>ratings[i+1]) candies[i] = max(candies[i],
candies[i+1]+1);
    }
    return accumulate(candies.begin(), candies.end(), 0);
}
```

# DesignDataStructure

## Design Data Structure

**Description:** It involves building custom data structures to efficiently handle specific operations, like managing data access, updates, and memory usage. Focusing on optimizing performance and resource management.

- [Design Twitter](#)
- [Design Browser History](#)
- [Design Circular Deque](#)
- [Snapshot Array](#)
- [LRU Cache](#)
- [LFU Cache](#)

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

Twitter() Initializes your twitter object.

void postTweet(int userId, int tweetId) Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.

List<Integer> getNewsFeed(int userId) Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themselves. Tweets must be ordered from most recent to least recent.

void follow(int followerId, int followeeId) The user with ID followerId started following the user with ID followeeId.

void unfollow(int followerId, int followeeId) The user with ID followerId started unfollowing the user with ID followeeId.

Input

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet",  
"getNewsFeed", "unfollow", "getNewsFeed"]
```

```
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

Output

```
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();
```

```

twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 1 tweet id -> [5]. return [5]
twitter.follow(1, 2);    // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 2 tweet ids -> [6, 5]. Tweet id 6 should precede tweet id 5
because it is posted after tweet id 5.
twitter.unfollow(1, 2); // User 1 unfollows user 2.
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 1 tweet id -> [5], since user 1 is no longer following user 2.
class Twitter {
private:
    int timestamp;
    unordered_map<int, unordered_set<int>> follows;
    unordered_map<int, vector<pair<int,int>>> tweets; // user ->
{(time, tweetId)}
public:
    Twitter() {
        timestamp = 0;
    }
    void postTweet(int userId, int tweetId) {
        tweets[userId].push_back({timestamp++, tweetId});
    }
    vector<int> getNewsFeed(int userId) {
        vector<pair<int,int>> feed;
        if(tweets.count(userId)) feed.insert(feed.end(),
tweets[userId].begin(), tweets[userId].end());
        for(auto f : follows[userId]) {
            if(tweets.count(f))
                feed.insert(feed.end(), tweets[f].begin(),
tweets[f].end());
        }
        sort(feed.rbegin(), feed.rend()); // most recent first
        vector<int> ans;
        for(int i=0;i<min(10,(int)feed.size());i++)
            ans.push_back(feed[i].second);
        return ans;
    }
};

```



```

    }
    void follow(int followerId, int followeeId) {
        if(followerId != followeeId)
            follows[followerId].insert(followeeId);
    }
    void unfollow(int followerId, int followeeId) {
        follows[followerId].erase(followeeId);
    }
};

```

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

BrowserHistory(string homepage) Initializes the object with the homepage of the browser.

void visit(string url) Visits url from the current page. It clears up all the forward history.

string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.

string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Input:

```

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

```

Output:

```

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

```

Explanation:

```

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

```

```

browserHistory.visit("google.com");           // You are in
"leetcode.com". Visit "google.com"
browserHistory.visit("facebook.com");         // You are in
"google.com". Visit "facebook.com"
browserHistory.visit("youtube.com");          // You are in
"facebook.com". Visit "youtube.com"
browserHistory.back(1);                       // You are in
"youtube.com", move back to "facebook.com" return "facebook.com"
browserHistory.back(1);                       // You are in
"facebook.com", move back to "google.com" return "google.com"
browserHistory.forward(1);                    // You are in
"google.com", move forward to "facebook.com" return "facebook.com"
browserHistory.visit("linkedin.com");          // You are in
"facebook.com". Visit "linkedin.com"
browserHistory.forward(2);                    // You are in
"linkedin.com", you cannot move forward any steps.
browserHistory.back(2);                       // You are in
"linkedin.com", move back two steps to "facebook.com" then to
"google.com". return "google.com"
browserHistory.back(7);                       // You are in
"google.com", you can move back only one step to "leetcode.com".
return "leetcode.com"
class BrowserHistory {
    vector<string> history;
    int curr;
public:
    BrowserHistory(string homepage) {
        history.push_back(homepage);
        curr = 0;
    }
    void visit(string url) {
        history.resize(curr+1); // remove forward
        history.push_back(url);
        curr++;
    }
    string back(int steps) {
        curr = max(0, curr-steps);
        return history[curr];
    }

```

```

    }
    string forward(int steps) {
        curr = min((int)history.size()-1, curr+steps);
        return history[curr];
    }
};

```

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

MyCircularDeque(int k) Initializes the deque with a maximum size of k.

boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.

boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.

boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.

boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.

int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.

int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.

boolean isEmpty() Returns true if the deque is empty, or false otherwise.

boolean isFull() Returns true if the deque is full, or false otherwise.

Input

```

["MyCircularDeque", "insertLast", "insertLast", "insertFront",
"insertFront", "getRear", "isFull", "deleteLast", "insertFront",
"getFront"]

```

```

[[3], [1], [2], [3], [4], [], [], [], [4], []]

```

Output

```

[null, true, true, true, false, 2, true, true, true, 4]

```

### Explanation

```
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
myCircularDeque.insertLast(1); // return True
myCircularDeque.insertLast(2); // return True
myCircularDeque.insertFront(3); // return True
myCircularDeque.insertFront(4); // return False, the queue is full.
myCircularDeque.getRear();      // return 2
myCircularDeque.isFull();       // return True
myCircularDeque.deleteLast();   // return True
myCircularDeque.insertFront(4); // return True
myCircularDeque.getFront();     // return 4
class MyCircularDeque {
    vector<int> dq;
    int frontIndex, rearIndex, capacity, size;
public:
    MyCircularDeque(int k) {
        capacity = k;
        dq.resize(k);
        frontIndex = 0;
        rearIndex = 0;
        size = 0;
    }
    bool insertFront(int value) {
        if(isFull()) return false;
        frontIndex = (frontIndex - 1 + capacity) % capacity;
        dq[frontIndex] = value;
        size++;
        return true;
    }
    bool insertLast(int value) {
        if(isFull()) return false;
        dq[rearIndex] = value;
        rearIndex = (rearIndex + 1) % capacity;
        size++;
        return true;
    }
    bool deleteFront() {
        if(isEmpty()) return false;
```

```

        frontIndex = (frontIndex + 1) % capacity;
        size--;
        return true;
    }
    bool deleteLast() {
        if(isEmpty()) return false;
        rearIndex = (rearIndex - 1 + capacity) % capacity;
        size--;
        return true;
    }
    int getFront() {
        if(isEmpty()) return -1;
        return dq[frontIndex];
    }
    int getRear() {
        if(isEmpty()) return -1;
        return dq[(rearIndex - 1 + capacity) % capacity];
    }
    bool isEmpty() { return size == 0; }
    bool isFull() { return size == capacity; }
};

```

Implement a SnapshotArray that supports the following interface:  
**SnapshotArray(int length)** initializes an array-like data structure with the given length. Initially, each element equals 0.  
**void set(index, val)** sets the element at the given index to be equal to val.

**int snap()** takes a snapshot of the array and returns the snap\_id: the total number of times we called snap() minus 1.

**int get(index, snap\_id)** returns the value at the given index, at the time we took the snapshot with the given snap\_id

Input: ["SnapshotArray","set","snap","set","get"]

[[3],[0,5],[],[0,6],[0,0]]

Output: [null,null,0,null,5]

Explanation:

```
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
```

```
snapshotArr.set(0,5); // Set array[0] = 5
```

```
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
```

```
snapshotArr.set(0,6);
```

```
snapshotArr.get(0,0); // Get the value of array[0] with snap_id = 0, return 5
```

```
class SnapshotArray {
    int snap_id;
    vector<vector<pair<int,int>>> arr; // index -> (snap_id, value)
public:
    SnapshotArray(int length) {
        snap_id = 0;
        arr.resize(length);
        for(int i=0;i<length;i++)
            arr[i].push_back({-1,0});
    }
    void set(int index, int val) {
        arr[index].push_back({snap_id, val});
    }
    int snap() {
        return snap_id++;
    }
    int get(int index, int id) {
        auto &v = arr[index];
        int l=0,r=v.size()-1, ans=0;
        while(l<=r){
            int m = (l+r)/2;
            if(v[m].first <= id){
                ans = v[m].second;
                l = m+1;
            } else r = m-1;
        }
        return ans;
    }
};
```

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

LRUCache(int capacity) Initialize the LRU cache with positive size capacity.

int get(int key) Return the value of the key if the key exists, otherwise return -1.

void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions get and put must each run in  $O(1)$  average time complexity.

Input

["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]

[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

Output

[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

```
lRUCache.put(1, 1); // cache is {1=1}
```

```
lRUCache.put(2, 2); // cache is {1=1, 2=2}
```

```
lRUCache.get(1);    // return 1
```

```
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
```

```
lRUCache.get(2);    // returns -1 (not found)
```

```
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
```

```
lRUCache.get(1);    // return -1 (not found)
```

```
lRUCache.get(3);    // return 3
```

```
lRUCache.get(4);    // return 4
```

```

class LRUCache {
    int cap;
    list<pair<int,int>> dq;
    unordered_map<int,list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int capacity) { cap = capacity; }
    int get(int key) {
        if(mp.find(key) == mp.end()) return -1;
        dq.splice(dq.begin(), dq, mp[key]); // move to front
        return mp[key]->second;
    }
    void put(int key, int value) {
        if(mp.find(key) != mp.end()){ // logic same as get()
            dq.splice(dq.begin(), dq, mp[key]);
            mp[key]->second = value;
        } else {
            if(dq.size() == cap){
                mp.erase(dq.back().first);
                dq.pop_back();
            }
            dq.push_front({key,value});
            mp[key] = dq.begin();
        }
    }
};

```

**[hard]**

Design and implement a data structure for a Least Frequently Used (LFU) cache. Implement the LFUCache class:

**LFUCache(int capacity)** Initializes the object with the capacity of the data structure. **int get(int key)** Gets the value of the key if the key exists in the cache. Otherwise, returns -1.

**void put(int key, int value)** Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.



To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it. The functions get and put must each run in  $O(1)$  average time complexity.

Input: ["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]

[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]

Output: [null, null, null, 1, null, -1, 3, null, -1, 3, 4]

Explanation: // cnt(x) = the use counter for key x. // cache=[] will show the last used order for tiebreakers (leftmost element is most recent)

```
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1);    // cache=[1,_], cnt(1)=1
lfu.put(2, 2);    // cache=[2,1], cnt(2)=1, cnt(1)=1
lfu.get(1);       // return 1
                  // cache=[1,2], cnt(2)=1, cnt(1)=2
lfu.put(3, 3);    // 2 is the LFU key because cnt(2)=1 is the
                  // smallest, invalidate 2.
                  // cache=[3,1], cnt(3)=1, cnt(1)=2
lfu.get(2);       // return -1 (not found)
lfu.get(3);       // return 3
                  // cache=[3,1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4);    // Both 1 and 3 have the same cnt, but 1 is LRU,
                  // invalidate 1.
                  // cache=[4,3], cnt(4)=1, cnt(3)=2
lfu.get(1);       // return -1 (not found)
lfu.get(3);       // return 3
                  // cache=[3,4], cnt(4)=1, cnt(3)=3
lfu.get(4);       // return 4
                  // cache=[4,3], cnt(4)=2, cnt(3)=3
```

```

class LFUCache {
    int cap, minFreq;
    unordered_map<int, pair<int, int>> keyValFreq;
    unordered_map<int, list<int>> freqKeys;
    unordered_map<int, list<int>::iterator> keyIter;
public:
    LFUCache(int capacity) { cap = capacity; minFreq = 0; }
    int get(int key) {
        if(keyValFreq.find(key)==keyValFreq.end()) return -1;
        int val = keyValFreq[key].first, freq =
keyValFreq[key].second;
        freqKeys[freq].erase(keyIter[key]);
        if(freqKeys[freq].empty()){
            freqKeys.erase(freq);
            if(minFreq == freq) minFreq++;
        }
        freq++;
        freqKeys[freq].push_front(key);
        keyIter[key] = freqKeys[freq].begin();
        keyValFreq[key].second = freq;
        return val;
    }
    void put(int key, int value) {
        if(cap==0) return;
        if(get(key)!=-1){
            keyValFreq[key].first = value;
            return;
        }
        if(keyValFreq.size()==cap){
            int k = freqKeys[minFreq].back();
            freqKeys[minFreq].pop_back();
            keyValFreq.erase(k);
            keyIter.erase(k);
        }
        keyValFreq[key] = {value, 1};
        freqKeys[1].push_front(key);
        keyIter[key] = freqKeys[1].begin();
        minFreq = 1;}};

```

SQL\_Qs

Fundamentals & Qs.

# Fundamentals

## Fundamentals

---

### Table of Contents

1. Sample Tables for Examples
2. SQL Basics
3. Aggregate Functions
4. Window Functions Deep Dive
5. Time/Date Functions
6. Common Table Expressions (CTEs)
7. Joins

### 1. Sample Tables for Examples {#sample-tables}

Let's create sample tables that we'll use throughout this guide:

#### employees table

| emp_id | name    | department | salary | hire_date  | manager_id |
|--------|---------|------------|--------|------------|------------|
| 1      | Alice   | Sales      | 60000  | 2020-01-15 | NULL       |
| 2      | Bob     | IT         | 75000  | 2019-03-20 | 1          |
| 3      | Charlie | Sales      | 55000  | 2021-06-10 | 1          |
| 4      | Diana   | IT         | 80000  | 2018-11-05 | 1          |
| 5      | Eve     | HR         | 50000  | 2020-08-25 | 1          |
| 6      | Frank   | Sales      | 62000  | 2019-12-01 | 1          |
| 7      | Grace   | IT         | 78000  | 2021-02-14 | 4          |
| 8      | Henry   | HR         | 52000  | 2022-04-30 | 5          |

#### sales table

| sale_id | emp_id | product | amount | sale_date  |
|---------|--------|---------|--------|------------|
| 1       | 1      | Laptop  | 1200   | 2023-01-10 |
| 2       | 3      | Mouse   | 25     | 2023-01-12 |

|   |   |          |      |            |
|---|---|----------|------|------------|
| 3 | 1 | Keyboard | 75   | 2023-01-15 |
| 4 | 6 | Monitor  | 350  | 2023-01-18 |
| 5 | 3 | Laptop   | 1100 | 2023-02-01 |
| 6 | 1 | Mouse    | 30   | 2023-02-05 |
| 7 | 6 | Keyboard | 80   | 2023-02-10 |
| 8 | 3 | Monitor  | 400  | 2023-02-15 |

## 2. SQL Basics {#sql-basics}

### SELECT Statement

```
sql
-- Basic SELECT
SELECT * FROM employees;

-- Select specific columns
SELECT name, department, salary FROM employees;

-- With WHERE clause
SELECT name, salary
FROM employees
WHERE department = 'IT' AND salary > 75000;

-- With ORDER BY
SELECT name, salary
FROM employees
ORDER BY salary DESC;

-- With LIMIT
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

### INSERT, UPDATE, DELETE

```

sql
-- INSERT
INSERT INTO employees (emp_id, name, department, salary,
hire_date) VALUES (9, 'Ivan', 'Sales', 58000, '2023-03-01');

-- UPDATE
UPDATE employees SET salary = salary * 1.1 WHERE department =
'IT';

-- DELETE
DELETE FROM employees WHERE emp_id = 9;

```

## DISTINCT and GROUP BY

```

sql
-- DISTINCT values
SELECT DISTINCT department FROM employees;

-- GROUP BY with HAVING
SELECT department, COUNT(*) as emp_count, AVG(salary) as
avg_salary FROM employees GROUP BY department
HAVING COUNT(*) > 2;

```

## 3. Aggregate Functions {#aggregate-functions}

### Common Aggregate Functions

```

sql
-- COUNT
SELECT COUNT(*) as total_employees FROM employees;
SELECT COUNT(DISTINCT department) as unique_departments FROM
employees;

-- SUM
SELECT SUM(salary) as total_payroll FROM employees;

-- AVG
SELECT AVG(salary) as average_salary FROM employees;

-- MIN/MAX

```



```

SELECT
    MIN(salary) as lowest_salary,
    MAX(salary) as highest_salary
FROM employees;

-- GROUP_CONCAT / STRING_AGG (varies by database)
-- MySQL
SELECT department, GROUP_CONCAT(name) as employees_list
FROM employees
GROUP BY department;

```

### Combining Aggregates

```

sql
SELECT
    department,
    COUNT(*) as emp_count,
    MIN(salary) as min_salary,
    MAX(salary) as max_salary,
    AVG(salary) as avg_salary,
    SUM(salary) as total_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;

```

## 4. Window Functions Deep Dive {#window-functions}

### Understanding OVER()

The **OVER()** clause defines a window (a set of rows) for the function to operate on. **OVER() with no partition -> Window = entire table**

```

sql
-- Aggregate function with OVER() - calculates for ALL rows
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER() as company_avg_salary
FROM employees;

```

Result:

| name    | department | salary | company_avg_salary |
|---------|------------|--------|--------------------|
| Alice   | Sales      | 60000  | 63625              |
| Bob     | IT         | 75000  | 63625              |
| Charlie | Sales      | 55000  | 63625              |
| ...     | ...        | ...    | 63625              |

### OVER() PARTITION BY

**PARTITION BY** divides the result set into partitions (groups) and performs calculations within each partition.

```
sql
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER(PARTITION BY department) as dept_avg_salary
FROM employees;
```

Result:

| name    | department | salary | dept_avg_salary |
|---------|------------|--------|-----------------|
| Alice   | Sales      | 60000  | 59000           |
| Charlie | Sales      | 55000  | 59000           |
| Frank   | Sales      | 62000  | 59000           |
| Bob     | IT         | 75000  | 77667           |
| Diana   | IT         | 80000  | 77667           |
| Grace   | IT         | 78000  | 77667           |
| Eve     | HR         | 50000  | 51000           |

|       |    |       |       |
|-------|----|-------|-------|
| Henry | HR | 52000 | 51000 |
|-------|----|-------|-------|

## ROW\_NUMBER()

Assigns a unique sequential integer to rows within a partition.

```
sql
-- Simple ROW_NUMBER()
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER(ORDER BY salary DESC) as salary_rank
FROM employees;
```

## ROW\_NUMBER() OVER() PARTITION BY

```
sql
-- ROW_NUMBER() with PARTITION BY
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary
DESC) as dept_rank
FROM employees;
```

Result:

| name  | department | salary | dept_rank |
|-------|------------|--------|-----------|
| Henry | HR         | 52000  | 1         |
| Eve   | HR         | 50000  | 2         |
| Diana | IT         | 80000  | 1         |
| Grace | IT         | 78000  | 2         |
| Bob   | IT         | 75000  | 3         |
| Frank | Sales      | 62000  | 1         |

|         |       |       |   |
|---------|-------|-------|---|
| Alice   | Sales | 60000 | 2 |
| Charlie | Sales | 55000 | 3 |

### **RANK() vs DENSE\_RANK() vs ROW\_NUMBER()**

Key differences when there are ties:

- ROW\_NUMBER(): Always assigns unique numbers (1,2,3,4...)
- RANK(): Assigns same rank to ties, then skips numbers (1,2,2,4...)
- DENSE\_RANK(): Assigns same rank to ties, no gaps (1,2,2,3...)

```
sql
-- Let's see the difference with an example having duplicate
salaries
WITH sales_summary AS (
    SELECT
        emp_id,
        SUM(amount) as total_sales
    FROM sales
    GROUP BY emp_id
)
SELECT
    e.name,
    s.total_sales,
    ROW_NUMBER() OVER(ORDER BY s.total_sales DESC) as row_num,
    RANK() OVER(ORDER BY s.total_sales DESC) as rank_num,
    DENSE_RANK() OVER(ORDER BY s.total_sales DESC) as
dense_rank_num
FROM employees e
JOIN sales_summary s ON e.emp_id = s.emp_id;
```

### **Advanced Window Functions**

```
sql
-- LEAD and LAG - access next/previous row
SELECT
    name,
    salary,
    LAG(salary, 1) OVER(ORDER BY salary) as previous_salary,
    LEAD(salary, 1) OVER(ORDER BY salary) as next_salary
```

```

FROM employees;

-- FIRST_VALUE and LAST_VALUE
SELECT
    name,
    department,
    salary,
    FIRST_VALUE(name) OVER(PARTITION BY department ORDER BY salary
DESC) as highest_paid_in_dept,
    LAST_VALUE(name) OVER(
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) as lowest_paid_in_dept
FROM employees;

-- NTILE - divide rows into n groups
SELECT
    name,
    salary,
    NTILE(4) OVER(ORDER BY salary DESC) as salary_quartile
FROM employees;

-- Running totals with SUM() OVER()
SELECT
    sale_date,
    amount,
    SUM(amount) OVER(ORDER BY sale_date) as running_total
FROM sales
ORDER BY sale_date;

-- Moving averages
SELECT
    sale_date,
    amount,
    AVG(amount) OVER(
        ORDER BY sale_date

```

```
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) as moving_avg_3_days
FROM sales;
```

## Window Frame Specification

```
sql
-- Different window frames
SELECT
    name,
    salary,
    department,
    -- Default frame: RANGE BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW
    SUM(salary) OVER(PARTITION BY department ORDER BY salary) as
default_sum,

    -- All rows in partition
    SUM(salary) OVER(
        PARTITION BY department
        ORDER BY salary
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) as total_dept_salary,

    -- Current row and 1 before
    AVG(salary) OVER(
        PARTITION BY department
        ORDER BY salary
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) as avg_with_prev,

    -- Current row and 1 after
    AVG(salary) OVER(
        PARTITION BY department
        ORDER BY salary
        ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING
    ) as avg_with_next
FROM employees;
```

---

## 5. Time/Date Functions {#time-functions}

### Common Date Functions

```
sql
-- Current date/time
SELECT
    CURRENT_DATE,
    CURRENT_TIME,
    CURRENT_TIMESTAMP,
    NOW(); -- MySQL

-- Extract parts
SELECT
    hire_date,
    YEAR(hire_date) as year,
    MONTH(hire_date) as month,
    DAY(hire_date) as day,
    DAYOFWEEK(hire_date) as day_of_week, -- MySQL
    EXTRACT(YEAR FROM hire_date) as year_extracted -- Standard SQL
FROM employees;

-- Date arithmetic
SELECT
    name,
    hire_date,
    DATE_ADD(hire_date, INTERVAL 1 YEAR) as first_anniversary, -- MySQL
    DATE_SUB(hire_date, INTERVAL 30 DAY) as month_before, -- MySQL
    DATEDIFF(CURRENT_DATE, hire_date) as days_employed, -- MySQL
FROM employees;

-- Formatting dates
SELECT
    name,
    DATE_FORMAT(hire_date, '%W, %M %d, %Y') as formatted_date, -- MySQL
FROM employees;
```

```
-- Date truncation
SELECT
    DATE_FORMAT(hire_date, '%Y-%m-01') as month_start_mysql --
MySQL
FROM employees;
```

## Time-based Analysis

```
sql
-- Employees hired by year and quarter
SELECT
    YEAR(hire_date) as hire_year,
    QUARTER(hire_date) as hire_quarter,
    COUNT(*) as employees_hired
FROM employees
GROUP BY YEAR(hire_date), QUARTER(hire_date)
ORDER BY hire_year, hire_quarter;

-- Calculate tenure
SELECT
    name,
    hire_date,
    TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) as
years_employed,
    TIMESTAMPDIFF(MONTH, hire_date, CURRENT_DATE) % 12 as
additional_months
FROM employees
ORDER BY years_employed DESC;
```

---

## 6. Common Table Expressions (CTEs) {#ctes}

CTEs make complex queries more readable and maintainable.

```
sql
-- Basic CTE
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 70000
)
SELECT department, COUNT(*) as high_earner_count
```



```

FROM high_earners
GROUP BY department;

-- Multiple CTEs
WITH
dept_stats AS (
    SELECT
        department,
        AVG(salary) as avg_salary,
        COUNT(*) as emp_count
    FROM employees
    GROUP BY department
),
company_stats AS (
    SELECT AVG(salary) as company_avg FROM employees
)
SELECT
    d.department,
    d.avg_salary,
    d.emp_count,
    c.company_avg,
    d.avg_salary - c.company_avg as diff_from_company_avg
FROM dept_stats d
CROSS JOIN company_stats c
ORDER BY diff_from_company_avg DESC;

-- Recursive CTE (employee hierarchy)
WITH RECURSIVE emp_hierarchy AS (
    -- Anchor: start with top-level employees
    SELECT
        emp_id,
        name,
        manager_id,
        1 as level
    FROM employees
    WHERE manager_id IS NULL

```

```

UNION ALL

-- Recursive: find employees of current level
SELECT
    e.emp_id,
    e.name,
    e.manager_id,
    h.level + 1
FROM employees e
JOIN emp_hierarchy h ON e.manager_id = h.emp_id
)
SELECT
    level,
    emp_id,
    name,
    manager_id
FROM emp_hierarchy
ORDER BY level, emp_id;

```

---

## 7. Joins {#joins}

### Types of Joins

```

sql
-- INNER JOIN
SELECT
    e.name,
    e.department,
    s.product,
    s.amount
FROM employees e
INNER JOIN sales s ON e.emp_id = s.emp_id;

-- LEFT JOIN
SELECT
    e.name,
    e.department,
    COALESCE(SUM(s.amount), 0) as total_sales

```

```

FROM employees e
LEFT JOIN sales s ON e.emp_id = s.emp_id
GROUP BY e.emp_id, e.name, e.department;

-- RIGHT JOIN (less common)
SELECT
    e.name,
    s.product,
    s.amount
FROM employees e
RIGHT JOIN sales s ON e.emp_id = s.emp_id;

-- FULL OUTER JOIN (not in MySQL)
SELECT
    COALESCE(e.name, 'No Employee') as name,
    COALESCE(s.product, 'No Sale') as product
FROM employees e
FULL OUTER JOIN sales s ON e.emp_id = s.emp_id;

-- CROSS JOIN
SELECT
    e.name,
    d.department
FROM employees e
CROSS JOIN (SELECT DISTINCT department FROM employees) d;

-- SELF JOIN
SELECT
    e1.name as employee,
    e2.name as manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.emp_id;

```

---

## Practical Examples Combining Everything

### Example 1: Top N per Group

```
sql
```

```
-- Get top 2 salaries per department
WITH ranked_salaries AS (
    SELECT
        name,
        department,
        salary,
        DENSE_RANK() OVER(PARTITION BY department ORDER BY salary
DESC) as salary_rank
    FROM employees
)
SELECT name, department, salary
FROM ranked_salaries
WHERE salary_rank <= 2
ORDER BY department, salary_rank;
```

## Example 2: Year-over-Year Comparison

```
sql
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(sale_date, '%Y-%m') as month,
        SUM(amount) as total_sales
    FROM sales
    GROUP BY DATE_FORMAT(sale_date, '%Y-%m')
)
SELECT
    month,
    total_sales,
    LAG(total_sales, 12) OVER(ORDER BY month) as
same_month_last_year,
    total_sales - LAG(total_sales, 12) OVER(ORDER BY month) as
yoy_change,
    ROUND(
        (total_sales - LAG(total_sales, 12) OVER(ORDER BY month))
/
        LAG(total_sales, 12) OVER(ORDER BY month) * 100,
    2
) as yoy_percent_change
FROM monthly_sales
```

```
ORDER BY month;
```

### Example 3: Percentile Calculation

```
sql
SELECT
    name,
    salary,
    PERCENT_RANK() OVER(ORDER BY salary) as percentile_rank,
    CUME_DIST() OVER(ORDER BY salary) as cumulative_distribution,
    NTILE(100) OVER(ORDER BY salary) as percentile_bucket
FROM employees;
```

### Example 4: Gap Analysis

```
sql
-- Find gaps in sequential IDs
WITH id_gaps AS (
    SELECT
        emp_id,
        LEAD(emp_id) OVER(ORDER BY emp_id) as next_id,
        LEAD(emp_id) OVER(ORDER BY emp_id) - emp_id as gap
    FROM employees
)
SELECT
    emp_id as last_before_gap,
    next_id as first_after_gap,
    gap - 1 as missing_ids_count
FROM id_gaps
WHERE gap > 1;
```

---

### Key Takeaways

1. Window Functions vs GROUP BY: Window functions don't reduce rows like GROUP BY; they add calculated columns while preserving all rows.
2. PARTITION BY vs GROUP BY: PARTITION BY divides data for window function calculations, GROUP BY aggregates and reduces rows.
3. ROW\_NUMBER vs RANK vs DENSE\_RANK: Use ROW\_NUMBER for unique numbering, RANK when you want gaps after ties, DENSE\_RANK for continuous ranking.

4. Performance: Window functions can be more efficient than self-joins or subqueries for calculations across rows.
  5. Frame Clauses: Understanding ROWS vs RANGE and frame boundaries is crucial for moving averages and running totals.
  6. CTEs: Use them to break complex queries into manageable, readable parts.
  7. Combining Techniques: Real-world queries often combine multiple concepts - CTEs with window functions, joins with aggregates, etc.
-

Qs

Write a solution to report the products that were only sold in the first quarter of 2019. That is, between 2019-01-01 and 2019-03-31 inclusive.

```
select distinct p.product_id as product_id, p.product_name as
product_name
from Product p join Sales s
on p.product_id = s.product_id
where s.sale_date between '2019-01-01' and '2019-03-31'
and p.product_id not in (
    select sx.product_id from Sales sx where
    sx.sale_date < '2019-01-01' or sx.sale_date > '2019-03-31'
)
-- select p.product_id, p.product_name
-- from Product p
-- join Sales s on p.product_id = s.product_id
-- group by p.product_id, p.product_name having min(s.sale_date)
>= '2019-01-01' and max(s.sale_date) <= '2019-03-31';
```

Write a solution to get the names of products that have at least 100 units ordered in February 2020 and their amount.

```
select p.product_name as product_name, SUM(ofil.unit) as unit
from
(
    select o.product_id, o.unit
    from Orders o
    where year(order_date) = 2020
    and month(order_date) = 2
) ofil
join Products p
on ofil.product_id = p.product_id
group by ofil.product_id
having sum(ofil.unit) >=100
```

Find all numbers that appear at least three times consecutively.

```
select distinct(l1.num) as ConsecutiveNums
```



```

from
Logs l1 join Logs l2 join Logs l3
on l1.id = l2.id-1 and l2.id = l3.id-1
and l1.num=l2.num
and l2.num=l3.num
-- SELECT DISTINCT num AS ConsecutiveNums
-- FROM (
-- SELECT num, a.id - ROW_NUMBER() OVER (PARTITION BY num ORDER BY
id) AS label
-- FROM Logs a
-- ) cte
-- GROUP BY label, num
-- HAVING COUNT(num) >= 3;

```

Write a solution to find the people who have the most friends and the most friends number. The test cases are generated so that only one person has the most friends.

```

with base as
(
    select requester_id as id /* idr */
    from RequestAccepted
    union all
    select acceptor_id as id /* ida */
    from RequestAccepted
)
select id, count(*) as num from base
/* name taken from first expression only, i.e idr, but for sake of
error safety, we will name it same*/
group by 1
order by 2 desc
limit 1

```

Write a solution to find the employees who earn more than their managers.

```

select e.name as Employee

```

```

from Employee e
inner join Employee m
on e.managerId = m.id
where e.salary > m.salary
| e.id | e.name | e.salary | e.managerId | m.id | m.name |
m.salary |
| ---- | ----- | -|-----| -|-----| ---- | ----- |
-----|
| 1     | Joe    | 70000    | 3           | 3     | Sam    | 60000    |
| 2     | Henry  | 80000    | 4           | 4     | Max    | 90000    |

```

Write a solution to report the movies with an odd-numbered ID and a description that is not "boring".

```

select * from
Cinema
where id%2!=0 and description != "boring"
order by rating desc

```

Write a solution to find the rank of the scores. The ranking should be calculated according to the following rules:

The scores should be ranked from the highest to the lowest.

If there is a tie between two scores, both should have the same ranking. After a tie, the next ranking number should be the next consecutive integer value. In other words, there should be no holes between ranks.

```

select score,
dense_rank() over (
    order by score desc
) as 'rank'
from Scores

```

Write a solution to report all the duplicate emails. Note that it's guaranteed that the email field is not NULL.

```
-- select email as Email
-- from Person
-- group by 1
-- having count(email) > 1
select distinct(p.email) as Email
from (
select email, row_number() over (partition by email) r
from Person
) p
where r > 1;
```

Write a solution to find all customers who never order anything.

```
select c.name as Customers
from Customers c left join Orders o
on c.id = o.customerId where o.customerId is null
```

Write a solution to find all dates' id with higher temperatures compared to its previous dates (yesterday).

```
select w2.id
from Weather w1 cross join Weather w2
where w2.temperature > w1.temperature
and datediff(w2.recordDate, w1.recordDate)=1;
```

Write a solution to report the name and bonus amount of each employee with a bonus less than 1000.

```
select e.name, b.bonus
from Employee e left join Bonus b
on e.empId = b.empId
where b.bonus < 1000 or b.bonus is null
```

-----

LLD

LLD: Reference:

- <https://github.com/ashishps1/awesome-low-level-design>
- <https://www.hellointerview.com/learn/low-level-design/in-a-hurry/introduction>
- GPT

## Basics

- A class is a blueprint, template, or recipe for creating objects. It defines what an object will contain (its data) and what it will be able to do (its behavior).
- Datatype Refreshers:

```
Primary datatypes: int, long, double, float, char, boolean
Wrapper classes: Integer, Boolean, Double
Other datatypes:
List<String> names = new ArrayList<>(); // or new LinkedList<>();
// Add
names.add("Alice");
// Insert at index
names.add(1, "Zara");
// Modify
names.set(2, "Bobby"); // replace element at index 2
// Remove
names.remove("Charlie"); // by value
names.remove(0);          // by index
// Iterate
for (String name : names)
    System.out.println(name);
// Contains
names.contains("Alice");
// Sort
Collections.sort(names);
names.sort(Comparator.reverseOrder());
// Sublist
List<String> sub = names.subList(0, 2);
-----
Set<String> countries = new HashSet<>();
// Add
```

```

countries.add("India");
countries.add("India"); // ignored (duplicate)
// Remove
countries.remove("USA");
// Iterate
for (String c : countries)
    System.out.println(c);
// Contains
countries.contains("India");
-----
Map<Integer, String> students = new HashMap<>();
// Add
students.put(101, "Alice");
// Modify (same key overwrites)
students.put(102, "Bobby");
// Remove
students.remove(102);
// Iterate:
for (Map.Entry<Integer, String> entry : students.entrySet()) {
    System.out.println(entry.getKey() + "->" + entry.getValue());
} // Over everything
for (Integer id: students.keySet()) { } // Iterate over key
for (String name: students.values()) { } // Iterate over value
// Using Map.of()
Map<String, Integer> ages = Map.of(
    "Alice", 25, "Bob", 30) // Immutable- Only supports 10 entries
Map<String, Integer> big = Map.ofEntries(
    Map.entry("A", 1),
    Map.entry("B", 2)) // Mutable - Supports > 10 entries
// Contains
students.containsKey(101);
students.containsValue("Alice");
// Get Or Default. Better than using get().
students.getDefault(105, "Unknown");
// Comparing operator
For primitives → use ==
int a = 5, b = 5;

```

```

System.out.println(a==b);
For objects→ ALWAYS use .equals(). '==' references memory address
String s1 = new String("hello"); String s2 = new String("hello");
System.out.println(s1==s2); // false (different memory)
System.out.println(s1.equals(s2)); // true (same content)
-----
Queue<String> queue = new LinkedList<>();
queue.add("Task1"); // add head
System.out.println(queue.poll()); // removes head (Task1)
System.out.println(queue.peek()); // view head/front element
// Iterate
for (String t : queue)
    System.out.println(t);
-----
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.pop();
stack.peek();
// Also has methods like addLast/addFirst, etc...
-----
Min-heap (default): PriorityQueue<Integer> pq = new
PriorityQueue<>();
Max-heap: PriorityQueue<Integer> pq = new
PriorityQueue<>(Comparator.reverseOrder());
-----
Have others like LinkedHashMap, EnumMap, etc., as well.
-----

```

- Exceptions: Only methods (functions) and constructors can explicitly throw exceptions in Java. A class itself does not throw an exception.

```

public void writeFile() throws IOException {
    // ... code that might fail ...
    if (condition) {
        throw new IOException("File access denied.");
    }
}

```

```

/*
try {
    int a = 10;
    int b = 0;
    int result = a / b; // This will cause ArithmeticException
    System.out.println(result);
}
catch (ArithmeticException e) {
    System.out.println("Error: xyz");
}
catch (Exception e) {
    System.out.println("General error: " + e.getMessage());
}
finally {
    System.out.println("This block always runs.");
}
*/

```

- **Java Streams:** Java stream is a pipeline that processes a collection in a functional way. Can think of it as a conveyor belt.
  - It has 3 parts: source -> intermediate operations that return stream -> terminal operations that produce result.
  - Source: `list.stream()`, `array.stream()`, [Stream.of\(...\)](#)
  - Intermediate operations: `filter`, `map`, `sorted`, `distinct`, `limit`, `skip`
  - Terminal operations: `collect`, `toList`, `findFirst`, `min`, `max`, `count`, `anyMatch`, `allMatch`, `forEach`. Eg:

Eg:

```

List<Integer> nums = List.of(5, 3, 8, 3, 1);
Optional<Integer> smallestEven =
    nums.stream()
        .filter(n -> n % 2 == 0) // keep only even numbers
        .sorted() // sorted stream
        .findFirst(); // terminal op → executes

```

Important ones:

```

list.stream().filter(x -> x > 10) // Keeps only elements that

```



```

match a condition.
    list.stream().map(x -> x * 2) // Transforms each element into
another value (or object).
    list.stream().sorted(Comparator.comparing(Person::getAge)) //
sort
    list.stream().distinct()
    list.stream().limit(5)
    List<Integer> ans =
list.stream().collect(Collectors.toList()); // or toSet()
    list.stream().anyMatch(x -> x == 5)
    list.stream().allMatch(x -> x > 0)

```

Other egs:

```

    eg1: return spots.values().stream().filter(spot ->
!spot.isOccupied() &&
spot.canFitVehicle(vehicle)).min(Comparator.comparing(ParkingSpot:
:getSpotSize));

```

```

    eg2: int sum = orders.stream().mapToInt(Order::getCost)
.sum();

```

-----

- Normal vs Static vs Abstract vs Interface Class
  - **Normal Class:** A regular class that: Can be instantiated using new, Provides full implementation. May contain: fields, constructors, fully implemented methods, overridden methods (optional). It cannot have abstract methods. When to use:
    - When objects of the class should exist directly.
    - When full implementation is known.
    - When no forced method implementation rule is needed.

Eg:

```

class Dog {
    void bark() { System.out.println("Woof"); }
}
Dog d = new Dog(); // ✓ valid

```

- **Static:** Static members belong to the class, not the object. Used without creating an object. Shared across all objects. Cannot access non-static variables/methods. You can have a static field/method/inner class (NOT parent outer class).
  - Use static when something is shared, does not depend on object state, or should be accessible without creating an object.

```
class Counter {
    static int count = 0;
}
// Other ex:
class MathUtil {
    static int square(int n) {
        return n * n;
    }
}
MathUtil.square(5); // used directly
```

- **Abstract Class:** A class declared with abstract keyword that: Cannot be instantiated (most important), Can contain: abstract methods (no body), normal methods (with body), fields, constructors. Must be extended by child classes.
  - Why abstract classes exist: Because sometimes a parent class is: Conceptual (not concrete); Common base for multiple subclasses. Examples: Vehicle → Car, Truck; Animal → Dog, Cat; Account → Savings, Current.
  - When to use abstract class: Subclasses share common fields and logic; You want a structured base for all subclasses; Parent should never be instantiated; Some methods require compulsory overriding; Codebase is large → avoids duplication (DRY) & enforces patterns.

Eg:

```
abstract class Vehicle {
```

```

String plate;
int wheels;
Vehicle(String plate, int wheels) {
    this.plate = plate;
    this.wheels = wheels;
}
abstract void move(); // must be implemented later
void printInfo() { // normal method
    System.out.println("General vehicle");
}
}
class Car extends Vehicle {
    Car(String plate) { super(plate, 4); }
    @Override
    void move() { System.out.println("Car moving"); }
}

```

- **Abstract Methods:** Declared without body; Can only exist inside an abstract class; Must be implemented by the first concrete subclass; Abstract class cannot include abstract method with body, only signature/declaration. If subclass doesn't implement abstract methods then compile-time error (unless subclass is also abstract).
- Do all abstract methods need implementation?: Yes, but ONLY in a concrete subclass. If subclass is also abstract → it may skip implementation.
- Can abstract class contain constructor?: Yes. Reason: When you create a child object, the parent portion must be initialized.

Eg:

```

abstract class Vehicle {...
    Vehicle(String plate) { this.plate = plate; }
}
class Car extends Vehicle {
    Car() { super("KA-01"); }
}
// Abstract class cannot be instantiated directly, but its

```

constructor is used during child object creation.

- Constructor rules?: If parent constructor takes (a, b) and child passes (a, b, c) → compile error. Constructor signatures must match exactly. Also as we know, constructor has no return type. And child class can be imagined as superset of parent as Child object contains: All parent fields; All parent methods; Its own fields and methods; It is one single object → parent + child portions.
- **Interface**: A pure behavior contract. Defines what a class must do, not how. Interfaces contain: abstract methods; default methods (Java 8+); static methods; constants (public static final). A class can implement multiple interfaces but extend only single class. (Hence multiple inheritance done via interface). Interface cannot hold common state (fields except constants). Interface = no identity / no shared data, only behavior. Interface cannot be instantiated. No constructors.
  - Use interface when: Unrelated classes need same behavior (Dog, Car, Drone can all be Movable); You want a pure contract; You need multiple inheritance of behavior; No shared fields or common state is needed.

Eg:

```
interface Drivable {  
    void drive();  
}  
class Car implements Drivable {  
    @Override  
    public void drive() { System.out.println("Car driving"); }  
}
```

\*Another Eg:\*

code

```
public interface PaymentGateway {  
    void initiatePayment(double amount);  
}  
public class StripePayment implements PaymentGateway {
```

```

        public void initiatePayment(double amount) {
            System.out.println("Processing payment via Stripe: $" +
amount);
        }
    }
    public class RazorpayPayment implements PaymentGateway {
        public void initiatePayment(double amount) {
            System.out.println("Processing payment via Razorpay: ₹" +
amount);
        }
    }
    public class CheckoutService {
        private PaymentGateway paymentGateway;
        public CheckoutService(PaymentGateway paymentGateway) {
            this.paymentGateway = paymentGateway;
        }
        public void checkout(double amount) {
            paymentGateway.initiatePayment(amount);
        }
    }
    public class Main {
        public static void main(String[] args) {
            PaymentGateway stripeGateway = new StripePayment();
            CheckoutService service = new
CheckoutService(stripeGateway);
            service.checkout(120.50); // Output: Processing payment
via Stripe: $120.5
            // Switch to Razorpay
            PaymentGateway razorpayGateway = new RazorpayPayment();
            service.setPaymentGateway(razorpayGateway);
            service.checkout(150.50); // Output: Processing payment
via Razorpay: ₹150.5
        }
    }
}

```

- A Combined/Mix Example:

```

abstract class Vehicle {
    String plate;
    int wheels;
    Vehicle(String plate, int wheels) {
        this.plate = plate;
        this.wheels = wheels;
    }
}
interface Drivable {
    void drive();
}
class Car extends Vehicle implements Drivable {
    Car(String plate) { super(plate, 4); }
    @Override
    public void drive() { System.out.println("Car driving"); }
}

```

- Misc stuff:

- Instant entry = Instant.now();
- UUID.randomUUID().toString()
- new Date().getTime()
- A final field must be assigned once and only once. Can be assigned during declaration time or inside constructor.  
Case: OK  

```
private final int x = 10; // Assigned at declaration
```

 Case: OK  

```
private final int x;
public MyClass() {
    x = 10; // Assigned in constructor
}
```

 Case: FALSE  

```
private final int x; // ❌ not assigned here
public MyClass() {
    // ❌ not assigned in constructor → compile error
}
```
- printf is the easiest way to format numbers, strings, and floats in Java -- just like C.

```

Integers: int n = 10; System.out.printf("Number=%d",n);
Float: double price = 12.346;
System.out.printf("Price=%f",price);
String: String name = "Suraj"; System.out.printf("Hi,%s",
name);
Char: System.out.printf("First letter: %c %n", 'A');
Escape sequences:
| Sequence | Meaning |
| ----- | ----- |
| `\\n` | new line (platform independent) |
| `\\%` | print a literal % |
| `\\t` | tab |
| `\\n` | new line (Unix) |
| ----- |

```

- **Enum:** It's a special data type that defines a collection of constant values under a single name. Are type-safe.

```

code
public enum VehicleSize {
    SMALL,
    MEDIUM,
    LARGE;
}
// Can access like: VehicleSize.MEDIUM -> Enum type. Other ways:
.name()/toString(): Returns the exact enum constant name as
written in the code. But toString() can be override to customize
display. Eg: VehicleSize.SMALL.name();
.ordinal(): Returns the index of the constant, based on
declaration order. Shouldn't be used in prod as if order changes
index changes. Safer option below: Eg:
public enum VehicleSize {
    SMALL(1),
    MEDIUM(2),
    LARGE(3);
    private final int level;
    VehicleSize(int level) {
        this.level = level;
    }
}

```

```

    }
    public int getLevel() {
        return level;
    }
}

```

Eg: `VehicleSize.SMALL.getLevel()` can be used. Live eg., would look like:

```

spotA.getSpotSize().getLevel() < spotB.getSpotSize().getLevel()
/*

```

SMALL, MEDIUM, and LARGE are objects. Each enum constant is an object of type `VehicleSize` just like: `new VehicleSize(1)`. But you cannot call `new`. Java automatically creates these objects for you. When Java loads this enum, it does: `VehicleSize SMALL = new VehicleSize(1)`. Fundamentally, these are singletons.

Why can't I write `new VehicleSize(1)` myself?: Because enums are a special restricted class, etc.

```

*/

```

- **Encapsulation:**

- Data hiding + Controlled access, via: Access Modifiers - `private`, `public`, `protected` | Getters & Setters.

Encapsulation focuses on how to protect and control access to data within a class. But if we extend that idea not just hiding data, but also hiding complexity itself; That's where Abstraction comes in.

code

```

class PaymentProcessor {
    private String cardNumber;
    private double amount;
    public PaymentProcessor(String cardNumber, double amount) {
        this.cardNumber = maskCardNumber(cardNumber);
        this.amount = amount;
    }
    public void processPayment() {
        System.out.println("Processing payment of " + amount + "
for card " + cardNumber);
    }
}

```



```

public class Main {
    public static void main(String[] args) {
        PaymentProcessor payment = new
PaymentProcessor("1234567812345678", 250.00);
        payment.processPayment();
    }
}

```

- **Abstraction:**

- Hiding Complexity + Showing Essentials. Achieved via abstract class or interfaces. Either of them cannot be instantiated. A class can extend only one abstract class but can implement multiple interfaces.

```

code
abstract class Vehicle {
    String brand;
    // Constructor
    Vehicle(String brand) {
        this.brand = brand;
    }
    // Abstract method (must be implemented by subclasses)
    abstract void start();
    // Concrete method (can be inherited)
    void displayBrand() {
        System.out.println("Brand: " + brand);
    }
}
// Subclass implementing the abstract method
class Car extends Vehicle {
    Car(String brand) {
        super(brand);
    }
    @Override
    void start() {
        System.out.println("Car is starting...");
    }
}

```

- **Inheritance:**

- Allows one class (called the subclass or child class) to inherit the properties and behaviors of another class (called the superclass or parent class).

code

```
class ElectricCar extends Car {  
    public void chargeBattery() {  
        System.out.println("Battery charging");  
    }  
}
```

- **Polymorphism:**

- Allows the same method name or interface to exhibit different behaviors depending on the object that is invoking it.

Compile-time Polymorphism (Static Binding), Also known as method overloading

code

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public double add(double a, double b) {  
        return a + b;  
    }  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Runtime Polymorphism (Dynamic Binding), Also known as method overriding

code

```
public interface NotificationSender {  
    void sendNotification(String message);  
}  
public class EmailSender implements NotificationSender {  
    public void sendNotification(String message) {  
        System.out.println("Sending EMAIL: " + message);  
    }  
}
```

```

    }
}
public class SmsSender implements NotificationSender {
    public void sendNotification(String message) {
        System.out.println("Sending SMS: " + message);
    }
}
public void notifyUser(NotificationSender sender, String message)
{
    sender.sendNotification(message);
}

```

## Class Relations: Association, Aggregation, Composition, Dependency Association

Definition: The most general and structural relationship, showing that instances of two classes are connected. It means one class is somehow related to another. No ownership is implied; classes are generally independent. Typically bidirectional (but can be unidirectional)

Dashed line in UML (---)

Example: Teacher Student Interaction

```

code
class Teacher {
    private String name;
    private List<Student> students; // Teacher knows students
    public void teach(Student student) {
        System.out.println(name + " teaching " + student.getName());
    }
}
class Student {
    private String name;
    private List<Teacher> teachers; // Student knows teachers
    public String getName() {
        return name;
    }
}
// Both Teacher and Student can exist independently

```

## Aggregation

Definition: A special form of Association that represents a "part-whole" or "has-a" relationship, but with weak ownership. The part (the contained object) can exist independently of the whole (the container object).

Hollow diamond in UML (◇---)

Example: Department has Employees

```
code
class Department {
    private String name;
    private List<Employee> employees; // Department has employees
    public Department(String name) {
        this.name = name;
        this.employees = new ArrayList<>();
    }
    public void addEmployee(Employee emp) {
        employees.add(emp);
    }
}

class Employee {
    private String name;
    private String id;
    public Employee(String name, String id) {
        this.name = name;
        this.id = id;
    }
}

// Usage:
Employee emp1 = new Employee("John", "E001");
Employee emp2 = new Employee("Jane", "E002");
Department dept = new Department("Engineering");
dept.addEmployee(emp1);
dept.addEmployee(emp2);
// Employees can exist even if department is destroyed
```

## Composition

Definition: A more restrictive and stronger form of Aggregation where the whole is solely responsible for the lifecycle of its part. The part cannot exist without the whole.

Filled diamond in UML (◆---)

Example: House owns Rooms

code

```
class House {
    private String address;
    private List<Room> rooms; // House owns rooms
    public House(String address) {
        this.address = address;
        this.rooms = new ArrayList<>();
        // Creating rooms inside constructor - strong ownership
        rooms.add(new Room("Living Room", 300));
        rooms.add(new Room("Bedroom", 200));
        rooms.add(new Room("Kitchen", 150));
    }
    // Inner class - Room cannot exist without House
    private class Room {
        private String name;
        private int squareFeet;
        public Room(String name, int squareFeet) {
            this.name = name;
            this.squareFeet = squareFeet;
        }
    }
}
// When House object is garbage collected, all Room objects are
destroyed too
```

## Dependency

Definition: The weakest form of relationship, indicating that one class uses another class, usually on a temporary basis, such as a local variable in a method or as a parameter to a method.

Dashed arrow in UML (---->)

No field reference maintained

code

```
class EmailService {
    // EmailService depends on EmailMsg and EmailServer
    public void sendEmail(EmailMsg message, EmailServer server) {
        // Uses EmailMessage & EmailServer but doesn't store them
    }
}
```

```

        server.connect();
        server.send(message);
        server.disconnect();
    }
}
class EmailMessage {
    private String to;
    private String subject;
    private String body;
    // Constructor and getters
}
class EmailServer {
    public void connect() { }
    public void send(EmailMessage msg) { }
    public void disconnect() { }
}
// EmailService doesn't maintain references to EmailMsg or
EmailServer

```

| Feature           | Association                | Aggregation<br>(Weak<br>"Has-A")         | Composition<br>(Strong<br>"Has-A")                 | Dependency<br>("Uses-A")                |
|-------------------|----------------------------|------------------------------------------|----------------------------------------------------|-----------------------------------------|
| Relationship Type | General connection         | Part-of<br>(Optional)                    | Part-of<br>(Mandatory)                             | Temporary link (E.g., Method parameter) |
| Ownership         | None implied; just a link. | Weak ownership; the part is independent. | Strong/Exclusive ownership; the part is dependent. | None; just temporary use.               |
| Part's Lifecycle  | Independent                | Independent                              | Dependent (Part dies with the whole).              | Independent                             |

|              |          |                                                |                                                |          |
|--------------|----------|------------------------------------------------|------------------------------------------------|----------|
| Part Sharing | Possible | Possible (Part can belong to multiple wholes). | Not possible (Part belongs to only one whole). | Possible |
|--------------|----------|------------------------------------------------|------------------------------------------------|----------|

Design principles:

- Don't Repeat Yourself (DRY) (Every piece of knowledge should have a single, unambiguous representation in the system.)
- Keep It Simple Stupid (KISS) (Systems work best when they are kept simple rather than made complex.)
- You Aren't Gonna Need It (YAGNI) (Don't add functionality until it's actually needed.)
- Law of Demeter (LoD) / Principle of Least Knowledge (A method should only talk to its immediate friends, not strangers.)

SOLID Principles:

### Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change. It should have only one job or responsibility.

BAD:

```
class Employee {
    private String name;
    private double salary;
    public double calculateSalary() { return salary * 12; }    //
calc
    public void saveToDatabase() { /* DB logic */ }           //
persist
    public String generateReport() { return name + salary; }  //
report
    public void sendEmail(String msg) { /* email logic */ }   //
notify
}
```

GOOD:

```
class Employee { String name; double salary; }
class SalaryCalculator {
    double calcAnnual(Employee e) { return e.salary * 12; }
```

```

}
class EmployeeRepository {
    void save(Employee e) { /* DB ops */ }
}
class ReportGenerator {
    String generate(Employee e) { return e.name + " " + e.salary; }
}
}
class EmailService {
    void send(String to, String msg) { /* email logic */ }
}

```

### Open/Closed Principle (OCP)

Definition: Software entities should be open for extension but closed for modification.

BAD:

```

class PaymentProcessor {
    public void processPayment(String paymentType, double amount)
    {
        if (paymentType.equals("CREDIT_CARD")) {
            // Credit card processing
            System.out.println("Processing credit card payment: "
+ amount);
        } else if (paymentType.equals("PAYPAL")) {
            // PayPal processing
            System.out.println("Processing PayPal payment: " +
amount);
        } else if (paymentType.equals("BITCOIN")) {
            // Bitcoin processing - had to modify this class!
            System.out.println("Processing Bitcoin payment: " +
amount);
        }
        // Every new payment method requires modifying this class
    }
}

```

GOOD:

// Open for extension, closed for modification



```
interface PaymentMethod {
    void processPayment(double amount);
}
class CreditCardPayment implements PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment: " +
amount);
    }
}
class PayPalPayment implements PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment: " +
amount);
    }
}
// New payment method - just add new class, no modification needed
class BitcoinPayment implements PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing Bitcoin payment: " +
amount);
    }
}
class PaymentProcessor {
    public void processPayment(PaymentMethod paymentMethod, double
amount) {
        paymentMethod.processPayment(amount);
        // No modification needed for new payment types!
    }
}
// Usage
PaymentProcessor processor = new PaymentProcessor();
processor.processPayment(new CreditCardPayment(), 100.0);
processor.processPayment(new BitcoinPayment(), 0.5);
```

## Liskov Substitution Principle (LSP)

Definition: Objects of a superclass should be replaceable with objects of its subclasses without breaking the application.

BAD:

```
class Rectangle {
    protected int width;
    protected int height;
    public void setWidth(int width) {
        this.width = width;
    }
    public void setHeight(int height) {
        this.height = height;
    }
    public int getArea() {
        return width * height;
    }
}
// Violates LSP!
class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width; // Changes behavior!
    }
    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height; // Changes behavior!
    }
}
// This breaks!
void testArea(Rectangle rect) {
    rect.setWidth(5);
    rect.setHeight(4);
    assert rect.getArea() == 20; // Fails for Square! Gets 16
}
```

GOOD:

```
interface Shape {
    int getArea();
}
class Rectangle implements Shape {
    private int width;
    private int height;
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
    public void setWidth(int width) {
        this.width = width;
    }
    public void setHeight(int height) {
        this.height = height;
    }
    @Override
    public int getArea() {
        return width * height;
    }
}
class Square implements Shape {
    private int side;
    public Square(int side) {
        this.side = side;
    }
    public void setSide(int side) {
        this.side = side;
    }
    @Override
    public int getArea() {
        return side * side;
    }
}
// Now both can be used correctly
void printArea(Shape shape) {
    System.out.println("Area: " + shape.getArea());
}
```

```
}
```

## Interface Segregation Principle (ISP)

Definition: No client should be forced to depend on methods it doesn't use. Create specific interfaces rather than one general-purpose interface.

BAD:

```
// Fat interface - forces implementation of unused methods
```

```
interface Worker {  
    void work();  
    void eat();  
    void sleep();  
    void attendMeeting();  
    void submitReport();  
}  
  
class HumanWorker implements Worker {  
    @Override  
    public void work() {  
        System.out.println("Working...");  
    }  
    @Override  
    public void eat() {  
        System.out.println("Eating lunch...");  
    }  
    @Override  
    public void sleep() {  
        System.out.println("Sleeping...");  
    }  
    @Override  
    public void attendMeeting() {  
        System.out.println("Attending meeting...");  
    }  
    @Override  
    public void submitReport() {  
        System.out.println("Submitting report...");  
    }  
}
```

```

class RobotWorker implements Worker {
    @Override
    public void work() {
        System.out.println("Working 24/7...");
    }
    @Override
    public void eat() {
        // Forced to implement but doesn't make sense!
        throw new UnsupportedOperationException("Robots don't
eat");
    }
    @Override
    public void sleep() {
        // Forced to implement but doesn't make sense!
        throw new UnsupportedOperationException("Robots don't
sleep");
    }
    @Override
    public void attendMeeting() {
        throw new UnsupportedOperationException("Robots don't
attend meetings");
    }
    @Override
    public void submitReport() {
        System.out.println("Generating automated report...");
    }
}

```

GOOD:

```

// Segregated interfaces
interface Workable {
    void work();
}
interface Eatable {
    void eat();
}
interface Sleepable {
    void sleep();
}

```

```
}
interface Reportable {
    void submitReport();
}
interface Attendable {
    void attendMeeting();
}
// Human implements all
class HumanWorker implements Workable, Eatable, Sleepable,
Attendable, Reportable {
    @Override
    public void work() {
        System.out.println("Working...");
    }
    @Override
    public void eat() {
        System.out.println("Eating lunch...");
    }
    @Override
    public void sleep() {
        System.out.println("Sleeping...");
    }
    @Override
    public void attendMeeting() {
        System.out.println("Attending meeting...");
    }
    @Override
    public void submitReport() {
        System.out.println("Submitting report...");
    }
}
// Robot only implements what it needs
class RobotWorker implements Workable, Reportable {
    @Override
    public void work() {
        System.out.println("Working 24/7...");
    }
}
```

```

@Override
public void submitReport() {
    System.out.println("Generating automated report...");
}
// No need to implement eat, sleep, or attendMeeting!
}

```

## Dependency Inversion Principle (DIP)

Definition:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

BAD:

```

// Low-level module
class MySQLDatabase {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}
// High-level module depends on low-level concrete class
class UserService {
    private MySQLDatabase database; // Tight coupling!
    public UserService() {
        this.database = new MySQLDatabase(); // Hard dependency
    }
    public void saveUser(String userData) {
        database.save(userData);
        // If we want to switch to PostgreSQL, we have to modify
        this class!
    }
}

```

GOOD:

```

// Abstraction
interface Database {
    void save(String data);
}

```

```

// Low-level modules implement abstraction
class MySQLDatabase implements Database {
    @Override
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}

class PostgreSQLDatabase implements Database {
    @Override
    public void save(String data) {
        System.out.println("Saving to PostgreSQL: " + data);
    }
}

class MongoDBDatabase implements Database {
    @Override
    public void save(String data) {
        System.out.println("Saving to MongoDB: " + data);
    }
}

// High-level module depends on abstraction
class UserService {
    private Database database; // Depends on abstraction!
    // Dependency injection
    public UserService(Database database) {
        this.database = database;
    }
    public void saveUser(String userData) {
        database.save(userData);
        // Can switch databases without modifying this class!
    }
}

// Usage
Database mysql = new MySQLDatabase();
UserService service1 = new UserService(mysql);
Database mongo = new MongoDBDatabase();
UserService service2 = new UserService(mongo);

```



## Design Patterns - Creational

### **\*\*Singleton Pattern**

Definition: Ensures a class has only one instance and provides a global point of access to it. Few ways; (Note: Singleton can be broken by reflection, serialization, and cloning. To protect it, we check instance in constructor, use readResolve(), and override clone())

❌ Not Thread-Safe (Lazy):

```
class Singleton {
    private static Singleton instance;
    private Singleton() {} // Private constructor
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton(); // Race condition!
        }
        return instance;
    }
}
```

✅ Thread-Safe (Synchronized Method):

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    // Synchronized - thread safe but slow
    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

✅ Thread-Safe (Double-Checked Locking) - BEST for lazy initialization:

```
public class Singleton {
    // volatile ensures visibility across threads
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
```

```

        if (instance == null) { // First check (no locking)
            synchronized (Singleton.class) {
                if (instance == null) { // Second check (with
locking)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

✅ Thread-Safe (Bill Pugh - Inner Static Class) - RECOMMENDED:

```

public class Singleton {
    private Singleton() {}
    // Inner static class - loaded only when getInstance() is
called
    private static class SingletonHelper {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return SingletonHelper.INSTANCE;
    }
}

```

✅ Thread-Safe (Eager Initialization):

```

public class Singleton {
    // Created at class loading time
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return INSTANCE;
    }
}

```

✅ Enum Singleton - BEST for serialization safety:

```

public enum Singleton {
    INSTANCE;
    public void doSomething() {
        System.out.println("Doing something...");
    }
}

```

```

    }
}
// Usage
Singleton.INSTANCE.doSomething();

```

## **\*\*Factory Method Pattern**

Definition: Defines an interface for creating objects, but lets subclasses decide which class to instantiate.

```

// Product
interface Notification {
    void send(String message);
}
// Concrete Products
class EmailNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending Email: " + message);
    }
}
class SMSNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
    }
}
class PushNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Sending Push Notification: " +
message);
    }
}
// Factory
class NotificationFactory {
    public static Notification createNotification(String type) {
        switch (type.toUpperCase()) {
            case "EMAIL":

```

```

        return new EmailNotification();
    case "SMS":
        return new SMSNotification();
    case "PUSH":
        return new PushNotification();
    default:
        throw new IllegalArgumentException("Unknown
notification type");
    }
}
}
// Usage
Notification notification =
NotificationFactory.createNotification("EMAIL");
notification.send("Hello World!");

```

## **\*\*Builder Pattern**

Definition: Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

```

public class Employee {
    // Required fields
    private final String name;
    private final String department;
    // Optional fields
    private final int age;
    private final Map<String, String> preferences;
    // Private constructor
    private Employee(Builder builder) {
        this.name = builder.name;
        this.department = builder.department;
        this.age = builder.age;
        this.preferences = builder.preferences;
    }
    // Getters
    public String getName(){ return name; }
    public String getDepartment(){ return department; }
}

```

```

    public int getAge(){ return age; }
    public Map<String, String> getPreferences(){ return
preferences;}
    // Builder class
    public static class Builder {
        // Required
        private final String name;
        private final String department;
        // Optional
        private int age = 0;
        private Map<String, String> preferences = Map.of();
        // Constructor for required params
        public Builder(String name, String department) {
            this.name = name;
            this.department = department;
        }
        public Builder age(int age) {
            this.age = age;
            return this;
        }
        public Builder preferences(Map<String, String>
preferences) {
            this.preferences = preferences;
            return this;
        }
        public Employee build() {
            return new Employee(this);
        }
    }
    @Override
    public String toString() {
        return "Employee{" +
            "name='" + name + '\'' +
            ", department='" + department + '\'' +
            ", age=" + age +
            ", preferences=" + preferences +
            '}';
    }

```

```

    }
    // Example usage
    public static void main(String[] args) {
        Employee emp1 = new Employee.Builder("Alice",
"Engineering").age(28).preferences(Map.of("Shift", "Morning",
"Laptop", "Mac")).build();
        Employee emp2 = new Employee.Builder("Bob", "HR").build();
    }
}

```

## Abstract Factory Pattern

Definition: Provides an interface for creating families of related objects (like UI components, database connections, or devices) without specifying their concrete classes.

```

// Abstract Products
interface Button {
    void render();
}
interface Checkbox {
    void render();
}
// Concrete Products
class LightButton implements Button {
    public void render() { System.out.println("Light Button"); }
}
class DarkButton implements Button {
    public void render() { System.out.println("Dark Button"); }
}
class LightCheckbox implements Checkbox {
    public void render() { System.out.println("Light Checkbox"); }
}
class DarkCheckbox implements Checkbox {
    public void render() { System.out.println("Dark Checkbox"); }
}
// Abstract Factory
interface UIFactory {
    Button createButton();
}

```

```

        Checkbox createCheckbox();
    }
    // Concrete Factories
    class LightUIFactory implements UIFactory {
        public Button createButton() { return new LightButton(); }
        public Checkbox createCheckbox() { return new LightCheckbox(); }
    }
    class DarkUIFactory implements UIFactory {
        public Button createButton() { return new DarkButton(); }
        public Checkbox createCheckbox() { return new DarkCheckbox(); }
    }
    // Client
    public class AbstractFactoryDemo {
        public static void main(String[] args) {
            UIFactory factory = new DarkUIFactory(); // can switch to
            LightUIFactory easily
            Button btn = factory.createButton();
            Checkbox chk = factory.createCheckbox();
            btn.render();
            chk.render();
        }
    }
}

```

## Prototype Pattern

Definition: Creates new objects by copying an existing object (prototype) rather than creating from scratch.

```

// import java.util.*;
// Prototype Interface
interface Prototype extends Cloneable {
    Prototype clone();
}
// Concrete Prototypes
class Circle implements Prototype {
    private int radius;
    public Circle(int radius) { this.radius = radius; }
}

```

```

@Override
public Prototype clone() {
    return new Circle(this.radius); // deep copy
}
public void draw() {
    System.out.println("Drawing Circle with radius " +
radius);
}
}
class Rectangle implements Prototype {
    private int width, height;
    public Rectangle(int width, int height) {
        this.width = width; this.height = height;
    }
    @Override
    public Prototype clone() {
        return new Rectangle(this.width, this.height);
    }
    public void draw() {
        System.out.println("Drawing Rectangle " + width + "x" +
height);
    }
}
// Client
public class PrototypeDemo {
    public static void main(String[] args) {
        Prototype circle1 = new Circle(10);
        Prototype circle2 = circle1.clone(); // copy instead of
new
        Prototype rect1 = new Rectangle(5, 8);
        Prototype rect2 = rect1.clone();
        ((Circle) circle2).draw();
        ((Rectangle) rect2).draw();
    }
}

```



## Design Patterns - Structural

### **\*\*Adapter Pattern**

Definition: Converts the interface of a class into another interface that clients expect. Allows classes with incompatible interfaces to work together.

```
// Target interface
interface MediaPlayer {
    void play(String fileName);
}

// Adaptee (incompatible interface)
class VLCPlayer {
    void playVlcFile(String fileName) {
        System.out.println("Playing VLC: " + fileName);
    }
}

// Adapter (composition)
class MediaAdapter implements MediaPlayer {
    private VLCPlayer vlc = new VLCPlayer();

    @Override
    public void play(String fileName) {
        vlc.playVlcFile(fileName); // adapts method name
    }
}

// Client
public class AdapterDemo {
    public static void main(String[] args) {
        MediaPlayer player = new MediaAdapter();
        player.play("song.vlc");
    }
}

// Here, MediaAdapter is tightly coupled to VlcPlayer (it knows
// exactly how to call it).
// But the client (AudioPlayer) only depends on the MediaPlayer
// interface -- not VlcPlayer or MediaAdapter.
// Inside the adapter class, there is a dependency on the adaptee
// -- but that coupling is localized, not spread to client code.
```

## **\*\*Facade Pattern**

Definition: Provides a unified, simplified interface to a complex subsystem. Makes the subsystem easier to use.

```
// Subsystem classes
class CPU {
    void start() { System.out.println("CPU starting..."); }
}
class Memory {
    void load() { System.out.println("Memory loading..."); }
}
class Disk {
    void read() { System.out.println("Disk reading data..."); }
}
// Facade
class ComputerFacade {
    private final CPU cpu = new CPU();
    private final Memory memory = new Memory();
    private final Disk disk = new Disk();

    public void startComputer() {
        System.out.println("Starting computer...");
        cpu.start();
        memory.load();
        disk.read();
        System.out.println("Computer ready!");
    }
}
// Client
public class FacadeDemo {
    public static void main(String[] args) {
        ComputerFacade computer = new ComputerFacade();
        computer.startComputer(); // One simple call
    }
}
```

## **\*\*Decorator Pattern**

Definition: Attaches additional responsibilities to an object dynamically. Provides a flexible alternative to subclassing for

extending functionality.

```
// Base
interface Coffee {
    String getDescription();
    double getCost();
}

class SimpleCoffee implements Coffee {
    public String getDescription() { return "Simple Coffee"; }
    public double getCost() { return 2.0; }
}

// Decorator
class MilkDecorator implements Coffee {
    private Coffee coffee;
    public MilkDecorator(Coffee coffee) { this.coffee = coffee; }
    // Use @override for safety
    public String getDescription() { return
coffee.getDescription() + ", Milk"; }
    public double getCost() { return coffee.getCost() + 0.5; }
}

class SugarDecorator implements Coffee {
    private Coffee coffee;
    public SugarDecorator(Coffee coffee) { this.coffee = coffee; }
    // Use @override for safety
    public String getDescription() { return
coffee.getDescription() + ", Sugar"; }
    public double getCost() { return coffee.getCost() + 0.2; }
}

// Use
public class Main {
    public static void main(String[] args) {
        Coffee coffee = new SimpleCoffee(); // base
        // Memory: SimpleCoffee{ }
        coffee = new MilkDecorator(coffee); // add milk
        // Memory: MilkDecorator{coffee: SimpleCoffee{ }}
        coffee = new SugarDecorator(coffee); // add sugar
        // Memory: SugarDecorator{ coffee: MilkDecorator{ coffee:
SimpleCoffee{ }}}}
```

```

        System.out.println(coffee.getDescription()); // Simple
        Coffee, Milk, Sugar
        System.out.println(coffee.getCost()); // 2.7
    }
}

```

## **\*\*Composite Pattern**

Definition: Composes objects into tree structures to represent part-whole hierarchies. Allows clients to treat individual objects and compositions uniformly.

Structure:

Component (interface)

└─ Leaf (File / Developer)

└─ Composite (Directory / Manager) → contains list of Components

```

interface FileSystem {
    void showDetails();
}
// Leaf
class File implements FileSystem {
    private String name;
    public File(String name) { this.name = name; }
    public void showDetails() { System.out.println("File: " +
name); }
}
// Composite
class Folder implements FileSystem {
    private String name;
    private List<FileSystem> items = new ArrayList<>();
    public Folder(String name) { this.name = name; }
    public void add(FileSystem item) { items.add(item); }
    public void showDetails() {
        System.out.println("Folder: " + name);
        for (FileSystem item : items) item.showDetails();
    }
}
// Usage
public class CompositeDemo {

```

```

    public static void main(String[] args) {
        FileSystem file1 = new File("resume.pdf");
        FileSystem file2 = new File("photo.jpg");
        Folder docs = new Folder("Documents");
        docs.add(file1);
        docs.add(file2);
        docs.showDetails();
    }
}

```

## Proxy Pattern

Definition: Provides a surrogate or placeholder for another object to control access, add security, caching, or lazy loading without changing the original object.

```

// Subject Interface
interface Image {
    void display();
}

// Real Subject
class RealImage implements Image {
    private String filename;
    public RealImage(String filename) {
        this.filename = filename;
        loadFromDisk(); // heavy operation
    }
    private void loadFromDisk() {
        System.out.println("Loading image from disk: " +
filename);
    }
    @Override
    public void display() {
        System.out.println("Displaying image: " + filename);
    }
}

// Proxy
class ProxyImage implements Image {
    private RealImage realImage;

```

```

    private String filename;
    public ProxyImage(String filename) {
        this.filename = filename;
    }
    @Override
    public void display() {
        // Lazy loading - load only when needed
        if (realImage == null) {
            realImage = new RealImage(filename);
        }
        realImage.display();
    }
}
// Client
public class ProxyDemo {
    public static void main(String[] args) {
        Image image = new ProxyImage("photo.jpg");
        System.out.println("First display:");
        image.display(); // Loads + displays
        System.out.println("\nSecond display:");
        image.display(); // Cached, no reload
    }
}

```

## Bridge Pattern

Definition: Decouples an abstraction from its implementation so that the two can vary independently.

```

// Implementor
interface Device {
    void turnOn();
    void turnOff();
    boolean isEnabled();
}
// Concrete Implementors
class TV implements Device {
    private boolean on = false;
    public void turnOn() { on = true; System.out.println("TV

```

```

turned ON"); }
    public void turnOff() { on = false; System.out.println("TV
turned OFF"); }
    public boolean isEnabled() { return on; }
}
class Radio implements Device {
    private boolean on = false;
    public void turnOn() { on = true; System.out.println("Radio
turned ON"); }
    public void turnOff() { on = false; System.out.println("Radio
turned OFF"); }
    public boolean isEnabled() { return on; }
}
// Abstraction
abstract class Remote {
    protected Device device;
    protected Remote(Device device) {
        this.device = device;
    }
    public abstract void togglePower();
}
// Refined Abstraction
class AdvancedRemote extends Remote {
    public AdvancedRemote(Device device) {
        super(device);
    }
    @Override
    public void togglePower() {
        if (device.isEnabled()) device.turnOff();
        else device.turnOn();
    }
}
// Client
public class BridgeDemo {
    public static void main(String[] args) {
        Remote remote = new AdvancedRemote(new TV());
        remote.togglePower(); // TV ON
    }
}

```

```

        remote.togglePower(); // TV OFF
        Remote radioRemote = new AdvancedRemote(new Radio());
        radioRemote.togglePower(); // Radio ON
    }
}

```

## Flyweight Pattern

Definition: Uses sharing to support large numbers of fine-grained objects efficiently. Reduces memory usage by sharing common state.

```

// import java.util.*;
// Flyweight
interface Shape {
    void draw(int x, int y);
}
// Concrete Flyweight
class Circle implements Shape {
    private String color; // intrinsic state (shared)
    public Circle(String color) {
        this.color = color;
    }
    @Override
    public void draw(int x, int y) {
        System.out.println("Drawing " + color + " circle at (" + x
+ ", " + y + ")");
    }
}
// Flyweight Factory
class ShapeFactory {
    private static final Map<String, Shape> circleMap = new
HashMap<>();
    public static Shape getCircle(String color) {
        return circleMap.computeIfAbsent(color, c -> {
            System.out.println("Creating new circle of color: " +
c);
            return new Circle(c);
        });
    }
}

```



```

}
// Client
public class FlyweightDemo {
    public static void main(String[] args) {
        String[] colors = {"Red", "Green", "Blue", "Red",
"Green"};

        for (String color : colors) {
            Shape circle = ShapeFactory.getCircle(color);
            circle.draw(new Random().nextInt(100), new
Random().nextInt(100));
        }
    }
}

```

## Design Patterns - Behavioral

### **\*\*Observer Pattern**

Definition: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

```

// Observer interface
interface Subscriber {
    void update(String videoTitle);
}
// Subject interface
interface Channel {
    void subscribe(Subscriber subscriber);
    void unsubscribe(Subscriber subscriber);
    void notifySubscribers();
}
// Concrete Subject
class YouTubeChannel implements Channel {
    private List<Subscriber> subscribers = new ArrayList<>();
    private String latestVideo;
    @Override
    public void subscribe(Subscriber subscriber) {
        subscribers.add(subscriber);
    }
}

```

```

    }
    @Override
    public void unsubscribe(Subscriber subscriber) {
        subscribers.remove(subscriber);
    }
    @Override
    public void notifySubscribers() {
        for (Subscriber s : subscribers) {
            s.update(latestVideo);
        }
    }
    public void uploadVideo(String title) {
        this.latestVideo = title;
        System.out.println("\nNew video uploaded: " + title);
        notifySubscribers();
    }
}

// Concrete Observers
class User implements Subscriber {
    private String name;
    public User(String name) {
        this.name = name;
    }
    @Override
    public void update(String videoTitle) {
        System.out.println(name + " got notified: New video - " +
videoTitle);
    }
}

// Usage
public class ObserverDemo {
    public static void main(String[] args) {
        YouTubeChannel channel = new YouTubeChannel();
        Subscriber user1 = new User("Alice");
        Subscriber user2 = new User("Bob");
        Subscriber user3 = new User("Charlie");
        channel.subscribe(user1);
    }
}

```

```

        channel.subscribe(user2);
        channel.subscribe(user3);
        channel.uploadVideo("Design Patterns in Java");
        channel.unsubscribe(user2);
        channel.uploadVideo("Observer Pattern Simplified!");
    }
}
// Output
New video uploaded: Design Patterns in Java
Alice got notified: New video - Design Patterns in Java
Bob got notified: New video - Design Patterns in Java
Charlie got notified: New video - Design Patterns in Java
New video uploaded: Observer Pattern Simplified!
Alice got notified: New video - Observer Pattern Simplified!
Charlie got notified: New video - Observer Pattern Simplified!

```

## **\*\*Strategy Pattern**

Definition: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can switch strategy (behavior) without changing its code.

```

// Strategy interface
interface TextFormatter {
    String format(String text);
}
// Concrete Strategies
class UpperCaseFormatter implements TextFormatter {
    public String format(String text) {
        return text.toUpperCase();
    }
}
class LowerCaseFormatter implements TextFormatter {
    public String format(String text) {
        return text.toLowerCase();
    }
}
class TitleCaseFormatter implements TextFormatter {
    public String format(String text) {
        return Arrays.stream(text.split(" "))

```

```

        .map(w -> Character.toUpperCase(w.charAt(0))
+ w.substring(1).toLowerCase())
        .collect(Collectors.joining(" "));
    }
}
// Context
class TextEditor {
    private TextFormatter formatter;
    public void setFormatter(TextFormatter formatter) {
        this.formatter = formatter;
    }
    public void publish(String text) {
        System.out.println("Formatted: " +
formatter.format(text));
    }
}
// Usage
public class StrategyDemo {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        editor.setFormatter(new UpperCaseFormatter());
        editor.publish("strategy pattern made easy");
        editor.setFormatter(new TitleCaseFormatter());
        editor.publish("strategy pattern made easy");
    }
}

```

## **\*\*Command Pattern**

Definition: Encapsulates a request as an object, allowing you to parameterize clients with different requests, queue requests, and support undoable operations.

```

// Command interface
interface Command {
    void execute();
}
// Receiver
class Light {
    void on() { System.out.println("Light is ON"); }
}

```

```

        void off() { System.out.println("Light is OFF"); }
    }
    // Concrete Commands
    class LightOnCommand implements Command {
        private Light light;
        public LightOnCommand(Light light) { this.light = light; }
        public void execute() { light.on(); }
    }
    class LightOffCommand implements Command {
        private Light light;
        public LightOffCommand(Light light) { this.light = light; }
        public void execute() { light.off(); }
    }
    // Invoker
    class RemoteButton {
        private Command command;
        public void setCommand(Command command) { this.command =
command; }
        public void press() { command.execute(); }
    }
    // Usage
    public class CommandDemo {
        public static void main(String[] args) {
            Light livingRoomLight = new Light();
            RemoteButton remote = new RemoteButton();
            remote.setCommand(new LightOnCommand(livingRoomLight));
            remote.press();    // Light is ON
            remote.setCommand(new LightOffCommand(livingRoomLight));
            remote.press();    // Light is OFF
        }
    }
    // Key Points to Remember
    // Command → encapsulates a request (execute() method).
    // Receiver → performs the real action.
    // Invoker → triggers the command (doesn't know how it's done).
    // Undo feature → can be added by storing executed commands.

```

## **\*\*State Pattern**

Definition: Allows an object to alter its behavior when its internal

state changes. The object will appear to change its class.

```
// State interface
interface TrafficLightState {
    void handleRequest(TrafficLight light);
}

// Concrete States
class RedLight implements TrafficLightState {
    @Override
    public void handleRequest(TrafficLight light) {
        System.out.println("RED → Stop");
        light.setState(new GreenLight());
    }
}

class GreenLight implements TrafficLightState {
    @Override
    public void handleRequest(TrafficLight light) {
        System.out.println("GREEN → Go");
        light.setState(new YellowLight());
    }
}

class YellowLight implements TrafficLightState {
    @Override
    public void handleRequest(TrafficLight light) {
        System.out.println("YELLOW → Slow down");
        light.setState(new RedLight());
    }
}

// Context
class TrafficLight {
    private TrafficLightState state;
    public TrafficLight() {
        state = new RedLight(); // initial state
    }
    public void setState(TrafficLightState state) {
        this.state = state;
    }
    public void change() {
```

```

        state.handleRequest(this);
    }
}
// Usage
public class StatePatternDemo {
    public static void main(String[] args) {
        TrafficLight light = new TrafficLight();

        // simulate cycles
        for (int i = 0; i < 5; i++) {
            light.change();
        }
    }
}

```

## **\*\*Chain of Responsibility Pattern**

Definition: Passes request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain.

```

// Handler interface
interface SupportHandler {
    void setNext(SupportHandler next);
    void handleRequest(String issue);
}
// Concrete Handlers
class BasicSupport implements SupportHandler {
    private SupportHandler next;
    @Override
    public void setNext(SupportHandler next) {
        this.next = next;
    }
    @Override
    public void handleRequest(String issue) {
        if (issue.equalsIgnoreCase("password reset")) {
            System.out.println("Basic Support: Handling password
reset.");
        } else if (next != null) {
            next.handleRequest(issue);
        }
    }
}

```

```

        } else {
            System.out.println("No handler available for: " +
issue);
        }
    }
}

class TechnicalSupport implements SupportHandler {
    private SupportHandler next;
    @Override
    public void setNext(SupportHandler next) {
        this.next = next;
    }
    @Override
    public void handleRequest(String issue) {
        if (issue.equalsIgnoreCase("network issue")) {
            System.out.println("Technical Support: Handling
network issue.");
        } else if (next != null) {
            next.handleRequest(issue);
        } else {
            System.out.println("No handler available for: " +
issue);
        }
    }
}

class ManagerSupport implements SupportHandler {
    @Override
    public void setNext(SupportHandler next) {
        // Last handler - no next
    }
    @Override
    public void handleRequest(String issue) {
        System.out.println("Manager: Escalating " + issue + " to
higher authorities.");
    }
}

// Usage

```



```

public class ChainDemo {
    public static void main(String[] args) {
        SupportHandler basic = new BasicSupport();
        SupportHandler tech = new TechnicalSupport();
        SupportHandler manager = new ManagerSupport();
        basic.setNext(tech);
        tech.setNext(manager);
        basic.handleRequest("password reset");
        basic.handleRequest("network issue");
        basic.handleRequest("billing problem");
    }
}

```

## Iterator Pattern

Definition: Provides a way to access elements of a collection sequentially without exposing its underlying representation.

```

// import java.util.*;
// Iterator interface
interface Iterator<T> {
    boolean hasNext();
    T next();
}
// Aggregate interface
interface Container<T> {
    Iterator<T> getIterator();
}
// Concrete Collection
class NameCollection implements Container<String> {
    private String[] names = {"Alice", "Bob", "Charlie", "Diana"};
    @Override
    public Iterator<String> getIterator() {
        return new NameIterator();
    }
}
// Concrete Iterator
private class NameIterator implements Iterator<String> {
    int index = 0;
    @Override
    public boolean hasNext() {

```

```

        return index < names.length;
    }
    @Override
    public String next() {
        return hasNext() ? names[index++] : null;
    }
}
}
// Usage
public class IteratorDemo {
    public static void main(String[] args) {
        NameCollection collection = new NameCollection();
        Iterator<String> iterator = collection.getIterator();

        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}

```

## Template Method Pattern

Definition: Defines the skeleton of an algorithm in a base class, letting subclasses override specific steps without changing the algorithm's structure.

```

// Abstract class (Template)
abstract class DataParser {
    // Template method (final to prevent override)
    public final void parseData() {
        readData();
        processData();
        saveData();
    }
    abstract void readData();
    abstract void processData();
    // Common step
    void saveData() {
        System.out.println("Saving processed data to database.");
    }
}

```

```

}
// Concrete classes
class CSVParser extends DataParser {
    void readData() {
        System.out.println("Reading data from CSV file.");
    }
    void processData() {
        System.out.println("Processing CSV data.");
    }
}
class JSONParser extends DataParser {
    void readData() {
        System.out.println("Reading data from JSON file.");
    }
    void processData() {
        System.out.println("Processing JSON data.");
    }
}
// Usage
public class TemplateDemo {
    public static void main(String[] args) {
        DataParser csvParser = new CSVParser();
        csvParser.parseData();
        System.out.println();
        DataParser jsonParser = new JSONParser();
        jsonParser.parseData();
    }
}

```

## Visitor Pattern

Definition: Lets you define a new operation without changing the classes of the elements on which it operates. Separates algorithms from the objects they operate on.

```

// Element interface
interface Shape {
    void accept(ShapeVisitor visitor);
}
// Concrete elements

```

```
class Circle implements Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    public void accept(ShapeVisitor visitor) {
        visitor.visit(this);
    }
}

class Rectangle implements Shape {
    double width, height;
    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
    public void accept(ShapeVisitor visitor) {
        visitor.visit(this);
    }
}

// Visitor interface
interface ShapeVisitor {
    void visit(Circle circle);
    void visit(Rectangle rectangle);
}

// Concrete visitors (operations)
class AreaCalculator implements ShapeVisitor {
    public void visit(Circle circle) {
        double area = Math.PI * circle.radius * circle.radius;
        System.out.println("Circle area: " + area);
    }
    public void visit(Rectangle rectangle) {
        double area = rectangle.width * rectangle.height;
        System.out.println("Rectangle area: " + area);
    }
}

class ShapePrinter implements ShapeVisitor {
    public void visit(Circle circle) {
        System.out.println("Drawing a Circle with radius " +
circle.radius);
    }
}
```

```

    }
    public void visit(Rectangle rectangle) {
        System.out.println("Drawing a Rectangle (" +
rectangle.width + "x" + rectangle.height + ")");
    }
}
// Usage
public class VisitorDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6)
        };
        ShapeVisitor areaCalc = new AreaCalculator();
        ShapeVisitor printer = new ShapePrinter();
        for (Shape s : shapes) {
            s.accept(areaCalc);
            s.accept(printer);
        }
    }
}

```

## Mediator Pattern

Definition: Defines an object that encapsulates how a set of objects interact. Promotes loose coupling by keeping objects from referring to each other explicitly.

```

// Mediator interface
interface ChatMediator {
    void showMessage(User user, String message);
}
// Concrete Mediator
class ChatRoom implements ChatMediator {
    public void showMessage(User user, String message) {
        System.out.println(user.getName() + ": " + message);
    }
}
// Colleague class
class User {

```

```

    private String name;
    private ChatMediator mediator;
    public User(String name, ChatMediator mediator) {
        this.name = name;
        this.mediator = mediator;
    }
    public String getName() { return name; }
    public void send(String message) {
        mediator.showMessage(this, message);
    }
}
// Usage
public class MediatorDemo {
    public static void main(String[] args) {
        ChatMediator chatRoom = new ChatRoom();

        User alice = new User("Alice", chatRoom);
        User bob = new User("Bob", chatRoom);

        alice.send("Hi Bob!");
        bob.send("Hey Alice, how are you?");
    }
}

```

## Memento Pattern

Definition: Captures and externalizes an object's internal state without violating encapsulation, allowing the object to be restored to this state later.

```

// Memento - stores state
class TextMemento {
    private final String state;
    public TextMemento(String state) { this.state = state; }
    public String getState() { return state; }
}
// Originator - creates/restores memento
class TextEditor {
    private String content = "";
    public void type(String words) { content += words; }
}

```

```

        public String getContent() { return content; }
        public TextMemento save() { return new TextMemento(content); }
        public void restore(TextMemento memento) { content =
memento.getState(); }
    }
    // Caretaker - manages mementos
    class EditorHistory {
        private final java.util.Stack<TextMemento> history = new
java.util.Stack<>();
        public void save(TextEditor editor) {
history.push(editor.save()); }
        public void undo(TextEditor editor) {
            if (!history.isEmpty()) editor.restore(history.pop());
        }
    }
    // Usage
    public class MementoDemo {
        public static void main(String[] args) {
            TextEditor editor = new TextEditor();
            EditorHistory history = new EditorHistory();
            editor.type("Hello ");
            history.save(editor);
            editor.type("World!");
            System.out.println(editor.getContent()); // Hello World!
            history.undo(editor);
            System.out.println(editor.getContent()); // Hello
        }
    }
}

```

---

HLD



## HLD

Good HLD Reference:

- <https://github.com/ashishps1/awesome-system-design-resources?tab=readme-ov-file#-system-design-interview-problems>
- [https://www.youtube.com/watch?v=Qi3Kzq\\_rlKQ&list=PLjTveVh7FakLGZ36GbWak\\_DMf\\_0xBZpGv&index=9](https://www.youtube.com/watch?v=Qi3Kzq_rlKQ&list=PLjTveVh7FakLGZ36GbWak_DMf_0xBZpGv&index=9)
- GPT

## 1. NETWORKING FUNDAMENTALS

### IP Addressing

- **IPv4:** 32-bit addresses (4 billion addresses), format: 192.168.1.1
- **IPv6:** 128-bit addresses (340 undecillion addresses), format: 2001:0db8:85a3::8a2e:0370:7334
- **Types:** Public (internet-facing), Private (internal network), Static (fixed), Dynamic (DHCP-assigned)

### OSI Model (7 Layers)

- **Application (L7):** User-facing protocols (HTTP, SMTP, FTP)
- **Presentation (L6):** Data translation, encryption, compression
- **Session (L5):** Connection management between applications
- **Transport (L4):** End-to-end communication (TCP/UDP)
- **Network (L3):** Routing, IP addressing
- **Data Link (L2):** Frame transmission, MAC addressing
- **Physical (L1):** Physical hardware, cables, signals

### TCP vs UDP

#### TCP (Transmission Control Protocol)

- Connection-oriented with 3-way handshake
- Reliable delivery with acknowledgments
- Ordered packets, flow control, congestion control
- Higher overhead, slower
- Use cases: HTTP, HTTPS, FTP, email

#### UDP (User Datagram Protocol)

- Connectionless, no handshake
- No delivery guarantee, packets may arrive out of order
- Lower overhead, faster

- Use cases: Video streaming, VoIP, gaming, DNS

## **Domain Name System (DNS)**

Translates domain names to IP addresses through hierarchical lookup.

### **DNS Hierarchy:**

1. Recursive Resolver (ISP DNS)
2. Root Nameserver (13 global root servers)
3. TLD Nameserver (.com, .org, .net)
4. Authoritative Nameserver (actual IP)

### **DNS Records:**

- **A Record:** Domain → IPv4
- **AAAA Record:** Domain → IPv6
- **CNAME:** Alias → Another domain
- **MX Record:** Mail server routing
- **NS Record:** Nameserver for domain
- **TXT Record:** Text info (SPF, DKIM)

**DNS Caching:** TTL-based caching at browser, OS, and resolver levels

---

## **2. SCALABILITY & PERFORMANCE**

### **Performance vs Scalability**

- **Performance Problem:** System slow for single user
- **Scalability Problem:** System fast for one user but slow under load
- Goal: Maximize throughput with acceptable latency

### **Latency vs Throughput**

- **Latency:** Time to perform an action (response time)
- **Throughput:** Number of actions per unit time (requests/sec)
- Trade-off: Higher throughput may increase latency

### **Vertical vs Horizontal Scaling**

#### **Vertical Scaling (Scale Up)**

- Add more power (CPU, RAM) to existing machine
- Pros: Simple, no code changes, data consistent
- Cons: Hardware limits, single point of failure, expensive, downtime

#### **Horizontal Scaling (Scale Out)**

- Add more machines to pool
- Pros: No limits, fault tolerant, cost-effective

- **Cons:** Complex, data consistency challenges, network overhead
- 

### 3. LOAD BALANCING

Distributes incoming traffic across multiple servers.

#### Algorithms

- **Round Robin:** Sequential distribution
- **Weighted Round Robin:** Based on server capacity
- **Least Connections:** Send to server with fewest active connections
- **Least Response Time:** Fastest response + fewest connections
- **IP Hash:** Based on client IP (sticky sessions)

#### Layer 4 vs Layer 7

##### Layer 4 (Transport Layer)

- Routes based on IP + Port (no content inspection)
- Faster, less resource intensive
- Simple NAT forwarding

##### Layer 7 (Application Layer)

- Routes based on content (headers, cookies, URL path)
- Content-based routing, SSL termination
- More features but higher overhead

#### High Availability

- **Active-Passive:** Standby server takes over on failure
  - **Active-Active:** All servers handle traffic simultaneously
- 

### 4. CACHING

Stores frequently accessed data in fast storage (RAM) to reduce latency.

#### Cache Levels

- **Client-side:** Browser cache, OS cache
- **CDN:** Geographically distributed edge servers
- **Web Server:** Reverse proxy cache (Varnish, Nginx)
- **Application:** In-memory cache (Redis, Memcached)
- **Database:** Query cache, buffer pool

#### Cache Strategies

##### Cache-Aside (Lazy Loading)

- App checks cache first, then DB on miss

- App responsible for loading cache
- Pros: Only requested data cached
- Cons: Cache miss penalty (3 trips), stale data possible

### **Write-Through**

- Write to cache and DB synchronously
- Pros: No stale data, cache always consistent
- Cons: Higher write latency, unused data cached

### **Write-Behind (Write-Back)**

- Write to cache, async write to DB
- Pros: Fast writes, high throughput
- Cons: Data loss risk if cache crashes

### **Refresh-Ahead**

- Automatically refresh before expiration
- Pros: Reduced latency for predicted data
- Cons: Wasteful if predictions wrong

### **Cache Eviction Policies**

- **LRU (Least Recently Used)**: Remove oldest unused
- **LFU (Least Frequently Used)**: Remove least accessed
- **FIFO**: Remove first added
- **Random**: Random removal

### **Cache Invalidation**

- **TTL (Time To Live)**: Expiration-based
- **Event-based**: Invalidate on data change
- **Manual**: Explicit deletion

---

## **5. CONTENT DELIVERY NETWORK (CDN)**

Globally distributed servers serving content from locations closest to users.

### **Push CDN**

- Content uploaded to CDN when changed
- Pros: Low latency, control over updates
- Cons: Higher storage costs, manual management
- Use: Low traffic sites, infrequent updates

### **Pull CDN**

- CDN fetches content on first user request

- Pros: Minimal storage, automatic updates
  - Cons: Slower first request, redundant traffic
  - Use: High traffic sites, frequent changes
- 

## 6. PROXY SERVERS

**Forward Proxy:** Sits between clients and internet, makes requests on behalf of clients.

- **Uses:** Content filtering, anonymity, bypass restrictions, caching

**Reverse Proxy:** Sits between internet and servers, receives client requests.

- **Benefits:** Load balancing, SSL termination, caching, security, compression
  - **Examples:** Nginx, HAProxy
- 

## 7. AVAILABILITY & RELIABILITY

### Availability Metrics

- **99% (2 nines):** 3.65 days downtime/year
- **99.9% (3 nines):** 8.77 hours downtime/year
- **99.99% (4 nines):** 52.6 minutes downtime/year
- **99.999% (5 nines):** 5.25 minutes downtime/year

### Availability Patterns

#### Failover

- **Active-Passive:** Heartbeat monitoring, passive takes over on failure
- **Active-Active:** Both servers handle traffic, better resource utilization

#### Replication

- Multiple copies across servers for redundancy
- Improves availability and read performance

**Components in Sequence:** Total availability =  $A \times B$  (decreases)

**Parallel:** Total availability =  $1 - (1-A)(1-B)$  (increases)

---

## 8. CONSISTENCY MODELS

### CAP Theorem

In distributed systems, choose 2 of 3:

- **Consistency:** All nodes see same data simultaneously
- **Availability:** Every request gets response
- **Partition Tolerance:** System works despite network failures

**CP Systems:** Consistency + Partition Tolerance (MongoDB, HBase, Redis)

**AP Systems:** Availability + Partition Tolerance (Cassandra, CouchDB, DynamoDB)

**CA Systems:** Not realistic in distributed systems (traditional RDBMS in single node)

### **PACELC Theorem**

Extension of CAP: If Partition, choose A or C; Else choose Latency or Consistency

### **Consistency Patterns**

- **Weak Consistency:** No guarantee when data will be visible (VoIP, gaming)
  - **Eventual Consistency:** Data will be consistent eventually (DNS, email, S3)
  - **Strong Consistency:** Immediate consistency, reads always get latest (RDBMS, file systems)
- 

## **9. DATABASES - SQL**

Structured data with predefined schema, ACID transactions.

### **ACID Properties**

- **Atomicity:** All or nothing transaction
- **Consistency:** Valid state transitions only
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed data persists

### **Database Scaling Techniques**

#### **Master-Slave Replication**

- Master: Handles writes + reads
- Slaves: Handle reads only (replicate from master)
- Pros: Scalable reads, backups
- Cons: Replication lag, master failure downtime

#### **Master-Master Replication**

- Both handle reads + writes
- Pros: Better write scalability, no single point
- Cons: Conflict resolution, consistency issues

#### **Federation (Functional Partitioning)**

- Split by function (users DB, products DB, orders DB)
- Pros: Less traffic per DB, more cache hits
- Cons: Complex joins across DBs, not for large tables

### **Sharding (Horizontal Partitioning)**

- Split data across multiple databases by key
- Strategies: Hash-based, List-based, Range-based
- Pros: Horizontal scalability, better performance
- Cons: Complex queries, rebalancing difficult, no joins

### **Denormalization**

- Add redundant data to avoid expensive joins
- Pros: Faster reads
- Cons: Data duplication, complex writes

### **Indexing**

- B-tree indexes for fast lookups
- Speeds up SELECT, WHERE, JOIN, ORDER BY
- Tradeoff: Faster reads, slower writes, more storage

### **Normalization (1NF, 2NF, 3NF, BCNF)**

- Remove data redundancy
- Ensures data integrity
- Reduces anomalies (insert, update, delete)
- May require more joins

---

## **10. DATABASES - NoSQL**

Schema-less, horizontally scalable, BASE model.

### **BASE Properties**

- **Basically Available:** System guarantees availability
- **Soft State:** State may change over time
- **Eventual Consistency:** Consistency achieved eventually

### **NoSQL Types**

#### **Key-Value Store**

- Simple hash table, O(1) reads/writes
- Examples: Redis, DynamoDB, Memcached
- Use: Caching, session storage, user preferences

#### **Document Store**

- JSON/XML documents with flexible schema
- Examples: MongoDB, CouchDB, Firestore
- Use: Content management, catalogs, user profiles

### **Column-Family Store**

- Data stored in column families, not rows
- Examples: Cassandra, HBase, Google Bigtable
- Use: Time-series data, analytics, big data

### **Graph Database**

- Nodes and relationships
- Examples: Neo4j, Amazon Neptune
- Use: Social networks, recommendation engines, fraud detection

### **SQL vs NoSQL Decision**

#### **Choose SQL when:**

- Structured data, strict schema needed
- Complex joins required
- ACID transactions critical
- Well-established patterns

#### **Choose NoSQL when:**

- Massive scale (TB/PB data)
- Flexible/dynamic schema
- High throughput needed
- Horizontal scaling required
- Simple access patterns

---

## **11. DISTRIBUTED SYSTEMS CONCEPTS**

### **Consistent Hashing**

Distributes keys across nodes with minimal redistribution when nodes added/removed.

#### **Process:**

1. Hash nodes and keys onto ring (0 to  $2^{32}$ )
2. Key goes to first node clockwise
3. Adding/removing node only affects adjacent keys

**Virtual Nodes:** Each physical node mapped multiple times to improve load distribution

**Benefits:** Minimal redistribution, better load balance, handles failures gracefully

### **Database Transactions**

**Transaction States:** Active → Partially Committed → Committed/Failed → Aborted/Terminated



### **Distributed Transactions:**

#### **Two-Phase Commit (2PC)**

- Phase 1: Prepare (all nodes vote)
- Phase 2: Commit (if all yes) or Abort
- Problem: Blocking protocol, coordinator failure

#### **Three-Phase Commit (3PC)**

- Adds pre-commit phase between prepare and commit
- Non-blocking, handles coordinator failure better

#### **Saga Pattern**

- Sequence of local transactions
  - Choreography: Event-driven
  - Orchestration: Central coordinator
  - Compensating transactions for rollback
- 

## **12. ARCHITECTURAL PATTERNS**

### **N-Tier Architecture**

Separates application into logical layers and physical tiers.

#### **Common Tiers:**

- **3-Tier:** Presentation → Business Logic → Data
- **2-Tier:** Client → Server (no middle tier)
- **1-Tier:** All on single machine

**Pros:** Scalable layers independently, better security, separation of concerns

**Cons:** Network latency, complexity, expensive

### **Monolith vs Microservices**

#### **Monolithic**

- Single deployable unit, tightly coupled
- Pros: Simple development/deployment, easy debugging, ACID transactions
- Cons: Hard to scale, technology lock-in, single point of failure

#### **Microservices**

- Independent services, loosely coupled, separate databases
- Pros: Independent scaling/deployment, technology flexibility, fault isolation
- Cons: Complex, distributed system challenges, eventual consistency

**Distributed Monolith:** Looks like microservices but tightly coupled (worst of

both)

### **Event-Driven Architecture (EDA)**

Services communicate through events via message broker.

#### **Components:**

- Event Producers: Publish events
- Event Router: Message broker (Kafka, RabbitMQ)
- Event Consumers: Subscribe and react

**Pros:** Loose coupling, scalable, easy to add consumers **Cons:** Complex, difficult error handling, eventual consistency

### **Event Sourcing**

Store events (state changes) instead of current state.

- Append-only event log
- Rebuild state by replaying events
- Complete audit trail
- Use: Financial systems, audit requirements

### **CQRS (Command Query Responsibility Segregation)**

Separate read and write models.

- Commands: Change state (writes)
- Queries: Read state (reads)
- Different databases for reads/writes
- Often used with Event Sourcing

**Pros:** Independent scaling, optimized models **Cons:** Complexity, eventual consistency, more infrastructure

---

## **13. API DESIGN & COMMUNICATION**

### **REST (Representational State Transfer)**

- Stateless, resource-based (nouns)
- HTTP methods: GET (read), POST (create), PUT (update/replace), PATCH (partial update), DELETE
- Status codes: 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error)

#### **Principles:**

- Uniform interface
- Stateless

- Cacheable
- Client-server separation
- Layered system

### GraphQL

- Query language for APIs
- Client specifies exact data needed
- Single endpoint, no over/under-fetching
- Pros: Flexible, efficient, strong typing
- Cons: Complexity, caching harder, learning curve

### gRPC

- Remote Procedure Call using Protocol Buffers
- HTTP/2 based, binary protocol
- Strong typing, code generation
- Pros: Fast, efficient, bi-directional streaming
- Cons: Not browser-friendly, binary format harder to debug

### RPC vs REST

- **RPC:** Action-oriented (verbs), tight coupling, faster
- **REST:** Resource-oriented (nouns), loose coupling, standard HTTP

## 14. MESSAGE-DRIVEN ARCHITECTURE

### Message Queue

Point-to-point (1:1), FIFO delivery, each message consumed once.

- **Use:** Task distribution, decoupling, load leveling
- **Examples:** RabbitMQ, Amazon SQS, ActiveMQ

#### Features:

- Dead letter queues
- Message persistence
- Priority queues
- Exactly-once delivery

### Publish-Subscribe (Pub/Sub)

One-to-many (1:N), messages pushed to all subscribers.

- **Use:** Event notification, fanout, real-time updates
- **Examples:** Apache Kafka, Google Pub/Sub, Redis Pub/Sub

#### Benefits:

- Dynamic subscribers
- Decoupled publishers/subscribers
- Message filtering

### **Message Broker**

Intermediary for message exchange, translation between protocols.

- Validates, stores, routes messages
- Supports both queue and pub/sub patterns

### **Enterprise Service Bus (ESB)**

Centralized integration hub for applications.

- Transformation, routing, orchestration
  - Pros: Standardized integration
  - Cons: Single point of failure, complexity, bottleneck
- 

## **15. API GATEWAY & BFF**

### **API Gateway**

Single entry point for all clients, handles cross-cutting concerns.

#### **Responsibilities:**

- Authentication & authorization
- Rate limiting & throttling
- Load balancing
- Request/response transformation
- Caching
- Logging & monitoring
- Service discovery
- Circuit breaking

**Pros:** Simplified client, centralized logic, security **Cons:** Single point of failure, complexity, latency

### **Backend for Frontend (BFF)**

Separate backend for each client type (web, mobile, IoT).

- Optimized for specific frontend needs
  - Formats data appropriately
  - Reduces client-side logic
- 

## **16. RELIABILITY PATTERNS**

## **Circuit Breaker**

Prevents cascading failures by stopping calls to failing service.

**States:**

- **Closed:** Normal operation
- **Open:** Requests fail immediately (service down)
- **Half-Open:** Test if service recovered

## **Rate Limiting**

Controls request rate to prevent abuse and overload.

**Algorithms:**

- **Token Bucket:** Tokens refill at constant rate
- **Leaky Bucket:** Requests leak at constant rate
- **Fixed Window:** Counter per time window
- **Sliding Window:** Overlapping windows
- **Sliding Window Log:** Precise but memory-intensive

## **Bulkhead Pattern**

Isolate resources to prevent total system failure.

- Separate connection pools per service
- Limit threads per operation
- Prevents resource exhaustion

## **Service Discovery**

Automatically detect service locations in dynamic environments.

- **Client-side:** Client queries registry (Consul, Eureka)
- **Server-side:** Load balancer queries registry
- Health checks, automatic registration/deregistration

---

# **17. STORAGE SYSTEMS**

## **Storage Types**

### **File Storage**

- Hierarchical directory structure
- Examples: NFS, Amazon EFS
- Use: Shared file access

### **Block Storage**

- Raw storage blocks, fixed size
- Examples: Amazon EBS, SAN

- Use: Databases, VMs

### **Object Storage**

- Flat namespace, metadata-rich
- Examples: Amazon S3, Azure Blob
- Use: Unstructured data, backups, media

### **RAID (Redundant Array of Independent Disks)**

- **RAID 0:** Striping (performance, no redundancy)
- **RAID 1:** Mirroring (redundancy, 50% capacity)
- **RAID 5:** Striping + parity (balance)
- **RAID 6:** Striping + double parity (2 disk failures)
- **RAID 10:** Striping + mirroring (best performance + redundancy)

### **Distributed File Systems**

#### **HDFS (Hadoop Distributed File System)**

- Block-based, replication for fault tolerance
- Write-once, read-many
- Use: Big data processing

---

## **18. ASYNC PROCESSING**

### **Long Polling**

Client requests, server holds until data available.

- Less overhead than polling
- Still blocking connection

### **WebSockets**

Full-duplex, persistent connection over single TCP.

- Real-time, bi-directional communication
- Use: Chat, gaming, live updates

### **Server-Sent Events (SSE)**

Server pushes updates to client over HTTP.

- Unidirectional (server → client)
- Automatic reconnection
- Use: Live feeds, notifications

---

## **19. SECURITY**

### **Best Practices**

- **Encryption:** TLS in transit, AES at rest
  - **Authentication:** OAuth 2.0, JWT, API keys
  - **Authorization:** RBAC, ABAC
  - **Input Validation:** Prevent XSS, SQL injection
  - **Rate Limiting:** Prevent DoS
  - **Least Privilege:** Minimal necessary permissions
  - **Security Headers:** CORS, CSP, HSTS
- 

## 20. IMPORTANT NUMBERS

### Latency Numbers

- L1 cache: 0.5 ns
- L2 cache: 7 ns
- RAM: 100 ns
- SSD random read: 150  $\mu$ s
- SSD sequential read: 1 ms
- HDD seek: 10 ms
- Network same datacenter: 0.5 ms
- Network CA  $\rightarrow$  Netherlands: 150 ms

### Capacity Estimates

- 1 KB =  $10^3$  bytes
- 1 MB =  $10^6$  bytes
- 1 GB =  $10^9$  bytes
- 1 TB =  $10^{12}$  bytes
- 1 PB =  $10^{15}$  bytes

### Throughput

- RAM: 4 GB/s sequential
- SSD: 1 GB/s sequential
- HDD: 30 MB/s sequential
- 1 Gbps network: 100 MB/s

# InterviewQsLearnings



## Interview Qs asked:

- (Some more job interview question/assignments solved saved in drive)

---

### Problem: API Marketplace Analytics Backend

We are building the backend for a cloud-based API Marketplace (similar to RapidAPI or AWS API Gateway) where developers can discover and use third-party APIs.

Let's start with the following functional requirements:

- > `registerApi(apiName [String])` -> An API is given an incremental integer ID when it is registered, starting with 1.
- > `invokeApi(apiId [Integer], developerId [Integer])` -> Assume any developer ID is valid and that the given API ID will have already been registered. -> This represents a developer successfully making a call to an API.
- > `printAnalyticsSummary()` -> This is used for an end-of-day billing and analytics report. -> The summary should be sorted (descending) by the number of unique developers who have invoked each API. -> The summary should include the API names and the number of unique developers, but the exact string formatting does not matter.
- > `printLastThreeInvokedApis(developerId [Integer])` -> Prints the names of the last three unique APIs the given developer has invoked. -> Must be ordered with the most recently invoked API first.

---

### Objective:

Design and implement a rate limiter using Erlang's GenServer that restricts the number of allowed requests from a given user (identified by a unique ID) within a specified time window.

### Requirements:

#### 1. Rate Limit Logic:

- Each user is allowed a maximum of 5 requests within a 10 second window.
  - If the user exceeds the limit, additional requests should be rejected until the window resets.
-

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

TimeMap() Initializes the object of the data structure.

void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.

All the timestamps timestamp of set call are strictly increasing.

String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp\_prev <= timestamp. If there are multiple such values, it returns the value associated with the largest timestamp\_prev. If there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]
```

```
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

```
map<string, vector<pair<string, int>>> mp
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

```
timeMap.set("foo", "bar", 1); // store the key "foo" and value "bar" along with timestamp = 1.
```

```
timeMap.get("foo", 1);      // return "bar"
```

```
timeMap.get("foo", 3);      // return "bar", since there is no value corresponding to foo at timestamp 3 and timestamp 2, then the only value is at timestamp 1 is "bar".
```

```
timeMap.set("foo", "bar2", 4); // store the key "foo" and value "bar2" along with timestamp = 4.
```

```
timeMap.get("foo", 4);      // return "bar2"
```

```
timeMap.get("foo", 5);      // return "bar2"
```

SOLN:

```
class TimeMap {
```

```

unordered_map<string, vector<pair<string,int>> timeDataSet;
void set(string key, string value, int time){
    timeDataSet[key].push_back({value, time});
}
void get(string key, int time){
    if(timeDataSet[key]==timeDataSet.end()) return NULL;
    vector<pair<int,int>> keyValue = timeDataSet[key]
    int size = keyValue.size();
    int lowValue = 0; int highValue = size-1;
    int targetTime = time;
    int globalLow = INT_MIN;
    while(lowValue<=highValue){
        int midValue = lowValue + (highValue-lowValue)/2
        pair<int,int> tempPair = timeDataSet[mid]
        if(tempPair.second == targetTime){
            return tempPair.first;
        }
        if(tempPair.second > target){
            highValue = midValue;
        }
        else{
            lowValue = midValue;
            globalLow = lowValue;
        }
    }
    if(globalLow!=INT_MIN){
        return vector[globalLow].first;
    }
}
};

```

Time complexity for get() -> O(1) since inserting in the Map

Time complexity for set() -> O(1) + O(LogN) -> Since using map and binary search approach

---

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob =  $1 + 3 = 4$ .

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob =  $2 + 9 + 1 = 12$ .

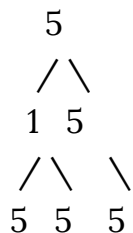
`[2,7,9,3,1]`

---

A subtree is univalue if:

All nodes in the subtree have the same value.

AND its left and right subtrees (if exist) are also univalue and match the current node's value. Your task is to write a function that counts how many univalue subtrees are present in a given binary tree



Input: `root = [5,1,5,5,5,null,5]`

Output: 4

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Example 1:

Input: nums = [100,4,200,1,3,2]

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: nums = [0,3,7,2,5,8,4,6,0,1]

Output: 9

Example 3:

Input: nums = [1,0,1,2]

Output: 3

---

Given two words w1 and w2. The baby can either erase a word or write a new word,

what is the minimum no of action he will perform to make these 2 words same.

S1 = "heap", s2 = "pea"

Output: 3

P and h deleted - 2 operation

P added in prior - 1 operation

---

You are building spotify shuffle feature

You have been given a playlist of songs

List songs = [s1,s2,s3,s4];

You have to play songs in shuffled manner in such a way that a song should be played again only after all other songs have been played.

---

Design a campaign management system (eg: samvad - multi tenant).

There could be 1000s of organizations, and each organization can have millions of users (scale).

Organizations could be having enterprise plan or other tier plans.

Organizations will send attributes related to user and campaign data.

And will tell, say campaign needs (schedule campaign) to run from 11 Dec to 16 Dec from 10am-5pm. Then the campaign would be sent to the user via whatsapp/telegram (basically some API call), etc. Also have SLAs, and check for throttling related areas. How to set up observability?. And 1 campaign should not affect other campaigns.

---

Leetcode qs asked:

<https://leetcode.com/problems/find-peak-element/description/>

<https://leetcode.com/problems/boats-to-save-people/description/>

<https://leetcode.com/problems/split-array-largest-sum/description/>

<https://leetcode.com/problems/number-of-visible-people-in-a-queue/description/>

<https://leetcode.com/problems/path-in-zigzag-labelled-binary-tree/description/>

<https://leetcode.com/problems/univalued-binary-tree/description/>

<https://algo.monster/liteproblems/250>

<https://www.geeksforgeeks.org/dsa/find-count-of-singly-subtrees/>

<https://leetcode.com/problems/longest-consecutive-sequence/description/>

---